

Tina4 for Python Developers

One Framework, Native Conventions, Shared Contracts

v3.13.138 • tina4.com

The Intelligent Native Application 4ramework

Table of Contents

Getting Started with Tina4 Python 72

1. What Is Tina4 Python . . . 72

2. Prerequisites and Installation . . . 72

    What You Need . . . 72

    Check What the CLI Found . . . 73

    Creating a New Project . . . 73

    Starting the Dev Server . . . 74

3. Project Structure Walkthrough . . . 75

4. Your First Route . . . 76

    Test It . . . 76

    Understanding What Happened . . . 77

    Adding More HTTP Methods . . . 77

5. Your First Template . . . 78

    Create a Base Layout . . . 78

    Create a Product Listing Page . . . 79

    Create the Route That Renders the Template . . . 79

    See It in the Browser . . . 80

    How Template Rendering Works . . . 80

    About tina4css . . . 80

6. Understanding .env . . . 81

7. The Dev Dashboard . . . 82

8. Manual Setup (No CLI) . . . 83

    Step 1: Install the Package . . . 83

    Step 2: Create app.py . . . 83

Step 3: Create the Folder Structure . . . 83
Step 4: Create .env . . . 84
Step 5: Run It . . . 84
9. Request & Response Fundamentals . . . 84
Reading Query Parameters . . . 84
Reading URL Path Parameters . . . 84
Reading the Request Body . . . 84
Reading Headers . . . 85
Sending JSON Responses . . . 85
Sending HTML / Template Responses . . . 85
Status Codes . . . 85
Worked Example: A Complete Route File . . . 85
10. Exercise: Greeting API + Product List Template . . . 87
Exercise Part A: Greeting API . . . 87
Exercise Part B: Product List Page . . . 88
11. Solutions . . . 88
Solution A: Greeting API . . . 88
Solution B: Product List Page . . . 89
12. Gotchas . . . 91
1. File not auto-discovered . . . 91
2. Route file did not import . . . 91
3. Template not found . . . 91
4. Port already in use . . . 91
5. Changes not reflected . . . 91
6. .env not loaded . . . 92
Routing **93**
1. How Routing Works in Tina4 . . . 93

2. HTTP Methods . . . 93
HEAD and OPTIONS: automatic, no boilerplate . . . 94
Explicit head() and options() registration . . . 95
3. Path Parameters . . . 95
Typed Parameters . . . 95
Typed Parameters in Action . . . 96
4. Query Parameters . . . 97
5. Route Groups . . . 98
6. Middleware on Routes . . . 99
Middleware on a Single Route . . . 99
Blocking Middleware . . . 99
Multiple Middleware . . . 100
7. Route Decorators: @noauth() and @secured() . . . 100
@noauth() -- Public Routes . . . 100
@secured() -- Protected GET Routes . . . 100
8. Route Chaining: .secure() and .cache() . . . 101
.secure() . . . 101
.cache() . . . 101
Chaining Both . . . 101
9. Wildcard and Catch-All Routes . . . 102
Wildcard Routes . . . 102
Catch-All Route (Custom 404) . . . 102
10. Route Listing via CLI . . . 102
11. Organizing Route Files . . . 103
Pattern 1: One File Per Resource . . . 103
Pattern 2: Subdirectories by Feature . . . 103
12. Exercise: Build a Full CRUD API for Products . . . 104

Requirements . . . 104
13. Solution . . . 105
14. Gotchas . . . 107
1. Trailing slashes matter . . . 107
2. Parameter names must be unique in a path . . . 107
3. Method conflicts . . . 107
4. Route handler must return a response . . . 107
5. Decorator order matters . . . 108
6. Forgetting async def . . . 108
7. Group prefix must start with a slash . . . 108
Request and Response **109**
1. The Two Objects You Use Everywhere . . . 109
2. The Request Object . . . 109
request.method . . . 109
request.path . . . 109
Path Parameters (Function Arguments) . . . 109
request.params . . . 110
request.body . . . 110
request.headers . . . 110
request.ip . . . 111
request.cookies . . . 111
request.files . . . 111
Handling Multiple File Uploads . . . 112
Upload Size Limits . . . 113
3. The Response Object . . . 113
response.json() . . . 113
response.html() . . . 113

response.text() . . . 114
response.render() . . . 114
response.redirect() . . . 114
response.file() . . . 114
response.xml() . . . 115
response.error() . . . 115
4. Status Codes . . . 115
5. Content Negotiation . . . 117
6. Custom Headers and Cookies . . . 117
Setting Response Headers . . . 117
Setting Cookies . . . 118
7. Input Validation . . . 119
The Validator Class . . . 119
8. Real-World Example: File Upload with Validation . . . 119
9. Exercise: Build a Contact Form API . . . 121
Requirements . . . 121
Test with: . . . 121
10. Solution . . . 122
11. Gotchas . . . 123
1. request.body is None for GET requests . . . 123
2. Forgetting the Content-Type header . . . 124
3. File upload with wrong field name . . . 124
4. response.redirect() in AJAX calls . . . 124
5. Cookie not being set . . . 124
6. Large file uploads fail . . . 124
7. Chaining response methods in wrong order . . . 125
8. Header names are lowercase . . . 125

Templates 126

1. Beyond JSON -- Rendering HTML . . . 126
2. The @template Decorator . . . 126
3. Variables and Output . . . 127
    - Basic Output . . . 127
    - Accessing Nested Properties . . . 127
    - Method Calls and Slicing . . . 127
    - Auto-Escaping . . . 127
4. Filters . . . 128
    - Complete Filter Reference . . . 128
        - String Filters . . . 128
        - Array Filters . . . 129
        - Encoding Filters . . . 130
        - Numeric Filters . . . 130
        - JSON Filters . . . 130
        - Dict Filters . . . 131
        - Other Filters . . . 131
    - Chaining Filters . . . 131
    - The default Filter . . . 131
    - Dumping Values for Debugging . . . 132
5. Control Tags . . . 132
    - if / elif / else . . . 132
    - for Loops . . . 133
    - for / else . . . 133
    - set -- Local Variables . . . 133
6. Template Inheritance . . . 134
    - Base Template . . . 134

Child Template . . . 134
Calling Parent Blocks . . . 135
7. Include and Macro . . . 135
include . . . 135
macro -- Reusable Template Functions . . . 135
8. Comments . . . 136
9. Special Tags . . . 136
{% raw %} -- Literal Output . . . 136
{% spaceless %} -- Remove Whitespace . . . 136
{% autoescape %} -- Control Escaping . . . 137
Whitespace Control . . . 137
10. tina4css . . . 137
Layout Classes . . . 137
Buttons . . . 138
Cards . . . 138
Alerts . . . 139
Forms . . . 139
Tables . . . 139
Spacing and Typography Utilities . . . 140
11. Exercise: Build a Product Catalog Page . . . 140
Requirements . . . 140
12. Solution . . . 141
13. Gotchas . . . 143
1. Whitespace in output . . . 143
2. Variable not defined error . . . 143
3. Extends must be the first tag . . . 143
4. Macro not found . . . 143

5. Filter produces wrong type . . . 144
6. Escaped HTML when you want raw output . . . 144
7. Include file path wrong . . . 144

Database **145**

1. From Lists to Real Data . . . 145
2. Configuration . . . 145
TINA4DATABASEURL . . . 145
Installing Database Drivers . . . 146
3. Creating a Connection . . . 146
Connection Pooling . . . 146
Extra Driver Options via **kwargs . . . 146
Firebird URL Forms . . . 147
Firebird Dual-Driver Support . . . 147
Lowercase Column Names . . . 147
Firebird Migration Support . . . 147
4. Running Queries . . . 148
fetch -- Get Multiple Rows . . . 148
DatabaseResult . . . 148
Properties . . . 148
Iteration . . . 148
Index Access . . . 148
Countable . . . 148
Conversion Methods . . . 149
Schema Metadata with column_info() . . . 149
fetch_one -- Get a Single Row . . . 149
execute -- Insert, Update, Delete . . . 150
5. Parameterised Queries . . . 150

6. Transactions . . . 150
7. Batch Operations with execute_many . . . 151
8. Insert, Update, and Delete Helpers . . . 152
insert . . . 152
update . . . 152
delete . . . 152
9. Schema Inspection . . . 153
get_tables() . . . 153
get_columns() . . . 153
table_exists() . . . 153
getdatabasetype() . . . 153
A Schema Info Endpoint . . . 153
10. Migrations . . . 154
File Naming . . . 154
Generating a Migration . . . 154
Writing the Up Migration . . . 154
Writing the Down Migration . . . 155
Running Migrations . . . 155
Checking Status . . . 155
Rolling Back . . . 155
Tracking Table . . . 155
Advanced SQL Splitting . . . 156
Migration Best Practices . . . 156
11. Query Caching . . . 156
12. Exercise: Build a Notes App API . . . 157
Requirements . . . 157
Test with: . . . 158

13. Solution . . . 158
Migration . . . 158
Routes . . . 159
14. Seeder -- Generating Test Data . . . 161
FakeData . . . 161
Deterministic Output . . . 162
Seeding a Table . . . 162
Overrides . . . 162
Seeding ORM Models . . . 162
When to Use It . . . 163
15. Gotchas . . . 163
1. SQLite boolean quirk . . . 163
2. lastinsertrowid() is SQLite-specific . . . 163
3. String vs integer comparison . . . 163
4. Connection not closed . . . 163
5. Migration order matters . . . 164
6. Missing down migration . . . 164
7. Failed migrations blocking progress . . . 164
8. SQL injection through string formatting . . . 164
ORM **165**
1. From SQL to Objects . . . 165
ORM at a Glance . . . 165
2. Defining a Model . . . 166
Field Types . . . 167
Field Options . . . 168
Field Mapping . . . 168
getdbcolumn and getdbdata . . . 169

find() vs where() -- naming convention . . . 169
auto_map and Case Conversion Utilities . . . 169
3. create_table -- Schema from Models . . . 170
4. CRUD Operations . . . 171
save -- Create or Update . . . 171
create -- Build and Save in One Step . . . 171
findbyid -- Fetch One Record by Primary Key . . . 171
find -- Query by Filter Dict . . . 172
where -- Query with SQL Conditions . . . 172
delete -- Remove a Record . . . 172
Listing Records . . . 172
ModelCollection -- The Page and the Total . . . 173
select_one -- Fetch a Single Record by SQL . . . 173
load -- Populate an Existing Instance . . . 174
count -- Count Records . . . 174
5. todict, tojson, and Other Serialisation . . . 174
to_dict . . . 174
to_json . . . 174
Other Serialisation Methods . . . 175
6. Relationships . . . 175
ForeignKeyField - Auto-Wired Relationships . . . 175
has_many . . . 176
has_one . . . 177
belongs_to . . . 177
7. Eager Loading . . . 178
Declarative Relationships with Descriptors . . . 178
Nested Eager Loading . . . 179

to_dict with Nested Includes . . . 179
8. Soft Delete . . . 179
Deleting and Restoring . . . 180
Including Soft-Deleted Records . . . 180
Counting with Soft Delete . . . 180
When to Use Soft Delete . . . 181
9. Auto-CRUD . . . 181
The auto_crud Flag . . . 181
Manual Registration . . . 181
Auto-Discovering Models . . . 182
What the Generated Routes Do . . . 182
Custom Routes Alongside Auto-CRUD . . . 183
Introspection . . . 183
10. Cached Queries . . . 183
11. Scopes . . . 184
12. Input Validation . . . 184
13. Exercise: Build a Blog with Relationships . . . 185
Requirements . . . 185
14. Solution . . . 186
15. Gotchas . . . 189
1. Forgetting to call save() . . . 189
2. findbyid() returns None . . . 189
3. find() vs findbyid() . . . 189
4. Circular imports with relationships . . . 190
5. to_dict() includes everything . . . 190
6. save() returns False on invalid data . . . 190
7. Foreign key not enforced . . . 190

8. N+1 query problem . . . 190
9. Auto-CRUD endpoint conflicts . . . 191
10. Soft-deleted records appearing in queries . . . 191
QueryBuilder Integration . . . 191
QueryBuilder 193
1. The Factory: from_table() . . . 193
2. Choosing Columns: select() . . . 193
3. Filtering: where() and or_where() . . . 194
OR conditions . . . 194
Conditions without parameters . . . 194
4. Joins: join() and left_join() . . . 194
5. Aggregation: group_by() and having() . . . 195
Filtering groups with having() . . . 195
6. Sorting: order_by() . . . 195
7. Pagination: limit() . . . 196
Pagination pattern . . . 196
8. Inspecting the SQL: to_sql() . . . 197
9. Execution Methods . . . 197
get() - Multiple rows . . . 197
first() - Single row . . . 197
count() - Row count . . . 198
exists() - Boolean check . . . 198
10. Chaining - Building Complex Queries . . . 198
11. Using with ORM Models . . . 199
When to use QueryBuilder vs ORM . . . 199
NoSQL: MongoDB Queries . . . 200
Operator Mapping . . . 200

Example . . . 200
Gotchas . . . 201
1. "select() replaced my columns" . . . 201
2. "My LIMIT is not in to_sql() output" . . . 201
3. "count() returns 0 but get() returns rows" . . . 201
4. "RuntimeError: No database connection provided" . . . 201
5. "or_where() on the first condition has no effect" . . . 202
6. "Parameters are in the wrong order" . . . 202
Exercise: Product Search API . . . 202
Solution . . . 203
Authentication **205**
1. Locking the Door . . . 205
2. JWT Tokens . . . 205
Generating a Token . . . 205
Token Expiry . . . 205
Validating a Token . . . 206
Reading the Payload . . . 206
The Secret Key and Algorithm . . . 206
3. Password Hashing . . . 206
Hashing a Password . . . 206
Checking a Password . . . 207
Registration Example . . . 207
4. The Login Flow . . . 208
5. Using Tokens in Requests . . . 209
6. Protecting Routes . . . 209
Auth Middleware . . . 209
Applying Middleware to Routes . . . 209

Applying Middleware to a Group . . . 210
7. @noauth and @secured Decorators . . . 210
@noauth -- Skip Authentication . . . 210
@secured -- Require Authentication for GET Routes . . . 211
Role-Based Authorization . . . 211
8. CSRF Protection . . . 211
Generating a Token . . . 211
Validating the Token . . . 212
When to Use CSRF Tokens . . . 212
9. Sessions . . . 213
Session Configuration . . . 213
Using Sessions . . . 213
Session Options . . . 214
10. Exercise: Build Login, Register, and Profile . . . 214
Requirements . . . 214
Test with: . . . 215
11. Solution . . . 215
Migration . . . 215
Routes . . . 215
12. Gotchas . . . 219
1. Token expiry confusion . . . 219
2. Secret key management . . . 219
3. CORS with authentication . . . 219
4. Storing tokens in localStorage . . . 219
5. Forgetting @noauth on login . . . 220
6. Password hash column too short . . . 220
7. Token in URL query parameters . . . 220

Sessions and Cookies 221

1. Remembering Your Users . . . 221

2. Session Configuration . . . 221

Available Backends . . . 221

Redis Configuration . . . 221

MongoDB Configuration . . . 222

Valkey Configuration . . . 222

Database Sessions . . . 222

Session Lifetime . . . 222

Session Cookie . . . 222

3. The Session API . . . 222

Full API Reference . . . 223

4. Reading and Writing Session Data . . . 223

Writing to the Session . . . 223

Reading from the Session . . . 223

Deleting Session Data . . . 224

Checking if a Key Exists . . . 224

Getting All Session Data . . . 224

5. Flash Messages . . . 224

Setting a Flash Message . . . 224

Reading a Flash Message . . . 224

Flash Messages in Templates . . . 225

6. Cookies . . . 225

Setting a Cookie . . . 225

Reading a Cookie . . . 225

Deleting a Cookie . . . 225

Cookie Options . . . 226

When to Use Cookies vs Sessions . . . 226
7. Remember Me . . . 227
8. Session Security . . . 228
Regenerate Session ID After Login . . . 228
Destroy a Session . . . 228
Secure Cookie Settings . . . 228
9. Exercise: Build a Shopping Cart . . . 229
Requirements . . . 229
Test with: . . . 230
10. Solution . . . 230
11. Gotchas . . . 233
1. Session lost between requests . . . 233
2. Session data is not JSON-serializable . . . 233
3. Cookie not sent on cross-origin requests . . . 233
4. File-based sessions on multi-server deployments . . . 233
5. Session cookie overwritten by another app . . . 234
6. Secure cookies on localhost . . . 234
7. Flash messages shown twice . . . 234
Middleware & Security **235**
1. The Pipeline Pattern . . . 235
2. Built-In Middleware . . . 235
CorsMiddleware . . . 235
RateLimiter . . . 236
3. Writing Custom Middleware . . . 236
Function-Based Middleware . . . 236
Class-Based Middleware . . . 237
Example: Request Timer (Class-Based) . . . 237

Example: Request Logger (Function-Based) . . . 238
Example: Security Headers (Class-Based) . . . 238
Example: JSON Content-Type Enforcer (Function-Based) . . . 238
Example: Request ID (Class-Based) . . . 238
Example: Input Sanitization (Class-Based) . . . 239
4. The @middleware Decorator . . . 239
5. Middleware on Route Groups . . . 240
6. Execution Order . . . 240
7. Short-Circuiting . . . 241
Conditional Short-Circuit . . . 242
8. Modifying Request and Response . . . 242
Adding Data to the Request . . . 242
Modifying the Response . . . 243
9. Real-World Example: JWT Authentication Middleware . . . 243
10. Real-World Example: API Key Middleware with Database Lookup . . . 244
11. Exercise: Build an API Key Middleware System . . . 244
Requirements . . . 244
Test with: . . . 245
12. Solution . . . 245
Migration . . . 245
Routes . . . 246
13. Gotchas . . . 248
1. Forgetting to await next_handler . . . 248
2. Middleware modifies response after it is sent . . . 248
3. Middleware applied to wrong routes . . . 248
4. Middleware execution order surprises . . . 248
5. Error in middleware breaks the chain . . . 248

6. Database connections in middleware . . . 249
7. Modifying request in middleware does not persist . . . 249
14. Security . . . 249
1. Secure-by-Default Routing . . . 249
Making a Write Route Public . . . 250
Protecting a GET Route . . . 250
The Rule . . . 250
2. CSRF Protection . . . 251
How It Works . . . 251
The Template . . . 251
The Middleware . . . 251
AJAX Requests . . . 251
Tokens in Query Strings - Blocked . . . 252
Skipping CSRF Validation . . . 252
Turning CSRF On . . . 252
3. Session-Bound Tokens . . . 252
How Stolen Tokens Fail . . . 252
4. Security Headers . . . 253
HSTS - Enforcing HTTPS . . . 253
Customising Headers . . . 253
5. SameSite Cookies . . . 254
6. Login Flow - Complete Example . . . 254
The Login Route . . . 255
The Login Form . . . 256
Protected Pages - Checking the Session . . . 256
Logout - Destroying the Session . . . 257
7. Handling Expired Sessions . . . 257

The Pattern: Redirect to Login, Then Back . . . 257
Token Refresh . . . 258
8. Rate Limiting . . . 258
9. CORS and Credentials . . . 259
10. Security Checklist . . . 259
Gotchas . . . 259
1. "My POST route returns 401 but I didn't add auth" . . . 259
2. "CSRF validation fails on AJAX requests" . . . 260
3. "I disabled CSRF but forms still fail" . . . 260
4. "My Content-Security-Policy blocks inline scripts" . . . 260
5. "User stays logged in after session expires" . . . 260
Exercise: Secure Contact Form . . . 260
Solution . . . 261
Caching **263**
1. From 800ms to 3ms . . . 263
2. The Request-Scoped Query Cache (On by Default) . . . 263
3. The Persistent DB Query Cache (Opt-In) . . . 264
When to use the persistent cache . . . 264
When not to use it . . . 265
4. Bypassing the Cache Per Query . . . 265
DB cache statistics . . . 265
5. Response Caching with ResponseCache Middleware . . . 266
Three ways to attach it . . . 266
Response cache headers . . . 267
Caching with query parameters . . . 267
What not to cache . . . 267
Response cache configuration . . . 268

6. Cache Backends . . . 268
Redis (and Valkey / Memcached / MongoDB) . . . 269
Graceful fallback . . . 269
7. Direct Cache API (Key/Value) . . . 269
cache_set . . . 269
cache_get . . . 270
cache_delete . . . 270
Real-World Pattern: Cache-Aside . . . 270
Key/value statistics . . . 270
8. Cache Invalidation Strategies . . . 271
Strategy 1: Time-Based Expiry (TTL) . . . 271
Strategy 2: Event-Based Invalidation . . . 271
Strategy 3: Write-Through Cache . . . 272
9. TTL Management . . . 272
Dynamic TTL . . . 273
10. Monitoring Cache Performance . . . 273
11. Combining Cache Layers . . . 273
12. Exercise: Cache an Expensive Product Listing Endpoint . . . 274
Requirements . . . 274
Test with: . . . 275
13. Solution . . . 275
14. Gotchas . . . 277
1. Caching Authenticated Responses . . . 277
2. Memory Cache Lost on Restart . . . 278
3. Stale Data After a Database Update . . . 278
4. Cache Key Collisions . . . 278
5. Serialization Overhead . . . 278

6. Forgetting the TTL on the Memory Backend . . . 278

Queues **279**

1. Not Everything Should Happen Right Now . . . 279

2. Queue Configuration . . . 279

Default (File Queue) . . . 279

Switching to RabbitMQ . . . 279

Switching to Kafka . . . 279

Switching to MongoDB . . . 280

3. Creating a Queue and Pushing Messages . . . 281

Priority and Delay . . . 281

Convenience Method: produce . . . 281

Queue Size . . . 281

4. Consuming Messages . . . 282

The consume Pattern . . . 282

Consume a Single Job by ID . . . 282

Manual Pop . . . 282

5. Priority Ordering . . . 283

6. Job Lifecycle . . . 283

Job Methods . . . 283

7. Automatic Retry and Dead Letters . . . 284

How Retry Works . . . 284

Retry Backoff . . . 284

Configuring Max Retries . . . 284

Inspecting Failed and Dead Jobs . . . 284

Reviving Dead Letters . . . 285

Counting and Purging by Status . . . 285

8. Queue in Route Handlers . . . 285

9. Switching Backends via .env . . . 286
Development: File (default) . . . 286
Production: RabbitMQ . . . 286
High-Scale Production: Kafka . . . 286
Production: MongoDB . . . 286
Environment variables the queue reads . . . 287
10. Produce and Consume Across Topics . . . 287
11. Exercise: Build an Email Queue . . . 288
Requirements . . . 288
Test with: . . . 288
12. Solution . . . 288
13. Gotchas . . . 290
1. Always call complete or fail . . . 290
2. Worker not picking up messages . . . 291
3. Payload too large . . . 291
4. Dead letters pile up . . . 291
5. File backend for production . . . 291
6. Environment-specific topic collision . . . 291
Events **292**
1. Decouple Everything with Events . . . 292
2. Registering Handlers with @on . . . 292
3. Firing Events with emit . . . 292
4. One-Time Handlers with @once . . . 293
5. Removing Handlers with off . . . 293
6. Priority Ordering . . . 293
7. Async Event Support with emit_async . . . 294
8. Real-World Pattern: Order Pipeline . . . 295

9. Exercise: User Lifecycle Events . . . 296
Requirements . . . 296
Test with: . . . 296
10. Solution . . . 296
11. Gotchas . . . 298
1. Emitting from a sync context with async handlers . . . 298
2. Handler order is non-deterministic at equal priority . . . 298
3. Forgetting once fires exactly once . . . 298
4. Mutating the payload in a handler . . . 298
Localization **299**
1. One App, Many Languages . . . 299
2. Locale Files . . . 299
3. The t() Function . . . 300
4. The I18n Class . . . 300
5. Setting the Locale at Runtime . . . 301
From a Query Parameter . . . 301
From the Accept-Language Header . . . 301
From a Session or Cookie . . . 302
6. The TINA4_LOCALE Environment Variable . . . 302
7. Interpolation with {placeholder} . . . 302
8. Fallback Behaviour . . . 303
9. Translating Templates . . . 303
10. Exercise: Multi-Language API Responses . . . 304
Requirements . . . 304
Test with: . . . 304
11. Solution . . . 304

12. Gotchas . . . 306
1. Missing locale file raises an error . . . 306
2. Placeholder name mismatch . . . 306
3. TINA4_LOCALE not set, t() uses "en" . . . 306
4. Locale files not reloading during development . . . 307
Structured Logging **308**
1. Stop Using print() . . . 308
2. Basic Usage . . . 308
3. Log Levels . . . 308
4. The TINA4LOGLEVEL Environment Variable . . . 309
5. Logging in Route Handlers . . . 309
6. File Output and Log Rotation . . . 310
7. Logging Exceptions . . . 310
8. Request Logging Middleware . . . 310
9. Log Levels by Environment . . . 311
10. Exercise: Add Logging to an Order API . . . 311
Requirements . . . 312
Test with: . . . 312
11. Solution . . . 312
12. Gotchas . . . 314
1. DEBUG logs flooding production . . . 314
2. Logging sensitive data . . . 314
3. Log file grows without rotation . . . 314
4. Forgetting to log exceptions . . . 314
Email with Messenger **315**
1. Every App Sends Email . . . 315

2. Messenger Configuration via .env . . . 315
Staging delivery safety . . . 316
Common Provider Configurations . . . 316
3. Constructor Override Pattern . . . 317
4. Sending Plain Text Email . . . 317
The send() Method Signature . . . 318
5. Sending HTML Email with Text Fallback . . . 318
6. Adding Attachments . . . 319
Attachments with Custom Names and Binary Content . . . 319
7. CC, BCC, and Reply-To . . . 320
8. Custom Headers . . . 320
Default Headers on an Instance . . . 320
Per-Email Headers . . . 321
9. Reading Inbox via IMAP . . . 321
Listing Inbox Messages . . . 321
Reading a Specific Message . . . 322
Searching Messages . . . 323
Other IMAP Operations . . . 323
Testing IMAP Connectivity . . . 323
10. Dev Mailbox Capture . . . 323
Sending from a development environment . . . 324
11. Using Templates for Email Content . . . 324
Rendering and Sending . . . 326
12. Sending Email via Queues . . . 327
13. Exercise: Build a Contact Form with Email Notification . . . 328
Requirements . . . 328
Test with: . . . 328

14. Solution . . . 328
15. Gotchas . . . 332
1. Gmail Blocks "Less Secure" Apps . . . 332
2. Emails Go to Spam . . . 332
3. HTML Email Looks Broken . . . 332
4. Attachment File Not Found . . . 332
5. Dev Mode Silently Intercepts Emails . . . 332
6. Email Template Variables Not Substituted . . . 333
7. Connection Timeout on Send . . . 333
8. IMAP Host Not Configured . . . 333
Frontend with tina4css **334**
1. The Problem with Frontend Toolchains . . . 334
2. What Ships with Tina4 . . . 334
3. The Grid System . . . 334
4. Components . . . 335
Navbar . . . 335
Cards . . . 335
Buttons . . . 336
Tables . . . 336
Alerts . . . 336
Modals . . . 337
Progress Bars . . . 337
Badges . . . 337
Forms . . . 338
5. SCSS Customization . . . 338
Live SCSS Compilation . . . 339
6. frond.js -- The JavaScript Helper . . . 339

AJAX Requests . . . 339
Form Submission via AJAX . . . 340
Token Management . . . 340
Loading Indicators . . . 340
WebSocket (Covered in Chapter 23) . . . 341
7. JavaScript API Reference . . . 341
Including the Scripts . . . 341
7.1 tina4.min.js -- Core Utilities . . . 341
loadPage(url, targetId) . . . 341
saveForm(formId, url, method) . . . 341
sendRequest(url, method, data, callback) . . . 342
7.2 frond.min.js -- Template Engine Client-Side Helpers . . . 342
AJAX Form Handling . . . 342
WebSocket Auto-Reconnect . . . 342
Token Refresh . . . 342
Dynamic Template Loading . . . 342
7.3 tina4js.min.js -- Reactive Frontend Framework . . . 343
Reactive State: signal(), computed(), effect() . . . 343
DOM Rendering: html Tagged Template . . . 343
Web Components: Tina4Element . . . 343
Fetch Wrapper: api() . . . 344
WebSocket Client: ws() . . . 344
Client-Side Routing: route(), navigate() . . . 344
8. Building an Admin Dashboard . . . 344
Base Template . . . 344
Dashboard Page . . . 345
Dashboard Route . . . 347

9. Dark Mode . . . 348
Respecting System Preference . . . 348
10. Responsive Design . . . 348
Responsive Sidebar . . . 348
Responsive Tables . . . 349
Hiding Elements by Screen Size . . . 349
11. Building a Users Page with AJAX . . . 349
The Template . . . 349
The Route . . . 351
12. HTML Builder -- Generating HTML in Code . . . 351
Direct Construction . . . 351
Builder Pattern . . . 352
Helper Functions . . . 352
Void Tags and Auto-Escaping . . . 352
Use Case -- HTML from a Route . . . 353
13. Exercise: Build an Admin Dashboard . . . 353
Requirements . . . 353
Test by: . . . 353
14. Solution . . . 354
15. Gotchas . . . 354
1. CSS Not Loading . . . 354
2. frond.js Functions Not Found . . . 355
3. Dark Mode Flickers on Page Load . . . 355
4. SCSS Not Compiling . . . 355
5. Modal Does Not Open . . . 355
6. Grid Columns Do Not Stack on Mobile . . . 355
7. Static Files Return 404 . . . 356

8. Form Data Not Reaching the Server . . . 356

9. Loading Indicator Never Disappears . . . 356

Testing **357**

1. Why Tests Matter More Than You Think . . . 357

2. Your First Test . . . 357

How It Works . . . 358

3. Assertion Methods . . . 358

assert_equal(actual, expected, message) . . . 358

assert_true(actual, expected, message) . . . 358

assert_false(actual, expected, message) . . . 358

assertraises(callable, exceptionclass, message) . . . 358

assertnotequal(actual, expected, message) . . . 359

assert_none(actual, expected, message) . . . 359

assertnotnone(actual, expected, message) . . . 359

4. Testing ORM Models . . . 359

Test Database . . . 361

5. Testing Routes . . . 361

Test Client Methods . . . 362

Response Object . . . 363

6. Testing Authentication . . . 363

7. Setup and Teardown . . . 364

8. Running Tests . . . 365

Run All Tests . . . 365

Run a Specific Test File . . . 365

Run a Specific Test Method . . . 365

Verbose Output . . . 365

Failed Test Output . . . 365

9. pytest Integration . . . 366
Code Coverage . . . 366
10. Testing Best Practices . . . 367
Test One Thing Per Test . . . 367
Use Descriptive Assertion Messages . . . 367
Isolate Tests . . . 367
11. Exercise: Test a User Model and Authentication Flow . . . 367
Requirements . . . 368
Expected output: . . . 368
12. Solution . . . 369
tests/testusermodel.py . . . 369
tests/testauthflow.py . . . 371
13. Gotchas . . . 372
1. Tests Run in Order . . . 372
2. Test Database vs Development Database . . . 373
3. Unique Constraint Failures . . . 373
4. Test Method Not Discovered . . . 373
5. Assertion Arguments Reversed . . . 373
6. Cannot Test Routes That Require Authentication Setup . . . 373
7. Tests Pass Locally but Fail in CI . . . 374
Scaffolding **375**
The CRUD Generator . . . 375
What Each File Contains . . . 375
Run It . . . 377
Individual Generators . . . 377
Model . . . 377
Route . . . 378

Migration . . . 378
Middleware . . . 378
Test . . . 378
Form . . . 378
View . . . 378
CRUD . . . 378
Auth . . . 379
AutoCRUD . . . 379
Usage . . . 379
The Auth Generator . . . 379
Field Types . . . 380
Table Naming Convention . . . 380
Combining Generators . . . 381
Exercise: Scaffold a Blog . . . 381
Gotchas . . . 381
Generators Do Not Overwrite . . . 381
Run Migrate After Generate . . . 382
File Naming Matters . . . 382
Field Changes Need New Migrations . . . 382
Singular Table Names . . . 382
Swagger / OpenAPI **383**
1. The 47-Endpoint Problem . . . 383
2. What Swagger/OpenAPI Is . . . 383
3. Accessing the Swagger UI . . . 383
4. Documenting Routes with Decorators . . . 384
@description -- Describe What an Endpoint Does . . . 384
@tags -- Organize Endpoints into Groups . . . 385

@example -- Document Request Body . . . 385
@example_response -- Document Response Body . . . 386
5. Documenting Path Parameters . . . 386
Documenting Query Parameters . . . 387
6. Authentication in Swagger . . . 387
7. Try-It-Out from the Swagger UI . . . 387
Authentication in Try-It-Out . . . 388
8. Customizing the Swagger Info Block . . . 388
9. Generating Client SDKs from the Spec . . . 389
Using OpenAPI Generator . . . 389
Other Generators . . . 390
10. Complete API Documentation Example . . . 390
11. Swagger Configuration . . . 392
12. Exercise: Document a User API . . . 392
Requirements . . . 392
Expected Result . . . 392
13. Solution . . . 393
14. Gotchas . . . 395
1. Swagger not showing up . . . 395
2. Route appears but has no documentation . . . 395
3. Decorator order causes issues . . . 395
4. Example does not match actual response . . . 395
5. Sensitive data in examples . . . 395
6. Swagger in production . . . 396
7. Too many tags . . . 396
8. SDK generation produces incorrect types . . . 396

API Client 397

1. Calling External APIs . . . 397
2. Basic Requests . . . 397
3. Reading the Response . . . 398
4. Authentication Headers . . . 398
    - Bearer Token . . . 398
    - Basic Authentication . . . 398
    - Custom Headers . . . 399
    - Mixing Auth and Custom Headers . . . 399
5. SSL Verification and Timeouts . . . 399
6. Sending Query Parameters . . . 399
7. Real-World Patterns . . . 400
    - Payment Gateway . . . 400
    - Weather Service . . . 401
8. Retry Logic . . . 401
9. Exercise: Weather Dashboard API . . . 402
    - Requirements . . . 402
    - Test with: . . . 402
10. Solution . . . 402
11. Gotchas . . . 404
    - 1. Not checking result["error"] . . . 404
    - 2. Using verify_ssl=False in production . . . 404
    - 3. No timeout set for slow external APIs . . . 404
    - 4. Hardcoding API keys . . . 404

GraphQL and SOAP 405

1. The Problem GraphQL Solves . . . 405
2. GraphQL vs REST -- A Quick Comparison . . . 405

3. Enabling GraphQL . . . 406
4. Defining a Schema . . . 406
5. Writing Resolvers . . . 407
Testing the Queries . . . 407
6. Mutations . . . 408
Mutation Resolvers . . . 409
Testing Mutations . . . 410
7. Nested Types and Relationships . . . 411
8. Auto-Generating Schema from ORM Models . . . 413
9. The GraphiQL Playground . . . 413
10. Query Variables . . . 414
11. Exercise: Build a GraphQL API for a Blog . . . 414
Requirements . . . 414
Test with these queries: . . . 415
12. Solution . . . 415
Schema . . . 415
Resolvers . . . 416
13. Input Validation . . . 418
What Gets Validated . . . 418
Example . . . 418
14. Field-Level Auth Directives . . . 418
Available Directives . . . 418
Passing Auth Context . . . 418
Using Directives in Queries . . . 419
15. Gotchas . . . 419
1. Schema File Not Found . . . 419

2. Resolver Not Called . . . 419
3. Nested Resolver Returns Wrong Data . . . 419
4. Mutation Input Not Parsed . . . 420
5. N+1 Query Problem . . . 420
6. GraphQL Playground Returns 404 . . . 420
7. Type Mismatch Between Schema and Resolver . . . 420
14. SOAP / WSDL Services . . . 421
What is SOAP? . . . 421
Defining a SOAP Service . . . 421
Type Annotations Map to XSD . . . 422
A More Complete Example . . . 422
Lifecycle Hooks . . . 423
Testing with curl . . . 423
When to Use SOAP vs REST vs GraphQL . . . 424
Real-time with WebSocket 425
1. The Refresh Button Problem . . . 425
2. What WebSocket Is . . . 425
3. @websocket -- WebSocket as a Route . . . 425
Starting the Server . . . 426
4. Connection Events . . . 426
Open . . . 426
Message . . . 426
Close . . . 427
A Complete Handler . . . 427
5. Sending to a Single Client . . . 428
6. Broadcasting to All Clients . . . 428
Broadcast Excluding Sender . . . 429

Sending JSON . . . 429
Closing a Connection . . . 430
Connection Methods Summary . . . 430
7. Path-Scoped Isolation . . . 430
8. Building a Live Chat . . . 431
WebSocket Handler . . . 431
9. Live Notifications . . . 433
10. Connecting from JavaScript . . . 434
Basic Connection . . . 434
Auto-Reconnect . . . 434
Using Native WebSocket . . . 435
11. A Complete Chat Page . . . 435
12. Exercise: Build a Real-Time Chat Room . . . 437
Requirements . . . 437
Test by: . . . 437
13. Solution . . . 438
14. Scaling with a Backplane . . . 440
Configuration . . . 440
Requirements . . . 440
When to Choose a Backplane . . . 441
15. Gotchas . . . 441
1. WebSocket Needs a Persistent Server . . . 441
2. Messages Are Strings, Not Dicts . . . 441
3. Connection Count Is Per-Path . . . 442
4. Broadcasting Does Not Scale Across Servers . . . 442
5. Large Messages Cause Disconnects . . . 442
6. Memory Leak from Tracking Connected Users . . . 442

7. No Authentication on WebSocket Connect . . . 442

Server-Sent Events (SSE) 444

1. The Polling Problem . . . 444

2. What SSE Is . . . 444

3. Your First Stream . . . 445

4. The SSE Protocol . . . 446

The data: Field . . . 446

The event: Field . . . 446

The id: Field . . . 446

The retry: Field . . . 446

The Delimiter . . . 447

5. Frontend: EventSource . . . 447

Basic Usage . . . 447

Named Events . . . 447

Closing the Connection . . . 448

With Authentication . . . 448

6. Real Examples . . . 448

Live Dashboard . . . 448

Build Progress . . . 449

LLM Chat Streaming . . . 449

File Processing Progress . . . 450

7. Custom Content Types . . . 451

NDJSON Streaming . . . 451

Consuming NDJSON on the Client . . . 451

8. SSE vs WebSocket . . . 452

9. Production Considerations . . . 453

Nginx Proxy Buffering . . . 453

Connection Limits . . . 453
Heartbeat Messages . . . 453
10. Exercise: Build a Live Metrics Dashboard . . . 454
Requirements . . . 454
Test by: . . . 454
11. Solution . . . 454
12. What You Learned . . . 456
WSDL / SOAP **457**
1. SOAP Is Not Dead . . . 457
2. Your First SOAP Service . . . 457
3. The @wsdl_operation Decorator . . . 457
4. Auto WSDL Generation at ?wsdl . . . 458
5. Calling the Service . . . 458
6. Lifecycle Hooks: onrequest and onresult . . . 459
7. Complex Types and Lists . . . 460
8. Error Handling . . . 460
9. Exercise: Build a Currency Conversion SOAP Service . . . 461
Requirements . . . 461
Test with: . . . 461
10. Solution . . . 461
11. Gotchas . . . 462
1. Missing SOAPAction header . . . 462
2. Namespace mismatch . . . 463
3. Type annotations missing . . . 463
4. Forgetting the ?wsdl route . . . 463
DI Container **464**

1. Stop Constructing Dependencies Inside Your Code . . . 464
2. The Container Class . . . 464
3. register() - Transient Services . . . 464
4. singleton() - Cached Services . . . 465
5. get() and has() . . . 465
6. reset() . . . 465
7. Building a Service Container for Your App . . . 466
8. Testing with the Container . . . 466
9. Full Example: Order Route with DI . . . 468
10. Exercise: Configurable Notification Service . . . 468
Requirements . . . 468
Test with: . . . 469
11. Solution . . . 469
12. Gotchas . . . 470
1. Mutable singleton shared across requests . . . 470
2. reset() removes singleton cache but not registrations . . . 470
3. Circular dependencies . . . 470
4. Importing container before .env is loaded . . . 470
Service Runner **471**
1. Work That Never Stops . . . 471
2. A Basic Background Service . . . 471
3. Start / Stop Lifecycle . . . 471
4. Queue Worker Service . . . 472
5. Scheduled Task Service . . . 473
6. Multiple Services . . . 474
7. Stopping Services Gracefully . . . 474

8. Integrating with the App . . . 475
9. Exercise: Cache-Warming Background Service . . . 475
Requirements . . . 475
Test with: . . . 476
10. Solution . . . 476
11. Gotchas . . . 477
1. Blocking the main thread . . . 477
2. No stop() method . . . 477
3. Exceptions crashing the service thread . . . 477
4. Service starts before .env is loaded . . . 477
MCP Dev Tools **478**
1. What is MCP? . . . 478
2. How It Works . . . 478
Localhost-Only Security . . . 478
3. Connecting AI Tools . . . 478
Claude Code . . . 478
Cursor . . . 479
Manual Connection . . . 479
4. Built-in Tools . . . 479
Database . . . 479
Routes . . . 480
Templates . . . 480
Files . . . 480
Migrations . . . 481
Data and ORM . . . 481
Infrastructure . . . 481
Debugging . . . 481

Project introspection . . . 482
Persistent code index . . . 482
Framework documentation (prose) . . . 482
Live API RAG (reflection) . . . 482
Plan management . . . 483
5. Protocol Details . . . 483
Streamable HTTP (current) . . . 484
Legacy HTTP+SSE (still supported) . . . 484
Messages . . . 484
Lifecycle . . . 484
6. Troubleshooting . . . 485
Building Custom MCP Servers **486**
1. Beyond Dev Tools . . . 486
2. Creating an MCP Server . . . 486
3. Registering Tools with @mcp_tool . . . 486
4. Registering Resources with @mcp_resource . . . 487
5. Class-Based MCP Services . . . 487
6. Securing MCP Endpoints . . . 488
7. Testing MCP Tools . . . 488
8. Complete Example: CRM MCP Server . . . 489
9. Best Practices . . . 490
Dev Tools **491**
1. Debugging at 2am . . . 491
2. Enabling the Dev Dashboard . . . 491
3. Dashboard Overview . . . 491
System Overview . . . 491

4. The Dev Toolbar . . . 492
Disabling the Toolbar . . . 492
5. Error Overlay . . . 492
Example Error . . . 493
Error Overlay in Production . . . 493
6. The Gallery . . . 493
7. Live Reload . . . 494
How It Works . . . 494
8. Hot-Patching with jurigged . . . 495
When Hot-Patching Cannot Help . . . 495
9. Request Inspector . . . 495
Request Details Panel . . . 496
Filtering Requests . . . 496
Request Replay . . . 496
10. SQL Query Runner . . . 496
Safety . . . 497
11. Queue Monitor . . . 497
12. System Info . . . 498
13. Exercise: Debug a Failing Route . . . 498
Setup . . . 498
Requirements . . . 499
Solution . . . 499
14. Gotchas . . . 500
1. Dev Dashboard Accessible on Network . . . 500
2. Live Reload Causes Connection Drops . . . 500
3. Error Overlay Shows in Production . . . 500
4. Request Inspector Slows Down the Server . . . 500

5. SQL Runner Executes Destructive Queries . . . 500
6. Hot-Patching Does Not Pick Up New Routes . . . 500
7. Template Changes Not Reflected . . . 501
8. Queue Monitor Shows No Jobs . . . 501

Tina4 CLI **502**

1. Getting a New Developer Up to Speed . . . 502
2. tina4 init -- Project Scaffolding . . . 502
Language Detection . . . 502
Explicit Language Selection . . . 503
Init into an Existing Directory . . . 503
3. tina4 serve -- Dev Server . . . 503
Options . . . 504
Serving a Project by Name . . . 504
Bypassing the CLI . . . 504
4. tina4 generate model -- ORM Scaffolding . . . 504
Adding Fields . . . 505
Field Types . . . 505
Options . . . 506
5. tina4 generate route -- CRUD Route Scaffolding . . . 506
Options . . . 508
6. tina4 generate migration -- Migration Scaffolding . . . 508
When to Generate a Migration . . . 508
7. tina4 generate middleware -- Middleware Scaffolding . . . 508
8. Generate a Model and Its Routes . . . 509
9. tina4 doctor -- Health Check . . . 509
10. tina4 test -- Running Tests . . . 510
Test Options . . . 510

11. tina4 routes -- Route Listing . . . 510
12. tina4 migrate -- Database Migrations . . . 511
Rollback . . . 511
Migration Table Auto-Upgrade . . . 511
Status . . . 511
13. Exercise: Scaffold a Feature in 5 Commands . . . 512
Requirements . . . 512
Expected Commands . . . 512
Solution . . . 512
14. Gotchas . . . 514
1. tina4 Command Not Found . . . 514
2. Wrong Language Detected . . . 514
3. Generated Files Overwrite Existing Code . . . 514
4. Migration Order Issues . . . 514
5. tina4 serve Uses Wrong Port . . . 514
6. Doctor Shows False Warnings . . . 514
7. Generate Creates Python 3.12+ Syntax . . . 515
8. Model Name Must Be PascalCase . . . 515
15. Documentation . . . 515
16. Test Port (Dual-Port Development) . . . 515
Vibe Coding with AI **516**
Why Tina4 is Built for AI-Assisted Development . . . 516
The Zero-Dependency Advantage . . . 516
CLAUDE.md -- The AI's Instruction Manual . . . 516
Skills -- Deep Framework Knowledge . . . 517
tina4-maintainer . . . 517

tina4-developer . . . 517
tina4-js . . . 517
The Convention Advantage . . . 517
Real-World Vibe Coding Session . . . 518
Python-Specific Patterns the AI Knows . . . 518
Route Decorators . . . 519
Template Rendering . . . 519
Middleware Attachment . . . 519
WebSocket Handlers . . . 519
Supported AI Tools . . . 520
The AI Chat in Dev Dashboard . . . 520
Tips for Effective Vibe Coding with Tina4 . . . 521
Exercise . . . 521
Gotchas . . . 521
1. No CLAUDE.md? . . . 521
2. AI Suggests Wrong Import Paths . . . 522
3. AI Hallucinates a Package . . . 522
4. AI Creates Files in Wrong Location . . . 522
5. AI Code Does Not Run . . . 522
6. AI Uses Synchronous Code . . . 522
7. AI Suggests Flask/Django Patterns . . . 522
The Philosophy . . . 523
Environment Variables 524
Core Server . . . 525
Secrets and Authentication . . . 527
Database . . . 528
CORS . . . 529

Routing . . . 529
Security Headers . . . 529
Rate Limiting . . . 530
Sessions . . . 530
Redis/Valkey session backend . . . 531
MongoDB session backend . . . 531
Templates . . . 532
Cache . . . 533
Queues . . . 534
Kafka queue backend . . . 534
RabbitMQ queue backend . . . 534
MongoDB queue backend . . . 535
WebSocket . . . 535
Email . . . 536
Logging . . . 537
Uploads . . . 538
Localisation . . . 538
Services (background tasks) . . . 539
Application AI Client and Developer AI Tools . . . 539
Swagger / OpenAPI . . . 541
GraphQL . . . 541
MCP (Model Context Protocol) . . . 542
Configuration Recipes . . . 542
PostgreSQL in production . . . 542
A shared cache on Redis . . . 542
Sessions that survive more than one box . . . 543

A queue on RabbitMQ or Kafka . . . 543
WebSocket broadcasts across instances . . . 543
Locked-down production headers . . . 543
The dev dashboard AI, kept local . . . 543
Minimal .env for Development . . . 544
Minimal .env for Production . . . 544
Feature Module Settings . . . 544
Deployment **546**
1. From Development to Production . . . 546
2. Production .env Configuration . . . 546
Key Differences from Development . . . 547
Sensitive Values . . . 547
3. ASGI Server Auto-Detection . . . 547
How Auto-Detection Works . . . 547
Uvicorn Configuration . . . 548
How Many Workers? . . . 548
4. Docker Deployment . . . 548
Official Base Image . . . 548
Dockerfile . . . 549
.dockerignore . . . 549
Building and Running . . . 550
Adding Database Drivers . . . 550
Docker Compose . . . 551
5. Health Checks . . . 551
6. Graceful Shutdown . . . 552
Configuring Shutdown Timeout . . . 552
Docker Stop Grace Period . . . 553

7. Log Rotation . . . 553
Using logrotate (Linux) . . . 553
Docker Logging . . . 553
8. Reverse Proxy with Nginx . . . 553
9. SSL/TLS with Let's Encrypt . . . 555
With Nginx and Certbot . . . 555
With Docker (Using Traefik as Reverse Proxy) . . . 555
Certificate Monitoring . . . 556
10. Scaling . . . 556
Multiple Workers . . . 556
Load Balancing with Nginx . . . 556
Docker Scaling . . . 557
Scaling Considerations . . . 557
11. Monitoring . . . 558
Log Aggregation . . . 558
Uptime Monitoring . . . 558
Application Performance Monitoring (APM) . . . 559
12. Exercise: Docker Deploy . . . 559
Requirements . . . 559
Solution . . . 560
HTTP Caching, Compression, and ETags . . . 560
13. Gotchas . . . 561
1. .env Not Loaded in Docker . . . 561
2. SQLite Database Lost on Container Restart . . . 561
3. WebSocket Connections Drop Behind Nginx . . . 561
4. Built-in Server Used in Production . . . 561
5. Static Files Slow . . . 561

6. Memory Usage Grows Over Time . . . 562
7. Container Starts Before Database Is Ready . . . 562
8. SSL Certificate Not Renewing . . . 562
9. Scaled Instances Have Different State . . . 562

Building a Complete App 563

1. Putting It All Together . . . 563
2. Planning the App . . . 563
Models . . . 563
Relationships . . . 563
Routes . . . 564
Templates . . . 564
3. Step 1: Init Project and Set Up Database . . . 564
Create Migrations . . . 564
4. Step 2: ORM Models . . . 565
5. Step 3: Authentication Routes . . . 567
Test Registration and Login . . . 568
6. Step 4: Auth Middleware . . . 568
7. Step 5: Task CRUD Routes . . . 569
Test the Task API . . . 571
8. Step 6: Dashboard Stats with Caching . . . 572
9. Step 7: WebSocket Real-Time Updates . . . 573
10. Step 8: Email Notifications . . . 574
11. Step 9: Dashboard Template . . . 575
12. Step 10: Tests . . . 577
13. Step 11: Docker Deployment . . . 579
14. The Complete Project Structure . . . 581

15. What You Built . . . 581
16. What to Build Next . . . 582
17. Closing Thoughts -- The Tina4 Philosophy . . . 583

Release Notes **584**

v3.13.138 (2026-09-24) - Safer requests, templates, and database operations . . . 584
v3.13.105 (2026-08-19) - Cross-API invariants, honest logs, portable tests . . . 585
Queue and ORM audit bugs — five fixed . . . 585
Route inspection scans, never boots . . . 585
Startup log tells the truth about @noauth / @secured . . . 586
Hot reload reaches every module that imports the changed one . . . 586
Firebird migration ledger is case-agnostic . . . 586
Test-harness fixes . . . 586
Skills grow a spine . . . 586
Upgrade notes . . . 587
v3.13.104 (2026-08-17) - OIDC SSO and PostGIS land . . . 587
OIDC / OpenID Connect — provider-neutral, session handoff . . . 587
PostGIS spatial support . . . 587
Swagger is SSO-aware . . . 587
v3.13.102 (2026-08-15) - Unified client and CI hardening . . . 587
v3.13.103 (2026-08-16) - Metrics you can trust, releases that cannot lie . . . 588
What changed . . . 588
What ships . . . 588
Failure is data . . . 589
Upgrade notes . . . 589
v3.13.100 (2026-08-14) - Frond keeps the whole page . . . 589
Template inheritance . . . 589
Bounded caches . . . 589

Frond extension scope . . . 589
AI skill installer . . . 589
Release consistency . . . 589
Upgrade notes . . . 590
v3.13.99 (2026-08-13) - Stability, parity, and secure by default . . . 590
Security . . . 590
Data integrity . . . 591
ORM and validation . . . 591
Request model - the big one . . . 592
Migrations . . . 592
HTTP . . . 592
Dev tooling . . . 592
Proven in all four . . . 593
Bug fixes . . . 593
Possible breaking . . . 593
Coming in 3.13.100 . . . 594
v3.13.98 (2026-08-11) - Skills discipline . . . 594
v3.13.97 (2026-08-07) - Behaviour corrections . . . 594
Queues . . . 595
DocStore . . . 595
Possible breaking . . . 595
v3.13.96 (2026-08-07) - One paginate envelope, one message shape . . . 595
Pagination . . . 595
Messenger . . . 595
Api client . . . 596
Swagger . . . 596
Migrations . . . 596

Server . . . 596
Build and skills . . . 596
Proven in all four . . . 597
Possible breaking . . . 597
v3.13.95 (2026-08-06) - Preparing for the 3.14 stable release . . . 597
Queues . . . 597
Logging . . . 597
Databases . . . 598
Sessions . . . 598
Document store and cache . . . 598
Local by default, production by environment variable . . . 598
Bulk insert . . . 598
Tooling . . . 599
Possible breaking . . . 599
v3.13.94 (2026-07-29) - A write with no filter is an error . . . 599
The write path (BREAKING) . . . 600
The Frond sandbox revokes capability, it does not skip a step . . . 600
Messenger: one name, one signature . . . 600
Base images that boot . . . 601
Frond, measured against the engines it replaces . . . 601
Also in this release . . . 601
Python specifics . . . 601
v3.13.92 (2026-07-27) - A token header that tells the truth about its signature . . . 602
The header names the algorithm that signed (python#105) . . . 602
TINA4JWTALGORITHM is read for real (python#106) . . . 602
nbf is enforced, with 60 seconds of leeway (python#107) . . . 602
The header cannot choose the verifier . . . 603

Overrides on authenticate_request are honoured . . . 603
A misconfigured algorithm raises (Breaking) . . . 603
The tests that hold it . . . 603
v3.13.91 (2026-07-27) - The offenders list finally points at code worth fixing . . . 603
A function is no longer charged for the functions inside it . . . 604
Ruby complexity was double what it should have been . . . 604
LOC means one thing again . . . 605
The dev dashboard reads the coupling it is given . . . 605
v3.13.90 (2026-07-27) - A scaffolded model and its migration finally agree . . . 605
One default, defined once (php#186, ruby#33, nodejs#38) . . . 606
A failed write no longer reports success . . . 606
v3.13.89 (2026-07-27) - A typo'd tag no longer renders what it was hiding . . . 606
An unknown tag raises instead of leaking its body (Breaking, Security) . . . 606
set works as a block (Fixed) . . . 607
The expression corpus grew to 82 . . . 607
v3.13.88 (2026-07-27) - json_encode gives you JSON again . . . 607
json_encode emits JSON, not entities (Breaking, reverts 3.13.87) . . . 608
A value JSON cannot represent becomes null (Fixed) . . . 608
to_json and tojson are the same filter . . . 608
The expression corpus grew to 79 . . . 609
What to change in your templates . . . 609
v3.13.87 (2026-07-27) - Frond expressions agree in all four frameworks . . . 609
Booleans print as true or false (Breaking) . . . 609
Both macro import forms now agree (Fixed) . . . 610
The not operator works in output (Fixed) . . . 610
json_encode escaping . . . 610
The gate that keeps this true . . . 610

v3.13.86 (2026-07-25) - The write-result contract is now consistent across the family 611

v3.13.85 (2026-07-24) - The dev-admin bundle ships once 611

Why the surviving file is still called .min.js 611

A gate, so the duplicate cannot come back 611

v3.13.84 (2026-07-24) - Every generated Dockerfile actually starts 612

serve no longer kills PID 1 (all four frameworks) 612

A gate, so this cannot rot again 613

Also in the CLI (tina4 v3.8.58) 613

v3.13.83 (2026-07-24) - Swagger UI stops serving itself in production 613

The banner advertises only what answers (python#99) 613

The CLI runs no watcher in production (tina4 v3.8.57) 614

A stale CA no longer turns a missing certificate into six failures (python#98) 614

v3.13.82 (2026-07-23) - MQTT 3.1.1: talk to any broker, no dependency 614

v3.13.81 (2026-07-21) - tina4 test fails the build when tests fail 615

v3.13.79 (2026-07-19) - Session cookies get Secure behind a proxy, and a renamed cookie is read back 615

v3.13.77 (2026-07-16) - A slow background task no longer runs on top of itself 616

v3.13.76 (2026-07-16) - Migrations apply again on a database created before 3.13.55 616

v3.13.75 (2026-07-14) - Static assets revalidate, so a deploy reaches users without a hard refresh 617

v3.13.74 (2026-07-13) - The dev dashboard connection tester works again 617

v3.13.73 (2026-07-13) - A failed migration re-applies cleanly 618

v3.13.72 (2026-07-12) - Frond number_format, filter precedence, and a sandbox fix 618

v3.13.71 (2026-07-11) - AI skills: sharper tina4_code guidance 619

v3.13.70 (2026-07-11) - Column defaults on INSERT, a Firebird charset override, and stacked Swagger metadata 619

ORM honours column defaults on INSERT (#165) 619

Firebird connection charset override (#160) 619

Stacked Swagger metadata all survives (#59) 619

v3.13.69 (2026-07-10) - The Api client grows up: uploads, downloads, and safe redirects . . . 619
Also shipping . . . 620
v3.13.68 (2026-07-10) - Parity release . . . 620
v3.13.67 (2026-07-10) - The MCP table browser, locked down . . . 620
v3.13.66 (2026-07-10) - A self-describing CLI, and generators that ship their tests . . . 621
The self-describing CLI . . . 621
Generators now ship a test with the code . . . 621
Databases . . . 621
Fixes . . . 622
Breaking . . . 622
v3.13.56 (2026-07-08) - Skills that own up when they drift . . . 622
v3.13.55 (2026-07-07) - One migration tracking schema on every engine . . . 623
v3.13.54 (2026-07-07) - Migrations honour the SET TERM directive . . . 623
v3.13.53 (2026-07-06) - JSONField: JSON document columns . . . 624
v3.13.52 (2026-07-04) - Frond live blocks, pgsql:// URL scheme, SCSS colour functions . . . 624
v3.13.51 (2026-07-03) - MCP Streamable HTTP transport, Firebird fixes . . . 625
v3.13.50 (2026-07-02) - Path route params match INTEGER primary keys on SQLite (Ruby fix) 626
v3.13.47 (2026-06-25) - Open-issue batch: migration comment splitting, global middleware, SCSS interpolation . . . 626
v3.13.46 (2026-06-24) - Atomic batch insert on SQLite + full batch-contract tests . . . 627
v3.13.45 (2026-06-24) - MSSQL insert row-count fix + real-service test hardening . . . 627
v3.13.44 (2026-06-24) - Real-service bug-fix sweep (no mocks) . . . 628
v3.13.43 (2026-06-22) - Queue lifecycle correctness + cross-framework unification . . . 629
v3.13.42 (2026-06-22) - Swagger configurability for external and public APIs . . . 629
v3.13.41 (2026-06-22) - Queue reservation/visibility timeout (at-least-once delivery) . . . 630
v3.13.40 (2026-06-22) - MCP security hardening + Swagger/OpenAPI overhaul . . . 631

v3.13.39 (2026-06-21) - Auto-migration, unified critical log level, fail-loud ORM, per-route WebSocket auth 632

v3.13.38 (2026-06-19) - Coordinated security & robustness release 633

v3.13.37 (2026-06-18) - Dev-admin editor: Ruby + more syntax highlighting 633

v3.13.36 (2026-06-18) - Instant WebSocket dev-reload 633

v3.13.35 (2026-06-17) - Live MCP endpoint for AI agents 634

v3.13.34 (2026-06-17) - Demo + onboarding fixes 634

v3.13.33 (2026-06-17) - Queues: priority pop + automatic dead-lettering (Warning: behavioural change) 634

v3.13.32 (2026-06-17) - Caching: per-query bypass + X-Cache headers (chapter rewritten to match code) 634

v3.13.31 (2026-06-17) - Documentation fixes (no functional change) 635

v3.13.30 (2026-06-16) - Typed route params now arrive coerced (Warning: behavioural change) 635

v3.13.29 (2026-06-16) - Live API search (api_search/api_class/api_method) now finds what you ask for 635

v3.13.28 (2026-06-16) - Frond: custom add_test now honoured by is 636

v3.13.27 (2026-06-16) - Frond template-engine parity fixes 636

v3.13.26 (2026-06-16) - pooling fix: standalone writes auto-commit; explicit transactions stay atomic 636

v3.13.24 (2026-06-15) - unified cache backends across response, KV, and persistent DB cache 637

v3.13.23 (2026-06-15) - request-scoped DB query cache, on by default 637

v3.13.22 (2026-06-15) - session default TTL standardised to 1 hour 638

v3.13.21 (2026-06-15) - security: never sign JWTs with a guessable default secret 638

v3.13.19 (2026-06-15) - return domain objects, construct from JSON, and one database binder 638

response(...) serializes domain objects 638

Construct a model from a JSON object string 638

Warning: Breaking - one database binder: bind_database 639

v3.13.16 (2026-06-15) - create_table() works on PostgreSQL + DatabaseResult index access 639

create_table() is now engine-aware 639

DatabaseResult is subscriptable . . . 640

v3.13.15 (2026-06-15) - Python only: PostgreSQL idle-in-transaction leak (#51) . . . 640

The slow-motion outage . . . 640

The fix . . . 640

Tests . . . 640

v3.13.14 (2026-06-13) - Logs reach stdout in containers + per-request logging + schema-qualified tables (#48) . . . 641

Per-request logging - on by default in dev . . . 641

What changed (stdout) . . . 641

Why it spanned all four . . . 641

Request logging parity . . . 642

Schema-qualified tables (#48) + a PostgreSQL fetch() regression . . . 642

Tests . . . 643

v3.13.12 (2026-06-11) - SQL safety + implicit ORM binding + fetch_all correctness . . . 644

fetch_all actually fetches ALL rows now (no silent 100-row truncation) . . . 644

Trailing ; is now stripped from user SQL in fetch() / fetch_one() . . . 644

Ruby ORM now auto-discovers TINA4_DATABASE_URL like the other three . . . 644

Cross-framework parity . . . 645

Tests . . . 645

v3.13.11 (2026-06-11) - ORM correctness pass . . . 645

#50.1 - Callable field defaults are now resolved per-instance . . . 645

#50.2 - save() correctly INSERTs natural (non-auto-increment) PKs . . . 646

#49.1 - Original cause logged when failure is inside an explicit transaction . . . 646

#49.2 - Database.fetch() populates last_error (mirror of execute()) . . . 646

BooleanField - engine-aware DDL on PG / MySQL / MSSQL . . . 646

Cross-framework parity . . . 647

Tests . . . 647

v3.13.10 (2026-06-11) - Python only . . . 647

1. Google Antigravity removed from AI_TOOLS - it reads AGENTS.md, not .antigravity/context.md . 648
2. uv.lock drift caught . . . 648
3. .gitignore for runtime broken-route artefacts . . . 648
Why Python-only . . . 648
Tests . . . 648
v3.13.9 (2026-06-10) . . . 649
The bug . . . 649
The fix . . . 649
Cross-framework parity . . . 650
Tests . . . 650
What you'll see when you re-install . . . 650
v3.13.8 (2026-06-10) - Python only . . . 650
The gap in v3.13.6 / v3.13.7 . . . 650
The fix: pre-flight heal . . . 651
Cross-framework . . . 651
Tests . . . 651
v3.13.7 (2026-06-10) . . . 652
NEW: tina4.request.error event . . . 652
FIX: Stack trace removed from production 500 body (CWE-209) . . . 652
Tests . . . 653
Background . . . 653
v3.13.6 (2026-06-09) . . . 653
PostgreSQL transaction errors no longer cascade (#46) . . . 653
Better driver install hints (#47) . . . 654
Tests . . . 654
v3.13.5 (2026-06-05) . . . 654
What changes . . . 654

Instance form still works . . . 655
clearRegistry() for test fixtures . . . 655
Implementation notes per framework . . . 655
Test count . . . 656
Upgrade . . . 656
v3.13.4 (2026-06-04) . . . 656
PY-10-02 - @middleware() no longer silently disables auth (SECURITY) . . . 656
PY-10-03 - request.headers is now case-insensitive . . . 656
PY-10-01 - Function-based middleware now runs . . . 657
Python chapter rewrites - book + docs . . . 657
Test count . . . 658
Upgrade . . . 658
v3.13.3 (2026-06-03) . . . 658
Env typed env-var helpers (tina4-python#43) . . . 658
Function-name in log lines (tina4-python#41) . . . 659
Test count . . . 659
Upgrade . . . 659
v3.13.2 (2026-06-03) . . . 659
SCSS calc() with mixed units (tina4-python#42, tina4-php#116, tina4-nodejs#1) . . . 659
Router.group docs taught a crashing pattern (tina4-python#44) . . . 660
DATABASEURL -> TINA4DATABASE_URL drift (tina4-python#45) . . . 660
Test count . . . 660
Upgrade . . . 660
v3.13.1 (2026-06-02) . . . 661
Convenience parity additions (Groups A / B) . . . 661
Decorator-style GraphQL resolvers across the family . . . 661
Class-based service pattern across the family . . . 661

PHP chapter rewrites (docs/php/ and book-2-php/) . . . 662
Test count . . . 662
Upgrade . . . 663
v3.13.0 (2026-06-01) . . . 663
The headline fire: Test class with HTTP helpers . . . 663
Auth.valid_token now returns the payload, not a bare bool - BREAKING . . . 663
Python-specific groups (mirrored to PHP/Ruby/Node in follow-up patches) . . . 664
PHP-specific: Tina4\Debug shim . . . 665
Documentation sweep . . . 665
Aspirational chapters rewritten . . . 666
tina4-js doc drift caught . . . 666
Test counts . . . 666
Upgrade . . . 667
v3.12.14 (2026-06-01) . . . 667
Python - tina4_python.test xUnit testing surface . . . 667
Cross-framework parity check (testing) . . . 668
PHP - :named placeholder translation across non-PDO adapters . . . 668
Cross-framework parity check (:named placeholders) . . . 669
Upgrade . . . 669
v3.12.13 (2026-05-29) . . . 669
Cross-framework dev-admin parity sweep (Tier 1-5) . . . 669
Folded-in from PHP 3.12.11 - file upload regression (tina4-book#139) . . . 671
Folded-in from PHP 3.12.12 + Python 3.12.13 - v2 tina4_migration auto-upgrade (#115) . . . 671
Folded-in from PHP 3.12.11 - request URL parity . . . 672
Upgrade . . . 672
v3.12.10 (2026-05-14) . . . 672
PHP - ORM->save() no longer swallows write failures (#114) . . . 672

Also in the PHP 3.12.7-3.12.9 patch line . . . 673
Python / Ruby / Node . . . 673
Upgrade . . . 673
v3.12.6 (2026-05-06) . . . 673
Python - psycopg2 % substitution no longer trips PL/pgSQL function bodies (#40) . . . 673
Long-standing tina4-js #37 confirmed fixed . . . 674
Upgrade . . . 674
v3.12.5 (2026-05-06) . . . 674
PHP - multipart bodies with file uploads now parse correctly (#135) . . . 674
Upgrade . . . 675
v3.12.4 (2026-05-06) . . . 675
25 new env vars across all 4 frameworks . . . 675
Logging - env-driven file output + rotation . . . 675
Documentation-truth CI gate now strict on both axes . . . 676
Tests added . . . 676
Upgrade path . . . 676
v3.12.3 (2026-05-05) . . . 676
Breaking changes (Ruby + PHP only) . . . 676
Doc parity - CLAUDE.md and book chapter 33 . . . 677
Other fixes . . . 677
Genuine gaps surfaced by the parity audit (follow-up, not blocking 3.12.3) . . . 677
Upgrade path . . . 678
v3.12.2 (2026-05-05) . . . 678
Firebird URL auto-detect . . . 678
Bug fix specific to PHP - mysqli localhost+port quirk . . . 679
Other fixes . . . 679
v3.12.1 (2026-05-04) . . . 679

v3.12.0 (2026-05-04) . . . 679
Why this release . . . 680
What changed . . . 680
Bug fixes shipped alongside the rename . . . 680
Migration - every renamed var . . . 681
Names that stay un-prefixed (not framework config) . . . 682
How to upgrade . . . 682
Parity . . . 682
v3.11.32 (2026-04-25) . . . 682
v3.11.13 (2026-04-16) . . . 683
v3.11.12 (2026-04-16) . . . 684
v3.11.11 (2026-04-16) . . . 685
v3.11.10 (2026-04-15) . . . 685
v3.11.9 (2026-04-15) . . . 686
v3.10.99 (2026-04-12) . . . 686
v3.10.97 (2026-04-11) . . . 687
v3.10.93 (2026-04-11) . . . 688
v3.10.92 (2026-04-10) . . . 688
v3.10.91 (2026-04-10) . . . 689
v3.10.90 (2026-04-09) . . . 689
v3.10.89 (2026-04-09) . . . 689
v3.10.86 (2026-04-09) . . . 690
v3.10.85 (2026-04-09) . . . 690
v3.10.84 (2026-04-09) . . . 690
v3.10.83 (2026-04-08) . . . 690
v3.10.70 (2026-04-06) . . . 691

Version History . . . 691
v3.10.68 (2026-04-03) - Full Parity Release . . . 691
v3.10.67 (2026-04-03) . . . 692
v3.10.66 (2026-04-03) . . . 692
v3.10.65 (2026-04-03) . . . 692
v3.10.60 (2026-04-03) . . . 693
v3.10.57 (2026-04-02) . . . 693
v3.10.54 (2026-04-02) . . . 693
v3.10.48 - April 2, 2026 . . . 694
Bug Fixes . . . 694
Test Coverage . . . 694
v3.10.46 - April 1, 2026 . . . 694
Test Coverage . . . 694
v3.10.45 - April 1, 2026 . . . 694
Bug Fixes . . . 694
v3.10.44 - April 1, 2026 . . . 695
New Features . . . 695
Bug Fixes . . . 695
Test Coverage . . . 695
v3.10.40 - April 1, 2026 . . . 696
Bug Fixes . . . 696
v3.10.39 - April 1, 2026 . . . 696
Breaking Changes . . . 696
New Features . . . 697
Test Coverage . . . 697
v3.10.38 - April 1, 2026 . . . 697
Code Metrics & Bubble Chart . . . 697

AI Context Installer . . . 698
Dashboard Improvements . . . 698
Cleanup . . . 698
v3.10.x - Previous Releases (March 28-31, 2026) . . . 698
Highlights . . . 698
Bug Fixes . . . 699
Breaking Changes . . . 701
v3.9.x - QueryBuilder and Sessions (March 26-27, 2026) . . . 702
Features . . . 702
Breaking Changes . . . 704
v3.8.x - Pooling, Validation, and Security (March 25-26, 2026) . . . 705
Features . . . 705
v3.7.x - Template Auto-Serve and Firebird Fixes (March 25, 2026) . . . 706
Features . . . 706
v3.6.x - API Parity (March 24, 2026) . . . 706
Breaking Changes . . . 706
v3.5.x - Bundled Frontend (March 24, 2026) . . . 708
Features . . . 708
v3.3.x - WebSockets, File Uploads, and Frond Improvements (March 24, 2026) . . . 708
Features . . . 708
Breaking Changes . . . 708
v3.2.x - Flexible Route Handlers and DevReload (March 24, 2026) . . . 710
Features . . . 710
Breaking Changes . . . 711
v3.1.x - Benchmarks and Internal Improvements (March 22, 2026) . . . 712
Features . . . 712
v3.0.0 - The Rewrite (March 22, 2026) . . . 712

What Changed . . . 712
Release Candidate History . . . 713
Upgrading from v2 to v3 714
1. Overview . . . 714
Step 0: Run the Automated Upgrade Command . . . 714
2. Package and Installation . . . 714
3. Project Structure Changes . . . 715
4. Routing Changes . . . 715
Decorators . . . 715
Auth Defaults . . . 716
Decorator Order . . . 716
Middleware . . . 716
Wildcard Routes . . . 716
5. Database Changes . . . 717
Connection URL . . . 717
Keyword Arguments . . . 717
Firebird Column Names . . . 717
Firebird Drivers . . . 717
Transactions . . . 718
Connection Pooling . . . 718
6. ORM Changes . . . 718
Field Definitions . . . 718
Binding the Database . . . 718
Auto Mapping . . . 719
Field Mapping . . . 719
Relationships . . . 719

7. Template Engine Changes . . . 719
Frond Replaces Template . . . 719
Custom Filters . . . 720
Custom Globals . . . 720
New Template Features . . . 720
8. Migration Tracking Table . . . 720
9. Authentication Changes . . . 721
10. Session Changes . . . 721
11. New Features in v3 . . . 722
Common Pitfalls . . . 722
1. POST/PUT/DELETE routes now require authentication . . . 722
2. Database connection strings changed . . . 723
3. Template engine renamed . . . 723
12. Step-by-Step Migration Checklist . . . 723
Feature Catalog **725**
Read the status correctly . . . 725
Where the features live . . . 726
Current parity boundary . . . 726
Dependency boundary . . . 727
Numbered catalog . . . 727
Foundation . . . 727
Database and providers . . . 727
ORM and data layer . . . 728
HTTP core . . . 728
HTTP policies . . . 729
HTTP runtime . . . 729

Frond template engine . . . 730
Frontend assets . . . 730
Authentication . . . 730
Sessions and providers . . . 731
Cache and providers . . . 731
Integrations and storage . . . 732
Enterprise authentication . . . 733
Application runtime . . . 733
CLI . . . 734
Developer runtime . . . 734
Testing and verification tools . . . 735
What parity means . . . 735
Real-time Collaboration (WebRTC) 736
1. The Media Server You Do Not Have to Run . . . 736
2. What You Get, and How It Differs from Raw WebSocket . . . 736
3. Mounting the Module: realtime() . . . 737
What each feature wires . . . 738
4. The Config Bootstrap: GET /api/rtc/config . . . 738
5. ICE and TURN Servers . . . 739
Environment variables . . . 739
6. Signalling: The Mesh Relay . . . 740
7. Chat: Secured Channels, Presence, History . . . 741
Chat history: GET {p}/api/channels/{id}/messages . . . 742
8. Files: Upload and Download . . . 742
Storage backends . . . 742
9. Auth, Identity, and the Data Model . . . 743

Data model . . . 743
10. A Complete Example . . . 744
Backend - app.py . . . 744
Browser - bootstrap from the config endpoint . . . 745
Browser - the chat side-panel (secured, needs a JWT) . . . 746
11. Footguns and Hard Rules . . . 746
Where to Go Next . . . 747
AI Client **748**
Configure a provider . . . 748
Complete one prompt . . . 748
Hold a chat . . . 749
Stream text . . . 749
Create embeddings . . . 750
Handle failures . . . 750
Timeouts and retries . . . 750
OpenID Connect SSO **752**
Configure SSO . . . 752
Protect a route . . . 752
Use the SSO client . . . 753
Map provider claims . . . 754
Provider recipes . . . 754
Keycloak . . . 754
Microsoft Entra ID . . . 754
Verification and failure rules . . . 754
Swagger . . . 754
GIS and PostGIS **755**

Prepare PostGIS . . . 755
Define a point field . . . 755
Point values . . . 755
Query by distance . . . 756
Return GeoJSON . . . 756
Limits in 3.13.104 . . . 757
IoT and MQTT **758**
Configure the broker . . . 758
Publish telemetry . . . 758
Subscribe and consume . . . 758
Retained state and Last Will . . . 759
TLS and trust . . . 759
Public operations . . . 760

Getting Started with Tina4 Python

1. What Is Tina4 Python

Tina4 Python is a zero-dependency web framework for Python 3.12+. The core package declares no required third-party dependencies; optional packages enable selected database, cache, queue, MongoDB, and S3 providers. Routing, ORM, template engine, authentication, queues, and WebSocket are all built in. Tina4 Python contributes to the framework family's 140-entry feature catalog, listed in Chapter 38: Complete Feature List; the catalog is an inventory, not a claim that every entry has reached parity.

It belongs to the Tina4 family: four backend implementations in Python, PHP, Ruby, and Node.js. They share contracts, project structure, template syntax, CLI commands, and `.env` variables. The parity audit records where an implementation still differs.

Tina4 Python follows Python convention: `snake_case` for methods and functions (`fetch_one()`, `soft_delete()`, `has_many()`), `PascalCase` for classes, `UPPER_SNAKE_CASE` for constants. Route handlers are decorated with `@get`, `@post`, and friends, and are written `async def` by convention -- the server is asyncio, so an `async` handler can `await` without blocking it.

By the end of this chapter, you will have a running Tina4 Python project with an API endpoint and a rendered HTML page.

---

2. Prerequisites and Installation

What You Need

Three tools. Nothing else.

- **Python 3.12 or later** -- check with:

```
python3 --version
```

You should see output like:

```
Python 3.x.x
```

If you see a version lower than 3.12, or no Python at all, install it from python.org/downloads. On Windows, tick *Add python.exe to PATH* in the installer, or the `tina4` CLI will not find it -- and Windows names the command `python`, not `python3`.

- **uv** -- a fast Python package manager and project tool. Check with:

```
uv --version
```

You should see:

```
uv 0.x.x
```

If uv is not installed, get it from docs.astral.sh/uv/getting-started/installation, which covers every platform including Homebrew, winget and pipx. Python and uv are other people's tools, so this

book links their instructions rather than copying them -- installers change, and a copy here would go stale without anyone noticing.

- **The Tina4 CLI** -- a Rust-based binary that manages all four Tina4 frameworks. This one is ours, so the commands are here:

macOS (Homebrew):

```
brew install tina4stack/tap/tina4
```

Linux / macOS (install script):

```
curl -fsSL https://tina4.com/install.sh | sh
```

Windows (PowerShell):

```
irm https://tina4.com/install.ps1 | iex
```

Verify the CLI is installed:

```
tina4 --version
```

```
tina4 3.x.x
```

The `tina4` CLI manages project scaffolding, development servers, migrations, and more across all Tina4 frameworks. Binaries are also on GitHub Releases.

Check What the CLI Found

Before scaffolding anything, ask the CLI what it can see:

```
tina4 doctor
```

```
  Tina4 Doctor - Environment Check

  Language     Status     Version              Pkg Mgr      Version
  ■■■
```

`tina4 init` takes a language and a **path**, and the path is not optional. It scaffolds a complete project at that path -- routes, templates, database, configuration -- builds a virtual environment inside it, and installs the dependency. There is no separate `uv sync` step.

`tina4 setup` is the guided alternative: it asks once where your projects should live, remembers the answer in `~/.tina4/setup.conf`, and scaffolds there. From then on `tina4 serve my-store` finds the project from any directory.

You should see:

```
■ Initialising python project at /home/you/my-store

■ Checking python runtime...
  ✓ python3 found

■ Checking package manager...
  ✓ uv found
  ✓ Created directory /home/you/my-store

■ Scaffolding python project...
  ✓ Created directory structure
  ✓ Created .env
  ✓ Created app.py
  ✓ Created .gitignore
  ✓ Created pyproject.toml
  ✓ Created Python scaffold

■ Installing dependencies...
  ■ Running: uv sync (in /home/you/my-store)
Using CPython 3.x.x interpreter at: /usr/bin/python3
Creating virtual environment at: .venv
Resolved 2 packages in 3.81s
Installed 1 package in 15ms
 + tina4-python==3.x.x
  ✓ Dependencies installed

✓ Project created at /home/you/my-store

  Start the server now? [Y/n]:
```

`Resolved 2 packages` is the whole dependency graph: your project and `tina4-python`. One package. No dependency tree. No version conflicts.

Everything the project needs lives under that one directory, `.venv/` included, so there is nothing installed globally and nothing to clean up elsewhere. Delete the directory and the project is gone.

Answer `n` for now -- we will start the server deliberately in the next section.

Starting the Dev Server

```
cd my-store
tina4 serve

  ______ _              __ __
 /_  __/(_)___  ____ _/ // /
  / /  / / __ \/ __ `/ // /_
 / /  / / / / / /_/ /__  __/
/_/  /_/_/ /_/\__,_/  /_/   ____________________________
```

```
Tina4 Python v3.x.x - The Intelligent Native Application 4ramework

Server:    http://localhost:7146 (asyncio)
Swagger:   http://localhost:7146/swagger
Dashboard: http://localhost:7146/__dev
Debug:     ON (Log level: ALL)
Test Port: http://localhost:8146 (stable - no hot-reload)
```

Open your browser to `http://localhost:7146`. The Tina4 welcome page greets you.

Open `http://localhost:7146/health` in your browser or curl it:

```
curl http://localhost:7146/health
```

```
{"status":"ok","version":"3.x.x","uptime":0.83,"framework":"tina4-python"}
```

`/health` and `/__health` both answer. It is a liveness probe for the process, so it reports no database or cache state.

Your Tina4 Python project is running.

---

3. Project Structure Walkthrough

Here is what you have after `tina4 init` and the first `tina4 serve`:

```
my-store/
■■■ app.py                  # Entry point -- imports the framework and calls run()
■■■ .env                    # Your configuration (gitignored)
■■■ .env.local              # Dev auth secret, written on first serve (gitignored)
■■■ .gitignore              # Pre-configured
■■■ pyproject.toml          # Python project definition (uv / pip)
■■■ uv.lock                 # Resolved dependency lock file
■■■ .venv/                  # Virtual environment (gitignored)
■■■ migrations/             # SQL migration files (project root)
■■■ src/
■   ■■■ routes/             # Your route handlers go here
■   ■■■ orm/                # Your ORM model classes go here
■   ■■■ seeds/              # Database seed files
■   ■■■ templates/          # Frond templates
■   ■   ■■■ errors/         # Your custom error pages
■   ■■■ public/             # Static files, served from /
■   ■   ■■■ js/
■   ■   ■■■ css/
■   ■   ■■■ images/
■   ■   ■■■ icons/
■   ■■■ scss/               # SCSS sources, compiled into src/public/css/
■   ■■■ locales/            # Translation files
■■■ data/                   # SQLite databases (gitignored)
■■■ logs/                   # Log files (gitignored)
■■■ secrets/                # JWT keys (gitignored)
■■■ tests/                  # Your test files
```

`init` writes `app.py`, `.env`, `.gitignore` and `pyproject.toml`; the convention directories are created and kept in place by the server. They start empty -- there is no sample route, template or locale file to delete.

Key directories:

- `src/routes/` -- Every `.py` file here is auto-loaded at startup. Drop your route definitions here. Organize into subdirectories if you want.
- `src/orm/` -- Every `.py` file here is auto-loaded. Drop your ORM model classes here.
- `src/templates/` -- Frond (Tina4's built-in template engine -- see Chapter 4: Templates) looks here when you call `response.render("my-page.html", data)`.
- `src/public/` -- Files served directly. `src/public/images/logo.png` lives at `/images/logo.png`.
- `data/` -- The default SQLite database (`app.db`) lives here. Gitignored because databases do not belong in version control.

---

4. Your First Route

Create the file `src/routes/greeting.py`:

```
from tina4_python.core.router import get

@get("/api/greeting/{name}")
async def greeting(name, request, response):
    from datetime import datetime
    return response.json({
        "message": f"Hello, {name}!",
        "timestamp": datetime.now().isoformat()
    })
```

In Python, path parameters arrive as function arguments. The parameter name in the function signature must match the `{name}` in the route pattern.

> ***Cross-language note.*** *Python is the odd one out here. PHP, Ruby, and Node read path parameters from* `$request->params` / `request.params` / `req.params` *instead. Same concept, two shapes, so pick the chapter that matches your runtime.*

Save the file. The dev server picks up the change through live reload. If not, restart with `tina4 serve`.

Test It

Open your browser to:

```
http://localhost:7146/api/greeting/Alice
```

You should see:

```
{
  "message": "Hello, Alice!",
  "timestamp": "2026-03-22T14:30:00.000000"
}
```

Or use curl:

```
curl http://localhost:7146/api/greeting/Alice
```

```
{"message":"Hello, Alice!","timestamp":"2026-03-22T14:30:00.000000"}
```

The browser shows pretty-printed JSON (browser extensions or dev mode). Curl shows compact JSON. Force pretty output with `?pretty=true`:

```
curl "http://localhost:7146/api/greeting/Alice?pretty=true"
```

```
{
  "message": "Hello, Alice!",
  "timestamp": "2026-03-22T14:30:00.000000"
}
```

Understanding What Happened

Five steps. No magic.

- You created a file in `src/routes/`. Tina4 auto-discovered it at startup.
- `@get("/api/greeting/{name}")` registered a GET route with a path parameter `{name}`.
- When you requested `/api/greeting/Alice`, the router matched the pattern and called your handler.
- The router extracted `"Alice"` from the URL and passed it as the `name` argument to your function.
- `response.json(...)` serialized the dictionary to JSON, set `Content-Type: application/json`, and returned a `200 OK`.

Adding More HTTP Methods

Update `src/routes/greeting.py`:

```
from tina4_python.core.router import get, post

@get("/api/greeting/{name}")
async def greeting(name, request, response):
    from datetime import datetime
    return response.json({
        "message": f"Hello, {name}!",
        "timestamp": datetime.now().isoformat()
    })

@post("/api/greeting")
async def create_greeting(request, response):
    name = request.body.get("name", "World")
    language = request.body.get("language", "en")

    greetings = {
        "en": "Hello",
        "es": "Hola",
        "fr": "Bonjour",
        "de": "Hallo",
        "ja": "Konnichiwa"
    }

    greeting_word = greetings.get(language, greetings["en"])

    return response.json({
        "message": f"{greeting_word}, {name}!",
        "language": language
    }, 201)
```

Test the POST endpoint:

```
curl -X POST http://localhost:7146/api/greeting \
  -H "Content-Type: application/json" \
  -d '{"name": "Carlos", "language": "es"}'
```

```
{"message":"Hola, Carlos!","language":"es"}
```

> ***On Windows**, run these from PowerShell as `curl.exe`, on a single line. Plain `curl` is an alias for `Invoke-WebRequest`, which takes different arguments, and `\` is not a line continuation character:*
>
> ``powershell curl.exe -X POST http://localhost:7146/api/greeting -H "Content-Type: application/json" -d '{"name": "Carlos", "language": "es"}' ``
>
> *The same applies to every `curl` example below. Everything else in this chapter -- the `tina4` CLI, `uv`, `.env` variables -- is identical on Linux, macOS and Windows.*

The HTTP status code is `201 Created` (the second argument to `response.json()`).

---

5. Your First Template

Tina4 uses the **Frond** template engine -- a zero-dependency, Twig-compatible engine built from scratch. If you have used Twig, Jinja2, or Nunjucks, Frond will feel familiar.

Create a Base Layout

Create `src/templates/base.html`:

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>{% block title %}My Store{% endblock %}</title>
    <link rel="stylesheet" href="/css/tina4.css">
    <style>
        body { font-family: -apple-system, BlinkMacSystemFont, "Segoe UI", Roboto, sans-serif; m
        .container { max-width: 960px; margin: 0 auto; padding: 20px; }
        .product-card { border: 1px solid #ddd; border-radius: 8px; padding: 16px; margin: 8px 0
        .product-card h3 { margin-top: 0; }
        .price { color: #2d8f2d; font-weight: bold; font-size: 1.2em; }
        .badge { display: inline-block; padding: 2px 8px; border-radius: 4px; font-size: 0.8em;
        .badge-success { background: #d4edda; color: #155724; }
        .badge-danger { background: #f8d7da; color: #721c24; }
        nav { background: #333; color: white; padding: 12px 20px; }
        nav a { color: white; text-decoration: none; margin-right: 16px; }
    </style>
</head>
<body>
    <nav>
```

```
        <a href="/">Home</a>
        <a href="/products">Products</a>
    </nav>
    <div class="container">
        {% block content %}{% endblock %}
    </div>
    <script src="/js/frond.js"></script>
</body>
</html>
```

This base layout defines two blocks (`title` and `content`) that child templates override. It includes `tina4.css` (the built-in CSS framework) and `frond.js` (the built-in JS helper library).

Create a Product Listing Page

Create `src/templates/products.html`:

```
{% extends "base.html" %}

{% block title %}Products - My Store{% endblock %}

{% block content %}
    <h1>Our Products</h1>
    <p>Showing {{ products | length }} product{{ "s" if products|length != 1 else "" }}</p>

    {% if products %}
        {% for product in products %}
            <div class="product-card">
                <h3>{{ product.name }}</h3>
                <p>{{ product.description }}</p>
                <p class="price">${{ "%.2f"|format(product.price) }}</p>
                {% if product.in_stock %}
                    <span class="badge badge-success">In Stock</span>
                {% else %}
                    <span class="badge badge-danger">Out of Stock</span>
                {% endif %}
            </div>
        {% endfor %}
    {% else %}
        <p>No products available at the moment.</p>
    {% endif %}
{% endblock %}
```

Create the Route That Renders the Template

Create `src/routes/pages.py`:

```
from tina4_python.core.router import get

@get("/products")
async def products_page(request, response):
    products = [
        {
            "name": "Wireless Keyboard",
            "description": "Ergonomic wireless keyboard with backlit keys.",
            "price": 79.99,
            "in_stock": True
        },
        {
            "name": "USB-C Hub",
```

```
            "description": "7-port USB-C hub with HDMI, SD card reader, and Ethernet.",
            "price": 49.99,
            "in_stock": True
        },
        {
            "name": "Monitor Stand",
            "description": "Adjustable aluminum monitor stand with cable management.",
            "price": 129.99,
            "in_stock": False
        },
        {
            "name": "Mechanical Mouse",
            "description": "High-precision wireless mouse with 16,000 DPI sensor.",
            "price": 59.99,
            "in_stock": True
        }
    ]

    return response.render("products.html", {"products": products})
```

See It in the Browser

Open `http://localhost:7146/products`. You should see:

- A dark navigation bar with "Home" and "Products" links
- The heading "Our Products"
- A subheading showing "Showing 4 products"
- Four product cards, each with name, description, price, and stock badge
- The "Monitor Stand" card shows a red "Out of Stock" badge
- The other three show green "In Stock" badges

How Template Rendering Works

Eight steps. All automatic.

- `response.render("products.html", {"products": products})` tells Frond to render `src/templates/products.html` with the given data.
- Frond sees `{% extends "base.html" %}` and loads the base template.
- The `{% block content %}` in `products.html` replaces the same block in `base.html`.
- `{{ product.name }}` outputs the value, auto-escaped for HTML safety.
- `{{ "%.2f"|format(product.price) }}` formats the number with 2 decimal places.
- `{% for product in products %}` loops through the list.
- `{% if product.in_stock %}` renders the stock badge conditionally.
- `{{ products | length }}` returns the item count.

About tina4css

The `tina4.css` file is Tina4's built-in CSS utility framework. Layout utilities. Typography. Common UI patterns. No Bootstrap. No Tailwind. No download.

It is served by the framework itself, so there is no copy in your project to find or maintain -- link it at `/css/tina4.css`. The same is true of `frond.js` at `/js/frond.js`.

---

6. Understanding .env

Open the `.env` file at the root of your project:

```
TINA4_DEBUG=true
```

That is likely all you see. The scaffold creates a minimal `.env` with debug mode enabled. Everything else uses defaults.

The important defaults for development:

Variable	Default Value	What It Means
`TINA4_PORT`	`7146`	Default server port (override with `tina4 serve --port`)
`TINA4_DATABASE_URL`	`sqlite:///data/app.db`	SQLite database in the `data/` directory
`TINA4_LOG_LEVEL`	`ALL`	All log messages are output
`TINA4_CORS_ORIGINS`	unset	Deny by default -- no cross-origin request is allowed until you list the origins (`*` has to be asked for)
`TINA4_RATE_LIMIT`	`100`	Requests allowed per window, per IP
`TINA4_RATE_WINDOW`	`60`	Window length in seconds, so the default is 100 requests per minute

Log levels control how much output Tina4 produces:

Level	Behaviour
`ALL` / `DEBUG`	Full verbose output. DevReload active (live-reload, error overlay, hot-patching).
`INFO`	Standard logging. Startup messages, request summaries.
`WARNING`	Warnings and errors only.
`ERROR`	Errors only. Minimal output.

Set `TINA4_LOG_LEVEL=DEBUG` during development for maximum visibility. Use `WARNING` or `ERROR` in production.

To change the port, use the CLI flag or `.env`:

```
tina4 serve --port 8080
```

Or add it to your `.env` file:

```
TINA4_DEBUG=true
TINA4_PORT=8080
```

Restart the server. It now runs on port 8080.

Setting the port. Three ways, and the framework default if you use none of them:

```
tina4 serve --port 8080        # for one run

TINA4_PORT=8080                # in .env, for every run
```

A bare `PORT` variable is also read, for hosts that set it, but it is deprecated and goes away in 3.14 -- use `TINA4_PORT`. The defaults per framework are PHP 7145, Python 7146, Ruby 7147 and Node.js 7148.

For the complete `.env` reference with all 68 variables, see Environment Variables.

---

7. The Dev Dashboard

With `TINA4_DEBUG=true` in your `.env`, Tina4 provides a built-in development dashboard. No additional environment variables are needed.

Restart the server and navigate to:

```
http://localhost:7146/__dev
```

The dashboard shows:

- **System Overview** -- framework version, Python version, uptime, memory usage, database status
- **Request Inspector** -- recent HTTP requests with method, path, status, duration, and request ID. Click any request to see full headers, body, database queries, and template renders.
- **Error Log** -- unhandled exceptions with stack traces and occurrence counts
- **Queue Manager** -- queue status (pending, reserved, failed, dead-letter messages)
- **WebSocket Monitor** -- active WebSocket connections with metadata
- **Routes** -- all registered routes with their methods, paths, and middleware

The dev dashboard shows you what your application is doing without adding print statements or log calls.

When you visit any HTML page (like `/products`), a **debug overlay** appears -- a toolbar at the bottom showing:

- Request details (method, URL, duration)
- Database queries executed (with timing)
- Template renders (with timing)
- Session data
- Recent log entries

This overlay is only visible when `TINA4_DEBUG=true`. Production never sees it.

---

8. Manual Setup (No CLI)

The `tina4` CLI scaffolds everything for you. But sometimes you create a project from scratch: an empty folder, a fresh virtual environment, no CLI installed yet. Here is the minimum you need.

Step 1: Install the Package

```
pip install tina4-python
```

Or with `uv`:

```
uv add tina4-python
```

Step 2: Create `app.py`

This is the entry point. Create a file called `app.py` in your project root:

```
"""Tina4 Application."""
from tina4_python.core import run

if __name__ == "__main__":
    run()
```

That is the entire file. The `run()` function starts the web server, scans for routes, and loads templates.

Step 3: Create the Folder Structure

Tina4 expects this layout:

```
my-project/
■■■ app.py
■■■ .env
■■■ src/
    ■■■ routes/       # Route files go here
    ■■■ templates/    # Twig templates go here
    ■■■ public/       # Static files (CSS, JS, images)
```

Create the directories:

```
mkdir -p src/routes src/templates src/public
```

On Windows, from PowerShell:

```
New-Item -ItemType Directory -Force -Path src/routes, src/templates, src/public
```

Step 4: Create `.env`

```
TINA4_DEBUG=true
```

Step 5: Run It

```
tina4 serve
```

The server starts on `http://localhost:7146`. You should see the Tina4 welcome page. From here, add route files in `src/routes/` and templates in `src/templates/`, the same way as a CLI-scaffolded project.

> ***Note:*** *Tina4 Python refuses to start without the Rust CLI. If you genuinely need to bypass it (e.g. inside a Docker image that already wraps the framework), set* `TINA4_OVERRIDE_CLIENT=true` *in* `.env` *and run* `uv run python app.py` *directly.*

---

9. Request & Response Fundamentals

Before jumping into the exercises, let's consolidate how route handlers work in Tina4 Python. Every handler receives two objects: `request` (what the client sent) and `response` (what you send back). Here is the complete picture.

Reading Query Parameters

Query parameters are the key-value pairs after the `?` in a URL. Access them through `request.params`:

```
# URL: /api/search?q=laptop&page=2
request.params.get("q", "")       # "laptop"
request.params.get("page", "1")   # "2" (always a string)
request.params.get("sort", "name") # "name" (default -- param was not sent)
```

Reading URL Path Parameters

Route patterns like `/users/{id}` capture segments of the URL. In Tina4 Python, path parameters arrive as function arguments:

```
from tina4_python.core.router import get

@get("/users/{id:int}/posts/{slug}")
async def user_post(id, slug, request, response):
    # id = 5 (int, because of :int type hint)
    # slug = "hello-world" (string)
    return response.json({"user_id": id, "slug": slug})
```

The `{id:int}` syntax tells Tina4 to convert the value to an integer. Without `:int`, it stays a string.

Reading the Request Body

POST, PUT, and PATCH requests carry a body. Tina4 parses JSON bodies into a dictionary automatically (as long as the client sends `Content-Type: application/json`):

```
from tina4_python.core.router import post

@post("/api/items")
```

```
async def create_item(request, response):
    name = request.body.get("name", "")
    price = request.body.get("price", 0)
    return response.json({"received_name": name, "received_price": price})
```

Reading Headers

Headers are available as a dictionary. Header names are case-insensitive:

```
content_type = request.headers.get("Content-Type", "not set")
auth_token = request.headers.get("Authorization", "")
custom = request.headers.get("X-Custom-Header", "")
```

Sending JSON Responses

`response.json()` converts a dictionary to JSON and sets the correct `Content-Type`. Pass a status code as the second argument:

```
return response.json({"id": 1, "name": "Widget"})        # 200 OK (default)
return response.json({"id": 1, "name": "Widget"}, 201)    # 201 Created
return response.json({"error": "Not found"}, 404)          # 404 Not Found
```

Sending HTML / Template Responses

`response.render()` renders a Frond template from `src/templates/` and passes data to it:

```
return response.render("products.html", {"products": product_list, "title": "Our Products"})
```

For raw HTML without a template file:

```
return response.html("<h1>Hello</h1><p>This works too.</p>")
```

Status Codes

The most common status codes you will use:

Code	Meaning	When to Use
`200`	OK	Successful GET (default)
`201`	Created	Successful POST that created something
`400`	Bad Request	Client sent invalid input
`404`	Not Found	Resource does not exist
`500`	Internal Server Error	Something broke on the server

Worked Example: A Complete Route File

Here is a full route file that ties everything together. It builds a small book lookup API with query parameters, path parameters, JSON responses, and proper status codes. Read through it before attempting the exercises -- it is your reference.

Create `src/routes/books.py`:

```
from tina4_python.core.router import get, post

# In-memory data store
books = [
    {"id": 1, "title": "Dune", "author": "Frank Herbert", "year": 1965},
    {"id": 2, "title": "Neuromancer", "author": "William Gibson", "year": 1984},
    {"id": 3, "title": "Snow Crash", "author": "Neal Stephenson", "year": 1992}
]


@get("/api/books")
async def list_books(request, response):
    """List all books. Supports ?author= filter and ?sort=year."""
    author = request.params.get("author", "")
    sort_by = request.params.get("sort", "")

    result = books

    # Filter by author if the query param is present
    if author:
        result = [b for b in result if author.lower() in b["author"].lower()]

    # Sort by year if requested
    if sort_by == "year":
        result = sorted(result, key=lambda b: b["year"])

    return response.json({"books": result, "count": len(result)})


@get("/api/books/{id:int}")
async def get_book(id, request, response):
    """Get a single book by ID. Returns 404 if not found."""
    book = next((b for b in books if b["id"] == id), None)

    if book is None:
        return response.json({"error": f"Book with id {id} not found"}, 404)

    return response.json(book)


@post("/api/books")
async def create_book(request, response):
    """Create a new book from the JSON body. Returns 201 on success."""
    title = request.body.get("title", "")
    author = request.body.get("author", "")
    year = request.body.get("year", 0)

    if not title or not author:
        return response.json({"error": "title and author are required"}, 400)

    new_book = {
        "id": max(b["id"] for b in books) + 1,
        "title": title,
        "author": author,
        "year": year
    }
    books.append(new_book)

    return response.json(new_book, 201)
```

Test it:

```
# List all books
curl http://localhost:7146/api/books

# Filter by author
curl "http://localhost:7146/api/books?author=gibson"

# Sort by year
curl "http://localhost:7146/api/books?sort=year"

# Get a single book
curl http://localhost:7146/api/books/2

# Get a book that does not exist (returns 404)
curl http://localhost:7146/api/books/99

# Create a new book
curl -X POST http://localhost:7146/api/books \
  -H "Content-Type: application/json" \
  -d '{"title": "Foundation", "author": "Isaac Asimov", "year": 1951}'
```

This example covers every building block the exercises use: reading query parameters, reading path parameters, reading the request body, returning JSON with different status codes, and handling missing data. Refer back to it as you work through the exercises below.

---

10. Exercise: Greeting API + Product List Template

Build the following two features from scratch, without looking at the examples above.

Exercise Part A: Greeting API

Create an API endpoint at `GET /api/greet` that:

- Accepts a query parameter `name` (e.g., `/api/greet?name=Sarah`)
- If `name` is missing, defaults to `"Stranger"`
- Returns JSON like:

```
{
  "greeting": "Welcome, Sarah!",
  "time_of_day": "afternoon"
}
```

- The `time_of_day` should be calculated from the server's current hour:
- 5:00 - 11:59 = "morning"
- 12:00 - 16:59 = "afternoon"
- 17:00 - 20:59 = "evening"
- 21:00 - 4:59 = "night"

Test your endpoint with:

```
curl "http://localhost:7146/api/greet?name=Sarah"
curl "http://localhost:7146/api/greet"
```

Exercise Part B: Product List Page

Create a page at `GET /store` that:

- Displays a list of at least 5 products (hardcoded for now)
- Each product has: name, category, price, and a boolean `featured` flag
- Featured products get a visual highlight (different background color, border, or badge)
- The page shows the total number of products and the number of featured products
- Uses template inheritance -- a layout template and a page template that extends it
- Includes `tina4.css` and `frond.js`

Your products data should look like this in your route handler:

```
products = [
    {"name": "Espresso Machine", "category": "Kitchen", "price": 299.99, "featured": True},
    {"name": "Yoga Mat", "category": "Fitness", "price": 29.99, "featured": False},
    {"name": "Standing Desk", "category": "Office", "price": 549.99, "featured": True},
    {"name": "Noise-Canceling Headphones", "category": "Electronics", "price": 199.99, "featured
    {"name": "Water Bottle", "category": "Fitness", "price": 24.99, "featured": False}
]
```

Expected browser output:

- A page titled "Our Store"
- Text showing "5 products, 3 featured"
- A list of product cards with name, category, price, and a "Featured" badge on highlighted items
- Featured products have a distinct visual style (your choice -- different border color, background, star icon, etc.)

---

11. Solutions

Solution A: Greeting API

Create `src/routes/greet.py`:

```
from tina4_python.core.router import get

@get("/api/greet")
async def greet(request, response):
    name = request.params.get("name", "Stranger")

    from datetime import datetime
    hour = datetime.now().hour

    if 5 <= hour < 12:
        time_of_day = "morning"
    elif 12 <= hour < 17:
        time_of_day = "afternoon"
    elif 17 <= hour < 21:
        time_of_day = "evening"
```

```
    else:
        time_of_day = "night"

    return response.json({
        "greeting": f"Welcome, {name}!",
        "time_of_day": time_of_day
    })
```

Test:

```
curl "http://localhost:7146/api/greet?name=Sarah"

{"greeting":"Welcome, Sarah!","time_of_day":"afternoon"}

curl "http://localhost:7146/api/greet"

{"greeting":"Welcome, Stranger!","time_of_day":"afternoon"}
```

Solution B: Product List Page

Create `src/templates/store-layout.html`:

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>{% block title %}Store{% endblock %}</title>
    <link rel="stylesheet" href="/css/tina4.css">
    <style>
        body { font-family: -apple-system, BlinkMacSystemFont, "Segoe UI", Roboto, sans-serif; m
        .container { max-width: 960px; margin: 0 auto; padding: 20px; }
        header { background: #1a1a2e; color: white; padding: 16px 20px; }
        header h1 { margin: 0; }
        .stats { color: #888; margin: 8px 0 20px; }
        .product-grid { display: grid; gap: 16px; }
        .product-card { background: white; border: 2px solid #e0e0e0; border-radius: 8px; paddin
        .product-card.featured { border-color: #ffc107; background: #fffdf0; }
        .product-name { font-size: 1.2em; font-weight: bold; margin: 0 0 4px; }
        .product-category { color: #666; font-size: 0.9em; }
        .product-price { color: #2d8f2d; font-weight: bold; font-size: 1.1em; margin-top: 8px; }
        .featured-badge { background: #ffc107; color: #333; padding: 2px 8px; border-radius: 4px
    </style>
</head>
<body>
    <header>
        <h1>{% block header %}Store{% endblock %}</h1>
    </header>
    <div class="container">
        {% block content %}{% endblock %}
    </div>
    <script src="/js/frond.js"></script>
</body>
</html>
```

Create `src/templates/store.html`:

```
{% extends "store-layout.html" %}

{% block title %}Our Store{% endblock %}
{% block header %}Our Store{% endblock %}
```

```
{% block content %}
    <p class="stats">{{ products | length }} products, {{ featured_count }} featured</p>

    <div class="product-grid">
        {% for product in products %}
            <div class="product-card{{ ' featured' if product.featured else '' }}">
                <p class="product-name">
                    {{ product.name }}
                    {% if product.featured %}
                        <span class="featured-badge">Featured</span>
                    {% endif %}
                </p>
                <p class="product-category">{{ product.category }}</p>
                <p class="product-price">${{ "%.2f"|format(product.price) }}</p>
            </div>
        {% endfor %}
    </div>
{% endblock %}
```

Create `src/routes/store.py`:

```
from tina4_python.core.router import get

@get("/store")
async def store_page(request, response):
    products = [
        {"name": "Espresso Machine", "category": "Kitchen", "price": 299.99, "featured": True},
        {"name": "Yoga Mat", "category": "Fitness", "price": 29.99, "featured": False},
        {"name": "Standing Desk", "category": "Office", "price": 549.99, "featured": True},
        {"name": "Noise-Canceling Headphones", "category": "Electronics", "price": 199.99, "feat
        {"name": "Water Bottle", "category": "Fitness", "price": 24.99, "featured": False}
    ]

    featured_count = sum(1 for p in products if p["featured"])

    return response.render("store.html", {
        "products": products,
        "featured_count": featured_count
    })
```

Open `http://localhost:7146/store` in your browser. You should see:

- A dark header reading "Our Store"
- Text showing "5 products, 3 featured"
- Five product cards in a grid
- Three cards (Espresso Machine, Standing Desk, Noise-Canceling Headphones) have a yellow border, light yellow background, and a "Featured" badge
- Two cards (Yoga Mat, Water Bottle) have a standard white background with gray border
- Each card shows the product name, category, and price formatted with two decimal places

---

12. Gotchas

1. File not auto-discovered

Problem: You created a route file but nothing happens when you visit the URL.

Cause: The file is not in `src/routes/`. It must be inside `src/routes/` (or a subdirectory), and the file must end with `.py`. Names beginning with `_` are treated as private and not loaded.

Fix: Move the file to `src/routes/your-file.py` and restart the server.

2. Route file did not import

Problem: The URL 404s.

Cause: The file did not import, so none of its routes registered -- a missing decorator import is the usual reason. The framework logs the reason and keeps serving the rest of the app.

Fix: Read the server output. The reason is there:

```
[ERROR] Failed to load /home/you/my-store/src/routes/greeting.py: name 'get' is not defined
```

Start your route file with the correct import: `from tina4_python.core.router import get, post` (include whichever decorators you need). A route that 404s unexpectedly is usually a file that did not load.

3. Template not found

Problem: `Template "my-page.html" not found` error.

Cause: The template file is not in `src/templates/`, or there is a typo in the filename.

Fix: Check that the file exists at `src/templates/my-page.html`. The name in `response.render()` is relative to `src/templates/`.

4. Port already in use

Problem: Another process is on port 7146.

Cause: Another process -- often a Tina4 server you forgot to stop.

Fix: Stop the other process, or move Tina4 with the CLI flag:

```
tina4 serve --port 8080
```

Or set `TINA4_PORT` in `.env` if you want it to stick.

5. Changes not reflected

Problem: You edited a file but the browser shows the old version.

Cause: Live reload may not be active. Browser caching can serve stale versions.

Fix: Hard-refresh the browser (`Ctrl+Shift+R` or `Cmd+Shift+R`). If that fails, restart the dev server with `Ctrl+C` and `tina4 serve`.

6. .env not loaded

Problem: Environment variables have no effect.

Cause: The `.env` file must be at the project root (same directory as `pyproject.toml`). A subdirectory placement hides it from Tina4.

Fix: Move `.env` to the project root.

Routing

1. How Routing Works in Tina4

Every web application maps URLs to code. A browser requests `/products`. The framework finds the handler for `/products`, runs it, sends back the result. That mapping is routing.

In Tina4 Python, routes live in Python files inside `src/routes/`. Every `.py` file in that directory (and its subdirectories) is auto-loaded at startup. No registration file. No central config. Drop a file in. It works.

The simplest route:

```
from tina4_python.core.router import get

@get("/hello")
async def hello(request, response):
    return response.json({"message": "Hello, World!"})
```

Save that as `src/routes/hello.py`, start the server with `tina4 serve`, and visit `http://localhost:7146/hello`:

```
{"message":"Hello, World!"}
```

One decorator. One function. Done.

---

2. HTTP Methods

Tina4 supports all five standard HTTP methods. Each has a decorator:

```
from tina4_python.core.router import get, post, put, patch, delete

@get("/products")
async def list_products(request, response):
    return response.json({"action": "list all products"})

@post("/products")
async def create_product(request, response):
    return response.json({"action": "create a product"}, 201)

@put("/products/{id}")
async def replace_product(id, request, response):
    return response.json({"action": f"replace product {id}"})

@patch("/products/{id}")
async def update_product(id, request, response):
    return response.json({"action": f"update product {id}"})

@delete("/products/{id}")
async def delete_product(id, request, response):
    return response.json({"action": f"delete product {id}"})
```

Test each one:

```
curl http://localhost:7146/products
```

```
{"action":"list all products"}

curl -X POST http://localhost:7146/products \
  -H "Content-Type: application/json" \
  -d '{"name": "Widget"}'

{"action":"create a product"}

curl -X PUT http://localhost:7146/products/42

{"action":"replace product 42"}

curl -X PATCH http://localhost:7146/products/42

{"action":"update product 42"}

curl -X DELETE http://localhost:7146/products/42

{"action":"delete product 42"}
```

`GET` reads. `POST` creates. `PUT` replaces. `PATCH` patches. `DELETE` removes. REST convention. Predictable API.

HEAD and OPTIONS: automatic, no boilerplate

Tina4 handles `HEAD` and `OPTIONS` for you. **You don't register them.** They follow from your `GET` / `POST` / etc. routes:

- `HEAD` is identical to `GET` except the response carries no body (RFC 9110 §9.3.2). Every `GET` route automatically responds to `HEAD`: the framework strips the response body on the way out and preserves `Content-Length` pointing at the byte count the equivalent `GET` would have sent. Cache validators, link checkers, and uptime probes work without you writing anything.
- `OPTIONS` on any registered path returns `204 No Content` with an `Allow:` header listing every method the path supports (RFC 9110 §9.3.7). Useful for clients that need to discover the API surface.
- **Wrong method on an existing path** returns `405 Method Not Allowed` with the same `Allow:` header (RFC 9110 §15.5.6 + §10.2.1). Sending `PUT` to a `GET`-only route used to return `404`, which was semantically wrong and confusing for link checkers. Now you get a real `405` that says exactly which methods are allowed.
- `TRACE` **and** `CONNECT` are rejected with `405` (we never support them, a security default for origin servers).

```
# HEAD on a GET route - same headers, empty body
curl -sI http://localhost:7146/products
# HTTP/1.1 200 OK
# Content-Type: application/json
# Content-Length: 33

# OPTIONS - discover what the path supports
curl -sI -X OPTIONS http://localhost:7146/products
# HTTP/1.1 204 No Content
# Allow: GET, POST, HEAD, OPTIONS

# Wrong method - clear 405 with Allow header
curl -sI -X PUT http://localhost:7146/products
# HTTP/1.1 405 Method Not Allowed
# Allow: GET, POST, HEAD, OPTIONS
```

Explicit `head()` and `options()` registration

The automatic behaviour is enough for 95% of apps. When you need custom HEAD or OPTIONS handlers, for example a cheap existence-check endpoint or a richer OPTIONS payload, register them explicitly:

```
from tina4_python.core.router import Router

# A HEAD handler that doesn't run the full GET body - useful for
# expensive endpoints where the client only needs to check existence
# or read validators (ETag, Last-Modified).
Router.head("/expensive/{id}", lambda req, res:
    res.header("ETag", compute_etag(req.params["id"]))
)

# An OPTIONS handler that returns more than just Allow - for example
# a CORS preflight with custom headers, or a discovery endpoint.
Router.options("/api/discovery", lambda req, res:
    res.json({"version": "v1", "endpoints": [...]})
)
```

The framework still strips the response body for `HEAD` handlers (RFC 9110 §9.3.2 is a MUST), so you can't accidentally leak content even if your handler returns a body.

---

3. Path Parameters

Path parameters capture values from the URL. Wrap the parameter name in curly braces:

```
from tina4_python.core.router import get

@get("/users/{id}/posts/{post_id}")
async def user_post(id, post_id, request, response):
    return response.json({
        "user_id": id,
        "post_id": post_id
    })
```

```
curl http://localhost:7146/users/5/posts/99
```

```
{"user_id":"5","post_id":"99"}
```

Notice `user_id` came back as the string `"5"`, not the integer `5`. Path parameters are strings by default. In Python, path parameters are passed as function arguments -- the parameter names in the function signature must match the `{name}` placeholders in the route pattern.

> ***Auto-casting:*** *Tina4 automatically casts path parameter values that are purely numeric to integers. For example, requesting `/users/42/posts/99` will pass `id` as the integer `42` and `post_id` as the integer `99` to your handler -- no explicit `:int` type hint required. The `:int` type hint adds validation (rejecting non-numeric values with a 404), but the auto-casting happens regardless.*

Typed Parameters

Enforce a type by adding a colon and the type after the parameter name:

```
from tina4_python.core.router import get
```

```
@get("/orders/{id:int}")
async def get_order(id, request, response):
    # id is already an integer thanks to :int
    return response.json({
        "order_id": id,
        "type": type(id).__name__
    })
```

```
curl http://localhost:7146/orders/42
```

```
{"order_id":42,"type":"int"}
```

Pass a non-integer value and the route does not match. A 404:

```
curl http://localhost:7146/orders/abc
```

```
{"error":"Not found","path":"/orders/abc","status":404}
```

Supported types:

Type	Matches	Auto-cast	Example
`int`	Digits only	Integer	`{id:int}` matches `42` but not `abc`
`float`	Decimal numbers	Float	`{price:float}` matches `19.99`
`path`	All remaining path segments (catch-all)	String	`{slug:path}` matches `docs/api/auth`

The `{name}` form (no type) matches any single path segment and returns it as a string.

Typed Parameters in Action

Here is a complete example showing the most commonly used typed parameters together:

```
from tina4_python.core.router import get

# Integer parameter -- only digits match, auto-cast to int
@get("/products/{id:int}")
async def get_product(id, request, response):
    # id is an integer, e.g. 42
    return response.json({
        "product_id": id,
        "type": type(id).__name__
    })

# Float parameter -- decimal numbers, auto-cast to float
@get("/products/{id:int}/price/{price:float}")
async def check_price(id, price, request, response):
    return response.json({
        "product_id": id,
        "price": price,
        "type": type(price).__name__
    })

# Path parameter -- catch-all, captures remaining segments as a string
```

```
@get("/files/{filepath:path}")
async def serve_file(filepath, request, response):
    # filepath could be "images/photos/cat.jpg"
    return response.json({
        "filepath": filepath,
        "type": type(filepath).__name__
    })
```

```
# Integer route -- matches digits, returns an int
curl http://localhost:7146/products/42
```

```
{"product_id":42,"type":"int"}
```

```
# Integer route -- non-integer gives a 404
curl http://localhost:7146/products/abc
```

```
{"error":"Not found","path":"/products/abc","status":404}
```

```
# Path catch-all -- captures everything after /files/
curl http://localhost:7146/files/images/photos/cat.jpg
```

```
{"filepath":"images/photos/cat.jpg","type":"str"}
```

The `:int` and `:float` types act as both a constraint and a converter. If the URL segment does not match the expected pattern, the route is skipped entirely and Tina4 moves on to the next registered route (or returns 404 if nothing matches). The `:path` type is greedy -- it consumes all remaining segments, making it ideal for file paths and documentation URLs.

---

4. Query Parameters

Query parameters are the key-value pairs after the `?` in a URL. Access them through `request.params`:

```
from tina4_python.core.router import get

@get("/search")
async def search(request, response):
    q = request.params.get("q", "")
    page = int(request.params.get("page", 1))
    limit = int(request.params.get("limit", 10))

    return response.json({
        "query": q,
        "page": page,
        "limit": limit,
        "offset": (page - 1) * limit
    })
```

```
curl "http://localhost:7146/search?q=keyboard&page=2&limit=20"
```

```
{"query":"keyboard","page":2,"limit":20,"offset":20}
```

If a query parameter is missing, `request.params.get("key")` returns `None`. Use `.get()` with a default value.

---

5. Route Groups

A set of routes sharing a common prefix belongs in a `group()`:

```
from tina4_python.core.router import Router

Router.group("/api/v1", lambda group: [
    group.get("/users", list_users),
    group.get("/users/{id:int}", get_user),
    group.post("/users", create_user),
    group.get("/products", list_products),
])

async def list_users(request, response):
    return response.json({"users": []})

async def get_user(id, request, response):
    return response.json({"user": {"id": id, "name": "Alice"}})

async def create_user(request, response):
    return response.json({"created": True}, 201)

async def list_products(request, response):
    return response.json({"products": []})
```

These routes register as `/api/v1/users`, `/api/v1/users/{id}`, and `/api/v1/products`. Short paths inside the group. Tina4 prepends the prefix. `Router.group()` is a classmethod that takes a prefix, a callback that receives a `RouteGroup` object, and an optional middleware list. The callback **must accept the group argument**: call `group.get(...)`, `group.post(...)`, etc. on it so the prefix and middleware propagate to nested routes. Using a zero-arg `lambda:` raises `TypeError: <lambda>() takes 0 positional arguments but 1 was given`.

```
curl http://localhost:7146/api/v1/users
```

```
{"users":[]}
```

```
curl http://localhost:7146/api/v1/products
```

```
{"products":[]}
```

Groups nest:

```
from tina4_python.core.router import Router

async def v1_status(request, response):
    return response.json({"version": "1.0"})

async def v2_status(request, response):
    return response.json({"version": "2.0"})

Router.group("/api", lambda api: [
    api.group("/v1", lambda v1: [
        v1.get("/status", v1_status),
    ]),
    api.group("/v2", lambda v2: [
        v2.get("/status", v2_status),
    ]),
])
```

```
curl http://localhost:7146/api/v1/status

{"version":"1.0"}

curl http://localhost:7146/api/v2/status

{"version":"2.0"}
```

---

6. Middleware on Routes

Middleware is code that runs before or after your route handler. Authentication. Logging. Rate limiting. Input validation. Anything that belongs on multiple routes. Chapter 10 covers middleware in depth. Here is how it connects to routes.

Middleware on a Single Route

Attach middleware with the `@middleware` decorator. Middleware classes use `before_*` and `after_*` methods:

```
from tina4_python.core.router import get, middleware
import time

class LogRequest:
    def before_request(self, request, response):
        print(f"[{time.strftime('%Y-%m-%d %H:%M:%S')}] {request.method} {request.path}")
        return request, response

    def after_request(self, request, response):
        print(f"  Completed: {response.status_code}")
        return request, response

@middleware(LogRequest)
@get("/api/data")
async def get_data(request, response):
    return response.json({"data": [1, 2, 3]})
```

Middleware classes have `before_*` methods (called before the handler) and `after_*` methods (called after). If a `before_*` method sets `response.status_code` to 400 or above, the handler is skipped entirely. Useful for blocking unauthorized requests.

Blocking Middleware

Middleware that checks for an API key:

```
from tina4_python.core.router import get, middleware

class RequireApiKey:
    def before_check(self, request, response):
        api_key = request.headers.get("x-api-key", "")
        if api_key != "my-secret-key":
            return request, response.json({"error": "Invalid API key"}, 401)
        return request, response

@middleware(RequireApiKey)
@get("/api/secret")
async def secret_data(request, response):
    return response.json({"secret": "The answer is 42"})
```

```
curl http://localhost:7146/api/secret

{"error":"Invalid API key"}
```

Status: `401 Unauthorized`.

```
curl http://localhost:7146/api/secret -H "X-API-Key: my-secret-key"

{"secret":"The answer is 42"}
```

Multiple Middleware

Chain multiple middleware classes as arguments:

```
@middleware(LogRequest, RequireApiKey)
@get("/api/important")
async def important_data(request, response):
    return response.json({"data": "important stuff"})
```

Middleware runs left to right: `LogRequest` first, then `RequireApiKey`, then the route handler. If any middleware's `before_*` method returns a response with status 400+, the chain stops and the handler is skipped.

---

7. Route Decorators: @noauth() and @secured()

Two decorators control authentication at the route level.

@noauth() -- Public Routes

When your application has global authentication middleware, `@noauth()` marks a route as public:

```
from tina4_python.core.router import get, noauth

@noauth()
@get("/api/public/info")
async def public_info(request, response):
    return response.json({
        "app": "My Store",
        "version": "1.0.0"
    })
```

`@noauth()` tells Tina4 to skip authentication checks for this route, even if global auth middleware is configured in `.env` or applied to the parent group.

@secured() -- Protected GET Routes

`@secured()` marks a GET route as requiring authentication:

```
from tina4_python.core.router import get, secured

@secured()
@get("/api/profile")
async def profile(request, response):
    # request.user is populated by the auth middleware
    return response.json({
        "user": request.user
```

```
})
```

By default, `POST`, `PUT`, `PATCH`, and `DELETE` routes are secured. `GET` routes are public unless you add `@secured()`. This matches the common pattern: reading data is public, modifying data requires authentication.

---

8. Route Chaining: .secure() and .cache()

Route decorators return a chainable object. Two methods you can call on any route: `.secure()` and `.cache()`.

.secure()

`.secure()` requires a valid bearer token in the `Authorization` header. If the token is missing or invalid, the route returns `401 Unauthorized` without ever reaching your handler:

```
from tina4_python.core.router import get

@get("/api/account")
async def get_account(request, response):
    return response.json({"account": request.user})

get_account.secure()
```

Or chain it inline using the `Router` class directly:

```
from tina4_python.core.router import Router

Router.get("/api/account", get_account).secure()
```

```
curl http://localhost:7146/api/account
# 401 Unauthorized

curl http://localhost:7146/api/account -H "Authorization: Bearer eyJhbGci..."
# 200 OK
```

.cache()

`.cache()` enables response caching for the route. Once the handler runs and produces a response, subsequent requests to the same URL return the cached result without re-executing the handler:

```
Router.get("/api/catalog", list_catalog).cache()
```

Chaining Both

Chain `.secure()` and `.cache()` together:

```
Router.get("/api/data", handler).secure().cache()
```

This route requires a bearer token and caches the response. Order does not matter -- `.cache().secure()` produces the same result.

---

9. Wildcard and Catch-All Routes

Wildcard Routes

Use `*` at the end of a path to match anything after it:

```
from tina4_python.core.router import get

@get("/docs/*")
async def docs_handler(request, response):
    path = request.params.get("*", "")
    return response.json({
        "section": "docs",
        "path": path
    })
```

```
curl http://localhost:7146/docs/getting-started
```

```
{"section":"docs","path":"getting-started"}
```

```
curl http://localhost:7146/docs/api/authentication/jwt
```

```
{"section":"docs","path":"api/authentication/jwt"}
```

Catch-All Route (Custom 404)

A catch-all handles any unmatched URL:

```
from tina4_python.core.router import get

@get("/*")
async def not_found(request, response):
    return response.json({
        "error": "Page not found",
        "path": request.path
    }, 404)
```

Define this route last (or in a file that sorts alphabetically after your other route files). Tina4 matches routes in registration order -- first match wins.

You can also create a custom 404 page by placing a template at `src/templates/errors/404.html`:

```
{% extends "base.html" %}

{% block title %}Not Found{% endblock %}

{% block content %}
    <h1>404 - Page Not Found</h1>
    <p>The page you are looking for does not exist.</p>
    <a href="/">Go back home</a>
{% endblock %}
```

Tina4 uses this template for any unmatched route when the file exists.

---

10. Route Listing via CLI

As your application grows, see all registered routes at a glance:

```
tina4 routes

Method   Path                          Middleware          Auth
------   ----                          ----------          ----
GET      /hello                        -                   public
GET      /products                     -                   public
POST     /products                     -                   secured
PUT      /products/{id}                -                   secured
PATCH    /products/{id}                -                   secured
DELETE   /products/{id}                -                   secured
GET      /api/v1/users                 -                   public
GET      /api/v1/users/{id:int}        -                   public
POST     /api/v1/users                 -                   secured
GET      /api/public/info              -                   @noauth
GET      /api/profile                  -                   @secured
GET      /search                       -                   public
GET      /docs/*                       -                   public
```

The `Auth` column shows whether a route is public, secured (default for non-GET methods), `@noauth`, or `@secured`.

---

11. Organizing Route Files

Organize route files any way you want. Tina4 loads every `.py` file in `src/routes/` recursively. Two common patterns:

Pattern 1: One File Per Resource

```
src/routes/
■■■ products.py     # All product routes
■■■ users.py        # All user routes
■■■ orders.py       # All order routes
■■■ pages.py        # HTML page routes
```

Pattern 2: Subdirectories by Feature

```
src/routes/
■■■ api/
■   ■■■ products.py
■   ■■■ users.py
■   ■■■ orders.py
■■■ admin/
■   ■■■ dashboard.py
■   ■■■ settings.py
■■■ pages/
    ■■■ home.py
    ■■■ about.py
```

Both work identically. The directory structure has no effect on URL paths -- only the route definitions inside the files matter. Choose whichever keeps your project navigable.

---

12. Exercise: Build a Full CRUD API for Products

Build a complete REST API for managing products. All data stored in a Python list (no database yet -- Chapter 5 adds that).

Requirements

Create a file `src/routes/product_api.py` with the following routes:

Method	Path	Description
`GET`	`/api/products`	List all products. Support `?category=` filter.
`GET`	`/api/products/{id:int}`	Get a single product by ID. Return 404 if not found.
`POST`	`/api/products`	Create a new product. Return 201.
`PUT`	`/api/products/{id:int}`	Replace a product. Return 404 if not found.
`DELETE`	`/api/products/{id:int}`	Delete a product. Return 204 with no body.

Each product has: `id` (int), `name` (string), `category` (string), `price` (float), `in_stock` (bool).

Start with this seed data:

```
products = [
    {"id": 1, "name": "Wireless Keyboard", "category": "Electronics", "price": 79.99, "in_stock"
    {"id": 2, "name": "Yoga Mat", "category": "Fitness", "price": 29.99, "in_stock": True},
    {"id": 3, "name": "Coffee Grinder", "category": "Kitchen", "price": 49.99, "in_stock": False
    {"id": 4, "name": "Standing Desk", "category": "Office", "price": 549.99, "in_stock": True},
    {"id": 5, "name": "Running Shoes", "category": "Fitness", "price": 119.99, "in_stock": True}
]
```

Test with:

```
# List all
curl http://localhost:7146/api/products

# Filter by category
curl "http://localhost:7146/api/products?category=Fitness"

# Get one
curl http://localhost:7146/api/products/3

# Create
curl -X POST http://localhost:7146/api/products \
  -H "Content-Type: application/json" \
  -d '{"name": "Desk Lamp", "category": "Office", "price": 39.99, "in_stock": true}'

# Update
curl -X PUT http://localhost:7146/api/products/3 \
  -H "Content-Type: application/json" \
```

```
  -d '{"name": "Burr Coffee Grinder", "category": "Kitchen", "price": 59.99, "in_stock": true}'

# Delete
curl -X DELETE http://localhost:7146/api/products/3

# Not found
curl http://localhost:7146/api/products/999
```

---

13. Solution

Create `src/routes/product_api.py`:

```
from tina4_python.core.router import get, post, put, delete

# In-memory product store (resets on server restart)
products = [
    {"id": 1, "name": "Wireless Keyboard", "category": "Electronics", "price": 79.99, "in_stock"
    {"id": 2, "name": "Yoga Mat", "category": "Fitness", "price": 29.99, "in_stock": True},
    {"id": 3, "name": "Coffee Grinder", "category": "Kitchen", "price": 49.99, "in_stock": False
    {"id": 4, "name": "Standing Desk", "category": "Office", "price": 549.99, "in_stock": True},
    {"id": 5, "name": "Running Shoes", "category": "Fitness", "price": 119.99, "in_stock": True}
]

next_id = 6


# List all products, optionally filter by category
@get("/api/products")
async def list_products(request, response):
    category = request.params.get("category")

    if category is not None:
        filtered = [p for p in products if p["category"].lower() == category.lower()]
        return response.json({"products": filtered, "count": len(filtered)})

    return response.json({"products": products, "count": len(products)})


# Get a single product by ID
@get("/api/products/{id:int}")
async def get_product(id, request, response):
    for product in products:
        if product["id"] == id:
            return response.json(product)

    return response.json({"error": "Product not found", "id": id}, 404)


# Create a new product
@post("/api/products")
async def create_product(request, response):
    global next_id
    body = request.body

    if not body.get("name"):
        return response.json({"error": "Name is required"}, 400)
```

```
    product = {
        "id": next_id,
        "name": body["name"],
        "category": body.get("category", "Uncategorized"),
        "price": float(body.get("price", 0)),
        "in_stock": bool(body.get("in_stock", True))
    }
    next_id += 1

    products.append(product)

    return response.json(product, 201)


# Replace a product
@put("/api/products/{id:int}")
async def replace_product(id, request, response):
    body = request.body

    for i, product in enumerate(products):
        if product["id"] == id:
            products[i] = {
                "id": id,
                "name": body.get("name", product["name"]),
                "category": body.get("category", product["category"]),
                "price": float(body.get("price", product["price"])),
                "in_stock": bool(body.get("in_stock", product["in_stock"]))
            }
            return response.json(products[i])

    return response.json({"error": "Product not found", "id": id}, 404)


# Delete a product
@delete("/api/products/{id:int}")
async def delete_product(id, request, response):
    for i, product in enumerate(products):
        if product["id"] == id:
            products.pop(i)
            return response.json(None, 204)

    return response.json({"error": "Product not found", "id": id}, 404)
```

Expected output for the test commands:

List all:

```
{"products":[{"id":1,"name":"Wireless Keyboard","category":"Electronics","price":79.99,"in_stock
```

Filter by category:

```
{"products":[{"id":2,"name":"Yoga Mat","category":"Fitness","price":29.99,"in_stock":true},{"id"
```

Get one:

```
{"id":3,"name":"Coffee Grinder","category":"Kitchen","price":49.99,"in_stock":false}
```

Create:

```
{"id":6,"name":"Desk Lamp","category":"Office","price":39.99,"in_stock":true}
```

(Status: `201 Created`)

Update:

```
{"id":3,"name":"Burr Coffee Grinder","category":"Kitchen","price":59.99,"in_stock":true}
```

Delete: empty response with status `204 No Content`.

Not found:

```
{"error":"Product not found","id":999}
```

(Status: `404 Not Found`)

---

14. Gotchas

1. Trailing slashes matter

Problem: `/products` works but `/products/` returns a 404 (or vice versa).

Cause: Tina4 treats `/products` and `/products/` as different routes by default.

Fix: Pick one convention and stick with it. If you want both to work, register the route without a trailing slash -- Tina4 redirects `/products/` to `/products` when `TINA4_TRAILING_SLASH_REDIRECT=true` is set in `.env`.

2. Parameter names must be unique in a path

Problem: `/users/{id}/posts/{id}` behaves wrong -- both parameters share a name.

Cause: The second `{id}` overwrites the first in `request.params`.

Fix: Use distinct names: `/users/{user_id}/posts/{post_id}`.

3. Method conflicts

Problem: You defined `@get("/items/{id}")` and `@get("/items/{action}")` and the wrong handler runs.

Cause: Both patterns match `/items/42`. The first one registered wins.

Fix: Use typed parameters to disambiguate: `@get("/items/{id:int}")` matches integers only, leaving `/items/export` free for the other route. Or restructure your paths: `/items/{id:int}` and `/items/actions/{action}`.

4. Route handler must return a response

Problem: Your route handler runs but the browser shows an empty page or a 500 error.

Cause: You forgot the `return` statement. Without `return`, the handler returns `None` and Tina4 has nothing to send back.

Fix: Every handler must return something: `return response.json(...)` or `return response.html(...)` or `return response.render(...)`.

---

5. Decorator order matters

Problem: Your `@middleware` decorator has no effect, or your `@noauth()` is ignored.

Cause: Python decorators apply bottom-up. Wrong stacking order breaks registration.

Fix: Put the route decorator (`@get`, `@post`, etc.) first (closest to the function), then additional decorators above it:

```
@middleware(require_api_key)  # Applied second (wraps the route)
@get("/api/secret")           # Applied first (registers the route)
async def secret(request, response):
    ...
```

6. Forgetting async def

Problem: Your route handler raises a `TypeError` about a coroutine or the response is a coroutine object instead of JSON.

Cause: You used `def` instead of `async def`.

Fix: Every route handler in Tina4 Python must be `async def`. The framework runs on an async server. Change `def my_handler(request, response):` to `async def my_handler(request, response):`.

7. Group prefix must start with a slash

Problem: `Router.group("api/v1", ...)` produces routes that do not match.

Cause: The group prefix should start with `/` for consistency.

Fix: Start group prefixes with `/`: `Router.group("/api/v1", ...)`.

Request and Response

1. The Two Objects You Use Everywhere

Every route handler receives two arguments: `request` and `response`. The request carries what the client sent. The response builds what you send back. These two objects handle every HTTP scenario you will face.

A file-sharing service demonstrates the pattern. A user uploads a document. You validate the file type. You check the session. You return success JSON or an error page. Every step flows through `request` and `response`.

---

2. The Request Object

The `request` object opens every piece of the incoming HTTP request. Headers, body, cookies, files, IP address -- all accessible through named properties.

request.method

The HTTP method arrives as an uppercase string:

```
from tina4_python.core.router import get, post

@get("/api/check")
async def check_method(request, response):
    return response.json({"method": request.method})
```

```
curl http://localhost:7146/api/check
```

```
{"method":"GET"}
```

request.path

The URL path strips query parameters:

```
@get("/api/info")
async def info(request, response):
    return response.json({"path": request.path})
```

```
curl "http://localhost:7146/api/info?foo=bar"
```

```
{"path":"/api/info"}
```

Path Parameters (Function Arguments)

Path parameters captured from the URL pattern arrive as function arguments. The parameter names in your function signature must match the `{name}` placeholders in the route pattern:

```
@get("/users/{id:int}/posts/{slug}")
async def user_post(id, slug, request, response):
    return response.json({
        "user_id": id,
        "slug": slug
    })
```

```
curl http://localhost:7146/users/5/posts/hello-world
```

```
{"user_id":5,"slug":"hello-world"}
```

request.params

Query string parameters live in a dictionary. Each key holds a single value unless the same key appears multiple times -- then Tina4 stores a list:

```
@get("/search")
async def search(request, response):
    return response.json({
        "q": request.params.get("q", ""),
        "page": int(request.params.get("page", 1)),
        "sort": request.params.get("sort", "relevance")
    })
```

```
curl "http://localhost:7146/search?q=laptop&page=2&sort=price"
```

```
{"q":"laptop","page":2,"sort":"price"}
```

request.body

The parsed request body. JSON requests become a dictionary. Form submissions contain form fields. GET requests produce `None`:

```
@post("/api/feedback")
async def feedback(request, response):
    name = request.body.get("name", "Anonymous")
    message = request.body.get("message", "")
    rating = request.body.get("rating", 0)

    return response.json({
        "received": {
            "name": name,
            "message": message,
            "rating": rating
        }
    }, 201)
```

```
curl -X POST http://localhost:7146/api/feedback \
  -H "Content-Type: application/json" \
  -d '{"name": "Alice", "message": "Great product!", "rating": 5}'
```

```
{"received":{"name":"Alice","message":"Great product!","rating":5}}
```

request.headers

A dictionary of HTTP headers. Tina4 normalizes header names to lowercase:

```
@get("/api/headers")
async def show_headers(request, response):
    return response.json({
        "content_type": request.headers.get("content-type", "not set"),
        "user_agent": request.headers.get("user-agent", "not set"),
        "accept": request.headers.get("accept", "not set"),
        "custom": request.headers.get("x-custom-header", "not set")
    })
```

```
curl http://localhost:7146/api/headers -H "X-Custom-Header: hello-tina4"
```

```
{"content_type":"not set","user_agent":"curl/8.1.2","accept":"*/*","custom":"hello-tina4"}
```

request.ip

The client's IP address. Tina4 respects `X-Forwarded-For` and `X-Real-IP` headers behind a reverse proxy:

```
@get("/api/whoami")
async def whoami(request, response):
    return response.json({"ip": request.ip})
```

```
curl http://localhost:7146/api/whoami
```

```
{"ip":"127.0.0.1"}
```

request.cookies

A dictionary of cookies the client sent:

```
@get("/api/cookies")
async def show_cookies(request, response):
    return response.json({
        "cookies": request.cookies,
        "session_id": request.cookies.get("session_id", "none")
    })
```

```
curl http://localhost:7146/api/cookies -b "session_id=abc123; theme=dark"
```

```
{"cookies":{"session_id":"abc123","theme":"dark"},"session_id":"abc123"}
```

request.files

For multipart file uploads, `request.files` holds the uploaded files. Each file is a dictionary with four keys: `filename`, `type`, `content` (raw bytes), and `size` (in bytes):

```
@post("/api/upload")
async def upload(request, response):
    if "document" not in request.files:
        return response.json({"error": "No file uploaded"}, 400)

    file = request.files["document"]

    return response.json({
        "filename": file["filename"],
        "content_type": file["type"],
        "size": file["size"]
    })
```

```
curl -X POST http://localhost:7146/api/upload \
  -F "document=@report.pdf"
```

```
{"filename":"report.pdf","content_type":"application/pdf","size":45231}
```

To save the file to disk:

```
@post("/api/upload/save")
async def upload_and_save(request, response):
    if "photo" not in request.files:
        return response.json({"error": "No photo uploaded"}, 400)

    file = request.files["photo"]

    # Validate file type
    allowed = ["image/jpeg", "image/png", "image/webp"]
```

```
    if file["type"] not in allowed:
        return response.json({"error": f"File type {file['type']} not allowed"}, 400)

    # Save to public directory
    import os
    save_path = os.path.join("src", "public", "images", file["filename"])
    with open(save_path, "wb") as f:
        f.write(file["content"])

    return response.json({
        "message": "Photo uploaded",
        "url": f"/images/{file['filename']}"
    }, 201)
```

Handling Multiple File Uploads

When a form sends multiple files, each file field name maps to one entry in `request.files`. Use distinct field names for each file input:

```
@post("/api/upload-many")
async def upload_many(request, response):
    results = []

    for key, file in request.files.items():
        if not isinstance(file, dict) or "filename" not in file:
            continue

        ext = os.path.splitext(file["filename"])[1]
        unique_name = f"{uuid.uuid4().hex}{ext}"
        save_path = os.path.join("src", "public", "uploads", unique_name)

        os.makedirs(os.path.dirname(save_path), exist_ok=True)
        with open(save_path, "wb") as f:
            f.write(file["content"])

        results.append({
            "original_name": file["filename"],
            "saved_as": unique_name,
            "url": f"/uploads/{unique_name}"
        })

    if not results:
        return response.json({"error": "No files uploaded"}, 400)

    return response.json({"uploaded": results, "count": len(results)}, 201)
```

```
curl -X POST http://localhost:7146/api/upload-many \
  -F "photo=@sunset.jpg" \
  -F "document=@invoice.pdf" \
  -F "avatar=@profile.png"
```

```
{
  "uploaded": [
    {"original_name": "sunset.jpg", "saved_as": "a1b2c3d4.jpg", "url": "/uploads/a1b2c3d4.jpg"},
    {"original_name": "invoice.pdf", "saved_as": "e5f6a7b8.pdf", "url": "/uploads/e5f6a7b8.pdf"},
    {"original_name": "profile.png", "saved_as": "c9d0e1f2.png", "url": "/uploads/c9d0e1f2.png"}
  ],
  "count": 3
}
```

The HTML form uses distinct `name` attributes for each file input. Each field name becomes a key in `request.files`.

Upload Size Limits

Tina4 enforces a maximum upload size through the `TINA4_MAX_UPLOAD_SIZE` environment variable. The value is in bytes. The default is `10485760` (10 MB).

```
TINA4_MAX_UPLOAD_SIZE=10485760
```

When a client sends a body larger than this limit, Tina4 raises a `PayloadTooLarge` exception and returns a `413 Payload Too Large` response. Your handler never runs. The check fires before body parsing begins.

To allow 50 MB uploads, set this in your `.env` file:

```
TINA4_MAX_UPLOAD_SIZE=52428800
```

Calculate the value by multiplying: megabytes times 1,048,576. For 25 MB, that is `26214400`. For 100 MB, `104857600`. Choose the smallest limit that covers your use case. Large limits invite abuse.

---

3. The Response Object

The `response` object builds HTTP responses. Every route handler must return one. Methods chain, so you can set headers, cookies, and content in a single expression.

response.json()

Return JSON data with an optional status code:

```
@get("/api/users")
async def users(request, response):
    users = [
        {"id": 1, "name": "Alice"},
        {"id": 2, "name": "Bob"}
    ]
    return response.json({"users": users, "count": len(users)})

@post("/api/users")
async def create_user(request, response):
    # 201 Created
    return response.json({"id": 3, "name": request.body["name"]}, 201)

@get("/api/error")
async def error_example(request, response):
    # 400 Bad Request
    return response.json({"error": "Something went wrong"}, 400)
```

response.html()

Return raw HTML:

```
@get("/welcome")
async def welcome(request, response):
    return response.html("""
```

```
        <!DOCTYPE html>
        <html>
        <body>
            <h1>Welcome to My Store</h1>
            <p>Browse our <a href="/products">products</a>.</p>
        </body>
        </html>
    """)
```

response.text()

Return plain text:

```
@get("/robots.txt")
async def robots(request, response):
    return response.text("User-agent: *\nAllow: /")
```

response.render()

Render a Frond template with data (Chapter 4: Templates covers the engine in full):

```
@get("/dashboard")
async def dashboard(request, response):
    return response.render("dashboard.html", {
        "user_name": "Alice",
        "notifications": 3,
        "recent_orders": [
            {"id": 101, "total": 59.99},
            {"id": 102, "total": 124.50}
        ]
    })
```

response.redirect()

Send the client to another URL:

```
@get("/old-page")
async def old_page(request, response):
    return response.redirect("/new-page")

@post("/api/logout")
async def logout(request, response):
    # 303 See Other -- redirect after POST
    return response.redirect("/login", 303)

@get("/moved")
async def moved(request, response):
    # 301 Permanent redirect
    return response.redirect("/new-location", 301)
```

The default status code is `302 Found` (temporary redirect).

response.file()

Send a file as the response:

```
@get("/download/report")
async def download_report(request, response):
    return response.file("data/reports/monthly-report.pdf")
```

Tina4 auto-detects the content type from the file extension. The browser displays or downloads based on the MIME type.

To force a download instead of inline display, pass a `download_name`:

```
@get("/download/data")
async def download_data(request, response):
    return response.file("data/export.csv", download_name="sales-data.csv")
```

This sets the `Content-Disposition: attachment` header with your chosen filename.

response.xml()

Return XML content:

```
@get("/api/feed")
async def feed(request, response):
    return response.xml("<feed><entry><title>Hello</title></entry></feed>")
```

response.error()

Return a structured error envelope:

```
@post("/api/things")
async def create_thing(request, response):
    return response.error("VALIDATION_FAILED", "Name is required", 400)
```

This produces:

```
{"error":true,"code":"VALIDATION_FAILED","message":"Name is required","status":400}
```

Three arguments: an error code string, a human-readable message, and the HTTP status code. Clients check `error: true` and switch on the `code` field.

---

4. Status Codes

Every response method accepts a status code. Here are the codes you will use most:

Code	Meaning	When to Use
`200`	OK	Default. Successful GET, PUT, PATCH.
`201`	Created	Successful POST that created a resource.
`204`	No Content	Successful DELETE. No body needed.
`301`	Moved Permanently	URL has changed forever.
`302`	Found	Temporary redirect.
`400`	Bad Request	Invalid input from the client.

`401`	Unauthorized	Missing or invalid authentication.
`403`	Forbidden	Authenticated but not permitted.
`404`	Not Found	Resource does not exist.
`409`	Conflict	Duplicate or conflicting data. Two users claim the same username.
`413`	Payload Too Large	Body exceeds `TINA4_MAX_UPLOAD_SIZE`.
`422`	Unprocessable Entity	Valid JSON but fails business rules. The data parses but the logic rejects it.
`500`	Internal Server Error	Something broke on the server.

```
# 200 OK (default)
return response.json({"ok": True})

# 201 Created
return response.json({"id": 1}, 201)

# 204 No Content
return response.json(None, 204)

# 400 Bad Request
return response.json({"error": "Invalid input"}, 400)

# 401 Unauthorized
return response.json({"error": "Login required"}, 401)

# 403 Forbidden
return response.json({"error": "Not allowed"}, 403)

# 404 Not Found
return response.json({"error": "Not found"}, 404)

# 409 Conflict
return response.json({"error": "Username already taken"}, 409)

# 422 Unprocessable Entity
return response.json({"error": "Start date must precede end date"}, 422)

# 500 Internal Server Error
return response.json({"error": "Server error"}, 500)
```

You can also chain the status with `response.status()`:

```
return response.status(201).json({"id": 7, "created": True})
```

---

5. Content Negotiation

One endpoint can return different formats. The client declares what it wants through the `Accept` header. Your handler inspects that header and picks the right response method:

```
@get("/api/products/{id:int}")
async def product_detail(id, request, response):
    product = {
        "id": id,
        "name": "Wireless Keyboard",
        "price": 79.99
    }

    accept = request.headers.get("accept", "application/json")

    if "text/html" in accept:
        return response.render("product-detail.html", {"product": product})
    elif "text/plain" in accept:
        text = f"Product #{id}: {product['name']} - ${product['price']}"
        return response.text(text)
    elif "application/xml" in accept:
        xml = f'<product><id>{id}</id><name>{product["name"]}</name><price>{product["price"]}</p
        return response.xml(xml)
    else:
        return response.json(product)
```

```
# JSON (default)
curl http://localhost:7146/api/products/1
```

```
{"id":1,"name":"Wireless Keyboard","price":79.99}
```

```
# Plain text
curl http://localhost:7146/api/products/1 -H "Accept: text/plain"
```

```
Product #1: Wireless Keyboard - $79.99
```

```
# HTML (renders the template)
curl http://localhost:7146/api/products/1 -H "Accept: text/html"
```

```
<!DOCTYPE html>
<html>...rendered template...</html>
```

The pattern: check `request.headers.get("accept")`, match against known MIME types, fall back to JSON. Most API clients send `application/json` or `*/*`. Browsers send `text/html`. The same route serves both.

---

6. Custom Headers and Cookies

Setting Response Headers

Add custom headers with the `header` method. Chain calls before the final response:

```
@get("/api/data")
async def data_with_headers(request, response):
    return response.header("X-Custom-Header", "my-value") \
```

```
                    .header("X-Request-Id", "abc-123") \
                    .json({"data": [1, 2, 3]})
```

```
curl -i http://localhost:7146/api/data
```

```
HTTP/1.1 200 OK
Content-Type: application/json
X-Custom-Header: my-value
X-Request-Id: abc-123

{"data":[1,2,3]}
```

Setting Cookies

Set cookies on the response:

```
@post("/api/login")
async def login(request, response):
    # Set a session cookie
    return response.cookie("session_id", "abc123",
        path="/",
        max_age=3600,
        http_only=True,
        secure=True,
        same_site="Lax"
    ).json({"message": "Logged in"})

@post("/api/logout")
async def logout(request, response):
    # Delete a cookie by setting max_age to 0
    return response.cookie("session_id", "",
        path="/",
        max_age=0
    ).json({"message": "Logged out"})
```

Cookie keyword arguments:

Argument	Type	Default	Description
`path`	str	`"/"`	URL path scope
`max_age`	int	`3600`	Lifetime in seconds (0 deletes the cookie)
`http_only`	bool	`True`	JavaScript cannot access the cookie
`secure`	bool	`False`	Cookie travels over HTTPS only
`same_site`	str	`"Lax"`	`"Strict"`, `"Lax"`, or `"None"`

---

7. Input Validation

Tina4 ships a `Validator` class for declarative input validation. Chain rules together, then check the result.

The Validator Class

```
from tina4_python.validator import Validator

@post("/api/users")
async def create_user(request, response):
    v = Validator(request.body)
    v.required("name", "email").email("email").min_length("name", 2)

    if not v.is_valid():
        return response.error("VALIDATION_FAILED", v.errors()[0]["message"], 400)

    # proceed with valid data
```

The `Validator` accepts the request body (a dictionary) and provides chainable methods:

Method	Description
`required(*fields)`	Fields must be present and non-empty
`email(field)`	Field must be a valid email address
`min_length(field, n)`	Field must have at least `n` characters
`max_length(field, n)`	Field must have at most `n` characters
`integer(field)`	Field must be an integer
`min(field, n)`	Numeric field must be >= `n`
`max(field, n)`	Numeric field must be <= `n`
`in_list(field, values)`	Field must be one of the allowed values
`regex(field, pattern)`	Field must match the regular expression

Call `v.is_valid()` to check all rules. Call `v.errors()` to get the list of validation failures -- each entry holds a `field` and `message` key.

Note that `required()` accepts multiple field names in one call: `v.required("name", "email", "subject")`. This validates all three fields in a single statement.

---

8. Real-World Example: File Upload with Validation

A complete file upload endpoint. It validates type and size, generates a unique filename, and returns the URL:

```
from tina4_python.core.router import post
import os
import uuid
```

```
UPLOAD_DIR = "src/public/uploads"
MAX_FILE_SIZE = 5 * 1024 * 1024  # 5 MB
ALLOWED_TYPES = {
    "image/jpeg": ".jpg",
    "image/png": ".png",
    "image/webp": ".webp",
    "application/pdf": ".pdf"
}

@post("/api/files/upload")
async def upload_file(request, response):
    if "file" not in request.files:
        return response.json({"error": "No file provided. Send a file with field name 'file'"},

    file = request.files["file"]

    # Check file type
    if file["type"] not in ALLOWED_TYPES:
        return response.json({
            "error": f"File type '{file['type']}' not allowed",
            "allowed": list(ALLOWED_TYPES.keys())
        }, 400)

    # Check file size
    if file["size"] > MAX_FILE_SIZE:
        return response.json({
            "error": f"File too large. Maximum size is {MAX_FILE_SIZE // (1024 * 1024)} MB",
            "size": file["size"]
        }, 400)

    # Generate a unique filename
    ext = ALLOWED_TYPES[file["type"]]
    unique_name = f"{uuid.uuid4().hex}{ext}"

    # Save the file
    os.makedirs(UPLOAD_DIR, exist_ok=True)
    save_path = os.path.join(UPLOAD_DIR, unique_name)
    with open(save_path, "wb") as f:
        f.write(file["content"])

    return response.json({
        "message": "File uploaded successfully",
        "file": {
            "original_name": file["filename"],
            "saved_as": unique_name,
            "url": f"/uploads/{unique_name}",
            "size": file["size"],
            "content_type": file["type"]
        }
    }, 201)
```

```
curl -X POST http://localhost:7146/api/files/upload \
  -F "file=@photo.jpg"
```

```
{
  "message": "File uploaded successfully",
  "file": {
    "original_name": "photo.jpg",
    "saved_as": "a1b2c3d4e5f6.jpg",
    "url": "/uploads/a1b2c3d4e5f6.jpg",
```

```
    "size": 245760,
    "content_type": "image/jpeg"
  }
}
```

---

9. Exercise: Build a Contact Form API

Build an API that processes contact form submissions with full validation.

Requirements

Create `src/routes/contact.py` with these endpoints:

Method	Path	Description
`POST`	`/api/contact`	Submit a contact form. Validate all fields. Return 201 on success.
`GET`	`/api/contact/submissions`	List all submissions. Support `?status=` filter.

The contact form body must include: `name` (required, 2-100 chars), `email` (required, must contain @), `subject` (required), `message` (required, 10+ chars), `urgency` (optional, one of "low", "medium", "high", default "medium").

Validation rules:

- Return 400 with an `errors` list if any field fails
- Store submissions in a Python list with an auto-incremented ID and timestamp
- Each submission starts with a `status` of "new"

Test with:

```
# Valid submission
curl -X POST http://localhost:7146/api/contact \
  -H "Content-Type: application/json" \
  -d '{"name": "Alice", "email": "alice@example.com", "subject": "Question", "message": "I have

# Invalid submission (missing fields, short message)
curl -X POST http://localhost:7146/api/contact \
  -H "Content-Type: application/json" \
  -d '{"name": "A", "email": "bad-email", "message": "Short"}'

# List all
curl http://localhost:7146/api/contact/submissions

# Filter by status
curl "http://localhost:7146/api/contact/submissions?status=new"
```

---

10. Solution

```
from tina4_python.core.router import get, post
from datetime import datetime

submissions = []
next_id = 1


@post("/api/contact")
async def submit_contact(request, response):
    global next_id
    body = request.body
    errors = []

    # Validate name
    name = body.get("name", "")
    if not name:
        errors.append("Name is required")
    elif len(name) < 2 or len(name) > 100:
        errors.append("Name must be between 2 and 100 characters")

    # Validate email
    email = body.get("email", "")
    if not email:
        errors.append("Email is required")
    elif "@" not in email:
        errors.append("Email must contain @")

    # Validate subject
    subject = body.get("subject", "")
    if not subject:
        errors.append("Subject is required")

    # Validate message
    message = body.get("message", "")
    if not message:
        errors.append("Message is required")
    elif len(message) < 10:
        errors.append("Message must be at least 10 characters")

    # Validate urgency
    urgency = body.get("urgency", "medium")
    if urgency not in ("low", "medium", "high"):
        errors.append("Urgency must be one of: low, medium, high")

    if errors:
        return response.json({"errors": errors}, 400)

    submission = {
        "id": next_id,
        "name": name,
        "email": email,
        "subject": subject,
        "message": message,
        "urgency": urgency,
        "status": "new",
        "created_at": datetime.now().isoformat()
    }
```

```
    next_id += 1
    submissions.append(submission)

    return response.json({
        "message": "Contact form submitted successfully",
        "submission": submission
    }, 201)


@get("/api/contact/submissions")
async def list_submissions(request, response):
    status = request.params.get("status")

    if status:
        filtered = [s for s in submissions if s["status"] == status]
        return response.json({"submissions": filtered, "count": len(filtered)})

    return response.json({"submissions": submissions, "count": len(submissions)})
```

Valid submission output:

```
{
  "message": "Contact form submitted successfully",
  "submission": {
    "id": 1,
    "name": "Alice",
    "email": "alice@example.com",
    "subject": "Question",
    "message": "I have a question about your product pricing.",
    "urgency": "high",
    "status": "new",
    "created_at": "2026-03-22T14:30:00.000000"
  }
}
```

(Status: `201 Created`)

Invalid submission output:

```
{
  "errors": [
    "Name must be between 2 and 100 characters",
    "Email must contain @",
    "Subject is required",
    "Message must be at least 10 characters"
  ]
}
```

(Status: `400 Bad Request`)

---

11. Gotchas

1. request.body is None for GET requests

Problem: Accessing `request.body` in a GET handler returns `None`.

Cause: GET requests carry no body. Browsers and curl send none by convention.

Fix: Use `request.params` for GET parameters. The body populates only for POST, PUT, and PATCH requests.

2. Forgetting the Content-Type header

Problem: `request.body` is empty even though you sent JSON.

Cause: The `Content-Type: application/json` header is missing. Without it, Tina4 does not parse the body as JSON.

Fix: Include `-H "Content-Type: application/json"` in your curl command. In JavaScript `fetch()`, set `headers: {"Content-Type": "application/json"}`.

3. File upload with wrong field name

Problem: `request.files["photo"]` raises a `KeyError`.

Cause: The form field name does not match the key you look up.

Fix: Check that the form uses `<input type="file" name="photo">` and curl uses `-F "photo=@file.jpg"`. The string after `-F` must match the key in `request.files`.

4. response.redirect() in AJAX calls

Problem: Your JavaScript `fetch()` call receives a redirect response, but the browser does not navigate.

Cause: `fetch()` follows redirects silently. It returns the final response. It does not trigger browser navigation.

Fix: For AJAX calls, return JSON with a redirect URL. Handle navigation in JavaScript: `window.location.href = data.redirect_url`. Use `response.redirect()` only for traditional form submissions and browser navigation.

5. Cookie not being set

Problem: You called `response.cookie()` but the cookie does not appear in the browser.

Cause: Setting `secure=True` restricts the cookie to HTTPS. On `http://localhost`, the browser drops it.

Fix: During development on HTTP, use `secure=False` (the default). Enable `secure=True` only in production with HTTPS configured.

6. Large file uploads fail

Problem: Uploading a large file returns a 413 error or empty `request.files`.

Cause: The file exceeds `TINA4_MAX_UPLOAD_SIZE`. The default limit is 10 MB (10,485,760 bytes).

Fix: Set `TINA4_MAX_UPLOAD_SIZE` in your `.env` file. For 50 MB: `TINA4_MAX_UPLOAD_SIZE=52428800`. The check happens before your handler runs -- there is nothing to catch in your route code.

---

7. Chaining response methods in wrong order

Problem: Headers or cookies set after calling `response.json()` have no effect.

Cause: `response.json()` finalizes and returns the response object. Calling methods on a separate line after the return is dead code.

Fix: Chain headers and cookies before the final response method:

```
# Correct
return response.header("X-Custom", "value").cookie("token", "abc").json({"ok": True})

# Wrong -- header is set after json() already returned
result = response.json({"ok": True})
response.header("X-Custom", "value")  # Too late
return result
```

8. Header names are lowercase

Problem: `request.headers.get("Content-Type")` returns `None` even though the header exists.

Cause: Tina4 normalizes all header names to lowercase during parsing. The ASGI protocol delivers them this way.

Fix: Use lowercase keys: `request.headers.get("content-type")`. This applies to all headers -- `"authorization"`, `"user-agent"`, `"x-custom-header"`.

Templates

1. Beyond JSON -- Rendering HTML

Every route so far returns JSON. That works for APIs. Web applications need HTML -- product listings, dashboards, login forms, email templates. Tina4 uses the **Frond** template engine for this work.

Frond is a zero-dependency template engine built from scratch. Its syntax matches Twig, Jinja2, and Nunjucks. Three constructs drive the entire engine: `{{ }}` for output, `{% %}` for logic, `{# #}` for comments. That is the whole grammar.

This chapter builds toward a product catalog page. Items in a grid. Featured products highlighted. Prices formatted. Layout inherited from a shared template. One engine handles it all.

---

2. The @template Decorator

The shortest path to a rendered page:

```
from tina4_python.core.router import get, template

@get("/about")
@template("about.html")
async def about_page(request, response):
    return {
        "title": "About Us",
        "company": "My Store",
        "founded": 2020
    }
```

> ***Decorator order matters.*** `@template` *must sit* ***below*** *the route decorator (*`@get`*/*`@post`*/...) so the template wrapper is what gets registered with the router. If* `@template` *is above the route decorator, the router only sees the raw handler and the template wrapping is never applied, so the page renders a bare dict.*

Create `src/templates/about.html`:

```
<!DOCTYPE html>
<html>
<head><title>{{ title }}</title></head>
<body>
    <h1>{{ title }}</h1>
    <p>{{ company }} was founded in {{ founded }}.</p>
</body>
</html>
```

Visit `http://localhost:7146/about` and the rendered page appears.

The `@template` decorator stacks above `@get` (or `@post`, etc.). When the handler returns a dictionary, the decorator passes that dict to `response.render()` with the named template. If the handler returns something other than a dict -- an already-built Response, for instance -- the decorator passes it through unchanged. You can also call `response.render()` directly in any

route handler. The decorator is shorthand.

---

3. Variables and Output

Basic Output

Double curly braces print a variable:

```
<h1>Hello, {{ name }}!</h1>
<p>Your balance is {{ balance }}.</p>
```

With data `{"name": "Alice", "balance": 150.50}`, this renders:

```
<h1>Hello, Alice!</h1>
<p>Your balance is 150.5.</p>
```

Accessing Nested Properties

Dot notation reaches into dictionaries and object attributes:

```
<p>{{ user.name }}</p>
<p>{{ user.address.city }}</p>
<p>{{ order.items.0.name }}</p>
```

With data:

```
{
    "user": {
        "name": "Alice",
        "address": {"city": "Cape Town"}
    },
    "order": {
        "items": [{"name": "Keyboard"}, {"name": "Mouse"}]
    }
}
```

Renders:

```
<p>Alice</p>
<p>Cape Town</p>
<p>Keyboard</p>
```

Method Calls and Slicing

Frond supports calling methods on values inside templates. If a dictionary value is callable, you invoke it with arguments:

```
{{ user.t("greeting") }}       {# calls user.t("greeting") #}
{{ text[:10] }}                {# slice syntax -- first 10 characters #}
{{ items[2:5] }}               {# slice from index 2 to 5 #}
```

Operators inside quoted function arguments (such as `+`, `-`, `*`) parse without breaking the expression.

Auto-Escaping

Frond escapes HTML characters in output by default. XSS attacks die here:

```
<p>{{ user_input }}</p>
```

With `{"user_input": "<script>alert('hacked')</script>"}`, this renders:

```
<p>&lt;script&gt;alert(&#39;hacked&#39;)&lt;/script&gt;</p>
```

The script tag becomes plain text. It never executes. If you need raw HTML output and you trust the source, use the `|safe` filter:

```
<div>{{ trusted_html | safe }}</div>
```

---

4. Filters

Filters transform output. The pipe `|` applies them:

```
{{ name | upper }}          {# ALICE #}
{{ name | lower }}          {# alice #}
{{ name | capitalize }}     {# Alice #}
{{ name | title }}          {# Alice Smith #}
{{ bio | trim }}            {# Removes leading/trailing whitespace #}
```

Complete Filter Reference

String Filters

Filter	Example	Description	
`upper`	`{{ name |	upper }}`	Convert to uppercase
`lower`	`{{ name |	lower }}`	Convert to lowercase
`capitalize`	`{{ name |	capitalize }}`	Capitalize first letter
`title`	`{{ name |	title }}`	Capitalize each word
`trim`	`{{ name |	trim }}`	Strip leading/trailing whitespace
`ltrim`	`{{ name |	ltrim }}`	Strip leading whitespace
`rtrim`	`{{ name |	rtrim }}`	Strip trailing whitespace
`slug`	`{{ title |	slug }}`	Convert to URL-friendly slug
`wordwrap(80)`	`{{ text |	wordwrap(80) }}`	Wrap text at N characters
`truncate(100)`	`{{ text |	truncate(100) }}`	Truncate to N characters with ellipsis
`nl2br`	`{{ text |	nl2br }}`	Convert newlines to ` ` tags
`striptags`	`{{ html |	striptags }}`	Remove all HTML tags
`replace("a", "b")`	`{{ text |	replace("old", "new") }}`	Replace occurrences of a substring

Array Filters

Filter	Example	Description	
`length`	`{{ items \	length }}`	Count items in array or string length
`reverse`	`{{ items \	reverse }}`	Reverse order of items
`sort`	`{{ items \	sort }}`	Sort items ascending
`shuffle`	`{{ items \	shuffle }}`	Randomly shuffle items
`first`	`{{ items \	first }}`	Get the first item
`last`	`{{ items \	last }}`	Get the last item
`join(", ")`	`{{ items \	join(", ") }}`	Join array items with separator
`split(",")`	`{{ csv \	split(",") }}`	Split string into array
`unique`	`{{ items \	unique }}`	Remove duplicate values
`filter`	`{{ items \	filter }}`	Remove falsy values from array
`map("name")`	`{{ items \	map("name") }}`	Extract a property from each item
`column("name")`	`{{ items \	column("name") }}`	Extract a column from array of objects
`batch(3)`	`{{ items \	batch(3) }}`	Group items into batches of N
`slice(0, 3)`	`{{ items \	slice(0, 3) }}`	Extract a slice from offset with length

Encoding Filters

Filter	Example	Description	
`escape` (`e`)	`{{ text | escape }}`	HTML-escape special characters
`raw` (`safe`)	`{{ html | raw }}`	Output without auto-escaping
`url_encode`	`{{ text | url_encode }}`	URL-encode a string
`base64_encode` (`base64encode`)	`{{ text | base64_encode }}`	Base64-encode a string
`base64_decode` (`base64decode`)	`{{ data | base64_decode }}`	Base64-decode a string
`md5`	`{{ text | md5 }}`	Compute MD5 hash
`sha256`	`{{ text | sha256 }}`	Compute SHA-256 hash

Numeric Filters

Filter	Example	Description	
`abs`	`{{ num | abs }}`	Absolute value
`round(2)`	`{{ price | round(2) }}`	Round to N decimal places
`number_format(2)`	`{{ price | number_format(2) }}`	Format with decimals and thousands separator
`int`	`{{ val | int }}`	Cast to integer
`float`	`{{ val | float }}`	Cast to float
`string`	`{{ val | string }}`	Cast to string

JSON Filters

Filter	Example	Description	
`json_encode`	`{{ data | json_encode }}`	Encode value as JSON string
`to_json` (`tojson`)	`{{ data | to_json }}`	Encode value as JSON string (alias)
`json_decode`	`{{ str | json_decode }}`	Decode JSON string to object
`js_escape`	`{{ text | js_escape }}`	Escape string for safe use in JavaScript

Dict Filters

Filter	Example	Description	
`keys`	`{{ obj | keys }}`		Get dictionary keys as array
`values`	`{{ obj | values }}`		Get dictionary values as array
`merge(other)`	`{{ defaults | merge(overrides) }}`		Merge two dictionaries

Other Filters

Filter	Example	Description	
`default("fallback")`	`{{ name | default("Guest") }}`		Fallback when value is empty or undefined
`date("Y-m-d")`	`{{ created | date("Y-m-d") }}`		Format a date value
`format(val)`	`{{ "%.2f" | format(price) }}`		Format string with value (sprintf-style)
`data_uri`	`{{ content | data_uri }}`		Convert to a data URI string
`dump`	`{{ var | dump }}` `or` `{{ dump(var) }}`		Debug output, gated on `TINA4_DEBUG=true` (see *Dumping Values*)
`form_token`	`{{ form_token() }}`		Generate a CSRF hidden input with token
`formTokenValue`	`{{ formTokenValue("context") }}`		Return the raw JWT token string
`to_json`	`{{ data | to_json }}`		JSON-encode a value (no double-escaping)
`js_escape`	`{{ text | js_escape }}`		Escape for safe use in JavaScript strings

Chaining Filters

Filters chain left to right:

```
{{ name | trim | lower | capitalize }}
{# "  alice smith  " → "alice smith" → "Alice smith" #}

{{ items | sort | reverse | first }}
{# Sort, reverse, take first = largest item #}
```

The default Filter

A fallback when a variable is empty or undefined:

```
{{ username | default("Guest") }}
```

```
{{ bio | default("No bio provided.") }}
{{ theme | default("light") }}
```

Dumping Values for Debugging

The `dump` helper lets you inspect any variable mid-template. Two interchangeable forms are supported:

```
{{ user | dump }}
{{ dump(user) }}
```

Both produce the same `<pre>`-wrapped, HTML-escaped `repr()` of the value. Handles nested dicts, lists, class instances, and cyclic references without crashing; Python's `repr()` prints `{...}` for back-edges.

```
{{ dump(order) }}

{# Output: #}
{# <pre>{'id': 42, 'items': [...], 'total': 99.99}</pre> #}
```

dump is gated on `TINA4_DEBUG=true`. In production (env var unset or `false`) **both** the filter and function form silently return an empty string. This prevents accidental leaks of internal state, object shapes, or sensitive values into rendered HTML if a developer leaves a `{{ dump(x) }}` call in a template.

```
# .env - dev
TINA4_DEBUG=true    # dump() outputs the value

# .env - production
TINA4_DEBUG=false   # dump() is a no-op
```

You can rely on this gate for safety, but treat `dump` as a development-only convenience. For structured output in production code paths, use `to_json`.

---

5. Control Tags

if / elif / else

```
{% if user.role == "admin" %}
    <span class="badge">Admin</span>
{% elif user.role == "moderator" %}
    <span class="badge">Moderator</span>
{% else %}
    <span class="badge">Member</span>
{% endif %}
```

Comparisons and logical operators:

```
{% if price > 100 and in_stock %}
    <p>Premium item, available now!</p>
{% endif %}

{% if not user.verified %}
    <p>Please verify your email.</p>
{% endif %}
```

```
{% if not items %}
    <p>Your cart is empty.</p>
{% endif %}
```

for Loops

```
{% for product in products %}
    <div class="product-card">
        <h3>{{ product.name }}</h3>
        <p>${{ "%.2f"|format(product.price) }}</p>
    </div>
{% endfor %}
```

Inside a for loop, the `loop` variable provides iteration context:

Variable	Description
`loop.index`	Current iteration (1-based)
`loop.index0`	Current iteration (0-based)
`loop.first`	True on the first iteration
`loop.last`	True on the last iteration
`loop.length`	Total number of items

```
<ol>
{% for item in items %}
    <li class="{{ 'first' if loop.first else '' }} {{ 'last' if loop.last else '' }}">
        {{ loop.index }}. {{ item.name }}
    </li>
{% endfor %}
</ol>
```

for / else

The `{% else %}` block inside `{% for %}` runs when the list is empty:

```
{% for product in products %}
    <div class="product-card">{{ product.name }}</div>
{% else %}
    <p>No products found.</p>
{% endfor %}
```

set -- Local Variables

Create or update a variable inside a template:

```
{% set greeting = "Hello" %}
{% set full_name = user.first_name ~ " " ~ user.last_name %}
{% set total = price * quantity %}
{% set discount = total - rebate %}

<p>{{ greeting }}, {{ full_name }}!</p>
<p>Total: {{ total }}, After discount: {{ discount }}</p>
```

The `~` operator concatenates strings. Arithmetic operators (`+`, `-`, `*`, `/`, `//`, `%`, `**`) work in `set` and expressions.

When combining filters with arithmetic, assign the filtered values first:

```
{% set dr = account.dr|default(0) %}
{% set cr = account.cr|default(0) %}
{% set balance = dr - cr %}
<p>Balance: {{ balance }}</p>
```

---

6. Template Inheritance

Template inheritance kills duplication. A base template defines blocks. Child templates override them. One layout file controls every page.

Base Template

Create `src/templates/base.html`:

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>{% block title %}My Store{% endblock %}</title>
    <link rel="stylesheet" href="/css/tina4.css">
    {% block head %}{% endblock %}
</head>
<body>
    <nav>
        <a href="/">Home</a>
        <a href="/products">Products</a>
        <a href="/about">About</a>
    </nav>

    <main>
        {% block content %}{% endblock %}
    </main>

    <footer>
        <p>&copy; 2026 My Store</p>
    </footer>

    <script src="/js/frond.js"></script>
    {% block scripts %}{% endblock %}
</body>
</html>
```

Child Template

Create `src/templates/home.html`:

```
{% extends "base.html" %}

{% block title %}Home - My Store{% endblock %}

{% block content %}
    <h1>Welcome to My Store</h1>
    <p>Browse our collection of quality products.</p>
{% endblock %}
```

When Frond renders `home.html`:

- It sees `{% extends "base.html" %}` and loads the base template.
- The `{% block title %}` in `home.html` replaces the one in `base.html`.
- The `{% block content %}` in `home.html` replaces the one in `base.html`.
- Blocks not overridden (`head`, `scripts`) keep their default content -- empty here.

Calling Parent Blocks

Use `{{ parent() }}` to include the parent block's content:

```
{% extends "base.html" %}

{% block head %}
    {{ parent() }}
    <link rel="stylesheet" href="/css/products.css">
{% endblock %}
```

The `head` block now contains everything from the base plus the extra stylesheet.

---

7. Include and Macro

include

Pull in another template file:

```
{% include "partials/header.html" %}

<main>
    <h1>Products</h1>
</main>

{% include "partials/footer.html" %}
```

Pass variables to included templates:

```
{% include "partials/product-card.html" with {"product": featured_product} %}
```

macro -- Reusable Template Functions

Macros are functions for templates. Define once, call everywhere:

Create `src/templates/macros/forms.html`:

```
{% macro input(name, label, type="text", value="", required=false) %}
    <div class="form-group">
        <label for="{{ name }}">{{ label }}{% if required %} *{% endif %}</label>
        <input type="{{ type }}" name="{{ name }}" id="{{ name }}"
               value="{{ value }}" {{ "required" if required else "" }}>
    </div>
{% endmacro %}

{% macro textarea(name, label, value="", rows=4) %}
    <div class="form-group">
        <label for="{{ name }}">{{ label }}</label>
```

```
        <textarea name="{{ name }}" id="{{ name }}" rows="{{ rows }}">{{ value }}</textarea>
    </div>
{% endmacro %}

{% macro button(text, type="submit", class="btn-primary") %}
    <button type="{{ type }}" class="btn {{ class }}">{{ text }}</button>
{% endmacro %}
```

Use them:

```
{% import "macros/forms.html" as forms %}

<form method="POST" action="/api/contact">
    {{ forms.input("name", "Your Name", required=true) }}
    {{ forms.input("email", "Email Address", type="email", required=true) }}
    {{ forms.textarea("message", "Your Message", rows=6) }}
    {{ forms.button("Send Message") }}
</form>
```

Change the macro once and every form in your application updates. Consistent markup across the entire project.

---

8. Comments

Template comments use `{# #}`. Frond strips them from the output:

```
{# This comment will not appear in the HTML source #}
<p>Visible content</p>

{#
    Multi-line comments work too.
    Use them to document template logic.
#}
```

HTML comments (`<!-- -->`) reach the browser. Frond comments never do.

---

9. Special Tags

{% raw %} -- Literal Output

Output literal `{{ }}` or `{% %}` without processing. This tag saves you when embedding Vue.js or Angular templates:

```
{% raw %}
    <div id="app">
        {{ message }}
    </div>
{% endraw %}
```

Frond outputs the literal text `{{ message }}`. No variable lookup. No expression parsing.

{% spaceless %} -- Remove Whitespace

Strip whitespace between HTML tags:

```
{% spaceless %}
    <div>
        <span>Hello</span>
    </div>
{% endspaceless %}
```

Output:

```
<div><span>Hello</span></div>
```

Inline elements create visible gaps when whitespace sits between them. The `spaceless` tag eliminates those gaps.

{% autoescape %} -- Control Escaping

Override auto-escaping for a block of content:

```
{% autoescape false %}
    {{ trusted_html }}
{% endautoescape %}
```

Everything inside outputs without HTML escaping. This works the same as `| raw` on every variable, but handles large blocks of trusted content with less repetition. Never use this with user-submitted data.

Whitespace Control

Template tags occupy a full line and produce blank lines in the output. Use `{%-` and `-%}` to strip surrounding whitespace:

```
{%- for item in items -%}
    <li>{{ item.name }}</li>
{%- endfor -%}
```

The `-` on the left strips whitespace before the tag. The `-` on the right strips whitespace after. The output contains no blank lines between list items.

---

10. tina4css

The `tina4.css` file is Tina4's built-in CSS utility framework. It ships with every project. Layout utilities. Typography. Spacing. Common UI patterns. No Bootstrap. No Tailwind. No separate download.

Include it in your base template:

```
<link rel="stylesheet" href="/css/tina4.css">
```

Layout Classes

The grid system uses a 12-column layout:

```
<div class="container">
    <div class="row">
        <div class="col-6">Left half</div>
        <div class="col-6">Right half</div>
    </div>
```

```
</div>
```

Flex layout for alignment:

```
<div class="flex justify-between items-center">
    <h1>Title</h1>
    <button class="btn btn-primary">Action</button>
</div>
```

CSS grid for card layouts:

```
<div class="grid grid-cols-3 gap-4">
    <div class="card">Item 1</div>
    <div class="card">Item 2</div>
    <div class="card">Item 3</div>
</div>
```

Buttons

Five button styles cover most use cases:

```
<button class="btn btn-primary">Primary</button>
<button class="btn btn-secondary">Secondary</button>
<button class="btn btn-success">Success</button>
<button class="btn btn-warning">Warning</button>
<button class="btn btn-danger">Danger</button>
```

Link-style buttons use the same classes on anchor tags:

```
<a href="/dashboard" class="btn btn-primary">Go to Dashboard</a>
<a href="/cancel" class="btn btn-secondary">Cancel</a>
```

Cards

Cards group related content with optional header and footer sections:

```
<div class="card">
    <div class="card-header">Order Summary</div>
    <div class="card-body">
        <p>3 items in your cart</p>
        <p class="text-primary">Total: $149.97</p>
    </div>
    <div class="card-footer">
        <button class="btn btn-primary">Checkout</button>
    </div>
</div>
```

Cards work well inside a grid for catalog-style layouts:

```
<div class="grid grid-cols-3 gap-4">
    {% for product in products %}
    <div class="card">
        <div class="card-header">{{ product.name }}</div>
        <div class="card-body">
            <p>${{ "%.2f"|format(product.price) }}</p>
        </div>
    </div>
    {% endfor %}
</div>
```

Alerts

Alert boxes communicate status messages to the user:

```
<div class="alert alert-success">Order placed. Check your email for confirmation.</div>
<div class="alert alert-danger">Payment failed. Your card was declined.</div>
<div class="alert alert-warning">Your session expires in 5 minutes.</div>
<div class="alert alert-info">New features are available. See the changelog.</div>
```

Forms

Form controls use `form-group` for spacing and `form-control` for input styling:

```
<form method="POST" action="/api/contact">
    <div class="form-group">
        <label for="name">Full Name</label>
        <input type="text" id="name" name="name" class="form-control" required>
    </div>

    <div class="form-group">
        <label for="email">Email Address</label>
        <input type="email" id="email" name="email" class="form-control" required>
    </div>

    <div class="form-group">
        <label for="message">Message</label>
        <textarea id="message" name="message" class="form-control" rows="4"></textarea>
    </div>

    <div class="form-group">
        <label for="priority">Priority</label>
        <select id="priority" name="priority" class="form-control">
            <option value="low">Low</option>
            <option value="medium" selected>Medium</option>
            <option value="high">High</option>
        </select>
    </div>

    <button type="submit" class="btn btn-primary">Send Message</button>
    <button type="reset" class="btn btn-secondary">Clear</button>
</form>
```

Tables

Tables gain borders and row striping with tina4css classes:

```
<table class="table">
    <thead>
        <tr>
            <th>#</th>
            <th>Product</th>
            <th>Price</th>
        </tr>
    </thead>
    <tbody>
        {% for product in products %}
        <tr>
            <td>{{ loop.index }}</td>
            <td>{{ product.name }}</td>
            <td>${{ "%.2f"|format(product.price) }}</td>
```

```
        </tr>
        {% endfor %}
    </tbody>
</table>
```

Spacing and Typography Utilities

```
{# Spacing #}
<div class="p-4 m-2">Padded and margined</div>
<div class="mt-4">Margin top</div>
<div class="mb-2">Margin bottom</div>
<div class="px-3">Horizontal padding</div>

{# Typography #}
<p class="text-lg text-gray-600">Large gray text</p>
<p class="text-center">Centered text</p>
<p class="text-right">Right-aligned text</p>
<span class="text-muted">Gray text</span>
<span class="text-primary">Primary color text</span>
```

No external dependencies. If you prefer Bootstrap or Tailwind, swap the `<link>` tag. Tina4 does not care which CSS framework you choose.

---

11. Exercise: Build a Product Catalog Page

Build a product catalog page with categories, filtering, and a detail view.

Requirements

- Create a route at `GET /catalog` that renders a product catalog page
- Create a route at `GET /catalog/{id:int}` that renders a single product detail page
- Use template inheritance with a base layout
- Display products in a grid with:
- Product name, price (formatted to 2 decimal places), category
- "In Stock" / "Out of Stock" badge
- Featured products get a highlighted border
- Show category filter links at the top (all categories from the data)
- Support `?category=` query parameter to filter products
- The detail page shows full product info including description
- Use at least one macro (e.g., for the product card)
- Use the `|default` filter somewhere meaningful

Use this data:

```
products = [
    {"id": 1, "name": "Espresso Machine", "category": "Kitchen", "price": 299.99, "in_stock": Tr
    {"id": 2, "name": "Yoga Mat", "category": "Fitness", "price": 29.99, "in_stock": True, "feat
    {"id": 3, "name": "Standing Desk", "category": "Office", "price": 549.99, "in_stock": True,
    {"id": 4, "name": "Noise-Canceling Headphones", "category": "Electronics", "price": 199.99,
```

```
    {"id": 5, "name": "Water Bottle", "category": "Fitness", "price": 24.99, "in_stock": True, "
    {"id": 6, "name": "Desk Lamp", "category": "Office", "price": 79.99, "in_stock": True, "feat
]
```

---

12. Solution

Create `src/templates/macros/catalog.html`:

```
{% macro product_card(product) %}
    <div class="product-card {{ 'featured' if product.featured else '' }}">
        {% if product.featured %}
            <span class="featured-badge">Featured</span>
        {% endif %}
        <h3><a href="/catalog/{{ product.id }}">{{ product.name }}</a></h3>
        <p class="category">{{ product.category }}</p>
        <p class="price">${{ "%.2f"|format(product.price) }}</p>
        {% if product.in_stock %}
            <span class="badge badge-success">In Stock</span>
        {% else %}
            <span class="badge badge-danger">Out of Stock</span>
        {% endif %}
    </div>
{% endmacro %}
```

Create `src/templates/catalog.html`:

```
{% extends "base.html" %}
{% import "macros/catalog.html" as catalog %}

{% block title %}{{ page_title | default("Product Catalog") }}{% endblock %}

{% block content %}
    <h1>{{ page_title | default("Product Catalog") }}</h1>

    <div class="category-filters">
        <a href="/catalog" class="{{ 'active' if active_category is not defined else '' }}">All<
        {% for cat in categories %}
            <a href="/catalog?category={{ cat }}"
               class="{{ 'active' if active_category == cat else '' }}">{{ cat }}</a>
        {% endfor %}
    </div>

    <p class="stats">Showing {{ products | length }} product{{ "s" if products|length != 1 else

    <div class="product-grid">
        {% for product in products %}
            {{ catalog.product_card(product) }}
        {% else %}
            <p>No products found in this category.</p>
        {% endfor %}
    </div>
{% endblock %}
```

Create `src/templates/product_detail.html`:

```
{% extends "base.html" %}

{% block title %}{{ product.name }} - My Store{% endblock %}
```

```
{% block content %}
    <a href="/catalog">&larr; Back to catalog</a>

    <div class="product-detail">
        <h1>{{ product.name }}</h1>
        <p class="category">{{ product.category }}</p>
        <p class="price">${{ "%.2f"|format(product.price) }}</p>
        {% if product.in_stock %}
            <span class="badge badge-success">In Stock</span>
        {% else %}
            <span class="badge badge-danger">Out of Stock</span>
        {% endif %}
        {% if product.featured %}
            <span class="featured-badge">Featured</span>
        {% endif %}
        <div class="description">
            <h2>Description</h2>
            <p>{{ product.description | default("No description available.") }}</p>
        </div>
    </div>
{% endblock %}
```

Create `src/routes/catalog.py`:

```
from tina4_python.core.router import get

products = [
    {"id": 1, "name": "Espresso Machine", "category": "Kitchen", "price": 299.99, "in_stock": Tr
    {"id": 2, "name": "Yoga Mat", "category": "Fitness", "price": 29.99, "in_stock": True, "feat
    {"id": 3, "name": "Standing Desk", "category": "Office", "price": 549.99, "in_stock": True,
    {"id": 4, "name": "Noise-Canceling Headphones", "category": "Electronics", "price": 199.99,
    {"id": 5, "name": "Water Bottle", "category": "Fitness", "price": 24.99, "in_stock": True, "
    {"id": 6, "name": "Desk Lamp", "category": "Office", "price": 79.99, "in_stock": True, "feat
]


@get("/catalog")
async def catalog_page(request, response):
    category = request.params.get("category")
    categories = sorted(set(p["category"] for p in products))

    if category:
        filtered = [p for p in products if p["category"].lower() == category.lower()]
    else:
        filtered = products

    return response.render("catalog.html", {
        "products": filtered,
        "categories": categories,
        "active_category": category,
        "page_title": f"{category} Products" if category else "Product Catalog"
    })


@get("/catalog/{id:int}")
async def product_detail(id, request, response):
    product_id = id

    for product in products:
```

```
        if product["id"] == product_id:
            return response.render("product_detail.html", {"product": product})

    return response.render("errors/404.html", {}, 404)
```

Open `http://localhost:7146/catalog` **in your browser.** You should see:

- A heading "Product Catalog"
- Category filter links: All, Electronics, Fitness, Kitchen, Office
- A count of products shown
- Product cards in a grid with names, prices, category labels, and stock badges
- Featured products wear a highlighted style and a "Featured" badge
- Clicking a product name navigates to the detail page
- Clicking a category link filters the list

---

13. Gotchas

1. Whitespace in output

Problem: Rendered HTML contains unexpected blank lines or spaces.

Cause: Template tags produce whitespace on the lines they occupy.

Fix: Use whitespace control with `{%-` and `-%}` to strip whitespace around tags:

```
{%- for item in items -%}
    <li>{{ item.name }}</li>
{%- endfor -%}
```

2. Variable not defined error

Problem: Frond raises an error when a variable does not exist in the context.

Cause: You used `{{ user.name }}` but did not pass `user` in the template data.

Fix: Use the `|default` filter: `{{ user.name | default("Guest") }}`. Or check first: `{% if user is defined %}{{ user.name }}{% endif %}`.

3. Extends must be the first tag

Problem: `{% extends "base.html" %}` has no effect. The page renders without the layout.

Cause: `{% extends %}` must be the first tag in the template. Any text, HTML, or tags before it cause Frond to treat the template as standalone.

Fix: Move `{% extends "base.html" %}` to the first line. Nothing before it.

4. Macro not found

Problem: `{{ forms.input(...) }}` produces an error about `forms` being undefined.

Cause: The `{% import %}` statement is missing, or the import path is wrong.

Fix: Add `{% import "macros/forms.html" as forms %}` at the top of the template (after `{% extends %}` if using inheritance). The path is relative to `src/templates/`.

5. Filter produces wrong type

Problem: `{{ "%.2f"|format(price) }}` shows an error instead of a formatted number.

Cause: The variable `price` is a string, not a number. Filters expect specific types.

Fix: Pass correct types from your route handler. Use `float(price)` in Python before passing to the template, or convert in the template: `{{ "%.2f"|format(price|float) }}`.

6. Escaped HTML when you want raw output

Problem: HTML content shows as text with visible `<tags>` instead of rendering.

Cause: Frond auto-escapes all `{{ }}` output to prevent XSS.

Fix: Use the `|safe` filter: `{{ trusted_html | safe }}`. Only use this with content you trust -- never with user input.

7. Include file path wrong

Problem: `{% include "header.html" %}` produces a "template not found" error even though the file exists.

Cause: The path in `{% include %}` is relative to `src/templates/`, not the current template file.

Fix: Use the full path from the templates root: `{% include "partials/header.html" %}` for a file at `src/templates/partials/header.html`.

Database

1. From Lists to Real Data

Every example so far stored data in Python lists. Restart the server. Data gone. A real application demands persistence.

Tina4 Python makes database access straightforward. Set a `TINA4_DATABASE_URL` in your `.env`. Create a `Database()` connection. Run SQL. No ORM required (Chapter 6 adds that). The SQL you already know carries over unchanged.

Picture a notes application. Users create, edit, and delete notes. Those notes must survive restarts, support searching, and handle concurrent users. A database delivers all three.

---

2. Configuration

TINA4*DATABASE*URL

Set your database connection in `.env`:

```
TINA4_DATABASE_URL=sqlite:///data/app.db
```

That is the default -- a SQLite database stored in `data/`. SQLite ships with Python. No install required.

Tina4 supports six database engines:

Engine	TINA4*DATABASE*URL Format	Package Required
SQLite	`sqlite:///data/app.db`	None (stdlib)
PostgreSQL	`postgresql://user:pass@host:5432/dbname`	`psycopg2`
MySQL	`mysql://user:pass@host:3306/dbname`	`mysql-connector-python`
MSSQL	`mssql://user:pass@host:1433/dbname`	`pymssql`
Firebird	`firebird://user:pass@host:3050/path/to/db.fdb`	`firebird-driver`
ODBC	`odbc://DSN_NAME`	`pyodbc`

Switch databases by changing the URL. Your Python code stays the same. All drivers implement the same adapter interface.

Installing Database Drivers

SQLite requires nothing. For other databases, install the driver with uv:

```
# PostgreSQL
uv add psycopg2-binary

# MySQL
uv add mysql-connector-python

# MSSQL
uv add pymssql

# Firebird
uv add firebird-driver

# ODBC
uv add pyodbc
```

---

3. Creating a Connection

```
from tina4_python.database.connection import Database

db = Database()
```

One line. `Database()` reads `TINA4_DATABASE_URL` from your `.env` and connects. If the SQLite file does not exist, the adapter creates one.

You can also pass a URL directly:

```
db = Database("sqlite:///data/test.db")
```

Connection Pooling

Applications that handle many concurrent requests benefit from connection pooling. Pass a `pool` argument:

```
db = Database("postgres://localhost/mydb", pool=5)  # 5 connections, round-robin
```

The `pool` parameter controls how many database connections the pool maintains:

- `pool=0` (the default) -- a single connection serves all queries
- `pool=N` (where N > 0) -- N connections rotate round-robin across queries

Pooled connections are thread-safe. Each query dispatches to the next available connection. This eliminates contention when multiple route handlers query the database at the same time.

Extra Driver Options via **kwargs

Any additional keyword arguments pass through to the underlying driver's `connect()` call. This suits engine-specific options:

```
db = Database("firebird://localhost:3050//data/legacy.fdb", charset="ISO8859_1")
```

Firebird URL Forms

Firebird is the awkward one in the stack: every other engine has a server-side database name (`postgres://host:port/dbname`), but Firebird wants either an absolute file path on the server, a Windows drive-letter path, or an alias. The classic URI form needs a double slash to keep the leading `/` of an absolute path through `urlparse`, which is unintuitive.

Tina4 normalises five equivalent forms. Pick whichever reads best:

```
# Classic double-slash absolute path -- the URL spec way
db = Database("firebird://SYSDBA:masterkey@localhost:3050//firebird/data/app.fdb")

# Single-slash absolute path -- what most people instinctively type
db = Database("firebird://SYSDBA:masterkey@localhost:3050/firebird/data/app.fdb")

# Windows drive-letter path
db = Database("firebird://SYSDBA:masterkey@host:3050/C:/Data/app.fdb")

# Windows with URL-encoded colon
db = Database("firebird://SYSDBA:masterkey@host:3050/C%3A/Data/app.fdb")

# Firebird alias (single token, no slashes)
db = Database("firebird://SYSDBA:masterkey@localhost:3050/employee")
```

For ops setups that keep the server URL and the database location in separate config layers -- or for Windows backslash paths that fight URL encoding -- set `TINA4_DATABASE_FIREBIRD_PATH`:

```
TINA4_DATABASE_FIREBIRD_PATH=C:\firebird\data\app.fdb
TINA4_DATABASE_URL=firebird://SYSDBA:masterkey@localhost:3050/ignored
```

The env override wins over whatever path is in the URL.

Firebird Dual-Driver Support

The Firebird adapter tries `firebird-driver` (the modern package) first and falls back to the legacy `fdb` package if `firebird-driver` is missing. Install whichever is available -- the adapter handles the difference internally.

Lowercase Column Names

Firebird returns column names in uppercase by default. Tina4 normalises them to lowercase, so `result["first_name"]` works regardless of how the column was defined in the schema.

Firebird Migration Support

As of 3.10.8, the migration runner handles Firebird correctly. It uses a generator (sequence) for auto-increment IDs instead of `AUTOINCREMENT`, and emits `VARCHAR(4096)` instead of `TEXT` (which Firebird does not support as a column type). No changes to existing migrations are needed -- the runner detects the engine on its own.

---

4. Running Queries

fetch -- Get Multiple Rows

```
from tina4_python.core.router import get
from tina4_python.database.connection import Database

@get("/api/notes")
async def list_notes(request, response):
    db = Database()
    notes = db.fetch("SELECT id, title, content, created_at FROM notes ORDER BY created_at DESC"

    return response.json({"notes": notes, "count": len(notes)})
```

`fetch()` returns a `DatabaseResult` containing a list of dictionaries. Each dictionary represents one row:

```
{
  "notes": [
    {"id": 1, "title": "Shopping List", "content": "Milk, eggs, bread", "created_at": "2026-03-2
    {"id": 2, "title": "Meeting Notes", "content": "Discuss Q2 roadmap", "created_at": "2026-03-
  ],
  "count": 2
}
```

DatabaseResult

`fetch()` returns a `DatabaseResult` object. It behaves like a list but carries extra metadata about the query.

Properties

```
result = db.fetch("SELECT * FROM users WHERE active = ?", [1])

result.records      # [{"id": 1, "name": "Alice"}, {"id": 2, "name": "Bob"}]
result.columns      # ["id", "name", "email", "active"]
result.count        # total number of matching rows
result.limit        # query limit (if set)
result.offset       # query offset (if set)
```

Iteration

A `DatabaseResult` is iterable. Use it directly in a `for` loop:

```
for user in result:
    print(user["name"])
```

Index Access

Access rows by index:

```
first_user = result[0]
```

Countable

`len()` works on the result:

```
print(len(result))  # number of records in this result set
```

Conversion Methods

```
result.to_list()              # plain list of dictionaries (same as result.records)
result.to_paginate()          # {"data": [...], "total": 42, "page": 1, "per_page": 20, ...}
result.to_paginate(page=2, per_page=10)  # custom page and page size
```

`to_list()` returns the records as a plain list. `to_paginate(page=1, per_page=20)` bundles the records with total count, page, per*page, total*pages, has*next, and has*prev in a single dictionary. It is built for paginated API responses.

Schema Metadata with column_info()

`column_info()` returns detailed metadata about the columns in the result set. The data loads lazily -- it queries the database schema only when you call the method for the first time:

```
info = result.column_info()
# [
#     {"name": "id", "type": "INTEGER", "size": None, "decimals": None, "nullable": False, "prim
#     {"name": "name", "type": "TEXT", "size": None, "decimals": None, "nullable": False, "prima
#     {"name": "email", "type": "TEXT", "size": 255, "decimals": None, "nullable": True, "primar
#     ...
# ]
```

Each column entry contains:

Field	Description
`name`	Column name
`type`	Database type (e.g. `INTEGER`, `TEXT`, `REAL`)
`size`	Maximum size (or `None` if not applicable)
`decimals`	Decimal places (or `None`)
`nullable`	Whether the column allows `NULL`
`primary_key`	Whether the column is part of the primary key

This powers dynamic forms, documentation generation, and data validation before insert.

fetch_one -- Get a Single Row

```
@get("/api/notes/{id:int}")
async def get_note(id, request, response):
    db = Database()
    note = db.fetch_one(
        "SELECT id, title, content, created_at FROM notes WHERE id = ?",
        [id]
    )

    if note is None:
        return response.json({"error": "Note not found"}, 404)

    return response.json(note)
```

`fetch_one()` returns a single dictionary or `None` if no row matches.

execute -- Insert, Update, Delete

```
from tina4_python.core.router import post, put, delete
from tina4_python.database.connection import Database

@post("/api/notes")
async def create_note(request, response):
    db = Database()
    body = request.body

    if not body.get("title"):
        return response.json({"error": "Title is required"}, 400)

    result = db.execute(
        "INSERT INTO notes (title, content) VALUES (?, ?)",
        [body["title"], body.get("content", "")]
    )

    note = db.fetch_one("SELECT * FROM notes WHERE id = ?", [result.last_id])

    return response.json({"message": "Note created", "note": note}, 201)
```

`execute()` runs an INSERT, UPDATE, or DELETE statement and returns a `DatabaseResult` with `affected_rows` and `last_id` properties.

---

5. Parameterised Queries

Never concatenate user input into SQL strings. Parameterised queries stand between you and SQL injection:

```
# WRONG -- SQL injection vulnerability
db.fetch(f"SELECT * FROM notes WHERE title = '{user_input}'")

# CORRECT -- parameterised query
db.fetch("SELECT * FROM notes WHERE title = ?", [user_input])
```

Positional parameters use the `?` placeholder. Pass a list of values as the second argument:

```
db.fetch(
    "SELECT * FROM notes WHERE category = ? AND created_at > ?",
    ["work", "2026-03-01"]
)
```

Tina4 handles parameter escaping and type conversion for all database engines.

---

6. Transactions

Multiple operations must succeed or fail together. Transactions enforce that contract:

```
@post("/api/transfer")
async def transfer_funds(request, response):
    db = Database()
    body = request.body

    from_account = body["from_account"]
```

```
    to_account = body["to_account"]
    amount = float(body["amount"])

    db.start_transaction()

    try:
        # Deduct from sender
        db.execute(
            "UPDATE accounts SET balance = balance - ? WHERE id = ? AND balance >= ?",
            [amount, from_account, amount]
        )

        # Check if deduction succeeded
        sender = db.fetch_one(
            "SELECT balance FROM accounts WHERE id = ?",
            [from_account]
        )

        if sender is None:
            db.rollback()
            return response.json({"error": "Insufficient funds or account not found"}, 400)

        # Credit receiver
        db.execute(
            "UPDATE accounts SET balance = balance + ? WHERE id = ?",
            [amount, to_account]
        )

        db.commit()

        return response.json({"message": f"Transferred {amount} successfully"})

    except Exception as e:
        db.rollback()
        return response.json({"error": str(e)}, 500)
```

The three transaction methods:

- `db.start_transaction()` -- begin a transaction
- `db.commit()` -- save all changes since the transaction started
- `db.rollback()` -- undo all changes since the transaction started

Without transactions, each `execute()` call auto-commits on its own.

---

7. Batch Operations with execute_many

Inserting or updating many rows at once calls for `execute_many()`. It batches operations for speed:

```
@post("/api/notes/import")
async def import_notes(request, response):
    db = Database()
    notes = request.body.get("notes", [])

    if not notes:
```

```
        return response.json({"error": "No notes provided"}, 400)

    params_list = [
        [note["title"], note.get("content", "")]
        for note in notes
    ]

    db.execute_many(
        "INSERT INTO notes (title, content) VALUES (?, ?)",
        params_list
    )

    return response.json({"message": f"Imported {len(notes)} notes"}, 201)
```

```
curl -X POST http://localhost:7146/api/notes/import \
  -H "Content-Type: application/json" \
  -d '{"notes": [{"title": "Note 1", "content": "First"}, {"title": "Note 2", "content": "Second
```

```
{"message":"Imported 3 notes"}
```

`execute_many()` runs far faster than `execute()` in a loop. One round trip instead of many.

---

8. Insert, Update, and Delete Helpers

Tina4 provides shorthand methods that cut the boilerplate:

insert

```
result = db.insert("notes", {
    "title": "Quick Note",
    "content": "Created with insert helper"
})
# result.last_id contains the new row's ID
```

Returns a `DatabaseResult`. Access `result.last_id` for the inserted row's ID.

update

```
result = db.update("notes", {"title": "Updated Title", "content": "New content"}, "id = ?", [1])
```

The third argument is the WHERE clause, the fourth is a list of parameter values. Returns a `DatabaseResult` with `affected_rows`.

delete

```
result = db.delete("notes", "id = ?", [1])
```

Returns a `DatabaseResult` with `affected_rows`.

These helpers eliminate boilerplate INSERT/UPDATE/DELETE SQL. For complex queries, `execute()` is always available.

---

9. Schema Inspection

Tina4 exposes methods to inspect your database structure at runtime. The `Database` object delegates to the underlying adapter, so these work across all six engines.

get_tables()

```
db = Database()
tables = db.get_tables()
```

Returns a list of table names:

```
["notes", "users", "tina4_migration"]
```

get_columns()

```
columns = db.get_columns("notes")
```

Returns column definitions as a list of dictionaries:

```
[
    {"name": "id", "type": "INTEGER", "nullable": False, "default": None, "primary_key": True},
    {"name": "title", "type": "TEXT", "nullable": False, "default": None, "primary_key": False},
    {"name": "content", "type": "TEXT", "nullable": True, "default": "''", "primary_key": False}
    {"name": "created_at", "type": "TEXT", "nullable": True, "default": "CURRENT_TIMESTAMP", "pr
]
```

Each entry includes the column name, data type, whether it accepts NULL, its default value, and whether it belongs to the primary key.

table_exists()

```
if db.table_exists("notes"):
    # Table exists, safe to query
    notes = db.fetch("SELECT * FROM notes")
```

Returns `True` if the table exists, `False` otherwise.

get*database*type()

```
engine = db.get_database_type()
# "sqlite", "postgresql", "mysql", "mssql", or "firebird"
```

Returns a lowercase string identifying the active database engine. This is useful when you need engine-specific SQL in a multi-database setup.

A Schema Info Endpoint

Combine these methods to build a schema browser:

```
from tina4_python.core.router import get
from tina4_python.database.connection import Database

@get("/api/schema")
async def schema_info(request, response):
    db = Database()
    tables = db.get_tables()

    schema = {}
    for table in tables:
```

```
        schema[table] = db.get_columns(table)

    return response.json({"tables": schema})
```

```
{
  "tables": {
    "notes": [
      {"name": "id", "type": "INTEGER", "nullable": false, "default": null, "primary_key": true}
      {"name": "title", "type": "TEXT", "nullable": false, "default": null, "primary_key": false
    ],
    "users": [
      {"name": "id", "type": "INTEGER", "nullable": false, "default": null, "primary_key": true}
      {"name": "email", "type": "TEXT", "nullable": false, "default": null, "primary_key": false
    ]
  }
}
```

Schema inspection powers admin dashboards, migration generators, and dynamic form builders. The database tells you its own structure -- no guesswork required.

---

10. Migrations

Migrations are SQL files that version your database schema. Write them once. Apply them in order. Roll them back when needed.

File Naming

Migration files live in a `migrations/` directory. Two naming patterns are supported:

Pattern	Example
Sequential	`000001_create_users.sql`
Timestamp	`20260322160000_create_notes.sql`

Files sort alphabetically when they run. Pick one pattern and stick with it -- `000001_` sorts before `20260322_`, so mixing them leads to unexpected execution order.

Generating a Migration

```
tina4 generate migration create_notes_table
```

This creates two files:

```
migrations/20260322160000_create_notes_table.sql
migrations/20260322160000_create_notes_table.down.sql
```

The first is your "up" migration. The second is the matching "down" migration for rollbacks.

Writing the Up Migration

Open the generated `.sql` file and add your schema changes:

```
-- migrations/20260322160000_create_notes_table.sql
CREATE TABLE notes (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
```

```
    title TEXT NOT NULL,
    content TEXT DEFAULT '',
    category TEXT DEFAULT 'general',
    created_at TEXT DEFAULT CURRENT_TIMESTAMP,
    updated_at TEXT DEFAULT CURRENT_TIMESTAMP
);

CREATE INDEX idx_notes_category ON notes (category);
CREATE INDEX idx_notes_created_at ON notes (created_at);
```

Writing the Down Migration

The `.down.sql` file undoes whatever the up migration did:

```
-- migrations/20260322160000_create_notes_table.down.sql
DROP INDEX IF EXISTS idx_notes_created_at;
DROP INDEX IF EXISTS idx_notes_category;
DROP TABLE IF EXISTS notes;
```

Down migrations are optional. Everything works without them -- until you try to roll back. At that point Tina4 fails with a clear error about the missing file.

The `.down.sql` file must share the exact same base name as the up file. If your up file is `20260322160000_create_notes_table.sql`, the down file must be `20260322160000_create_notes_table.down.sql`.

Running Migrations

```
tina4 migrate
# or
tina4python migrate
```

Tina4 finds all pending `.sql` files in `migrations/`, sorts them alphabetically, and executes them in order. Each run receives a **batch number**. The batch groups every migration applied in that single run.

Checking Status

```
tina4python migrate:status
```

This shows which migrations have been applied and which are still pending. Run it before deploying to see what will execute.

Rolling Back

```
tina4python migrate:rollback
```

Rollback undoes the **entire last batch**. If your last `tina4 migrate` run applied three migration files, rollback reverts all three by running their `.down.sql` counterparts in reverse order.

Tracking Table

Tina4 creates a `tina4_migration` table in your database to track what has run:

Column	Purpose
`id`	Primary key

`migration_id`	The migration filename
`description`	Human-readable description
`batch`	Which batch this migration belonged to
`executed_at`	When it ran
`passed`	`1` if it succeeded, `0` if it failed

Failed migrations are recorded with `passed = 0`. On the next `tina4 migrate` run, the runner retries them.

Advanced SQL Splitting

Tina4's migration runner is not a naive line splitter. It correctly handles:

- `$$` **delimited blocks** -- PostgreSQL stored procedures and functions that contain semicolons
- `//` **blocks** -- alternative delimiter blocks
- `/* */` **block comments** -- skipped during splitting
- `--` **line comments** -- skipped during splitting

Write PostgreSQL stored procedures in your migration files without worrying about the runner choking on internal semicolons.

Migration Best Practices

- **One migration per change** -- do not pile multiple table changes into one file
- **Always write a down migration** -- so you can roll back cleanly
- **Never edit a migration that has been applied** -- create a new migration instead
- **Use descriptive names** -- `create_users_table`, `add_email_to_orders`, `create_category_index`
- **Pick one naming pattern** -- use either `000001_` or `YYYYMMDDHHMMSS_`, not both

---

11. Query Caching

Expensive queries that return the same result on every call deserve caching. Tina4 builds caching in. Enable it via environment variables in your `.env`:

```
TINA4_DB_CACHE=true
TINA4_DB_CACHE_TTL=30
```

When caching is active, all `fetch()` and `fetch_one()` calls cache their results. The `TINA4_DB_CACHE_TTL` value (in seconds) controls how long results stay cached. Write operations (`execute()`, `insert()`, `update()`, `delete()`) invalidate the entire cache.

To clear the cache manually (for example, after external data changes):

```
@post("/api/notes")
```

```
async def create_note(request, response):
    db = Database()
    result = db.execute(
        "INSERT INTO notes (title, content, category) VALUES (?, ?, ?)",
        [request.body["title"], request.body.get("content", ""), request.body.get("category", "g
    )

    # Cache is auto-invalidated on writes, but you can also clear manually:
    db.cache_clear()

    note = db.fetch_one("SELECT * FROM notes WHERE id = ?", [result.last_id])
    return response.json({"note": note}, 201)
```

Check cache performance with `db.cache_stats()`. It returns hits, misses, size, and TTL.

---

12. Exercise: Build a Notes App API

Build a complete notes application API with database persistence.

Requirements

- Create a migration for a `notes` table with: `id`, `title` (required), `content`, `category` (default "general"), `pinned` (boolean, default false), `created_at`, `updated_at`
- Build these endpoints:

Method	Path	Description
`GET`	`/api/notes`	List all notes. Support `?category=` and `?pinned=true` filters.
`GET`	`/api/notes/{id:int}`	Get a single note. 404 if not found.
`POST`	`/api/notes`	Create a note. Title required. Return 201.
`PUT`	`/api/notes/{id:int}`	Update a note. Return 404 if not found.
`DELETE`	`/api/notes/{id:int}`	Delete a note. Return 204.
`GET`	`/api/notes/categories`	List all distinct categories with counts.

Test with:

```
# Create notes
curl -X POST http://localhost:7146/api/notes \
  -H "Content-Type: application/json" \
  -d '{"title": "Shopping List", "content": "Milk, eggs, bread", "category": "personal"}'

curl -X POST http://localhost:7146/api/notes \
  -H "Content-Type: application/json" \
  -d '{"title": "Sprint Planning", "content": "Review backlog, assign tasks", "category": "work"

# List all
curl http://localhost:7146/api/notes

# Filter by category
curl "http://localhost:7146/api/notes?category=work"

# Filter by pinned
curl "http://localhost:7146/api/notes?pinned=true"

# Get categories
curl http://localhost:7146/api/notes/categories

# Update
curl -X PUT http://localhost:7146/api/notes/1 \
  -H "Content-Type: application/json" \
  -d '{"title": "Updated Shopping List", "content": "Milk, eggs, bread, butter"}'

# Delete
curl -X DELETE http://localhost:7146/api/notes/2
```

---

13. Solution

Migration

Generate the migration:

```
tina4 generate migration create_notes_table
```

Edit `migrations/20260322160000_create_notes_table.sql`:

```
CREATE TABLE notes (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    title TEXT NOT NULL,
    content TEXT DEFAULT '',
    category TEXT NOT NULL DEFAULT 'general',
    pinned INTEGER NOT NULL DEFAULT 0,
    created_at TEXT DEFAULT CURRENT_TIMESTAMP,
    updated_at TEXT DEFAULT CURRENT_TIMESTAMP
);

CREATE INDEX idx_notes_category ON notes (category);
CREATE INDEX idx_notes_pinned ON notes (pinned);
```

Edit `migrations/20260322160000_create_notes_table.down.sql`:

```
DROP INDEX IF EXISTS idx_notes_pinned;
DROP INDEX IF EXISTS idx_notes_category;
```

```
DROP TABLE IF EXISTS notes;
```

Run the migration:

```
tina4 migrate
```

Routes

Create `src/routes/notes.py`:

```
from tina4_python.core.router import get, post, put, delete
from tina4_python.database.connection import Database


@get("/api/notes")
async def list_notes(request, response):
    db = Database()
    category = request.params.get("category")
    pinned = request.params.get("pinned")

    sql = "SELECT * FROM notes"
    params = []
    conditions = []

    if category:
        conditions.append("category = ?")
        params.append(category)

    if pinned == "true":
        conditions.append("pinned = 1")

    if conditions:
        sql += " WHERE " + " AND ".join(conditions)

    sql += " ORDER BY pinned DESC, created_at DESC"

    notes = db.fetch(sql, params)

    return response.json({"notes": notes, "count": len(notes)})


@get("/api/notes/categories")
async def list_categories(request, response):
    db = Database()
    categories = db.fetch(
        "SELECT category, COUNT(*) as count FROM notes GROUP BY category ORDER BY count DESC"
    )
    return response.json({"categories": categories})


@get("/api/notes/{id:int}")
async def get_note(request, response):
    db = Database()
    note = db.fetch_one(
        "SELECT * FROM notes WHERE id = ?",
        [request.params["id"]]
    )

    if note is None:
        return response.json({"error": "Note not found"}, 404)
```

```
    return response.json(note)


@post("/api/notes")
async def create_note(request, response):
    db = Database()
    body = request.body

    if not body.get("title"):
        return response.json({"error": "Title is required"}, 400)

    result = db.execute(
        "INSERT INTO notes (title, content, category, pinned) VALUES (?, ?, ?, ?)",
        [body["title"], body.get("content", ""), body.get("category", "general"), 1 if body.get(
    )

    note = db.fetch_one("SELECT * FROM notes WHERE id = ?", [result.last_id])
    db.cache_clear()

    return response.json({"message": "Note created", "note": note}, 201)


@put("/api/notes/{id:int}")
async def update_note(request, response):
    db = Database()
    note_id = request.params["id"]
    body = request.body

    existing = db.fetch_one("SELECT * FROM notes WHERE id = ?", [note_id])
    if existing is None:
        return response.json({"error": "Note not found"}, 404)

    db.execute(
        """UPDATE notes
           SET title = ?, content = ?, category = ?,
               pinned = ?, updated_at = CURRENT_TIMESTAMP
           WHERE id = ?""",
        [
            body.get("title", existing["title"]),
            body.get("content", existing["content"]),
            body.get("category", existing["category"]),
            1 if body.get("pinned", existing["pinned"]) else 0,
            note_id
        ]
    )

    updated = db.fetch_one("SELECT * FROM notes WHERE id = ?", [note_id])
    db.cache_clear()

    return response.json({"message": "Note updated", "note": updated})


@delete("/api/notes/{id:int}")
async def delete_note(request, response):
    db = Database()
    note_id = request.params["id"]

    existing = db.fetch_one("SELECT * FROM notes WHERE id = ?", [note_id])
    if existing is None:
```

```
        return response.json({"error": "Note not found"}, 404)

    db.execute("DELETE FROM notes WHERE id = ?", [note_id])
    db.cache_clear()

    return response.json(None, 204)
```

Expected output for creating a note:

```
{
  "message": "Note created",
  "note": {
    "id": 1,
    "title": "Shopping List",
    "content": "Milk, eggs, bread",
    "category": "personal",
    "pinned": 0,
    "created_at": "2026-03-22 16:00:00",
    "updated_at": "2026-03-22 16:00:00"
  }
}
```

(Status: `201 Created`)

Expected output for categories:

```
{
  "categories": [
    {"category": "personal", "count": 1},
    {"category": "work", "count": 1}
  ]
}
```

---

14. Seeder -- Generating Test Data

Testing with an empty database tells you nothing. Testing with hand-typed rows is slow and brittle. The `FakeData` class generates realistic test data, and `seed_table()` inserts it in bulk.

FakeData

```
from tina4_python.seeder import FakeData

fake = FakeData()

fake.name()       # "Grace Lopez"
fake.email()      # "bob.anderson@demo.net"
fake.phone()      # "+1 (547) 382-9104"
fake.sentence()   # "Magna exercitation lorem ipsum dolor sit amet consectetur."
fake.paragraph()  # Four sentences of filler text
fake.integer()    # 7342
fake.decimal()    # 481.29
fake.date()       # "2023-07-14"
fake.uuid()       # "a3f1b2c4-d5e6-f7a8-b9c0-d1e2f3a4b5c6"
fake.address()    # "742 Oak Ave, Tokyo"
fake.boolean()    # True
```

Every method draws from built-in word banks -- no network calls, no external packages.

Deterministic Output

Pass a seed to get reproducible results. The same seed produces the same sequence every time:

```
fake = FakeData(seed=42)
fake.name()   # Always "Wendy White" with seed 42
fake.email()  # Always the same email with seed 42
```

Deterministic data means deterministic assertions. This matters for tests.

Seeding a Table

`seed_table()` combines `FakeData` with your database. Pass a field map -- a dictionary where each key is a column name and each value is a callable that generates data:

```
from tina4_python.seeder import FakeData, seed_table
from tina4_python.database.connection import Database

db = Database()
fake = FakeData(seed=1)

seed_table(db, "users", 100, {
    "name": fake.name,
    "email": fake.email,
    "phone": fake.phone,
    "bio": fake.sentence,
})
```

This inserts 100 rows into the `users` table. Each row calls `fake.name()`, `fake.email()`, and so on to generate its values. The function commits after all rows are inserted.

Overrides

Static values that apply to every row go in the `overrides` dictionary:

```
seed_table(db, "users", 50,
    field_map={
        "name": fake.name,
        "email": fake.email,
    },
    overrides={
        "role": "member",
        "active": 1,
    },
)
```

Every row gets `role = "member"` and `active = 1`. The field map generates the rest.

Seeding ORM Models

Use `seed_orm()` for one model and `seed_models()` for a related model graph:

```
from tina4_python.seeder import seed_orm, seed_models

seed_orm(User, count=20, seed=42, clear=True)
seed_models([User, Order], count=20, seed=42, clear=True)
```

`seed_models()` orders parents before children and clears in reverse order so foreign keys remain valid. Its `seed` controls the `FakeData` instance it creates. `seed_table()` deliberately

has no seed argument because its generators belong to the caller; create `FakeData(seed=42)` and pass its methods in the field map, as shown above.

FakeData is suitable for tests, development, demos, and load data. It is not cryptographically secure: never use it to create production passwords, tokens, API keys, or other secrets.

When to Use It

- Populating a development database with realistic data
- Writing integration tests that need rows in the database
- Load testing with thousands of records
- Demos and screenshots that look real without using real data

---

15. Gotchas

1. SQLite boolean quirk

Problem: Boolean values come back as `0` and `1` instead of `false` and `true` in JSON.

Cause: SQLite has no native boolean type. It stores booleans as integers.

Fix: This is expected. In your route handler, convert them: `note["pinned"] = bool(note["pinned"])`. Or handle it in the frontend. The ORM (Chapter 6) does this conversion with `BooleanField`.

2. last*insert*rowid() is SQLite-specific

Problem: `SELECT * FROM notes WHERE id = last_insert_rowid()` does not work on PostgreSQL or MySQL.

Cause: `last_insert_rowid()` is a SQLite function. Other databases use different mechanisms.

Fix: Use `db.insert()` which returns the last inserted ID regardless of engine. Or use database-specific syntax: PostgreSQL uses `RETURNING id` in the INSERT statement, MySQL uses `LAST_INSERT_ID()`.

3. String vs integer comparison

Problem: `WHERE id = ?` does not find the row even though the ID exists.

Cause: Path parameters arrive as strings by default. If `id` is `"5"` (string) and the column is an integer, some databases handle this differently.

Fix: Use typed path parameters (`{id:int}`) so the value is already an integer, or cast explicitly: `[int(request.params["id"])]`.

4. Connection not closed

Problem: After many requests, the application runs out of database connections.

Cause: You create `Database()` instances without them being cleaned up.

Fix: Tina4's `Database()` manages connection pooling internally. Creating `Database()` in each handler is fine because it reuses connections from the pool. If you see connection issues, check that you are not holding transactions open too long.

5. Migration order matters

Problem: A migration fails because it references a table that does not exist yet.

Cause: Migrations run in alphabetical order. If migration B depends on the table created by migration A, migration A must sort earlier.

Fix: Use `tina4 generate migration`, which prefixes every file with a `YYYYMMDDHHMMSS_` timestamp so new migrations sort after older ones. Do not mix `000001_` and `YYYYMMDDHHMMSS_` patterns in the same project -- `000001_` sorts before `20240315_`, which scrambles your intended order.

6. Missing down migration

Problem: `tina4python migrate:rollback` fails with an error about a missing file.

Cause: The `.down.sql` file does not exist for the migration being rolled back.

Fix: Create a `.down.sql` file with the exact same base name as the up migration. If your up file is `000001_create_users.sql`, the down file must be `000001_create_users.down.sql`. It should undo exactly what the up migration did. For `CREATE TABLE`, the down is `DROP TABLE IF EXISTS`. For `ALTER TABLE ADD COLUMN`, the down is `ALTER TABLE DROP COLUMN` (though SQLite does not support dropping columns -- in that case, recreate the table).

7. Failed migrations blocking progress

Problem: A migration failed and now `tina4 migrate` keeps retrying it.

Cause: Failed migrations are recorded in the `tina4_migration` table with `passed = 0`. Tina4 retries them on the next `migrate` run.

Fix: Fix the SQL in the migration file, then run `tina4 migrate` again. The failed migration retries. If you need to skip it entirely, update its `passed` column to `1` in the `tina4_migration` table manually -- but fix the root cause first.

8. SQL injection through string formatting

Problem: Your application is vulnerable to SQL injection attacks.

Cause: You used f-strings or string concatenation to build SQL queries with user input: `f"WHERE name = '{name}'"`.

Fix: Use parameterised queries: `"WHERE name = ?", [name]`. This is the single most important security practice for database code. Tina4 handles escaping and quoting for you.

ORM

1. From SQL to Objects

The last chapter was raw SQL. It works. It also gets repetitive. Every insert demands an INSERT statement. Every update demands an UPDATE. Every fetch maps column names to dictionary keys. Over and over.

Tina4's ORM turns database rows into Python objects. Define a model class with fields. The ORM writes the SQL. It stays SQL-first -- you can drop to raw SQL at any moment -- but for the 90% case of CRUD operations, the ORM handles the grunt work.

Configure the database before the models are used. The configuration-first path is an environment entry such as `TINA4_DATABASE_URL=sqlite:///data/app.db`. For an explicit connection, bind it once during application startup:

```
from tina4_python import Database
from tina4_python.orm import bind_database

bind_database(Database("sqlite:///data/app.db"))
```

If neither path supplies a database, ORM operations fail with guidance to call `bind_database()` or set `TINA4_DATABASE_URL`; they do not guess a connection.

Picture a blog. Authors, posts, comments. Authors own many posts. Posts own many comments. Comments belong to posts. Modeling these relationships with raw SQL means JOINs and manual foreign key management. The ORM makes this declarative.

ORM at a Glance

Every operation in this chapter works on the `Note` model you'll define in Section 2. Here's the short list before the long one:

Operation	Python
Create the table	`Note.create_table()`
Find by primary key	`Note.find_by_id(1)`
Filter by attributes	`Note.find({"category": "work"})`
Raw SQL where clause	`Note.where("category = ?", ["work"])`
Build and save	`Note.create(title="x")`
Save an instance	`note.save()`
Fetch every row	`Note.all()`
Delete a record	`note.delete()`

Count rows	`Note.count()`

`find()` takes attribute names and applies the field map; `where()` takes raw SQL and skips translation. The PHP, Ruby and Node.js books run the same operations in their own syntax, so the names line up if you ever move between them.

---

2. Defining a Model

Create a model file in `src/orm/`. Every `.py` file in that directory is auto-loaded.

Create `src/orm/note.py`:

```
from tina4_python.orm import ORM, IntegerField, StringField, BooleanField, DateTimeField

class Note(ORM):
    table_name = "notes"

    id = IntegerField(primary_key=True, auto_increment=True)
    title = StringField(required=True, max_length=200)
    content = StringField(default="")
    category = StringField(default="general")
    pinned = BooleanField(default=False)
    created_at = DateTimeField()
    updated_at = DateTimeField()
```

A complete model. Here is what each piece does:

- `table_name` -- the database table this model maps to. If omitted, the ORM uses the lowercase class name (e.g. `Contact` -> `contact`).
- `primary_key=True` on a field marks it as the primary key (defaults to `id` if none is specified)
- Each field is a class-level attribute with a field type

Field Types

Field Type	Python Type	SQL Type	Description
`IntegerField`	`int`	`INTEGER`	Whole numbers
`StringField`	`str`	`VARCHAR(255)`	Text strings
`NumericField`	`float`	`REAL`	Decimal numbers
`BooleanField`	`bool`	Engine-dependent: `BOOLEAN` on PostgreSQL and MySQL, `BIT` on MSSQL, `INTEGER` (0/1) on SQLite and Firebird	True/False
`DateTimeField`	`datetime`	`DATETIME`	Date and time
`TextField`	`str`	`TEXT`	Long text
`BlobField`	`bytes`	`BLOB`	Binary data
`ForeignKeyField`	`int`	`INTEGER`	Foreign key - auto-wires `belongs_to` and `has_many` (see *Relationships*)

Verbose names (`IntegerField`, `StringField`, `BooleanField`) are the standard. Short aliases (`IntField`, `StrField`, `BoolField`) also work.

When you compare a `BooleanField` column in raw SQL, pass a Python `True` or `False`, never `1` or `0`. SQLite accepts either, but PostgreSQL stores a native `BOOLEAN` and rejects `pinned = 1` with `operator does not exist: boolean = integer`. The ORM's own filters, such as `find({"pinned": True})`, convert for you.

Field Options

Option	Type	Description
`primary_key`	`bool`	Marks this field as the primary key
`required`	`bool`	Field must have a value (not None)
`default`	any	Default value when not provided
`max_length`	`int`	Maximum string length
`min_length`	`int`	Minimum string length
`min_value`	number	Minimum numeric value
`max_value`	number	Maximum numeric value
`choices`	list	Allowed values
`auto_increment`	`bool`	Auto-incrementing integer
`regex`	`str`	Pattern the value must match
`validator`	callable	Custom validation function

Field Mapping

When your Python attribute names do not match the database column names, use `field_mapping` to define the translation. `field_mapping` is a dict that maps Python attribute names to DB column names.

```
from tina4_python.orm import ORM, IntegerField, StringField

class User(ORM):
    table_name = "user_accounts"
    field_mapping = {
        "first_name": "fname",      # Python attr -> DB column
        "last_name": "lname",
        "email_address": "email",
    }

    id = IntegerField(primary_key=True, auto_increment=True)
    first_name = StringField(required=True)
    last_name = StringField(required=True)
    email_address = StringField(required=True)
```

With this mapping, `user.first_name` reads from and writes to the `fname` column. The ORM handles the conversion in both directions -- on `find_by_id()`, `save()`, `select()`, and `to_dict()`. This is useful with legacy databases or third-party schemas where you cannot rename the columns.

A common use case is Firebird or Oracle, which store column names in uppercase:

```
from tina4_python.orm import ORM, Field, StringField

class Account(ORM):
    table_name = "ACCOUNTS"
    field_mapping = {
        "account_no":   "ACCOUNTNO",
        "store_name":   "STORENAME",
        "credit_limit": "CREDITLIMIT",
    }
    account_no   = StringField()
    store_name   = StringField()
    credit_limit = Field(float, default=0.0)
```

Python code uses clean snake_case names (`account.account_no`, `account.credit_limit`). The ORM maps them to the uppercase DB columns automatically.

*get*db*column and* get*db*data

Two internal helpers make field_mapping available in custom code:

```
# Get the DB column name for a Python attribute
col = account._get_db_column("account_no")   # "ACCOUNTNO"

# Get a dict of all fields using DB column names as keys
data = account._get_db_data()
# {"ACCOUNTNO": "A001", "STORENAME": "Main Store", "CREDITLIMIT": 5000.0}
```

These are mainly used internally by `save()` and `create_table()`, but are available if you need them in custom queries.

find() vs where() -- naming convention

The two query methods have a deliberate difference in how they handle column names:

- `find(filter_dict)` uses **Python attribute names**. The ORM translates them via `field_mapping`.
- `where(filter_sql)` uses **raw DB column names** in the SQL string. No translation is done.

```
# find() -- use Python attribute names
accounts = Account.find({"account_no": "A001"})   # translates to ACCOUNTNO = ?

# where() -- use DB column names directly in the SQL
accounts = Account.where("ACCOUNTNO = ?", ["A001"])  # raw SQL, no translation
```

This means `find()` is portable across database engines, while `where()` gives you full control of the SQL.

auto_map and Case Conversion Utilities

The `auto_map` flag exists on the ORM base class for cross-language parity with the PHP and Node.js versions. In Python it is a no-op because Python convention already uses `snake_case`, which matches database column names.

For cases where you need to convert between naming conventions (for example, when serialising to a camelCase JSON API), two utility functions are available:

```
from tina4_python.orm.model import snake_to_camel, camel_to_snake
```

```
snake_to_camel("first_name")   # "firstName"
camel_to_snake("firstName")    # "first_name"
```

---

3. create_table -- Schema from Models

A model describes a table. It doesn't build one. The table has to exist before the first `save()` or query, and that goes for every model in this chapter - `Note` here, `Author` and `BlogPost` in Section 6, `Task` in Section 8, `Product` in Section 12, and the three blog models in the solution.

You can create the table straight from the model definition:

```
Note.create_table()
```

This generates and runs the `CREATE TABLE` SQL for your field definitions, and skips a table that's already there. It suits development and testing. For production, use migrations (Chapter 5) so schema changes are version-controlled.

A small script in the project root does the job once. A plain script doesn't read `.env` on its own, so load it first:

```
# create_tables.py
from tina4_python.dotenv import load_env
load_env()

from src.orm.note import Note

Note.create_table()
```

```
uv run python create_tables.py
```

If you skip this step, nothing crashes and nothing is stored either. `save()` returns `False`, and `note.last_error` holds the database's complaint. On SQLite, PostgreSQL and MySQL it ends with the fix: `table 'notes' does not exist; call Note.create_table() or run a migration`. Check the return value of `save()` and you'll never lose a row quietly.

Because `create_table()` only creates a missing table, it won't add a column to a table that already exists. Change a model after its table is built and you need a migration, or a fresh database while you're still experimenting.

---

4. CRUD Operations

save -- Create or Update

```
from tina4_python.core.router import post, put
from src.orm.note import Note

@post("/api/notes")
async def create_note(request, response):
    note = Note()
    note.title = request.body["title"]
    note.content = request.body.get("content", "")
    note.category = request.body.get("category", "general")
    note.pinned = request.body.get("pinned", False)
    note.save()

    return response({"message": "Note created", "note": note.to_dict()}, 201)
```

`save()` detects whether the record is new (INSERT) or existing (UPDATE) based on whether the primary key has a value. It runs `validate()` first (Section 12), so an invalid model never reaches the database. It returns `self` on success, so you can chain calls. It returns `False` on failure, and `note.last_error` (or `note.get_error()`) tells you why.

This route is a `POST`, and Tina4 secures every `POST`, `PUT`, `PATCH` and `DELETE` route by default. Called without a valid `Authorization: Bearer <token>` header it answers `401` before the handler runs. Chapter 8 shows how to issue tokens. While you experiment locally you can put `@noauth()` (from `tina4_python.core.router`) above `@post` to open a route, then take it away again before you ship.

create -- Build and Save in One Step

When you have a dict of data ready, `create()` builds the model and saves it in one call:

```
note = Note.create({
    "title": "Quick Note",
    "content": "Created in one step",
    "category": "general"
})
```

You can also pass keyword arguments:

```
note = Note.create(title="Quick Note", content="One step", category="general")
```

find*by*id -- Fetch One Record by Primary Key

```
from tina4_python.core.router import get
from src.orm.note import Note

@get("/api/notes/{id:int}")
async def get_note(id, request, response):
    note = Note.find_by_id(id)

    if note is None:
        return response({"error": "Note not found"}, 404)

    return response(note.to_dict())
```

`find_by_id()` takes a primary key value and returns a model instance, or `None` if no row matches. If soft delete is enabled, it excludes soft-deleted records.

Use `find_or_fail()` when you want a `ValueError` raised instead of `None`:

```
note = Note.find_or_fail(id)  # Raises ValueError if not found
```

find -- Query by Filter Dict

The `find()` method accepts a dictionary of column-value pairs and returns a list of matching records:

```
# Find all notes in the "work" category
work_notes = Note.find({"category": "work"})

# Find with pagination and ordering
recent = Note.find({"pinned": True}, limit=10, order_by="created_at DESC")

# Find all records (no filter)
all_notes = Note.find()
```

where -- Query with SQL Conditions

For more complex queries, `where()` takes a SQL WHERE clause with `?` placeholders:

```
notes = Note.where("category = ?", ["work"])
```

delete -- Remove a Record

```
from tina4_python.core.router import delete as delete_route
from src.orm.note import Note

@delete_route("/api/notes/{id:int}")
async def delete_note(id, request, response):
    note = Note.find_by_id(id)

    if note is None:
        return response({"error": "Note not found"}, 404)

    note.delete()

    return response(None, 204)
```

Listing Records

```
@get("/api/notes")
async def list_notes(request, response):
    category = request.query.get("category")

    if category:
        notes = Note.where("category = ?", [category])
    else:
        notes = Note.all()

    return response({
        "notes": [note.to_dict() for note in notes],
        "count": len(notes)
    })
```

`where()` takes a WHERE clause with `?` placeholders and a list of parameters. It returns a `ModelCollection` (covered just below). `all()` fetches all records. Both support pagination:

```
# With pagination
notes = Note.where("category = ?", ["work"], limit=20, offset=40)

# Fetch all with pagination
notes = Note.all(limit=20, offset=0)

# SQL-first query -- full control over the SQL
notes = Note.select(
    "SELECT * FROM notes WHERE pinned = ? ORDER BY created_at DESC",
    [True], limit=20, offset=0
)
```

ModelCollection -- The Page and the Total

Pagination hides an awkward gap. You ask for 20 rows and you get 20 rows, but a pager needs to say "page 3 of 13", and that means knowing the total number of matching rows, not the 20 sitting on this page. The old answer was a second `COUNT(*)` query with the same filter written out again by hand.

Tina4 closes the gap. `where()`, `select()`, `find()` (the filter form), `all()`, and `with_trashed()` all return a `ModelCollection` -- a subclass of `list`, so nothing you already wrote changes. You iterate it, index it, slice it, call `len()` on it, and serialise it to JSON exactly as before. It just carries one extra thing: the total for the filter, independent of `limit` and `offset`.

```
rows = Note.where("pinned = ?", [True], limit=20, offset=40)  # a page of up to 20 models
rows.get_total_records()   # 250 -- the whole matching set, ignoring limit/offset
```

That total is free. Every one of those methods already runs a `COUNT(*)` probe when it fetches the page, and the ORM used to hydrate the models and throw the count away. `ModelCollection` keeps it instead, so `get_total_records()` fires no second query. It is a method, not a `.count` property, on purpose: `list` already has a `count()` method, so a `.count` attribute would shadow a built-in.

The single-record finders are untouched. `find(pk)`, `find_by_id()`, `find_or_fail()`, `select_one()`, and `load()` still return one model or `None`.

Call `to_paginate()` for a ready-made pagination envelope. It hands back the same seven keys as `db.fetch(...).to_paginate()`, so a route paginates the same way whether the data came through the ORM or through raw SQL:

```
@get("/api/notes")
async def list_notes(request, response):
    page = Note.where("category = ?", ["work"], limit=20, offset=40)
    return response(page.to_paginate())
    # {"records": [...20...], "total": 250, "page": 3, "per_page": 20,
    #  "total_pages": 13, "limit": 20, "offset": 40}
```

The keys (`records`, `total`, `page`, `per_page`, `total_pages`, `limit`, `offset`) are snake_case and identical in all four frameworks, so a client reads the same JSON everywhere.

select_one -- Fetch a Single Record by SQL

When you need exactly one record from a custom SQL query:

```
note = Note.select_one("SELECT * FROM notes WHERE title = ?", ["Shopping List"])
```

Returns a model instance or `None`.

load -- Populate an Existing Instance

The `load()` method fills an existing model instance from the database:

```
note = Note()
note.id = 42
note.load()  # Loads data for id=42

# Or with a filter string
note = Note()
note.load("title = ?", ["Shopping List"])
```

Returns `True` if a record was found, `False` otherwise.

count -- Count Records

```
total = Note.count()
work_count = Note.count("category = ?", ["work"])
```

Respects soft delete -- only counts non-deleted records.

---

5. to*dict, to*json, and Other Serialisation

to_dict

Convert a model instance to a dictionary:

```
note = Note.find_by_id(1)

data = note.to_dict()
# {"id": 1, "title": "Shopping List", "content": "Milk, eggs", "category": "personal", "pinned":
```

The `include` parameter adds relationship data to the output (see Eager Loading below). Pass a list of relationship names:

```
# Include relationships in the dict
data = note.to_dict(include=["comments"])
```

to_json

Convert directly to a JSON string:

```
json_string = note.to_json()
# '{"id": 1, "title": "Shopping List", ...}'
```

Other Serialisation Methods

Method	Returns	Description
`to_dict(include=None)`	`dict`	Primary dict method with optional relationship includes
`to_assoc(include=None)`	`dict`	Alias for `to_dict()`
`to_object()`	`dict`	Alias for `to_dict()`
`to_json(include=None)`	`str`	JSON string
`to_array()`	`list`	Flat list of values (no keys)
`to_list()`	`list`	Alias for `to_array()`

---

6. Relationships

Tina4 ORM supports three relationship types: `has_many`, `has_one`, and `belongs_to`. Each works in two styles:

- **Imperative**: call the method on an instance when you need a one-off lookup
- **Declarative**: define the relationship as a class attribute using descriptor functions, accessed as a simple attribute, lazy-loaded on first access

Both styles support eager loading via `include=["relationship_name"]`.

ForeignKeyField - Auto-Wired Relationships

Declaring a column with `ForeignKeyField(to=OtherModel)` automatically wires both sides of the relationship. The declaring model gets a `belongs_to` accessor (the column name with `_id` stripped), and the referenced model gets a `has_many` accessor (the declaring class name lowercased with `s` appended, or whatever you pass via `related_name=`).

```
from tina4_python.orm import ORM, IntegerField, StringField, ForeignKeyField

class Author(ORM):
    table_name = "authors"
    id = IntegerField(primary_key=True, auto_increment=True)
    name = StringField(required=True)

class BlogPost(ORM):
    table_name = "posts"
    id = IntegerField(primary_key=True, auto_increment=True)
    title = StringField(required=True)
    author_id = ForeignKeyField(to=Author, related_name="posts")
```

With that single `ForeignKeyField` declaration, two accessors are auto-wired:

- `post.author` - returns the `Author` instance (belongs_to)
- `author.posts` - returns a list of `BlogPost` instances (has_many)

No manual `has_many` or `belongs_to` calls required. Both tables need to exist first: `Author.create_table()`, then `BlogPost.create_table()`.

```
post = BlogPost.find_by_id(1)
print(post.author.name)         # "Alice"

author = Author.find_by_id(1)
for p in author.posts:
    print(p.title)
```

has_many

An author has many posts:

Create `src/orm/author.py`:

```
from tina4_python.orm import ORM, IntegerField, StringField, DateTimeField

class Author(ORM):
    table_name = "authors"

    id = IntegerField(primary_key=True, auto_increment=True)
    name = StringField(required=True)
    email = StringField(required=True)
    bio = StringField(default="")
    created_at = DateTimeField()
```

Create `src/orm/blog_post.py`:

```
from tina4_python.orm import ORM, IntegerField, StringField, DateTimeField

class BlogPost(ORM):
    table_name = "posts"

    id = IntegerField(primary_key=True, auto_increment=True)
    author_id = IntegerField(required=True)
    title = StringField(required=True, max_length=300)
    slug = StringField(required=True)
    content = StringField(default="")
    status = StringField(default="draft", choices=["draft", "published", "archived"])
    created_at = DateTimeField()
    updated_at = DateTimeField()
```

This `BlogPost` maps to the same `posts` table as the short one above, with more columns. `create_table()` skips a table that already exists, so if you built the short version first, drop the `posts` table (or start a fresh database) before you create this one. The rest of the chapter uses this fuller shape.

Now use `has_many` to get an author's posts:

```
@get("/api/authors/{id:int}")
async def get_author(id, request, response):
    author = Author.find_by_id(id)

    if author is None:
        return response({"error": "Author not found"}, 404)

    posts = author.has_many(BlogPost, "author_id")
```

```
    data = author.to_dict()
    data["posts"] = [post.to_dict() for post in posts]

    return response(data)
```

```
{
  "id": 1,
  "name": "Alice",
  "email": "alice@example.com",
  "bio": "Tech writer",
  "posts": [
    {"id": 1, "title": "Getting Started with Tina4", "slug": "getting-started", "status": "publi
    {"id": 2, "title": "Advanced Routing", "slug": "advanced-routing", "status": "draft"}
  ]
}
```

has_one

A user has one profile:

```
profile = user.has_one(Profile, "user_id")
```

Returns a single model instance or `None`.

belongs_to

A post belongs to an author:

```
@get("/api/posts/{id:int}")
async def get_post(id, request, response):
    post = BlogPost.find_by_id(id)

    if post is None:
        return response({"error": "Post not found"}, 404)

    author = post.belongs_to(Author, "author_id")

    data = post.to_dict()
    data["author"] = author.to_dict() if author else None

    return response(data)
```

```
{
  "id": 1,
  "author_id": 1,
  "title": "Getting Started with Tina4",
  "slug": "getting-started",
  "content": "...",
  "status": "published",
  "author": {
    "id": 1,
    "name": "Alice",
    "email": "alice@example.com"
  }
}
```

---

7. Eager Loading

Calling relationship methods inside a loop creates the N+1 problem. Load 10 authors. Call `has_many(BlogPost, "author_id")` for each one. That fires 11 queries -- 1 for authors, 10 for posts. The page drags.

The `include` parameter on `all()`, `where()`, `find_by_id()`, and `select()` solves this. It eager-loads relationships in bulk:

```
@get("/api/authors")
async def list_authors(request, response):
    # Pass a list of relationship names - ORM batch-loads all posts in 2 queries total
    authors = Author.all(include=["posts"])

    data = []
    for author in authors:
        author_dict = author.to_dict(include=["posts"])
        data.append(author_dict)

    return response({"authors": data})
```

Without eager loading, 10 authors and their posts cost 11 queries. With eager loading: 2 queries. That is the difference between a fast page and a slow one.

Each name in `include` must be a relationship the model declares, either through a `ForeignKeyField` (Section 6) or a descriptor (next). In the example above, `posts` exists because of the `ForeignKeyField(to=Author, related_name="posts")` from Section 6. A name the model doesn't declare is skipped without an error, so a typo shows up as a missing key in the output rather than a crash.

Declarative Relationships with Descriptors

The imperative `has_many()`, `has_one()`, and `belongs_to()` methods called on instances work for one-off lookups. For models where relationships are always needed, declare them as class attributes using the descriptor functions imported from `tina4_python.orm`:

```
from tina4_python.orm import ORM, IntegerField, StringField, DateTimeField
from tina4_python.orm import has_many, has_one, belongs_to

class Author(ORM):
    table_name = "authors"

    id    = IntegerField(primary_key=True, auto_increment=True)
    name  = StringField(required=True)
    email = StringField(required=True)

    # Declare the relationship once on the class
    posts = has_many("BlogPost", foreign_key="author_id")


class BlogPost(ORM):
    table_name = "posts"

    id        = IntegerField(primary_key=True, auto_increment=True)
    author_id = IntegerField(required=True)
    title     = StringField(required=True)
```

```
    # Lazy-load the parent author
    author   = belongs_to("Author", foreign_key="author_id")
    # Lazy-load comments
    comments = has_many("Comment", foreign_key="post_id")
```

With declarative descriptors, accessing the relationship is a simple attribute read:

```
author = Author.find_by_id(1)
for post in author.posts:          # lazy-loads on first access
    print(post.title)

post = BlogPost.find_by_id(10)
print(post.author.name)            # lazy-loads the related Author
```

Eager loading works through the `include` parameter. Pass a list of relationship names:

```
# Eager load posts when fetching all authors
authors = Author.all(include=["posts"])

# Eager load author and comments when finding a single post
post = BlogPost.find_by_id(1, include=["author", "comments"])
```

Nested Eager Loading

Dot notation loads multiple levels deep:

```
# Load authors, their posts, and each post's comments
authors = Author.all(include=["posts", "posts.comments"])
```

Authors, their posts, and each post's comments. Three queries total instead of hundreds.

to_dict with Nested Includes

When eager loading is active, `to_dict(include=...)` embeds the related data:

```
post = BlogPost.find_by_id(1, include=["author", "comments"])
data = post.to_dict(include=["author", "comments"])
```

```
{
  "id": 1,
  "title": "Getting Started with Tina4",
  "author": {
    "id": 1,
    "name": "Alice",
    "email": "alice@example.com"
  },
  "comments": [
    {"id": 1, "body": "Great post!", "author_name": "Bob"}
  ]
}
```

---

8. Soft Delete

Sometimes a record needs to disappear from queries without leaving the database. Soft delete handles this. The row stays. A flag marks it as deleted. Queries skip it.

```
from tina4_python.orm import ORM, IntegerField, StringField, BooleanField

class Task(ORM):
```

```
    table_name = "tasks"
    soft_delete = True  # Enable soft delete

    id = IntegerField(primary_key=True, auto_increment=True)
    title = StringField(required=True)
    completed = BooleanField(default=False)
    is_deleted = IntegerField(default=0)  # Required for soft delete (0 = active, 1 = deleted)
    created_at = StringField()
```

When `soft_delete = True`, the ORM changes its behaviour:

- `task.delete()` sets `is_deleted` to `1` instead of running a DELETE query
- `Task.all()`, `Task.where()`, and `Task.find_by_id()` filter out records where `is_deleted = 1`
- `task.restore()` sets `is_deleted` back to `0` and makes the record visible again
- `task.force_delete()` permanently removes the row from the database
- `Task.with_trashed()` includes soft-deleted records in query results

Deleting and Restoring

```
# Soft delete -- sets is_deleted = 1, row stays in the database
task = Task.find_by_id(1)
task.delete()

# Restore -- sets is_deleted = 0, record is visible again
task.restore()

# Permanently delete -- removes the row, no recovery possible
task.force_delete()
```

`restore()` is the inverse of `delete()`. It sets `is_deleted` back to `0` and commits the change. The record reappears in all standard queries.

Including Soft-Deleted Records

Standard queries (`all()`, `where()`, `find_by_id()`) exclude soft-deleted records. When you need to see everything -- for admin dashboards, audit logs, or data recovery -- use `with_trashed()`:

```
# All tasks, including soft-deleted ones
all_tasks = Task.with_trashed()

# Soft-deleted tasks matching a condition
deleted_tasks = Task.with_trashed("completed = ?", [True])
```

`with_trashed()` accepts the same filter parameters as `where()`. The only difference: it ignores the `is_deleted` filter that standard queries apply.

Counting with Soft Delete

The `count()` class method respects soft delete. It only counts non-deleted records:

```
active_count = Task.count()
active_done = Task.count("completed = ?", [True])
```

When to Use Soft Delete

Soft delete suits data that users might want to recover -- emails, documents, user accounts. It also serves audit requirements where regulations demand retention. For temporary data (sessions, cache entries, logs), hard delete keeps the table lean.

---

9. Auto-CRUD

Writing the same five REST endpoints for every model gets tedious. Auto-CRUD generates them from your model class. Define the model. Register it. Five routes appear.

The auto_crud Flag

The simplest approach -- set `auto_crud = True` on your model class:

```
class Note(ORM):
    table_name = "notes"
    auto_crud = True  # Generates REST endpoints automatically

    id = IntegerField(primary_key=True, auto_increment=True)
    title = StringField(required=True)
    content = StringField(default="")
```

The moment Python loads this class, the ORM metaclass detects `auto_crud = True` and registers it with `AutoCrud`. Five routes appear at `/api/notes` with no additional code.

Here is a more complete example with a `Product` model:

```
from tina4_python.orm import ORM, Field, IntegerField, StringField

class Product(ORM):
    table_name = "products"
    auto_crud  = True   # registers /api/products routes automatically

    id    = IntegerField(primary_key=True, auto_increment=True)
    name  = StringField(required=True)
    price = Field(float, default=0.0)
```

This registers five endpoints at `/api/products` with no route files needed.

Manual Registration

You can also register models explicitly using `AutoCrud.register()`:

```
from tina4_python.crud import AutoCrud
from src.orm.note import Note

AutoCrud.register(Note)
```

Both approaches produce the same result:

Method	Path	Description
`GET`	`/api/notes`	List all with pagination (`limit`, `offset` params)

`GET`	`/api/notes/{id}`	Get one by primary key
`POST`	`/api/notes`	Create a new record
`PUT`	`/api/notes/{id}`	Update a record
`DELETE`	`/api/notes/{id}`	Delete a record

The endpoint prefix derives from the table name. The `notes` table becomes `/api/notes`. Pass a custom prefix to change it:

```
AutoCrud.register(Note, prefix="/api/v2")
# Routes: /api/v2/notes, /api/v2/notes/{id}, etc.
```

The generated write routes (`POST`, `PUT`, `DELETE`) follow the same secure-by-default rule as your own routes: without a valid Bearer token they answer `401`. Pass `public=True` to open them on purpose:

```
AutoCrud.register(Note, public=True)
```

Auto-Discovering Models

Rather than registering each model by hand, point `AutoCrud.discover()` at your models directory. It scans every `.py` file, finds ORM subclasses, and registers them all:

```
from tina4_python.crud import AutoCrud

AutoCrud.discover("src/orm", prefix="/api")
```

Every ORM model in `src/orm/` gets five REST endpoints. No route files needed.

What the Generated Routes Do

GET /api/notes returns paginated results:

```
curl "http://localhost:7146/api/notes?limit=10&offset=0"
```

```
{
  "records": [
    {"id": 1, "title": "Shopping List", "content": "Milk, eggs", "category": "personal", "pinned
    {"id": 2, "title": "Sprint Plan", "content": "Review backlog", "category": "work", "pinned":
  ],
  "total": 2,
  "page": 1,
  "per_page": 10,
  "total_pages": 1,
  "limit": 10,
  "offset": 0
}
```

That's the same seven-key envelope `to_paginate()` returns (Section 4).

POST /api/notes validates input before saving. It's a write route, so send a token:

```
curl -X POST http://localhost:7146/api/notes \
  -H "Content-Type: application/json" \
  -H "Authorization: Bearer $TOKEN" \
  -d '{"title": "New Note", "content": "Created via auto-CRUD"}'
```

If validation fails (for example, a required field is missing), the endpoint returns a 422 with error details:

```
{"error": "Validation failed", "detail": ["title is required"]}
```

DELETE /api/notes/1 respects soft delete. If the model has `soft_delete = True`, the record is marked deleted instead of removed.

Custom Routes Alongside Auto-CRUD

If you need special logic for one endpoint (custom validation, side effects, complex queries), define that route yourself and let Auto-CRUD handle the rest. How the two meet depends on the path you write.

At startup the server imports `src/orm/` before `src/routes/`, so a model with `auto_crud = True` registers its five routes first. When a route file then registers the **same method and the same path string**, the later registration replaces the earlier one, and your handler wins:

```
from tina4_python.core.router import get

@get("/api/notes/{id}")          # identical to the Auto-CRUD path: replaces it
async def get_note(id, request, response):
    ...
```

A different pattern for the same URL does **not** replace anything. Both routes stay registered, the router takes the first match, and the Auto-CRUD route was there first:

```
@get("/api/notes/{id:int}")      # different path string: Auto-CRUD's /api/notes/{id} still answ
async def get_note(id, request, response):
    ...
```

So copy the Auto-CRUD path exactly (`/api/notes/{id}`, with a plain `{id}`) when you override one of its routes, or give your route a path of its own.

Introspection

Check which models are registered:

```
registered = AutoCrud.models()
# {"notes": <class 'Note'>, "users": <class 'User'>}
```

---

10. Cached Queries

For expensive queries that don't change often, `cached()` caches the results in memory with a TTL:

```
# Cache for 60 seconds
popular = Note.cached(
    "SELECT * FROM notes WHERE pinned = ? ORDER BY created_at DESC",
    [True], ttl=60, limit=20
)
```

Clear the cache when data changes:

```
Note.clear_cache()
```

---

11. Scopes

Scopes are reusable query filters baked into the model:

```
class BlogPost(ORM):
    table_name = "posts"

    id = IntegerField(primary_key=True, auto_increment=True)
    title = StringField(required=True)
    status = StringField(default="draft")
    created_at = DateTimeField()

    @classmethod
    def published(cls):
        return cls.where("status = ?", ["published"])

    @classmethod
    def drafts(cls):
        return cls.where("status = ?", ["draft"])

    @classmethod
    def recent(cls, days=7):
        return cls.where(
            "created_at > datetime('now', ?)",
            [f"-{days} days"]
        )
```

Use them in your routes:

```
@get("/api/posts/published")
async def published_posts(request, response):
    posts = BlogPost.published()
    return response({"posts": [p.to_dict() for p in posts]})

@get("/api/posts/recent")
async def recent_posts(request, response):
    days = int(request.query.get("days", 7))
    posts = BlogPost.recent(days)
    return response({"posts": [p.to_dict() for p in posts]})
```

You can also register scopes dynamically with the `scope()` class method:

```
BlogPost.scope("active", "status != ?", ["archived"])

# Now call it:
active_posts = BlogPost.active()
```

Scopes keep query logic in the model where it belongs. Route handlers stay thin.

---

12. Input Validation

Field definitions carry validation rules. Call `validate()` before `save()` and the ORM checks every constraint:

```
from tina4_python.orm import ORM, IntegerField, StringField, NumericField

class Product(ORM):
    table_name = "products"

    id = IntegerField(primary_key=True, auto_increment=True)
    name = StringField(required=True, min_length=2, max_length=200)
    sku = StringField(required=True, regex=r"^[A-Z]{2}-\d{4}$")  # e.g., EL-1234
    price = NumericField(required=True, min_value=0.01, max_value=999999.99)
    category = StringField(choices=["Electronics", "Kitchen", "Office", "Fitness"])

@post("/api/products")
async def create_product(request, response):
    product = Product()
    product.name = request.body.get("name")
    product.sku = request.body.get("sku")
    product.price = request.body.get("price")
    product.category = request.body.get("category")

    errors = product.validate()
    if errors:
        return response({"errors": errors}, 400)

    product.save()
    return response({"product": product.to_dict()}, 201)
```

If validation fails, `validate()` returns a list of error messages:

```
{
  "errors": [
    "name must be at least 2 characters",
    "sku does not match the required format",
    "price must be at least 0.01",
    "category must be one of [\"Electronics\",\"Kitchen\",\"Office\",\"Fitness\"]"
  ]
}
```

---

13. Exercise: Build a Blog with Relationships

Build a blog API with authors, posts, and comments.

Requirements

- Create these models:

Author: `id`, `name` (required), `email` (required), `bio`, `created_at`

Post: `id`, `author_id` (integer foreign key), `title` (required, max 300), `slug` (required), `content`, `status` (choices: draft/published/archived, default draft), `created_at`, `updated_at`

Comment: `id`, `post_id` (integer foreign key), `author_name` (required), `author_email` (required), `body` (required, min 5 chars), `created_at`

- Build these endpoints:

Method	Path	Description

`POST`	`/api/authors`	Create an author
`GET`	`/api/authors/{id:int}`	Get author with their posts
`POST`	`/api/posts`	Create a post (requires author_id)
`GET`	`/api/posts`	List published posts with author info
`GET`	`/api/posts/{id:int}`	Get post with author and comments
`POST`	`/api/posts/{id:int}/comments`	Add comment to a post

---

14. Solution

Create `src/orm/author.py`:

```
from tina4_python.orm import ORM, IntegerField, StringField, DateTimeField

class Author(ORM):
    table_name = "authors"

    id = IntegerField(primary_key=True, auto_increment=True)
    name = StringField(required=True, min_length=2)
    email = StringField(required=True)
    bio = StringField(default="")
    created_at = DateTimeField()
```

Create `src/orm/blog_post.py`:

```
from tina4_python.orm import ORM, IntegerField, StringField, DateTimeField

class BlogPost(ORM):
    table_name = "posts"

    id = IntegerField(primary_key=True, auto_increment=True)
    author_id = IntegerField(required=True)
    title = StringField(required=True, max_length=300)
    slug = StringField(required=True)
    content = StringField(default="")
    status = StringField(default="draft", choices=["draft", "published", "archived"])
    created_at = DateTimeField()
    updated_at = DateTimeField()

    @classmethod
    def published(cls):
        return cls.where("status = ?", ["published"])
```

Create `src/orm/comment.py`:

```
from tina4_python.orm import ORM, IntegerField, StringField, DateTimeField
```

```
class Comment(ORM):
    table_name = "comments"

    id = IntegerField(primary_key=True, auto_increment=True)
    post_id = IntegerField(required=True)
    author_name = StringField(required=True)
    author_email = StringField(required=True)
    body = StringField(required=True, min_length=5)
    created_at = DateTimeField()
```

Build the three tables before the first request. Extend `create_tables.py` from Section 3 and run it once with `uv run python create_tables.py`:

```
# create_tables.py
from tina4_python.dotenv import load_env
load_env()

from src.orm.author import Author
from src.orm.blog_post import BlogPost
from src.orm.comment import Comment

for model in (Author, BlogPost, Comment):
    model.create_table()
```

Create `src/routes/blog.py`:

```
from tina4_python.core.router import get, post
from src.orm.author import Author
from src.orm.blog_post import BlogPost
from src.orm.comment import Comment


@post("/api/authors")
async def create_author(request, response):
    author = Author()
    author.name = request.body.get("name")
    author.email = request.body.get("email")
    author.bio = request.body.get("bio", "")

    errors = author.validate()
    if errors:
        return response({"errors": errors}, 400)

    author.save()
    return response({"author": author.to_dict()}, 201)


@get("/api/authors/{id:int}")
async def get_author(id, request, response):
    author = Author.find_by_id(id)

    if author is None:
        return response({"error": "Author not found"}, 404)

    posts = BlogPost.where("author_id = ?", [author.id])

    data = author.to_dict()
    data["posts"] = [p.to_dict() for p in posts]

    return response(data)
```

```
@post("/api/posts")
async def create_post(request, response):
    body = request.body

    # Verify author exists
    author = Author.find_by_id(body.get("author_id"))
    if author is None:
        return response({"error": "Author not found"}, 404)

    blog_post = BlogPost()
    blog_post.author_id = body["author_id"]
    blog_post.title = body.get("title")
    blog_post.slug = body.get("slug")
    blog_post.content = body.get("content", "")
    blog_post.status = body.get("status", "draft")

    errors = blog_post.validate()
    if errors:
        return response({"errors": errors}, 400)

    blog_post.save()
    return response({"post": blog_post.to_dict()}, 201)


@get("/api/posts")
async def list_posts(request, response):
    posts = BlogPost.published()
    data = []

    for p in posts:
        post_dict = p.to_dict()
        author = p.belongs_to(Author, "author_id")
        post_dict["author"] = author.to_dict() if author else None
        data.append(post_dict)

    return response({"posts": data, "count": len(data)})


@get("/api/posts/{id:int}")
async def get_post(id, request, response):
    blog_post = BlogPost.find_by_id(id)

    if blog_post is None:
        return response({"error": "Post not found"}, 404)

    author = blog_post.belongs_to(Author, "author_id")
    comments = blog_post.has_many(Comment, "post_id")

    data = blog_post.to_dict()
    data["author"] = author.to_dict() if author else None
    data["comments"] = [c.to_dict() for c in comments]
    data["comment_count"] = len(comments)

    return response(data)


@post("/api/posts/{id:int}/comments")
async def add_comment(id, request, response):
    blog_post = BlogPost.find_by_id(id)
```

```
    if blog_post is None:
        return response({"error": "Post not found"}, 404)

    comment = Comment()
    comment.post_id = id
    comment.author_name = request.body.get("author_name")
    comment.author_email = request.body.get("author_email")
    comment.body = request.body.get("body")

    errors = comment.validate()
    if errors:
        return response({"errors": errors}, 400)

    comment.save()
    return response({"comment": comment.to_dict()}, 201)
```

The three `POST` routes are secured by default, so test them with an `Authorization: Bearer <token>` header (Chapter 8). The `GET` routes answer without one.

---

15. Gotchas

1. Forgetting to call save()

Problem: You set properties on a model but the database does not change.

Cause: Setting `note.title = "New Title"` only changes the Python object. The database remains unchanged until you call `note.save()`.

Fix: Call `save()` after modifying properties. Check the return value -- `save()` returns `self` on success and `False` on failure.

2. find*by*id() returns None

Problem: You call `Note.find_by_id(id)` but get `None` instead of a note object.

Cause: `find_by_id()` returns `None` when no row matches the given primary key. If soft delete is enabled, `find_by_id()` also excludes soft-deleted records.

Fix: Check for `None` after `find_by_id()`: `if note is None: return 404`. Use `find_or_fail()` if you want a `ValueError` raised instead.

3. find() vs find*by*id()

Problem: You call `Note.find(42)` expecting a single record, but get unexpected results.

Cause: `find()` takes a dict filter (`find({"id": 42})`), not a bare primary key value. For single-record lookups by primary key, use `find_by_id(42)`.

Fix: Use `find_by_id(id)` for primary key lookups. Use `find({"column": value})` for filter-based queries.

4. Circular imports with relationships

Problem: `from src.orm.post import BlogPost` in `author.py` and `from src.orm.author import Author` in `post.py` causes an `ImportError`.

Cause: Python cannot handle circular imports at module level.

Fix: Import inside the method that uses the relationship, not at the top of the file. Or pass the model class as a parameter in the route handler where you use both models.

5. to_dict() includes everything

Problem: `user.to_dict()` includes `password_hash` in the API response.

Cause: `to_dict()` includes all fields by default.

Fix: Build the response dict manually, omitting sensitive fields: `{"id": user.id, "name": user.name, "email": user.email}`. Or create a helper method on your model class that returns only safe fields.

6. save() returns False on invalid data

Problem: `save()` returns `False` and the row never reaches the database, but nothing raised.

Cause: `save()` runs `validate()` first. If any field fails its rules, it refuses the write, logs the reason, and returns `False` instead of raising. The same happens when the database rejects the write (a missing table, a `NOT NULL` column).

Fix: Check the return value. `model.last_error` (or `model.get_error()`) holds the reason. In a route handler, call `errors = model.validate()` before `save()` when you want to send the field errors back to the client as a `400`.

7. Foreign key not enforced

Problem: You save a post with `author_id = 999` and it succeeds, even though no author with ID 999 exists.

Cause: SQLite does not enforce foreign key constraints by default. The ORM defines the relationship through `has_many`/`belongs_to` methods, but the database itself may not enforce it.

Fix: Enable SQLite foreign keys with `PRAGMA foreign_keys = ON;` in a migration, or validate the foreign key in your route handler before saving.

8. N+1 query problem

Problem: Listing 100 authors with their posts runs 101 queries (1 for authors + 100 for posts), and the page loads slowly.

Cause: You call `author.has_many(BlogPost, "author_id")` inside a loop for each author.

Fix: Use eager loading with the `include` parameter on `all()`, `where()`, or `select()`. Or fetch all posts in a single query and group them manually:

```
authors = Author.all()
all_posts = BlogPost.select(
```

```
    "SELECT * FROM posts WHERE author_id IN (" + ",".join(str(a.id) for a in authors) + ")"
)
posts_by_author = {}
for post in all_posts:
    posts_by_author.setdefault(post.author_id, []).append(post)
```

9. Auto-CRUD endpoint conflicts

Problem: Custom route at `/api/notes/{id}` stops working after registering Auto-CRUD for the Note model.

Cause: Both routes match the same URL, and the router answers with the first match. Auto-CRUD registers while `src/orm/` loads, before your files in `src/routes/`, so its `/api/notes/{id}` is first in line. Your `/api/notes/{id:int}` is a different path string, so it doesn't replace the Auto-CRUD route. It just never gets a turn.

Fix: Declare the custom route with the exact Auto-CRUD path, `/api/notes/{id}`. Re-registering the same method and path replaces the earlier route, so yours wins. Or give the custom route a path of its own.

10. Soft-deleted records appearing in queries

Problem: You soft-deleted a record, but queries still return it.

Cause: Soft delete requires the `soft_delete = True` flag on the model class and an `is_deleted = IntegerField(default=0)` field. Without both, soft delete is inactive.

Fix: Verify both the `soft_delete = True` flag and the `is_deleted = IntegerField(default=0)` field exist on the model. The column stores `0` for active records and `1` for deleted ones.

QueryBuilder Integration

ORM models provide a `query()` class method that returns a `QueryBuilder` pre-configured with the model's table name and database connection. This gives you a fluent API for building complex queries without writing raw SQL:

```
# Fluent query builder from ORM
results = User.query() \
    .select("id", "name", "email") \
    .where("active = ?", [True]) \
    .order_by("name") \
    .limit(50) \
    .get()

# First matching record
user = User.query() \
    .where("email = ?", ["alice@example.com"]) \
    .first()

# Count
total = User.query() \
    .where("role = ?", ["admin"]) \
    .count()
```

```
# Check existence
exists = User.query() \
    .where("email = ?", ["test@example.com"]) \
    .exists()
```

See the QueryBuilder chapter for the full fluent API including joins, grouping, having, and MongoDB support.

QueryBuilder

Every database query starts as a string. Small queries stay readable. But the moment you add optional filters, pagination, sorting, and joins, string concatenation turns your code into an unreadable mess of `if` statements and f-strings. One missed space, one misplaced comma, and the query breaks.

QueryBuilder solves this. It gives you a fluent, chainable API that assembles SQL for you. You describe what you want (columns, conditions, joins, ordering) and QueryBuilder produces the correct SQL string with properly separated parameters. No concatenation. No injection risk. No debugging whitespace.

---

1. The Factory: from_table()

Every QueryBuilder chain starts with `from_table()`. It takes a table name and an optional database connection.

```
from tina4_python.query_builder import QueryBuilder

qb = QueryBuilder.from_table("users", db)
```

The first argument is the table name. The second is your database connection, the same `Database` object you use everywhere else. If you omit the database, QueryBuilder will fall back to the global ORM database (set via `bind_database()`). If neither exists, it raises a `RuntimeError` when you try to execute.

`from_table()` returns a fresh `QueryBuilder` instance. Every method you call on it returns the same instance, so you can chain.

---

2. Choosing Columns: select()

By default, QueryBuilder selects all columns (`*`). Use `select()` to narrow the result.

```
qb = QueryBuilder.from_table("users", db) \
    .select("id", "name", "email")
```

Pass column names as separate arguments, not a list. Each call to `select()` replaces the previous column selection.

```
# This selects only "email", not "id", "name", "email"
qb = QueryBuilder.from_table("users", db) \
    .select("id", "name") \
    .select("email")
```

If you want all columns, skip `select()` entirely. The default is `*`.

---

3. Filtering: where() and or_where()

`where()` adds a condition joined with `AND`. Use `?` placeholders for parameter values.

```
result = QueryBuilder.from_table("users", db) \
    .where("active = ?", [1]) \
    .where("age > ?", [18]) \
    .get()
```

This produces:

```
SELECT * FROM users WHERE active = ? AND age > ?
```

Parameters `[1, 18]` are passed separately to the database driver. No string interpolation. No injection.

OR conditions

Use `or_where()` when you need an OR clause.

```
result = QueryBuilder.from_table("users", db) \
    .where("role = ?", ["admin"]) \
    .or_where("role = ?", ["superadmin"]) \
    .get()
```

Produces:

```
SELECT * FROM users WHERE role = ? OR role = ?
```

The first condition in the chain never gets a connector prefix. Every subsequent `where()` adds `AND`, and every `or_where()` adds `OR`.

Conditions without parameters

Some conditions do not need parameters. Pass the condition string alone.

```
qb.where("is_deleted = 0")
```

---

4. Joins: join() and left_join()

`join()` adds an `INNER JOIN`. `left_join()` adds a `LEFT JOIN`. Both take a table name and an ON clause.

```
result = QueryBuilder.from_table("orders", db) \
    .select("orders.id", "users.name", "orders.total") \
    .join("users", "users.id = orders.user_id") \
    .where("orders.total > ?", [100]) \
    .get()
```

Produces:

```
SELECT orders.id, users.name, orders.total
FROM orders
INNER JOIN users ON users.id = orders.user_id
WHERE orders.total > ?
```

For optional relationships, where a matching row might not exist, use `left_join()`.

```
result = QueryBuilder.from_table("users", db) \
    .select("users.name", "profiles.avatar") \
    .left_join("profiles", "profiles.user_id = users.id") \
    .get()
```

You can chain multiple joins. They appear in the SQL in the order you add them.

```
result = QueryBuilder.from_table("orders", db) \
    .join("users", "users.id = orders.user_id") \
    .join("products", "products.id = orders.product_id") \
    .left_join("discounts", "discounts.order_id = orders.id") \
    .get()
```

---

5. Aggregation: group_by() and having()

`group_by()` groups rows by a column. Call it once per column.

```
result = QueryBuilder.from_table("orders", db) \
    .select("user_id", "COUNT(*) as order_count", "SUM(total) as revenue") \
    .group_by("user_id") \
    .get()
```

Produces:

```
SELECT user_id, COUNT(*) as order_count, SUM(total) as revenue
FROM orders
GROUP BY user_id
```

To group by multiple columns, chain multiple calls.

```
qb.group_by("user_id").group_by("status")
```

Filtering groups with having()

`having()` filters after aggregation. It works like `where()` but applies to grouped results.

```
result = QueryBuilder.from_table("orders", db) \
    .select("user_id", "COUNT(*) as order_count") \
    .group_by("user_id") \
    .having("COUNT(*) > ?", [5]) \
    .get()
```

Produces:

```
SELECT user_id, COUNT(*) as order_count
FROM orders
GROUP BY user_id
HAVING COUNT(*) > ?
```

Parameters in `having()` are kept separate from `where()` parameters internally but merged at execution time. The order is always correct.

---

6. Sorting: order_by()

`order_by()` takes a column name and optional direction as a single string.

```
result = QueryBuilder.from_table("users", db) \
    .order_by("name ASC") \
    .get()
```

Chain multiple calls for multi-column sorting. They appear in the SQL in order.

```
result = QueryBuilder.from_table("products", db) \
    .order_by("category ASC") \
    .order_by("price DESC") \
    .get()
```

Produces:

```
SELECT * FROM products ORDER BY category ASC, price DESC
```

If you omit the direction, your database's default applies (usually `ASC`).

---

7. Pagination: limit()

`limit()` sets the maximum number of rows. Pass an optional second argument for the offset.

```
# First page: 10 rows starting at row 0
result = QueryBuilder.from_table("users", db) \
    .limit(10) \
    .get()

# Second page: 10 rows starting at row 10
result = QueryBuilder.from_table("users", db) \
    .limit(10, 10) \
    .get()

# Third page
result = QueryBuilder.from_table("users", db) \
    .limit(10, 20) \
    .get()
```

When you call `get()`, the limit and offset values are passed to `db.fetch()`. If you do not call `limit()`, the default is 100 rows starting at offset 0.

Pagination pattern

A common pattern for API endpoints:

```
@get("/api/users")
async def list_users(request, response):
    page = int(request.params.get("page", 1))
    per_page = int(request.params.get("per_page", 20))
    offset = (page - 1) * per_page

    result = QueryBuilder.from_table("users", db) \
        .select("id", "name", "email") \
        .where("active = ?", [1]) \
        .order_by("name ASC") \
        .limit(per_page, offset) \
        .get()

    total = QueryBuilder.from_table("users", db) \
        .where("active = ?", [1]) \
```

```
        .count()

    return response({
        "users": result.to_list(),
        "total": total,
        "page": page,
        "per_page": per_page,
    })
```

---

8. Inspecting the SQL: to_sql()

Before executing, you can inspect the generated SQL with `to_sql()`. This is useful for debugging.

```
qb = QueryBuilder.from_table("users", db) \
    .select("id", "name") \
    .where("active = ?", [1]) \
    .order_by("name ASC") \
    .limit(10)

print(qb.to_sql())
```

Output:

```
SELECT id, name FROM users WHERE active = ? ORDER BY name ASC
```

Note that `to_sql()` does not include `LIMIT` or `OFFSET` in the string. Those values are passed as arguments to `db.fetch()` at execution time. The SQL string shows everything else: columns, joins, conditions, grouping, having, and ordering.

`to_sql()` does not execute anything. It does not require a database connection. Use it freely for logging and debugging.

---

9. Execution Methods

Four methods execute the query against the database.

get() - Multiple rows

Returns a `DatabaseResult` object. Use `.records` for the list of dicts, `.to_list()` for a plain list, or iterate directly.

```
result = QueryBuilder.from_table("users", db) \
    .where("active = ?", [1]) \
    .get()

for row in result:
    print(row["name"])
```

first() - Single row

Returns a single dict, or `None` if no rows match.

```
user = QueryBuilder.from_table("users", db) \
```

```
    .where("email = ?", ["alice@example.com"]) \
    .first()

if user:
    print(user["name"])
```

count() - Row count

Returns an integer. Internally rewrites the select to `COUNT(*) as cnt` and reads the result.

```
total = QueryBuilder.from_table("orders", db) \
    .where("status = ?", ["pending"]) \
    .count()

print(f"{total} pending orders")
```

exists() - Boolean check

Returns `True` if at least one row matches, `False` otherwise. Calls `count()` under the hood.

```
if QueryBuilder.from_table("users", db) \
    .where("email = ?", ["alice@example.com"]) \
    .exists():
    print("User exists")
```

---

10. Chaining - Building Complex Queries

Every method returns `self`, so you can chain everything into a single expression. Here is a realistic example that combines most features.

```
result = QueryBuilder.from_table("orders", db) \
    .select(
        "orders.id",
        "users.name as customer",
        "products.name as product",
        "orders.quantity",
        "orders.total",
        "orders.created_at",
    ) \
    .join("users", "users.id = orders.user_id") \
    .join("products", "products.id = orders.product_id") \
    .where("orders.status = ?", ["completed"]) \
    .where("orders.created_at > ?", ["2025-01-01"]) \
    .order_by("orders.created_at DESC") \
    .limit(50) \
    .get()
```

You can also build queries conditionally. Since each method returns the same instance, store it in a variable and add clauses as needed.

```
@get("/api/products")
async def search_products(request, response):
    qb = QueryBuilder.from_table("products", db) \
        .select("id", "name", "price", "category")

    # Apply filters only if provided
    category = request.params.get("category")
```

```
    if category:
        qb.where("category = ?", [category])

    min_price = request.params.get("min_price")
    if min_price:
        qb.where("price >= ?", [float(min_price)])

    max_price = request.params.get("max_price")
    if max_price:
        qb.where("price <= ?", [float(max_price)])

    search = request.params.get("q")
    if search:
        qb.where("name LIKE ?", [f"%{search}%"])

    # Always sort and paginate
    qb.order_by("name ASC")
    qb.limit(20)

    return response(qb.get().to_list())
```

This is where QueryBuilder shines. Without it, you would be concatenating SQL fragments with `if` checks and tracking parameter positions manually.

---

11. Using with ORM Models

If your ORM models are bound to a database via `bind_database()`, QueryBuilder can use that connection automatically. You do not need to pass `db` explicitly.

```
from tina4_python.query_builder import QueryBuilder

# No db argument - uses the global ORM database
result = QueryBuilder.from_table("users") \
    .where("active = ?", [1]) \
    .get()
```

When you call `get()`, `first()`, `count()`, or `exists()` without a database connection, QueryBuilder checks the global ORM database. If it finds one, it uses it. If not, it raises `RuntimeError: QueryBuilder: No database connection provided.`

This means you can use QueryBuilder in the same files as your ORM models without importing or passing the database around.

When to use QueryBuilder vs ORM

Use the ORM when you are working with a single model: loading, saving, deleting records. The ORM gives you objects with attributes.

Use QueryBuilder when you need joins across tables, aggregations, complex filtering, or you want a `DatabaseResult` instead of model instances.

```
# ORM - single model operations
user = User.find(1)
user.name = "Alice"
user.save()
```

```
# QueryBuilder - cross-table query with joins
result = QueryBuilder.from_table("users", db) \
    .select("users.name", "COUNT(orders.id) as order_count") \
    .join("orders", "orders.user_id = users.id") \
    .group_by("users.name") \
    .having("COUNT(orders.id) > ?", [10]) \
    .order_by("order_count DESC") \
    .get()
```

NoSQL: MongoDB Queries

The QueryBuilder can generate MongoDB-compatible query documents with `to_mongo()`. This returns a dict containing the filter, projection, sort, limit, and skip -- ready to pass to PyMongo or any MongoDB driver.

Operator Mapping

SQL Operator	MongoDB Operator
`=`	Exact match
`!=`	`$ne`
`>`	`$gt`
`<`	`$lt`
`>=`	`$gte`
`<=`	`$lte`
`LIKE`	`$regex`
`IN`	`$in`
`IS NULL`	`$exists: false`
`IS NOT NULL`	`$exists: true`

Example

```
from tina4_python import QueryBuilder

query = (
    QueryBuilder.from_table("users")
    .select("name", "email")
    .where("age > ?", [25])
    .where("status = ?", ["active"])
    .order_by("name ASC")
    .limit(10)
    .offset(5)
)

mongo = query.to_mongo()
```

The returned dict:

```
{
    "filter": {"age": {"$gt": 25}, "status": "active"},
    "projection": {"name": 1, "email": 1},
    "sort": [("name", 1)],
    "limit": 10,
    "skip": 5,
}
```

Pass it directly to PyMongo:

```
collection = db["users"]
cursor = collection.find(
    mongo["filter"],
    mongo["projection"]
).sort(mongo["sort"]).limit(mongo["limit"]).skip(mongo["skip"])
```

Gotchas

1. "select() replaced my columns"

Cause: Each call to `select()` replaces the column list. It does not append.

Fix: Pass all columns in a single `select()` call.

```
# Wrong - only selects "email"
qb.select("id", "name").select("email")

# Right - selects all three
qb.select("id", "name", "email")
```

2. "My LIMIT is not in to_sql() output"

Cause: `to_sql()` builds the SQL string but does not include `LIMIT` or `OFFSET`. Those values are passed as separate arguments to `db.fetch()` when you call `get()`.

Fix: This is expected behaviour. If you need to see the full query for debugging, print both `to_sql()` and the limit/offset values.

3. "count() returns 0 but get() returns rows"

Cause: `count()` temporarily replaces the select columns with `COUNT(*) as cnt`. If you have a `GROUP BY` clause, the count query counts groups, not total rows. The first group's count is returned, which may be unexpected.

Fix: For simple row counts without grouping, `count()` works correctly. For grouped queries, use `get()` and check the result length instead.

4. "RuntimeError: No database connection provided"

Cause: You called an execution method (`get()`, `first()`, `count()`, `exists()`) without passing a database to `from_table()` and without having called `bind_database()`.

Fix: Either pass the database explicitly:

```
QueryBuilder.from_table("users", db).get()
```

Or ensure `bind_database(db)` has been called in your `app.py` before the query runs.

5. "or_where() on the first condition has no effect"

Cause: The first condition in the chain never gets a connector prefix (`AND` or `OR`). Whether you use `where()` or `or_where()` for the first condition, the result is the same.

Fix: This is by design. The connector only matters from the second condition onward.

6. "Parameters are in the wrong order"

Cause: `having()` parameters are stored separately from `where()` parameters. At execution time, they are merged as `where_params + having_params`. If you mix calls in an unusual order, the parameter positions might not match your expectations.

Fix: Add all `where()` conditions before `having()` conditions. This matches the natural SQL order and keeps parameters aligned.

Exercise: Product Search API

Build a product search endpoint that:

- Accepts optional query parameters: `category`, `min_price`, `max_price`, `sort` (column name), `order` (`asc` or `desc`), `page`, `per_page`.
- Returns matching products with their category name (joined from a `categories` table).
- Includes total count and pagination metadata in the response.
- Returns an empty list (not an error) when no products match.

Solution

```
# src/routes/products.py
from tina4_python.core.router import get
from tina4_python.query_builder import QueryBuilder

ALLOWED_SORT_COLUMNS = {"name", "price", "created_at"}

@get("/api/products")
async def search_products(request, response):
    page = max(int(request.params.get("page", 1)), 1)
    per_page = min(int(request.params.get("per_page", 20)), 100)
    offset = (page - 1) * per_page

    # Base query with join
    qb = QueryBuilder.from_table("products", db) \
        .select(
            "products.id",
            "products.name",
            "products.price",
            "categories.name as category",
            "products.created_at",
        ) \
        .left_join("categories", "categories.id = products.category_id")

    # Count query - same filters, no join needed for count
    count_qb = QueryBuilder.from_table("products", db)

    # Apply filters to both queries
    category = request.params.get("category")
    if category:
        qb.where("categories.name = ?", [category])
        count_qb.join("categories", "categories.id = products.category_id")
        count_qb.where("categories.name = ?", [category])

    min_price = request.params.get("min_price")
    if min_price:
        qb.where("products.price >= ?", [float(min_price)])
        count_qb.where("products.price >= ?", [float(min_price)])

    max_price = request.params.get("max_price")
    if max_price:
        qb.where("products.price <= ?", [float(max_price)])
        count_qb.where("products.price <= ?", [float(max_price)])

    # Sorting - validate column name to prevent injection
    sort_col = request.params.get("sort", "name")
    if sort_col not in ALLOWED_SORT_COLUMNS:
        sort_col = "name"

    sort_dir = request.params.get("order", "asc").upper()
    if sort_dir not in ("ASC", "DESC"):
        sort_dir = "ASC"

    qb.order_by(f"products.{sort_col} {sort_dir}")

    # Paginate
    qb.limit(per_page, offset)

    # Execute
```

```
result = qb.get()
total = count_qb.count()

return response({
    "products": result.to_list(),
    "total": total,
    "page": page,
    "per_page": per_page,
    "pages": (total + per_page - 1) // per_page,
})
```

The sort column is validated against a whitelist. The direction is constrained to `ASC` or `DESC`. User input never touches the SQL directly; it either goes through `?` placeholders or gets checked against known-safe values. QueryBuilder handles the assembly. The route handler stays readable.

Authentication

1. Locking the Door

Every endpoint built so far is public. Anyone with the URL can read, create, update, and delete data. Fine for a tutorial. Unacceptable for production.

A real application needs to know two things: who is making the request, and whether they are allowed to make it. This chapter covers Tina4's authentication system. JWT tokens. Password hashing. Middleware-based route protection. CSRF tokens for forms. Session management.

---

2. JWT Tokens

Tina4 uses JSON Web Tokens (JWT) for authentication. A JWT is a signed string carrying a payload -- user ID, role, expiry. The server mints the token at login. The client sends it with every request. The server verifies the signature without touching the database.

Generating a Token

```
from tina4_python.auth import get_token

payload = {
    "user_id": 42,
    "email": "alice@example.com",
    "role": "admin"
}

token = get_token(payload)
```

`get_token()` signs the payload with HS256 (HMAC-SHA256) and returns a JWT string like:

```
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJ1c2VyX2lkIjo0MiwiZW1haWwiOiJhbGljZUBleGFtcGxlLmNvbSIsInJ
```

The token has three parts separated by dots: header, payload, and signature. The signature ensures the token has not been tampered with.

Token Expiry

By default, tokens expire after 60 minutes. Configure this in `.env`:

```
TINA4_TOKEN_EXPIRES_IN=60
```

The value is in **minutes**. Common settings:

Value	Duration
`15`	15 minutes
`60`	1 hour (default)
`1440`	24 hours
`10080`	7 days

Validating a Token

```
payload = Auth.valid_token(token)
# Returns the payload dict if the token is valid, None if invalid or expired
```

`valid_token()` returns the decoded payload on success, not a boolean. This lets you validate and read the token in one step. Returns `None` if the token is invalid or expired.

Reading the Payload

```
payload = Auth.get_payload(token)
```

Returns the decoded payload dictionary **without validation** -- it just decodes the token:

```
{
    "user_id": 42,
    "email": "alice@example.com",
    "role": "admin",
    "iat": 1711112600,  # issued at (Unix timestamp)
    "exp": 1711116200   # expires at (Unix timestamp)
}
```

If the token cannot be decoded, `get_payload()` returns `None`.

> ***Important:*** `get_payload()` *does not verify the signature or check expiry. Use* `valid_token()` *when you need to confirm the token is trustworthy.*

The Secret Key and Algorithm

Tina4 Python uses **HS256** (HMAC-SHA256) for JWT signing. It uses only the standard library -- zero external dependencies.

Set the secret key in `.env`:

```
TINA4_SECRET=my-super-secret-key-at-least-32-chars
```

If no `TINA4_SECRET` is set, Tina4 falls back to generating a random key at `secrets/jwt.key` on first run. Setting `TINA4_SECRET` explicitly is recommended for production so all server instances share the same key.

Keep this key secret. If someone gets it, they can forge tokens.

---

3. Password Hashing

Plain-text passwords are a liability. Tina4 provides two functions for secure password handling:

Hashing a Password

```
from tina4_python.auth import Auth

hashed = Auth.hash_password("my-secure-password")
# Returns: "$2b$12$abc123...long-hash-string..."
```

Uses PBKDF2 from the standard library -- no external dependencies. Each hash includes a random salt, so hashing the same password twice produces different results.

Checking a Password

```
is_correct = Auth.check_password("my-secure-password", stored_hash)
# Returns True if the password matches the hash
```

Registration Example

```
from tina4_python.core.router import post, noauth
from tina4_python.auth import Auth
from tina4_python.database.connection import Database

@post("/api/register")
@noauth()
async def register(request, response):
    body = request.body

    # Validate input
    if not body.get("name") or not body.get("email") or not body.get("password"):
        return response.json({"error": "Name, email, and password are required"}, 400)

    if len(body["password"]) < 8:
        return response.json({"error": "Password must be at least 8 characters"}, 400)

    db = Database()

    # Check if email already exists
    existing = db.fetch_one("SELECT id FROM users WHERE email = ?", [body["email"]])
    if existing is not None:
        return response.json({"error": "Email already registered"}, 409)

    # Hash the password
    password_hash = Auth.hash_password(body["password"])

    # Create the user
    result = db.execute(
        "INSERT INTO users (name, email, password_hash) VALUES (?, ?, ?)",
        [body["name"], body["email"], password_hash]
    )

    user = db.fetch_one("SELECT id, name, email FROM users WHERE id = ?", [db.get_last_id()])

    return response.json({
        "message": "Registration successful",
        "user": user
    }, 201)
```

```
curl -X POST http://localhost:7146/api/register \
  -H "Content-Type: application/json" \
  -d '{"name": "Alice", "email": "alice@example.com", "password": "securePass123"}'
```

```
{
  "message": "Registration successful",
  "user": {"id": 1, "name": "Alice", "email": "alice@example.com"}
}
```

---

4. The Login Flow

The complete login flow. Client sends credentials. Server validates them. Server returns a JWT token.

```
from tina4_python.core.router import post, noauth
from tina4_python.auth import Auth, get_token
from tina4_python.database.connection import Database

@post("/api/login")
@noauth()
async def login(request, response):
    body = request.body

    if not body.get("email") or not body.get("password"):
        return response.json({"error": "Email and password are required"}, 400)

    db = Database()

    # Find the user
    user = db.fetch_one(
        "SELECT id, name, email, password_hash FROM users WHERE email = ?",
        [body["email"]]
    )

    if user is None:
        return response.json({"error": "Invalid email or password"}, 401)

    # Check the password
    if not Auth.check_password(body["password"], user["password_hash"]):
        return response.json({"error": "Invalid email or password"}, 401)

    # Generate a token
    token = get_token({
        "user_id": user["id"],
        "email": user["email"],
        "name": user["name"]
    })

    return response.json({
        "message": "Login successful",
        "token": token,
        "user": {
            "id": user["id"],
            "name": user["name"],
            "email": user["email"]
        }
    })
```

Notice `@noauth`. The login endpoint must be public. You cannot require a token to get a token.

```
curl -X POST http://localhost:7146/api/login \
  -H "Content-Type: application/json" \
  -d '{"email": "alice@example.com", "password": "securePass123"}'
```

```
{
  "message": "Login successful",
  "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...",
  "user": {
```

```
    "id": 1,
    "name": "Alice",
    "email": "alice@example.com"
  }
}
```

The client stores this token (in localStorage, a cookie, or memory) and sends it with subsequent requests.

---

5. Using Tokens in Requests

The client sends the token in the `Authorization` header with the `Bearer` prefix:

```
curl http://localhost:7146/api/profile \
  -H "Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9..."
```

---

6. Protecting Routes

Auth Middleware

Create a reusable auth middleware:

```
from tina4_python.auth import Auth

async def auth_middleware(request, response, next_handler):
    auth_header = request.headers.get("Authorization", "")

    if not auth_header or not auth_header.startswith("Bearer "):
        return response.json({"error": "Authorization header required"}, 401)

    token = auth_header[7:]  # Remove "Bearer " prefix

    payload = Auth.valid_token(token)
    if payload is None:
        return response.json({"error": "Invalid or expired token"}, 401)

    request.user = payload  # Attach user data to the request

    return await next_handler(request, response)
```

Applying Middleware to Routes

```
from tina4_python.core.router import get, middleware

@get("/api/profile")
@middleware(auth_middleware)
async def profile(request, response):
    return response.json({
        "user_id": request.user["user_id"],
        "email": request.user["email"],
        "name": request.user["name"]
    })

# Without token -- 401
curl http://localhost:7146/api/profile
```

```
{"error":"Authorization header required"}
```

```
# With valid token -- 200
curl http://localhost:7146/api/profile \
  -H "Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9..."
```

```
{
  "user_id": 1,
  "email": "alice@example.com",
  "name": "Alice"
}
```

Applying Middleware to a Group

```
from tina4_python.core.router import get, group, middleware

@group("/api/admin")
@middleware(auth_middleware)
def admin_routes():

    @get("/stats")
    async def admin_stats(request, response):
        return response.json({"active_users": 42})

    @get("/logs")
    async def admin_logs(request, response):
        return response.json({"logs": []})
```

Every route in the group requires a valid token.

---

7. @noauth and @secured Decorators

As introduced in Chapter 2, these decorators control authentication at the route level.

@noauth -- Skip Authentication

Use `@noauth` for public endpoints that should bypass any global or group-level auth:

```
from tina4_python.core.router import get, post, noauth

@get("/api/public/health")
@noauth()
async def health(request, response):
    return response.json({"status": "ok"})

@post("/api/login")
@noauth()
async def login(request, response):
    # Login logic
    pass

@post("/api/register")
@noauth()
async def register(request, response):
    # Registration logic
    pass
```

@secured -- Require Authentication for GET Routes

By default, POST, PUT, PATCH, and DELETE routes are considered secured. GET routes are public. Use `@secured` to explicitly protect a GET route:

```
from tina4_python.core.router import get, secured

@get("/api/me")
@secured()
async def me(request, response):
    # This GET route requires authentication
    return response.json(request.user)
```

Role-Based Authorization

Combine auth middleware with role checks:

```
from tina4_python.auth import Auth

def require_role(role):
    async def role_middleware(request, response, next_handler):
        auth_header = request.headers.get("Authorization", "")
        if not auth_header or not auth_header.startswith("Bearer "):
            return response.json({"error": "Authorization required"}, 401)

        token = auth_header[7:]
        payload = Auth.valid_token(token)
        if payload is None:
            return response.json({"error": "Invalid or expired token"}, 401)

        if payload.get("role") != role:
            return response.json({"error": f"Forbidden. Required role: {role}"}, 403)

        request.user = payload
        return await next_handler(request, response)

    return role_middleware
```

Use it as middleware:

```
from tina4_python.core.router import delete as delete_route, middleware

@delete_route("/api/users/{id:int}")
@middleware(require_role("admin"))
async def delete_user(request, response):
    # Only admins can delete users
    return response.json({"deleted": True})
```

---

8. CSRF Protection

Traditional form-based applications (not SPAs) need CSRF protection. Tina4 provides it through form tokens.

Generating a Token

In your template, include the CSRF token in every form:

```
<form method="POST" action="/profile/update">
    {{ form_token() }}

    <div class="form-group">
        <label for="name">Name</label>
        <input type="text" name="name" id="name" value="{{ user.name }}">
    </div>

    <button type="submit">Update Profile</button>
</form>
```

`{{ form_token() }}` renders a hidden input field:

```
<input type="hidden" name="_token" value="abc123randomtoken456">
```

Validating the Token

In your route handler, check the token:

```
from tina4_python.core.router import post
from tina4_python.auth import Auth

@post("/profile/update")
async def update_profile(request, response):
    # Validate CSRF token
    if not Auth.validate_form_token(request.body.get("_token", "")):
        return response.json({"error": "Invalid form token. Please refresh and try again."}, 403

    # Process the form...
    return response.redirect("/profile")
```

The CSRF token is tied to the session and expires after a single use. A malicious site cannot forge a form submission because it cannot guess the token.

> ***Note (3.10.9):*** *Form token validation internally uses* `Auth.valid_token_static()`*, a classmethod that does not require an* `Auth` *instance. Earlier versions incorrectly called the instance method, which could fail when no request context was available. If you validate form tokens manually, prefer* `Auth.valid_token_static(token)` *for reliability.*

When to Use CSRF Tokens

Use CSRF tokens for:

- HTML forms submitted by browsers
- Any POST/PUT/DELETE from server-rendered pages

You do not need CSRF tokens for:

- API endpoints that use JWT (the Bearer token already proves the request is intentional)
- Single-page applications that use `fetch()` with custom headers

---

9. Sessions

Tina4 supports server-side sessions for storing per-user state between requests. JWTs handle API authentication. Sessions handle stateful web pages. Both work side by side.

Session Configuration

Set the session backend in `.env`:

```
# File-based sessions (default)
TINA4_SESSION_BACKEND=file

# Redis
TINA4_SESSION_BACKEND=redis
TINA4_SESSION_REDIS_HOST=localhost
TINA4_SESSION_REDIS_PORT=6379

# MongoDB
TINA4_SESSION_BACKEND=mongodb
TINA4_SESSION_REDIS_HOST=localhost
TINA4_SESSION_REDIS_PORT=27017

# Valkey
TINA4_SESSION_BACKEND=valkey
TINA4_SESSION_REDIS_HOST=localhost
TINA4_SESSION_REDIS_PORT=6379
```

File-based sessions work out of the box. No extra dependencies. For production deployments with multiple servers, use Redis or Valkey so sessions are shared across instances.

Using Sessions

Access session data via `request.session`:

```
from tina4_python.core.router import get, post

@post("/login-form")
async def login_form(request, response):
    # After validating credentials...
    request.session["user_id"] = 42
    request.session["user_name"] = "Alice"
    request.session["logged_in"] = True

    return response.redirect("/dashboard")

@get("/dashboard")
async def dashboard(request, response):
    if not request.session.get("logged_in"):
        return response.redirect("/login")

    return response.render("dashboard.html", {
        "user_name": request.session["user_name"]
    })

@post("/logout")
async def logout(request, response):
    # Clear all session data
    request.session.clear()
```

```
    return response.redirect("/login")
```

Session Options

```
TINA4_SESSION_TTL=3600       # Session lifetime in seconds (default: 3600)
TINA4_SESSION_NAME=tina4_session  # Cookie name for the session ID
```

---

10. Exercise: Build Login, Register, and Profile

Build a complete authentication system with registration, login, profile viewing, and password changing.

Requirements

- Create a `users` table migration with: `id`, `name`, `email` (unique), `password_hash`, `role` (default "user"), `created_at`
- Build these endpoints:

Method	Path	Auth	Description
`POST`	`/api/register`	@noauth	Create an account. Validate name, email, password (min 8 chars).
`POST`	`/api/login`	@noauth	Login. Return JWT token.
`GET`	`/api/profile`	secured	Get current user's profile from token.
`PUT`	`/api/profile`	secured	Update name and email.
`PUT`	`/api/profile/password`	secured	Change password. Require current password.

- Create auth middleware that extracts the user from the JWT and attaches it to `request.user`.

Test with:

```
# Register
curl -X POST http://localhost:7146/api/register \
  -H "Content-Type: application/json" \
  -d '{"name": "Alice", "email": "alice@example.com", "password": "securePass123"}'

# Login
curl -X POST http://localhost:7146/api/login \
  -H "Content-Type: application/json" \
  -d '{"email": "alice@example.com", "password": "securePass123"}'

# Save the token from login response, then:

# Get profile
curl http://localhost:7146/api/profile \
  -H "Authorization: Bearer YOUR_TOKEN_HERE"

# Update profile
curl -X PUT http://localhost:7146/api/profile \
  -H "Authorization: Bearer YOUR_TOKEN_HERE" \
  -H "Content-Type: application/json" \
  -d '{"name": "Alice Smith"}'

# Change password
curl -X PUT http://localhost:7146/api/profile/password \
  -H "Authorization: Bearer YOUR_TOKEN_HERE" \
  -H "Content-Type: application/json" \
  -d '{"current_password": "securePass123", "new_password": "evenMoreSecure456"}'

# Try with no token (should fail)
curl http://localhost:7146/api/profile
```

---

11. Solution

Migration

Create `migrations/20260322160000_create_users_table.sql`:

```
-- UP
CREATE TABLE users (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    name TEXT NOT NULL,
    email TEXT NOT NULL UNIQUE,
    password_hash TEXT NOT NULL,
    role TEXT NOT NULL DEFAULT 'user',
    created_at TEXT DEFAULT CURRENT_TIMESTAMP
);

-- DOWN
DROP TABLE IF EXISTS users;
```

```
tina4 migrate
```

Routes

Create `src/routes/auth.py`: _____

```
from tina4_python.core.router import get, post, put, noauth, middleware
from tina4_python.auth import Auth, get_token
from tina4_python.database.connection import Database


async def auth_middleware(request, response, next_handler):
    auth_header = request.headers.get("Authorization", "")

    if not auth_header or not auth_header.startswith("Bearer "):
        return response.json({"error": "Authorization required. Send: Authorization: Bearer <tok

    token = auth_header[7:]

    payload = Auth.valid_token(token)
    if payload is None:
        return response.json({"error": "Invalid or expired token. Please login again."}, 401)

    request.user = payload

    return await next_handler(request, response)


@post("/api/register")
@noauth()
async def register(request, response):
    body = request.body

    errors = []
    if not body.get("name"):
        errors.append("Name is required")
    if not body.get("email"):
        errors.append("Email is required")
    if not body.get("password"):
        errors.append("Password is required")
    elif len(body["password"]) < 8:
        errors.append("Password must be at least 8 characters")

    if errors:
        return response.json({"errors": errors}, 400)

    db = Database()

    existing = db.fetch_one("SELECT id FROM users WHERE email = ?", [body["email"]])
    if existing is not None:
        return response.json({"error": "Email already registered"}, 409)

    password_hash = Auth.hash_password(body["password"])

    result = db.execute(
        "INSERT INTO users (name, email, password_hash) VALUES (?, ?, ?)",
        [body["name"], body["email"], password_hash]
    )

    user = db.fetch_one("SELECT id, name, email, role, created_at FROM users WHERE id = ?", [db.

    return response.json({"message": "Registration successful", "user": user}, 201)


@post("/api/login")
```

```
@noauth()
async def login(request, response):
    body = request.body

    if not body.get("email") or not body.get("password"):
        return response.json({"error": "Email and password are required"}, 400)

    db = Database()

    user = db.fetch_one(
        "SELECT id, name, email, password_hash, role FROM users WHERE email = ?",
        [body["email"]]
    )

    if user is None or not Auth.check_password(body["password"], user["password_hash"]):
        return response.json({"error": "Invalid email or password"}, 401)

    token = get_token({
        "user_id": user["id"],
        "email": user["email"],
        "name": user["name"],
        "role": user["role"]
    })

    return response.json({
        "message": "Login successful",
        "token": token,
        "user": {
            "id": user["id"],
            "name": user["name"],
            "email": user["email"],
            "role": user["role"]
        }
    })


@get("/api/profile")
@middleware(auth_middleware)
async def get_profile(request, response):
    db = Database()

    user = db.fetch_one(
        "SELECT id, name, email, role, created_at FROM users WHERE id = ?",
        [request.user["user_id"]]
    )

    if user is None:
        return response.json({"error": "User not found"}, 404)

    return response.json(user)


@put("/api/profile")
@middleware(auth_middleware)
async def update_profile(request, response):
    db = Database()
    body = request.body
    user_id = request.user["user_id"]
```

```
    if body.get("email"):
        existing = db.fetch_one(
            "SELECT id FROM users WHERE email = ? AND id != ?",
            [body["email"], user_id]
        )
        if existing is not None:
            return response.json({"error": "Email already in use by another account"}, 409)

    current = db.fetch_one("SELECT * FROM users WHERE id = ?", [user_id])

    db.execute(
        "UPDATE users SET name = ?, email = ? WHERE id = ?",
        [body.get("name", current["name"]), body.get("email", current["email"]), user_id]
    )

    updated = db.fetch_one(
        "SELECT id, name, email, role, created_at FROM users WHERE id = ?",
        [user_id]
    )

    return response.json({"message": "Profile updated", "user": updated})


@put("/api/profile/password")
@middleware(auth_middleware)
async def change_password(request, response):
    db = Database()
    body = request.body
    user_id = request.user["user_id"]

    if not body.get("current_password") or not body.get("new_password"):
        return response.json({"error": "Current password and new password are required"}, 400)

    if len(body["new_password"]) < 8:
        return response.json({"error": "New password must be at least 8 characters"}, 400)

    user = db.fetch_one("SELECT password_hash FROM users WHERE id = ?", [user_id])

    if not Auth.check_password(body["current_password"], user["password_hash"]):
        return response.json({"error": "Current password is incorrect"}, 401)

    new_hash = Auth.hash_password(body["new_password"])

    db.execute(
        "UPDATE users SET password_hash = ? WHERE id = ?",
        [new_hash, user_id]
    )

    return response.json({"message": "Password changed successfully"})
```

Expected output for register:

```
{
  "message": "Registration successful",
  "user": {
    "id": 1,
    "name": "Alice",
    "email": "alice@example.com",
    "role": "user",
```

```
    "created_at": "2026-03-22 16:00:00"
  }
}
```

(Status: `201 Created`)

Expected output for profile without token:

```
{"error":"Authorization required. Send: Authorization: Bearer <token>"}
```

(Status: `401 Unauthorized`)

---

12. Gotchas

1. Token expiry confusion

Problem: Tokens that worked yesterday now return 401.

Cause: The default token lifetime is 60 minutes (`TINA4_TOKEN_EXPIRES_IN=60`). After that, the token is invalid even if the signature is correct.

Fix: Issue a new token at login. If your application needs long-lived sessions, use refresh tokens: a short-lived access token (15 minutes) paired with a long-lived refresh token (7 days) that can be used to get a new access token without re-entering credentials.

2. Secret key management

Problem: Tokens generated on one server are invalid on another, or tokens stop working after a deployment.

Cause: Each server generated its own random `secrets/jwt.key` file. Or the key file was deleted/regenerated during deployment.

Fix: Set `TINA4_SECRET` in `.env` explicitly and use the same value across all servers. Store it in your deployment secrets manager (not in version control). If the key changes, all existing tokens become invalid and users must log in again.

3. CORS with authentication

Problem: Frontend requests with the `Authorization` header fail with a CORS error, even though `TINA4_CORS_ORIGINS=*` is set.

Cause: When the browser sends an `Authorization` header, it first sends a preflight `OPTIONS` request. The server must respond to the OPTIONS request with the correct CORS headers, including `Access-Control-Allow-Headers: Authorization`.

Fix: Tina4 handles preflight requests automatically. Make sure `TINA4_CORS_ORIGINS` is set correctly. If it is still failing, check that you are not overriding CORS headers in middleware.

4. Storing tokens in localStorage

Problem: Your token is stolen via an XSS attack because it was stored in `localStorage`.

Cause: Any JavaScript on the page can read `localStorage`, including injected scripts from an XSS vulnerability.

Fix: Store tokens in `httpOnly` cookies when possible -- they cannot be accessed by JavaScript. For SPAs that must use `localStorage`, implement strict Content Security Policy headers and sanitize all user input.

5. Forgetting @noauth on login

Problem: Your login endpoint returns 401 -- you cannot log in because the endpoint requires authentication.

Cause: If you have global auth middleware, the login endpoint needs the `@noauth` decorator to bypass it.

Fix: Add `@noauth` to your login and register routes. Without it, users cannot authenticate because authentication is required to authenticate -- a catch-22.

6. Password hash column too short

Problem: Registration fails with a database error about the password hash being too long.

Cause: PBKDF2 hashes can be long. If your `password_hash` column is defined as `VARCHAR(50)`, it gets truncated.

Fix: Use `TEXT` for the password hash column, or at minimum `VARCHAR(255)`. Never constrain the hash length.

7. Token in URL query parameters

Problem: Tokens in URLs like `/api/profile?token=eyJ...` leak through browser history, server logs, and the Referer header.

Cause: Query parameters are visible in many places where headers are not.

Fix: Always send tokens in the `Authorization` header, never in the URL. The only exception is WebSocket connections, where the initial HTTP upgrade request cannot carry custom headers -- use a short-lived token for that case.

Sessions and Cookies

1. Remembering Your Users

JWT tokens handle APIs. Traditional web applications need more. A shopping cart that persists across pages. A flash message that appears once after a redirect. A "remember me" checkbox on the login form. These features run on sessions and cookies -- server-side state tied to a browser.

Picture an e-commerce site. A customer adds three items to their cart. Navigates to a product page. Comes back to the cart. Without sessions, the cart is empty every time. Sessions give the server memory. Each browser gets its own state, preserved across requests.

Sessions are auto-started. Every route handler receives `request.session` ready to use. No manual setup. No configuration required for the default file backend.

---

2. Session Configuration

The default backend is file-based sessions. They work out of the box with no additional dependencies.

To change the backend, set `TINA4_SESSION_BACKEND` in `.env`:

```
TINA4_SESSION_BACKEND=file
```

Available Backends

Backend	Value	Package Required	Best For
File	`file`	None	Development, single server
Redis	`redis`	`redis`	Production, multi-server
MongoDB	`mongodb`	`pymongo`	Production, document stores
Valkey	`valkey`	`valkey`	Production, Redis alternative
Database	`database`	None	Production, using existing DB

Redis Configuration

```
TINA4_SESSION_BACKEND=redis
TINA4_SESSION_REDIS_HOST=localhost
TINA4_SESSION_REDIS_PORT=6379
TINA4_SESSION_REDIS_PASSWORD=
```

Install the Redis driver:

```
uv add redis
```

MongoDB Configuration

```
TINA4_SESSION_BACKEND=mongodb
TINA4_SESSION_REDIS_HOST=localhost
TINA4_SESSION_REDIS_PORT=27017
TINA4_SESSION_MONGO_DB=tina4_sessions
```

Valkey Configuration

```
TINA4_SESSION_BACKEND=valkey
TINA4_SESSION_REDIS_HOST=localhost
TINA4_SESSION_REDIS_PORT=6379
```

Database Sessions

```
TINA4_SESSION_BACKEND=database
```

Stores sessions in the `tina4_session` table using your existing database connection (`TINA4_DATABASE_URL`). The table is auto-created on first use.

Session Lifetime

```
TINA4_SESSION_TTL=3600  # 1 hour in seconds (default)
```

Session Cookie

The session cookie is named `tina4_session`. It is `HttpOnly` and `SameSite=Lax` by default. Tina4 manages it automatically -- you never need to set or read it yourself.

---

3. The Session API

Access session data through `request.session`. It is available in every route handler with zero configuration.

Full API Reference

Method	Description
`request.session.set(key, value)`	Store a value
`request.session.get(key, default)`	Retrieve a value (with optional default)
`request.session.delete(key)`	Remove a key
`request.session.has(key)`	Check if a key exists
`request.session.clear()`	Remove all session data
`request.session.destroy()`	Destroy the session entirely
`request.session.save()`	Persist session data (auto-called after response)
`request.session.regenerate()`	Generate a new session ID, preserve data
`request.session.flash(key, value)`	Set flash data (one-time read)
`request.session.flash(key)`	Read and remove flash data
`request.session.all()`	Get all session data as a dictionary

---

4. Reading and Writing Session Data

Writing to the Session

```
from tina4_python.core.router import post

@post("/login-form")
async def login_form(request, response):
    # After validating credentials...
    request.session.set("user_id", 42)
    request.session.set("user_name", "Alice")
    request.session.set("role", "admin")
    request.session.set("logged_in", True)

    return response.redirect("/dashboard")
```

Reading from the Session

```
from tina4_python.core.router import get

@get("/dashboard")
async def dashboard(request, response):
    if not request.session.get("logged_in"):
        return response.redirect("/login")

    return response.render("dashboard.html", {
        "user_name": request.session.get("user_name"),
        "role": request.session.get("role")
    })
```

Deleting Session Data

```
# Delete a single key
request.session.delete("temp_data")

# Clear all session data (logout)
@post("/logout")
async def logout(request, response):
    request.session.clear()
    return response.redirect("/login")
```

Checking if a Key Exists

```
if request.session.has("user_id"):
    user_id = request.session.get("user_id")
else:
    user_id = None

# Or use .get() with a default
user_id = request.session.get("user_id", None)
```

Getting All Session Data

```
all_data = request.session.all()
```

---

5. Flash Messages

Flash messages are session values that exist for exactly one read. Set them before a redirect. Read them on the next request. They vanish automatically after being read.

Setting a Flash Message

```
@post("/api/contact")
async def submit_contact(request, response):
    # Process the form...

    request.session.flash("message", "Your message has been sent!")
    request.session.flash("message_type", "success")

    return response.redirect("/contact")
```

Reading a Flash Message

```
@get("/contact")
async def contact_page(request, response):
    flash_message = request.session.flash("message")
    flash_type = request.session.flash("message_type") or "info"

    return response.render("contact.html", {
        "flash_message": flash_message,
        "flash_type": flash_type
    })
```

Calling `request.session.flash(key)` with only a key reads the value and removes it in one step. The next time the user visits `/contact`, the flash message will be gone.

Flash Messages in Templates

```
{% if flash_message %}
    <div class="alert alert-{{ flash_type }}">
        {{ flash_message }}
    </div>
{% endif %}
```

---

6. Cookies

Cookies are small data fragments stored in the browser. Unlike sessions, cookies travel with every request and are visible to JavaScript (unless `httpOnly` is set).

Setting a Cookie

```
from tina4_python.core.router import post

@post("/preferences")
async def save_preferences(request, response):
    theme = request.body.get("theme", "light")
    language = request.body.get("language", "en")

    return response.cookie("theme", theme, {
        "max_age": 365 * 24 * 60 * 60,  # 1 year
        "path": "/",
        "samesite": "Lax"
    }).cookie("language", language, {
        "max_age": 365 * 24 * 60 * 60,
        "path": "/",
        "samesite": "Lax"
    }).json({"message": "Preferences saved"})
```

Reading a Cookie

```
from tina4_python.core.router import get

@get("/")
async def home(request, response):
    theme = request.cookies.get("theme", "light")
    language = request.cookies.get("language", "en")

    return response.render("home.html", {
        "theme": theme,
        "language": language
    })
```

Deleting a Cookie

Set `max_age` to 0:

```
@post("/clear-preferences")
async def clear_preferences(request, response):
    return response.cookie("theme", "", {"max_age": 0, "path": "/"}) \
                   .cookie("language", "", {"max_age": 0, "path": "/"}) \
                   .json({"message": "Preferences cleared"})
```

Cookie Options

Option	Type	Description
`max_age`	int	Lifetime in seconds. 0 = delete. Omit = session cookie (deleted when browser closes).
`path`	str	URL path scope. `/` means the whole site. `/admin` means only under `/admin`.
`domain`	str	Domain scope. `.example.com` includes subdomains.
`secure`	bool	Only send over HTTPS. Always `True` in production.
`httponly`	bool	Not accessible via JavaScript. Use for session cookies.
`samesite`	str	`"Strict"`, `"Lax"`, or `"None"`. Controls cross-site sending.

When to Use Cookies vs Sessions

Use Case	Use Cookies	Use Sessions
User preferences (theme, language)	Yes	No
Shopping cart	No	Yes
Authentication state	No (use JWT in header)	Yes (for form-based auth)
Flash messages	No	Yes
Tracking consent	Yes	No
Sensitive data (user ID, role)	No	Yes

The rule: sensitive data or data the client should not see goes in sessions. Non-sensitive data that should persist beyond session expiry goes in cookies.

---

7. Remember Me

The "remember me" pattern extends the session lifetime when the user checks a box on the login form:

```
from tina4_python.core.router import post
from tina4_python.auth import Auth, get_token
from tina4_python.database.connection import Database

@post("/login-form")
async def login_form(request, response):
    body = request.body
    db = Database()

    user = db.fetch_one(
        "SELECT id, name, email, password_hash FROM users WHERE email = ?",
        [body.get("email")]
    )

    if user is None or not Auth.check_password(body.get("password", ""), user["password_hash"]):
        request.session.flash("message", "Invalid email or password")
        request.session.flash("message_type", "error")
        return response.redirect("/login")

    # Set session data
    request.session.set("user_id", user["id"])
    request.session.set("user_name", user["name"])
    request.session.set("logged_in", True)

    # Handle "remember me"
    remember = body.get("remember_me") == "on"

    if remember:
        # Generate a long-lived token
        remember_token = get_token({"user_id": user["id"]})

        # Store it in a cookie that lasts 30 days
        return response.cookie("remember_token", remember_token, {
            "max_age": 30 * 24 * 60 * 60,
            "httponly": True,
            "secure": True,
            "path": "/",
            "samesite": "Lax"
        }).redirect("/dashboard")

    return response.redirect("/dashboard")
```

Then, on every page load, check for the remember token if the session has expired:

```
async def remember_me_middleware(request, response, next_handler):
    if not request.session.get("logged_in"):
        remember_token = request.cookies.get("remember_token")

        if remember_token and Auth.valid_token(remember_token):
            payload = Auth.get_payload(remember_token)
            db = Database()
            user = db.fetch_one(
                "SELECT id, name FROM users WHERE id = ?",
                [payload["user_id"]]
```

```
            )

            if user:
                request.session.set("user_id", user["id"])
                request.session.set("user_name", user["name"])
                request.session.set("logged_in", True)

    return await next_handler(request, response)
```

---

8. Session Security

Regenerate Session ID After Login

To prevent session fixation attacks, regenerate the session ID after login:

```
@post("/login-form")
async def login_form(request, response):
    # After validating credentials...

    # Regenerate session ID (prevents session fixation)
    request.session.regenerate()

    request.session.set("user_id", user["id"])
    request.session.set("logged_in", True)

    return response.redirect("/dashboard")
```

Destroy a Session

To completely destroy a session (not just clear its data):

```
@post("/logout")
async def logout(request, response):
    request.session.destroy()
    return response.redirect("/login")
```

Secure Cookie Settings

The session cookie (`tina4_session`) is `HttpOnly` and `SameSite=Lax` by default. For production, ensure HTTPS:

```
TINA4_SESSION_SECURE=true     # Only send over HTTPS
TINA4_SESSION_HTTPONLY=true   # Not accessible via JavaScript (default)
TINA4_SESSION_SAMESITE=Lax   # Prevent CSRF via cross-site requests (default)
```

`TINA4_SESSION_SAMESITE` controls cross-site cookie behavior:

Value	Behavior
`Strict`	Never sent with cross-site requests. Safest. Breaks some flows (clicking links from email).
`Lax`	Sent with top-level navigations (clicking links) but not cross-site API calls. Good default.

`None`	Always sent. Requires `TINA4_SESSION_SECURE=true`. Only for cross-site cookie access.

---

9. Exercise: Build a Shopping Cart

Build a shopping cart using sessions.

Requirements

- Create these endpoints:

Method	Path	Description
`GET`	`/cart`	View cart contents (rendered HTML page)
`POST`	`/cart/add`	Add an item to the cart
`POST`	`/cart/update`	Update item quantity
`POST`	`/cart/remove`	Remove an item from the cart
`POST`	`/cart/clear`	Clear the entire cart
`GET`	`/cart/api`	Get cart as JSON (for AJAX)

- The cart is stored in the session via `request.session.set("cart", [...])`
- Each cart item has: `product_id`, `name`, `price`, `quantity`
- Adding an existing product increments its quantity
- The cart page shows items, quantities, line totals, and a grand total
- Use flash messages for feedback ("Item added", "Cart cleared", etc.)

Use this product data for validation:

```
PRODUCTS = {
    1: {"name": "Wireless Keyboard", "price": 79.99},
    2: {"name": "USB-C Hub", "price": 49.99},
    3: {"name": "Monitor Stand", "price": 129.99},
    4: {"name": "Mechanical Mouse", "price": 59.99},
    5: {"name": "Desk Lamp", "price": 39.99}
}
```

Test with:

```
# Add items
curl -X POST http://localhost:7146/cart/add \
  -H "Content-Type: application/json" \
  -d '{"product_id": 1, "quantity": 2}' \
  -c cookies.txt -b cookies.txt

curl -X POST http://localhost:7146/cart/add \
  -H "Content-Type: application/json" \
  -d '{"product_id": 3, "quantity": 1}' \
  -c cookies.txt -b cookies.txt

# View cart
curl http://localhost:7146/cart/api -b cookies.txt

# Update quantity
curl -X POST http://localhost:7146/cart/update \
  -H "Content-Type: application/json" \
  -d '{"product_id": 1, "quantity": 3}' \
  -c cookies.txt -b cookies.txt

# Remove item
curl -X POST http://localhost:7146/cart/remove \
  -H "Content-Type: application/json" \
  -d '{"product_id": 3}' \
  -c cookies.txt -b cookies.txt

# Clear cart
curl -X POST http://localhost:7146/cart/clear -c cookies.txt -b cookies.txt
```

---

10. Solution

Create `src/routes/cart.py`:

```
from tina4_python.core.router import get, post

PRODUCTS = {
    1: {"name": "Wireless Keyboard", "price": 79.99},
    2: {"name": "USB-C Hub", "price": 49.99},
    3: {"name": "Monitor Stand", "price": 129.99},
    4: {"name": "Mechanical Mouse", "price": 59.99},
    5: {"name": "Desk Lamp", "price": 39.99}
}


def get_cart(session):
    return session.get("cart", [])


def save_cart(session, cart):
    session.set("cart", cart)


def calculate_totals(cart):
    for item in cart:
        item["line_total"] = round(item["price"] * item["quantity"], 2)
```

```
    grand_total = round(sum(item["line_total"] for item in cart), 2)
    return cart, grand_total


@get("/cart")
async def view_cart(request, response):
    cart = get_cart(request.session)
    cart, grand_total = calculate_totals(cart)
    flash_message = request.session.flash("message")
    flash_type = request.session.flash("message_type") or "info"

    return response.render("cart.html", {
        "cart": cart,
        "grand_total": grand_total,
        "item_count": sum(item["quantity"] for item in cart),
        "flash_message": flash_message,
        "flash_type": flash_type
    })


@get("/cart/api")
async def cart_api(request, response):
    cart = get_cart(request.session)
    cart, grand_total = calculate_totals(cart)

    return response.json({
        "cart": cart,
        "grand_total": grand_total,
        "item_count": sum(item["quantity"] for item in cart)
    })


@post("/cart/add")
async def add_to_cart(request, response):
    body = request.body
    product_id = body.get("product_id")
    quantity = int(body.get("quantity", 1))

    if product_id not in PRODUCTS:
        return response.json({"error": "Product not found"}, 404)

    if quantity < 1:
        return response.json({"error": "Quantity must be at least 1"}, 400)

    product = PRODUCTS[product_id]
    cart = get_cart(request.session)

    # Check if product already in cart
    for item in cart:
        if item["product_id"] == product_id:
            item["quantity"] += quantity
            save_cart(request.session, cart)
            request.session.flash("message", f"Updated {product['name']} quantity")
            request.session.flash("message_type", "success")
            return response.json({"message": f"Updated {product['name']} quantity", "cart": cart

    # Add new item
    cart.append({
        "product_id": product_id,
```

```
        "name": product["name"],
        "price": product["price"],
        "quantity": quantity
    })
    save_cart(request.session, cart)

    request.session.flash("message", f"Added {product['name']} to cart")
    request.session.flash("message_type", "success")

    return response.json({"message": f"Added {product['name']}", "cart": cart}, 201)


@post("/cart/update")
async def update_cart(request, response):
    body = request.body
    product_id = body.get("product_id")
    quantity = int(body.get("quantity", 1))

    cart = get_cart(request.session)

    for item in cart:
        if item["product_id"] == product_id:
            if quantity < 1:
                cart.remove(item)
                request.session.flash("message", f"Removed {item['name']} from cart")
            else:
                item["quantity"] = quantity
                request.session.flash("message", f"Updated {item['name']} quantity to {quantity}
            request.session.flash("message_type", "success")
            save_cart(request.session, cart)
            return response.json({"message": "Cart updated", "cart": cart})

    return response.json({"error": "Product not in cart"}, 404)


@post("/cart/remove")
async def remove_from_cart(request, response):
    product_id = request.body.get("product_id")
    cart = get_cart(request.session)

    for item in cart:
        if item["product_id"] == product_id:
            cart.remove(item)
            save_cart(request.session, cart)
            request.session.flash("message", f"Removed {item['name']} from cart")
            request.session.flash("message_type", "success")
            return response.json({"message": f"Removed {item['name']}", "cart": cart})

    return response.json({"error": "Product not in cart"}, 404)


@post("/cart/clear")
async def clear_cart(request, response):
    save_cart(request.session, [])
    request.session.flash("message", "Cart cleared")
    request.session.flash("message_type", "info")
    return response.json({"message": "Cart cleared", "cart": []})
```

Expected output for cart API after adding items:

```
{
  "cart": [
    {"product_id": 1, "name": "Wireless Keyboard", "price": 79.99, "quantity": 2, "line_total":
    {"product_id": 3, "name": "Monitor Stand", "price": 129.99, "quantity": 1, "line_total": 129
  ],
  "grand_total": 289.97,
  "item_count": 3
}
```

---

11. Gotchas

1. Session lost between requests

Problem: Data you stored in the session is gone on the next request.

Cause: The session cookie is not being sent back by the client. This happens when using curl without `-c cookies.txt -b cookies.txt`, or when the browser blocks cookies.

Fix: For curl testing, use `-c cookies.txt -b cookies.txt` to save and send cookies. In the browser, make sure cookies are not blocked for your site.

2. Session data is not JSON-serializable

Problem: Storing a Python object (like a datetime or a custom class) in the session causes an error.

Cause: Sessions are serialized to JSON (or pickled). Complex Python objects cannot be serialized directly.

Fix: Convert objects to strings or dictionaries before storing: `request.session.set("login_time", datetime.now().isoformat())`.

3. Cookie not sent on cross-origin requests

Problem: Your frontend at `http://localhost:3000` calls your API at `http://localhost:7146`, but cookies are not included.

Cause: Browsers do not send cookies cross-origin by default. You need both CORS configuration and explicit `credentials: "include"` in fetch.

Fix: Set `CORS_CREDENTIALS=true` in `.env` and use `fetch(url, { credentials: "include" })` in JavaScript. Also set `TINA4_CORS_ORIGINS` to the specific frontend origin (not `*` -- wildcard does not work with credentials).

4. File-based sessions on multi-server deployments

Problem: A user is logged in on server A but logged out on server B.

Cause: File-based sessions are stored on the local filesystem. Each server has its own session files.

Fix: Switch to Redis, Valkey, MongoDB, or database-backed sessions so all servers share the same session store.

5. Session cookie overwritten by another app

Problem: Your session keeps getting reset even though you set it correctly.

Cause: Another application on the same domain uses the same session cookie name.

Fix: The session cookie name (`tina4_session`) is managed by Tina4. If you need to change it, set `TINA4_SESSION_NAME=myapp_session` in `.env`.

6. Secure cookies on localhost

Problem: Setting `"secure": True` on a cookie means it never gets sent during development.

Cause: Secure cookies are only sent over HTTPS. `http://localhost` is not HTTPS.

Fix: Do not set `"secure": True` during development. Use environment-based configuration: `"secure": os.getenv("TINA4_DEBUG") != "true"`.

7. Flash messages shown twice

Problem: A flash message appears on the page, but when you refresh, it shows again.

Cause: You read the flash message but did not remove it. Using `request.session.get()` reads without deleting.

Fix: Use `request.session.flash("message")` to read flash data. It reads and deletes in one step. Do not use `request.session.get()` for flash messages.

Middleware & Security

1. The Pipeline Pattern

Every HTTP request passes through a series of gates before reaching your route handler. Rate limiter. Body parser. Auth check. Logger. These gates are middleware -- code that wraps your route handler and runs before, after, or both.

Picture a public API. Every request hits a rate limit check. Some endpoints require an API key. All responses need CORS headers. Errors need logging. Without middleware, that logic lives in every handler. Duplicated. Scattered. Fragile. With middleware, you write it once and attach it where it belongs.

Tina4 Python ships with built-in middleware (CORS, rate limiting) and lets you write your own. This chapter covers both.

---

2. Built-In Middleware

Tina4 Python ships with two built-in middleware classes in `tina4_python.core.middleware`: `CorsMiddleware` and `RateLimiter`. Both are configured via environment variables.

CorsMiddleware

CORS (Cross-Origin Resource Sharing) controls which websites can call your API from a browser. When React at `http://localhost:3000` calls your Tina4 API at `http://localhost:7146`, the browser sends a preflight `OPTIONS` request first. Wrong headers and the browser blocks everything.

Configure via `.env`:

```
TINA4_CORS_ORIGINS=https://app.example.com,https://admin.example.com
TINA4_CORS_METHODS=GET,POST,PUT,PATCH,DELETE,OPTIONS
TINA4_CORS_HEADERS=Content-Type,Authorization,X-Request-ID
TINA4_CORS_MAX_AGE=86400
TINA4_CORS_CREDENTIALS=true
```

With these settings, only `app.example.com` and `admin.example.com` can make cross-origin requests to your API. The browser automatically handles preflight `OPTIONS` requests.

For development, you can allow all origins:

```
TINA4_CORS_ORIGINS=*
```

The CORS middleware is active by default. You do not need to register it manually.

You can also use `CorsMiddleware` programmatically in your own middleware:

```
from tina4_python.core.middleware import CorsMiddleware

cors = CorsMiddleware()

# Check if an origin is allowed
```

```
allowed = cors.allowed_origin("https://app.example.com")

# Apply CORS headers to a response
cors.apply(request, response)

# Check if this is a preflight request
if cors.is_preflight(request):
    # Handle OPTIONS request
    pass
```

RateLimiter

The rate limiter prevents abuse by limiting how many requests a single IP can make in a sliding time window. It uses an in-memory store that is thread-safe.

Configure via `.env`:

```
TINA4_RATE_LIMIT=60
TINA4_RATE_WINDOW=60
```

This allows 60 requests per 60-second window per IP address. When a client exceeds the limit, they receive a `429 Too Many Requests` response with a `Retry-After` header.

Rate limit headers are included in every response:

```
X-RateLimit-Limit: 60
X-RateLimit-Remaining: 57
X-RateLimit-Reset: 60
```

You can use the `RateLimiter` class directly for custom rate limiting logic:

```
from tina4_python.core.middleware import RateLimiter

limiter = RateLimiter()

allowed, info = limiter.check(request.ip)
if not allowed:
    return response.json({
        "error": "Too many requests",
        "retry_after": info["reset"]
    }, 429)

# Add rate limit headers to the response
limiter.apply_headers(response, info)
```

Like CORS, the rate limiter is active by default based on your `.env` configuration.

---

3. Writing Custom Middleware

Tina4 Python supports two styles of middleware: function-based and class-based.

Function-Based Middleware

A middleware function takes three arguments: `request`, `response`, and `next_handler`. It must return a response.

```
async def my_middleware(request, response, next_handler):
```

```
    # Code that runs BEFORE the route handler
    print("Before handler")

    # Call the next middleware or the route handler
    result = await next_handler(request, response)

    # Code that runs AFTER the route handler
    print("After handler")

    return result
```

Class-Based Middleware

Class-based middleware uses a naming convention: static methods prefixed with `before_` run before the route handler, and methods prefixed with `after_` run after it. Each method receives `(request, response)` and returns `(request, response)`.

```
class MyMiddleware:
    @staticmethod
    def before_check(request, response):
        """Runs before the route handler."""
        print("Before handler")
        return request, response

    @staticmethod
    def after_cleanup(request, response):
        """Runs after the route handler."""
        print("After handler")
        return request, response
```

If a `before_*` method returns a response with an error status (>= 400), the route handler is skipped entirely. This is short-circuiting.

Example: Request Timer (Class-Based)

```
import time

class TimingMiddleware:
    @staticmethod
    def before_start_timer(request, response):
        request._start_time = time.time()
        return request, response

    @staticmethod
    def after_add_timing(request, response):
        elapsed = time.time() - getattr(request, "_start_time", time.time())
        response.add_header("X-Response-Time", f"{elapsed:.3f}s")
        return request, response
```

Example: Request Logger (Function-Based)

```
from datetime import datetime

async def log_middleware(request, response, next_handler):
    timestamp = datetime.now().strftime("%Y-%m-%d %H:%M:%S")
    print(f"[{timestamp}] {request.method} {request.path}")
    print(f"  Headers: {dict(request.headers)}")

    if request.body:
        print(f"  Body: {request.body}")

    result = await next_handler(request, response)

    print(f"  Status: {result.status_code if hasattr(result, 'status_code') else 'unknown'}")

    return result
```

Example: Security Headers (Class-Based)

```
class SecurityHeaders:
    @staticmethod
    def after_security(request, response):
        response.add_header("X-Content-Type-Options", "nosniff")
        response.add_header("X-Frame-Options", "DENY")
        response.add_header("X-XSS-Protection", "1; mode=block")
        response.add_header("Strict-Transport-Security", "max-age=31536000")
        return request, response
```

Example: JSON Content-Type Enforcer (Function-Based)

```
async def require_json(request, response, next_handler):
    if request.method in ("POST", "PUT", "PATCH"):
        content_type = request.headers.get("Content-Type", "")
        if "application/json" not in content_type:
            return response.json({
                "error": "Content-Type must be application/json"
            }, 415)

    return await next_handler(request, response)
```

> ***Note**: `request.headers` is case-insensitive (HTTP per RFC 7230). `headers.get("Content-Type")`, `headers.get("content-type")`, and `headers.get("CONTENT-TYPE")` all return the same value.*

Example: Request ID (Class-Based)

```
import secrets

class RequestIdMiddleware:
    @staticmethod
    def before_inject_id(request, response):
        request.request_id = secrets.token_hex(8)
        return request, response

    @staticmethod
    def after_add_header(request, response):
        response.add_header("X-Request-ID", getattr(request, "request_id", ""))
        return request, response
```

Example: Input Sanitization (Class-Based)

```
import html

class InputSanitizer:
    @staticmethod
    def before_sanitize(request, response):
        if request.body and isinstance(request.body, dict):
            request.body = InputSanitizer._sanitize_dict(request.body)
        return request, response

    @staticmethod
    def _sanitize_dict(data):
        sanitized = {}
        for key, value in data.items():
            if isinstance(value, str):
                sanitized[key] = html.escape(value)
            elif isinstance(value, dict):
                sanitized[key] = InputSanitizer._sanitize_dict(value)
            else:
                sanitized[key] = value
        return sanitized
```

---

4. The @middleware Decorator

Apply middleware to a single route with the `@middleware` decorator. Both function-based and class-based middleware work:

```
from tina4_python.core.router import get, post, middleware

# Function-based middleware
@middleware(timer_middleware)
@get("/api/data")
async def get_data(request, response):
    return response.json({"data": [1, 2, 3]})

# Class-based middleware
@middleware(TimingMiddleware)
@get("/api/stats")
async def get_stats(request, response):
    return response.json({"stats": {}})
```

Apply multiple middleware by passing them as separate arguments:

```
@middleware(RequestIdMiddleware, SecurityHeaders, require_json)
@post("/api/items")
async def create_item(request, response):
    return response.json({"item": request.body}, 201)
```

You can mix function-based and class-based middleware freely. They run in the order you list them. In the example above: `RequestIdMiddleware` runs first (before and after hooks), then `SecurityHeaders`, then `require_json`, then the route handler.

> ***Note**: `@middleware()` is purely additive; it does **not** change a route's auth requirement. POST/PUT/PATCH/DELETE routes stay Bearer-token-gated by default even when you add custom middleware. Use ~~`@noauth()` to open a write route~~, `@secured()` to lock a*

read route.

---

5. Middleware on Route Groups

Apply middleware to all routes in a group:

```
from tina4_python.core.router import Router, get, post, middleware

def api_v1(group):

    @group.get("/users")
    async def list_users(request, response):
        return response.json({"users": []})

    @group.post("/users")
    async def create_user(request, response):
        return response.json({"created": True}, 201)

    @group.get("/products")
    async def list_products(request, response):
        return response.json({"products": []})

Router.group("/api/v1", api_v1, middleware=[TimingMiddleware, SecurityHeaders])
```

Every route inside the group now has `TimingMiddleware` and `SecurityHeaders` applied. You can still add route-specific middleware on top:

```
def api_v1(group):

    @group.get("/public")
    async def public_endpoint(request, response):
        # Only group middleware runs
        return response.json({"public": True})

    @middleware(AuthMiddleware)
    @group.post("/admin")
    async def admin_endpoint(request, response):
        # Group middleware + AuthMiddleware both run
        return response.json({"admin": True})

Router.group("/api/v1", api_v1, middleware=[RequestIdMiddleware])
```

Group middleware always runs before route-specific middleware. This means authentication checks at the group level cannot be bypassed by individual routes.

---

6. Execution Order

Stacked middleware forms a nested pipeline. Requests travel inward. Responses travel outward:

```
Request arrives
  → log_middleware (before)
    → auth_middleware (before)
      → timer_middleware (before)
        → route handler
```

```
      → timer_middleware (after)
    → auth_middleware (after)
  → log_middleware (after)
Response sent
```

Each middleware wraps around the next one. The outermost middleware runs first on the way in and last on the way out.

Here is a concrete example showing the order:

```
async def middleware_a(request, response, next_handler):
    print("A: before")
    result = await next_handler(request, response)
    print("A: after")
    return result

async def middleware_b(request, response, next_handler):
    print("B: before")
    result = await next_handler(request, response)
    print("B: after")
    return result

@get("/test")
@middleware(middleware_a, middleware_b)
async def test(request, response):
    print("Handler")
    return response.json({"ok": True})
```

When you request `/test`, the console shows:

```
A: before
B: before
Handler
B: after
A: after
```

---

7. Short-Circuiting

Skip `next_handler` and the chain stops cold. The route handler never runs. This is how blocking middleware works:

```
async def maintenance_mode(request, response, next_handler):
    import os
    if os.getenv("MAINTENANCE_MODE") == "true":
        return response.json({
            "error": "Service is under maintenance. Please try again later."
        }, 503)

    return await next_handler(request, response)
```

When `MAINTENANCE_MODE=true`, every request gets a 503 response without reaching any route handler.

Conditional Short-Circuit

```
async def require_api_key(request, response, next_handler):
    api_key = request.headers.get("X-API-Key", "")

    if not api_key:
        return response.json({"error": "API key required"}, 401)

    # Validate the key against a database or config
    valid_keys = ["key-abc-123", "key-def-456", "key-ghi-789"]
    if api_key not in valid_keys:
        return response.json({"error": "Invalid API key"}, 403)

    # Attach the key info to the request for the handler to use
    request.api_key = api_key

    return await next_handler(request, response)
```

```
# No key -- 401
curl http://localhost:7146/api/data
```

```
{"error":"API key required"}
```

```
# Invalid key -- 403
curl http://localhost:7146/api/data -H "X-API-Key: wrong-key"
```

```
{"error":"Invalid API key"}
```

```
# Valid key -- 200
curl http://localhost:7146/api/data -H "X-API-Key: key-abc-123"
```

```
{"data":[1,2,3]}
```

---

8. Modifying Request and Response

Middleware can modify the request before it reaches the handler, and the response before it reaches the client.

Adding Data to the Request

```
async def inject_user_agent(request, response, next_handler):
    ua = request.headers.get("User-Agent", "")

    request.is_mobile = "Mobile" in ua or "Android" in ua or "iPhone" in ua
    request.is_bot = "bot" in ua.lower() or "spider" in ua.lower()

    return await next_handler(request, response)
```

Now the route handler can access `request.is_mobile` and `request.is_bot`.

Modifying the Response

```
async def add_security_headers(request, response, next_handler):
    result = await next_handler(request, response)

    # Add security headers to every response
    return response.header("X-Content-Type-Options", "nosniff") \
                   .header("X-Frame-Options", "DENY") \
                   .header("X-XSS-Protection", "1; mode=block") \
                   .header("Strict-Transport-Security", "max-age=31536000; includeSubDomains")
```

---

9. Real-World Example: JWT Authentication Middleware

This class-based middleware verifies JWT tokens on protected routes. It uses the `before_*` / `after_*` convention.

```
from tina4_python.auth import Auth

class JwtAuthMiddleware:
    @staticmethod
    def before_verify_token(request, response):
        auth_header = request.headers.get("authorization", "")

        if not auth_header or not auth_header.startswith("Bearer "):
            return request, response({"error": "Authorization header required"}, 401)

        token = auth_header[7:]
        payload = Auth.valid_token(token)

        if payload is None:
            return request, response({"error": "Invalid or expired token"}, 401)

        # Attach the decoded payload to the request
        request.user = payload
        return request, response
```

Apply it to a group of protected routes:

```
from tina4_python.core.router import Router, get, post, middleware

def protected_routes():

    @get("/profile")
    async def get_profile(request, response):
        return response({"user": request.user})

    @post("/settings")
    async def update_settings(request, response):
        user_id = request.user["sub"]
        return response({"updated": True, "user_id": user_id})

Router.group("/api/protected", protected_routes, middleware=[JwtAuthMiddleware])
```

The middleware short-circuits with 401 if the token is missing or invalid. The route handler never runs. If the token is valid, the decoded payload is available as `request.user`.

---

10. Real-World Example: API Key Middleware with Database Lookup

```
from tina4_python.database.connection import Database
from datetime import datetime

async def api_key_middleware(request, response, next_handler):
    api_key = request.headers.get("X-API-Key", "")

    if not api_key:
        return response.json({
            "error": "API key required. Send it in the X-API-Key header."
        }, 401)

    db = Database()
    key_record = db.fetch_one(
        "SELECT id, name, rate_limit, is_active FROM api_keys WHERE key_value = ?",
        [api_key]
    )

    if key_record is None:
        return response.json({"error": "Invalid API key"}, 403)

    if not key_record["is_active"]:
        return response.json({"error": "API key has been deactivated"}, 403)

    # Update last used timestamp
    db.execute(
        "UPDATE api_keys SET last_used_at = ?, request_count = request_count + 1 WHERE id = ?",
        [datetime.now().isoformat(), key_record["id"]]
    )

    # Attach key info to request
    request.api_key_id = key_record["id"]
    request.api_key_name = key_record["name"]

    return await next_handler(request, response)
```

---

11. Exercise: Build an API Key Middleware System

Build a complete API key system with key management and usage tracking.

Requirements

- Create a migration for an `api_keys` table: `id`, `name`, `key_value` (unique), `is_active` (default true), `rate_limit` (default 100), `request_count` (default 0), `last_used_at`, `created_at`
- Build these endpoints:

Method	Path	Middleware	Description

`POST`	`/admin/api-keys`	Auth	Create a new API key (generate random key)
`GET`	`/admin/api-keys`	Auth	List all API keys with usage stats
`DELETE`	`/admin/api-keys/{id:int}`	Auth	Deactivate an API key
`GET`	`/api/data`	API Key	Protected endpoint -- requires valid API key
`GET`	`/api/status`	API Key	Another protected endpoint

- The API key middleware should:
- Check `X-API-Key` header
- Validate against the database
- Reject deactivated keys
- Track usage (increment count, update last*used*at)
- Attach key info to the request

Test with:

```
# Create an API key (requires auth token from Chapter 8)
curl -X POST http://localhost:7146/admin/api-keys \
  -H "Authorization: Bearer YOUR_AUTH_TOKEN" \
  -H "Content-Type: application/json" \
  -d '{"name": "Mobile App"}'

# Use the API key
curl http://localhost:7146/api/data \
  -H "X-API-Key: THE_GENERATED_KEY"

# List keys with stats
curl http://localhost:7146/admin/api-keys \
  -H "Authorization: Bearer YOUR_AUTH_TOKEN"
```

---

12. Solution

Migration

Create `migrations/20260322170000_create_api_keys_table.sql`:

```
-- UP
CREATE TABLE api_keys (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    name TEXT NOT NULL,
```

```
    key_value TEXT NOT NULL UNIQUE,
    is_active INTEGER NOT NULL DEFAULT 1,
    rate_limit INTEGER NOT NULL DEFAULT 100,
    request_count INTEGER NOT NULL DEFAULT 0,
    last_used_at TEXT,
    created_at TEXT DEFAULT CURRENT_TIMESTAMP
);

CREATE UNIQUE INDEX idx_api_keys_key ON api_keys (key_value);

-- DOWN
DROP INDEX IF EXISTS idx_api_keys_key;
DROP TABLE IF EXISTS api_keys;
```

Routes

Create `src/routes/api_keys.py`:

```
from tina4_python.core.router import get, post, delete as delete_route, middleware
from tina4_python.auth import Auth
from tina4_python.database.connection import Database
from datetime import datetime
import secrets


async def auth_middleware(request, response, next_handler):
    auth_header = request.headers.get("Authorization", "")
    if not auth_header or not auth_header.startswith("Bearer "):
        return response.json({"error": "Authorization required"}, 401)

    token = auth_header[7:]
    if not Auth.valid_token(token):
        return response.json({"error": "Invalid or expired token"}, 401)

    request.user = Auth.get_payload(token)
    return await next_handler(request, response)


async def api_key_middleware(request, response, next_handler):
    api_key = request.headers.get("X-API-Key", "")

    if not api_key:
        return response.json({"error": "API key required. Send in X-API-Key header."}, 401)

    db = Database()
    key_record = db.fetch_one(
        "SELECT id, name, is_active FROM api_keys WHERE key_value = ?",
        [api_key]
    )

    if key_record is None:
        return response.json({"error": "Invalid API key"}, 403)

    if not key_record["is_active"]:
        return response.json({"error": "API key has been deactivated"}, 403)

    db.execute(
        "UPDATE api_keys SET last_used_at = ?, request_count = request_count + 1 WHERE id = ?",
        [datetime.now().isoformat(), key_record["id"]]
```

```
    )

    request.api_key_id = key_record["id"]
    request.api_key_name = key_record["name"]

    return await next_handler(request, response)


@post("/admin/api-keys")
@middleware(auth_middleware)
async def create_api_key(request, response):
    db = Database()
    name = request.body.get("name", "Unnamed Key")
    key_value = f"tk_{secrets.token_hex(24)}"

    db.execute(
        "INSERT INTO api_keys (name, key_value) VALUES (?, ?)",
        [name, key_value]
    )

    key = db.fetch_one("SELECT * FROM api_keys WHERE id = last_insert_rowid()")

    return response.json({"message": "API key created", "key": key}, 201)


@get("/admin/api-keys")
@middleware(auth_middleware)
async def list_api_keys(request, response):
    db = Database()
    keys = db.fetch("SELECT id, name, key_value, is_active, rate_limit, request_count, last_used
    return response.json({"keys": keys, "count": len(keys)})


@delete_route("/admin/api-keys/{id:int}")
@middleware(auth_middleware)
async def deactivate_api_key(request, response):
    db = Database()
    key_id = request.params["id"]

    existing = db.fetch_one("SELECT id FROM api_keys WHERE id = ?", [key_id])
    if existing is None:
        return response.json({"error": "API key not found"}, 404)

    db.execute("UPDATE api_keys SET is_active = 0 WHERE id = ?", [key_id])
    return response.json({"message": "API key deactivated"})


@get("/api/data")
@middleware(api_key_middleware)
async def api_data(request, response):
    return response.json({
        "data": [1, 2, 3, 4, 5],
        "api_key": request.api_key_name
    })


@get("/api/status")
@middleware(api_key_middleware)
async def api_status(request, response):
```

```
    return response.json({
        "status": "operational",
        "api_key": request.api_key_name
    })
```

---

13. Gotchas

1. Forgetting to await next_handler

Problem: The route handler never runs, or you get an error about a coroutine object.

Cause: You called `next_handler(request, response)` without `await`.

Fix: Always use `await next_handler(request, response)`. Since Tina4 Python is async, every middleware and handler must be awaited.

2. Middleware modifies response after it is sent

Problem: Headers or cookies you add in the "after" phase of middleware do not appear in the response.

Cause: The response was already finalized by the route handler.

Fix: In the "after" phase, modify the result returned by `next_handler`, not the original `response` object. Some modifications may need to happen in the "before" phase instead.

3. Middleware applied to wrong routes

Problem: Your API key middleware runs on public routes that should not require a key.

Cause: The middleware is applied to the group, and the public route is inside that group.

Fix: Move public routes outside the group, or use `@noauth` on specific routes to bypass authentication middleware. For custom middleware like API key checks, you need to handle exemptions manually by checking the path in the middleware.

4. Middleware execution order surprises

Problem: Your auth check runs after your logging middleware, but you wanted it to run first.

Cause: Middleware in `@middleware(a, b, c)` runs in left-to-right order: `a` wraps `b` wraps `c` wraps handler.

Fix: Put the middleware you want to run first at the leftmost position: `@middleware(auth_middleware, log_middleware)`.

5. Error in middleware breaks the chain

Problem: An unhandled exception in middleware causes a 500 error without reaching the route handler.

Cause: If middleware throws an exception before calling `next_handler`, no subsequent middleware or the handler runs.

Fix: Wrap risky middleware code in try/except and return an appropriate error response instead of letting the exception propagate.

6. Database connections in middleware

Problem: Opening a database connection in middleware that runs on every request causes connection pool exhaustion.

Cause: Each middleware call creates a new `Database()` instance.

Fix: Tina4's `Database()` uses connection pooling internally, so this is usually safe. But if you are seeing issues, cache your database lookups (like API keys) in memory with a TTL instead of querying on every request.

7. Modifying request in middleware does not persist

Problem: You set `request.custom_field = "value"` in middleware, but the route handler does not see it.

Cause: In some edge cases, the request object may be copied between middleware stages.

Fix: Use `request.custom_field` consistently. If it is not persisting, check that you are modifying the same request object that is passed to `next_handler`. Do not create a new request object.

---

14. Security

Every route you write is a door. Chapter 8 gave you locks. Chapter 9 gave you session keys. The first half of this chapter gave you guards. This section ties them together into a defence that works without thinking about it.

Tina4 ships secure by default. POST routes require authentication. CSRF tokens protect forms once you turn them on. Security headers harden every response. The framework does the boring security work so you focus on building features. But you need to understand what it does, and why, so you don't accidentally undo it.

---

1. Secure-by-Default Routing

Every POST, PUT, PATCH, and DELETE route requires a valid `Authorization: Bearer` token. No configuration needed. No decorator to remember. The framework enforces this before your handler runs.

```
from tina4_python.core.router import post

@post("/api/orders")
async def create_order(request, response):
    # This handler ONLY runs if the request carries a valid Bearer token.
    # Without one, the framework returns 401 before your code executes.
    return response({"created": True}, 201)
```

Test it without a token:

```
curl -X POST http://localhost:7146/api/orders \
  -H "Content-Type: application/json" \
  -d '{"product": "widget"}'
# 401 Unauthorized
```

Test it with a valid token:

```
curl -X POST http://localhost:7146/api/orders \
  -H "Content-Type: application/json" \
  -H "Authorization: Bearer eyJhbGciOiJIUzI1NiJ9..." \
  -d '{"product": "widget"}'
# 201 Created
```

GET routes are public by default. Anyone can read. Writing requires proof of identity.

Making a Write Route Public

Some endpoints need to accept unauthenticated writes: webhooks, registration forms, public contact forms. Use `@noauth()`:

```
from tina4_python.core.router import post, noauth

@noauth()
@post("/api/webhooks/stripe")
async def stripe_webhook(request, response):
    # No token required. Stripe can POST here freely.
    return response({"received": True})
```

Protecting a GET Route

Admin dashboards, user profiles, account settings: some pages need protection even though they only read data. Use `@secured()`:

```
from tina4_python.core.router import get, secured

@secured()
@get("/api/admin/users")
async def admin_users(request, response):
    # Requires a valid Bearer token, even though it's a GET.
    return response({"users": []})
```

The Rule

Method	Default	Override
GET, HEAD, OPTIONS	Public	`@secured()` to protect
POST, PUT, PATCH, DELETE	Auth required	`@noauth()` to open

Two decorators. One rule. No surprises.

---

2. CSRF Protection

Cross-Site Request Forgery tricks a user's browser into submitting a form to your server. The browser sends cookies automatically, including session cookies. Without CSRF protection, an attacker's page can submit forms as your logged-in user.

Tina4 blocks this with form tokens.

How It Works

- Your template renders a hidden token using `{{ form_token() }}`.
- The browser submits the token with the form data.
- The `CsrfMiddleware` validates the token before the route handler runs.
- Invalid or missing tokens receive a `403 Forbidden` response.

The Template

```
<form method="POST" action="/api/profile">
    {{ form_token() }}
    <input type="text" name="name" placeholder="Your name">
    <button type="submit">Save</button>
</form>
```

The `{{ form_token() }}` call generates a hidden input field containing a signed JWT. The token is bound to the current session; a token from one session cannot be used in another.

The Middleware

CSRF protection is off by default. Set `TINA4_CSRF=true` to turn it on; from then on every POST, PUT, PATCH, and DELETE request must include a valid form token. The middleware checks two places:

- **Request body**: `request.body["formToken"]`
- **Request header**: `X-Form-Token`

If the token is missing or invalid, the middleware returns 403 before your handler runs.

AJAX Requests

For JavaScript-driven forms, send the token as a header:

```
// frond.min.js handles this automatically via saveForm()
// For manual AJAX, extract the token from the hidden field:
const token = document.querySelector('input[name="formToken"]').value;

fetch("/api/profile", {
    method: "POST",
    headers: {
        "Content-Type": "application/json",
        "X-Form-Token": token
    },
    body: JSON.stringify({ name: "Alice" })
});
```

Tokens in Query Strings - Blocked

Tokens must never appear in URLs. Query strings are logged in server access logs, browser history, and referrer headers. A token in the URL is a token anyone can steal.

Tina4 rejects any request that carries `formToken` in the query string and logs a warning:

```
CSRF token found in query string - rejected for security.
Use POST body or X-Form-Token header instead.
```

Skipping CSRF Validation

Three scenarios skip CSRF checks automatically:

- **GET, HEAD, OPTIONS** - Safe methods don't modify state.
- **Routes with** `@noauth()` - Public write endpoints don't need CSRF (they have no session to protect).
- **Requests with a valid Bearer token** - API clients authenticate with tokens, not cookies. CSRF only matters for cookie-based sessions.

Turning CSRF On

CSRF stays off until you turn it on. Turn it on for anything a browser can reach:

```
TINA4_CSRF=true
```

The cost is one hidden field per form. The protection is worth it. `TINA4_CSRF=false` turns it off again. Leave it off only for internal microservices behind a firewall, where no browser ever touches the API.

---

3. Session-Bound Tokens

A form token alone prevents cross-site forgery. But what if someone steals a token from a form? Session binding stops them.

When `{{ form_token() }}` generates a token, it embeds the current session ID in the JWT payload. The CSRF middleware checks that the session ID in the token matches the session ID of the request. A token stolen from one session cannot be replayed in another.

This happens automatically. No configuration. No extra code.

How Stolen Tokens Fail

- Attacker visits your site, gets a form token for session `abc-123`.
- Attacker sends that token from their own session `xyz-789`.
- The middleware compares: `abc-123 != xyz-789`, rejected with 403.

The token is cryptographically valid. But it belongs to the wrong session. The binding catches it.

---

4. Security Headers

Every response from Tina4 carries security headers. The `SecurityHeadersMiddleware` injects them before the response reaches the browser. No opt-in required.

Header	Default Value	Purpose
`X-Frame-Options`	`SAMEORIGIN`	Prevents clickjacking - your pages cannot be embedded in iframes on other domains.
`X-Content-Type-Options`	`nosniff`	Stops browsers from guessing content types. A script is a script, not HTML.
`Content-Security-Policy`	`default-src 'self'`	Controls which resources the browser loads. Blocks inline scripts from injected HTML.
`Referrer-Policy`	`strict-origin-when-cross-origin`	Limits referrer data sent to external sites. Protects internal URLs from leaking.
`X-XSS-Protection`	`0`	Disabled. Modern CSP replaces this legacy header. Keeping it on can introduce vulnerabilities.
`Permissions-Policy`	`camera=(), microphone=(), geolocation=()`	Disables browser APIs your app does not need.

HSTS - Enforcing HTTPS

Strict Transport Security tells the browser to always use HTTPS. Disabled by default (it breaks local development on HTTP). Enable it in production:

```
TINA4_HSTS=31536000
```

This sets a one-year HSTS policy with `includeSubDomains`. Once a browser sees this header, it refuses to connect over HTTP, even if the user types `http://`.

Customising Headers

Override any header via environment variables:

```
TINA4_FRAME_OPTIONS=DENY
TINA4_CSP=default-src 'self'; script-src 'self' https://cdn.example.com
TINA4_REFERRER_POLICY=no-referrer
TINA4_PERMISSIONS_POLICY=camera=(), microphone=(), geolocation=(), payment=()
```

---

5. SameSite Cookies

Session cookies control who can send them. The `SameSite` attribute tells the browser when to include the cookie in requests.

Value	Behaviour
`Strict`	Cookie sent only on same-site requests. Even clicking a link from email to your site won't include the cookie. The user must navigate directly.
`Lax`	Cookie sent on same-site requests and top-level navigations (clicking a link). Not sent on cross-site AJAX or form POSTs from other domains.
`None`	Cookie sent on all requests, including cross-site. Requires `Secure` flag (HTTPS only).

Tina4 defaults to `Lax`. This blocks cross-site form submissions (CSRF) while allowing normal link navigation. Users click a link to your site from an email and they stay logged in. An attacker's page submits a hidden form and the cookie is withheld.

For most applications, `Lax` is the right choice. Change it only if you understand the trade-offs.

---

6. Login Flow - Complete Example

Authentication, sessions, tokens, and security converge in the login flow. Here is a complete implementation.

The Login Route

```
from tina4_python.core.router import post, noauth
from tina4_python.auth import Auth, get_token

@noauth()
@post("/api/login")
async def login(request, response):
    email = request.body.get("email", "")
    password = request.body.get("password", "")

    if not email or not password:
        return response({"error": "Email and password required"}, 400)

    # Look up user (replace with your database query)
    user = db.fetch_one(
        "SELECT id, email, password_hash, role FROM users WHERE email = ?",
        [email]
    )

    if not user:
        return response({"error": "Invalid credentials"}, 401)

    # Verify password
    if not Auth.check_password(password, user["password_hash"]):
        return response({"error": "Invalid credentials"}, 401)

    # Generate token with user claims
    token = get_token(
        {"sub": user["id"], "email": user["email"], "role": user["role"]}
    )

    # Store user in session
    request.session.set("user_id", user["id"])
    request.session.set("email", user["email"])
    request.session.set("role", user["role"])
    request.session.save()

    return response({"token": token, "user": {"id": user["id"], "email": user["email"]}})
```

The `@noauth()` decorator opens this route to unauthenticated requests. The handler validates credentials and issues a token. The session stores the user identity for server-side lookups.

The Login Form

```
{% extends "base.twig" %}
{% block content %}
<div class="container mt-5" style="max-width: 400px;">
    <h2>Login</h2>
    <form id="loginForm">
        {{ form_token() }}
        <div class="mb-3">
            <label for="email">Email</label>
            <input type="email" name="email" id="email" class="form-control"
                   placeholder="you@example.com" required>
        </div>
        <div class="mb-3">
            <label for="password">Password</label>
            <input type="password" name="password" id="password" class="form-control"
                   placeholder="Your password" required>
        </div>
        <button type="button" class="btn btn-primary w-100"
                onclick="saveForm('loginForm', '/api/login', 'loginMsg', handleLogin)">
            Sign In
        </button>
        <div id="loginMsg" class="mt-3"></div>
    </form>
</div>

<script>
function handleLogin(result) {
    if (result.token) {
        localStorage.setItem("token", result.token);
        window.location.href = "/dashboard";
    }
}
</script>
{% endblock %}
```

Protected Pages - Checking the Session

```
from tina4_python.core.router import get

@get("/dashboard")
async def dashboard(request, response):
    user_id = request.session.get("user_id")

    if not user_id:
        return response.redirect("/login")

    return response.render("dashboard.twig", {
        "email": request.session.get("email"),
        "role": request.session.get("role"),
    })
```

Logout - Destroying the Session

```
from tina4_python.core.router import post, noauth

@noauth()
@post("/api/logout")
async def logout(request, response):
    request.session.destroy()
    return response({"logged_out": True})
```

---

7. Handling Expired Sessions

Sessions expire. Tokens expire. The user clicks a link and finds themselves staring at a broken page or a cryptic error. A good security implementation handles expiry gracefully.

The Pattern: Redirect to Login, Then Back

When a session expires mid-use, the user should:

- See a login page, not an error.
- Log in again.
- Land on the page they were trying to reach, not the home page.

```
from urllib.parse import quote

@get("/account/settings")
async def account_settings(request, response):
    user_id = request.session.get("user_id")

    if not user_id:
        # Remember where they wanted to go
        return_url = quote(request.url)
        return response.redirect(f"/login?redirect={return_url}")

    return response.render("settings.twig", {"user_id": user_id})
```

The login handler reads the `redirect` parameter after successful authentication:

```
@noauth()
@post("/api/login")
async def login(request, response):
    # ... validate credentials ...

    redirect_url = request.params.get("redirect", "/dashboard")

    return response({
        "token": token,
        "redirect": redirect_url
    })
```

The login form JavaScript redirects to the saved URL:

```
function handleLogin(result) {
    if (result.token) {
        localStorage.setItem("token", result.token);
        window.location.href = result.redirect || "/dashboard";
    }
```

```
}
```

Token Refresh

Tokens expire based on `TINA4_TOKEN_LIMIT` (default: 60 minutes). The `frond.min.js` frontend library handles token refresh automatically: every response includes a `FreshToken` header with a new token. The client stores it and uses it for the next request.

For custom AJAX code, read the header yourself:

```
const res = await fetch("/api/data", {
    headers: { "Authorization": "Bearer " + localStorage.getItem("token") }
});

const freshToken = res.headers.get("FreshToken");
if (freshToken) {
    localStorage.setItem("token", freshToken);
}
```

---

8. Rate Limiting

Brute-force login attempts. Credential stuffing. API abuse. Rate limiting stops all of them.

Tina4 includes a sliding-window rate limiter that tracks requests per IP address. It activates automatically.

```
TINA4_RATE_LIMIT=100
TINA4_RATE_WINDOW=60
```

One hundred requests per sixty seconds per IP. Exceed the limit and the server returns `429 Too Many Requests` with headers telling the client when to retry:

```
X-RateLimit-Limit: 100
X-RateLimit-Remaining: 0
X-RateLimit-Reset: 45
```

For login routes, consider a stricter limit:

```
from tina4_python.core.router import post, noauth, middleware

class LoginRateLimit:
    @staticmethod
    def before_rate_check(request, response):
        # Custom rate limiter: 5 attempts per 60 seconds for login
        ip = request.ip
        # ... implement per-route rate limiting ...
        return request, response

@noauth()
@middleware(LoginRateLimit)
@post("/api/login")
async def login(request, response):
    # ... login logic ...
    pass
```

---

9. CORS and Credentials

When your frontend runs on a different origin than your API (common in development), CORS controls whether the browser sends cookies and auth headers.

Tina4 handles CORS automatically. The relevant security settings:

```
TINA4_CORS_ORIGINS=*
TINA4_CORS_CREDENTIALS=true
```

Two rules to remember:

- `TINA4_CORS_ORIGINS=*` **with** `TINA4_CORS_CREDENTIALS=true` is invalid per the CORS spec. Tina4 handles this: when origin is `*`, the credentials header is not sent. But in production, list your actual origins.
- **Cookies need** `SameSite=None; Secure` for true cross-origin requests. If your API is on `api.example.com` and your frontend is on `app.example.com`, the default `Lax` cookie works because they share the same registrable domain. Different domains need `SameSite=None`.

Production CORS:

```
TINA4_CORS_ORIGINS=https://app.example.com,https://admin.example.com
TINA4_CORS_CREDENTIALS=true
```

---

10. Security Checklist

Before you deploy, verify:

- [ ] `TINA4_SECRET` is set to a long, random string, not the default.
- [ ] `TINA4_DEBUG=false` in production.
- [ ] `TINA4_HSTS=31536000` if serving over HTTPS.
- [ ] `TINA4_CORS_ORIGINS` lists your actual domains, not `*`.
- [ ] `TINA4_CSRF=true` for any browser-facing application (CSRF is off by default).
- [ ] Login route uses `@noauth()` and validates credentials before issuing tokens.
- [ ] Session is regenerated after login (prevents session fixation).
- [ ] Passwords are hashed with `Auth.hash_password()`, never stored in plain text.
- [ ] File uploads are validated and size-limited (`TINA4_MAX_UPLOAD_SIZE`).
- [ ] Rate limiting is active on login and registration routes.
- [ ] Expired sessions redirect to login with a return URL.

Gotchas

1. "My POST route returns 401 but I didn't add auth"

Cause: Tina4 requires authentication on all write routes by default.

Fix: Add `@noauth()` above the route decorator if the endpoint should be public. Otherwise, send a valid Bearer token with the request.

2. "CSRF validation fails on AJAX requests"

Cause: The form token is not included in the request.

Fix: Send the token as an `X-Form-Token` header. If using `frond.min.js`, call `saveForm()`; it handles tokens automatically.

3. "I disabled CSRF but forms still fail"

Cause: The route still requires Bearer auth (separate from CSRF). CSRF and auth are independent checks.

Fix: Either send a Bearer token or add `@noauth()` to the route.

4. "My Content-Security-Policy blocks inline scripts"

Cause: The default CSP is `default-src 'self'`, which blocks inline `<script>` tags and `onclick` handlers.

Fix: Move scripts to external `.js` files (the right approach) or relax the CSP:

```
TINA4_CSP=default-src 'self'; script-src 'self' 'unsafe-inline'
```

Prefer external scripts. Inline scripts are an XSS vector.

5. "User stays logged in after session expires"

Cause: The frontend stores a JWT in localStorage. The token is still valid even after the session is destroyed server-side.

Fix: Check the session on every page load. If the session is gone, redirect to login regardless of the token. Tokens authenticate API calls; sessions track server-side state. Both must be valid.

Exercise: Secure Contact Form

Build a public contact form that:

- Does not require login (`@noauth()`).
- Validates CSRF tokens (form includes `{{ form_token() }}`).
- Rate-limits submissions to 3 per minute per IP.
- Stores messages in the database.
- Returns a success message.

Solution

```
# src/routes/contact.py
from tina4_python.core.router import post, get, noauth, template

@get("/contact")
@template("contact.twig")
async def contact_page(request, response):
    return {"title": "Contact Us"}


@noauth()
@post("/api/contact")
async def submit_contact(request, response):
    name = request.body.get("name", "").strip()
    email = request.body.get("email", "").strip()
    message = request.body.get("message", "").strip()

    if not name or not email or not message:
        return response({"error": "All fields are required"}, 400)

    db.insert("contact_messages", {
        "name": name,
        "email": email,
        "message": message,
    })
    db.commit()

    return response({"success": True, "message": "Thank you for your message"})
```

```
{# src/templates/contact.twig #}
{% extends "base.twig" %}
{% block title %}Contact Us{% endblock %}
{% block content %}
<div class="container mt-5" style="max-width: 500px;">
    <h2>{{ title }}</h2>
    <form id="contactForm">
        {{ form_token() }}
        <div class="mb-3">
            <label for="name">Name</label>
            <input type="text" name="name" id="name" class="form-control"
                   placeholder="Jane Smith" required>
        </div>
        <div class="mb-3">
            <label for="email">Email</label>
            <input type="email" name="email" id="email" class="form-control"
                   placeholder="jane@example.com" required>
        </div>
        <div class="mb-3">
            <label for="message">Message</label>
            <textarea name="message" id="message" class="form-control" rows="4"
                      placeholder="How can we help?" required></textarea>
        </div>
        <button type="button" class="btn btn-primary"
                onclick="saveForm('contactForm', '/api/contact', 'contactMsg')">
            Send Message
        </button>
        <div id="contactMsg" class="mt-3"></div>
    </form>
</div>
```

```
{% endblock %}
```

The form is public. The CSRF token is present. The `@noauth()` decorator opens the route. The middleware validates the token. The database stores the message. The user sees confirmation.

Five moving parts. Zero security holes. The framework handles the rest.

Caching

1. From 800ms to 3ms

Your product catalog page runs 12 database queries. 800 milliseconds to render. Every visitor triggers the same queries, the same template rendering, the same JSON serialization -- for data that changes once a day.

Add caching. The first request takes 800ms. The next 10,000 take 3ms each. A 266x improvement from one line of configuration.

Caching stores the result of expensive work for reuse. Tina4 gives you three layers, each solving a different problem:

- **Request-scoped query cache** -- on by default. Dedupes identical database reads inside a single request.
- **Persistent DB query cache** -- opt-in. Shares query results across requests (and across server instances).
- **HTTP response cache** -- the `ResponseCache` middleware. Stores entire HTTP responses and serves them without touching your handler.

You can layer all three, or use just the one you need.

---

2. The Request-Scoped Query Cache (On by Default)

Every `Database` connection caches query results for the life of a single request. This layer is **on by default** -- you do not configure anything.

When two handlers (or a handler and a middleware, or a template and a route) run the same `SELECT` during one request, the database is hit once. The second call returns the first call's result.

```
from tina4_python.core.router import get
from tina4_python.database import Database

@get("/api/dashboard")
async def dashboard(request, response):
    db = Database()

    # First call hits the database
    users = db.fetch_all("SELECT * FROM users")

    # Same SQL + params during the same request -> served from the
    # request cache, no second round-trip
    users_again = db.fetch_all("SELECT * FROM users")

    return response({"count": len(users)})
```

The cache key comes from the SQL string and its parameters. Different SQL or different parameters mean different keys.

Two rules keep it safe:

- It is **cleared at the start of every request**, so it never serves one request's data to another.
- It is **flushed on any write** (`execute`, `insert`, `update`, `delete`), so a read after a write in the same request sees the new data.

Tuning:

```
TINA4_AUTO_CACHING=true        # default - set to false to turn this layer off
TINA4_AUTO_CACHING_TTL=5       # safety TTL in seconds (default: 5)
```

The TTL is a safety net for code that runs **outside** an HTTP request -- scripts, workers, queue consumers -- where there is no "start of request" to clear the cache. Inside a request, the per-request clear does the work.

---

3. The Persistent DB Query Cache (Opt-In)

The request-scoped cache disappears at the end of each request. When you want query results to survive across requests -- and to be shared across multiple server instances -- enable the persistent DB cache.

```
TINA4_DB_CACHE=true
TINA4_DB_CACHE_TTL=30          # default: 30 seconds
```

With this on, identical queries return cached results across requests until the TTL expires:

```
from tina4_python.core.router import get
from tina4_python.database import Database

@get("/api/categories")
async def list_categories(request, response):
    db = Database()

    # First request: executes the query
    # Later requests within the TTL: served from the cache
    categories = db.fetch_all("SELECT * FROM categories ORDER BY name")

    return response({"categories": categories})
```

The persistent cache routes through the same backend system as the response cache. Point it at a shared store so several instances share one cache with global write-invalidation:

```
TINA4_DB_CACHE_BACKEND=redis                    # memory (default), file, redis, valkey, memcach
TINA4_DB_CACHE_URL=redis://localhost:6379       # connection string for the chosen backend
```

When `TINA4_DB_CACHE_BACKEND` is left at `memory`, the cache lives in-process (fast, but not shared between instances).

When to use the persistent cache

- Read-heavy apps where the same queries run on every request
- Reference data that changes rarely (categories, countries, settings)
- Dashboard queries that aggregate large datasets

When not to use it

- Write-heavy data that changes constantly
- Queries with real-time requirements (live inventory, live prices)
- Anything that must always return the latest row

---

4. Bypassing the Cache Per Query

Both cache layers can be skipped for a single call. Pass `no_cache=True` and that call neither reads from nor writes to the cache -- it always hits the database:

```
db = Database()

# fetch - full signature
fresh = db.fetch("SELECT * FROM products", params=None, limit=100, offset=0, no_cache=True)

# fetch_one - single row, always fresh
row = db.fetch_one("SELECT * FROM products WHERE id = ?", [42], no_cache=True)

# fetch_all - list of dicts, always fresh
rows = db.fetch_all("SELECT * FROM products", params=None, limit=0, offset=0, no_cache=True)
```

Use this for the one query that must read live data while the rest of your app benefits from caching.

DB cache statistics

`db.cache_stats()` reports the state of the query cache for that connection:

```
from tina4_python.core.router import get
from tina4_python.database import Database

@get("/api/db/cache-stats")
async def db_cache_stats(request, response):
    db = Database()
    return response(db.cache_stats())
```

```
{
  "enabled": true,
  "mode": "request",
  "hits": 128,
  "misses": 42,
  "size": 17,
  "ttl": 5,
  "backend": "memory"
}
```

The fields are exactly:

- `enabled` -- whether any query caching is active
- `mode` -- `"request"` (request-scoped only), `"persistent"` (cross-request), or `"off"`
- `hits` / `misses` -- counters for this connection
- `size` -- number of cached entries

- `ttl` -- the active TTL in seconds
- `backend` -- the storage backend name

Flush the query cache and reset the counters with `db.cache_clear()`.

---

5. Response Caching with ResponseCache Middleware

The fastest cache is at the HTTP response level. The `ResponseCache` middleware stores the complete response and serves it on later requests without calling your handler at all.

Attach it as a string in the middleware list, with an optional TTL after the colon:

```
from tina4_python.core.router import get
from datetime import datetime, timezone

@get("/api/products", middleware=["ResponseCache:300"])
async def list_products(request, response):
    # This handler runs 12 database queries and takes 800ms.
    # With ResponseCache, it runs once every 5 minutes.
    print("Handler called -- this should only appear once every 5 minutes")

    products = [
        {"id": 1, "name": "Wireless Keyboard", "price": 79.99},
        {"id": 2, "name": "USB-C Hub", "price": 49.99},
        {"id": 3, "name": "Monitor Stand", "price": 129.99}
    ]

    return response({"products": products, "generated_at": datetime.now(timezone.utc).isoformat(
```

`"ResponseCache:300"` caches the response for 300 seconds (5 minutes). During that window:

- The first request runs the handler (800ms)
- The next 10,000 requests serve the cached response (3ms each)
- After 300 seconds, the cache expires and the next request runs the handler again

```
curl http://localhost:7146/api/products
```

```
{
  "products": [
    {"id": 1, "name": "Wireless Keyboard", "price": 79.99},
    {"id": 2, "name": "USB-C Hub", "price": 49.99},
    {"id": 3, "name": "Monitor Stand", "price": 129.99}
  ],
  "generated_at": "2026-03-22T14:30:00+00:00"
}
```

Call it again within 5 minutes and the `generated_at` timestamp is identical -- the handler did not run.

Three ways to attach it

The string form (no import) and the class form both work.

```
from tina4_python.core.router import get, middleware, cached
from tina4_python.cache import ResponseCache
```

```
# 1. String in the middleware list (TTL after the colon)
@get("/api/a", middleware=["ResponseCache:300"])
async def route_a(request, response):
    return response({"ok": True})

# 2. The @middleware decorator with the class (uses TINA4_CACHE_TTL, default 60s)
@middleware(ResponseCache)
@get("/api/b")
async def route_b(request, response):
    return response({"ok": True})

# 3. The @cached decorator for a per-route TTL override
@cached(max_age=120)
@get("/api/c")
async def route_c(request, response):
    return response({"ok": True})
```

Response cache headers

On a cached route, the middleware adds two headers:

```
X-Cache: HIT
X-Cache-TTL: 247
```

- `X-Cache` is `HIT` (served from cache) or `MISS` (your handler ran)
- `X-Cache-TTL` is the seconds the entry has left on a HIT, or the TTL it was stored with on a MISS

The middleware does **not** set `Cache-Control`. Browser- and CDN-cache directives are your application's call -- add them yourself if you want them.

Caching with query parameters

The cache key is the method plus the full URL including the query string. `/api/products?page=1` and `/api/products?page=2` cache separately:

```
@get("/api/products", middleware=["ResponseCache:300"])
async def list_products(request, response):
    page = int(request.params.get("page", 1))
    return response({
        "page": page,
        "products": [],
        "generated_at": datetime.now(timezone.utc).isoformat()
    })
```

```
curl "http://localhost:7146/api/products?page=1"  # MISS, stores page=1
curl "http://localhost:7146/api/products?page=2"  # MISS, stores page=2
curl "http://localhost:7146/api/products?page=1"  # HIT
```

Only `GET` responses with a 200 status are cached by default.

What not to cache

Do not put `ResponseCache` on:

- **POST, PUT, PATCH, DELETE routes** -- only GET responses are cached
- **User-specific endpoints** -- `/api/profile` returns different data per user, but the key is URL-only

- **Real-time data** -- stock prices, live scores, chat messages
- **Authenticated endpoints** -- unless every user shares the same response

```
# GOOD: public, rarely changing data
@get("/api/categories", middleware=["ResponseCache:3600"])
async def list_categories(request, response):
    return response({"categories": []})

# BAD: user-specific data -- do NOT cache the whole response
@get("/api/profile", middleware=["auth_middleware"])
async def get_profile(request, response):
    return response(request.user)
```

Response cache configuration

```
TINA4_CACHE_TTL=60             # default response TTL in seconds (default: 60)
TINA4_CACHE_MAX_ENTRIES=1000   # max cached entries before LRU eviction (default: 1000)
```

---

6. Cache Backends

Both the response cache and the key/value API (Section 7) share one set of backends, selected with `TINA4_CACHE_BACKEND`:

Backend	Value	Notes
Memory	`memory` (default)	In-process LRU. Fastest. Lost on restart.
File	`file`	JSON files on disk. Survives restarts, zero infrastructure.
Redis	`redis`	Shared across instances. Survives restarts.
Valkey	`valkey`	Redis-protocol compatible.
Memcached	`memcached`	Text protocol over TCP. Unauthenticated.
MongoDB	`mongodb`	TTL collection. Requires `pymongo`.
Database	`database`	A `tina4_cache` table in any Tina4-supported DB.

```
# Memory - the default, nothing to set
TINA4_CACHE_BACKEND=memory

# File
TINA4_CACHE_BACKEND=file
TINA4_CACHE_DIR=data/cache     # directory for the file backend (default: data/cache)
```

Redis (and Valkey / Memcached / MongoDB)

Network backends take a single connection URL. Credentials may be embedded in the URL or supplied separately:

```
TINA4_CACHE_BACKEND=redis
TINA4_CACHE_URL=redis://[user:pass@]host:port/db   # e.g. redis://localhost:6379/0
TINA4_CACHE_USERNAME=                                # optional, if not in the URL
TINA4_CACHE_PASSWORD=                                # optional, if not in the URL
```

The URL accepts a username, a password (`redis://:pass@host`), and a database number (`/0`). Valkey, Memcached, and MongoDB use the same `TINA4_CACHE_URL` with their own schemes (`valkey://`, `memcached://`, `mongodb://`). Memcached is unauthenticated. For the `database` backend, `TINA4_CACHE_URL` is a SQL URL that falls back to `TINA4_DATABASE_URL`.

Your code never changes. `cache_get`, `cache_set`, and the `ResponseCache` middleware work the same on every backend -- only the storage changes.

Graceful fallback

If a backend's driver is missing or its service or credentials are unreachable, Tina4 logs a warning and falls back to the **file** backend -- a real, persistent cache, never a silent no-op. Your app keeps caching even when Redis is down.

---

7. Direct Cache API (Key/Value)

For caching anything that is not a database query or an HTTP response, use the key/value helpers from `tina4_python.cache`.

cache_set

```
from tina4_python.cache import cache_set

# Cache a value for 300 seconds
cache_set("product:42", {
    "id": 42,
    "name": "Wireless Keyboard",
    "price": 79.99,
    "in_stock": True
}, ttl=300)

# Cache a string
cache_set("exchange_rate:USD_EUR", "0.92", ttl=3600)

# No TTL given -> uses TINA4_CACHE_TTL (default 60s)
cache_set("app:config", {"theme": "dark", "lang": "en"})
```

cache_get

```
from tina4_python.cache import cache_get, cache_set

product = cache_get("product:42")
# Returns the cached value, or None if missing or expired

if product is None:
    product = fetch_product_from_database(42)
    cache_set("product:42", product, ttl=300)
```

cache_delete

```
from tina4_python.cache import cache_delete

cache_delete("product:42")            # returns True if the key existed
```

Real-World Pattern: Cache-Aside

The most common pattern is cache-aside (lazy loading): check the cache, fall back to the database, store the result.

```
from tina4_python.core.router import get
from tina4_python.cache import cache_get, cache_set
from tina4_python.database import Database

@get("/api/products/{product_id}")
async def get_product(request, response):
    product_id = request.params["product_id"]
    cache_key = f"product:{product_id}"

    # 1. Try the cache
    product = cache_get(cache_key)
    if product is not None:
        return response({**product, "source": "cache"})

    # 2. Miss -- fetch from the database
    db = Database()
    product = db.fetch_one(
        "SELECT id, name, category, price, in_stock FROM products WHERE id = ?",
        [product_id]
    )
    if product is None:
        return response({"error": "Product not found"}, 404)

    # 3. Store for next time
    cache_set(cache_key, product, ttl=600)
    return response({**product, "source": "database"})
```

First call returns `"source": "database"`; the next returns `"source": "cache"`.

Key/value statistics

`cache_stats()` from `tina4_python.cache` reports the backend's counters:

```
from tina4_python.cache import cache_stats

stats = cache_stats()

{
```

```
  "hits": 15234,
  "misses": 891,
  "size": 42,
  "backend": "memory"
}
```

The fields are exactly `hits`, `misses`, `size`, and `backend`. Flush everything with `clear_cache()`:

```
from tina4_python.cache import clear_cache

clear_cache()   # flush all entries and reset stats
```

---

8. Cache Invalidation Strategies

Cache invalidation is the hard problem. A stale cache serves outdated data; premature invalidation throws away the performance gain. Three strategies handle the trade-off.

Strategy 1: Time-Based Expiry (TTL)

The simplest. Set a TTL and let the entry expire on its own:

```
cache_set("products:featured", featured_products, ttl=600)  # expires in 10 minutes
```

Good when near-real-time accuracy is fine. A 10-minute delay on the featured list rarely matters.

Strategy 2: Event-Based Invalidation

Clear the cache when the underlying data changes:

```
from tina4_python.core.router import put
from tina4_python.cache import cache_delete
from tina4_python.database import Database

@put("/api/products/{product_id}")
async def update_product(request, response):
    product_id = request.params["product_id"]
    body = request.body
    db = Database()

    db.execute(
        "UPDATE products SET name = ?, price = ? WHERE id = ?",
        [body["name"], body["price"], product_id]
    )

    # Invalidate this product and any list caches that include it
    cache_delete(f"product:{product_id}")
    cache_delete("products:all")
    cache_delete("products:featured")

    updated = db.fetch_one("SELECT * FROM products WHERE id = ?", [product_id])
    return response(updated)
```

The most accurate strategy -- the cache is fresh right after a write. The cost is discipline: you must invalidate everywhere the data could be cached.

Strategy 3: Write-Through Cache

Update the cache at the same moment as the database:

```
@put("/api/products/{product_id}")
async def update_product(request, response):
    product_id = request.params["product_id"]
    body = request.body
    db = Database()

    db.execute(
        "UPDATE products SET name = ?, price = ? WHERE id = ?",
        [body["name"], body["price"], product_id]
    )

    updated = db.fetch_one("SELECT * FROM products WHERE id = ?", [product_id])

    # Write the new data straight to the cache instead of deleting
    cache_set(f"product:{product_id}", updated, ttl=600)
    return response(updated)
```

The cache always holds the latest data, so there is no miss after an update -- the next read comes from the warm cache.

---

9. TTL Management

The right TTL depends on how often the data changes and how much staleness you can tolerate:

Data Type	Suggested TTL	Reasoning
Static config (categories, countries)	3600 (1 hour)	Changes rarely, stale data is harmless
Product catalog	300 (5 min)	Updates several times per day
User profile	60 (1 min)	Users expect changes to appear quickly
Search results	120 (2 min)	Balance freshness and performance
Dashboard stats	30 (30 sec)	Near-real-time but expensive to compute
Exchange rates	60 (1 min)	Updates often, slight delay is acceptable
Shopping cart	0 (no cache)	Must always reflect current state

Dynamic TTL

Adjust the TTL based on the data:

```
from tina4_python.cache import cache_get, cache_set

def get_cached_product(product_id):
    cache_key = f"product:{product_id}"
    product = cache_get(cache_key)
    if product is not None:
        return product

    product = fetch_product_from_database(product_id)

    # Popular products: shorter TTL (more likely to change)
    # Inactive products: longer TTL (rarely change)
    ttl = 60 if product["view_count"] > 1000 else 3600
    cache_set(cache_key, product, ttl=ttl)
    return product
```

---

10. Monitoring Cache Performance

Expose the stats to verify caching is helping:

```
from tina4_python.core.router import get
from tina4_python.cache import cache_stats

@get("/api/cache/stats")
async def get_cache_stats(request, response):
    return response(cache_stats())
```

```
curl http://localhost:7146/api/cache/stats
```

```
{
  "hits": 15234,
  "misses": 891,
  "size": 42,
  "backend": "memory"
}
```

Compute the hit rate yourself: `hits / (hits + misses)`. Above 90% means your strategy works. Below 80% means TTLs are too short, the cache is too small, or you are caching data that is not read often enough to pay off.

---

11. Combining Cache Layers

For maximum performance, stack the layers. Each one catches what the layer above it missed.

```
from datetime import datetime, timezone
from tina4_python.core.router import get
from tina4_python.cache import cache_get, cache_set
from tina4_python.database import Database
import math

@get("/api/catalog", middleware=["ResponseCache:60"])
```

```
async def get_catalog(request, response):
    page = int(request.params.get("page", 1))
    cache_key = f"catalog:page:{page}"

    # Layer 1: key/value cache
    catalog = cache_get(cache_key)
    if catalog is not None:
        return response({**catalog, "cache": "kv"})

    # Layer 2: database query (request-scoped cache always on;
    # persistent cache too if TINA4_DB_CACHE=true)
    db = Database()
    limit = 20
    offset = (page - 1) * limit

    products = db.fetch_all(
        """SELECT p.*, c.name as category_name
           FROM products p
           JOIN categories c ON p.category_id = c.id
           WHERE p.active = 1
           ORDER BY p.created_at DESC
           LIMIT ? OFFSET ?""",
        [limit, offset]
    )
    total = db.fetch_one("SELECT COUNT(*) as count FROM products WHERE active = 1")

    catalog = {
        "products": products,
        "page": page,
        "total": total["count"],
        "pages": math.ceil(total["count"] / limit),
        "generated_at": datetime.now(timezone.utc).isoformat()
    }

    cache_set(cache_key, catalog, ttl=300)
    return response({**catalog, "cache": "none"})
```

The layers, from outermost to innermost:

- **ResponseCache** (60s) -- the whole HTTP response is served from cache. No Python runs.
- **Key/value cache** (300s) -- if the response cache expired, skip the database queries.
- **DB query cache** -- individual query results are cached even when the key/value layer missed.

The first visitor after a full expiry waits 800ms. Everyone else gets the response in under 5ms.

---

12. Exercise: Cache an Expensive Product Listing Endpoint

Build a product listing endpoint that caches at multiple levels.

Requirements

- Create a `GET /api/store/products` endpoint that:
- Accepts query parameters: `category`, `page`, `limit`

- Returns a list of products with pagination metadata
- Uses the key/value API (`cache_get`/`cache_set`) with a 5-minute TTL
- Includes a `source` field in the response (`"cache"` or `"database"`)
- Create a `POST /api/store/products` endpoint that:
- Creates a new product
- Invalidates the relevant cache entries
- Create a `GET /api/store/cache-stats` endpoint that returns cache statistics

Test with:

```
# First call -- cache miss, slow
curl "http://localhost:7146/api/store/products?category=Electronics&page=1"

# Second call -- cache hit, fast
curl "http://localhost:7146/api/store/products?category=Electronics&page=1"

# Different category -- cache miss
curl "http://localhost:7146/api/store/products?category=Fitness&page=1"

# Create a product -- should invalidate cache
curl -X POST http://localhost:7146/api/store/products \
  -H "Content-Type: application/json" \
  -d '{"name": "Smart Watch", "category": "Electronics", "price": 299.99}'

# Same query again -- cache miss (invalidated by the POST)
curl "http://localhost:7146/api/store/products?category=Electronics&page=1"

# Check cache stats
curl http://localhost:7146/api/store/cache-stats
```

---

13. Solution

Create `src/routes/store_cached.py`:

```
import json
import time
import hashlib
from datetime import datetime, timezone
from tina4_python.core.router import get, post
from tina4_python.cache import cache_get, cache_set, cache_delete, cache_stats


def get_product_store():
    return [
        {"id": 1, "name": "Wireless Keyboard", "category": "Electronics", "price": 79.99, "in_st
        {"id": 2, "name": "Yoga Mat", "category": "Fitness", "price": 29.99, "in_stock": True},
        {"id": 3, "name": "Coffee Grinder", "category": "Kitchen", "price": 49.99, "in_stock": F
        {"id": 4, "name": "Standing Desk", "category": "Electronics", "price": 549.99, "in_stock
        {"id": 5, "name": "Running Shoes", "category": "Fitness", "price": 119.99, "in_stock": T
        {"id": 6, "name": "Bluetooth Speaker", "category": "Electronics", "price": 39.99, "in_st
        {"id": 7, "name": "Resistance Bands", "category": "Fitness", "price": 14.99, "in_stock":
        {"id": 8, "name": "French Press", "category": "Kitchen", "price": 34.99, "in_stock": Tru
```

```
    ]


@get("/api/store/products")
async def list_store_products(request, response):
    category = request.params.get("category")
    page = int(request.params.get("page", 1))
    limit = int(request.params.get("limit", 20))

    # Build a cache key from the query parameters
    key_data = json.dumps({"category": category, "page": page, "limit": limit})
    cache_key = f"store:products:{hashlib.md5(key_data.encode()).hexdigest()}"

    # Try the cache first
    cached = cache_get(cache_key)
    if cached is not None:
        return response({**cached, "source": "cache"})

    # Simulate an expensive database query
    time.sleep(0.1)  # 100ms delay

    products = get_product_store()
    if category is not None:
        products = [p for p in products if p["category"].lower() == category.lower()]

    total = len(products)
    offset = (page - 1) * limit
    products = products[offset:offset + limit]

    import math
    result = {
        "products": products,
        "page": page,
        "limit": limit,
        "total": total,
        "pages": math.ceil(total / limit),
        "generated_at": datetime.now(timezone.utc).isoformat()
    }

    # Cache for 5 minutes
    cache_set(cache_key, result, ttl=300)
    return response({**result, "source": "database"})


@post("/api/store/products")
async def create_store_product(request, response):
    body = request.body
    if not body.get("name"):
        return response({"error": "Name is required"}, 400)

    import random
    product = {
        "id": random.randint(100, 9999),
        "name": body["name"],
        "category": body.get("category", "General"),
        "price": float(body.get("price", 0)),
        "in_stock": True
    }
```

```
    # Invalidate all product list caches
    categories = ["Electronics", "Fitness", "Kitchen", "General"]
    for cat in categories:
        for p in range(1, 6):
            key_data = json.dumps({"category": cat, "page": p, "limit": 20})
            cache_delete(f"store:products:{hashlib.md5(key_data.encode()).hexdigest()}")

    # Also invalidate the unfiltered list
    for p in range(1, 6):
        key_data = json.dumps({"category": None, "page": p, "limit": 20})
        cache_delete(f"store:products:{hashlib.md5(key_data.encode()).hexdigest()}")

    return response({
        "message": "Product created",
        "product": product,
        "cache_invalidated": True
    }, 201)


@get("/api/store/cache-stats")
async def store_cache_stats(request, response):
    return response(cache_stats())
```

First call (cache miss):

```
curl "http://localhost:7146/api/store/products?category=Electronics&page=1"
```


```
{
  "products": [
    {"id": 1, "name": "Wireless Keyboard", "category": "Electronics", "price": 79.99, "in_stock"
    {"id": 4, "name": "Standing Desk", "category": "Electronics", "price": 549.99, "in_stock": t
    {"id": 6, "name": "Bluetooth Speaker", "category": "Electronics", "price": 39.99, "in_stock"
  ],
  "page": 1,
  "limit": 20,
  "total": 3,
  "pages": 1,
  "generated_at": "2026-03-22T14:30:00+00:00",
  "source": "database"
}
```


Second call (cache hit): the same payload, but `source` changes from `"database"` to `"cache"`. The handler skipped the simulated query.

---

14. Gotchas

1. Caching Authenticated Responses

Problem: User A's profile is served to User B because the response was cached.

Cause: `ResponseCache` keys on the URL only. If `/api/profile` returns different data per user but every request hits the same URL, the first user's response is served to everyone.

Fix: Do not put `ResponseCache` on user-specific endpoints. Use the key/value API with a per-user key instead: `cache_set(f"profile:{user_id}", data, ttl=300)`.

2. Memory Cache Lost on Restart

Problem: After a restart, performance drops until the cache warms up.

Cause: The memory backend lives in-process and disappears when the process restarts. Every request is a miss until data is cached again.

Fix: In production, use a persistent backend -- `TINA4_CACHE_BACKEND=redis` (shared and durable) or `file` (durable, zero infrastructure). Or run a warmup script that pre-populates hot keys on startup.

3. Stale Data After a Database Update

Problem: You updated a product's price, but the API still returns the old one.

Cause: A cached value still holds the old data and has not expired.

Fix: Invalidate or update the cache when you change the underlying data. Use `cache_delete(...)` after a write, or write through with `cache_set(...)`. The request-scoped DB cache already flushes on writes within the same request; the key/value and persistent caches do not -- you manage those.

4. Cache Key Collisions

Problem: Two different queries return the same cached data.

Cause: The keys are not specific enough. Using `"products"` for both the full list and a filtered list collides.

Fix: Put every relevant parameter in the key -- `"products:category:Electronics:page:1:limit:20"` -- or hash the parameters: `f"products:{hashlib.md5(json.dumps(params).encode()).hexdigest()}"`.

5. Serialization Overhead

Problem: Caching makes some requests slower, not faster.

Cause: The cached object is large. Serializing and deserializing it costs more than recomputing it.

Fix: Cache only what is expensive to produce. If the original work takes 5ms and cache serialization takes 10ms, caching loses. Measure before and after.

6. Forgetting the TTL on the Memory Backend

Problem: Cache entries never expire and memory grows.

Cause: You called `cache_set("key", value, ttl=0)` (or relied on a zero TTL). A zero TTL means no expiry, so the entry lives until eviction or restart.

Fix: Pass a real TTL: `cache_set("key", value, ttl=300)`. Even for data that "never changes," use a long TTL like 86400 (24 hours) as a safety net. The memory backend caps total entries at `TINA4_CACHE_MAX_ENTRIES` and evicts the least-recently-used entry when full.

Queues

1. Not Everything Should Happen Right Now

Some tasks are too slow for an HTTP request. Sending an email: 2 seconds. Generating a PDF report: 10 seconds. Processing a large CSV upload: a minute. Run these inside a route handler and the user stares at a spinner. Or the request times out.

Queues solve this. Push a message describing the work. A separate worker picks it up and does the job in the background. The user gets an instant response: "Your report is being generated."

Picture a store that sends order confirmations, generates invoices, and syncs inventory with a warehouse. None of these should block checkout. Each one becomes a queue message. A worker processes it on its own schedule.

---

2. Queue Configuration

Tina4 Python includes a built-in queue that requires zero additional setup. The default backend is file-based.

Default (File Queue)

The file queue works out of the box with no extra configuration:

```
# No TINA4_QUEUE_BACKEND needed -- file is the default
```

The first time you push a message, Tina4 creates the queue storage.

Switching to RabbitMQ

When your application outgrows the file queue (millions of messages, multiple services, pub/sub patterns), switch to RabbitMQ:

```
TINA4_QUEUE_BACKEND=rabbitmq
TINA4_QUEUE_URL=amqp://user:pass@localhost:5672
```

Install the client library:

```
uv add pika
```

Switching to Kafka

For stream processing, event sourcing, or very high throughput:

```
TINA4_QUEUE_BACKEND=kafka
TINA4_KAFKA_BROKERS=localhost:9092
```

Install the client library:

```
uv add confluent-kafka
```

Switching to MongoDB

```
TINA4_QUEUE_BACKEND=mongodb
TINA4_QUEUE_URL=mongodb://user:pass@localhost:27017/tina4
```

Install the driver:

```
uv add pymongo
```

The backend is configured through environment variables, but the providers do not pretend to support operations their brokers cannot perform. The portable core is message delivery and acknowledgement; management operations depend on the provider.

Provider	Supported contract	Explicitly unavailable
File / lite	Every queue operation	None
MongoDB	Every queue operation	None
RabbitMQ	Push, consume, complete, fail/reject, stable non-destructive dead letters, `retry()`, broker size (a count the broker reports, which can trail a push by a moment), close	Priority, delay, pop by ID, `purge()`, `clear()`, failed-job listing, retry failed
Kafka	Delivery, acknowledge, fail, stable non-destructive dead letters; `size()` honestly returns `0`	Priority, delay, pop by ID, `purge()`, `clear()`, failed-job listing, retry failed

This capability contract is defined by ADR-0022, ADR-0023, and ADR-0024. An unsupported operation raises an error naming the backend and operation. It never succeeds as a no-op or returns a misleading empty result.

Kafka has two more habits worth knowing before you test against it. It keeps no queue depth, so `size()` is always `0` there, even with messages waiting. And the very first `pop()` on a topic has to wait for the broker to assign the consumer group its partitions. Tina4 waits up to `TINA4_KAFKA_ASSIGN_TIMEOUT` seconds (default `15`) for that on the first call, so the first job can take a few seconds to arrive, and after that each poll is quick.

---

3. Creating a Queue and Pushing Messages

```
from tina4_python.queue import Queue

queue = Queue(topic="emails")

# Push a message
message_id = queue.push({
    "to": "alice@example.com",
    "subject": "Order Confirmation",
    "body": "Your order #1234 has been confirmed."
})
```

The `topic` argument names the queue. The payload is any dictionary that can be serialized to JSON.

> ***Backend configuration:*** *Set* `TINA4_QUEUE_BACKEND` *to* `file` *(default),* `rabbitmq`*,* `kafka`*, or* `mongodb`*, and every* `Queue` *you build picks it up. For the file backend, the* `TINA4_QUEUE_PATH` *environment variable controls the storage directory (default: data/queue). The constructor also takes a* `backend` *argument, and an explicit value wins over the environment:* `Queue(topic="emails", backend="file")` *stays on the file backend whatever* `TINA4_QUEUE_BACKEND` *says. Leave it out and your code moves between environments with a* `.env` *change alone. See Section 2 and Section 9 for full details.*

Priority and Delay

`push` accepts two optional arguments:

```
# Higher priority is processed first
queue.push({"to": "vip@example.com", "subject": "Urgent"}, priority=10)

# Hold the job for 60 seconds before it becomes available
queue.push({"to": "later@example.com", "subject": "Reminder"}, delay_seconds=60)
```

`priority` defaults to `0`. `pop` and `consume` return the highest-priority job first; ties break oldest-first. See Section 5.

`delay_seconds` defaults to `0`. The file backend honors the delay; the job stays hidden until the time arrives. External brokers (RabbitMQ, Kafka, MongoDB) manage their own delivery timing.

Convenience Method: produce

The `produce` method pushes to a specific topic without creating a separate Queue instance:

```
queue = Queue(topic="emails")
queue.produce("invoices", {"order_id": 101, "format": "pdf"})
```

Queue Size

Check how many pending messages are in the queue:

```
count = queue.size()
```

---

4. Consuming Messages

The consume Pattern

The `consume` method is a generator that yields jobs one at a time. It polls the queue continuously and sleeps when empty, so you need no outer loop. Each job must be explicitly completed or failed:

```
from tina4_python.queue import Queue

queue = Queue(topic="emails")

for job in queue.consume("emails"):
    try:
        send_email(job.payload["to"], job.payload["subject"], job.payload["body"])
        job.complete()
    except Exception as e:
        job.fail(str(e))
```

This loop runs forever, processing jobs as they arrive. To drain the queue once and stop when it is empty, pass `poll_interval=0`:

```
for job in queue.consume("emails", poll_interval=0):
    process(job)
    job.complete()
```

You can also stop after a fixed number of jobs with `iterations`:

```
for job in queue.consume("emails", iterations=5):
    process(job)
    job.complete()
```

Consume a Single Job by ID

Pass `job_id` to process one specific job. It yields that job once, then returns:

```
for job in queue.consume("emails", job_id="specific-job-id"):
    process(job)
    job.complete()
```

This works on the file and MongoDB backends. RabbitMQ and Kafka can't pick one message out of the middle of a queue, so there `consume(job_id=...)` (and `queue.pop_by_id()`) raises `NotImplementedError` naming the backend, rather than quietly yielding nothing.

Manual Pop

For full control, pop a single message. `pop` returns the highest-priority available job, or `None` when the queue is empty:

```
job = queue.pop()

if job is not None:
    try:
        send_email(job.payload["to"], job.payload["subject"])
        job.complete()
    except Exception as e:
        job.fail(str(e))
```

---

5. Priority Ordering

`pop` and `consume` do not return jobs in plain insert order. They return the **highest-priority** available job first. When two jobs share the same priority, the **older** one wins.

```
queue = Queue(topic="tasks")

queue.push({"label": "normal"})                # priority 0
queue.push({"label": "urgent"}, priority=10)   # priority 10
queue.push({"label": "also normal"})           # priority 0

queue.pop().payload["label"]   # "urgent"        (highest priority)
queue.pop().payload["label"]   # "normal"        (oldest of the priority-0 pair)
queue.pop().payload["label"]   # "also normal"
```

A delayed job (pushed with `delay_seconds`) stays hidden until its time arrives, regardless of priority.

Priority ordering is enforced by the file backend. External brokers store the priority on each message but follow their own delivery semantics.

---

6. Job Lifecycle

A job moves through these statuses:

```
push() -> PENDING -> pop()/consume() -> job.complete() -> done (removed)
                                      -> job.fail()     -> attempts += 1
                                                                |
                                            attempts < max_retries
                                                                |
                                                      re-enqueued -> PENDING
                                                                |
                                                 attempts >= max_retries
                                                                |
                                                            DEAD LETTER
```

Job Methods

When you receive a job from `consume` or `pop`, you have these methods:

- `job.complete()` -- mark the job as done. Terminal: the job is removed and never comes back.
- `job.fail(reason)` -- record a failed attempt. Increments `attempts` and either re-enqueues the job or dead-letters it (see Section 7).
- `job.reject(reason)` -- alias for `fail`.
- `job.retry(delay_seconds=0)` -- manually re-queue the job, optionally after a delay. Bypasses the retry limit.

Read the payload with `job.payload`. The fields `job.id`, `job.attempts`, and `job.error` are also available.

Always call `complete()` or `fail()`. If you call neither, the job has already left the queue (it was claimed on pop) and will not be retried.

---

7. Automatic Retry and Dead Letters

How Retry Works

`job.fail()` does the retry bookkeeping for you. Each call increments the job's `attempts` count. While `attempts` is below `max_retries`, the job is automatically re-enqueued, so the next `pop()` or `consume()` picks it up again. Once `attempts` reaches `max_retries`, the job moves to the dead-letter store.

This means a normal `consume` loop retries failed jobs on its own. No manual `retry_failed()` call is needed:

```
queue = Queue(topic="emails", max_retries=3)

for job in queue.consume("emails"):
    try:
        send_email(job.payload)
        job.complete()
    except Exception as e:
        job.fail(str(e))   # retried up to 3 times, then dead-lettered
```

With `max_retries=3`, a job that keeps failing is attempted 3 times. On the third failure it lands in dead letters.

Retry Backoff

By default a failed job is re-enqueued immediately and the next loop iteration retries it. To space retries out, pass `retry_backoff` (in seconds) to the constructor:

```
queue = Queue(topic="emails", max_retries=5, retry_backoff=30)
```

Now each automatic re-enqueue holds the job for 30 seconds before it becomes available again. `retry_backoff` applies to the file backend.

Configuring Max Retries

The `Queue` constructor accepts `max_retries` (default: 3):

```
queue = Queue(topic="emails", max_retries=5)
```

Inspecting Failed and Dead Jobs

Two methods let you see where jobs are in the retry cycle:

```
# Jobs that failed at least once but are still being retried
# (0 < attempts < max_retries)
retrying = queue.failed()

# Jobs that exhausted their retries (attempts >= max_retries)
dead_jobs = queue.dead_letters()
```

`failed()` returns plain dicts. `dead_letters()` returns `Job` objects, so you can iterate them like any other job:

```
for job in queue.dead_letters():
    print(f"Dead job: {job.id}")
    print(f"  Payload: {job.payload}")
    print(f"  Attempts: {job.attempts}")
    print(f"  Error: {job.error}")
```

Reviving Dead Letters

Auto-retry stops at the dead-letter store. To put dead jobs back on the queue, do it explicitly:

```
# Re-queue every dead-letter job
queue.retry()

# Re-queue one specific job by ID
queue.retry(job_id)

# Re-queue dead jobs that are still under the retry limit
queue.retry_failed()
```

Counting and Purging by Status

`size` and `purge` accept a status. Two sets matter: the pending queue and the dead-letter store.

```
queue.size("pending")    # jobs waiting to be processed, retries included
queue.size("dead")       # dead-letter jobs

queue.purge("pending")   # drop everything still waiting
queue.purge("dead")      # clear the dead-letter store
```

`"failed"` is an alias for `"dead"` here. `size("failed")` counts the dead-letter store and `purge("failed")` empties it. Neither touches the jobs `failed()` lists. Those jobs are still being retried, so they sit in the pending queue, and `size("pending")` already counts them. To see them on their own, call `failed()`.

---

8. Queue in Route Handlers

The most common pattern is pushing messages from route handlers:

```
from tina4_python.core.router import post, noauth
from tina4_python.queue import Queue

queue = Queue(topic="emails")

@noauth()   # open for this walkthrough only; see the note below
@post("/api/orders")
async def create_order(request, response):
    body = request.body

    # Create the order in the database
    order_id = 101  # Simulated

    # Send confirmation email
    queue.push({
```

```
        "type": "order_confirmation",
        "to": body["email"],
        "order_id": order_id,
        "total": body["total"]
    })

    # Generate invoice on a different topic
    queue.produce("invoices", {
        "order_id": order_id,
        "format": "pdf"
    })

    # Sync with warehouse on a different topic
    queue.produce("warehouse_sync", {
        "order_id": order_id,
        "items": body["items"]
    })

    return response.json({
        "message": "Order created",
        "order_id": order_id
    }, 201)
```

The user gets an instant response. The email, invoice, and warehouse sync happen in the background.

Tina4 secures every `POST`, `PUT`, `PATCH` and `DELETE` route by default. Without `@noauth()` this route answers `401` to any request that doesn't carry a valid `Authorization: Bearer <token>` header, and the queue code never runs. The decorator opens it so you can try it with a plain `curl`. A real checkout route keeps its protection and takes a token (Chapter 8).

---

9. Switching Backends via .env

Switching backends is a config change, not a code change.

Development: File (default)

```
# No config needed -- file is the default
```

Production: RabbitMQ

```
TINA4_QUEUE_BACKEND=rabbitmq
TINA4_QUEUE_URL=amqp://user:pass@rabbitmq.internal:5672
```

High-Scale Production: Kafka

```
TINA4_QUEUE_BACKEND=kafka
TINA4_KAFKA_BROKERS=kafka-1:9092,kafka-2:9092,kafka-3:9092
```

Production: MongoDB

```
TINA4_QUEUE_BACKEND=mongodb
TINA4_QUEUE_URL=mongodb://user:pass@mongo.internal:27017/tina4
```

Environment variables the queue reads

Variable	Used by	Purpose
`TINA4_QUEUE_BACKEND`	all	Selects the backend: `file` (default), `rabbitmq`, `kafka`, `mongodb`
`TINA4_QUEUE_PATH`	file	Storage directory for the file backend (default: `data/queue`)
`TINA4_QUEUE_URL`	rabbitmq, mongodb, kafka	Connection URL for the broker
`TINA4_KAFKA_BROKERS`	kafka	Comma-separated broker list (overrides `TINA4_QUEUE_URL`)

Switching backends is a `.env` change, not a code change, for the core of the queue: `push()`, `consume()`, `pop()`, `job.complete()`, `job.fail()` and `dead_letters()` behave the same on all four. The management calls are where brokers differ. Check the table in Section 2 before you lean on priority, delay, `consume(job_id=...)`, `purge()`, `clear()`, `failed()` or `retry_failed()` against RabbitMQ or Kafka. Those raise an error that names the backend, so you'll find out on the first call, not in production data.

---

10. Produce and Consume Across Topics

The `Queue` class provides `produce()` and `consume()` methods for cross-topic messaging:

```
from tina4_python.queue import Queue

queue = Queue(topic="default")

# Produce onto a specific topic
queue.produce("emails", {"to": "alice@example.com", "subject": "Hello"})

# Consume from a specific topic, then stop once it is empty
for job in queue.consume("emails", poll_interval=0):
    process(job)
    job.complete()
```

The `produce()` method pushes a job onto any named topic. The `consume()` method yields available jobs from a topic as a generator. `poll_interval=0` makes it a single pass, as in Section 4, so a script that runs this block finishes. Leave it out and `consume()` keeps polling for new jobs forever, which is what a worker wants and what a one-off script doesn't.

---

11. Exercise: Build an Email Queue

Build an email queue system that sends emails in the background, including failure handling.

Requirements

- Create these endpoints:

Method	Path	Description
`POST`	`/api/emails/send`	Queue an email for sending
`GET`	`/api/emails/queue`	List pending email count
`GET`	`/api/emails/dead`	List dead letter jobs
`POST`	`/api/emails/retry`	Revive dead-letter jobs

- The email payload should include: `to` (required), `subject` (required), `body` (required)
- Create a consumer that processes the queue, simulating occasional failures
- When an email to a specific address fails repeatedly, it should end up in dead letters

Test with:

```
# Queue an email
curl -X POST http://localhost:7146/api/emails/send \
  -H "Content-Type: application/json" \
  -d '{"to": "alice@example.com", "subject": "Welcome!", "body": "Thanks for signing up."}'

# Check queue size
curl http://localhost:7146/api/emails/queue

# Check dead letters
curl http://localhost:7146/api/emails/dead

# Revive dead letters
curl -X POST http://localhost:7146/api/emails/retry
```

---

12. Solution

Create `src/routes/email_queue.py`:

```
from tina4_python.core.router import get, post, noauth
from tina4_python.queue import Queue

queue = Queue(topic="emails", max_retries=3)


@noauth()   # open so the curl tests work; protect it with a token in a real app
@post("/api/emails/send")
async def queue_email(request, response):
    body = request.body

    errors = []
```

```
    if not body.get("to"):
        errors.append("'to' is required")
    if not body.get("subject"):
        errors.append("'subject' is required")
    if not body.get("body"):
        errors.append("'body' is required")

    if errors:
        return response.json({"errors": errors}, 400)

    message_id = queue.push({
        "to": body["to"],
        "subject": body["subject"],
        "body": body["body"]
    })

    return response.json({
        "message": "Email queued for sending",
        "message_id": message_id
    }, 201)


@get("/api/emails/queue")
async def email_queue_size(request, response):
    count = queue.size()
    return response.json({"pending": count})


@get("/api/emails/dead")
async def email_dead_letters(request, response):
    items = []
    for job in queue.dead_letters():
        items.append({
            "id": job.id,
            "payload": job.payload,
            "attempts": job.attempts,
            "error": job.error
        })
    return response.json({"dead_letters": items, "count": len(items)})


@noauth()
@post("/api/emails/retry")
async def retry_dead_emails(request, response):
    queue.retry()
    return response.json({"message": "Dead-letter emails re-queued"})
```

Create the consumer as its own script in the project root, `email_worker.py`, **not** under `src/`. When `tina4 serve` starts it imports every `.py` file under `src/` to find routes and models. A worker there would be imported too, its endless `consume()` loop would start inside the server, and the server would never finish booting.

```
from tina4_python.dotenv import load_env
from tina4_python.queue import Queue
import time


def run():
    queue = Queue(topic="emails", max_retries=3)
```

```
    for job in queue.consume("emails"):
        payload = job.payload

        print(f"Sending email to {payload['to']}...")
        print(f"  Subject: {payload['subject']}")
        print(f"  Body: {payload['body'][:50]}...")

        try:
            # Simulate sending (replace with real email logic)
            time.sleep(1)

            # Simulate failure for a specific address
            if payload["to"] == "bad@example.com":
                raise Exception("SMTP connection refused")

            print(f"  Email sent to {payload['to']} successfully!")
            job.complete()

        except Exception as e:
            print(f"  Failed: {e}")
            job.fail(str(e))


if __name__ == "__main__":
    load_env()   # same .env as the server, so both use the same queue backend
    run()
```

Run it in a second terminal, next to `tina4 serve`:

```
uv run python email_worker.py
```

The worker polls until you stop it with Ctrl-C. The `if __name__ == "__main__":` guard means importing the file never starts the loop, so nothing breaks if it ends up on an import path by accident.

The consumer loop retries on its own. A job to `bad@example.com` fails, gets re-enqueued, and is retried. After three attempts `queue.dead_letters()` returns it and the `/api/emails/dead` endpoint shows it. You investigate, fix the address, and call `/api/emails/retry` to put it back on the queue.

---

13. Gotchas

1. Always call complete or fail

Problem: A failed job is never retried, or you lose track of it.

Cause: Your consumer does not call `job.complete()` or `job.fail()`. The job was claimed on pop, so it has already left the pending queue; without `fail()` it is neither retried nor dead-lettered.

Fix: Always call one of `job.complete()`, `job.fail(reason)`, or `job.reject(reason)` in your consumer loop. `fail()` handles the retry and dead-letter bookkeeping for you.

2. Worker not picking up messages

Problem: Messages are pushed but nothing happens.

Cause: No consumer process is running, or the consumer is listening on a different topic.

Fix: Make sure the consumer is running. Check that the topic name in `queue.push()` matches the topic name in `queue.consume()`.

3. Payload too large

Problem: Pushing a message with a large payload is slow.

Cause: The payload is serialized to JSON and stored in the backend. Very large payloads slow down the queue.

Fix: Keep payloads small. Store files on disk or in object storage and put the file path in the payload. Payloads should be metadata, not data.

4. Dead letters pile up

Problem: Dead letters accumulate and nobody notices.

Cause: Jobs that exhaust `max_retries` become dead letters and stay there until you act.

Fix: Monitor dead letters with `queue.dead_letters()` or `queue.size("dead")`. Set up an alert when the count exceeds a threshold. Investigate the root cause, fix it, then call `queue.retry()` to revive them or `queue.purge("dead")` to clear them.

5. File backend for production

Problem: Multiple workers cause contention on file-based queue storage.

Cause: File-based storage supports one writer at a time.

Fix: For production with multiple workers, switch to RabbitMQ, Kafka, or MongoDB via the `TINA4_QUEUE_BACKEND` environment variable. The file backend is fine for development and single-worker setups.

6. Environment-specific topic collision

Problem: Development and staging environments process each other's messages.

Cause: Both environments use the same backend and the same topic names.

Fix: Use separate backends per environment, or prefix topic names with the environment: `Queue(topic="dev_emails")`.

Events

1. Decouple Everything with Events

Your order route creates an order, sends a confirmation email, deducts inventory, and notifies the warehouse. Six hundred lines of intertwined logic. Add a new requirement, loyalty points, and you are touching the order route again.

Events flip this around. The order route fires `order.placed` and walks away. Separate handlers pick it up: one sends the email, another deducts inventory, a third notifies the warehouse. None of them know about each other.

Tina4's event system is synchronous by default and supports async via `emit_async`. No broker required.

---

2. Registering Handlers with @on

```
from tina4_python.core.events import on, emit

@on("user.created")
def welcome_email(payload):
    print(f"Sending welcome email to {payload['email']}")

@on("user.created")
def create_profile(payload):
    print(f"Creating profile for {payload['user_id']}")
```

Register as many handlers as you need for the same event. They all run when the event fires.

---

3. Firing Events with emit

```
from tina4_python.core.router import post
from tina4_python.core.events import emit

@post("/api/users")
async def register_user(request, response):
    body = request.body

    user = {
        "user_id": 42,
        "email": body["email"],
        "name": body["name"]
    }

    # Fire the event -- all registered handlers run now
    emit("user.created", user)

    return response({"message": "User created", "user_id": user["user_id"]}, 201)
```

```
curl -X POST http://localhost:7146/api/users \
  -H "Content-Type: application/json" \
```

```
  -d '{"email": "alice@example.com", "name": "Alice"}'
```

Output in the server log:

```
Sending welcome email to alice@example.com
Creating profile for 42
```

---

4. One-Time Handlers with @once

A handler registered with `once` fires exactly once and then automatically unregisters itself:

```
from tina4_python.core.events import once, emit

@once("app.started")
def run_migrations(payload):
    print("Running database migrations...")

emit("app.started", {})   # Runs run_migrations
emit("app.started", {})   # Does nothing -- handler was removed
```

Useful for initialisation tasks, first-visit tracking, or features that should only trigger once per session.

---

5. Removing Handlers with off

Remove a handler by function reference:

```
from tina4_python.core.events import on, off, emit

def log_purchase(payload):
    print(f"Purchase logged: {payload['order_id']}")

on("order.placed", log_purchase)

emit("order.placed", {"order_id": 101})  # Handler runs

off("order.placed", log_purchase)

emit("order.placed", {"order_id": 102})  # Handler does not run
```

---

6. Priority Ordering

When multiple handlers listen to the same event, you control which one runs first using the `priority` parameter. Lower numbers run first. Default priority is `10`.

```
from tina4_python.core.events import on, emit

@on("order.placed", priority=1)
def validate_stock(payload):
    print(f"[priority 1] Validating stock for order {payload['order_id']}")

@on("order.placed", priority=5)
```

```
def charge_payment(payload):
    print(f"[priority 5] Charging payment for order {payload['order_id']}")

@on("order.placed", priority=10)
def send_confirmation(payload):
    print(f"[priority 10] Sending confirmation for order {payload['order_id']}")

emit("order.placed", {"order_id": 200})
```

Output:

```
[priority 1] Validating stock for order 200
[priority 5] Charging payment for order 200
[priority 10] Sending confirmation for order 200
```

Stock is validated before payment is charged, payment is charged before the confirmation email goes out.

---

7. Async Event Support with emit_async

For handlers that do I/O (sending HTTP requests, writing to a database, publishing to a message broker) use `emit_async` with async handlers:

```
import asyncio
from tina4_python.core.events import on, emit_async

@on("order.placed")
async def notify_warehouse(payload):
    # Simulate async HTTP call to warehouse API
    await asyncio.sleep(0.1)
    print(f"Warehouse notified for order {payload['order_id']}")

@on("order.placed")
async def update_analytics(payload):
    await asyncio.sleep(0.05)
    print(f"Analytics updated for order {payload['order_id']}")
```

Call `emit_async` from an async route handler:

```
from tina4_python.core.router import post
from tina4_python.core.events import emit_async

@post("/api/orders")
async def create_order(request, response):
    body = request.body

    order = {
        "order_id": 300,
        "customer_id": body["customer_id"],
        "total": body["total"],
        "items": body["items"]
    }

    # All async handlers run concurrently
    await emit_async("order.placed", order)

    return response({"message": "Order placed", "order_id": order["order_id"]}, 201)
```

`emit_async` gathers all async handlers and runs them concurrently using `asyncio.gather`. Synchronous handlers registered on the same event run first (in priority order), then all async handlers run in parallel.

---

8. Real-World Pattern: Order Pipeline

A realistic e-commerce order with five independent concerns:

```
from tina4_python.core.events import on, emit_async

@on("order.placed", priority=1)
def reserve_stock(payload):
    print(f"Reserving {len(payload['items'])} items")

@on("order.placed", priority=2)
def charge_customer(payload):
    print(f"Charging ${payload['total']} to customer {payload['customer_id']}")

@on("order.placed", priority=5)
async def send_confirmation_email(payload):
    import asyncio
    await asyncio.sleep(0)  # Yield to event loop
    print(f"Confirmation sent to {payload['email']}")

@on("order.placed", priority=5)
async def sync_warehouse(payload):
    import asyncio
    await asyncio.sleep(0)
    print(f"Warehouse sync queued for order {payload['order_id']}")

@on("order.placed", priority=10)
def award_loyalty_points(payload):
    points = int(payload['total'] * 10)
    print(f"Awarding {points} loyalty points to customer {payload['customer_id']}")
```

The order route:

```
from tina4_python.core.router import post
from tina4_python.core.events import emit_async

@post("/api/orders")
async def place_order(request, response):
    body = request.body

    order = {
        "order_id": 301,
        "customer_id": body["customer_id"],
        "email": body["email"],
        "total": body["total"],
        "items": body["items"]
    }

    await emit_async("order.placed", order)

    return response({"order_id": order["order_id"], "status": "placed"}, 201)
```

Adding loyalty points or a fraud check means adding a new `@on` handler. The route file never changes.

---

9. Exercise: User Lifecycle Events

Build a user registration flow using events.

Requirements

- Create a `POST /api/register` endpoint that:
- Accepts `email`, `name`, `password`
- Emits `user.created` with `{user_id, email, name}`
- Returns `{"user_id": ..., "status": "created"}`
- Register three handlers for `user.created`:
- Priority 1: log the event with timestamp
- Priority 2: simulate sending a welcome email
- Priority 5: simulate creating a default profile
- Create a `POST /api/users/{user_id}/deactivate` endpoint that emits `user.deactivated`
- Register a one-time handler for `app.ready` that prints a startup message

Test with:

```
# Register a user
curl -X POST http://localhost:7146/api/register \
  -H "Content-Type: application/json" \
  -d '{"email": "bob@example.com", "name": "Bob", "password": "secret"}'

# Deactivate a user
curl -X POST http://localhost:7146/api/users/42/deactivate
```

---

10. Solution

Create `src/routes/user_events.py`:

```
from datetime import datetime, timezone
from tina4_python.core.router import post
from tina4_python.core.events import on, once, emit, emit_async


@once("app.ready")
def on_app_ready(payload):
    print(f"[{datetime.now(timezone.utc).isoformat()}] Application ready.")


@on("user.created", priority=1)
def log_user_created(payload):
```

```
    print(f"[{datetime.now(timezone.utc).isoformat()}] user.created: {payload['email']}")


@on("user.created", priority=2)
def send_welcome_email(payload):
    print(f"Welcome email dispatched to {payload['email']}")


@on("user.created", priority=5)
def create_default_profile(payload):
    print(f"Default profile created for user {payload['user_id']}")


@on("user.deactivated")
def log_deactivation(payload):
    print(f"User {payload['user_id']} deactivated at {datetime.now(timezone.utc).isoformat()}")


@post("/api/register")
async def register_user(request, response):
    body = request.body

    if not body.get("email") or not body.get("name"):
        return response({"error": "email and name are required"}, 400)

    user = {
        "user_id": 42,
        "email": body["email"],
        "name": body["name"]
    }

    emit("user.created", user)

    return response({"user_id": user["user_id"], "status": "created"}, 201)


@post("/api/users/{user_id}/deactivate")
async def deactivate_user(request, response):
    user_id = request.params["user_id"]

    emit("user.deactivated", {"user_id": user_id})

    return response({"user_id": user_id, "status": "deactivated"})


# Runs when Tina4 imports this file at startup, after the handler above is registered
emit("app.ready", {})
```

The `emit("app.ready", {})` sits at the bottom of the same file, below the `@once("app.ready")` handler. Tina4 loads the `.py` files under `src/` in sorted path order, so an emit in a file that loads earlier, such as `src/app.py`, fires before the handler is registered and nothing prints. The emit must come after its handler. In development, a hot reload that re-imports this file fires it again.

---

11. Gotchas

1. Emitting from a sync context with async handlers

Problem: `emit()` is called from a synchronous function but some handlers are `async def`. They will not run.

Fix: Use `emit_async()` from an async context for events that have async handlers. Synchronous handlers always run regardless of which emit variant you use.

2. Handler order is non-deterministic at equal priority

Problem: Two handlers at `priority=5` run in a different order than expected.

Fix: Give them distinct priority values. Priority uniqueness within an event gives you guaranteed ordering.

3. Forgetting once fires exactly once

Problem: A handler registered with `@once` runs for the first `emit` of its event and never again in that process, so a second `emit("app.ready", {})` reaches nothing.

Fix: `@once` handlers remove themselves after firing. For a handler that must run on every emit, use `@on` instead.

4. Mutating the payload in a handler

Problem: A high-priority handler modifies the payload dict and downstream handlers see the mutated version.

Fix: This is intentional and useful (e.g., adding `payment_id` to the payload before the confirmation handler runs). If you want to prevent mutation, pass `payload.copy()` or use a frozen data structure.

Localization

1. One App, Many Languages

Your application is live. A customer in Berlin arrives and sees English. A customer in Tokyo sees English. A customer in São Paulo sees English.

Tina4 localization lets you write translations once and switch the active locale at runtime. Store JSON files in `src/locales/`, call `t()` at the point of use, and set the active language from a query parameter, a cookie, or an environment variable. No third-party libraries required.

---

2. Locale Files

Create one JSON file per locale in `src/locales/`. The filename is the locale code.

```
src/
  locales/
    en.json
    de.json
    ja.json
    pt.json
```

`src/locales/en.json`:

```
{
  "welcome": "Welcome",
  "greeting": "Hello, {name}!",
  "order": {
    "placed": "Your order has been placed.",
    "total": "Order total: {currency}{amount}",
    "item_count": "{count} item in your order",
    "item_count_plural": "{count} items in your order"
  },
  "errors": {
    "not_found": "The requested resource was not found.",
    "unauthorized": "You are not authorized to access this resource."
  }
}
```

`src/locales/de.json`:

```
{
  "welcome": "Willkommen",
  "greeting": "Hallo, {name}!",
  "order": {
    "placed": "Ihre Bestellung wurde aufgegeben.",
    "total": "Bestellsumme: {currency}{amount}",
    "item_count": "{count} Artikel in Ihrer Bestellung",
    "item_count_plural": "{count} Artikel in Ihrer Bestellung"
  },
  "errors": {
    "not_found": "Die angeforderte Ressource wurde nicht gefunden.",
    "unauthorized": "Sie sind nicht berechtigt, auf diese Ressource zuzugreifen."
  }
```

```
}
```

---

3. The t() Function

```
from tina4_python.i18n import t

# Simple key
message = t("welcome")
# "Welcome"

# Nested key with dot notation
error = t("errors.not_found")
# "The requested resource was not found."

# Interpolation with {placeholder}
greeting = t("greeting", name="Alice")
# "Hello, Alice!"

order_total = t("order.total", currency="$", amount="99.99")
# "Order total: $99.99"
```

Keys use dot notation for nested structures. Placeholders in curly braces are replaced with keyword arguments.

---

4. The I18n Class

For more control (setting the locale per request, loading custom locale paths) use the `I18n` class directly:

```
from tina4_python.i18n import I18n

i18n = I18n(locale="de", path="src/locales")

print(i18n.t("welcome"))
# "Willkommen"

print(i18n.t("greeting", name="Max"))
# "Hallo, Max!"

# Switch locale at runtime
i18n.set_locale("en")
print(i18n.t("welcome"))
# "Welcome"
```

The `I18n` class loads locale files lazily. Setting the locale to `"de"` loads `src/locales/de.json` the first time a translation is requested. Subsequent requests for the same locale use the in-memory cache.

---

5. Setting the Locale at Runtime

From a Query Parameter

The simplest approach: read `?lang=` from the request and set the locale.

```
from tina4_python.core.router import get
from tina4_python.i18n import I18n

@get("/api/welcome")
async def welcome(request, response):
    lang = request.params.get("lang", "en")
    i18n = I18n(locale=lang, path="src/locales")

    return response({
        "message": i18n.t("greeting", name="World"),
        "locale": lang
    })
```

```
curl "http://localhost:7146/api/welcome?lang=en"
# {"message": "Hello, World!", "locale": "en"}

curl "http://localhost:7146/api/welcome?lang=de"
# {"message": "Hallo, World!", "locale": "de"}
```

From the Accept-Language Header

A more standard approach uses the browser's language preference:

```
from tina4_python.core.router import get
from tina4_python.i18n import I18n

def detect_locale(request, default="en"):
    accept = request.headers.get("Accept-Language", default)
    # "de-DE,de;q=0.9,en;q=0.8" -> "de"
    primary = accept.split(",")[0].split("-")[0].strip()
    supported = ["en", "de", "ja", "pt"]
    return primary if primary in supported else default

@get("/api/dashboard")
async def dashboard(request, response):
    locale = detect_locale(request)
    i18n = I18n(locale=locale, path="src/locales")

    return response({
        "welcome": i18n.t("welcome"),
        "locale": locale
    })
```

From a Session or Cookie

```
@get("/api/home")
async def home(request, response):
    locale = request.session.get("locale", "en")
    i18n = I18n(locale=locale, path="src/locales")

    return response({"message": i18n.t("welcome")})

@post("/api/locale")
async def set_locale(request, response):
    locale = request.body.get("locale", "en")
    request.session["locale"] = locale
    return response({"locale": locale})
```

---

6. The TINA4_LOCALE Environment Variable

Set the application's default locale in `.env`:

```
TINA4_LOCALE=de
```

When `TINA4_LOCALE` is set, calls to `t()` without an explicit locale use German as the default. This is the locale loaded at application startup.

```
from tina4_python.i18n import t

# Uses TINA4_LOCALE from .env (e.g., "de")
message = t("welcome")
# "Willkommen"
```

---

7. Interpolation with {placeholder}

Placeholders are written as `{name}` in the JSON file and filled via keyword arguments to `t()`:

```
from tina4_python.i18n import I18n

i18n = I18n(locale="en", path="src/locales")

# Single placeholder
greeting = i18n.t("greeting", name="Alice")
# "Hello, Alice!"

# Multiple placeholders
total = i18n.t("order.total", currency="€", amount="149.99")
# "Order total: €149.99"
```

Missing placeholders are left as-is:

```
i18n.t("order.total", currency="$")
# "Order total: ${amount}"  -- {amount} not supplied
```

Extra keyword arguments are silently ignored.

---

8. Fallback Behaviour

When a key is missing from the active locale, Tina4 falls back through a chain:

- Active locale (`de`)
- Base language (`de` if locale was `de_AT`)
- Default locale (`en`, or whatever you passed as `default_locale=` when creating the `I18n` instance)
- The raw key string itself

```
TINA4_LOCALE=de
```

```
// en.json
{
  "beta_feature": "This feature is in beta."
}

// de.json
{
  "welcome": "Willkommen"
  // beta_feature not yet translated
}
```

```
i18n = I18n(locale="de", path="src/locales")
i18n.t("beta_feature")
# Falls back to en.json: "This feature is in beta."

i18n.t("completely.missing.key")
# Returns "completely.missing.key" -- the key itself
```

Your app never crashes on a missing translation. It returns the raw key string, which your translation team can search for to find gaps.

---

9. Translating Templates

In Frond templates, call `t()` directly:

```
<h1>{{ t("welcome") }}</h1>
<p>{{ t("greeting", name=user.name) }}</p>

<div class="order-summary">
  <p>{{ t("order.placed") }}</p>
  <p>{{ t("order.total", currency="$", amount=order.total) }}</p>
</div>
```

Pass the locale through the template context:

```
@get("/dashboard")
async def dashboard(request, response):
    locale = request.params.get("lang", "en")
    i18n = I18n(locale=locale, path="src/locales")

    return response.render("dashboard.html", {
        "t": i18n.t,
        "user": {"name": "Alice"},
```

```
        "locale": locale
    })
```

---

10. Exercise: Multi-Language API Responses

Build an API that returns translated error messages and UI strings based on the request locale.

Requirements

- Create locale files for `en`, `fr`, and `es` with these keys:
- `errors.not_found`
- `errors.validation`
- `user.created`
- `user.greeting`
- Create `GET /api/users/{user_id}` that:
- Detects locale from `?lang=` param (default: `en`)
- Returns a user or a translated 404 message
- Create `POST /api/users` that:
- Validates required fields and returns translated validation errors
- Returns a translated success message on creation

Test with:

```
# English (default)
curl "http://localhost:7146/api/users/999"
# {"error": "The requested resource was not found."}

# French
curl "http://localhost:7146/api/users/999?lang=fr"
# {"error": "La ressource demandée est introuvable."}

# Spanish
curl "http://localhost:7146/api/users/999?lang=es"
# {"error": "El recurso solicitado no fue encontrado."}

# Create user in French
curl -X POST "http://localhost:7146/api/users?lang=fr" \
  -H "Content-Type: application/json" \
  -d '{"email": "alice@example.com", "name": "Alice"}'
```

---

11. Solution

`src/locales/en.json`:

```
{
  "errors": {
    "not_found": "The requested resource was not found.",
```

```
    "validation": "Validation failed: {fields}"
  },
  "user": {
    "created": "User account created successfully.",
    "greeting": "Hello, {name}!"
  }
}
```

src/locales/fr.json:

```
{
  "errors": {
    "not_found": "La ressource demandée est introuvable.",
    "validation": "Échec de validation : {fields}"
  },
  "user": {
    "created": "Compte utilisateur créé avec succès.",
    "greeting": "Bonjour, {name} !"
  }
}
```

src/locales/es.json:

```
{
  "errors": {
    "not_found": "El recurso solicitado no fue encontrado.",
    "validation": "Error de validación: {fields}"
  },
  "user": {
    "created": "Cuenta de usuario creada con éxito.",
    "greeting": "¡Hola, {name}!"
  }
}
```

src/routes/users_i18n.py:

```
from tina4_python.core.router import get, post
from tina4_python.i18n import I18n

USERS = {
    1: {"id": 1, "name": "Alice", "email": "alice@example.com"},
    2: {"id": 2, "name": "Bob", "email": "bob@example.com"}
}

SUPPORTED_LOCALES = ["en", "fr", "es"]

def get_i18n(request):
    lang = request.params.get("lang", "en")
    if lang not in SUPPORTED_LOCALES:
        lang = "en"
    return I18n(locale=lang, path="src/locales")


@get("/api/users/{user_id}")
async def get_user(request, response):
    i18n = get_i18n(request)
    user_id = int(request.params["user_id"])

    user = USERS.get(user_id)
    if user is None:
```

```
        return response({"error": i18n.t("errors.not_found")}, 404)

    return response({
        "user": user,
        "greeting": i18n.t("user.greeting", name=user["name"])
    })


@post("/api/users")
async def create_user(request, response):
    i18n = get_i18n(request)
    body = request.body

    missing = [f for f in ["email", "name"] if not body.get(f)]
    if missing:
        return response({
            "error": i18n.t("errors.validation", fields=", ".join(missing))
        }, 400)

    new_user = {
        "id": max(USERS.keys()) + 1,
        "name": body["name"],
        "email": body["email"]
    }
    USERS[new_user["id"]] = new_user

    return response({
        "message": i18n.t("user.created"),
        "user": new_user
    }, 201)
```

---

12. Gotchas

1. Missing locale file raises an error

Problem: `I18n(locale="zh", ...)` crashes because `src/locales/zh.json` does not exist.

Fix: Pass `default_locale="en"` when creating the `I18n` instance. Always validate user-supplied locale values against a supported list before passing them to `I18n.set_locale()`.

2. Placeholder name mismatch

Problem: JSON has `{firstName}` but code calls `t("greeting", first_name="Alice")`. The placeholder is not replaced.

Fix: Placeholder names in JSON and keyword argument names in Python must match exactly. Use consistent naming conventions, either camelCase or snake_case throughout your locale files.

3. TINA4_LOCALE not set, t() uses "en"

Problem: `t()` returns English strings even though your app targets a different language.

Fix: Set `TINA4_LOCALE=de` (or your target locale) in `.env`. Without it, `t()` defaults to `"en"`.

4. Locale files not reloading during development

Problem: You updated a locale file but the app still serves the old translation.

Fix: The `I18n` class caches locale files in memory. Restart the dev server or set a shorter cache duration during development. In production this is the correct behaviour.

Structured Logging

1. Stop Using print()

`print("user logged in")` gets the job done in development. In production, it produces an undated, unlabelled stream of text with no severity, no context, and no way to filter. A crash at 3am gives you nothing useful.

Structured logging adds timestamps, levels, and machine-readable output to every message. Set the level in `.env`. Rotate logs by size. Write to a file. Filter by severity. Tina4's `Log` class does all of this without configuration beyond a single environment variable.

---

2. Basic Usage

```
from tina4_python.debug import Log

Log.info("Application started")
Log.debug("Request received", path="/api/users", method="GET")
Log.warning("Rate limit approaching", remaining=10)
Log.error("Database connection failed", host="db.internal", port=5432)
```

Output:

```
2026-04-02 09:14:01 [INFO]    Application started
2026-04-02 09:14:01 [DEBUG]   Request received path=/api/users method=GET
2026-04-02 09:14:01 [WARNING] Rate limit approaching remaining=10
2026-04-02 09:14:01 [ERROR]   Database connection failed host=db.internal port=5432
```

Every message includes a UTC timestamp and level. Keyword arguments become structured key-value pairs appended to the message.

---

3. Log Levels

There are five levels, in ascending severity:

Level	Method	When to Use
`ALL`	-	All levels (used only as a filter setting)
`DEBUG`	`Log.debug()`	Verbose diagnostic data; development only
`INFO`	`Log.info()`	Normal operational events
`WARNING`	`Log.warning()`	Unexpected conditions that are not errors

`ERROR`	`Log.error()`	Failures that need investigation

Setting `TINA4_LOG_LEVEL=WARNING` suppresses `DEBUG` and `INFO` messages. Only warnings and errors reach the log file.

---

4. The TINA4*LOG*LEVEL Environment Variable

```
# Development: see everything
TINA4_LOG_LEVEL=ALL

# Staging: see info and above
TINA4_LOG_LEVEL=INFO

# Production: warnings and errors only
TINA4_LOG_LEVEL=WARNING
```

The level is read at startup. Changing it requires a server restart (or, if you implement a reload endpoint, a signal to the process).

---

5. Logging in Route Handlers

```
from tina4_python.core.router import get, post
from tina4_python.debug import Log

@post("/api/orders")
async def create_order(request, response):
    body = request.body

    Log.info("Order creation started", customer_id=body.get("customer_id"))

    if not body.get("items"):
        Log.warning("Order rejected: no items", customer_id=body.get("customer_id"))
        return response({"error": "At least one item is required"}, 400)

    try:
        # Simulate order processing
        order_id = 1001
        total = sum(item.get("price", 0) for item in body["items"])

        Log.info("Order created", order_id=order_id, total=total, item_count=len(body["items"]))

        return response({"order_id": order_id, "total": total}, 201)

    except Exception as exc:
        Log.error("Order creation failed", error=str(exc), customer_id=body.get("customer_id"))
        return response({"error": "Internal server error"}, 500)
```

Log structured data alongside the message. When something goes wrong you can grep or filter by `order_id`, `customer_id`, or `error` without parsing free-form text.

---

6. File Output and Log Rotation

Write logs to a file:

```
TINA4_LOG_FILE=logs/app.log
```

Tina4 creates the file (and the `logs/` directory) if it does not exist. Logs are written to both the file and stdout.

Enable rotation by size:

```
TINA4_LOG_FILE=logs/app.log
TINA4_LOG_MAX_SIZE=10485760
TINA4_LOG_KEEP=5
```

`TINA4_LOG_MAX_SIZE=10485760` rotates the log file when it reaches 10 MB. `TINA4_LOG_KEEP=5` keeps the five most recent rotated files before deleting the oldest. These settings cap total log storage at roughly 50 MB.

Rotated files are named `app.log.1`, `app.log.2`, and so on.

---

7. Logging Exceptions

Log a full exception with stack trace using `Log.error` and Python's `traceback` module:

```
import traceback
from tina4_python.debug import Log

try:
    result = 1 / 0
except Exception as exc:
    Log.error(
        "Unhandled exception",
        error=str(exc),
        traceback=traceback.format_exc()
    )
```

Output:

```
2026-04-02 09:14:01 [ERROR] Unhandled exception error=division by zero traceback=Traceback (most
  File "src/routes/orders.py", line 34, in create_order
    result = 1 / 0
ZeroDivisionError: division by zero
```

The full stack trace is attached as a structured field. Log aggregators can extract, index, and alert on it.

---

8. Request Logging Middleware

Log every incoming request automatically:

```
from tina4_python.debug import Log

async def request_logger(request, response, next_handler):
```

```
    import time
    start = time.monotonic()
    result = await next_handler(request, response)
    elapsed = round((time.monotonic() - start) * 1000, 2)

    Log.info(
        "Request completed",
        method=request.method,
        path=request.url,
        status=getattr(result, "status_code", 200),
        duration_ms=elapsed
    )

    return result
```

Register it globally in your app:

```
from tina4_python.core.router import use
use(request_logger)
```

Every request now produces a structured log line:

```
2026-04-02 09:14:01 [INFO] Request completed method=POST path=/api/orders status=201 duration_ms
2026-04-02 09:14:02 [INFO] Request completed method=GET path=/api/users status=200 duration_ms=2
```

Filter by path, method, or status code in any log aggregation tool.

---

9. Log Levels by Environment

A typical multi-environment setup:

```
# .env.development
TINA4_LOG_LEVEL=ALL
TINA4_LOG_FILE=

# .env.staging
TINA4_LOG_LEVEL=INFO
TINA4_LOG_FILE=logs/staging.log
TINA4_LOG_MAX_SIZE=10485760
TINA4_LOG_KEEP=3

# .env.production
TINA4_LOG_LEVEL=WARNING
TINA4_LOG_FILE=logs/app.log
TINA4_LOG_MAX_SIZE=52428800
TINA4_LOG_KEEP=10
```

Development logs everything to stdout. Staging logs info and above to a rotating file. Production only logs warnings and errors to a large rotating file kept for 10 rotations.

---

10. Exercise: Add Logging to an Order API

Add structured logging to an order management API.

Requirements

- Create `POST /api/shop/orders` that:
- Logs the start of each request with `customer_id`
- Logs validation failures at `WARNING` level
- Logs successful orders at `INFO` level with `order_id`, `total`, `item_count`
- Logs any exceptions at `ERROR` level with full error context
- Create `GET /api/shop/orders/{order_id}` that:
- Logs cache hits and misses at `DEBUG` level
- Logs 404s at `WARNING` level
- Set up the request logging middleware globally

Test with:

```
# Valid order
curl -X POST http://localhost:7146/api/shop/orders \
  -H "Content-Type: application/json" \
  -d '{"customer_id": 1, "items": [{"name": "Keyboard", "price": 79.99}]}'

# Invalid order (no items)
curl -X POST http://localhost:7146/api/shop/orders \
  -H "Content-Type: application/json" \
  -d '{"customer_id": 1}'

# Get order
curl http://localhost:7146/api/shop/orders/1001
```

---

11. Solution

Create `src/routes/shop_orders.py`:

```
import traceback
from tina4_python.core.router import get, post
from tina4_python.debug import Log
from tina4_python.cache import cache_get, cache_set

ORDER_STORE = {}
NEXT_ORDER_ID = 1001


@post("/api/shop/orders")
async def create_shop_order(request, response):
    body = request.body
    customer_id = body.get("customer_id")

    Log.info("Order creation started", customer_id=customer_id)

    if not body.get("items"):
        Log.warning("Order rejected: missing items", customer_id=customer_id)
        return response({"error": "At least one item is required"}, 400)
```

```
    if not customer_id:
        Log.warning("Order rejected: missing customer_id")
        return response({"error": "customer_id is required"}, 400)

    try:
        global NEXT_ORDER_ID
        order_id = NEXT_ORDER_ID
        NEXT_ORDER_ID += 1

        items = body["items"]
        total = round(sum(item.get("price", 0) for item in items), 2)

        order = {
            "order_id": order_id,
            "customer_id": customer_id,
            "items": items,
            "total": total,
            "status": "placed"
        }
        ORDER_STORE[order_id] = order
        cache_set(f"order:{order_id}", order, ttl=300)

        Log.info(
            "Order created",
            order_id=order_id,
            customer_id=customer_id,
            total=total,
            item_count=len(items)
        )

        return response(order, 201)

    except Exception as exc:
        Log.error(
            "Order creation failed",
            customer_id=customer_id,
            error=str(exc),
            traceback=traceback.format_exc()
        )
        return response({"error": "Internal server error"}, 500)


@get("/api/shop/orders/{order_id}")
async def get_shop_order(request, response):
    order_id = int(request.params["order_id"])

    cached = cache_get(f"order:{order_id}")
    if cached:
        Log.debug("Order cache hit", order_id=order_id)
        return response(cached)

    Log.debug("Order cache miss", order_id=order_id)

    order = ORDER_STORE.get(order_id)
    if order is None:
        Log.warning("Order not found", order_id=order_id)
        return response({"error": "Order not found"}, 404)

    cache_set(f"order:{order_id}", order, ttl=300)
```

```
    return response(order)
```

---

12. Gotchas

1. DEBUG logs flooding production

Problem: Hundreds of debug messages per second fill the log file and slow the server.

Fix: Set `TINA4_LOG_LEVEL=WARNING` in production. Debug calls are no-ops when the level is above `DEBUG`.

2. Logging sensitive data

Problem: `Log.info("User logged in", password=request.body["password"])` writes passwords to the log file.

Fix: Never log passwords, tokens, card numbers, or PII. Log identifiers and metadata only: `Log.info("User logged in", user_id=user["id"])`.

3. Log file grows without rotation

Problem: `TINA4_LOG_FILE` is set but `TINA4_LOG_MAX_SIZE` is not. The log file grows until the disk is full.

Fix: Always set `TINA4_LOG_MAX_SIZE` and `TINA4_LOG_KEEP` alongside `TINA4_LOG_FILE` in production.

4. Forgetting to log exceptions

Problem: A route returns 500 but there is no log entry explaining why.

Fix: Wrap the handler body in `try/except`. Log the exception with `Log.error(..., error=str(exc), traceback=traceback.format_exc())` before returning the 500 response.

Email with Messenger

1. Every App Sends Email

Signup confirmations. Password resets. Weekly digests. Invoices with PDF attachments. Every application needs email. Nobody enjoys building it.

SMTP configuration. Plain text fallbacks. Attachment encoding. Connection timeouts. The details pile up fast. Tina4's `Messenger` class absorbs all of it. Configure through `.env`. Create an instance. Send. Without an SMTP host, Messenger captures mail in the dev mailbox. An SMTP host enables real delivery, so staging needs an explicit safety setting.

---

2. Messenger Configuration via .env

All email configuration lives in `.env`:

```
TINA4_MAIL_HOST=smtp.example.com
TINA4_MAIL_PORT=587
TINA4_MAIL_USERNAME=your-email@example.com
TINA4_MAIL_PASSWORD=your-app-password
TINA4_MAIL_ENCRYPTION=tls
TINA4_MAIL_FROM=noreply@example.com
TINA4_MAIL_FROM_NAME=My Store
```

Variable	Description	Common Values
`TINA4_MAIL_HOST`	SMTP server hostname	`smtp.gmail.com`, `smtp.mailgun.org`, `smtp.sendgrid.net`
`TINA4_MAIL_PORT`	SMTP port	`587` (TLS), `465` (SSL), `25` (unencrypted)
`TINA4_MAIL_USERNAME`	Login username	Usually your email address
`TINA4_MAIL_PASSWORD`	Login password or app-specific password	App passwords for Gmail
`TINA4_MAIL_ENCRYPTION`	Encryption method	`tls` (recommended), `ssl`, `none`
`TINA4_MAIL_FROM`	Default "From" address	`noreply@yourdomain.com`
`TINA4_MAIL_FROM_NAME`	Default "From" display name	`My Store`, `Acme Corp`
`TINA4_MAIL_CAPTURE`	Disable SMTP and capture each message locally	`true` on staging when no external delivery is allowed
`TINA4_MAIL_REDIRECT_TO`	Replace every SMTP recipient with a safety list	`qa@example.com,product@example.com`

Messenger also accepts legacy `SMTP_*` prefixed variables as fallback. The `TINA4_MAIL_*` prefix takes priority.

Staging delivery safety

`TINA4_DEBUG` controls developer tools. It does not disable SMTP delivery. Pick one explicit mail policy for staging:

```
# No message leaves the application. Inspect it in the dev mailbox.
TINA4_MAIL_CAPTURE=true
```

Capture wins even when an SMTP host exists. A missing SMTP host also captures mail. `TINA4_MAILBOX_DIR` only selects the dev mailbox directory; it does not turn capture on or off.

Use real SMTP without risking customer delivery by replacing all recipients:

```
TINA4_MAIL_REDIRECT_TO=qa@example.com,product@example.com
```

Tina4 trims each comma-separated address and drops blank entries. On the SMTP path, the safety list replaces `To`; Tina4 clears `Cc` and `Bcc` and stores all original recipients in `X-Tina4-Original-To`. An unset or empty list leaves delivery unchanged. If capture and redirect are both set, capture wins and nothing reaches SMTP.

Common Provider Configurations

Gmail:

```
TINA4_MAIL_HOST=smtp.gmail.com
TINA4_MAIL_PORT=587
TINA4_MAIL_USERNAME=your-email@gmail.com
TINA4_MAIL_PASSWORD=your-app-password
TINA4_MAIL_ENCRYPTION=tls
```

Gmail requires an "App Password" (not your regular password) when two-factor authentication is enabled.

Mailgun:

```
TINA4_MAIL_HOST=smtp.mailgun.org
TINA4_MAIL_PORT=587
TINA4_MAIL_USERNAME=postmaster@mg.yourdomain.com
TINA4_MAIL_PASSWORD=your-mailgun-smtp-password
TINA4_MAIL_ENCRYPTION=tls
```

SendGrid:

```
TINA4_MAIL_HOST=smtp.sendgrid.net
TINA4_MAIL_PORT=587
TINA4_MAIL_USERNAME=apikey
TINA4_MAIL_PASSWORD=your-sendgrid-api-key
TINA4_MAIL_ENCRYPTION=tls
```

---

3. Constructor Override Pattern

Different emails need different SMTP accounts. Transactional emails from one server. Marketing from another. Override the configuration in the constructor:

```
from tina4_python.messenger import Messenger

# Uses .env defaults
mailer = Messenger()

# Override specific settings
marketing_mailer = Messenger(
    host="smtp.mailgun.org",
    port=587,
    username="marketing@mg.yourdomain.com",
    password="marketing-smtp-password",
    encryption="tls",
    from_address="newsletter@yourdomain.com",
    from_name="My Store Newsletter"
)
```

Constructor arguments take priority over `.env` values. Any argument you omit falls back to the environment variable.

---

4. Sending Plain Text Email

The simplest email:

```
from tina4_python.core.router import post
from tina4_python.messenger import Messenger

@post("/api/contact")
async def contact_form(request, response):
    body = request.body

    mailer = Messenger()

    result = mailer.send(
        to=body["email"],
        subject="Contact Form Submission",
        body=f"Name: {body['name']}\n"
             f"Email: {body['email']}\n"
             f"Message:\n{body['message']}"
    )

    if result["success"]:
        return response({"message": "Email sent successfully"})

    return response({"error": "Failed to send email", "details": result["error"]}, 500)
```

```
curl -X POST http://localhost:7146/api/contact \
  -H "Content-Type: application/json" \
  -d '{"name": "Alice", "email": "alice@example.com", "message": "I love your products!"}'
```

```
{"message":"Email sent successfully"}
```

The `send()` method returns a dict with three keys: `"success"` (boolean), `"error"` (string or None), and `"message_id"` (string or None).

The send() Method Signature

```
mailer.send(
    to,                 # Recipient(s) -- string or list of strings
    subject,            # Email subject line
    body,               # Email body (plain text or HTML)
    html=False,         # If True, body is treated as HTML
    text=None,          # Plain text alternative (when body is HTML)
    cc=None,            # CC recipient(s) -- string or list
    bcc=None,           # BCC recipient(s) -- string or list
    reply_to=None,      # Reply-To address
    attachments=None,   # List of file paths or dicts
    headers=None        # Additional email headers (dict)
)
```

---

5. Sending HTML Email with Text Fallback

Most emails should carry HTML with a plain text fallback. Email clients that cannot render HTML display the text version instead:

```
from tina4_python.messenger import Messenger

mailer = Messenger()

html_body = """
<html>
<body style="font-family: Arial, sans-serif; max-width: 600px; margin: 0 auto;">
    <div style="background: #1a1a2e; color: white; padding: 20px; text-align: center;">
        <h1 style="margin: 0;">Welcome to My Store!</h1>
    </div>
    <div style="padding: 20px;">
        <p>Hi Alice,</p>
        <p>Thank you for creating your account. We are excited to have you!</p>
        <p>Here is what you can do next:</p>
        <ul>
            <li>Browse our <a href="https://mystore.com/products">product catalog</a></li>
            <li>Set up your <a href="https://mystore.com/profile">profile</a></li>
            <li>Check out our <a href="https://mystore.com/deals">current deals</a></li>
        </ul>
        <p>If you have any questions, reply to this email and we will get back to you within 24
        <p>Cheers,<br>The My Store Team</p>
    </div>
    <div style="background: #f5f5f5; padding: 12px; text-align: center; font-size: 12px; color:
        <p>You received this because you signed up at mystore.com</p>
    </div>
</body>
</html>
"""

text_body = (
    "Hi Alice,\n\n"
    "Thank you for creating your account. We are excited to have you!\n\n"
    "Here is what you can do next:\n"
```

```
    "- Browse our product catalog: https://mystore.com/products\n"
    "- Set up your profile: https://mystore.com/profile\n"
    "- Check out our current deals: https://mystore.com/deals\n\n"
    "If you have any questions, reply to this email.\n\n"
    "Cheers,\nThe My Store Team"
)

result = mailer.send(
    to="alice@example.com",
    subject="Welcome to My Store!",
    body=html_body,
    html=True,
    text=text_body
)
```

Pass `html=True` to tell Messenger the body contains HTML. The `text` parameter provides the plain text alternative. Messenger builds a `multipart/alternative` message that carries both versions.

---

6. Adding Attachments

Attach files by providing their paths:

```
from tina4_python.messenger import Messenger

mailer = Messenger()

result = mailer.send(
    to="accounting@example.com",
    subject="Monthly Invoice #1042",
    body="<h2>Invoice #1042</h2><p>Please find the invoice attached.</p>",
    html=True,
    attachments=[
        "/path/to/invoices/invoice-1042.pdf",
        "/path/to/reports/monthly-summary.csv"
    ]
)
```

Messenger reads each file, determines its MIME type, and encodes it for email transmission. Each entry can be a file path string, a `Path` object, or a dict for more control.

Attachments with Custom Names and Binary Content

```
result = mailer.send(
    to="alice@example.com",
    subject="Your Export",
    body="<p>Here is your data export.</p>",
    html=True,
    attachments=[
        {
            "filename": "my-store-export.csv",
            "content": csv_bytes,
            "mime": "text/csv"
        }
    ]
)
```

The dict format accepts `filename` (display name), `content` (raw bytes), and `mime` (MIME type string). The recipient sees the file named `my-store-export.csv` regardless of how it was generated.

---

7. CC, BCC, and Reply-To

```
from tina4_python.messenger import Messenger

mailer = Messenger()

result = mailer.send(
    to="alice@example.com",
    subject="Team Meeting Notes",
    body="<p>Here are the notes from today's meeting.</p>",
    html=True,
    cc=["bob@example.com", "charlie@example.com"],
    bcc=["manager@example.com"],
    reply_to="alice@example.com"
)
```

- **cc**: List of email addresses to carbon copy. All recipients see CC addresses.
- **bcc**: List of email addresses to blind carbon copy. Recipients cannot see BCC addresses.
- **reply_to**: When the recipient clicks "Reply", this address fills the "To" field instead of the "From" address.

Both `cc` and `bcc` accept a single string or a list of strings.

---

8. Custom Headers

Messenger supports two methods for adding headers. The `add_header()` method sets a default header on all emails sent by that instance. The `headers` parameter on `send()` sets headers for a single email.

Default Headers on an Instance

```
from tina4_python.messenger import Messenger

mailer = Messenger()
mailer.add_header("X-App-Name", "My Store")
mailer.add_header("X-Environment", "production")

# Every email from this instance carries both headers
mailer.send(to="alice@example.com", subject="Test", body="Hello")
```

Per-Email Headers

```
result = mailer.send(
    to="customer@example.com",
    subject="Your Support Ticket #123",
    body="We are looking into your issue.",
    reply_to="support@mystore.com",
    headers={
        "X-Ticket-Id": "123",
        "X-Priority": "1",
        "X-Mailer": "Tina4 Messenger"
    }
)
```

Per-email headers merge with default headers. If both define the same key, the per-email value wins.

Custom headers serve several purposes. Tracking headers like `X-Ticket-Id` let you correlate emails with support tickets. Priority headers influence some email clients' display. Bulk-sending headers like `Precedence: bulk` help mail servers classify newsletters.

---

9. Reading Inbox via IMAP

Messenger reads email through IMAP. Configure the IMAP server in `.env`:

```
TINA4_MAIL_IMAP_HOST=imap.example.com
TINA4_MAIL_IMAP_PORT=993
```

Messenger reuses `TINA4_MAIL_USERNAME` and `TINA4_MAIL_PASSWORD` for IMAP authentication. You can also override the IMAP host and port in the constructor:

```
mailer = Messenger(imap_host="imap.gmail.com", imap_port=993)
```

Listing Inbox Messages

The `inbox()` method fetches message headers from the mailbox:

```
from tina4_python.core.router import get
from tina4_python.messenger import Messenger

@get("/api/inbox")
async def get_inbox(request, response):
    mailer = Messenger()

    emails = mailer.inbox(limit=20, offset=0)

    messages = []
    for email in emails:
        messages.append({
            "uid": email["uid"],
            "from": email["from"],
            "subject": email["subject"],
            "date": email["date"],
            "snippet": email["snippet"],
            "seen": email["seen"]
        })
```

```
    return response({"messages": messages, "count": len(messages)})
```

```
curl http://localhost:7146/api/inbox
```

```
{
  "messages": [
    {
      "uid": "12345",
      "from": "customer@example.com",
      "subject": "Order question",
      "date": "2026-03-22T10:30:00+00:00",
      "snippet": "Hi, I have a question about my recent order...",
      "seen": false
    }
  ],
  "count": 1
}
```

The `inbox()` method returns messages newest-first. Each message contains `uid`, `subject`, `from`, `to`, `date`, `snippet` (first 150 characters of the body), and `seen` (boolean).

Reading a Specific Message

```
@get("/api/inbox/{uid}")
async def get_email(request, response):
    mailer = Messenger()
    uid = request.params["uid"]

    email = mailer.read(uid, mark_read=True)

    if not email:
        return response({"error": "Email not found"}, 404)

    return response({
        "uid": email["uid"],
        "from": email["from"],
        "to": email["to"],
        "cc": email["cc"],
        "subject": email["subject"],
        "date": email["date"],
        "body_html": email["body_html"],
        "body_text": email["body_text"],
        "attachments": [
            {"filename": a["filename"], "size": a["size"], "content_type": a["content_type"]}
            for a in email.get("attachments", [])
        ]
    })
```

The `read()` method fetches the full message including body and attachments. Pass `mark_read=False` to leave the message unread.

Searching Messages

```
@get("/api/inbox/search")
async def search_inbox(request, response):
    mailer = Messenger()

    results = mailer.search(
        subject=request.query.get("q"),
        sender=request.query.get("from"),
        unseen_only=request.query.get("unread") == "true",
        limit=20
    )

    return response({"messages": results, "count": len(results)})
```

The `search()` method accepts `subject`, `sender`, `since` (date string "DD-Mon-YYYY"), `before`, and `unseen_only` as filters. All filters combine with AND logic.

Other IMAP Operations

```
mailer = Messenger()

# Count unread messages
count = mailer.unread()

# Mark a message as read
mailer.mark_read("12345")

# Mark a message as unread
mailer.mark_unread("12345")

# Delete a message
mailer.delete("12345")

# List all mailbox folders
folders = mailer.folders()
# ["INBOX", "Sent", "Drafts", "Trash", "Spam"]
```

Testing IMAP Connectivity

```
mailer = Messenger()
result = mailer.test_imap_connection()
if result["success"]:
    print("IMAP connection works")
else:
    print(f"IMAP failed: {result['error']}")
```

---

10. Dev Mailbox Capture

Messenger captures outgoing mail when no SMTP host exists or when `TINA4_MAIL_CAPTURE=true`. `TINA4_DEBUG` only makes the dev dashboard available; it does not change delivery.

Navigate to `/__dev` and look for the "Mail" section. You see:

- Every email "sent" during the current session

- The To, CC, and BCC addresses
- The subject and body (both HTML and plain text)
- Attachments (viewable inline)
- The timestamp

Use `create_messenger()` to load the environment policy and capture without an SMTP host:

```
from tina4_python.messenger import create_messenger

mailer = create_messenger()
# No SMTP host or TINA4_MAIL_CAPTURE=true: captures locally
# SMTP host configured and capture off: sends through SMTP
```

Sending from a development environment

Configure an SMTP host and leave `TINA4_MAIL_CAPTURE` unset or false. The `TINA4_DEBUG` value does not matter to delivery. To test SMTP without reaching customers, set `TINA4_MAIL_REDIRECT_TO` to controlled inboxes.

---

11. Using Templates for Email Content

Hardcoded HTML in Python strings is ugly and hard to maintain. Templates fix this.

Create `src/templates/emails/welcome.html`:

```
<!DOCTYPE html>
<html>
<head>
    <meta charset="UTF-8">
</head>
<body style="font-family: Arial, sans-serif; max-width: 600px; margin: 0 auto; background: #f5f5
    <div style="background: #1a1a2e; color: white; padding: 24px; text-align: center; border-rad
        <h1 style="margin: 0; font-size: 24px;">Welcome, {{ name }}!</h1>
    </div>
    <div style="background: white; padding: 24px; border-radius: 0 0 8px 8px;">
        <p>Hi {{ name }},</p>
        <p>Your account has been created successfully. Here are your details:</p>

        <table style="width: 100%; border-collapse: collapse; margin: 16px 0;">
            <tr>
                <td style="padding: 8px; border-bottom: 1px solid #eee; font-weight: bold;">Emai
                <td style="padding: 8px; border-bottom: 1px solid #eee;">{{ email }}</td>
            </tr>
            <tr>
                <td style="padding: 8px; border-bottom: 1px solid #eee; font-weight: bold;">Acco
                <td style="padding: 8px; border-bottom: 1px solid #eee;">#{{ user_id }}</td>
            </tr>
            <tr>
                <td style="padding: 8px; border-bottom: 1px solid #eee; font-weight: bold;">Sign
                <td style="padding: 8px; border-bottom: 1px solid #eee;">{{ signed_up_at }}</td>
            </tr>
        </table>

        <p>Get started by exploring:</p>
```

```
        <ul>
            <li><a href="{{ base_url }}/products" style="color: #1a1a2e;">Our product catalog</a
            <li><a href="{{ base_url }}/profile" style="color: #1a1a2e;">Your profile settings</
        </ul>

        {% if promo_code %}
            <div style="background: #d4edda; padding: 16px; border-radius: 4px; margin: 16px 0;"
                <strong>Special offer!</strong> Use code <code>{{ promo_code }}</code> for 10% o
            </div>
        {% endif %}

        <p>Cheers,<br>The {{ app_name }} Team</p>
    </div>

    <div style="text-align: center; padding: 12px; color: #888; font-size: 12px;">
        <p>You received this because you signed up at {{ app_name }}.</p>
        <p><a href="{{ base_url }}/unsubscribe?token={{ unsubscribe_token }}" style="color: #888
    </div>
</body>
</html>
```

Rendering and Sending

```
import os
import secrets
from datetime import datetime
from tina4_python.core.router import post, template
from tina4_python.messenger import Messenger

@post("/api/register")
async def register_user(request, response):
    body = request.body

    # Create user (database logic)
    user_id = 42

    # Render the email template
    email_data = {
        "name": body["name"],
        "email": body["email"],
        "user_id": user_id,
        "signed_up_at": datetime.now().strftime("%B %d, %Y"),
        "base_url": os.getenv("APP_URL", "http://localhost:7146"),
        "app_name": "My Store",
        "promo_code": "WELCOME10",
        "unsubscribe_token": secrets.token_hex(16)
    }

    html_body = template("emails/welcome.html", **email_data)

    # Send the email
    mailer = Messenger()
    result = mailer.send(
        to=body["email"],
        subject=f"Welcome to My Store, {body['name']}!",
        body=html_body,
        html=True,
        text=f"Hi {body['name']},\n\nWelcome to My Store! "
             f"Your account (#{user_id}) has been created.\n\n"
             f"Cheers,\nThe My Store Team"
    )

    return response({
        "message": "Registration successful",
        "email_sent": result["success"],
        "user_id": user_id
    }, 201)
```

```
curl -X POST http://localhost:7146/api/register \
  -H "Content-Type: application/json" \
  -d '{"name": "Alice", "email": "alice@example.com", "password": "securePass123"}'
```

```
{
  "message": "Registration successful",
  "email_sent": true,
  "user_id": 42
}
```

With `TINA4_MAIL_CAPTURE=true`, the email stays in the dev mailbox instead of reaching a real inbox. Enable `TINA4_DEBUG=true` to inspect the rendered HTML and verify the template values

in the dashboard.

---

12. Sending Email via Queues

In production, never send email inside a route handler. The SMTP call blocks the response. Use the queue system from Chapter 12:

```
from tina4_python.core.router import post, template
from tina4_python.queue import Queue
from tina4_python.messenger import Messenger

# In the route handler, just queue the email
@post("/api/register")
async def register_user(request, response):
    body = request.body
    user_id = 42  # Simulated

    queue = Queue(topic="emails")
    queue.push({
        "template": "emails/welcome.html",
        "to": body["email"],
        "subject": f"Welcome to My Store, {body['name']}!",
        "data": {
            "name": body["name"],
            "email": body["email"],
            "user_id": user_id,
            "signed_up_at": "March 22, 2026",
            "base_url": "http://localhost:7146",
            "app_name": "My Store",
            "promo_code": "WELCOME10"
        }
    })

    return response({"message": "Registration successful", "user_id": user_id}, 201)


# The consumer sends the actual email
@Queue.consume("emails")
async def send_email_job(job):
    payload = job.payload

    html_body = template(payload["template"], **payload["data"])

    mailer = Messenger()
    result = mailer.send(
        to=payload["to"],
        subject=payload["subject"],
        body=html_body,
        html=True
    )

    if not result["success"]:
        print(f"Email failed: {result['error']}")
        return False  # Retry

    print(f"Email sent to {payload['to']}")
```

```
    return True
```

The route handler returns in under 50 milliseconds. The queue worker sends the email on its own timeline. If the SMTP server is down, retries happen automatically.

---

13. Exercise: Build a Contact Form with Email Notification

Build a contact form that sends an email notification when submitted.

Requirements

- Create a `GET /contact` page that renders a contact form with fields: name, email, subject, and message
- Create a `POST /contact` endpoint that:
- Validates all fields are present
- Sends an email notification to the site admin (`admin@example.com`)
- The email should include all form fields, formatted in HTML
- Shows a flash message on success
- Redirects back to the contact page
- Create an email template at `src/templates/emails/contact-notification.html` that formats the contact submission

Test with:

```
# View the form
curl http://localhost:7146/contact

# Submit the form
curl -X POST http://localhost:7146/contact \
  -H "Content-Type: application/json" \
  -d '{"name": "Bob", "email": "bob@example.com", "subject": "Product inquiry", "message": "Do y

# Check the dev dashboard for the intercepted email
# Navigate to http://localhost:7146/__dev
```

---

14. Solution

Create `src/templates/emails/contact-notification.html`:

```
<!DOCTYPE html>
<html>
<head><meta charset="UTF-8"></head>
<body style="font-family: Arial, sans-serif; max-width: 600px; margin: 0 auto;">
    <div style="background: #1a1a2e; color: white; padding: 16px; text-align: center;">
        <h2 style="margin: 0;">New Contact Form Submission</h2>
    </div>
    <div style="padding: 20px; background: white;">
        <table style="width: 100%; border-collapse: collapse;">
```

```
            <tr>
                <td style="padding: 10px; border-bottom: 1px solid #eee; font-weight: bold; widt
                <td style="padding: 10px; border-bottom: 1px solid #eee;">{{ name }} ({{ email }
            </tr>
            <tr>
                <td style="padding: 10px; border-bottom: 1px solid #eee; font-weight: bold;">Sub
                <td style="padding: 10px; border-bottom: 1px solid #eee;">{{ subject }}</td>
            </tr>
            <tr>
                <td style="padding: 10px; border-bottom: 1px solid #eee; font-weight: bold;">Dat
                <td style="padding: 10px; border-bottom: 1px solid #eee;">{{ submitted_at }}</td
            </tr>
        </table>

        <div style="margin-top: 16px; padding: 16px; background: #f8f9fa; border-radius: 4px;">
            <h3 style="margin-top: 0;">Message</h3>
            <p style="white-space: pre-wrap;">{{ message }}</p>
        </div>

        <p style="margin-top: 16px; color: #888; font-size: 12px;">
            Reply directly to this email to respond to {{ name }} at {{ email }}.
        </p>
    </div>
</body>
</html>
```

Create `src/templates/contact.html`:

```
{% extends "base.html" %}

{% block title %}Contact Us{% endblock %}

{% block content %}
    <h1>Contact Us</h1>

    {% if flash %}
        <div style="padding: 12px; border-radius: 4px; margin-bottom: 16px;
            {% if flash.type == 'success' %}background: #d4edda; color: #155724;{% endif %}
            {% if flash.type == 'error' %}background: #f8d7da; color: #721c24;{% endif %}">
            {{ flash.message }}
        </div>
    {% endif %}

    <form method="POST" action="/contact" style="max-width: 500px;">
        <div style="margin-bottom: 12px;">
            <label for="name" style="display: block; margin-bottom: 4px; font-weight: bold;">Nam
            <input type="text" name="name" id="name" required
                   style="width: 100%; padding: 8px; border: 1px solid #ddd; border-radius: 4px;
        </div>

        <div style="margin-bottom: 12px;">
            <label for="email" style="display: block; margin-bottom: 4px; font-weight: bold;">Em
            <input type="email" name="email" id="email" required
                   style="width: 100%; padding: 8px; border: 1px solid #ddd; border-radius: 4px;
        </div>

        <div style="margin-bottom: 12px;">
            <label for="subject" style="display: block; margin-bottom: 4px; font-weight: bold;">
            <input type="text" name="subject" id="subject" required
```

```
                    style="width: 100%; padding: 8px; border: 1px solid #ddd; border-radius: 4px;
        </div>

        <div style="margin-bottom: 12px;">
            <label for="message" style="display: block; margin-bottom: 4px; font-weight: bold;">
            <textarea name="message" id="message" rows="6" required
                      style="width: 100%; padding: 8px; border: 1px solid #ddd; border-radius: 4
        </div>

        <button type="submit"
                style="padding: 10px 20px; background: #1a1a2e; color: white; border: none; bord
            Send Message
        </button>
    </form>
{% endblock %}
```

Create `src/routes/contact.py`:

```
import os
from datetime import datetime
from tina4_python.core.router import get, post, template
from tina4_python.messenger import Messenger

@get("/contact")
async def contact_page(request, response):
    flash = request.session.get("_flash")
    if flash:
        del request.session["_flash"]

    return response(template("contact.html", flash=flash))


@post("/contact")
async def contact_submit(request, response):
    body = request.body

    # Validate
    errors = []
    if not body.get("name"):
        errors.append("Name is required")
    if not body.get("email"):
        errors.append("Email is required")
    if not body.get("subject"):
        errors.append("Subject is required")
    if not body.get("message"):
        errors.append("Message is required")

    if errors:
        request.session["_flash"] = {
            "type": "error",
            "message": "Please fill in all fields: " + ", ".join(errors)
        }
        return response.redirect("/contact")

    # Render the email template
    html_body = template("emails/contact-notification.html",
        name=body["name"],
        email=body["email"],
        subject=body["subject"],
```

```
        message=body["message"],
        submitted_at=datetime.now().strftime("%B %d, %Y at %I:%M %p")
    )

    # Send the email
    mailer = Messenger()
    admin_email = os.getenv("ADMIN_EMAIL", "admin@example.com")

    result = mailer.send(
        to=admin_email,
        subject=f"Contact Form: {body['subject']}",
        body=html_body,
        html=True,
        reply_to=body["email"],
        text=f"Contact form submission from {body['name']} ({body['email']}):\n\n"
             f"Subject: {body['subject']}\n\n"
             f"Message:\n{body['message']}"
    )

    if result["success"]:
        request.session["_flash"] = {
            "type": "success",
            "message": "Thank you for your message! We will get back to you shortly."
        }
    else:
        request.session["_flash"] = {
            "type": "error",
            "message": "Sorry, there was a problem sending your message. Please try again later.
        }

    return response.redirect("/contact")
```

Testing:

- Open `http://localhost:7146/contact` in your browser
- Fill in the form and submit
- You should see a green "Thank you" flash message
- Open `http://localhost:7146/__dev` to see the intercepted email
- The email shows the sender details, subject, message, and formatted HTML

API test:

```
curl -X POST http://localhost:7146/contact \
  -H "Content-Type: application/json" \
  -d '{"name": "Bob", "email": "bob@example.com", "subject": "Product inquiry", "message": "Do y
  -c cookies.txt -b cookies.txt
```

The response is a `302` redirect to `/contact`. Follow the redirect to see the flash message:

```
curl http://localhost:7146/contact -b cookies.txt
```

The HTML response includes the success flash message.

---

15. Gotchas

1. Gmail Blocks "Less Secure" Apps

Problem: Sending via Gmail fails with "Authentication failed" or "Username and Password not accepted".

Cause: Gmail blocks SMTP access from apps that do not use OAuth2 by default. Your regular password will not work when two-factor authentication is enabled.

Fix: Generate an "App Password" in your Google Account settings (Security > 2-Step Verification > App Passwords). Use this 16-character password as `TINA4_MAIL_PASSWORD`. This is separate from your regular Google password.

2. Emails Go to Spam

Problem: Emails land in the recipient's spam folder.

Cause: Your sending domain lacks proper DNS records (SPF, DKIM, DMARC) or you send from a free email provider (Gmail, Yahoo).

Fix: Use a dedicated sending domain with proper DNS records. Set up SPF, DKIM, and DMARC records. Use a transactional email service -- Mailgun, SendGrid, or Amazon SES -- that handles email reputation for you.

3. HTML Email Looks Broken

Problem: The email looks fine in Gmail but broken in Outlook or Apple Mail.

Cause: Email clients have different HTML/CSS support. CSS flexbox, grid, and many modern properties do not work in email.

Fix: Use inline styles (not external stylesheets or `<style>` blocks). Use table-based layouts for complex designs. Test with an email preview tool. Keep it simple -- most transactional emails do not need elaborate designs.

4. Attachment File Not Found

Problem: `Messenger.send()` returns an error about a missing file.

Cause: The attachment path is relative or incorrect. The file does not exist at the specified location.

Fix: Use absolute paths for attachments. Verify the file exists before calling `send()`: `if not os.path.exists(path): ...`. If the file is generated dynamically (a PDF, for example), make sure the generation completes before sending.

5. Dev Mode Silently Intercepts Emails

Problem: You configured SMTP but no emails arrive. No errors either.

Cause: `TINA4_MAIL_CAPTURE=true` or a missing SMTP host sends mail to the dev mailbox. `TINA4_DEBUG` does not control delivery.

Fix: Check `/__dev` for captured mail. To use SMTP, configure `TINA4_MAIL_HOST` and remove `TINA4_MAIL_CAPTURE`. On staging, set `TINA4_MAIL_REDIRECT_TO` so tests cannot reach customer addresses.

6. Email Template Variables Not Substituted

Problem: The email body shows `{{ name }}` literally instead of the user's name.

Cause: You passed the raw template file content instead of rendering it through the template engine.

Fix: Use `template("emails/template.html", **data)` to render the template with variables substituted. Do not read the file with `open()` -- that gives you the raw template source.

7. Connection Timeout on Send

Problem: `Messenger.send()` hangs for 30 seconds and then fails with a timeout error.

Cause: The SMTP server is unreachable from your network. The port may be blocked by a firewall, or the hostname may be wrong.

Fix: Test SMTP connectivity with `mailer.test_connection()`. Verify the hostname, port, and encryption settings. Check that your firewall allows outbound connections on the SMTP port. If you sit behind a corporate firewall, port 587 or 465 might be blocked -- ask your network administrator.

8. IMAP Host Not Configured

Problem: Calling `mailer.inbox()` raises `MessengerError: IMAP host not configured`.

Cause: You did not set `TINA4_MAIL_IMAP_HOST` in `.env` or pass `imap_host` to the constructor.

Fix: Add `TINA4_MAIL_IMAP_HOST=imap.example.com` and `TINA4_MAIL_IMAP_PORT=993` to `.env`. The IMAP host is separate from the SMTP host -- many providers use different hostnames for sending and reading.

Frontend with tina4css

1. The Problem with Frontend Toolchains

Your client wants a dashboard. You know the drill. Install Node.js. Run `npm install`. Wait for 200MB of `node_modules`. Configure webpack or Vite. Set up PostCSS. Add a CSS framework. Pray nothing breaks when you upgrade a dependency six months from now.

Tina4 skips all of that. The framework ships with **tina4css** -- a Bootstrap-compatible CSS framework -- and **frond.js** -- a lightweight JavaScript helper. Both arrive when you scaffold a project. No npm. No webpack. No build step. Link the files. Start building.

This chapter ends with a complete admin dashboard. Sidebar. Navigation. Cards. Tables. Modals. Dark mode. Progress bars. AJAX-driven user management. Zero npm dependencies.

---

2. What Ships with Tina4

Run `tina4 init python`. Several files appear in your project:

```
src/public/
■■■ css/
■   ■■■ tina4.css        # The CSS framework
■■■ js/
■   ■■■ tina4.min.js     # Core AJAX utilities
■   ■■■ frond.min.js     # Template engine client-side helpers
■   ■■■ tina4js.min.js   # Reactive frontend framework (tina4-js)
■■■ scss/
    ■■■ tina4.scss       # SCSS source (optional, for customization)
```

Include them in any template:

```
<link rel="stylesheet" href="/css/tina4.css">
<script src="/js/tina4.min.js"></script>
<script src="/js/frond.min.js"></script>
```

Include only what you need. See section 7 for the full JavaScript API reference. No CDN. No package manager. No version conflicts.

---

3. The Grid System

tina4css uses a 12-column responsive grid. The class names match Bootstrap. Know Bootstrap? You know tina4css.

```
<div class="container">
    <div class="row">
        <div class="col-md-4">
            <p>One third</p>
        </div>
        <div class="col-md-4">
            <p>One third</p>
        </div>
```

```
        <div class="col-md-4">
            <p>One third</p>
        </div>
    </div>
</div>
```

Breakpoints:

Class Prefix	Screen Width	Typical Device
`col-`	All sizes	Phones and up
`col-sm-`	>= 576px	Large phones
`col-md-`	>= 768px	Tablets
`col-lg-`	>= 992px	Desktops
`col-xl-`	>= 1200px	Large desktops

Columns stack vertically on screens smaller than their breakpoint. A `col-md-6` element takes half the row on tablets and up, full width on phones.

---

4. Components

Navbar

```
<nav class="navbar navbar-dark bg-dark">
    <div class="container">
        <a class="navbar-brand" href="/">My Dashboard</a>
        <ul class="navbar-nav">
            <li class="nav-item"><a class="nav-link" href="/dashboard">Dashboard</a></li>
            <li class="nav-item"><a class="nav-link" href="/products">Products</a></li>
            <li class="nav-item"><a class="nav-link" href="/settings">Settings</a></li>
        </ul>
    </div>
</nav>
```

Use `navbar-light bg-light` for a light theme, or `navbar-dark bg-primary` for a colored background.

Cards

```
<div class="card">
    <div class="card-header">Monthly Revenue</div>
    <div class="card-body">
        <h2 class="card-title">$12,450</h2>
        <p class="card-text">Up 12% from last month</p>
    </div>
    <div class="card-footer text-muted">Updated 5 minutes ago</div>
</div>
```

Buttons

```
<button class="btn btn-primary">Save</button>
<button class="btn btn-secondary">Cancel</button>
<button class="btn btn-danger">Delete</button>
<button class="btn btn-success">Publish</button>
<button class="btn btn-outline-primary">Outlined</button>
<button class="btn btn-sm btn-primary">Small</button>
<button class="btn btn-lg btn-primary">Large</button>
```

Tables

```
<table class="table table-striped table-hover">
    <thead>
        <tr>
            <th>Name</th>
            <th>Category</th>
            <th>Price</th>
            <th>Status</th>
        </tr>
    </thead>
    <tbody>
        <tr>
            <td>Wireless Keyboard</td>
            <td>Electronics</td>
            <td>$79.99</td>
            <td><span class="badge bg-success">In Stock</span></td>
        </tr>
        <tr>
            <td>Standing Desk</td>
            <td>Furniture</td>
            <td>$549.99</td>
            <td><span class="badge bg-danger">Out of Stock</span></td>
        </tr>
    </tbody>
</table>
```

Table variants: `table-bordered`, `table-striped`, `table-hover`, `table-sm` (compact), `table-responsive` (wraps in a scrollable container on small screens). Mix and match.

Alerts

```
<div class="alert alert-success">Product created.</div>
<div class="alert alert-danger">Failed to save changes.</div>
<div class="alert alert-warning alert-dismissible">
    Your trial expires in 3 days.
    <button type="button" class="close" data-dismiss="alert">&times;</button>
</div>
<div class="alert alert-info">A new version is available.</div>
```

Modals

```
<button class="btn btn-primary" data-toggle="modal" data-target="#confirmModal">Delete Product</

<div class="modal" id="confirmModal">
    <div class="modal-dialog">
        <div class="modal-content">
            <div class="modal-header">
                <h5 class="modal-title">Confirm Delete</h5>
                <button class="btn-close" data-dismiss="modal"></button>
            </div>
            <div class="modal-body">
                <p>Are you sure you want to delete this product? This action cannot be undone.</
            </div>
            <div class="modal-footer">
                <button class="btn btn-secondary" data-dismiss="modal">Cancel</button>
                <button class="btn btn-danger">Delete</button>
            </div>
        </div>
    </div>
</div>
```

tina4css includes the JavaScript for modal toggling. No jQuery required.

Progress Bars

Progress bars visualize completion, upload status, or system health. The outer `progress` container holds a `progress-bar` fill element.

```
<div class="progress">
    <div class="progress-bar bg-success" style="width: 75%">75%</div>
</div>

<div class="progress">
    <div class="progress-bar bg-warning progress-bar-striped" style="width: 45%">
        45% - Uploading...
    </div>
</div>
```

Set the width with inline `style`. Add `bg-success`, `bg-info`, `bg-warning`, or `bg-danger` for color. Add `progress-bar-striped` for animated stripes.

Badges

```
<span class="badge bg-primary">Primary</span>
<span class="badge bg-success">Active</span>
<span class="badge bg-warning">Pending</span>
<span class="badge bg-danger">Overdue</span>
<span class="badge bg-info">Info</span>
<span class="badge bg-dark">Dark</span>
```

Forms

```
<form>
    <div class="form-group">
        <label for="name" class="form-label">Product Name</label>
        <input type="text" class="form-control" id="name" placeholder="Enter product name">
    </div>

    <div class="form-group">
        <label for="category" class="form-label">Category</label>
        <select class="form-control" id="category">
            <option value="">Select a category</option>
            <option value="electronics">Electronics</option>
            <option value="furniture">Furniture</option>
        </select>
    </div>

    <div class="form-group">
        <label for="description" class="form-label">Description</label>
        <textarea class="form-control" id="description" rows="4"></textarea>
    </div>

    <div class="form-check">
        <input type="checkbox" class="form-check-input" id="featured">
        <label class="form-check-label" for="featured">Featured product</label>
    </div>

    <button type="submit" class="btn btn-primary mt-3">Save Product</button>
</form>
```

---

5. SCSS Customization

The default tina4css works out of the box. To customize colors, fonts, or spacing, edit the SCSS source.

Edit `src/public/scss/tina4.scss`:

```
// Override variables before importing the framework
$primary: #2d6a4f;
$secondary: #52b788;
$dark: #1b4332;
$font-family-base: 'Inter', sans-serif;
$border-radius: 8px;

// Import the framework
@import 'tina4-base';
```

Compile SCSS to CSS:

```
tina4 scss
```

```
Compiling SCSS...
  src/public/scss/tina4.scss -> src/public/css/tina4.css
Done (0.12s)
```

The compiled CSS replaces the default `tina4.css`. Your custom colors and fonts take effect across the entire application.

Live SCSS Compilation

During development, run SCSS compilation in watch mode:

```
tina4 scss --watch
```

Every save to a `.scss` file triggers a recompile. Combined with Tina4's live reload, changes appear in the browser within a second.

---

6. frond.js -- The JavaScript Helper

`frond.js` is a lightweight JavaScript library that ships with Tina4. It handles AJAX requests, form submission, JWT token management, and loading indicators. No jQuery. No Axios. No other dependency.

AJAX Requests

```
// GET request
frond.get("/api/products", function (data) {
    console.log("Products:", data);
});

// GET with error handling
frond.get("/api/products", function (data) {
    console.log(data);
}, function (error) {
    console.error("Failed:", error);
});

// POST request
frond.post("/api/products", {
    name: "New Product",
    price: 29.99
}, function (data) {
    console.log("Created:", data);
});

// PUT request
frond.put("/api/products/1", {
    name: "Updated Product"
}, function (data) {
    console.log("Updated:", data);
});

// DELETE request
frond.delete("/api/products/1", function (data) {
    console.log("Deleted:", data);
});
```

Form Submission via AJAX

```
<form id="product-form" data-frond-submit="/api/products" data-frond-method="POST">
    <input type="text" name="name" placeholder="Product name">
    <input type="number" name="price" placeholder="Price">
    <button type="submit">Create</button>
</form>

<script src="/js/frond.min.js"></script>
<script>
    frond.onFormSuccess("product-form", function (data) {
        alert("Product created: " + data.name);
    });

    frond.onFormError("product-form", function (error) {
        alert("Error: " + error.message);
    });
</script>
```

The `data-frond-submit` attribute tells frond.js to intercept the form submission and send it as an AJAX request. No page reload. frond.js serializes all form fields as JSON.

Token Management

frond.js manages JWT tokens. Store a token after login, and frond.js attaches it to every request.

```
// Store the token (usually after login)
frond.setToken("eyJhbGciOiJIUzI1NiIs...");

// All subsequent requests include:
// Authorization: Bearer eyJhbGciOiJIUzI1NiIs...
frond.get("/api/profile", function (data) {
    console.log("Profile:", data);
});

// Clear the token (logout)
frond.clearToken();
```

frond.js stores the token in `localStorage` and includes it as a `Bearer` token in the `Authorization` header on every request.

Loading Indicators

frond.js can show and hide a loading element during AJAX requests. Pass a CSS selector in the options object.

```
// Show a loading state while fetching
frond.get("/api/products", function (data) {
    renderProducts(data.products);
}, null, {
    loading: "#loadingSpinner"  // CSS selector for loading element
});
```

The element with id `loadingSpinner` appears while the request flies and disappears when it completes. Pair it with a spinner or "Loading..." text in your HTML:

```
<div id="loadingSpinner" class="text-center p-4" style="display: none;">
    Loading...
</div>
```

frond.js toggles `display: block` and `display: none` on the element. No extra CSS needed.

WebSocket (Covered in Chapter 23)

```
const ws = frond.ws("/ws/chat/general");
ws.on("message", function (data) {
    console.log("Message:", JSON.parse(data));
});
```

Connections drop. frond.js reconnects with exponential backoff. If the server restarts or the network blips, the client reconnects without intervention.

---

7. JavaScript API Reference

Tina4 ships three JavaScript files. Each serves a different purpose. Use them independently or together.

Including the Scripts

```
<script src="/js/tina4.min.js"></script>
<script src="/js/frond.min.js"></script>
<script src="/js/tina4js.min.js"></script>
```

All three live in `src/public/js/` and are served from `/js/`. Include only what you need.

7.1 tina4.min.js -- Core Utilities

Low-level helpers for AJAX page loading and form submission. Use this when you want simple dynamic page updates without a full framework.

`loadPage(url, targetId)`

Fetches HTML from `url` and injects it into the element with the given `id`.

```
<nav>
    <a href="#" onclick="loadPage('/dashboard', 'content')">Dashboard</a>
    <a href="#" onclick="loadPage('/settings', 'content')">Settings</a>
</nav>
<div id="content"><!-- pages load here --></div>

<script src="/js/tina4.min.js"></script>
```

`saveForm(formId, url, method)`

Serializes a form and submits it via AJAX. Prevents the default page reload.

```
<form id="product-form">
    <input type="text" name="name" placeholder="Product name">
    <input type="number" name="price" placeholder="Price">
    <button type="button" onclick="saveForm('product-form', '/api/products', 'POST')">
        Save
    </button>
</form>
```

`sendRequest(url, method, data, callback)`

Generic AJAX helper for any HTTP method. Returns the response to a callback function.

```
sendRequest("/api/products", "GET", null, function (response) {
    console.log("Products:", JSON.parse(response));
});

sendRequest("/api/products", "POST", { name: "Widget", price: 9.99 }, function (response) {
    console.log("Created:", JSON.parse(response));
});
```

7.2 frond.min.js -- Template Engine Client-Side Helpers

A companion to the Frond template engine. Handles AJAX form interception, WebSocket connections with auto-reconnect, JWT token refresh, and dynamic template loading.

AJAX Form Handling

Forms with `data-frond-submit` are intercepted. No page reload. No boilerplate.

```
<form id="login-form" data-frond-submit="/api/login" data-frond-method="POST">
    <input type="text" name="username" placeholder="Username">
    <input type="password" name="password" placeholder="Password">
    <button type="submit">Log In</button>
</form>

<script src="/js/frond.min.js"></script>
<script>
    frond.onFormSuccess("login-form", function (data) {
        frond.setToken(data.token);
        window.location.href = "/dashboard";
    });
</script>
```

WebSocket Auto-Reconnect

```
const ws = frond.ws("/ws/notifications");
ws.on("message", function (data) {
    const notification = JSON.parse(data);
    alert(notification.text);
});
// If the server restarts or the network blips, frond.js reconnects.
```

Token Refresh

JWT tokens stored via `frond.setToken()` are attached to every request. When a token expires, frond.js triggers a refresh before retrying the request.

```
frond.setToken("eyJhbGciOiJIUzI1NiIs...");
// All subsequent frond.get/post/put/delete calls include the token.
// When the token expires, frond.js calls the refresh endpoint.
```

Dynamic Template Loading

Load server-rendered Frond templates into any element without a full page reload.

```
frond.loadTemplate("/templates/user-card", { userId: 42 }, "user-panel");
```

```
// Fetches the rendered template and injects it into #user-panel.
```

7.3 tina4js.min.js -- Reactive Frontend Framework

A standalone reactive framework for building interactive client-side applications. Provides signals, computed values, effects, Web Components, client-side routing, and built-in fetch and WebSocket wrappers. This is the **tina4-js** project.

Reactive State: `signal()`, `computed()`, `effect()`

```
import { signal, computed, effect } from "/js/tina4js.min.js";

const count = signal(0);
const doubled = computed(() => count.value * 2);

effect(() => {
    console.log(`Count: ${count.value}, Doubled: ${doubled.value}`);
});

count.value = 5; // logs "Count: 5, Doubled: 10"
```

DOM Rendering: `html` Tagged Template

```
import { signal, html } from "/js/tina4js.min.js";

const name = signal("World");

const app = html`
    <div>
        <h1>Hello, ${name}!</h1>
        <input value="${name}" oninput="${(e) => name.value = e.target.value}" />
    </div>
`;

document.getElementById("app").append(app);
```

Web Components: `Tina4Element`

```
import { Tina4Element, signal, html } from "/js/tina4js.min.js";

class CounterButton extends Tina4Element {
    setup() {
        this.count = signal(0);
    }
    render() {
        return html`
            <button onclick="${() => this.count.value++}">
                Clicked ${this.count} times
            </button>
        `;
    }
}
customElements.define("counter-button", CounterButton);
```

Use it in HTML:

```
<counter-button></counter-button>
<script type="module" src="/js/counter-button.js"></script>
```

Fetch Wrapper: `api()`

```
import { api } from "/js/tina4js.min.js";

const products = await api("/api/products");            // GET
await api("/api/products", { method: "POST", body: { name: "Widget" } });
```

WebSocket Client: `ws()`

```
import { ws } from "/js/tina4js.min.js";

const socket = ws("/ws/chat");
socket.on("message", (data) => console.log(data));
socket.send({ text: "Hello" });
```

Client-Side Routing: `route()`, `navigate()`

```
import { route, navigate, html } from "/js/tina4js.min.js";

route("/", () => html`<h1>Home</h1>`);
route("/about", () => html`<h1>About</h1>`);

// Navigate programmatically
navigate("/about");
```

---

8. Building an Admin Dashboard

A complete admin dashboard. The kind of page that powers the backend of every web application.

Base Template

Create `src/templates/base.html`:

```
<!DOCTYPE html>
<html lang="en" data-theme="light">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>{% block title %}Dashboard{% endblock %}</title>
    <link rel="stylesheet" href="/css/tina4.css">
    <script>
        var t = localStorage.getItem("theme");
        if (t) document.documentElement.setAttribute("data-theme", t);
    </script>
</head>
<body>
    <nav class="navbar navbar-dark bg-dark">
        <div class="container-fluid">
            <a class="navbar-brand" href="/admin">TaskFlow Admin</a>
            <ul class="navbar-nav">
                <li class="nav-item"><a class="nav-link" href="/admin">Dashboard</a></li>
                <li class="nav-item"><a class="nav-link" href="/admin/products">Products</a></li
                <li class="nav-item"><a class="nav-link" href="/admin/users">Users</a></li>
                <li class="nav-item">
                    <button class="btn btn-sm btn-outline-light" onclick="toggleDarkMode()">Dark
                </li>
```

```
            </ul>
        </div>
    </nav>

    <div class="container-fluid mt-4">
        <div class="row">
            <div class="col-md-2">
                {% block sidebar %}
                <div class="list-group">
                    <a href="/admin" class="list-group-item list-group-item-action">Overview</a>
                    <a href="/admin/products" class="list-group-item list-group-item-action">Pro
                    <a href="/admin/orders" class="list-group-item list-group-item-action">Order
                    <a href="/admin/customers" class="list-group-item list-group-item-action">Cu
                    <a href="/admin/reports" class="list-group-item list-group-item-action">Repo
                    <a href="/admin/settings" class="list-group-item list-group-item-action">Set
                </div>
                {% endblock %}
            </div>
            <div class="col-md-10">
                {% block content %}{% endblock %}
            </div>
        </div>
    </div>

    <script src="/js/frond.min.js"></script>
    <script>
        function toggleDarkMode() {
            var html = document.documentElement;
            var current = html.getAttribute("data-theme");
            html.setAttribute("data-theme", current === "dark" ? "light" : "dark");
            localStorage.setItem("theme", current === "dark" ? "light" : "dark");
        }
    </script>
    {% block scripts %}{% endblock %}
</body>
</html>
```

Dashboard Page

Create `src/templates/dashboard.html`:

```
{% extends "base.html" %}

{% block title %}Dashboard - Admin{% endblock %}

{% block content %}
    <h1>Dashboard</h1>

    <div class="row mb-4">
        <div class="col-md-3">
            <div class="card">
                <div class="card-body">
                    <h6 class="card-subtitle text-muted">Total Products</h6>
                    <h2 class="card-title">{{ stats.total_products }}</h2>
                </div>
            </div>
        </div>
        <div class="col-md-3">
            <div class="card">
```

```
            <div class="card-body">
                <h6 class="card-subtitle text-muted">Total Orders</h6>
                <h2 class="card-title">{{ stats.total_orders }}</h2>
            </div>
        </div>
    </div>
    <div class="col-md-3">
        <div class="card">
            <div class="card-body">
                <h6 class="card-subtitle text-muted">Revenue</h6>
                <h2 class="card-title">${{ stats.revenue }}</h2>
            </div>
        </div>
    </div>
    <div class="col-md-3">
        <div class="card">
            <div class="card-body">
                <h6 class="card-subtitle text-muted">Active Users</h6>
                <h2 class="card-title">{{ stats.active_users }}</h2>
            </div>
        </div>
    </div>
</div>

<div class="row">
    <div class="col-md-8">
        <div class="card">
            <div class="card-header">Recent Orders</div>
            <div class="card-body">
                <table class="table table-striped">
                    <thead>
                        <tr>
                            <th>Order</th>
                            <th>Customer</th>
                            <th>Amount</th>
                            <th>Status</th>
                        </tr>
                    </thead>
                    <tbody>
                        {% for order in recent_orders %}
                        <tr>
                            <td>#{{ order.id }}</td>
                            <td>{{ order.customer }}</td>
                            <td>${{ order.amount }}</td>
                            <td>
                                <span class="badge bg-{{ order.badge }}">{{ order.status }}<
                            </td>
                        </tr>
                        {% endfor %}
                    </tbody>
                </table>
            </div>
        </div>
    </div>
    <div class="col-md-4">
        <div class="card">
            <div class="card-header">Quick Actions</div>
            <div class="card-body">
                <a href="/admin/products/new" class="btn btn-primary btn-block mb-2">Add Pro
```

```
                    <a href="/admin/orders" class="btn btn-outline-primary btn-block mb-2">View
                    <a href="/admin/reports" class="btn btn-outline-secondary btn-block">Generat
                </div>
            </div>

            <div class="card mt-3">
                <div class="card-header">System Health</div>
                <div class="card-body">
                    <p><strong>CPU:</strong></p>
                    <div class="progress mb-3">
                        <div class="progress-bar bg-success" style="width: {{ stats.cpu_usage }}
                            {{ stats.cpu_usage }}%
                        </div>
                    </div>
                    <p><strong>Memory:</strong></p>
                    <div class="progress mb-3">
                        <div class="progress-bar bg-info" style="width: {{ stats.memory_usage }}
                            {{ stats.memory_usage }}%
                        </div>
                    </div>
                    <p><strong>Disk:</strong></p>
                    <div class="progress">
                        <div class="progress-bar bg-warning" style="width: {{ stats.disk_usage }
                            {{ stats.disk_usage }}%
                        </div>
                    </div>
                </div>
            </div>
        </div>
    </div>
{% endblock %}
```

Dashboard Route

Create `src/routes/admin.py`:

```
from tina4_python.core.router import get, template

@get("/admin")
async def admin_dashboard(request, response):
    stats = {
        "total_products": 156,
        "total_orders": 1243,
        "revenue": "24,580",
        "active_users": 89,
        "cpu_usage": 42,
        "memory_usage": 68,
        "disk_usage": 55
    }

    recent_orders = [
        {"id": 1042, "customer": "Alice Johnson", "amount": "129.99", "status": "Shipped", "badg
        {"id": 1041, "customer": "Bob Smith", "amount": "549.99", "status": "Processing", "badge
        {"id": 1040, "customer": "Carol White", "amount": "79.99", "status": "Delivered", "badge
        {"id": 1039, "customer": "Dave Brown", "amount": "34.99", "status": "Cancelled", "badge"
    ]

    return response(template("dashboard.html", stats=stats, recent_orders=recent_orders))
```

Start the server and visit `http://localhost:7146/admin`. You see a sidebar, stat cards, a data table, quick action buttons, and system health progress bars. Zero npm dependencies.

---

9. Dark Mode

tina4css supports dark mode through a single attribute on the `<html>` element:

```
<!-- Light mode (default) -->
<html data-theme="light">

<!-- Dark mode -->
<html data-theme="dark">
```

The toggle function from the base template handles switching:

```
function toggleDarkMode() {
    var html = document.documentElement;
    var current = html.getAttribute("data-theme");
    html.setAttribute("data-theme", current === "dark" ? "light" : "dark");
    localStorage.setItem("theme", current === "dark" ? "light" : "dark");
}
```

Dark mode transforms every surface. Backgrounds darken. Text colors invert. Borders shift. Cards, tables, buttons -- all adjust contrast. Zero CSS rules required from you.

Respecting System Preference

Match the user's operating system dark mode setting:

```
var saved = localStorage.getItem("theme");
if (saved) {
    document.documentElement.setAttribute("data-theme", saved);
} else if (window.matchMedia("(prefers-color-scheme: dark)").matches) {
    document.documentElement.setAttribute("data-theme", "dark");
}
```

---

10. Responsive Design

tina4css is mobile-first. Components stack on small screens and expand on larger ones.

Responsive Sidebar

On mobile, the sidebar collapses into a toggle:

```
<button class="btn btn-dark d-md-none" onclick="toggleSidebar()">Menu</button>

<div id="sidebar" class="d-none d-md-block col-md-2">
    <!-- sidebar content -->
</div>

<script>
function toggleSidebar() {
    var sidebar = document.getElementById("sidebar");
    sidebar.classList.toggle("d-none");
```

```
}
</script>
```

Responsive Tables

Tables scroll horizontally on small screens:

```
<div class="table-responsive">
    <table class="table">
        <!-- table content -->
    </table>
</div>
```

Hiding Elements by Screen Size

```
<div class="d-none d-md-block">Only visible on tablet and up</div>
<div class="d-md-none">Only visible on mobile</div>
```

---

11. Building a Users Page with AJAX

A user management page. Data loads via AJAX using frond.js. No full page reloads. The table populates from the API. Users create, edit, and delete records through modals and inline actions.

The Template

Create `src/templates/admin/users.html`:

```
{% extends "base.html" %}

{% block title %}Users - Admin{% endblock %}

{% block content %}
    <div class="card">
        <div class="card-header d-flex justify-content-between align-items-center">
            <span>All Users</span>
            <button class="btn btn-sm btn-primary" data-toggle="modal" data-target="#addUserModa
                Add User
            </button>
        </div>
        <div class="card-body p-0">
            <div id="loadingSpinner" class="text-center p-4" style="display: none;">
                Loading...
            </div>
            <table class="table table-hover m-0" id="usersTable">
                <thead>
                    <tr>
                        <th>ID</th>
                        <th>Name</th>
                        <th>Email</th>
                        <th>Created</th>
                        <th>Actions</th>
                    </tr>
                </thead>
                <tbody id="usersBody">
                    <!-- Populated by JavaScript -->
                </tbody>
            </table>
```

```
        </div>
    </div>

    <!-- Add User Modal -->
    <div class="modal" id="addUserModal">
        <div class="modal-dialog">
            <div class="modal-content">
                <div class="modal-header">
                    <h5 class="modal-title">Add New User</h5>
                    <button type="button" class="btn-close" data-dismiss="modal"></button>
                </div>
                <div class="modal-body">
                    <form id="addUserForm">
                        <div class="form-group">
                            <label for="userName">Name</label>
                            <input type="text" class="form-control" name="name" id="userName" re
                        </div>
                        <div class="form-group">
                            <label for="userEmail">Email</label>
                            <input type="email" class="form-control" name="email" id="userEmail"
                        </div>
                    </form>
                </div>
                <div class="modal-footer">
                    <button class="btn btn-secondary" data-dismiss="modal">Cancel</button>
                    <button class="btn btn-primary" onclick="createUser()">Create User</button>
                </div>
            </div>
        </div>
    </div>

    <div id="alertArea" class="mt-3"></div>
{% endblock %}

{% block scripts %}
<script>
    function loadUsers() {
        frond.get("/api/users", function (data) {
            var tbody = document.getElementById("usersBody");
            tbody.innerHTML = "";

            if (data.data && data.data.length > 0) {
                data.data.forEach(function (user) {
                    var row = '<tr>'
                        + '<td>' + user.id + '</td>'
                        + '<td>' + user.name + '</td>'
                        + '<td>' + user.email + '</td>'
                        + '<td>' + user.created_at + '</td>'
                        + '<td>'
                        + '<button class="btn btn-sm btn-outline-primary" onclick="editUser(' +
                        + '<button class="btn btn-sm btn-outline-danger" onclick="deleteUser(' +
                        + '</td>'
                        + '</tr>';
                    tbody.innerHTML += row;
                });
            } else {
                tbody.innerHTML = '<tr><td colspan="5" class="text-center p-4">No users found</t
            }
        }, null, { loading: "#loadingSpinner" });
```

```
    }

    function createUser() {
        var name = document.getElementById("userName").value;
        var email = document.getElementById("userEmail").value;

        frond.post("/api/users", { name: name, email: email }, function (data) {
            document.getElementById("alertArea").innerHTML =
                '<div class="alert alert-success">User "' + data.name + '" created.</div>';
            loadUsers();
        }, function (error) {
            document.getElementById("alertArea").innerHTML =
                '<div class="alert alert-danger">Error creating user.</div>';
        });
    }

    function deleteUser(id) {
        if (confirm("Are you sure you want to delete this user?")) {
            frond.delete("/api/users/" + id, function () {
                loadUsers();
            });
        }
    }

    // Load users on page load
    loadUsers();
</script>
{% endblock %}
```

The Route

```
from tina4_python.core.router import get, template

@get("/admin/users")
async def admin_users(request, response):
    return response(template("admin/users.html"))
```

This page loads users via an AJAX call to `/api/users` (provided by auto-CRUD on the User model from Chapter 6). The loading indicator appears while the request flies. The table populates when data arrives. Users add records through a modal form and delete with confirmation -- all without full page reloads.

---

12. HTML Builder -- Generating HTML in Code

Sometimes you need HTML from a route handler without a template -- a dynamic email body, an AJAX fragment, or a one-off snippet. String concatenation is fragile. The `HTMLElement` class builds HTML with auto-escaping.

Direct Construction

```
from tina4_python.HtmlElement import HTMLElement

el = HTMLElement("div", {"class": "card"}, ["Hello"])
str(el)  # <div class="card">Hello</div>
```

The constructor takes three arguments: tag name, an attribute dictionary, and a list of children (strings or other `HTMLElement` instances).

Builder Pattern

Call an element to append children and return a new element:

```
page = HTMLElement("div")(
    HTMLElement("h1")("Dashboard"),
    HTMLElement("p")("Welcome back."),
)
```

Pass a dictionary to merge attributes:

```
link = HTMLElement("a")({"href": "/home", "class": "nav-link"}, "Home")
# <a href="/home" class="nav-link">Home</a>
```

Helper Functions

Typing `HTMLElement` everywhere gets verbose. `add_html_helpers()` injects shorthand functions -- `_div()`, `_p()`, `_a()`, `_span()`, `_h1()`, `_table()`, and every other standard HTML tag -- into your module:

```
from tina4_python.HtmlElement import add_html_helpers

add_html_helpers(globals())

html = _div({"class": "alert alert-success"},
    _strong("Done!"),
    _p("Your changes have been saved."),
)
```

The first argument can be an attribute dictionary. Everything else becomes children.

Void Tags and Auto-Escaping

Void tags render without a closing tag:

```
HTMLElement("img", {"src": "logo.png", "alt": "Logo"})
# <img src="logo.png" alt="Logo">

HTMLElement("br")
# <br>
```

All attribute values and text children are HTML-escaped. User input is safe by default -- no XSS vectors from forgotten escaping.

Use Case -- HTML from a Route

```
from tina4_python.core.router import get
from tina4_python.HtmlElement import add_html_helpers

add_html_helpers(globals())

@get("/api/status-badge")
async def status_badge(request, response):
    status = request.params.get("status", "unknown")
    color = "success" if status == "active" else "danger"

    badge = _span({"class": f"badge bg-{color}"}, status)
    return response(str(badge))
```

No template file needed. The builder handles escaping, nesting, and rendering in one place.

---

13. Exercise: Build an Admin Dashboard

Build an admin dashboard for a product management system.

Requirements

- Create a base template with:
- A dark navbar with the app name and navigation links
- A sidebar with menu items (Dashboard, Products, Orders, Settings)
- A main content area
- Dark mode toggle that persists across page loads
- Create a dashboard page at `GET /admin` with:
- Four stat cards (Products, Orders, Revenue, Users)
- A table showing recent orders with status badges
- Progress bars showing system health (CPU, Memory, Disk)
- Create a users page at `GET /admin/users` with:
- A table of users loaded via AJAX using frond.js
- An "Add User" button that opens a modal with a form
- AJAX form submission that refreshes the table without a page reload
- A loading indicator while data fetches
- Use tina4css classes throughout (no custom CSS needed)

Test by:

- Visit `http://localhost:7146/admin` -- see the dashboard with stats, orders, and progress bars
- Click "Dark Mode" -- the entire page switches to dark theme

---

- Refresh the page -- dark mode persists
- Resize the browser to mobile width -- the sidebar collapses
- Visit `http://localhost:7146/admin/users` -- see the user table loaded via AJAX

---

14. Solution

The base template, dashboard page, and users page are shown in sections 8, 11, and above. The product list page follows the same AJAX pattern from section 11, but for products instead of users.

For the users page, use auto-CRUD on the User model (`auto_crud = True`) so the API endpoints exist at `/api/users`. Load the table with `frond.get("/api/users", ...)` and handle form submission with `frond.post("/api/users", ...)`.

Add the product list route:

```
@get("/admin/products")
async def admin_products(request, response):
    search = request.params.get("search", "")
    selected_category = request.params.get("category", "")

    products = [
        {"id": 1, "name": "Wireless Keyboard", "category": "Electronics", "price": "79.99", "in_
        {"id": 2, "name": "Standing Desk", "category": "Furniture", "price": "549.99", "in_stock
        {"id": 3, "name": "Coffee Grinder", "category": "Kitchen", "price": "49.99", "in_stock":
        {"id": 4, "name": "Yoga Mat", "category": "Fitness", "price": "29.99", "in_stock": True}
        {"id": 5, "name": "USB-C Hub", "category": "Electronics", "price": "49.99", "in_stock":
    ]

    if selected_category:
        products = [p for p in products if p["category"] == selected_category]

    if search:
        products = [p for p in products if search.lower() in p["name"].lower()]

    categories = ["Electronics", "Furniture", "Kitchen", "Fitness"]

    return response(template("products.html",
        products=products,
        categories=categories,
        search=search,
        selected_category=selected_category
    ))
```

---

15. Gotchas

1. CSS Not Loading

Problem: The page looks unstyled -- no colors, no layout, plain text.

Cause: The path to `tina4.css` is wrong. The file lives in `src/public/css/` but the link points elsewhere.

Fix: Tina4 serves everything in `src/public/` from the root path. Link to `/css/tina4.css` (not `/src/public/css/tina4.css`). Verify the file exists at `src/public/css/tina4.css`.

2. frond.js Functions Not Found

Problem: `frond.get is not a function` or `frond is not defined` in the browser console.

Cause: The `frond.min.js` script tag is missing or placed after the code that uses it.

Fix: Include `<script src="/js/frond.min.js"></script>` before any script that calls `frond.*`. Put it at the bottom of the body, before your custom scripts.

3. Dark Mode Flickers on Page Load

Problem: The page loads in light mode and then flashes to dark mode.

Cause: The dark mode JavaScript runs after the page renders. The browser paints the light theme first, then switches.

Fix: Add the theme detection script in the `<head>` (before the body renders):

```
<head>
    <script>
        var t = localStorage.getItem("theme");
        if (t) document.documentElement.setAttribute("data-theme", t);
    </script>
</head>
```

4. SCSS Not Compiling

Problem: You edited `tina4.scss` but the CSS did not change.

Cause: SCSS does not compile on its own. You need to run `tina4 scss` or use `--watch` mode.

Fix: Run `tina4 scss --watch` during development. For production builds, run `tina4 scss` as part of your build process.

5. Modal Does Not Open

Problem: Clicking the button does nothing -- the modal stays hidden.

Cause: The `data-toggle` and `data-target` attributes require frond.js to be loaded. Without it, no JavaScript handles the modal toggle.

Fix: Ensure `frond.min.js` is loaded. Verify the `data-target` matches the modal's `id` exactly (including the `#` prefix).

6. Grid Columns Do Not Stack on Mobile

Problem: Columns stay side-by-side on phone screens instead of stacking vertically.

Cause: You used `col-4` instead of `col-md-4`. The `col-4` class applies at all screen sizes.

Fix: Use responsive prefixes: `col-md-4` means "one-third on medium screens and up, full width on small screens."

7. Static Files Return 404

Problem: CSS, JS, or image files return 404 Not Found.

Cause: The files are not in the `src/public/` directory.

Fix: Static files must be in `src/public/`. The URL path maps to the file path within that directory. `/css/tina4.css` maps to `src/public/css/tina4.css`.

8. Form Data Not Reaching the Server

Problem: `frond.post()` sends the request but `request.body` is empty on the server.

Cause: frond.js sends JSON by default. Passing a `FormData` object or building the data object wrong prevents correct parsing.

Fix: Pass a plain JavaScript object to `frond.post()`. Do not use `new FormData()` -- frond.js handles serialization. Make sure your object keys match what the server expects.

9. Loading Indicator Never Disappears

Problem: The loading spinner shows but never hides after the AJAX request completes.

Cause: The CSS selector passed to the `loading` option does not match any element, or the element uses a CSS class to toggle visibility instead of inline `display`.

Fix: frond.js toggles `display: block` and `display: none`. Make sure the element exists and its initial style is `display: none`. Use an `id` selector: `{ loading: "#loadingSpinner" }`.

Testing

1. Why Tests Matter More Than You Think

Friday afternoon. Your client reports a critical bug in production. You fix it -- one line of code. But did that fix break something else? 47 routes. 12 ORM models. 3 middleware functions. Clicking through every page: an hour. Running the test suite: 2 seconds.

```
tina4 test

============================= test session starts ==============================
platform darwin -- Python 3.13.5, pytest-9.0.2, pluggy-1.6.0
rootdir: /path/to/your-project
configfile: pyproject.toml
collected 9 items

tests/test_product.py .....                                              [ 55%]
tests/test_auth.py ....                                                  [100%]

============================== 9 passed in 0.34s ===============================
```

Everything still works. You deploy with confidence. Weekend intact.

Tina4 ships a `Test` class layered on top of pytest. Pytest handles discovery, parallelism, and the standard `./F/E` progress dots; the `Test` class gives you the HTTP test client and the `assert_equal` / `assert_true` / `assert_not_none` helpers from the chapter. `tina4 test` just runs pytest against the `tests/` directory.

---

2. Your First Test

Tests live in the `tests/` directory. Every `.py` file in that directory is auto-discovered when you run `tina4 test`.

Create `tests/test_basic.py`:

```
from tina4_python.test import Test, assert_equal, assert_true

class BasicTest(Test):

    def test_addition(self):
        assert_equal(2 + 2, 4, "Basic addition should work")

    def test_string_contains(self):
        greeting = "Hello, World!"
        assert_true("World" in greeting, "Greeting should contain 'World'")

    def test_array_length(self):
        items = [1, 2, 3, 4, 5]
        assert_equal(len(items), 5, "List should have 5 items")
```

Run it:

```
tina4 test
```

```
============================= test session starts ==============================
platform darwin -- Python 3.13.5, pytest-9.0.2, pluggy-1.6.0
collected 3 items

tests/test_basic.py ...                                                  [100%]

============================== 3 passed in 0.02s ===============================
```

How It Works

- Your test class extends `Test`.
- Every method that starts with `test` is a test case. The method name is converted to a readable label: `test_addition` stays as `test_addition`.
- Inside each test, you call assertion functions to verify behavior.
- If all assertions pass, the test passes. If any assertion fails, the test fails and you see the failure message.

---

3. Assertion Methods

Tina4's testing framework provides these assertion functions:

assert_equal(actual, expected, message)

Checks that two values are equal.

```
assert_equal(4, 4, "Should be equal")              # PASS
assert_equal("hello", "hello", "Strings match")    # PASS
assert_equal(4, 5, "Not equal")                     # FAIL
```

assert_true(actual, expected, message)

Checks that a value is truthy.

```
assert_true(True, "Should be true")                # PASS
assert_true(1, "1 is truthy")                      # PASS
assert_true("yes", "Non-empty string is truthy")   # PASS
assert_true(False, "This fails")                   # FAIL
assert_true(0, "Zero is falsy")                    # FAIL
```

assert_false(actual, expected, message)

Checks that a value is falsy.

```
assert_false(False, "Should be false")             # PASS
assert_false(0, "Zero is falsy")                   # PASS
assert_false("", "Empty string is falsy")          # PASS
assert_false(True, "This fails")                   # FAIL
```

assert*raises(callable, exception*class, message)

Checks that a function raises a specific exception.

```
assert_raises(
    lambda: int("not-a-number"),
    ValueError,
```

```
    "Should raise ValueError"
)

assert_raises(
    lambda: 10 / 0,
    ZeroDivisionError,
    "Should raise on division by zero"
)
```

assert*not*equal(actual, expected, message)

Checks that two values are not equal.

```
assert_not_equal("hello", "world", "Strings differ")  # PASS
assert_not_equal(4, 4, "Same values")                  # FAIL
```

assert_none(actual, expected, message)

Checks that a value is None.

```
assert_none(None, "Should be None")     # PASS
assert_none("hello", "Not None")        # FAIL
```

assert*not*none(actual, expected, message)

Checks that a value is not None.

```
assert_not_none("hello", "Has value")   # PASS
assert_not_none(None, "Is None")        # FAIL
```

---

4. Testing ORM Models

Let us test a Product model. The test creates records, loads them, updates them, and deletes them.

Create `tests/test_product.py`:

```
from tina4_python.test import Test, assert_equal, assert_true, assert_not_none
from src.orm.Product import Product   # assumes src/orm/Product.py exists

class ProductTest(Test):

    def test_create_product(self):
        product = Product()
        product.name = "Test Widget"
        product.category = "Testing"
        product.price = 19.99
        product.in_stock = True
        product.save()

        assert_not_none(product.id, "Product should have an ID after save")
        assert_true(product.id > 0, "Product ID should be positive")

    def test_load_product(self):
        product = Product()
        product.name = "Load Test Widget"
        product.category = "Testing"
        product.price = 29.99
```

```
    product.save()

    loaded = Product.find(product.id)

    assert_equal(loaded.name, "Load Test Widget", "Name should match")
    assert_equal(loaded.category, "Testing", "Category should match")
    assert_equal(loaded.price, 29.99, "Price should match")

def test_update_product(self):
    product = Product()
    product.name = "Update Test Widget"
    product.price = 10.00
    product.save()

    product_id = product.id

    product.name = "Updated Widget"
    product.price = 15.00
    product.save()

    reloaded = Product.find(product_id)

    assert_equal(reloaded.name, "Updated Widget", "Name should be updated")
    assert_equal(reloaded.price, 15.00, "Price should be updated")

def test_delete_product(self):
    product = Product()
    product.name = "Delete Me"
    product.price = 5.00
    product.save()

    product_id = product.id
    product.delete()

    gone = Product.find(product_id)

    assert_true(gone is None, "Deleted product should not be loadable")

def test_select_with_filter(self):
    p1 = Product()
    p1.name = "Filter Test A"
    p1.category = "FilterCat"
    p1.price = 10.00
    p1.save()

    p2 = Product()
    p2.name = "Filter Test B"
    p2.category = "FilterCat"
    p2.price = 20.00
    p2.save()

    products, count = Product.where("category = ?", ["FilterCat"])

    assert_true(len(products) >= 2, "Should find at least 2 FilterCat products")

    names = [p.name for p in products]
    assert_true("Filter Test A" in names, "Should include Filter Test A")
    assert_true("Filter Test B" in names, "Should include Filter Test B")
```

Run it:

```
tina4 test

============================= test session starts ==============================
platform darwin -- Python 3.13.5, pytest-9.0.2, pluggy-1.6.0
collected 5 items

tests/test_product.py .....                                              [100%]

============================== 5 passed in 0.18s ===============================
```

Test Database

By default, `tina4 test` uses a separate test database so your development data is not affected. The test database is created at `data/test.db` (SQLite) and is reset before each test run. If you want to use a different database for tests, set it in `.env`:

```
TINA4_DATABASE_URL=sqlite:///data/test.db
```

---

5. Testing Routes

Tina4 provides a test client for making HTTP requests to your routes without starting a server.

Create `tests/test_routes.py`:

```
from tina4_python.test import Test, assert_equal, assert_true, assert_not_none
import json

class RouteTest(Test):

    def test_health_endpoint(self):
        resp = self.get("/health")

        assert_equal(resp.status, 200, "Health check should return 200")

        body = json.loads(resp.body)
        assert_equal(body["status"], "ok", "Status should be 'ok'")
        assert_not_none(body.get("version"), "Should include version")

    def test_get_products(self):
        resp = self.get("/api/products")

        assert_equal(resp.status, 200, "Should return 200")

        body = json.loads(resp.body)
        assert_true("data" in body or "products" in body, "Should contain product data")

    def test_create_product(self):
        resp = self.post("/api/products", json={
            "name": "Route Test Product",
            "category": "Testing",
            "price": 42.00
        })

        assert_equal(resp.status, 201, "Should return 201 Created")
```

```
        body = json.loads(resp.body)
        assert_equal(body["name"], "Route Test Product", "Name should match")
        assert_equal(body["price"], 42.00, "Price should match")

    def test_get_product_not_found(self):
        resp = self.get("/api/products/99999")

        assert_equal(resp.status, 404, "Should return 404 for missing product")

    def test_create_product_validation(self):
        resp = self.post("/api/products", json={})

        assert_equal(resp.status, 400, "Should return 400 for empty body")

    def test_delete_product(self):
        # Create a product first
        create_resp = self.post("/api/products", json={
            "name": "To Be Deleted",
            "price": 1.00
        })
        body = json.loads(create_resp.body)
        product_id = body["id"]

        # Delete it
        delete_resp = self.delete(f"/api/products/{product_id}")
        assert_equal(delete_resp.status, 204, "Should return 204 No Content")

        # Verify it is gone
        get_resp = self.get(f"/api/products/{product_id}")
        assert_equal(get_resp.status, 404, "Should return 404 after deletion")
```

Test Client Methods

The test client provides methods for all HTTP verbs:

```
# GET request
resp = self.get("/api/products")

# GET with query parameters
resp = self.get("/api/products?category=Electronics&page=2")

# POST with JSON body
resp = self.post("/api/products", json={"name": "Widget", "price": 9.99})

# PUT with JSON body
resp = self.put("/api/products/1", json={"name": "Updated Widget"})

# PATCH with JSON body
resp = self.patch("/api/products/1", json={"price": 12.99})

# DELETE
resp = self.delete("/api/products/1")

# Request with custom headers
resp = self.get("/api/profile", headers={
    "Authorization": "Bearer eyJhbGciOiJIUzI1NiIs..."
})
```

Response Object

The response object has these properties:

```
resp.status        # HTTP status code (200, 201, 404, etc.)
resp.body          # Response body as raw bytes
resp.text()        # Decode resp.body to str
resp.json()        # Parse resp.body as JSON (dict or list)
resp.headers       # Response headers as a dict (lowercased keys)
resp.content_type  # Content-Type header value
```

---

6. Testing Authentication

Create `tests/test_auth.py`:

```
from tina4_python.test import Test, assert_equal, assert_true, assert_not_none
import json

class AuthTest(Test):

    def test_login_with_valid_credentials(self):
        resp = self.post("/api/auth/login", json={
            "email": "admin@example.com",
            "password": "correct-password"
        })

        assert_equal(resp.status, 200, "Login should succeed")

        body = json.loads(resp.body)
        assert_not_none(body.get("token"), "Should return a JWT token")
        assert_true(len(body["token"]) > 50, "Token should be a substantial string")

    def test_login_with_invalid_password(self):
        resp = self.post("/api/auth/login", json={
            "email": "admin@example.com",
            "password": "wrong-password"
        })

        assert_equal(resp.status, 401, "Should reject invalid password")

    def test_login_with_missing_fields(self):
        resp = self.post("/api/auth/login", json={
            "email": "admin@example.com"
        })

        assert_true(
            resp.status in (400, 401),
            "Should reject missing password"
        )

    def test_protected_route_without_token(self):
        resp = self.get("/api/profile")

        assert_equal(resp.status, 401, "Should reject unauthenticated request")

    def test_protected_route_with_valid_token(self):
        # Login first to get a token
```

```
        login_resp = self.post("/api/auth/login", json={
            "email": "admin@example.com",
            "password": "correct-password"
        })
        login_body = json.loads(login_resp.body)
        token = login_body["token"]

        # Access protected route with token
        resp = self.get("/api/profile", headers={
            "Authorization": f"Bearer {token}"
        })

        assert_equal(resp.status, 200, "Should allow authenticated request")

        body = json.loads(resp.body)
        assert_equal(body["user"]["email"], "admin@example.com", "Should return user data")

    def test_protected_route_with_invalid_token(self):
        resp = self.get("/api/profile", headers={
            "Authorization": "Bearer invalid.token.here"
        })

        assert_equal(resp.status, 401, "Should reject invalid token")
```

---

7. Setup and Teardown

Use `set_up()` and `tear_down()` methods to run code before and after each test:

```
from tina4_python.test import Test, assert_equal, assert_not_none
import time

class UserTest(Test):

    def set_up(self):
        # Runs before each test
        user = User()
        user.name = "Test User"
        user.email = f"test-{int(time.time())}@example.com"
        user.save()
        self.user_id = user.id

    def tear_down(self):
        # Runs after each test
        if self.user_id:
            user = User.find(self.user_id)
            if user:
                user.delete()

    def test_user_exists(self):
        user = User.find(self.user_id)
        assert_not_none(user.id, "User should exist")
        assert_equal(user.name, "Test User", "Name should match")

    def test_update_user(self):
        user = User.find(self.user_id)
        user.name = "Updated Name"
        user.save()
```

```
        reloaded = User.find(self.user_id)
        assert_equal(reloaded.name, "Updated Name", "Name should be updated")
```

`set_up()` runs before every test method, and `tear_down()` runs after every test method, regardless of whether the test passed or failed. This keeps tests isolated -- each test starts with a clean state.

---

8. Running Tests

Run All Tests

```
tina4 test
```

Run a Specific Test File

```
tina4 test --file tests/test_product.py
```

Run a Specific Test Method

```
tina4 test --file tests/test_product.py --method test_create_product
```

Verbose Output

```
tina4 test --verbose
```

```
============================= test session starts ==============================
platform darwin -- Python 3.13.5, pytest-9.0.2, pluggy-1.6.0
collected 5 items

tests/test_product.py::ProductTest::test_create_product PASSED           [ 20%]
tests/test_product.py::ProductTest::test_load_product PASSED             [ 40%]
tests/test_product.py::ProductTest::test_update_product PASSED           [ 60%]
tests/test_product.py::ProductTest::test_delete_product PASSED           [ 80%]
tests/test_product.py::ProductTest::test_select_with_filter PASSED       [100%]

============================== 5 passed in 0.16s ===============================
```

`--verbose` prints one line per test instead of the compact dot-progress.

Failed Test Output

When a test fails, pytest shows exactly what went wrong:

```
=================================== FAILURES ===================================
_____________________ ProductTest.test_load_product ____________________________

self = <test_product.ProductTest object at 0x10a2c1bb0>

    def test_load_product(self):
        product = Product.find(1)
>       assert_equal(product.name, "Load Test Widget", "Name should match")
E       AssertionError: Name should match
E       Expected: 'Load Test Widget'
E       Actual:   'Wrong Name'

tests/test_product.py:34: AssertionError
=========================== short test summary info ============================
FAILED tests/test_product.py::ProductTest::test_load_product - AssertionError: Name should match
```

```
========================= 1 failed, 2 passed in 0.12s ==========================
```

The failure message shows the failing assertion, expected vs actual, and the file and line number. This is standard pytest formatting.

---

9. pytest Integration

If your team already uses pytest, Tina4 tests can run alongside it. Install pytest:

```
uv add --dev pytest
```

Create `pyproject.toml` test configuration:

```
[tool.pytest.ini_options]
testpaths = ["tests"]
python_files = ["test_*.py"]
python_classes = ["*Test"]
python_functions = ["test_*"]
```

Tina4's `Test` class is compatible with pytest. Your test files work with both `tina4 test` and `pytest` without modification.

```
tina4 test
```

```
========================= test session starts ==========================
collected 5 items

tests/test_product.py .....                                        [100%]

========================= 5 passed in 0.34s ============================
```

Code Coverage

With pytest, you can generate code coverage reports:

```
uv add --dev pytest-cov
tina4 test --cov=src --cov-report=term
```

```
---------- coverage: ... ----------
Name                      Stmts   Miss  Cover
-------------------------------------------
src/routes/products.py       45      5    89%
src/orm/product.py           28      0   100%
src/middleware/auth.py        32      8    75%
-------------------------------------------
TOTAL                        105     13    88%
```

For HTML coverage reports:

```
tina4 test --cov=src --cov-report=html
```

Open `htmlcov/index.html` in your browser to see a visual breakdown of which lines are covered by tests.

---

10. Testing Best Practices

Test One Thing Per Test

Each test method should verify one behavior. If it fails, you know exactly what broke.

```
# Good: each test verifies one thing
def test_create_product_returns_201(self):
    resp = self.post("/api/products", json={"name": "Widget", "price": 9.99})
    assert_equal(resp.status, 201, "Should return 201")

def test_create_product_returns_created_product(self):
    resp = self.post("/api/products", json={"name": "Widget", "price": 9.99})
    body = json.loads(resp.body)
    assert_equal(body["name"], "Widget", "Should return the product name")

# Avoid: testing multiple unrelated things
def test_everything(self):
    # Creates, reads, updates, deletes, checks auth, validates input...
    # When this fails, you don't know which part broke
    pass
```

Use Descriptive Assertion Messages

```
# Good: tells you what went wrong
assert_equal(user.name, "Alice", "User name should be 'Alice' after creation")

# Bad: unhelpful when it fails
assert_equal(user.name, "Alice", "Failed")
```

Isolate Tests

Each test should create its own data and clean up after itself. Never depend on data from another test or from the development database.

```
# Good: creates its own data
def test_delete_product(self):
    product = Product()
    product.name = "Temporary"
    product.price = 1.00
    product.save()

    product.delete()

    check = Product.find(product.id)
    assert_true(check is None, "Product should be deleted")

# Bad: depends on data from another test or the dev database
def test_delete_product(self):
    product = Product.find(1)  # Assumes product with ID 1 exists
    product.delete()
```

---

11. Exercise: Test a User Model and Authentication Flow

Write a complete test suite for a User model and authentication system.

Requirements

Create `tests/test_user_model.py` with these tests:

- **test*create*user** -- Create a user with name, email. Verify the user gets an ID and all fields are correct.
- **test*duplicate*email** -- Try to create two users with the same email. Verify the second one raises an exception or returns an error.
- **test*update*user** -- Create a user, update their name, reload and verify.
- **test*delete*user** -- Create a user, delete them, verify they cannot be loaded.
- **test*select*users** -- Create 3 users, query all, verify count is at least 3.

Create `tests/test_auth_flow.py` with these tests:

- **test*register*new_user** -- POST to `/api/auth/register` with name, email, password. Verify 201 status.
- **test*register*duplicate_email** -- Register the same email twice. Verify the second returns 409 Conflict.
- **test*login*success** -- Login with correct credentials. Verify you get a JWT token.
- **test*login*failure** -- Login with wrong password. Verify 401 status.
- **test*access*protected_route** -- Use the token from login to access `/api/profile`. Verify 200 status and correct user data.
- **test*access*with*expired*token** -- Use a manually crafted expired token. Verify 401 status.

Expected output:

```
tina4 test

============================= test session starts ==============================
platform darwin -- Python 3.13.5, pytest-9.0.2, pluggy-1.6.0
collected 11 items

tests/test_user_model.py .....                                           [ 45%]
tests/test_auth_flow.py ......                                           [100%]

============================== 11 passed in 0.52s ==============================
```

---

12. Solution

tests/test*user*model.py

```
import uuid
from tina4_python.test import Test, assert_equal, assert_true, assert_not_none, assert_raises

class UserModelTest(Test):

    def test_create_user(self):
        user = User()
        user.name = "Test User"
        user.email = f"testuser-{uuid.uuid4().hex[:8]}@example.com"
        user.save()

        assert_not_none(user.id, "User should have an ID after save")
        assert_equal(user.name, "Test User", "Name should match")
        assert_true("@example.com" in user.email, "Email should be set")

    def test_duplicate_email(self):
        email = f"duplicate-{uuid.uuid4().hex[:8]}@example.com"

        user1 = User()
        user1.name = "First User"
        user1.email = email
        user1.save()

        def create_duplicate():
            user2 = User()
            user2.name = "Second User"
            user2.email = email
            user2.save()

        assert_raises(create_duplicate, Exception, "Should reject duplicate email")

    def test_update_user(self):
        user = User()
        user.name = "Original Name"
        user.email = f"update-{uuid.uuid4().hex[:8]}@example.com"
        user.save()

        user_id = user.id
        user.name = "New Name"
        user.save()

        reloaded = User.find(user_id)
        assert_equal(reloaded.name, "New Name", "Name should be updated")

    def test_delete_user(self):
        user = User()
        user.name = "Delete Me"
        user.email = f"delete-{uuid.uuid4().hex[:8]}@example.com"
        user.save()

        user_id = user.id
        user.delete()

        gone = User.find(user_id)
        assert_true(gone is None, "Deleted user should not exist")
```

```
def test_select_users(self):
    for i in range(3):
        user = User()
        user.name = f"Select Test User {i}"
        user.email = f"select-test-{i}-{uuid.uuid4().hex[:8]}@example.com"
        user.save()

    users, count = User.where("1=1")

    assert_true(len(users) >= 3, "Should have at least 3 users")
```

tests/test*auth*flow.py

```
import uuid
import json
from tina4_python.test import Test, assert_equal, assert_true, assert_not_none

class AuthFlowTest(Test):

    def set_up(self):
        self.test_email = f"auth-test-{uuid.uuid4().hex[:8]}@example.com"
        self.test_password = "SecurePassword123!"

    def test_register_new_user(self):
        resp = self.post("/api/auth/register", json={
            "name": "Auth Test User",
            "email": self.test_email,
            "password": self.test_password
        })

        assert_equal(resp.status, 201, "Registration should return 201")

        body = json.loads(resp.body)
        assert_equal(body["name"], "Auth Test User", "Should return user name")
        assert_not_none(body.get("id"), "Should return user ID")

    def test_register_duplicate_email(self):
        email = f"dup-{uuid.uuid4().hex[:8]}@example.com"

        self.post("/api/auth/register", json={
            "name": "First",
            "email": email,
            "password": self.test_password
        })

        resp = self.post("/api/auth/register", json={
            "name": "Second",
            "email": email,
            "password": self.test_password
        })

        assert_equal(resp.status, 409, "Duplicate email should return 409")

    def test_login_success(self):
        email = f"login-{uuid.uuid4().hex[:8]}@example.com"

        self.post("/api/auth/register", json={
            "name": "Login User",
            "email": email,
            "password": self.test_password
        })

        resp = self.post("/api/auth/login", json={
            "email": email,
            "password": self.test_password
        })

        assert_equal(resp.status, 200, "Login should return 200")

        body = json.loads(resp.body)
```

```
        assert_not_none(body.get("token"), "Should return a token")

    def test_login_failure(self):
        resp = self.post("/api/auth/login", json={
            "email": "nobody@example.com",
            "password": "wrong"
        })

        assert_equal(resp.status, 401, "Invalid login should return 401")

    def test_access_protected_route(self):
        email = f"profile-{uuid.uuid4().hex[:8]}@example.com"

        self.post("/api/auth/register", json={
            "name": "Profile User",
            "email": email,
            "password": self.test_password
        })

        login_resp = self.post("/api/auth/login", json={
            "email": email,
            "password": self.test_password
        })

        token = json.loads(login_resp.body)["token"]

        resp = self.get("/api/profile", headers={
            "Authorization": f"Bearer {token}"
        })

        assert_equal(resp.status, 200, "Should allow access with valid token")

        body = json.loads(resp.body)
        assert_equal(body["user"]["email"], email, "Should return correct user")

    def test_access_with_expired_token(self):
        resp = self.get("/api/profile", headers={
            "Authorization": "Bearer expired.invalid.token"
        })

        assert_equal(resp.status, 401, "Should reject invalid token")
```

---

13. Gotchas

1. Tests Run in Order

Problem: Test B depends on data created in Test A, but sometimes Test B fails.

Cause: While tests within a class run in the order they are defined, test classes may run in any order. If Test B depends on Test A being in a different class, this is fragile.

Fix: Each test should create its own data. Never depend on another test's side effects. Use `set_up()` to create the data each test needs.

2. Test Database vs Development Database

Problem: Running tests deletes your development data.

Cause: Tests are running against the same database as your development server.

Fix: Tina4 uses a separate test database by default (`data/test.db`). If your tests are modifying development data, check that `TINA4_DATABASE_URL` is set correctly in `.env`, or that you are running `tina4 test` (not executing test files manually with `python`).

3. Unique Constraint Failures

Problem: Tests fail with "UNIQUE constraint failed" on the email field.

Cause: Previous test runs left data in the test database, or two tests create records with the same email.

Fix: Use `uuid.uuid4().hex[:8]` or `int(time.time())` to generate unique values in tests:

```
user.email = f"test-{uuid.uuid4().hex[:8]}@example.com"
```

4. Test Method Not Discovered

Problem: You wrote a test method but it does not appear in the output.

Cause: The method name does not start with `test`. Tina4 only runs methods that begin with `test` (case-sensitive).

Fix: Rename the method to start with `test`: `test_create_product`, `test_login_failure`, etc.

5. Assertion Arguments Reversed

Problem: The failure message shows "Expected: 404, Actual: 200" but that seems backward.

Cause: You passed the arguments in the wrong order. `assert_equal(actual, expected)` -- actual comes first, expected comes second.

Fix: Follow the convention: `assert_equal(resp.status, 200, "message")`. The first argument is what you got, the second is what you expected.

6. Cannot Test Routes That Require Authentication Setup

Problem: Tests for protected routes fail because the authentication middleware expects a database table or JWT secret that does not exist in the test environment.

Cause: The test database is empty -- it does not have the users table or the JWT secret is not configured.

Fix: Use `set_up()` to create the necessary data:

```
def set_up(self):
    user = User()
    user.create_table()

    user.name = "Test Admin"
    user.email = "admin@test.com"
    user.password_hash = hash_password("password")
```

```
    user.save()
```

7. Tests Pass Locally but Fail in CI

Problem: All tests pass on your machine but fail in continuous integration.

Cause: The CI environment may have a different Python version, missing packages, or different filesystem permissions. Also, tests that depend on time or random values can be flaky.

Fix: Make tests deterministic. Avoid relying on the current time, filesystem state, or external services. If you must test time-dependent behavior, mock the clock or use fixed timestamps. Check that your CI environment matches your local Python version and has all required packages.

Scaffolding

One command. Six files. A working CRUD feature with routes, templates, tests, and Swagger docs -- ready to run.

That is Tina4's scaffolding system. It generates the boilerplate you write by hand in every project: models, migrations, routes, forms, views, and tests. You describe what you want. The generators produce it.

The CRUD Generator

This is the generator most developers reach for first. It creates everything a feature needs in one shot.

```
tina4python generate crud Product --fields "name:string,price:float"
```

That single command creates six files:

#	File	Purpose
1	`src/orm/Product.py`	ORM model with typed fields
2	`migrations/20260401_create_product.sql`	UP migration (CREATE TABLE)
3	`migrations/20260401_create_product.down.sql`	DOWN migration (DROP TABLE)
4	`src/routes/products.py`	CRUD routes with Swagger annotations
5	`src/templates/products/form.html`	Form template with typed inputs and `form_token`
6	`src/templates/products/view.html`	List and detail templates
7	`tests/test_products.py`	pytest stubs for all CRUD operations

What Each File Contains

The **model** maps the `product` table to a Python class:

```
from tina4_python import ORM

class Product(ORM):
    table_name = "product"
    fields = {
        "id": "integer",
```

```
        "name": "string",
        "price": "float",
        "created_at": "datetime",
        "updated_at": "datetime"
    }
```

The **migration** creates the table:

```
-- migrations/20260401_create_product.sql
CREATE TABLE product (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    name VARCHAR(255),
    price FLOAT,
    created_at DATETIME DEFAULT CURRENT_TIMESTAMP,
    updated_at DATETIME DEFAULT CURRENT_TIMESTAMP
);
```

The **down migration** reverses it:

```
-- migrations/20260401_create_product.down.sql
DROP TABLE IF EXISTS product;
```

The **routes** file wires up five endpoints with Swagger docs:

```
from tina4_python import get, post, put, delete
from src.orm.Product import Product

@get("/api/products", description="List all products")
async def get_products(request, response):
    products = Product().select()
    return response(products)

@get("/api/products/{id}", description="Get a product by ID")
async def get_product(request, response):
    product = Product()
    product.id = request.params["id"]
    product.load()
    return response(product)

@post("/api/products", description="Create a product")
async def create_product(request, response):
    product = Product(request.body)
    product.save()
    return response(product, 201)

@put("/api/products/{id}", description="Update a product")
async def update_product(request, response):
    product = Product(request.body)
    product.id = request.params["id"]
    product.save()
    return response(product)

@delete("/api/products/{id}", description="Delete a product")
async def delete_product(request, response):
    product = Product()
    product.id = request.params["id"]
    product.delete()
    return response(None, 204)
```

The **form template** renders typed inputs with CSRF protection:

```
<form method="POST" action="/api/products">
    <input type="hidden" name="form_token" value="{{ form_token }}">
    <label>Name</label>
    <input type="text" name="name" required>
    <label>Price</label>
    <input type="number" step="0.01" name="price" required>
    <button type="submit">Save</button>
</form>
```

The **test file** stubs out CRUD assertions:

```
import pytest
from tina4_python import App

@pytest.fixture
def client():
    app = App()
    return app.test_client()

def test_create_product(client):
    response = client.post("/api/products", json={"name": "Widget", "price": 9.99})
    assert response.status_code == 201

def test_list_products(client):
    response = client.get("/api/products")
    assert response.status_code == 200

def test_get_product(client):
    response = client.get("/api/products/1")
    assert response.status_code == 200

def test_update_product(client):
    response = client.put("/api/products/1", json={"name": "Updated Widget"})
    assert response.status_code == 200

def test_delete_product(client):
    response = client.delete("/api/products/1")
    assert response.status_code == 204
```

Run It

After generating, run the migration and start the server:

```
tina4python migrate
tina4python serve
```

Open Swagger UI at `http://localhost:7146/swagger` and test every endpoint. The scaffolded code works out of the box.

Individual Generators

The CRUD generator calls several smaller generators under the hood. You can call each one directly when you need a single piece.

Model

```
tina4python generate model Product --fields "name:string,price:float"
```

Creates three files: the ORM model (`src/orm/Product.py`), the UP migration, and the DOWN migration. No routes, no templates, no tests.

Route

```
tina4python generate route products --model Product
```

Creates one file: `src/routes/products.py` with CRUD endpoints and Swagger annotations. The model must exist first.

Migration

```
tina4python generate migration add_category_to_product
```

Creates two files: `migrations/20260401_add_category_to_product.sql` and `migrations/20260401_add_category_to_product.down.sql`. Both are empty stubs. You write the SQL.

Middleware

```
tina4python generate middleware AuthLog
```

Creates one file with before and after stubs:

```
from tina4_python import middleware

@middleware(before=True)
async def auth_log_before(request):
    print(f"Request: {request.method} {request.url}")
    return request

@middleware(after=True)
async def auth_log_after(request, response):
    print(f"Response: {response.status_code}")
    return response
```

Test

```
tina4python generate test products --model Product
```

Creates one file: `tests/test_products.py` with pytest CRUD stubs.

Form

```
tina4python generate form Product --fields "name:string,price:float"
```

Creates one file: `src/templates/products/form.html` with typed inputs and `form_token`.

View

```
tina4python generate view Product --fields "name:string,price:float"
```

Creates two templates: a list view and a detail view in `src/templates/products/`.

CRUD

```
tina4python generate crud Product --fields "name:string,price:float"
```

Shorthand for running all generators at once: model, migration, route, form, view, and test.

Auth

```
tina4python generate auth
```

Generates the full authentication scaffold: User model, migrations, login/register/logout routes, templates, and tests.

AutoCRUD

AutoCRUD automatically generates REST API endpoints from your ORM models:

- `GET /api/{table}` - List with pagination (`?limit=10&offset=0`)
- `GET /api/{table}/{id}` - Get single record
- `POST /api/{table}` - Create record
- `PUT /api/{table}/{id}` - Update record
- `DELETE /api/{table}/{id}` - Delete record

Usage

```
from tina4_python.crud import AutoCrud
AutoCrud.register(User)
AutoCrud.discover("src/orm", prefix="/api")
```

`AutoCrud.register(Model)` wires up a single model. `AutoCrud.discover(path)` scans a directory and registers every ORM model it finds. Both approaches mount the five standard endpoints automatically, with no route files needed.

The Auth Generator

Authentication needs more than one file. The auth generator creates seven:

```
tina4python generate auth
```

#	File	Purpose
1	`src/orm/User.py`	User model with hashed password field
2	`migrations/20260401_create_user.sql`	UP migration
3	`migrations/20260401_create_user.down.sql`	DOWN migration
4	`src/routes/auth.py`	Login, register, logout routes
5	`src/templates/auth/login.html`	Login form

6	`src/templates/auth/register.html`	Registration form
7	`tests/test_auth.py`	Auth flow tests

The generated routes handle password hashing, JWT token creation, and session management. The templates include CSRF tokens. The tests cover registration, login, invalid credentials, and logout.

Run the migration, start the server, and you have working auth:

```
tina4python migrate
tina4python serve
```

Field Types

Generators accept these field types. Each type maps to a specific column type in migrations, input type in forms, and display format in views.

Field Type	Migration Column	Form Input	View Display
`string`	`VARCHAR(255)`	`<input type="text">`	Plain text
`int`	`INTEGER`	`<input type="number">`	Number
`float`	`FLOAT`	`<input type="number" step="0.01">`	Decimal
`bool`	`BOOLEAN`	`<input type="checkbox">`	Yes / No
`text`	`TEXT`	`<textarea>`	Paragraph
`datetime`	`DATETIME`	`<input type="datetime-local">`	Formatted date
`blob`	`BLOB`	`<input type="file">`	Download link

Table Naming Convention

Tina4 uses singular table names. The model name `Product` maps to the table `product`. The model name `OrderItem` maps to `order_item`. The generator handles the conversion.

Combining Generators

Sometimes you want a model with routes but no form. Run the individual generators one after another. Each adds one piece:

```
tina4python generate model Product --fields "name:string,price:float"
tina4python generate route products --model Product
```

`generate model` writes the migration by default. Pass `--no-migration` to leave it out. Add `generate form`, `generate view` or `generate test` for the other pieces.

The `generate crud` command builds the full set in one step.

Exercise: Scaffold a Blog

Build a blog with three resources using generators.

Step 1: Generate the auth system.

```
tina4python generate auth
```

Step 2: Scaffold the Post resource.

```
tina4python generate crud Post --fields "title:string,body:text,published:bool"
```

Step 3: Scaffold the Category resource.

```
tina4python generate crud Category --fields "name:string,description:text"
```

Step 4: Add a migration to link posts to categories.

```
tina4python generate migration add_category_id_to_post
```

Edit the migration to add the foreign key:

```
ALTER TABLE post ADD COLUMN category_id INTEGER REFERENCES category(id);
```

Step 5: Run all migrations and start the server.

```
tina4python migrate
tina4python serve
```

You now have a working blog with authentication, posts, categories, and Swagger documentation. Total commands: five. Total hand-written SQL: one line.

Gotchas

Generators Do Not Overwrite

If a file exists, the generator skips it and prints a warning. This protects your edits. To regenerate, delete the file first.

Run Migrate After Generate

The model generator creates migration files. Those files do nothing until you run `tina4python migrate`. Generate and migrate are separate steps by design.

File Naming Matters

The generator derives file names from the model name. `Product` becomes `Product.py` for the model and `products.py` for routes. Do not rename generated files unless you update all imports.

Field Changes Need New Migrations

Changing `--fields` and re-running the generator does not update existing migrations. Create a new migration with `generate migration` and write the ALTER TABLE by hand.

Singular Table Names

Tina4 uses singular table names: `product`, not `products`. The route paths use plural (`/api/products`), but the table stays singular. The generator handles this split.

Swagger / OpenAPI

1. The 47-Endpoint Problem

Your team ships 47 API endpoints. The frontend developer asks what each one accepts. You email a spreadsheet. It rots. You write a wiki page. Nobody updates it. You add comments to the code. Nobody reads them.

Swagger kills this problem for good. It generates interactive API documentation from decorators on your route files. The docs stay current because they live inside the code itself. Your frontend developer browses every endpoint, sees expected request and response formats, and tests endpoints from the browser.

Tina4 Python auto-generates a Swagger UI at `/swagger` from your route decorators. No build step. No extra tooling. Write the decorators. The documentation appears.

---

2. What Swagger/OpenAPI Is

OpenAPI is a specification for describing REST APIs. Swagger is the toolset that reads OpenAPI specs and produces documentation, client SDKs, and server stubs.

An OpenAPI spec describes:

- Every endpoint (path + HTTP method)
- Parameters (path, query, header, body)
- Responses (status codes, body schemas)
- Data schemas (what a "User" or "Product" looks like)
- Authentication requirements
- Grouping and tagging

The spec follows a standard JSON structure. Tools across the industry consume it -- Postman, Insomnia, code generators, testing frameworks, API gateways. One spec feeds them all.

Tina4 builds this spec from Python decorators on your routes. No JSON or YAML by hand. The framework inspects your decorators at startup and constructs the full OpenAPI 3.0.3 document. You write Python. Tina4 writes the spec.

---

3. Accessing the Swagger UI

When `TINA4_DEBUG=true`, the Swagger UI appears at:

```
http://localhost:7146/swagger
```

Open it in your browser. You see all registered routes, organized by tags, with request and response details.

The underlying OpenAPI JSON spec lives at:

```
http://localhost:7146/swagger/json
```

This raw JSON feeds tools that import API definitions. Postman, Insomnia, and code generators all consume it.

```
curl http://localhost:7146/swagger/json
```

```
{
  "openapi": "3.0.3",
  "info": {
    "title": "My Store API",
    "version": "1.0.0"
  },
  "paths": {
    "/api/products": {
      "get": {
        "summary": "List all products",
        "responses": {
          "200": {
            "description": "Successful response"
          }
        }
      }
    }
  }
}
```

---

4. Documenting Routes with Decorators

@description -- Describe What an Endpoint Does

```
from tina4_python.core.router import get, post
from tina4_python.swagger import description

@get("/api/users")
@description("List all users", "Returns a paginated list of all registered users. Supports filte
async def list_users(request, response):
    return response.json({"users": [], "count": 0})
```

The first argument is a short summary (shown in the route list). The second is a detailed description (shown when you expand the route).

@tags -- Organize Endpoints into Groups

```
from tina4_python.swagger import tags

@get("/api/users")
@tags("Users")
@description("List all users")
async def list_users(request, response):
    return response.json({"users": []})

@post("/api/users")
@tags("Users")
@description("Create a new user")
async def create_user(request, response):
    return response.json({"user": request.body}, 201)

@get("/api/products")
@tags("Products")
@description("List all products")
async def list_products(request, response):
    return response.json({"products": []})
```

Tags group related endpoints in the Swagger UI. All "Users" endpoints appear under one collapsible section, all "Products" under another. Without tags, every endpoint sits in one flat list. With tags, the UI becomes navigable.

@example -- Document Request Body

```
from tina4_python.swagger import example

@post("/api/users")
@tags("Users")
@description("Create a new user", "Registers a new user account. Email must be unique.")
@example({
    "name": "Alice Smith",
    "email": "alice@example.com",
    "password": "securePass123",
    "role": "user"
})
async def create_user(request, response):
    return response.json({"user": request.body}, 201)
```

The `@example` decorator shows a sample request body in the Swagger UI. Developers click "Try it out" and the example pre-fills the input fields.

@example_response -- Document Response Body

```
from tina4_python.swagger import example_response

@get("/api/users/{id:int}")
@tags("Users")
@description("Get a user by ID")
@example_response(200, {
    "id": 1,
    "name": "Alice Smith",
    "email": "alice@example.com",
    "role": "user",
    "created_at": "2026-03-22T14:30:00"
})
@example_response(404, {
    "error": "User not found"
})
async def get_user(request, response):
    user_id = request.params["id"]
    return response.json({"id": user_id, "name": "Alice"})
```

Stack multiple `@example_response` decorators for different status codes. Developers see what to expect for both success and failure.

---

5. Documenting Path Parameters

The framework detects path parameters from the route pattern. Add descriptions with the `@description` decorator's extended syntax:

```
@get("/api/users/{id:int}/posts/{status}")
@tags("Posts")
@description(
    "Get user posts by status",
    "Returns all posts for a specific user filtered by status.",
    params={
        "id": "The user's unique identifier (integer)",
        "status": "Post status filter: 'draft', 'published', or 'archived'"
    }
)
async def user_posts(request, response):
    return response.json({"posts": []})
```

Documenting Query Parameters

```
@get("/api/products")
@tags("Products")
@description(
    "List products",
    "Returns a filtered and paginated list of products.",
    query={
        "category": {"type": "string", "description": "Filter by category name", "required": Fal
        "min_price": {"type": "number", "description": "Minimum price filter", "required": False
        "max_price": {"type": "number", "description": "Maximum price filter", "required": False
        "page": {"type": "integer", "description": "Page number (default: 1)", "required": False
        "limit": {"type": "integer", "description": "Items per page (default: 20)", "required":
    }
)
async def list_products(request, response):
    return response.json({"products": [], "count": 0})
```

---

6. Authentication in Swagger

Routes marked with `@secured` show a lock icon in the Swagger UI. Developers click "Authorize", paste their JWT token, and all subsequent requests include the header.

```
from tina4_python.core.router import get, secured
from tina4_python.swagger import description, tags

@get("/api/profile")
@secured()
@tags("Auth")
@description("Get current user profile", "Requires a valid JWT token in the Authorization header
async def get_profile(request, response):
    return response.json({"user": request.user})
```

Public routes marked with `@noauth` show as unlocked:

```
from tina4_python.core.router import post, noauth

@post("/api/login")
@noauth()
@tags("Auth")
@description("Login", "Authenticate with email and password. Returns a JWT token.")
@example({"email": "alice@example.com", "password": "securePass123"})
@example_response(200, {
    "message": "Login successful",
    "token": "eyJhbGciOiJIUzI1NiIs...",
    "user": {"id": 1, "name": "Alice", "email": "alice@example.com"}
})
@example_response(401, {"error": "Invalid email or password"})
async def login(request, response):
    return response.json({"token": "..."})
```

---

7. Try-It-Out from the Swagger UI

Every endpoint in the Swagger UI has a "Try it out" button. Click it and the interface transforms.

- Input fields expand for every parameter
- Example values pre-fill (when provided via `@example`)
- Edit parameters, headers, and the request body
- Click "Execute" -- the actual HTTP request fires against your running server
- The response appears: status code, headers, body

This turns your documentation into a live testing tool. No Postman needed. No curl commands to remember.

Authentication in Try-It-Out

Endpoints that require auth display a lock icon. Click the "Authorize" button at the top of the Swagger UI. Paste your JWT token or API key. The UI stores it and includes the `Authorization` header on every subsequent request.

The workflow for testing a secured endpoint:

- Call your `/api/login` endpoint through Swagger to get a token
- Click "Authorize" and paste the token
- Test any protected endpoint -- the token travels with each request
- Click "Authorize" again and "Logout" to clear it

Tina4 auto-detects auth requirements from `@secured` and `@noauth` decorators. Secured routes show the lock. Public routes show an open lock. Routes without either decorator inherit the default security scheme.

---

8. Customizing the Swagger Info Block

The Swagger UI header and OpenAPI spec carry metadata about your API. Configure this metadata through environment variables in `.env`:

```
TINA4_SWAGGER_TITLE=My Store API
TINA4_SWAGGER_DESCRIPTION=REST API for managing products, orders, and users
TINA4_SWAGGER_VERSION=1.0.0
SWAGGER_DEV_URL=http://localhost:7146
```

These values appear in the OpenAPI spec under the `info` block:

```
{
  "openapi": "3.0.3",
  "info": {
    "title": "My Store API",
    "description": "REST API for managing products, orders, and users",
    "version": "1.0.0"
  },
  "servers": [
    {
      "url": "http://localhost:7146"
    }
  ]
```

```
}
```

Variable	Purpose	Default
`TINA4_SWAGGER_TITLE`	API name shown in the UI header	`Tina4 API`
`TINA4_SWAGGER_DESCRIPTION`	Brief description below the title	(empty)
`TINA4_SWAGGER_VERSION`	API version number	`1.0.0`
`SWAGGER_DEV_URL`	Server URL for the spec	`http://localhost:7146`

When you version your API, update `TINA4_SWAGGER_VERSION` so consumers know which version they target. The title and description give context -- a developer who opens your Swagger page should know what the API does before scrolling.

---

9. Generating Client SDKs from the Spec

The OpenAPI spec at `/swagger/json` feeds code generation tools. One spec produces client libraries in any language.

Using OpenAPI Generator

```
npm install -g @openapitools/openapi-generator-cli

# TypeScript client
openapi-generator-cli generate \
  -i http://localhost:7146/swagger/json \
  -g typescript-fetch \
  -o ./frontend/api-client

# Python client
openapi-generator-cli generate \
  -i http://localhost:7146/swagger/json \
  -g python \
  -o ./python-client
```

The generated code carries types. IDE autocompletion works:

```
const api = new ProductsApi();

const product = await api.getProductById({ id: 42 });
console.log(product.name);  // TypeScript knows this is a string

const newProduct = await api.createProduct({
    name: "Widget",
    category: "General",
    price: 9.99,
    inStock: true
});
```

Update your decorators. Regenerate. The client stays in sync with the server.

Other Generators

The ecosystem supports dozens of languages and frameworks:

Generator	Output
`typescript-fetch`	Browser-ready TypeScript client
`typescript-axios`	Axios-based TypeScript client
`python`	Python client with type hints
`swift5`	iOS/macOS client
`kotlin`	Android/JVM client
`csharp-netcore`	.NET client
`go`	Go client

Every generator reads the same spec. Your API documentation becomes the single source of truth for every consumer.

---

10. Complete API Documentation Example

Here is a fully documented User API with all decorator features:

```
from tina4_python.core.router import get, post, put, delete, noauth, secured, middleware
from tina4_python.swagger import description, tags, example, example_response
from tina4_python.auth import Auth
from tina4_python.database.connection import Database


async def auth_middleware(request, response, next_handler):
    auth_header = request.headers.get("Authorization", "")
    if not auth_header or not auth_header.startswith("Bearer "):
        return response.json({"error": "Authorization required"}, 401)
    token = auth_header[7:]
    if not Auth.valid_token(token):
        return response.json({"error": "Invalid or expired token"}, 401)
    request.user = Auth.get_payload(token)
    return await next_handler(request, response)


@post("/api/users")
@noauth()
@tags("Users")
@description(
    "Register a new user",
    "Creates a new user account. Email must be unique. Password must be at least 8 characters."
)
@example({
    "name": "Alice Smith",
    "email": "alice@example.com",
    "password": "securePass123"
```

```
})
@example_response(201, {
    "message": "Registration successful",
    "user": {"id": 1, "name": "Alice Smith", "email": "alice@example.com", "role": "user"}
})
@example_response(400, {"errors": ["Password must be at least 8 characters"]})
@example_response(409, {"error": "Email already registered"})
async def register_user(request, response):
    # Registration logic...
    return response.json({"message": "Registration successful"}, 201)


@get("/api/users")
@middleware(auth_middleware)
@tags("Users")
@description(
    "List all users",
    "Returns a paginated list of users. Admin only.",
    query={
        "role": {"type": "string", "description": "Filter by role", "required": False},
        "page": {"type": "integer", "description": "Page number", "required": False},
        "limit": {"type": "integer", "description": "Results per page", "required": False}
    }
)
@example_response(200, {
    "users": [
        {"id": 1, "name": "Alice", "email": "alice@example.com", "role": "admin"},
        {"id": 2, "name": "Bob", "email": "bob@example.com", "role": "user"}
    ],
    "count": 2,
    "page": 1,
    "total_pages": 1
})
async def list_users(request, response):
    return response.json({"users": []})


@get("/api/users/{id:int}")
@middleware(auth_middleware)
@tags("Users")
@description(
    "Get a user by ID",
    "Returns the full profile of a single user.",
    params={"id": "The user's unique identifier"}
)
@example_response(200, {
    "id": 1, "name": "Alice Smith", "email": "alice@example.com",
    "role": "user", "created_at": "2026-03-22T14:30:00"
})
@example_response(404, {"error": "User not found"})
async def get_user(request, response):
    return response.json({"id": request.params["id"]})


@put("/api/users/{id:int}")
@middleware(auth_middleware)
@tags("Users")
@description("Update a user", "Update user profile. Users can only update themselves. Admins can
@example({"name": "Alice Johnson", "email": "alice.j@example.com"})
```

```
@example_response(200, {"message": "User updated", "user": {"id": 1, "name": "Alice Johnson"}})
@example_response(404, {"error": "User not found"})
async def update_user(request, response):
    return response.json({"message": "User updated"})


@delete("/api/users/{id:int}")
@middleware(auth_middleware)
@tags("Users")
@description("Delete a user", "Permanently removes a user account. Admin only.")
@example_response(204, None)
@example_response(404, {"error": "User not found"})
async def delete_user(request, response):
    return response.json(None, 204)
```

---

11. Swagger Configuration

Control Swagger behavior in `.env`:

```
# Swagger is only available when debug mode is on (default behavior)
TINA4_DEBUG=true

# Custom API title and version shown in Swagger UI
TINA4_SWAGGER_TITLE=My Store API
TINA4_SWAGGER_VERSION=1.0.0
TINA4_SWAGGER_DESCRIPTION=REST API for the My Store e-commerce platform
```

---

12. Exercise: Document a User API

Take the authentication routes from Chapter 8 (register, login, profile, update profile, change password) and add full Swagger documentation.

Requirements

- Add `@tags("Auth")` to all auth-related endpoints
- Add `@description()` with both summary and detailed description
- Add `@example()` for all POST and PUT endpoints
- Add `@example_response()` for success and error cases on every endpoint
- Document query parameters on any endpoint that accepts them
- Visit `/swagger` and verify all routes appear with examples

Expected Result

Open `http://localhost:7146/swagger`. You should see:

- An "Auth" section with 5 endpoints
- Each endpoint carries a summary and description
- POST/PUT endpoints show example request bodies

- Every endpoint shows example responses for success and error cases
- The "Authorize" button works for testing protected endpoints

---

13. Solution

Update `src/routes/auth.py` with Swagger decorators (showing the decorator additions -- the function bodies remain the same as Chapter 8):

```
from tina4_python.core.router import get, post, put, noauth, middleware
from tina4_python.swagger import description, tags, example, example_response
from tina4_python.auth import Auth
from tina4_python.database.connection import Database


async def auth_middleware(request, response, next_handler):
    auth_header = request.headers.get("Authorization", "")
    if not auth_header or not auth_header.startswith("Bearer "):
        return response.json({"error": "Authorization required"}, 401)
    token = auth_header[7:]
    if not Auth.valid_token(token):
        return response.json({"error": "Invalid or expired token"}, 401)
    request.user = Auth.get_payload(token)
    return await next_handler(request, response)


@post("/api/register")
@noauth()
@tags("Auth")
@description(
    "Register a new account",
    "Creates a new user account. All fields are required. Password must be at least 8 characters
)
@example({
    "name": "Alice Smith",
    "email": "alice@example.com",
    "password": "securePass123"
})
@example_response(201, {
    "message": "Registration successful",
    "user": {"id": 1, "name": "Alice Smith", "email": "alice@example.com", "role": "user", "crea
})
@example_response(400, {"errors": ["Password must be at least 8 characters"]})
@example_response(409, {"error": "Email already registered"})
async def register(request, response):
    # ... same as Chapter 8 ...
    pass


@post("/api/login")
@noauth()
@tags("Auth")
@description(
    "Login",
    "Authenticate with email and password. Returns a JWT token valid for 1 hour. Include the tok
)
@example({"email": "alice@example.com", "password": "securePass123"})
```

```
@example_response(200, {
    "message": "Login successful",
    "token": "eyJhbGciOiJIUzI1NiIs...",
    "user": {"id": 1, "name": "Alice Smith", "email": "alice@example.com", "role": "user"}
})
@example_response(400, {"error": "Email and password are required"})
@example_response(401, {"error": "Invalid email or password"})
async def login(request, response):
    # ... same as Chapter 8 ...
    pass


@get("/api/profile")
@middleware(auth_middleware)
@tags("Auth")
@description(
    "Get current user profile",
    "Returns the profile of the currently authenticated user. Requires a valid JWT token."
)
@example_response(200, {
    "id": 1, "name": "Alice Smith", "email": "alice@example.com",
    "role": "user", "created_at": "2026-03-22 16:00:00"
})
@example_response(401, {"error": "Authorization required"})
async def get_profile(request, response):
    # ... same as Chapter 8 ...
    pass


@put("/api/profile")
@middleware(auth_middleware)
@tags("Auth")
@description(
    "Update profile",
    "Update the current user's name and/or email. Only the fields you include will be updated."
)
@example({"name": "Alice Johnson", "email": "alice.j@example.com"})
@example_response(200, {
    "message": "Profile updated",
    "user": {"id": 1, "name": "Alice Johnson", "email": "alice.j@example.com", "role": "user"}
})
@example_response(409, {"error": "Email already in use by another account"})
async def update_profile(request, response):
    # ... same as Chapter 8 ...
    pass


@put("/api/profile/password")
@middleware(auth_middleware)
@tags("Auth")
@description(
    "Change password",
    "Change the current user's password. Requires the current password for verification. New pas
)
@example({"current_password": "securePass123", "new_password": "evenMoreSecure456"})
@example_response(200, {"message": "Password changed successfully"})
@example_response(400, {"error": "New password must be at least 8 characters"})
@example_response(401, {"error": "Current password is incorrect"})
async def change_password(request, response):
```

```
    # ... same as Chapter 8 ...
    pass
```

---

14. Gotchas

1. Swagger not showing up

Problem: Visiting `/swagger` returns a 404.

Cause: `TINA4_DEBUG` is not set to `true` in your `.env`. The Swagger UI only appears in debug mode.

Fix: Set `TINA4_DEBUG=true` in `.env` and restart the server.

2. Route appears but has no documentation

Problem: A route shows up in Swagger but carries no description, examples, or parameter documentation.

Cause: The route has no Swagger decorators (`@description`, `@example`, etc.).

Fix: Add at least `@description()` and `@tags()` to every route you want documented. Without these, Swagger shows the route with minimal information.

3. Decorator order causes issues

Problem: Adding `@description` or `@tags` breaks the route registration.

Cause: Swagger decorators must sit below the route decorator. Python applies decorators bottom to top, so the route decorator executes first.

Fix: Follow this order (bottom to top): route decorator first, then middleware, then Swagger decorators:

```
@tags("Users")               # Applied last
@description("List users")   # Applied third
@middleware(auth_middleware)  # Applied second
@get("/api/users")           # Applied first
async def list_users(...):
    ...
```

4. Example does not match actual response

Problem: The example response in Swagger differs from what the endpoint returns.

Cause: You updated the route handler but forgot to update the `@example_response` decorator.

Fix: Keep examples in sync with your actual response format. Decorators are not enforced by the runtime. Consider writing tests that validate your responses match the documented examples.

5. Sensitive data in examples

Problem: Your `@example` decorator contains a real password or API key.

Cause: You copy-pasted from a test and forgot to replace real values.

Fix: Use placeholder values in examples: `"password": "securePass123"`, `"token": "eyJhbGciOiJIUzI1NiIs..."`. Never include real credentials.

6. Swagger in production

Problem: Your production API exposes the Swagger UI, revealing all endpoints and their parameters.

Cause: `TINA4_DEBUG=true` in production.

Fix: Set `TINA4_DEBUG=false` in production. The Swagger UI disappears. If you need API docs in production, export the OpenAPI JSON and host it separately with access controls.

7. Too many tags

Problem: Your Swagger UI has 20 tags. Navigation becomes painful.

Cause: You created a separate tag for every resource, sub-resource, and action.

Fix: Use broad tags that group related functionality: "Users", "Products", "Orders", "Auth". Most APIs need 5-10 tags. Avoid tags like "User Profile", "User Settings", "User Notifications" -- use "Users" for all of them.

8. SDK generation produces incorrect types

Problem: The generated TypeScript client has `any` types everywhere.

Cause: Your `@example_response` data uses generic strings instead of typed values.

Fix: Use correct types in examples: strings for text, numbers for numeric values, booleans for flags, arrays for lists. The generator infers types from example values. `"price": 9.99` produces `number`. `"price": "9.99"` produces `string`.

API Client

1. Calling External APIs

Every real application talks to something external. A payment processor. A weather service. A shipping carrier. A CRM. You write the HTTP call, parse the response, handle timeouts, retry on failure, add auth headers.

Tina4's `Api` class makes outbound HTTP requests from your server-side code. It returns a consistent response format, handles auth in one line, and supports SSL and timeout configuration, with no extra libraries.

---

2. Basic Requests

```
from tina4_python.api import Api

api = Api()

# GET
result = api.get("https://api.example.com/products")

# POST with JSON body
result = api.post("https://api.example.com/orders", {
    "product_id": 42,
    "quantity": 3
})

# PUT
result = api.put("https://api.example.com/orders/101", {
    "status": "shipped"
})

# PATCH
result = api.patch("https://api.example.com/orders/101", {
    "tracking_number": "1Z999AA10123456784"
})

# DELETE
result = api.delete("https://api.example.com/orders/101")
```

Every method returns the same structure:

```
{
    "http_code": 200,
    "body": { ... },     # Parsed JSON, or raw string if not JSON
    "headers": { ... },  # Response headers
    "error": None        # None on success, error string on failure
}
```

---

3. Reading the Response

```
from tina4_python.api import Api

api = Api()
result = api.get("https://jsonplaceholder.typicode.com/posts/1")

if result["error"]:
    print(f"Request failed: {result['error']}")
else:
    print(f"Status: {result['http_code']}")
    print(f"Title: {result['body']['title']}")
```

Check `result["error"]` first. If it is `None`, the request succeeded. The HTTP status code is in `result["http_code"]`. The parsed response is in `result["body"]`.

```
# In a route handler
from tina4_python.core.router import get
from tina4_python.api import Api

@get("/api/posts/{post_id}")
async def proxy_post(request, response):
    api = Api()
    post_id = request.params["post_id"]

    result = api.get(f"https://jsonplaceholder.typicode.com/posts/{post_id}")

    if result["error"]:
        return response({"error": result["error"]}, 502)

    if result["http_code"] == 404:
        return response({"error": "Post not found"}, 404)

    return response(result["body"])
```

---

4. Authentication Headers

Bearer Token

```
from tina4_python.api import Api

api = Api(bearer_token="eyJhbGciOiJSUzI1NiJ9...")

result = api.get("https://api.example.com/me")
```

The `Authorization: Bearer <token>` header is sent with every request made by this `Api` instance.

Basic Authentication

```
api = Api(username="api_user", password="secret123")

result = api.get("https://api.example.com/data")
```

Sends `Authorization: Basic <base64(username:password)>` with every request.

Custom Headers

```
api = Api(headers={
    "X-API-Key": "my-api-key-here",
    "X-Client-Version": "1.0.0",
    "Accept": "application/json"
})

result = api.get("https://api.example.com/data")
```

Custom headers are merged with any authentication headers and sent with every request.

Mixing Auth and Custom Headers

```
api = Api(
    bearer_token="eyJhbGciOiJSUzI1NiJ9...",
    headers={"X-Request-Source": "tina4-app"}
)
```

---

5. SSL Verification and Timeouts

```
# Disable SSL verification (dev only, never in production)
api = Api(verify_ssl=False)

# Set a 10-second timeout
api = Api(timeout=10)

# Both
api = Api(verify_ssl=False, timeout=5)
```

The default timeout is 30 seconds. The default SSL behaviour is to verify certificates (`verify_ssl=True`).

Never disable SSL verification in production. If an external API has a self-signed certificate, obtain the CA bundle and pass it explicitly, or ask the provider for their CA bundle.

---

6. Sending Query Parameters

Pass query parameters as a dictionary to the `params` argument:

```
api = Api()

result = api.get("https://api.example.com/products", params={
    "category": "Electronics",
    "page": 1,
    "limit": 20,
    "in_stock": True
})
# Requests: GET /products?category=Electronics&page=1&limit=20&in_stock=True
```

---

7. Real-World Patterns

Payment Gateway

```
import os
from tina4_python.api import Api

class PaymentGateway:
    def __init__(self):
        self.api = Api(
            bearer_token=os.environ["PAYMENT_API_KEY"],
            timeout=15
        )
        self.base = "https://api.payment-provider.com/v1"

    def charge(self, amount_cents, currency, card_token):
        result = self.api.post(f"{self.base}/charges", {
            "amount": amount_cents,
            "currency": currency,
            "source": card_token
        })

        if result["error"]:
            return {"success": False, "error": result["error"]}

        if result["http_code"] not in (200, 201):
            return {
                "success": False,
                "error": result["body"].get("message", "Payment declined")
            }

        return {
            "success": True,
            "charge_id": result["body"]["id"],
            "status": result["body"]["status"]
        }

    def refund(self, charge_id, amount_cents=None):
        body = {"charge": charge_id}
        if amount_cents:
            body["amount"] = amount_cents

        result = self.api.post(f"{self.base}/refunds", body)
        return result["body"]
```

Weather Service

```
import os
from tina4_python.api import Api

class WeatherService:
    def __init__(self):
        self.api = Api(timeout=5)
        self.api_key = os.environ["OPENWEATHER_API_KEY"]
        self.base = "https://api.openweathermap.org/data/2.5"

    def get_current(self, city):
        result = self.api.get(f"{self.base}/weather", params={
            "q": city,
            "appid": self.api_key,
            "units": "metric"
        })

        if result["error"] or result["http_code"] != 200:
            return None

        data = result["body"]
        return {
            "city": data["name"],
            "temp_c": data["main"]["temp"],
            "description": data["weather"][0]["description"],
            "humidity": data["main"]["humidity"]
        }
```

Usage in a route:

```
from tina4_python.core.router import get

weather = WeatherService()

@get("/api/weather/{city}")
async def get_weather(request, response):
    city = request.params["city"]
    data = weather.get_current(city)

    if data is None:
        return response({"error": "Weather data unavailable"}, 502)

    return response(data)
```

---

8. Retry Logic

For transient failures, add retry logic around `Api` calls:

```
import time
from tina4_python.api import Api
from tina4_python.debug import Log

def api_get_with_retry(url, max_retries=3, backoff=1.0, **kwargs):
    api = Api(**kwargs)
    last_error = None

    for attempt in range(1, max_retries + 1):
```

```
        result = api.get(url)

        if not result["error"] and result["http_code"] < 500:
            return result

        last_error = result["error"] or f"HTTP {result['http_code']}"
        Log.warning("API call failed, retrying", url=url, attempt=attempt, error=last_error)

        if attempt < max_retries:
            time.sleep(backoff * attempt)

    Log.error("API call failed after retries", url=url, error=last_error)
    return {"http_code": 0, "body": None, "headers": {}, "error": last_error}
```

---

9. Exercise: Weather Dashboard API

Build a route that fetches weather for multiple cities and returns a dashboard-ready response.

Requirements

- Create `GET /api/dashboard/weather` that:
- Accepts a `cities` query parameter (comma-separated)
- Calls a mock weather API for each city
- Returns aggregated results
- Returns 502 if any city fails
- Use proper error checking on every API call
- Add a 5-second timeout and Bearer token auth

Test with:

```
curl "http://localhost:7146/api/dashboard/weather?cities=London,Berlin,Tokyo"
```

---

10. Solution

Create `src/routes/weather_dashboard.py`:

```
from tina4_python.core.router import get
from tina4_python.api import Api
from tina4_python.debug import Log

# Simulate a weather API with mock data
MOCK_WEATHER = {
    "london": {"city": "London", "temp_c": 12.3, "description": "Cloudy", "humidity": 78},
    "berlin": {"city": "Berlin", "temp_c": 8.1, "description": "Clear", "humidity": 55},
    "tokyo": {"city": "Tokyo", "temp_c": 18.7, "description": "Partly cloudy", "humidity": 65},
    "paris": {"city": "Paris", "temp_c": 14.0, "description": "Rainy", "humidity": 82},
}

def fetch_weather(city_name):
    # In production this would be a real API call:
```

```
    # api = Api(bearer_token=os.environ["WEATHER_API_KEY"], timeout=5)
    # return api.get(f"https://api.weather.com/v1/current?city={city_name}")

    key = city_name.lower().strip()
    data = MOCK_WEATHER.get(key)
    if data is None:
        return {"http_code": 404, "body": None, "headers": {}, "error": f"City not found: {city_
    return {"http_code": 200, "body": data, "headers": {}, "error": None}


@get("/api/dashboard/weather")
async def weather_dashboard(request, response):
    cities_param = request.params.get("cities", "")

    if not cities_param:
        return response({"error": "Provide at least one city via ?cities=City1,City2"}, 400)

    cities = [c.strip() for c in cities_param.split(",") if c.strip()]
    results = []
    errors = []

    for city in cities:
        Log.debug("Fetching weather", city=city)
        result = fetch_weather(city)

        if result["error"]:
            Log.warning("Weather fetch failed", city=city, error=result["error"])
            errors.append({"city": city, "error": result["error"]})
        else:
            results.append(result["body"])

    if errors:
        return response({
            "error": "One or more cities could not be fetched",
            "failed": errors,
            "succeeded": results
        }, 502)

    return response({
        "cities": results,
        "count": len(results)
    })
```

```
curl "http://localhost:7146/api/dashboard/weather?cities=London,Berlin,Tokyo"
```

```
{
  "cities": [
    {"city": "London", "temp_c": 12.3, "description": "Cloudy", "humidity": 78},
    {"city": "Berlin", "temp_c": 8.1, "description": "Clear", "humidity": 55},
    {"city": "Tokyo", "temp_c": 18.7, "description": "Partly cloudy", "humidity": 65}
  ],
  "count": 3
}
```

---

11. Gotchas

1. Not checking result["error"]

Problem: Code accesses `result["body"]["id"]` but the request timed out, so `body` is `None`.

Fix: Always check `result["error"]` before accessing `result["body"]`. Treat any non-None error as a failure.

2. Using verify_ssl=False in production

Problem: A man-in-the-middle intercepts requests to a payment gateway because SSL verification is disabled.

Fix: Only use `verify_ssl=False` against local services or during development. Never disable SSL for external APIs.

3. No timeout set for slow external APIs

Problem: An external API hangs for 90 seconds. Your route handler blocks for 90 seconds. All workers are eventually held waiting.

Fix: Always set `timeout` to a sensible value (5-15 seconds for most external APIs). Return a 502 or 504 to the caller if the external API does not respond in time.

4. Hardcoding API keys

Problem: `api = Api(bearer_token="sk-live-abc123...")` exposes the key in source control.

Fix: Read credentials from environment variables: `bearer_token=os.environ["PAYMENT_API_KEY"]`. Never hardcode secrets.

GraphQL and SOAP

1. The Problem GraphQL Solves

Your mobile app needs a list of products. Each product carries a name, price, 20 image URLs, a full description, 15 review objects, and 8 fields you do not need. REST sends all of it -- 50KB of JSON when you needed 2KB. On a mobile connection, that waste stings.

Now your web dashboard needs the same products plus category, stock status, and supplier info. REST forces you to make three requests and stitch them together, or build a custom endpoint with query parameter parsing.

GraphQL ends both problems. The client asks for the fields it needs. The server returns those fields. One endpoint. One request. One response shaped to fit.

Tina4 includes a built-in GraphQL engine. No external packages. No Strawberry. No Ariadne. Part of the framework.

---

2. GraphQL vs REST -- A Quick Comparison

Aspect	REST	GraphQL
Endpoints	One per resource (`/products`, `/users`)	One endpoint (`/graphql`)
Data shape	Server decides	Client decides
Over-fetching	Common (get all fields)	Never (get only requested fields)
Under-fetching	Common (multiple requests needed)	Never (get related data in one query)
Versioning	`/api/v1/`, `/api/v2/`	Schema evolves, no versioning needed
Caching	HTTP caching works naturally	Requires client-side cache management
Learning curve	Low	Moderate

REST is still great for simple APIs. GraphQL shines when clients have diverse data needs -- mobile apps, dashboards, third-party integrations.

---

3. Enabling GraphQL

GraphQL is available by default in Tina4. The engine serves requests at `/graphql`. If you want to change the endpoint, set it in `.env`:

```
TINA4_GRAPHQL_ENDPOINT=/graphql
```

To verify it is working, start the server and send a test query:

```
curl -X POST http://localhost:7146/graphql \
  -H "Content-Type: application/json" \
  -d '{"query": "{ __schema { queryType { name } } }"}'
```

```
{
  "data": {
    "__schema": {
      "queryType": {
        "name": "Query"
      }
    }
  }
}
```

If you see that response, GraphQL is running.

---

4. Defining a Schema

A GraphQL schema defines the types of data your API can return and the queries and mutations clients can execute.

Create `src/graphql/schema.graphql`:

```
type Product {
    id: Int!
    name: String!
    category: String!
    price: Float!
    inStock: Boolean!
    createdAt: String
}

type Query {
    products: [Product!]!
    product(id: Int!): Product
}
```

This schema says:

- A `Product` has six fields. The `!` means the field is non-nullable.
- The `Query` type has two operations: `products` returns a list of products, and `product(id)` returns a single product (or null if not found).

Tina4 auto-discovers `.graphql` files in `src/graphql/`. You do not need to register them.

---

5. Writing Resolvers

A resolver is the function that runs when a query or mutation is executed. Resolvers live in `src/graphql/` as Python files.

Create `src/graphql/resolvers.py`:

```
from tina4_python.graphql import GraphQL

# Resolve the "products" query
@GraphQL.resolve("Query", "products")
async def resolve_products(root, args):
    products = Product.where("1=1")
    return [p.to_dict() for p in products]


# Resolve the "product" query
@GraphQL.resolve("Query", "product")
async def resolve_product(root, args):
    product = Product.find(args["id"])

    if product is None:
        return None

    return product.to_dict()
```

The `@GraphQL.resolve()` decorator takes two arguments:

- **Type name** -- The GraphQL type this resolver belongs to (`Query`, `Mutation`, or a custom type).
- **Field name** -- The field within the type this resolver handles.

The resolver function receives `root` (the parent value, if any) and `args` (the query arguments).

Testing the Queries

List all products:

```
curl -X POST http://localhost:7146/graphql \
  -H "Content-Type: application/json" \
  -d '{"query": "{ products { id name price inStock } }"}'
```

```
{
  "data": {
    "products": [
      {"id": 1, "name": "Wireless Keyboard", "price": 79.99, "inStock": true},
      {"id": 2, "name": "USB-C Hub", "price": 49.99, "inStock": true},
      {"id": 3, "name": "Standing Desk", "price": 549.99, "inStock": false}
    ]
  }
}
```

Notice: the response only contains the four fields we asked for (`id`, `name`, `price`, `inStock`), not `category` or `createdAt`. That is GraphQL in action.

Get a single product with all fields:

```
curl -X POST http://localhost:7146/graphql \
  -H "Content-Type: application/json" \
```

```
  -d '{"query": "{ product(id: 1) { id name category price inStock createdAt } }"}'
```

```
{
  "data": {
    "product": {
      "id": 1,
      "name": "Wireless Keyboard",
      "category": "Electronics",
      "price": 79.99,
      "inStock": true,
      "createdAt": "2026-03-22 14:30:00"
    }
  }
}
```

Request a product that does not exist:

```
curl -X POST http://localhost:7146/graphql \
  -H "Content-Type: application/json" \
  -d '{"query": "{ product(id: 999) { id name } }"}'
```

```
{
  "data": {
    "product": null
  }
}
```

---

6. Mutations

Mutations are how clients create, update, or delete data. They are defined in the schema alongside queries.

Update `src/graphql/schema.graphql`:

```
type Product {
    id: Int!
    name: String!
    category: String!
    price: Float!
    inStock: Boolean!
    createdAt: String
}

input ProductInput {
    name: String!
    category: String
    price: Float!
    inStock: Boolean
}

input ProductUpdateInput {
    name: String
    category: String
    price: Float
    inStock: Boolean
}

type DeleteResult {
```

```
    success: Boolean!
    message: String!
}

type Query {
    products: [Product!]!
    product(id: Int!): Product
    productsByCategory(category: String!): [Product!]!
}

type Mutation {
    createProduct(input: ProductInput!): Product!
    updateProduct(id: Int!, input: ProductUpdateInput!): Product
    deleteProduct(id: Int!): DeleteResult!
}
```

Key concepts:

- `input` **types** define the shape of data the client sends. They are like regular types but used for arguments.
- `Mutation` **type** lists all write operations. By convention, mutations are named with verbs: `createProduct`, `updateProduct`, `deleteProduct`.
- Fields in `ProductUpdateInput` are all optional (no `!`) because the client may only want to update one field.

Mutation Resolvers

Add to `src/graphql/resolvers.py`:

```
@GraphQL.resolve("Mutation", "createProduct")
async def resolve_create_product(root, args):
    input_data = args["input"]

    product = Product()
    product.name = input_data["name"]
    product.category = input_data.get("category", "Uncategorized")
    product.price = float(input_data["price"])
    product.in_stock = bool(input_data.get("inStock", True))
    product.save()

    return product.to_dict()


@GraphQL.resolve("Mutation", "updateProduct")
async def resolve_update_product(root, args):
    product = Product.find(args["id"])

    if product is None:
        return None

    input_data = args["input"]
    if "name" in input_data:
        product.name = input_data["name"]
    if "category" in input_data:
        product.category = input_data["category"]
    if "price" in input_data:
        product.price = float(input_data["price"])
```

```
    if "inStock" in input_data:
        product.in_stock = bool(input_data["inStock"])
    product.save()

    return product.to_dict()


@GraphQL.resolve("Mutation", "deleteProduct")
async def resolve_delete_product(root, args):
    product = Product.find(args["id"])

    if product is None:
        return {"success": False, "message": "Product not found"}

    product.delete()
    return {"success": True, "message": "Product deleted"}


@GraphQL.resolve("Query", "productsByCategory")
async def resolve_products_by_category(root, args):
    products = Product.where("category = ?", [args["category"]])
    return [p.to_dict() for p in products]
```

Testing Mutations

Create a product:

```
curl -X POST http://localhost:7146/graphql \
  -H "Content-Type: application/json" \
  -d '{
    "query": "mutation { createProduct(input: { name: \"Desk Lamp\", category: \"Office\", price
  }'
```

```
{
  "data": {
    "createProduct": {
      "id": 4,
      "name": "Desk Lamp",
      "price": 39.99
    }
  }
}
```

Update a product:

```
curl -X POST http://localhost:7146/graphql \
  -H "Content-Type: application/json" \
  -d '{
    "query": "mutation { updateProduct(id: 4, input: { price: 44.99, inStock: false }) { id name
  }'
```

```
{
  "data": {
    "updateProduct": {
      "id": 4,
      "name": "Desk Lamp",
      "price": 44.99,
      "inStock": false
    }
  }
```

```
}
```

Delete a product:

```
curl -X POST http://localhost:7146/graphql \
  -H "Content-Type: application/json" \
  -d '{
    "query": "mutation { deleteProduct(id: 4) { success message } }"
  }'
```

```
{
  "data": {
    "deleteProduct": {
      "success": true,
      "message": "Product deleted"
    }
  }
}
```

---

7. Nested Types and Relationships

The real power of GraphQL: traversing relationships in a single query. Authors, posts, comments -- all in one request.

Update `src/graphql/schema.graphql`:

```
type User {
    id: Int!
    name: String!
    email: String!
    posts: [Post!]!
}

type Post {
    id: Int!
    title: String!
    body: String!
    published: Boolean!
    author: User!
    comments: [Comment!]!
    commentCount: Int!
}

type Comment {
    id: Int!
    authorName: String!
    body: String!
    post: Post!
}

type Query {
    posts: [Post!]!
    post(id: Int!): Post
    user(id: Int!): User
}

type Mutation {
```

```
    createPost(userId: Int!, title: String!, body: String!, published: Boolean): Post!
    addComment(postId: Int!, authorName: String!, body: String!): Comment!
}
```

Add resolvers for the nested types:

```
@GraphQL.resolve("Query", "posts")
async def resolve_posts(root, args):
    posts, count = Post.where("published = ?", [1])
    return [p.to_dict() for p in posts]


@GraphQL.resolve("Query", "post")
async def resolve_post(root, args):
    post = Post.find(args["id"])
    return post.to_dict() if post else None


# Resolve Post.author (nested field)
@GraphQL.resolve("Post", "author")
async def resolve_post_author(post, args):
    user = User.find(post["user_id"])
    return user.to_dict() if user else None


# Resolve Post.comments (nested field)
@GraphQL.resolve("Post", "comments")
async def resolve_post_comments(post, args):
    comments, count = Comment.where("post_id = ?", [post["id"]])
    return [c.to_dict() for c in comments]


# Resolve Post.commentCount (computed field)
@GraphQL.resolve("Post", "commentCount")
async def resolve_post_comment_count(post, args):
    comments, count = Comment.where("post_id = ?", [post["id"]])
    return count


# Resolve User.posts (nested field)
@GraphQL.resolve("User", "posts")
async def resolve_user_posts(user, args):
    posts, count = Post.where("user_id = ?", [user["id"]])
    return [p.to_dict() for p in posts]
```

Now clients can query across relationships in a single request:

```
curl -X POST http://localhost:7146/graphql \
  -H "Content-Type: application/json" \
  -d '{
    "query": "{ posts { id title author { name email } comments { authorName body } commentCount
  }'
```

```
{
  "data": {
    "posts": [
      {
        "id": 1,
        "title": "My First Post",
        "author": {
```

```
          "name": "Alice",
          "email": "alice@example.com"
        },
        "comments": [
          {"authorName": "Bob", "body": "Great post!"}
        ],
        "commentCount": 1
      }
    ]
  }
}
```

One request. Exactly the fields the client needs. No over-fetching. No under-fetching.

---

8. Auto-Generating Schema from ORM Models

If your ORM models have `auto_crud = True`, Tina4 can auto-generate GraphQL types and basic CRUD resolvers for them. Enable this in `.env`:

```
TINA4_GRAPHQL_AUTO_SCHEMA=true
```

With this setting, every ORM model with `auto_crud = True` automatically gets:

- A GraphQL type with fields matching the model properties
- A query to list all records (e.g., `products`)
- A query to get one by ID (e.g., `product(id: Int!)`)
- A mutation to create (e.g., `createProduct(input: ProductInput!)`)
- A mutation to update (e.g., `updateProduct(id: Int!, input: ProductUpdateInput!)`)
- A mutation to delete (e.g., `deleteProduct(id: Int!)`)

You can still define custom resolvers that override the auto-generated ones. Custom resolvers always take precedence.

---

9. The GraphiQL Playground

When `TINA4_DEBUG=true`, Tina4 serves a GraphiQL interactive playground at:

```
http://localhost:7146/graphql/playground
```

GraphiQL gives you:

- A query editor with syntax highlighting and auto-completion
- A documentation explorer showing all types, queries, and mutations
- A results panel showing the response
- Query history

This is the fastest way to explore and test your GraphQL API during development. Type a query on the left, click the play button, and see the results on the right. The documentation explorer on the right sidebar shows every type, field, and argument available in your schema.

GraphiQL is only available when `TINA4_DEBUG=true`. In production, it is disabled automatically.

---

10. Query Variables

For production use, clients should send query variables separately from the query string. This prevents injection and allows query caching.

```
curl -X POST http://localhost:7146/graphql \
  -H "Content-Type: application/json" \
  -d '{
    "query": "query GetProduct($id: Int!) { product(id: $id) { id name price } }",
    "variables": {"id": 1}
  }'
```

```
{
  "data": {
    "product": {
      "id": 1,
      "name": "Wireless Keyboard",
      "price": 79.99
    }
  }
}
```

The query uses `$id` as a placeholder, and the actual value comes from the `variables` object. This is safer and more efficient than string interpolation.

---

11. Exercise: Build a GraphQL API for a Blog

Build a complete GraphQL API for a blog with posts, authors, and comments.

Requirements

- **Schema** -- Define types for `User`, `Post`, and `Comment` with the following fields:
- `User`: id, name, email, posts (nested)
- `Post`: id, title, body, published, author (nested), comments (nested), commentCount (computed)
- `Comment`: id, authorName, body, createdAt
- **Queries**:
- `posts` -- List all published posts
- `post(id: Int!)` -- Get a single post by ID
- `user(id: Int!)` -- Get a user with their posts
- **Mutations**:

- `createPost(userId: Int!, title: String!, body: String!, published: Boolean)` -- Create a new post
- `addComment(postId: Int!, authorName: String!, body: String!)` -- Add a comment to a post
- Use the ORM models from Chapter 6 (User, Post, Comment).

Test with these queries:

```
# List published posts with author names
curl -X POST http://localhost:7146/graphql \
  -H "Content-Type: application/json" \
  -d '{"query": "{ posts { id title author { name } commentCount } }"}'

# Get a specific post with all comments
curl -X POST http://localhost:7146/graphql \
  -H "Content-Type: application/json" \
  -d '{"query": "{ post(id: 1) { title body author { name email } comments { authorName body } }

# Create a post
curl -X POST http://localhost:7146/graphql \
  -H "Content-Type: application/json" \
  -d '{"query": "mutation { createPost(userId: 1, title: \"GraphQL is great\", body: \"Here is w

# Add a comment
curl -X POST http://localhost:7146/graphql \
  -H "Content-Type: application/json" \
  -d '{"query": "mutation { addComment(postId: 1, authorName: \"Carol\", body: \"Nice article!\"
```

---

12. Solution

Schema

Create `src/graphql/blog_schema.graphql`:

```
type User {
    id: Int!
    name: String!
    email: String!
    posts: [Post!]!
}

type Post {
    id: Int!
    title: String!
    body: String!
    published: Boolean!
    author: User!
    comments: [Comment!]!
    commentCount: Int!
    createdAt: String
}

type Comment {
    id: Int!
    authorName: String!
```

```
    body: String!
    createdAt: String
}

type Query {
    posts: [Post!]!
    post(id: Int!): Post
    user(id: Int!): User
}

type Mutation {
    createPost(userId: Int!, title: String!, body: String!, published: Boolean): Post!
    addComment(postId: Int!, authorName: String!, body: String!): Comment!
}
```

Resolvers

Create `src/graphql/blog_resolvers.py`:

```
from tina4_python.graphql import GraphQL


@GraphQL.resolve("Query", "posts")
async def resolve_posts(root, args):
    posts, count = Post.where("published = ?", [1])
    return [p.to_dict() for p in posts]


@GraphQL.resolve("Query", "post")
async def resolve_post(root, args):
    post = Post.find(args["id"])
    return post.to_dict() if post else None


@GraphQL.resolve("Query", "user")
async def resolve_user(root, args):
    user = User.find(args["id"])
    return user.to_dict() if user else None


@GraphQL.resolve("Post", "author")
async def resolve_post_author(post, args):
    user = User.find(post["user_id"])
    return user.to_dict() if user else None


@GraphQL.resolve("Post", "comments")
async def resolve_post_comments(post, args):
    comments, count = Comment.where("post_id = ?", [post["id"]])
    return [c.to_dict() for c in comments]


@GraphQL.resolve("Post", "commentCount")
async def resolve_post_comment_count(post, args):
    comments, count = Comment.where("post_id = ?", [post["id"]])
    return count


@GraphQL.resolve("User", "posts")
```

```
async def resolve_user_posts(user, args):
    posts, count = Post.where("user_id = ?", [user["id"]])
    return [p.to_dict() for p in posts]


@GraphQL.resolve("Mutation", "createPost")
async def resolve_create_post(root, args):
    user = User.find(args["userId"])

    if user is None:
        raise Exception("User not found")

    post = Post()
    post.user_id = args["userId"]
    post.title = args["title"]
    post.body = args["body"]
    post.published = bool(args.get("published", False))
    post.save()

    return post.to_dict()


@GraphQL.resolve("Mutation", "addComment")
async def resolve_add_comment(root, args):
    post = Post.find(args["postId"])

    if post is None:
        raise Exception("Post not found")

    comment = Comment()
    comment.post_id = args["postId"]
    comment.author_name = args["authorName"]
    comment.body = args["body"]
    comment.save()

    return comment.to_dict()
```

Expected output for `{ posts { id title author { name } commentCount } }`:

```
{
  "data": {
    "posts": [
      {
        "id": 1,
        "title": "My First Post",
        "author": {"name": "Alice"},
        "commentCount": 1
      },
      {
        "id": 2,
        "title": "GraphQL is great",
        "author": {"name": "Alice"},
        "commentCount": 0
      }
    ]
  }
}
```

---

13. Input Validation

Tina4's GraphQL engine validates arguments against their declared types before the resolver runs. If validation fails, the resolver is skipped and a clean error is returned.

What Gets Validated

- **Non-null enforcement**: `ID!` means the argument cannot be null or empty
- **Scalar type checking**: `Int` must be numeric, `Boolean` must be bool, `String` must be scalar
- **Type coercion**: string `"123"` coerces to Int `123`, string `"true"` coerces to Boolean `True`
- **List item validation**: `[Int!]` checks each item in the list

Example

```
gql.schema.add_query("user", {"id": "ID!"}, "User", lambda r, a, c: find_user(a["id"]))
```

Query with missing required argument:

```
{ user { id name } }
```

Response:

```
{
  "data": { "user": null },
  "errors": [
    { "message": "Argument 'id' on field 'user' is required (type: ID!)", "path": ["user"] }
  ]
}
```

The resolver never runs. The client gets a clear error describing what went wrong and where.

---

14. Field-Level Auth Directives

Control access to individual fields using directives. No middleware setup, no wrapper functions: add a directive to the query and Tina4 checks the context.

Available Directives

Directive	Meaning
`@auth`	Field requires any authenticated user
`@role(role: "admin")`	Field requires a specific role
`@guest`	Field is only accessible to unauthenticated users

Passing Auth Context

The `execute()` method accepts a `context` parameter. Pass the authenticated user from your request handler:

```
from tina4_python import get
from tina4_python.graphql import GraphQL

gql = GraphQL()

@get("/api/graphql")
async def graphql_handler(request, response):
    body = request.body
    context = {}
    if request.user:
        context["user"] = {"id": request.user.id, "role": request.user.role}

    result = gql.execute(body["query"], body.get("variables"), context)
    return response.json(result)
```

Using Directives in Queries

```
# Only authenticated users can see this field
{ profile @auth { id name email } }

# Only admins can see salary
{ employees { name salary @role(role: "admin") } }

# Only guests see the registration prompt
{ registrationBanner @guest }
```

When a directive check fails, the field returns `null` and is silently excluded from the response. No error is added; the field simply doesn't appear, as if it doesn't exist for that user.

---

15. Gotchas

1. Schema File Not Found

Problem: Queries return errors about unknown types or fields.

Cause: The `.graphql` file is not in `src/graphql/`, or has a syntax error that prevents parsing.

Fix: Make sure schema files are in `src/graphql/` and end with `.graphql`. Check for syntax errors -- a missing `!` or unmatched brace will cause the entire schema to fail silently.

2. Resolver Not Called

Problem: A query returns `null` even though data exists.

Cause: The resolver is not registered, or the type/field names do not match the schema exactly. GraphQL is case-sensitive.

Fix: Verify that `@GraphQL.resolve("Query", "products")` matches `type Query { products: ... }` exactly. Check capitalization.

3. Nested Resolver Returns Wrong Data

Problem: `post.author` returns the entire post instead of the user.

Cause: The nested resolver receives the parent object as its first argument (`post`), not a fresh model. If you return the parent instead of loading the related record, you get the wrong data.

Fix: In a nested resolver, always load the related record from the database using the foreign key from the parent:

```
@GraphQL.resolve("Post", "author")
async def resolve_post_author(post, args):
    user = User.find(post["user_id"])  # Use the foreign key from the parent
    return user.to_dict() if user else None
```

4. Mutation Input Not Parsed

Problem: `args["input"]` is `None` inside a mutation resolver.

Cause: The mutation signature in the schema uses an `input` type, but the client query passes arguments directly instead of wrapping them in an `input` object.

Fix: Match the query to the schema. If the schema says `createProduct(input: ProductInput!)`, the client must send `createProduct(input: { name: "Widget", price: 9.99 })`, not `createProduct(name: "Widget", price: 9.99)`.

5. N+1 Query Problem

Problem: Loading 50 posts with their authors generates 51 database queries (1 for posts + 50 for authors).

Cause: Each nested resolver runs independently. When you resolve `author` for each post, that is one query per post.

Fix: Use the `include` parameter in your ORM queries to eager-load relationships. Tina4's GraphQL engine injects the current selection set into context as `context["__selections"]`, so `fromOrm` resolvers can detect which relationships the client requested and batch-load them:

```
# The fromOrm resolver automatically checks __selections and eager-loads
gql.schema.from_orm(Post)  # Generates queries that use include= when sub-fields match relations
```

For custom resolvers, check `context.get("__selections")` and pre-load related data in a single query.

6. GraphQL Playground Returns 404

Problem: `/graphql/playground` returns a 404 error.

Cause: `TINA4_DEBUG` is set to `false`. The playground is only available in debug mode.

Fix: Set `TINA4_DEBUG=true` in your `.env` file. The playground is intentionally disabled in production.

7. Type Mismatch Between Schema and Resolver

Problem: A field declared as `Int!` in the schema receives a string from the resolver.

Cause: Python's dynamic typing means your resolver might return `"42"` (a string) for an `Int!` field.

Fix: Tina4's input validation now coerces argument types automatically (string `"123"` becomes Int `123`). For return values, `to_dict()` handles type casting for ORM model properties. For computed fields, cast explicitly:

```
return {
    "id": int(product.id),
    "price": float(product.price),
    "inStock": bool(product.in_stock)
}
```

---

14. SOAP / WSDL Services

What is SOAP?

SOAP (Simple Object Access Protocol) is an XML-based messaging protocol for exchanging structured data between systems. It predates REST and GraphQL, but remains common in enterprise integrations -- banking, healthcare, government, and ERP systems all rely on SOAP services.

A WSDL (Web Services Description Language) file describes a SOAP service: what operations are available, what parameters they accept, and what they return. Clients use the WSDL to auto-generate code for calling the service.

Tina4 includes a zero-dependency SOAP 1.1 server that generates WSDL definitions automatically from Python classes and type annotations. No XML authoring required.

Defining a SOAP Service

Create a class that extends `WSDL`. Each method decorated with `@wsdl_operation` becomes a SOAP operation. The decorator takes a dict describing the response fields and their types.

```
from tina4_python.wsdl import WSDL, wsdl_operation
from tina4_python.core.router import get, post

class Calculator(WSDL):
    @wsdl_operation({"Result": int})
    def Add(self, a: int, b: int):
        return {"Result": a + b}

    @wsdl_operation({"Result": int})
    def Multiply(self, a: int, b: int):
        return {"Result": a * b}

@get("/calculator")
@post("/calculator")
async def calculator_endpoint(request, response):
    service = Calculator(request)
    return response(service.handle())
```

That is the entire service. The `handle()` method inspects the request:

- **GET** (or `?wsdl` query parameter) -- returns the auto-generated WSDL definition
- **POST** with SOAP XML body -- parses the XML, finds the operation, converts parameters, calls the method, and returns a SOAP XML response

Type Annotations Map to XSD

Python type annotations on method parameters control how incoming XML values are converted. The WSDL generator also uses them to produce correct XSD type declarations.

Python type	XSD type
`str`	`xsd:string`
`int`	`xsd:int`
`float`	`xsd:double`
`bool`	`xsd:boolean`
`bytes`	`xsd:base64Binary`
`List[T]`	Element of type T (repeated)
`Optional[T]`	Type T (nillable)

A More Complete Example

```
from typing import List, Optional
from tina4_python.wsdl import WSDL, wsdl_operation
from tina4_python.core.router import get, post

class UserService(WSDL):
    @wsdl_operation({"Name": str, "Email": str, "Active": bool})
    def GetUser(self, user_id: int):
        user = User.find(user_id)
        if user:
            return {
                "Name": user.name,
                "Email": user.email,
                "Active": bool(user.active),
            }
        return {"Name": "", "Email": "", "Active": False}

    @wsdl_operation({"Total": int, "Average": float, "Error": Optional[str]})
    def SumList(self, Numbers: List[int]):
        if not Numbers:
            return {"Total": 0, "Average": 0.0, "Error": "Empty list"}
        return {
            "Total": sum(Numbers),
            "Average": sum(Numbers) / len(Numbers),
            "Error": None,
        }

@get("/api/users/soap")
@post("/api/users/soap")
async def user_soap(request, response):
    service = UserService(request)
    return response(service.handle())
```

Lifecycle Hooks

Override `on_request` and `on_result` to add validation, logging, or transformation around every operation call.

```
class AuditedService(WSDL):
    def on_request(self, request):
        print(f"SOAP request from {request.headers.get('x-forwarded-for', 'unknown')}")

    def on_result(self, result):
        result["Timestamp"] = datetime.now().isoformat()
        return result

    @wsdl_operation({"Status": str, "Timestamp": str})
    def Ping(self):
        return {"Status": "ok"}
```

Testing with curl

Fetch the WSDL definition:

```
curl http://localhost:7146/calculator?wsdl
```

Call the Add operation with a SOAP request:

```
curl -X POST http://localhost:7146/calculator \
  -H "Content-Type: text/xml" \
  -d '<?xml version="1.0" encoding="UTF-8"?>
<soap:Envelope xmlns:soap="http://schemas.xmlsoap.org/soap/envelope/">
  <soap:Body>
    <Add>
      <a>5</a>
      <b>3</b>
    </Add>
  </soap:Body>
</soap:Envelope>'
```

Response:

```
<?xml version="1.0" encoding="UTF-8"?>
<soap:Envelope xmlns:soap="http://schemas.xmlsoap.org/soap/envelope/">
  <soap:Body>
    <AddResponse>
      <Result>8</Result>
    </AddResponse>
  </soap:Body>
</soap:Envelope>
```

If the operation name is wrong or the XML is malformed, the service returns a SOAP fault:

```
<soap:Fault>
  <faultcode>Client</faultcode>
  <faultstring>Unknown operation: Subtract</faultstring>
</soap:Fault>
```

When to Use SOAP vs REST vs GraphQL

Scenario	Use
New API for web/mobile clients	REST or GraphQL
Integrating with legacy enterprise systems (SAP, banks, government)	SOAP
Clients require a machine-readable service contract	SOAP (WSDL)
Simple internal microservices	REST
Clients with diverse data needs	GraphQL

SOAP is rarely the right choice for new APIs, but when you need to expose a service that legacy systems can consume, Tina4 makes it straightforward -- define a class, annotate your types, and the framework handles the XML.

Real-time with WebSocket

1. The Refresh Button Problem

Your project management app needs live updates. Someone moves a card from "In Progress" to "Done." Everyone else on the team should see it. No page refresh. No polling. No waiting.

Traditional HTTP is request-response. The client asks. The server answers. The server cannot push data on its own. WebSocket breaks that wall. A persistent, bi-directional connection between browser and server. Either side sends messages at any time. The connection stays open until one side closes it.

Tina4 treats WebSocket the same way it treats routing. A decorator. A handler. The path determines which handler processes the connection.

---

2. What WebSocket Is

HTTP works like this:

```
Client: "Give me /api/products"
Server: "Here are the products" (connection closes)
Client: "Give me /api/products" (new connection)
Server: "Here are the products" (connection closes)
```

WebSocket works like this:

```
Client: "I want to upgrade to WebSocket on /ws/chat"
Server: "Upgrade accepted. Connection open."
Client: "Hello everyone!"
Server: "Alice says: Hello everyone!"  (pushed to all clients)
Server: "Bob joined the chat"  (pushed to all clients, at any time)
Client: "Goodbye!"
Server: "Alice says: Goodbye!"
Client: (closes connection)
```

Key differences:

- **Persistent**: The connection stays open. No repeated handshakes.
- **Bi-directional**: The server can push data without the client asking.
- **Low overhead**: After the initial handshake, messages are tiny (no HTTP headers per message).
- **Real-time**: Messages arrive within milliseconds.

---

3. @websocket -- WebSocket as a Route

In Tina4, you define WebSocket handlers using the `@websocket` decorator:

```
from tina4_python.core.router import websocket
```

```
@websocket("/ws/echo")
async def echo_handler(connection, event, data):
    if event == "message":
        await connection.send(f"Echo: {data}")
```

This is the simplest WebSocket handler: it receives a message and sends it back with "Echo: " prepended. The handler receives three arguments:

- **connection**: The WebSocket connection object. Use it to send messages.
- **event**: The event type: `"open"`, `"message"`, or `"close"`.
- **data**: The message data (only present for `"message"` events).

Starting the Server

WebSocket runs alongside your HTTP server:

```
tina4 serve
```

```
  Tina4 Python v3.0.0
  HTTP server running at http://0.0.0.0:7146
  WebSocket server running at ws://0.0.0.0:7146
  Press Ctrl+C to stop
```

Both HTTP and WebSocket share the same port. The server detects the protocol upgrade request and routes it to the correct handler.

---

4. Connection Events

Every WebSocket connection goes through three lifecycle events:

Open

Fires when a client connects:

```
import json
from tina4_python.core.router import websocket

@websocket("/ws/notifications")
async def notifications_handler(connection, event, data):
    if event == "open":
        print(f"Client connected: {connection.id}")
        await connection.send(json.dumps({
            "type": "welcome",
            "message": "Connected to notifications",
            "connection_id": connection.id
        }))
```

Message

Fires when a client sends data:

```
import json
from datetime import datetime, timezone
from tina4_python.core.router import websocket

@websocket("/ws/notifications")
```

```
async def notifications_handler(connection, event, data):
    if event == "open":
        await connection.send(json.dumps({
            "type": "welcome",
            "connection_id": connection.id
        }))

    if event == "message":
        message = json.loads(data)
        print(f"Received from {connection.id}: {data}")

        if message.get("type") == "ping":
            await connection.send(json.dumps({
                "type": "pong",
                "timestamp": datetime.now(timezone.utc).isoformat()
            }))
```

Close

Fires when a client disconnects:

```
import json
from tina4_python.core.router import websocket

@websocket("/ws/notifications")
async def notifications_handler(connection, event, data):
    if event == "open":
        print(f"Client connected: {connection.id}")

    if event == "message":
        print(f"Message from {connection.id}: {data}")

    if event == "close":
        print(f"Client disconnected: {connection.id}")
        # Clean up: remove from tracking, notify others, etc.
```

A Complete Handler

Here is a handler that responds to all three events:

```
import json
from datetime import datetime, timezone
from tina4_python.core.router import websocket

@websocket("/ws/chat")
async def chat_handler(connection, event, data):
    if event == "open":
        print(f"[Chat] New connection: {connection.id}")
        await connection.send(json.dumps({
            "type": "system",
            "message": "Welcome to the chat!",
            "your_id": connection.id
        }))

    elif event == "message":
        message = json.loads(data)
        print(f"[Chat] {connection.id}: {message.get('text', data)}")

        await connection.send(json.dumps({
            "type": "message",
```

```
            "from": connection.id,
            "text": message.get("text", data),
            "timestamp": datetime.now(timezone.utc).isoformat()
        }))

    elif event == "close":
        print(f"[Chat] Disconnected: {connection.id}")
```

---

5. Sending to a Single Client

`connection.send()` sends a message to the specific client that triggered the event:

```
import json
import psutil
from datetime import datetime, timezone
from tina4_python.core.router import websocket

@websocket("/ws/private")
async def private_handler(connection, event, data):
    if event == "message":
        message = json.loads(data)
        action = message.get("action", "")

        if action == "get-time":
            await connection.send(json.dumps({
                "type": "time",
                "server_time": datetime.now(timezone.utc).isoformat()
            }))

        if action == "get-status":
            await connection.send(json.dumps({
                "type": "status",
                "uptime": 3600,
                "connections": 42,
                "memory_mb": round(psutil.Process().memory_info().rss / 1024 / 1024, 2)
            }))
```

Only the client that sent the message receives the response. Other connected clients do not see it.

---

6. Broadcasting to All Clients

`connection.broadcast()` sends a message to every client connected to the same WebSocket path:

```
import json
from datetime import datetime, timezone
from tina4_python.core.router import websocket

@websocket("/ws/announcements")
async def announcements_handler(connection, event, data):
    if event == "open":
        await connection.broadcast(json.dumps({
            "type": "system",
```

```
            "message": "A new user joined",
            "online_count": connection.connection_count()
        }))

    if event == "message":
        message = json.loads(data)

        await connection.broadcast(json.dumps({
            "type": "announcement",
            "from": connection.id,
            "text": message.get("text", ""),
            "timestamp": datetime.now(timezone.utc).isoformat()
        }))

    if event == "close":
        await connection.broadcast(json.dumps({
            "type": "system",
            "message": "A user left",
            "online_count": connection.connection_count()
        }))
```

When one client sends a message, every client connected to `/ws/announcements` receives it. This is the foundation for chat rooms, live dashboards, and collaborative editing.

Broadcast Excluding Sender

Sometimes you want to send to everyone except the client that triggered the event:

```
if event == "message":
    message = json.loads(data)

    # Send to sender (confirmation)
    await connection.send(json.dumps({
        "type": "sent",
        "text": message.get("text")
    }))

    # Send to everyone else
    await connection.broadcast(json.dumps({
        "type": "message",
        "from": message.get("username", "Anonymous"),
        "text": message.get("text"),
        "timestamp": datetime.now(timezone.utc).isoformat()
    }), exclude_self=True)
```

The `exclude_self=True` argument to `broadcast()` excludes the sender. The sender gets a "sent" confirmation, and everyone else gets the "message".

Sending JSON

Use `connection.send_json()` to send a Python dict or list as a JSON string without manually calling `json.dumps()`:

```
@websocket("/ws/status")
async def status_handler(connection, event, data):
    if event == "open":
        await connection.send_json({
            "type": "welcome",
            "connection_id": connection.id
```

```
        })

    if event == "message":
        await connection.send_json({
            "type": "ack",
            "received": data
        })
```

`send_json()` serialises the data to JSON for you. It is equivalent to `connection.send(json.dumps(data))` but saves you the import and the call.

Closing a Connection

Use `connection.close()` to close the connection from the server side:

```
@websocket("/ws/secure")
async def secure_handler(connection, event, data):
    if event == "open":
        token = connection.params.get("token")
        if not token or not valid_token(token):
            await connection.send_json({"error": "Unauthorized"})
            await connection.close()
            return

        await connection.send_json({"type": "welcome"})
```

Connection Methods Summary

Method	Description
`await connection.send(message)`	Send a string message to this connection only
`await connection.send_json(data)`	Send a dict/list as JSON to this connection only
`await connection.broadcast(message)`	Send to all connections on the same path
`await connection.broadcast(message, exclude_self=True)`	Send to all except this connection
`await connection.close()`	Close this connection from the server side
`connection.id`	Unique identifier for this connection
`connection.params`	Path parameters extracted from the URL
`connection.connection_count()`	Number of active connections on this path

---

7. Path-Scoped Isolation

Different WebSocket paths are completely isolated. Clients connected to `/ws/chat/room-1` do not see messages from `/ws/chat/room-2`:

```
import json
from tina4_python.core.router import websocket

@websocket("/ws/chat/{room}")
async def room_handler(connection, event, data):
    room = connection.params["room"]

    if event == "open":
        print(f"[Room {room}] New connection: {connection.id}")
        await connection.broadcast(json.dumps({
            "type": "system",
            "message": f"Someone joined room {room}",
            "room": room,
            "online": connection.connection_count()
        }))

    if event == "message":
        message = json.loads(data)

        await connection.broadcast(json.dumps({
            "type": "message",
            "room": room,
            "from": message.get("username", "Anonymous"),
            "text": message.get("text", ""),
            "timestamp": datetime.now(timezone.utc).isoformat()
        }))

    if event == "close":
        await connection.broadcast(json.dumps({
            "type": "system",
            "message": f"Someone left room {room}",
            "room": room,
            "online": connection.connection_count()
        }))
```

Connect to different rooms:

```
ws://localhost:7146/ws/chat/general    -- general chat
ws://localhost:7146/ws/chat/random     -- random chat
ws://localhost:7146/ws/chat/dev-team   -- dev team chat
```

Broadcasting in `/ws/chat/general` only reaches clients connected to `/ws/chat/general`. The `dev-team` and `random` rooms are separate.

Chat rooms. Project-specific notifications. Per-user channels. No extra configuration. The URL path is the isolation boundary.

---

8. Building a Live Chat

Here is a complete chat application with usernames, typing indicators, and message history:

WebSocket Handler

Create `src/routes/chat_ws.py`:

```
import json
from datetime import datetime, timezone
```

```
from tina4_python.core.router import websocket

chat_users = {}

@websocket("/ws/livechat/{room}")
async def livechat_handler(connection, event, data):
    room = connection.params["room"]

    if event == "open":
        chat_users[connection.id] = {
            "id": connection.id,
            "username": "Anonymous",
            "room": room,
            "joined_at": datetime.now(timezone.utc).isoformat()
        }

        await connection.send(json.dumps({
            "type": "welcome",
            "message": f"Connected to room: {room}",
            "your_id": connection.id,
            "online": connection.connection_count()
        }))

    if event == "message":
        message = json.loads(data)
        msg_type = message.get("type", "message")

        if msg_type == "set-username":
            old_name = chat_users[connection.id]["username"]
            chat_users[connection.id]["username"] = message["username"]

            await connection.broadcast(json.dumps({
                "type": "system",
                "message": f"{old_name} is now known as {message['username']}"
            }))

        if msg_type == "message":
            username = chat_users[connection.id]["username"]

            await connection.broadcast(json.dumps({
                "type": "message",
                "from": username,
                "from_id": connection.id,
                "text": message.get("text", ""),
                "timestamp": datetime.now(timezone.utc).isoformat()
            }))

        if msg_type == "typing":
            username = chat_users[connection.id]["username"]

            await connection.broadcast(json.dumps({
                "type": "typing",
                "username": username
            }), exclude_self=True)

    if event == "close":
        username = chat_users.get(connection.id, {}).get("username", "Unknown")
        chat_users.pop(connection.id, None)
```

```
        await connection.broadcast(json.dumps({
            "type": "system",
            "message": f"{username} left the chat",
            "online": connection.connection_count()
        }))
```

---

9. Live Notifications

WebSocket is great for pushing notifications to users in real time:

```
import json
from datetime import datetime, timezone
from tina4_python.core.router import websocket, post, push_to_websocket

@websocket("/ws/notifications/{user_id}")
async def notification_handler(connection, event, data):
    user_id = connection.params["user_id"]

    if event == "open":
        print(f"[Notifications] User {user_id} connected")
        await connection.send(json.dumps({
            "type": "connected",
            "message": "Listening for notifications"
        }))

    if event == "message":
        message = json.loads(data)
        if message.get("type") == "mark-read":
            print(f"[Notifications] User {user_id} read notification {message['id']}")


@post("/api/orders/{order_id}/ship")
async def ship_order(request, response):
    order_id = request.params["order_id"]
    user_id = request.body.get("user_id", 0)

    # Update order status in database...

    # Send real-time notification via WebSocket
    await push_to_websocket(f"/ws/notifications/{user_id}", json.dumps({
        "type": "notification",
        "title": "Order Shipped",
        "message": f"Your order #{order_id} has been shipped!",
        "action_url": f"/orders/{order_id}",
        "timestamp": datetime.now(timezone.utc).isoformat()
    }))

    return response({"message": "Order shipped, user notified"})
```

The `push_to_websocket()` function lets your HTTP handlers send messages to WebSocket clients. This bridges the gap between traditional request-response endpoints and real-time notifications.

---

10. Connecting from JavaScript

Tina4 provides `frond.js`, a built-in JavaScript helper library that includes WebSocket support:

Basic Connection

```
<script src="/js/frond.js"></script>
<script>
    const ws = frond.ws("/ws/chat/general");

    ws.on("open", function () {
        console.log("Connected to chat");
    });

    ws.on("message", function (data) {
        const message = JSON.parse(data);
        console.log("Received:", message);

        if (message.type === "message") {
            addMessageToUI(message.from, message.text, message.timestamp);
        }

        if (message.type === "system") {
            addSystemMessage(message.message);
        }
    });

    ws.on("close", function () {
        console.log("Disconnected from chat");
    });

    function sendMessage(text) {
        ws.send(JSON.stringify({
            type: "message",
            text: text
        }));
    }

    function setUsername(name) {
        ws.send(JSON.stringify({
            type: "set-username",
            username: name
        }));
    }
</script>
```

Auto-Reconnect

`frond.js` automatically reconnects when the connection drops:

```
const ws = frond.ws("/ws/notifications/42", {
    reconnect: true,           // Enable auto-reconnect (default: true)
    reconnectInterval: 3000,   // Retry every 3 seconds
    maxReconnectAttempts: 10   // Give up after 10 attempts
});

ws.on("reconnect", function (attempt) {
    console.log("Reconnecting... attempt " + attempt);
});
```

```
ws.on("reconnect_failed", function () {
    console.log("Failed to reconnect after maximum attempts");
});
```

Using Native WebSocket

If you prefer not to use `frond.js`, the native WebSocket API works too:

```
const ws = new WebSocket("ws://localhost:7146/ws/chat/general");

ws.onopen = function () {
    console.log("Connected");
    ws.send(JSON.stringify({ type: "set-username", username: "Alice" }));
};

ws.onmessage = function (event) {
    const message = JSON.parse(event.data);
    console.log("Received:", message);
};

ws.onclose = function () {
    console.log("Disconnected");
};

ws.onerror = function (error) {
    console.error("WebSocket error:", error);
};
```

`frond.js` gives you auto-reconnect, message buffering during reconnection, and a cleaner event API. Native WebSocket does not.

---

11. A Complete Chat Page

Here is a full chat page using templates and WebSocket:

Create `src/templates/chat.html`:

```
{% extends "base.html" %}

{% block title %}Chat - {{ room }}{% endblock %}

{% block content %}
    <h1>Chat Room: {{ room }}</h1>
    <p id="status">Connecting...</p>
    <p id="online">Online: 0</p>

    <div id="messages" style="border: 1px solid #ddd; height: 400px; overflow-y: scroll; padding
    </div>

    <div id="typing" style="height: 20px; color: #888; font-style: italic; margin-bottom: 4px;">

    <form id="chat-form" style="display: flex; gap: 8px;">
        <input type="text" id="message-input" placeholder="Type a message..."
               style="flex: 1; padding: 8px; border: 1px solid #ddd; border-radius: 4px;">
        <button type="submit" style="padding: 8px 16px; background: #333; color: white; border:
            Send
        </button>
```

```
</form>

<script src="/js/frond.js"></script>
<script>
    const room = "{{ room }}";
    const ws = frond.ws("/ws/livechat/" + room);
    const messagesDiv = document.getElementById("messages");
    const statusEl = document.getElementById("status");
    const onlineEl = document.getElementById("online");
    const typingEl = document.getElementById("typing");

    let username = prompt("Enter your username:") || "Anonymous";

    ws.on("open", function () {
        statusEl.textContent = "Connected";
        ws.send(JSON.stringify({ type: "set-username", username: username }));
    });

    ws.on("message", function (data) {
        const msg = JSON.parse(data);

        if (msg.type === "message") {
            const div = document.createElement("div");
            div.style.marginBottom = "8px";
            div.innerHTML = "<strong>" + msg.from + ":</strong> " + msg.text;
            messagesDiv.appendChild(div);
            messagesDiv.scrollTop = messagesDiv.scrollHeight;
        }

        if (msg.type === "system") {
            const div = document.createElement("div");
            div.style.cssText = "color: #888; font-style: italic; margin-bottom: 8px;";
            div.textContent = msg.message;
            messagesDiv.appendChild(div);
            messagesDiv.scrollTop = messagesDiv.scrollHeight;
        }

        if (msg.type === "typing") {
            typingEl.textContent = msg.username + " is typing...";
            setTimeout(function () { typingEl.textContent = ""; }, 2000);
        }

        if (msg.online !== undefined) {
            onlineEl.textContent = "Online: " + msg.online;
        }
    });

    ws.on("close", function () {
        statusEl.textContent = "Disconnected. Reconnecting...";
    });

    document.getElementById("chat-form").addEventListener("submit", function (e) {
        e.preventDefault();
        const input = document.getElementById("message-input");
        if (input.value.trim()) {
            ws.send(JSON.stringify({ type: "message", text: input.value }));
            input.value = "";
        }
    });
```

```
        document.getElementById("message-input").addEventListener("input", function () {
            ws.send(JSON.stringify({ type: "typing" }));
        });
    </script>
{% endblock %}
```

Create the route to serve the page:

```
from tina4_python.core.router import get, template

@get("/chat/{room}")
async def chat_page(request, response):
    room = request.params["room"]
    return response(template("chat.html", room=room))
```

Visit `http://localhost:7146/chat/general` in two browser tabs. Type in one tab and watch the message appear in the other instantly.

---

12. Exercise: Build a Real-Time Chat Room

Build a WebSocket chat room with the following features:

Requirements

- WebSocket endpoint at `/ws/room/{room_name}` that handles:
- `open`: Send welcome message with connection count
- `message` with type `set-name`: Set the user's display name
- `message` with type `chat`: Broadcast the message to all clients with the sender's name
- `close`: Broadcast that the user left
- HTTP endpoint at `GET /room/{room_name}` that serves an HTML chat page
- The chat page should:
- Prompt for a username on load
- Display messages in real time
- Show system messages (join/leave) in a different style
- Show the count of online users

Test by:

- Open `http://localhost:7146/room/test` in two browser tabs
- Set different usernames in each
- Send messages from both tabs and verify they appear in both
- Close one tab and verify the "user left" message appears in the other

---

13. Solution

Create `src/routes/chat_room.py`:

```
import json
from datetime import datetime, timezone
from tina4_python.core.router import websocket, get, template

room_users = {}

@websocket("/ws/room/{room_name}")
async def room_ws_handler(connection, event, data):
    room = connection.params["room_name"]
    key = f"{room}:{connection.id}"

    if event == "open":
        room_users[key] = "Anonymous"

        await connection.send(json.dumps({
            "type": "system",
            "message": f"Welcome to room: {room}. Set your name with: "
                       '{"type": "set-name", "name": "YourName"}',
            "online": connection.connection_count()
        }))

        await connection.broadcast(json.dumps({
            "type": "system",
            "message": "A new user joined",
            "online": connection.connection_count()
        }), exclude_self=True)

    if event == "message":
        msg = json.loads(data)
        msg_type = msg.get("type", "chat")

        if msg_type == "set-name":
            old_name = room_users.get(key, "Anonymous")
            new_name = msg.get("name", "Anonymous")
            room_users[key] = new_name

            await connection.broadcast(json.dumps({
                "type": "system",
                "message": f"{old_name} changed their name to {new_name}"
            }))

        if msg_type == "chat":
            username = room_users.get(key, "Anonymous")

            await connection.broadcast(json.dumps({
                "type": "chat",
                "from": username,
                "text": msg.get("text", ""),
                "timestamp": datetime.now(timezone.utc).strftime("%H:%M:%S")
            }))

    if event == "close":
        username = room_users.get(key, "Anonymous")
        room_users.pop(key, None)
```

```
        await connection.broadcast(json.dumps({
            "type": "system",
            "message": f"{username} left the room",
            "online": connection.connection_count()
        }))


@get("/room/{room_name}")
async def room_page(request, response):
    room = request.params["room_name"]
    return response(template("room.html", room=room))
```

Create `src/templates/room.html`:

```
{% extends "base.html" %}

{% block title %}Room: {{ room }}{% endblock %}

{% block content %}
    <h1>Room: {{ room }}</h1>
    <p id="online" style="color: #666;">Online: 0</p>

    <div id="messages" style="border: 1px solid #ddd; height: 400px; overflow-y: auto; padding:
    </div>

    <form id="form" style="display: flex; gap: 8px;">
        <input type="text" id="input" placeholder="Type a message..."
               style="flex: 1; padding: 10px; border: 1px solid #ddd; border-radius: 4px; font-s
        <button type="submit"
                style="padding: 10px 20px; background: #1a1a2e; color: white; border: none; bord
            Send
        </button>
    </form>

    <script src="/js/frond.js"></script>
    <script>
        const room = "{{ room }}";
        const username = prompt("Choose a username:") || "Anonymous";
        const ws = frond.ws("/ws/room/" + room);
        const msgs = document.getElementById("messages");

        ws.on("open", function () {
            ws.send(JSON.stringify({ type: "set-name", name: username }));
        });

        ws.on("message", function (raw) {
            const msg = JSON.parse(raw);
            const div = document.createElement("div");
            div.style.marginBottom = "8px";

            if (msg.type === "chat") {
                div.innerHTML = "<strong>" + msg.from + "</strong> <span style='color:#999;font-
            } else if (msg.type === "system") {
                div.style.color = "#888";
                div.style.fontStyle = "italic";
                div.textContent = msg.message;
            }

            msgs.appendChild(div);
```

```
            msgs.scrollTop = msgs.scrollHeight;

            if (msg.online !== undefined) {
                document.getElementById("online").textContent = "Online: " + msg.online;
            }
        });

        document.getElementById("form").addEventListener("submit", function (e) {
            e.preventDefault();
            const input = document.getElementById("input");
            if (input.value.trim()) {
                ws.send(JSON.stringify({ type: "chat", text: input.value }));
                input.value = "";
            }
        });
    </script>
{% endblock %}
```

Open `http://localhost:7146/room/test` in two browser tabs. Set different usernames. Send messages from one tab and watch them appear in both. Close one tab and verify the "left the room" message appears.

---

14. Scaling with a Backplane

When you run a single server instance, `broadcast()` reaches every connected client. But in production you often run multiple instances behind a load balancer. Each instance only knows about its own connections. A message broadcast on instance A never reaches clients connected to instance B.

A backplane solves this. It relays WebSocket messages across all instances using the shared `tina4:ws` pub/sub channel. Tina4 supports Redis and NATS adapters. The file-free local path remains the default; a network backplane needs its broker and the matching client package.

Configuration

Set two environment variables in your `.env`:

```
TINA4_WS_BACKPLANE=redis
TINA4_WS_BACKPLANE_URL=redis://localhost:6379
```

For NATS:

```
TINA4_WS_BACKPLANE=nats
TINA4_WS_BACKPLANE_URL=nats://localhost:4222
```

Every `broadcast()` call delivers to local connections first, then publishes an envelope to the backplane. Sibling instances discard the sender's echo and relay the message to their own local connections. Your WebSocket routes do not change.

Requirements

Install only the client for the adapter your application selects:

```
uv add redis
uv add nats-py
```

These are application dependencies, not Tina4 core dependencies. Tina4 imports them only when it starts the selected adapter. An explicit selection is a deployment contract: an unknown adapter, missing package, or unreachable broker fails startup with `Configured WebSocket backplane failed`. Tina4 never claims distributed delivery while running local-only.

When to Choose a Backplane

Deployment	Choice
One Tina4 process or container	Leave `TINA4_WS_BACKPLANE` unset
Multiple instances and Redis is already operated	`redis`
Multiple instances and NATS is already operated	`nats`
Unsure which broker to introduce	Redis is usually the simpler starting point

The signal is the process count, not traffic volume: once clients can land on different Tina4 instances, broadcasts need a shared backplane. Do not install NATS solely because Tina4 supports it; choose it when the deployment already uses NATS or needs NATS as its wider event bus.

Verify the setup with two Tina4 processes. Connect one WebSocket client to each process, broadcast from the first, and confirm the second client receives the frame. A single-process test proves only local delivery.

If `TINA4_WS_BACKPLANE` is not set, Tina4 broadcasts only to local connections. This is the correct default for a single instance.

---

15. Gotchas

1. WebSocket Needs a Persistent Server

Problem: WebSocket connections drop immediately or do not work at all.

Cause: You are running behind a traditional WSGI server like Gunicorn with sync workers. WSGI does not support persistent connections. Each request is handled and the connection is closed.

Fix: Use `tina4 serve` which runs Tina4's async server that handles both HTTP and WebSocket. For production, use an ASGI server like uvicorn or hypercorn (Tina4 auto-detects them). You can still use Nginx as a reverse proxy, but it must be configured for WebSocket proxying with `proxy_set_header Upgrade $http_upgrade` and `proxy_set_header Connection "upgrade"`.

2. Messages Are Strings, Not Dicts

Problem: `data` in the message handler is a string, not a Python dict.

Cause: WebSocket transmits raw strings. If the client sends JSON, you receive the JSON string, not a decoded dict.

Fix: Always `json.loads(data)` when you expect JSON messages. Always `json.dumps()` when you send structured data. WebSocket does not know about content types -- it is just bytes.

3. Connection Count Is Per-Path

Problem: `connection.connection_count()` returns a lower number than expected.

Cause: Connection count is scoped to the WebSocket path. Clients on `/ws/chat/room-1` are counted separately from `/ws/chat/room-2`.

Fix: This is by design. Each path is an isolated group. If you need a global connection count across all paths, maintain a counter in a shared variable or use a module-level dict.

4. Broadcasting Does Not Scale Across Servers

Problem: Users connected to different server instances do not see each other's messages.

Cause: `TINA4_WS_BACKPLANE` is unset, so `connection.broadcast()` correctly uses local-only mode. Multiple instances behind a load balancer each maintain their own connection sets.

Fix: Use a pub/sub backend like Redis to relay messages across server instances. Each server subscribes to a Redis channel, and broadcast messages are published to Redis so all servers receive them.

5. Large Messages Cause Disconnects

Problem: The connection drops when sending a large message.

Cause: WebSocket servers typically have a maximum message size. The default is usually around 1MB. If your message exceeds this, the server closes the connection.

Fix: Keep messages small (under 64KB is a good target). For large data transfers, use HTTP endpoints instead of WebSocket. WebSocket is designed for small, frequent messages -- not bulk data transfer.

6. Memory Leak from Tracking Connected Users

Problem: The server's memory usage grows over time and eventually crashes.

Cause: You store user data in a dict (like `chat_users`) but do not clean it up when users disconnect. If the `close` event handler does not remove the user from the dict, the dict grows indefinitely.

Fix: Always clean up in the `close` handler. Remove disconnected users from tracking dicts: `chat_users.pop(connection.id, None)`. Test by connecting and disconnecting repeatedly to verify memory stays stable.

7. No Authentication on WebSocket Connect

Problem: Anyone can connect to your WebSocket endpoint and see all messages.

Cause: The WebSocket upgrade request does not carry your JWT token in the `Authorization` header (browsers do not support custom headers on WebSocket connections).

Fix: Pass the token as a query parameter: `ws://localhost:7146/ws/chat?token=eyJ...`. In your `open` handler, validate the token and disconnect if invalid. Use a short-lived token specifically for WebSocket connections. Alternatively, authenticate via an HTTP endpoint first, store the session, and check the session cookie during the WebSocket upgrade.

Server-Sent Events (SSE)

1. The Polling Problem

Your monitoring dashboard needs live metrics. CPU usage. Memory. Active connections. The numbers change every few seconds.

The obvious solution: poll. A timer fires every three seconds. The browser sends a GET request. The server queries the database. The server responds. The browser updates the page. Repeat.

That works. It also wastes resources. Nineteen out of twenty polls return the same data. Each one opens a connection, sends headers, waits for a response, and closes the connection. The server does the same work whether the data changed or not.

WebSocket solves this. A persistent, bi-directional connection. The server pushes when data changes. But WebSocket is designed for two-way communication. Chat rooms. Collaborative editing. Live gaming. Your dashboard only needs one direction: server to client.

Server-Sent Events (SSE) is the middle ground. One-way streaming over plain HTTP. The server pushes. The client listens. The browser reconnects automatically if the connection drops. No upgrade handshake. No special protocol. Just HTTP with a twist.

---

2. What SSE Is

HTTP works like this:

```
Client: "Give me /api/metrics"
Server: "Here are the metrics" (connection closes)
Client: "Give me /api/metrics" (new connection, 3 seconds later)
Server: "Here are the metrics" (connection closes)
```

WebSocket works like this:

```
Client: "Upgrade to WebSocket on /ws/metrics"
Server: "Upgrade accepted. Connection open."
Client: "Subscribe to CPU metrics"
Server: "CPU: 42%"
Server: "CPU: 38%"  (pushed at any time)
Client: "Unsubscribe"
```

SSE works like this:

```
Client: "Give me /events (keep the connection open)"
Server: "data: CPU 42%"
Server: "data: CPU 38%"  (pushed, no client request)
Server: "data: CPU 45%"  (keeps flowing)
Client: (just listens)
```

The key differences:

Feature	HTTP Polling	WebSocket	SSE
Direction	Client to server	Both ways	Server to client

Connection	New each time	Persistent	Persistent
Protocol	HTTP	WebSocket (upgrade)	HTTP
Auto-reconnect	No	No (manual)	Yes (built-in)
Binary data	Yes	Yes	No (text only)
Browser support	Universal	Universal	Universal (except IE)
Complexity	Simple	Moderate	Simple

SSE uses plain HTTP. No protocol upgrade. No special server support. The server sets `Content-Type: text/event-stream` and keeps the connection open. The browser's `EventSource` API handles reconnection, event parsing, and last-event-ID tracking.

---

3. Your First Stream

Create `src/routes/events.py`:

```
import asyncio
from tina4_python import get

@get("/events")
async def stream_events(request, response):
    async def generate():
        for i in range(10):
            yield f"data: {{\"count\": {i}, \"message\": \"tick\"}}\n\n"
            await asyncio.sleep(1)
        yield "data: {\"done\": true}\n\n"

    return response.stream(generate())
```

Three things happen here:

- `generate()` is an async generator. Each `yield` produces one SSE message.
- `response.stream()` tells Tina4 to keep the connection open and send each chunk as it arrives.
- The framework sets `Content-Type: text/event-stream`, `Cache-Control: no-cache`, and `Connection: keep-alive` for you.

Start the server and test with curl:

```
curl -N http://localhost:7146/events
```

You see one message per second:

```
data: {"count": 0, "message": "tick"}

data: {"count": 1, "message": "tick"}

data: {"count": 2, "message": "tick"}
```

Each message arrives the moment the server yields it. No buffering. No waiting for the full response.

---

4. The SSE Protocol

SSE messages are plain text with a simple format. Each message is one or more field lines followed by a blank line.

The `data:` Field

The most common field. Contains the message payload.

```
data: Hello, world
```

Multiple `data:` lines in one message get joined with newlines:

```
data: line one
data: line two
```

The client receives this as `"line one\nline two"`.

The `event:` Field

Names the event type. Without it, the browser fires `onmessage`. With it, the browser fires a named event listener.

```
yield "event: metrics\ndata: {\"cpu\": 42}\n\n"
yield "event: alert\ndata: {\"level\": \"high\", \"message\": \"CPU spike\"}\n\n"
```

On the client:

```
source.addEventListener("metrics", (e) => {
    const data = JSON.parse(e.data);
    updateDashboard(data);
});

source.addEventListener("alert", (e) => {
    const data = JSON.parse(e.data);
    showAlert(data.message);
});
```

Named events let you multiplex different data types on a single connection.

The `id:` Field

Sets the last event ID. If the connection drops, the browser sends `Last-Event-ID` in the reconnection request. Your server can resume from where it left off.

```
yield f"id: {i}\ndata: {{\"count\": {i}}}\n\n"
```

The `retry:` Field

Tells the browser how many milliseconds to wait before reconnecting after a disconnect.

```
yield "retry: 5000\n\n"  # reconnect after 5 seconds
```

The Delimiter

Every SSE message ends with two newlines: `\n\n`. This tells the browser that the message is complete. A single `\n` separates fields within one message.

```
# One complete SSE message:
yield "event: update\nid: 42\ndata: {\"value\": 100}\n\n"

# Broken down:
# event: update\n     ← event type
# id: 42\n            ← event ID
# data: {"value": 100}\n  ← payload
# \n                   ← end of message (blank line)
```

---

5. Frontend: EventSource

The browser provides `EventSource` for consuming SSE streams. It handles connection management, reconnection, and message parsing.

Basic Usage

```
const source = new EventSource("/events");

source.onmessage = function (event) {
    const data = JSON.parse(event.data);
    console.log("Received:", data);
};

source.onerror = function () {
    console.log("Connection lost. Reconnecting...");
};

source.onopen = function () {
    console.log("Connected");
};
```

`EventSource` reconnects automatically when the connection drops. The `onerror` handler fires, the browser waits (default 3 seconds), then reconnects. Your server receives a new request with the `Last-Event-ID` header if you sent event IDs.

Named Events

```
const source = new EventSource("/events");

source.addEventListener("metrics", function (event) {
    updateChart(JSON.parse(event.data));
});

source.addEventListener("notification", function (event) {
    showToast(JSON.parse(event.data));
});

source.addEventListener("heartbeat", function (event) {
    // Connection is alive
});
```

Closing the Connection

```
source.close();
```

The browser stops reconnecting. The server receives a client disconnect.

With Authentication

`EventSource` does not support custom headers. Pass tokens as query parameters:

```
const token = getAuthToken();
const source = new EventSource(`/events?token=${token}`);
```

On the server:

```
@get("/events")
async def secure_events(request, response):
    token = request.query.get("token")
    payload = Auth.valid_token(token)
    if not payload:
        return response({"error": "Unauthorized"}, 401)

    async def generate():
        while True:
            yield f"data: {{\"user\": \"{payload['email']}\"}}\n\n"
            await asyncio.sleep(5)

    return response.stream(generate())
```

---

6. Real Examples

Live Dashboard

Stream database metrics every five seconds:

```
import asyncio
from tina4_python import get
from tina4_python.database import Database

@get("/events/dashboard")
async def dashboard_stream(request, response):
    async def generate():
        db = Database()
        while True:
            orders = db.fetch_one("SELECT count(*) as total FROM orders")
            revenue = db.fetch_one("SELECT sum(total) as revenue FROM orders WHERE date(created_

            yield f"data: {{\"orders\": {orders['total']}, \"revenue\": {revenue['revenue'] or 0
            await asyncio.sleep(5)

    return response.stream(generate())
```

The client updates the dashboard numbers without polling. One connection. One query every five seconds. No wasted requests.

Build Progress

Stream deployment status as it happens:

```
import asyncio
import json
from tina4_python import get

@get("/events/deploy/{build_id}")
async def deploy_stream(request, response):
    build_id = request.params["build_id"]

    async def generate():
        steps = [
            ("pull", "Pulling latest code..."),
            ("install", "Installing dependencies..."),
            ("test", "Running tests..."),
            ("build", "Building application..."),
            ("deploy", "Deploying to production..."),
            ("done", "Deployment complete"),
        ]

        for step, message in steps:
            yield f"event: progress\ndata: {json.dumps({'step': step, 'message': message, 'build
            await asyncio.sleep(2)  # simulate work

    return response.stream(generate())
```

The browser shows a progress bar that updates in real time. Each step arrives as it completes. The user watches the deployment unfold instead of staring at a spinner.

LLM Chat Streaming

Stream AI responses token by token. This is how ChatGPT works:

```
import asyncio
import json
import urllib.request
from tina4_python import get

@get("/events/chat")
async def chat_stream(request, response):
    prompt = request.query.get("q", "Hello")

    async def generate():
        # Call the LLM API with streaming enabled
        req_data = json.dumps({
            "model": "claude-sonnet-4-20250514",
            "max_tokens": 512,
            "stream": True,
            "messages": [{"role": "user", "content": prompt}],
        }).encode()

        req = urllib.request.Request(
            "https://api.anthropic.com/v1/messages",
            data=req_data,
            headers={
                "Content-Type": "application/json",
                "x-api-key": os.environ["ANTHROPIC_API_KEY"],
```

```
                "anthropic-version": "2023-06-01",
            },
        )

        with urllib.request.urlopen(req) as resp:
            for line in resp:
                line = line.decode().strip()
                if line.startswith("data: "):
                    chunk = json.loads(line[6:])
                    if chunk.get("type") == "content_block_delta":
                        text = chunk["delta"].get("text", "")
                        yield f"data: {json.dumps({'token': text})}\n\n"
                        await asyncio.sleep(0)

        yield "data: {\"done\": true}\n\n"

    return response.stream(generate())
```

The frontend appends each token as it arrives:

```
const source = new EventSource(`/events/chat?q=${encodeURIComponent(question)}`);
const output = document.getElementById("response");

source.onmessage = function (event) {
    const data = JSON.parse(event.data);
    if (data.done) {
        source.close();
        return;
    }
    output.textContent += data.token;
};
```

The response builds character by character. The user reads as the AI writes. No waiting for the full answer.

File Processing Progress

Upload a CSV, stream the processing status:

```
import asyncio
import json
from tina4_python import post

@post("/api/import")
async def import_csv(request, response):
    file = request.files.get("csv")
    if not file:
        return response({"error": "No file"}, 400)

    lines = file["content"].decode().strip().split("\n")
    total = len(lines) - 1  # minus header

    async def generate():
        for i, line in enumerate(lines[1:], 1):
            # Process each row
            cols = line.split(",")
            # ... insert into database ...
            yield f"data: {json.dumps({'processed': i, 'total': total, 'percent': round(i/total*
            await asyncio.sleep(0)  # yield control
```

```
        yield f"data: {json.dumps({'done': True, 'total': total})}\n\n"

    return response.stream(generate(), content_type="text/event-stream")
```

The browser shows a progress bar that fills as each row is processed. The user sees exactly where the import stands.

---

7. Custom Content Types

SSE defaults to `text/event-stream`, but `response.stream()` accepts any content type. Use this for newline-delimited JSON (NDJSON), chunked binary, or custom protocols.

NDJSON Streaming

```
import asyncio
import json
from tina4_python import get

@get("/api/export")
async def export_stream(request, response):
    async def generate():
        db = Database()
        result = db.fetch("SELECT * FROM products", limit=1000)
        for row in result.records:
            yield json.dumps(row) + "\n"
            await asyncio.sleep(0)

    return response.stream(generate(), content_type="application/x-ndjson")
```

Each line is a complete JSON object. The client reads line by line. No need to parse a giant array. Memory stays flat regardless of how many rows you export.

Consuming NDJSON on the Client

```
async function streamProducts() {
    const response = await fetch("/api/export");
    const reader = response.body.getReader();
    const decoder = new TextDecoder();
    let buffer = "";

    while (true) {
        const { done, value } = await reader.read();
        if (done) break;

        buffer += decoder.decode(value, { stream: true });
        const lines = buffer.split("\n");
        buffer = lines.pop(); // keep incomplete line

        for (const line of lines) {
            if (line.trim()) {
                const product = JSON.parse(line);
                addToTable(product);
            }
        }
    }
}
```

---

8. SSE vs WebSocket

Both keep a persistent connection. Both push data in real time. The choice depends on the direction of communication.

Use Case	Choose	Why
Live dashboard	SSE	Server pushes metrics. Client only listens.
Chat application	WebSocket	Both sides send messages.
Notification feed	SSE	Server pushes. Client marks as read via separate HTTP POST.
Collaborative editor	WebSocket	Both sides send cursor positions and edits.
Build/deploy progress	SSE	Server streams status. Client watches.
LLM token streaming	SSE	Server streams tokens. Client displays them.
Online gaming	WebSocket	Low-latency, bi-directional input and state.
Stock ticker	SSE	Server pushes prices. Client displays.
File upload progress	HTTP (native)	Browser tracks upload. Server tracks processing via SSE.

Rule of thumb: If the client only listens, use SSE. If the client sends data back on the same connection, use WebSocket.

SSE has one advantage WebSocket does not: automatic reconnection. The browser's `EventSource` reconnects without any code. WebSocket requires manual reconnection logic (or `frond.js` which handles it for you).

---

9. Production Considerations

Nginx Proxy Buffering

Nginx buffers responses by default. SSE messages accumulate in the buffer and arrive in batches instead of one at a time. Tina4 sets `X-Accel-Buffering: no` on streaming responses, which tells Nginx to disable buffering for that response.

If you configure Nginx manually:

```
location /events/ {
    proxy_pass http://localhost:7146;
    proxy_buffering off;
    proxy_cache off;
    proxy_set_header Connection '';
    proxy_http_version 1.1;
    chunked_transfer_encoding off;
}
```

Connection Limits

Each SSE connection is a persistent HTTP connection. Most servers limit concurrent connections. The default asyncio server handles thousands. In production:

- Monitor open connections
- Set timeouts on long-running streams
- Close streams when the client no longer needs them

```
@get("/events/metrics")
async def metrics_with_timeout(request, response):
    async def generate():
        for _ in range(3600):  # max 1 hour (one message per second)
            yield f"data: {{\"cpu\": {get_cpu()}}}\n\n"
            await asyncio.sleep(1)
        yield "event: timeout\ndata: {\"message\": \"Stream expired. Reconnect.\"}\n\n"

    return response.stream(generate())
```

Heartbeat Messages

Some proxies and load balancers close idle connections after a timeout (typically 30-60 seconds). Send a heartbeat comment to keep the connection alive:

```
async def generate():
    counter = 0
    while True:
        if has_new_data():
            yield f"data: {json.dumps(get_data())}\n\n"
        else:
            yield ": heartbeat\n\n"  # SSE comment (colon prefix) - ignored by EventSource
        counter += 1
        await asyncio.sleep(5)
```

Lines starting with `:` are SSE comments. The browser ignores them, but they keep the connection alive through proxies.

---

10. Exercise: Build a Live Metrics Dashboard

Build a dashboard that streams server metrics to the browser.

Requirements

- SSE endpoint at `GET /events/server-metrics` that streams every 3 seconds:
- Current timestamp
- A random CPU percentage (simulate with `random.randint(10, 90)`)
- A random memory percentage
- A random request count
- HTML page at `GET /dashboard` that:
- Connects to the SSE endpoint
- Displays the four metrics in cards
- Updates the values in real time (no page refresh)
- Shows a "Connected" / "Reconnecting..." status indicator

Test by:

- Open `http://localhost:7146/dashboard`
- Watch the numbers update every 3 seconds
- Stop the server and verify "Reconnecting..." appears
- Restart the server and verify it reconnects and resumes updating

---

11. Solution

Create `src/routes/server_metrics.py`:

```
import asyncio
import json
import random
from datetime import datetime, timezone
from tina4_python import get

@get("/events/server-metrics")
async def server_metrics_stream(request, response):
    async def generate():
        yield "retry: 3000\n\n"
        counter = 0
        while True:
            yield f"id: {counter}\ndata: {json.dumps({
                'timestamp': datetime.now(timezone.utc).isoformat(),
                'cpu': random.randint(10, 90),
                'memory': random.randint(30, 85),
                'requests': random.randint(100, 5000),
            })}\n\n"
            counter += 1
```

```
            await asyncio.sleep(3)

    return response.stream(generate())


@get("/dashboard")
async def dashboard_page(request, response):
    html = """
    <!DOCTYPE html>
    <html>
    <head><title>Live Dashboard</title></head>
    <body style="font-family: system-ui; max-width: 600px; margin: 40px auto; padding: 0 20px;">
        <h1>Server Metrics</h1>
        <p id="status" style="color: #888;">Connecting...</p>
        <div style="display: grid; grid-template-columns: 1fr 1fr; gap: 16px; margin-top: 20px;"
            <div style="background: #f5f5f5; padding: 20px; border-radius: 8px;">
                <div style="color: #888; font-size: 14px;">CPU</div>
                <div id="cpu" style="font-size: 32px; font-weight: 700;">--</div>
            </div>
            <div style="background: #f5f5f5; padding: 20px; border-radius: 8px;">
                <div style="color: #888; font-size: 14px;">Memory</div>
                <div id="memory" style="font-size: 32px; font-weight: 700;">--</div>
            </div>
            <div style="background: #f5f5f5; padding: 20px; border-radius: 8px;">
                <div style="color: #888; font-size: 14px;">Requests/min</div>
                <div id="requests" style="font-size: 32px; font-weight: 700;">--</div>
            </div>
            <div style="background: #f5f5f5; padding: 20px; border-radius: 8px;">
                <div style="color: #888; font-size: 14px;">Last Update</div>
                <div id="time" style="font-size: 16px; font-weight: 500;">--</div>
            </div>
        </div>
        <script>
            const source = new EventSource("/events/server-metrics");
            const status = document.getElementById("status");

            source.onopen = function () {
                status.textContent = "Connected";
                status.style.color = "#22c55e";
            };

            source.onmessage = function (event) {
                const data = JSON.parse(event.data);
                document.getElementById("cpu").textContent = data.cpu + "%";
                document.getElementById("memory").textContent = data.memory + "%";
                document.getElementById("requests").textContent = data.requests.toLocaleString()
                document.getElementById("time").textContent = new Date(data.timestamp).toLocaleT
            };

            source.onerror = function () {
                status.textContent = "Reconnecting...";
                status.style.color = "#ef4444";
            };
        </script>
    </body>
    </html>
    """
    return response(html)
```

Open `http://localhost:7146/dashboard`. The numbers update every three seconds. Stop the server. The status turns red: "Reconnecting..." Start it again. The status turns green. The numbers resume.

No polling. No WebSocket. No JavaScript timers. The browser handles everything.

---

12. What You Learned

- **SSE streams data from server to client** over a single HTTP connection. No upgrade. No special protocol.
- `response.stream(generator)` keeps the connection open and flushes each chunk as the generator yields it.
- **The SSE protocol** uses `data:`, `event:`, `id:`, and `retry:` fields. Messages end with `\n\n`.
- `EventSource` on the client handles connection, parsing, and automatic reconnection.
- **Named events** let you multiplex different data types on one stream.
- **Custom content types** let you stream NDJSON, binary, or any format.
- **SSE is for one-way server pushes.** Use WebSocket when the client needs to send data back on the same connection.
- **Heartbeat comments** keep connections alive through proxies. Tina4 sets `X-Accel-Buffering: no` to disable nginx buffering.

One direction. One connection. Real-time data. The server speaks. The client listens. The rest is infrastructure.

WSDL / SOAP

1. SOAP Is Not Dead

Banks. Governments. Insurance companies. Logistics providers. Decades of enterprise software runs on SOAP. If you integrate with any of them, you speak SOAP. Tina4 makes it bearable.

The `WSDL` class exposes your Python functions as SOAP 1.1 operations. It auto-generates the WSDL document at `?wsdl`. You write normal Python functions with type annotations. Tina4 handles the XML envelope, the SOAP headers, and the schema generation.

---

2. Your First SOAP Service

Tina4's WSDL service is a class you subclass. Methods marked with `@wsdl_operation` become SOAP operations. Each operation declares its response schema as a dict mapping field names to Python types.

```
from tina4_python.wsdl import WSDL, wsdl_operation
from tina4_python.core.router import get, post


class Calculator(WSDL):
    @wsdl_operation({"Result": int})
    def Add(self, a: int, b: int):
        return {"Result": a + b}

    @wsdl_operation({"Result": int})
    def Multiply(self, a: int, b: int):
        return {"Result": a * b}
```

Mount it on a route. The same handler answers both the WSDL definition and SOAP invocations: `Calculator(request).handle()` inspects the request and returns the right thing.

```
@get("/api/soap/calculator")
@post("/api/soap/calculator")
async def calculator(request, response):
    service = Calculator(request)
    return response(service.handle())
```

Visit `http://localhost:7146/api/soap/calculator?wsdl` to see the generated WSDL document. POST a SOAP envelope to the same URL to invoke an operation.

---

3. The @wsdl_operation Decorator

`@wsdl_operation` marks a method on a `WSDL` subclass as a SOAP operation. The single argument is the response schema: a dict mapping each output field name to its Python type.

```
class PriceService(WSDL):
    @wsdl_operation({"Price": float, "Currency": str})
    def GetProductPrice(self, product_id: str, currency: str):
```

```
        prices = {"USD": 79.99, "EUR": 73.99, "GBP": 62.99}
        return {"Price": prices.get(currency, prices["USD"]), "Currency": currency}
```

Method-parameter type annotations become the request-message schema. The operation name is the method name. Supported types: `str`, `int`, `float`, `bool`, plus `List[T]` and `Optional[T]` for collections and nullable fields.

---

4. Auto WSDL Generation at ?wsdl

Tina4 generates the WSDL document from your type annotations. When a SOAP client sends a GET request with the `?wsdl` query string, the handler returns the XML document automatically.

```
curl "http://localhost:7146/api/soap/calculator?wsdl"
```

```
<?xml version="1.0" encoding="UTF-8"?>
<definitions name="CalculatorService"
  targetNamespace="http://example.com/calculator"
  xmlns="http://schemas.xmlsoap.org/wsdl/"
  xmlns:soap="http://schemas.xmlsoap.org/wsdl/soap/"
  xmlns:tns="http://example.com/calculator"
  xmlns:xsd="http://www.w3.org/2001/XMLSchema">

  <types>
    <xsd:schema targetNamespace="http://example.com/calculator">
      <xsd:element name="AddRequest">
        <xsd:complexType>
          <xsd:sequence>
            <xsd:element name="a" type="xsd:int"/>
            <xsd:element name="b" type="xsd:int"/>
          </xsd:sequence>
        </xsd:complexType>
      </xsd:element>
      <xsd:element name="AddResponse">
        <xsd:complexType>
          <xsd:sequence>
            <xsd:element name="result" type="xsd:int"/>
          </xsd:sequence>
        </xsd:complexType>
      </xsd:element>
      <!-- Multiply types... -->
    </xsd:schema>
  </types>

  <!-- Bindings, ports, and service definition... -->
</definitions>
```

Most SOAP clients (SOAPUI, Java's JAX-WS, .NET's `wsdl.exe`) can import this URL directly and generate client stubs.

---

5. Calling the Service

A raw SOAP request with curl:

```
curl -X POST http://localhost:7146/api/soap/calculator \
  -H "Content-Type: text/xml; charset=utf-8" \
  -H "SOAPAction: \"Add\"" \
  -d '<?xml version="1.0" encoding="utf-8"?>
<soap:Envelope xmlns:soap="http://schemas.xmlsoap.org/soap/envelope/"
               xmlns:tns="http://example.com/calculator">
  <soap:Body>
    <tns:AddRequest>
      <tns:a>15</tns:a>
      <tns:b>27</tns:b>
    </tns:AddRequest>
  </soap:Body>
</soap:Envelope>'
```

Response:

```
<?xml version="1.0" encoding="utf-8"?>
<soap:Envelope xmlns:soap="http://schemas.xmlsoap.org/soap/envelope/">
  <soap:Body>
    <AddResponse xmlns="http://example.com/calculator">
      <result>42</result>
    </AddResponse>
  </soap:Body>
</soap:Envelope>
```

---

6. Lifecycle Hooks: on*request and on*result

Override `on_request` and `on_result` in your subclass to inspect or modify the SOAP envelope before invocation and the result before serialisation.

```
from tina4_python.wsdl import WSDL, wsdl_operation
from tina4_python.debug import Log


class OrderService(WSDL):
    def on_request(self, request):
        """Runs before each operation. Validate auth, audit, etc."""
        Log.info("SOAP request received", path=getattr(request, "path", ""))

    def on_result(self, result):
        """Transform or strip fields before serialisation."""
        Log.info("SOAP operation completed")
        return result

    @wsdl_operation({"OrderId": str})
    def CreateOrder(self, customer_id: str, product_id: str, quantity: int):
        return {"OrderId": f"ORD-{customer_id}-{product_id}"}
```

Use `on_request` for:

- Authentication / API key validation from SOAP headers
- Input sanitisation
- Request logging and audit trails

Use `on_result` for:

- Response transformation
- Stripping internal fields before returning
- Result logging

---

7. Complex Types and Lists

When an operation needs to return structured records or collections, describe the shape in the `@wsdl_operation` response schema. Use `List[T]` for collections and `Optional[T]` for nullable fields.

```
from typing import List, Optional
from tina4_python.wsdl import WSDL, wsdl_operation


class ProductService(WSDL):
    @wsdl_operation({
        "Id": str,
        "Name": str,
        "Price": float,
        "InStock": bool,
        "Error": Optional[str],
    })
    def GetProduct(self, product_id: str):
        catalog = {
            "KB-001": {"Id": "KB-001", "Name": "Wireless Keyboard", "Price": 79.99, "InStock": T
            "HUB-002": {"Id": "HUB-002", "Name": "USB-C Hub", "Price": 49.99, "InStock": False,
        }
        return catalog.get(product_id, {"Id": "", "Name": "", "Price": 0.0, "InStock": False, "E

    @wsdl_operation({"Total": int, "Average": float, "Error": Optional[str]})
    def SumList(self, Numbers: List[int]):
        if not Numbers:
            return {"Total": 0, "Average": 0.0, "Error": "Empty list"}
        return {"Total": sum(Numbers), "Average": sum(Numbers) / len(Numbers), "Error": None}
```

The dict you return must match the declared schema keys. Tina4 generates the WSDL `<complexType>` definitions from the schema dict and the method's parameter annotations.

---

8. Error Handling

Raise a Python exception inside an operation to return a SOAP fault:

```
class OrderService(WSDL):
    @wsdl_operation({"OrderId": str})
    def CreateOrder(self, customer_id: str, product_id: str, quantity: int):
        if quantity <= 0:
            raise ValueError("Quantity must be greater than zero")

        if not check_stock(product_id, quantity):
            raise RuntimeError(f"Insufficient stock for {product_id}")

        return {"OrderId": f"ORD-{customer_id}-{product_id}"}
```

Any raised exception becomes a SOAP fault response:

```
<soap:Body>
  <soap:Fault>
    <faultcode>soap:Server</faultcode>
    <faultstring>Quantity must be greater than zero</faultstring>
  </soap:Fault>
</soap:Body>
```

---

9. Exercise: Build a Currency Conversion SOAP Service

Create a SOAP service that converts amounts between currencies.

Requirements

- Create a `WSDL` instance for a `CurrencyService` at `/api/soap/currency`
- Implement two operations:
- `Convert(amount: float, from_currency: str, to_currency: str) -> float`
- `GetRate(from_currency: str, to_currency: str) -> float`
- Add `on_request` logging
- Raise `ValueError` for unknown currency codes

Test with:

```
# Get the WSDL
curl "http://localhost:7146/api/soap/currency?wsdl"

# Convert $100 USD to EUR
curl -X POST http://localhost:7146/api/soap/currency \
  -H "Content-Type: text/xml" \
  -H "SOAPAction: \"Convert\"" \
  -d '<soap:Envelope xmlns:soap="http://schemas.xmlsoap.org/soap/envelope/"
                     xmlns:tns="http://example.com/currency">
    <soap:Body>
      <tns:ConvertRequest>
        <tns:amount>100.0</tns:amount>
        <tns:from_currency>USD</tns:from_currency>
        <tns:to_currency>EUR</tns:to_currency>
      </tns:ConvertRequest>
    </soap:Body>
  </soap:Envelope>'
```

---

10. Solution

Create `src/routes/currency_soap.py`:

```
from tina4_python.core.router import get, post
from tina4_python.wsdl import WSDL, wsdl_operation
from tina4_python.debug import Log

RATES = {
```

```
    ("USD", "EUR"): 0.92,
    ("USD", "GBP"): 0.79,
    ("USD", "JPY"): 149.50,
    ("EUR", "USD"): 1.09,
    ("EUR", "GBP"): 0.86,
    ("GBP", "USD"): 1.27,
    ("GBP", "EUR"): 1.17,
}
SUPPORTED = {"USD", "EUR", "GBP", "JPY"}


class CurrencyService(WSDL):
    def on_request(self, request):
        Log.info("SOAP currency request received")

    def _lookup_rate(self, from_currency: str, to_currency: str) -> float:
        from_currency = from_currency.upper()
        to_currency = to_currency.upper()

        if from_currency not in SUPPORTED:
            raise ValueError(f"Unknown currency: {from_currency}")
        if to_currency not in SUPPORTED:
            raise ValueError(f"Unknown currency: {to_currency}")
        if from_currency == to_currency:
            return 1.0

        rate = RATES.get((from_currency, to_currency))
        if rate is None:
            raise ValueError(f"No rate available for {from_currency} -> {to_currency}")
        return rate

    @wsdl_operation({"Rate": float})
    def GetRate(self, from_currency: str, to_currency: str):
        return {"Rate": self._lookup_rate(from_currency, to_currency)}

    @wsdl_operation({"Amount": float, "Rate": float})
    def Convert(self, amount: float, from_currency: str, to_currency: str):
        rate = self._lookup_rate(from_currency, to_currency)
        return {"Amount": round(amount * rate, 2), "Rate": rate}


@get("/api/soap/currency")
@post("/api/soap/currency")
async def currency(request, response):
    return response(CurrencyService(request).handle())
```

---

11. Gotchas

1. Missing SOAPAction header

Problem: The SOAP request returns a fault saying the operation was not found.

Fix: Include the `SOAPAction` header in every POST request. Its value must match the operation name, wrapped in quotes: `SOAPAction: "Add"`.

2. Namespace mismatch

Problem: The WSDL is generated with namespace `http://example.com/calculator` but the request body uses a different namespace. The operation is not matched.

Fix: Copy the `targetNamespace` from the WSDL document exactly into the request body's `xmlns:tns` attribute. SOAP is namespace-strict.

3. Type annotations missing

Problem: A parameter with no type annotation causes an error during WSDL generation.

Fix: All parameters and return types must be annotated. Supported types: `str`, `int`, `float`, `bool`, and dataclasses whose fields use those types.

4. Forgetting the ?wsdl route

Problem: The SOAP client cannot import the WSDL because the GET handler is missing.

Fix: Register both a `@get` handler (for `?wsdl`) and a `@post` handler (for SOAP operations) on the same path.

DI Container

1. Stop Constructing Dependencies Inside Your Code

```
@post("/api/orders")
async def create_order(request, response):
    db = Database.get_connection()      # tight coupling
    mailer = Messenger()                # constructed inline
    payments = PaymentGateway(          # hardcoded config
        api_key=os.environ["PAYMENT_KEY"]
    )
    ...
```

Every route constructs its own dependencies. Testing requires monkey-patching or mocking at the module level. Swapping the payment gateway means touching every route that uses it.

A DI container moves construction out of the route. You register services once, at startup. Routes ask the container for what they need by name.

---

2. The Container Class

```
from tina4_python.container import Container

container = Container()
```

The `Container` is a lightweight registry. It does not scan files, read XML, or require annotations on your classes. You register explicitly. You retrieve explicitly.

---

3. register() - Transient Services

A transient service creates a new instance every time `get()` is called:

```
from tina4_python.container import Container

container = Container()

# Register a factory function
container.register("logger", lambda: Logger(level="INFO"))

# Each call creates a fresh instance
logger1 = container.get("logger")
logger2 = container.get("logger")

assert logger1 is not logger2  # Different objects
```

Use transient registration for services that must not share state: request-scoped objects, disposable HTTP clients, or mock implementations in tests.

---

4. singleton() - Cached Services

A singleton creates the instance once and returns the same object on every subsequent call:

```
import os
from tina4_python.container import Container
from tina4_python.api import Api

container = Container()

# Register a singleton factory
container.singleton("payment_gateway", lambda: Api(
    bearer_token=os.environ["PAYMENT_API_KEY"],
    timeout=15
))

# Both calls return the exact same object
gw1 = container.get("payment_gateway")
gw2 = container.get("payment_gateway")

assert gw1 is gw2  # Same object
```

Use singleton registration for: database connections, HTTP clients, configuration objects, external service adapters.

---

5. get() and has()

```
# Retrieve a service
service = container.get("payment_gateway")

# Check before retrieving
if container.has("email_service"):
    mailer = container.get("email_service")
    mailer.send(...)
```

`get()` raises `KeyError` if the service is not registered. `has()` lets you check first, or use it to provide a default:

```
mailer = container.get("email_service") if container.has("email_service") else None
```

---

6. reset()

Clear the singleton cache without removing registrations. Useful in tests to force fresh construction:

```
container.singleton("db", lambda: Database.connect())

# In tests: reset between test cases so each gets a clean DB
container.reset()

db = container.get("db")  # New connection created
```

`reset()` clears all cached singletons. Transient registrations are unaffected (they never cache). Call `reset()` in test teardown to prevent state from leaking between tests.

---

7. Building a Service Container for Your App

Define your container in one place:

```
# src/services.py
import os
from tina4_python.container import Container
from tina4_python.api import Api

container = Container()

# Database connection (singleton -- one connection reused)
container.singleton("db", lambda: Database.get_connection())

# Payment gateway (singleton -- one client, shared config)
container.singleton("payments", lambda: Api(
    bearer_token=os.environ["PAYMENT_API_KEY"],
    timeout=15
))

# Email service (singleton)
container.singleton("mailer", lambda: Api(
    bearer_token=os.environ["SENDGRID_API_KEY"]
))

# Request logger (transient -- new instance per use)
container.register("request_logger", lambda: RequestLogger(
    level=os.environ.get("LOG_LEVEL", "INFO")
))
```

Import `container` wherever you need it:

```
# src/routes/orders.py
from src.services import container

@post("/api/orders")
async def create_order(request, response):
    payments = container.get("payments")
    mailer = container.get("mailer")
    ...
```

---

8. Testing with the Container

Swap implementations for tests without changing any route code:

```
# tests/test_orders.py
import pytest
from src.services import container

class MockPaymentGateway:
    def post(self, url, body):
```

```
        return {
            "http_code": 200,
            "body": {"charge_id": "ch_test_001", "status": "succeeded"},
            "headers": {},
            "error": None
        }

class MockMailer:
    def __init__(self):
        self.sent = []

    def post(self, url, body):
        self.sent.append(body)
        return {"http_code": 200, "body": {"id": "msg_001"}, "headers": {}, "error": None}


@pytest.fixture(autouse=True)
def reset_container():
    # Clear singleton cache before each test
    container.reset()
    yield
    container.reset()


def test_create_order_success(client):
    mock_mailer = MockMailer()

    # Override with test doubles
    container.singleton("payments", lambda: MockPaymentGateway())
    container.singleton("mailer", lambda: mock_mailer)

    resp = client.post("/api/orders", json={
        "customer_id": 1,
        "items": [{"product_id": "KB-001", "quantity": 1, "price": 79.99}],
        "email": "alice@example.com"
    })

    assert resp.status_code == 201
    assert resp.json()["order_id"] is not None
    assert len(mock_mailer.sent) == 1  # Confirmation email was sent
```

The route code never changes. The container is the only seam.

---

9. Full Example: Order Route with DI

```
# src/routes/orders_di.py
from tina4_python.core.router import post
from tina4_python.debug import Log
from src.services import container


@post("/api/di/orders")
async def create_order_di(request, response):
    body = request.body

    if not body.get("customer_id") or not body.get("items"):
        return response({"error": "customer_id and items are required"}, 400)

    payments = container.get("payments")
    mailer = container.get("mailer")

    total = round(sum(item.get("price", 0) * item.get("quantity", 1) for item in body["items"]),

    # Charge payment
    charge_result = payments.post("https://api.payment-provider.com/v1/charges", {
        "amount": int(total * 100),
        "currency": "usd"
    })

    if charge_result["error"] or charge_result["http_code"] not in (200, 201):
        Log.error("Payment failed", customer_id=body["customer_id"], total=total)
        return response({"error": "Payment failed"}, 402)

    charge_id = charge_result["body"]["charge_id"]
    order_id = f"ORD-{body['customer_id']}-{charge_id}"

    # Send confirmation
    mailer.post("https://api.sendgrid.com/v3/mail/send", {
        "to": body.get("email"),
        "subject": f"Order Confirmation {order_id}",
        "body": f"Your order of ${total} has been confirmed."
    })

    Log.info("Order created", order_id=order_id, total=total)

    return response({
        "order_id": order_id,
        "total": total,
        "charge_id": charge_id
    }, 201)
```

---

10. Exercise: Configurable Notification Service

Build a notification service that can switch between email and SMS providers via the container.

Requirements

- Define two notifier classes: `EmailNotifier` and `SmsNotifier`, each with a `send(to, message)` method

- Register the active notifier as `"notifier"` in the container (controlled by a `NOTIFIER` env var)
- Create `POST /api/notify` that retrieves `"notifier"` from the container and sends a message
- Write a test that registers a mock notifier and verifies messages are sent

Test with:

```
# Set NOTIFIER=email or NOTIFIER=sms in .env
curl -X POST http://localhost:7146/api/notify \
  -H "Content-Type: application/json" \
  -d '{"to": "alice@example.com", "message": "Your order has shipped!"}'
```

---

11. Solution

```
# src/notifiers.py
import os

class EmailNotifier:
    def send(self, to, message):
        print(f"[EMAIL] To: {to} | Message: {message}")
        return {"channel": "email", "to": to}

class SmsNotifier:
    def send(self, to, message):
        print(f"[SMS] To: {to} | Message: {message}")
        return {"channel": "sms", "to": to}

def make_notifier():
    channel = os.environ.get("NOTIFIER", "email")
    if channel == "sms":
        return SmsNotifier()
    return EmailNotifier()
```

```
# src/services.py (additions)
from src.notifiers import make_notifier
container.singleton("notifier", make_notifier)
```

```
# src/routes/notify.py
from tina4_python.core.router import post
from src.services import container

@post("/api/notify")
async def notify(request, response):
    body = request.body

    if not body.get("to") or not body.get("message"):
        return response({"error": "to and message are required"}, 400)

    notifier = container.get("notifier")
    result = notifier.send(body["to"], body["message"])

    return response({"sent": True, **result})
```

```
# tests/test_notify.py
from src.services import container
```

```
class MockNotifier:
    def __init__(self):
        self.messages = []
    def send(self, to, message):
        self.messages.append({"to": to, "message": message})
        return {"channel": "mock", "to": to}

def test_notify_sends_message(client):
    mock = MockNotifier()
    container.reset()
    container.singleton("notifier", lambda: mock)

    resp = client.post("/api/notify", json={"to": "bob@example.com", "message": "Hello!"})

    assert resp.status_code == 200
    assert mock.messages[0]["to"] == "bob@example.com"
    container.reset()
```

---

12. Gotchas

1. Mutable singleton shared across requests

Problem: A singleton holds per-request state and leaks it between concurrent requests.

Fix: Use `register()` (transient) for anything that holds request-specific state. Singletons should be stateless or thread-safe.

2. reset() removes singleton cache but not registrations

Problem: After `container.reset()`, calling `container.has("payments")` still returns `True` but the next `get()` creates a new instance.

Fix: This is correct behaviour. `reset()` only clears the cache. The factory function is still registered and will run again on the next `get()`. Use `reset()` deliberately in test teardown.

3. Circular dependencies

Problem: Service A's factory calls `container.get("B")` and B's factory calls `container.get("A")`. Both hang or raise a recursion error.

Fix: Restructure so that dependencies are one-directional. Extract shared logic into a third service that neither A nor B depends back upon.

4. Importing container before .env is loaded

Problem: `container.singleton("payments", lambda: Api(bearer_token=os.environ["PAYMENT_KEY"]))` raises `KeyError` because `.env` has not been loaded yet.

Fix: Tina4 loads `.env` before importing route files. If you initialise the container in a module that is imported before startup, use a lambda (lazy factory) so `os.environ` is not accessed until `get()` is called.

Service Runner

1. Work That Never Stops

Some tasks run on a schedule. Some run indefinitely. A queue worker that drains emails. A cache warmer that refreshes product data every five minutes. These are background services: long-lived processes that run alongside the web server but are not tied to any HTTP request.

Tina4's service runner gives these tasks a consistent start/stop lifecycle. Register services. Start them. Stop them gracefully. They run in background threads so they do not block the web server.

---

2. A Basic Background Service

A service is any class with a `run()` method. Start it with the runner and it executes in the background:

```
import time
from tina4_python.service import ServiceRunner

class HeartbeatService:
    def __init__(self, interval=30):
        self.interval = interval
        self.running = False

    def run(self):
        self.running = True
        while self.running:
            print(f"Heartbeat at {time.strftime('%H:%M:%S')}")
            time.sleep(self.interval)

    def stop(self):
        self.running = False


runner = ServiceRunner()
runner.register("heartbeat", HeartbeatService(interval=30))
runner.start()
```

`runner.start()` launches all registered services in background threads and returns immediately. The web server keeps handling requests. The heartbeat ticks every 30 seconds.

---

3. Start / Stop Lifecycle

Every service implements `run()` and `stop()`. The runner calls `run()` in a thread when started and `stop()` when shutting down.

```
import threading
import time
from tina4_python.service import ServiceRunner

class PollingService:
```

```
    def __init__(self, poll_url, interval=60):
        self.poll_url = poll_url
        self.interval = interval
        self._running = threading.Event()

    def run(self):
        self._running.set()
        while self._running.is_set():
            try:
                self._poll()
            except Exception as exc:
                print(f"Poll failed: {exc}")

            # Wait for interval or until stop() clears the event
            self._running.wait(timeout=self.interval)

    def stop(self):
        self._running.clear()

    def _poll(self):
        from tina4_python.api import Api
        api = Api(timeout=10)
        result = api.get(self.poll_url)
        if result["http_code"] == 200:
            print(f"Poll ok: {result['body']}")
        else:
            print(f"Poll error: {result['http_code']}")
```

Using `threading.Event` instead of a boolean for the stop signal is safer: `wait(timeout=n)` wakes up immediately when the event is cleared, so `stop()` takes effect within milliseconds rather than waiting for the next sleep to expire.

---

4. Queue Worker Service

The most common background service is a queue worker:

```
import time
from tina4_python.service import ServiceRunner
from tina4_python.queue import Queue
from tina4_python.debug import Log


class EmailWorker:
    def __init__(self):
        self.queue = Queue(topic="emails", max_retries=3)
        self._running = False

    def run(self):
        self._running = True
        Log.info("Email worker started")

        while self._running:
            job = self.queue.pop()

            if job is None:
                # No pending jobs -- wait briefly before checking again
```

```
                time.sleep(1)
                continue

            try:
                self._send_email(job.payload)
                job.complete()
                Log.info("Email sent", to=job.payload["to"])
            except Exception as exc:
                job.fail(str(exc))
                Log.warning("Email failed", to=job.payload.get("to"), error=str(exc))

    def stop(self):
        self._running = False
        Log.info("Email worker stopped")

    def _send_email(self, payload):
        # Replace with real email logic
        import time
        time.sleep(0.1)  # Simulate sending
        if payload.get("to") == "bounce@example.com":
            raise RuntimeError("Recipient address rejected")


# Register and start
runner = ServiceRunner()
runner.register("email_worker", EmailWorker())
runner.start()
```

---

5. Scheduled Task Service

Run a function on a fixed schedule:

```
import time
import threading
from tina4_python.service import ServiceRunner
from tina4_python.debug import Log


class ScheduledTask:
    def __init__(self, name, task_fn, interval_seconds):
        self.name = name
        self.task_fn = task_fn
        self.interval = interval_seconds
        self._stop_event = threading.Event()

    def run(self):
        Log.info("Scheduled task started", task=self.name, interval=self.interval)
        while not self._stop_event.is_set():
            try:
                self.task_fn()
            except Exception as exc:
                Log.error("Scheduled task failed", task=self.name, error=str(exc))
            self._stop_event.wait(timeout=self.interval)

    def stop(self):
        self._stop_event.set()
        Log.info("Scheduled task stopped", task=self.name)
```

```
def warm_cache():
    from tina4_python.cache import cache_set
    Log.debug("Cache warming started")
    # Fetch expensive data and pre-populate the cache
    cache_set("featured_products", fetch_featured_products(), ttl=300)
    Log.debug("Cache warming complete")

def fetch_featured_products():
    return [
        {"id": 1, "name": "Wireless Keyboard", "price": 79.99},
        {"id": 2, "name": "USB-C Hub", "price": 49.99}
    ]


runner = ServiceRunner()
runner.register("cache_warmer", ScheduledTask("cache_warmer", warm_cache, interval_seconds=240))
runner.start()
```

---

6. Multiple Services

Register as many services as you need:

```
from tina4_python.service import ServiceRunner

runner = ServiceRunner()
runner.register("email_worker", EmailWorker())
runner.register("sms_worker", SmsWorker())
runner.register("cache_warmer", ScheduledTask("cache_warmer", warm_cache, 240))
runner.register("health_check", PollingService("https://api.internal/health", 30))

runner.start()
print("All services started")
```

All four services start concurrently. Each runs in its own thread.

---

7. Stopping Services Gracefully

Stop all services at once:

```
runner.stop()
```

`stop()` calls `stop()` on each registered service and waits for their threads to finish. Use this in your application shutdown handler:

```
import signal

def shutdown_handler(signum, frame):
    print("Shutting down services...")
    runner.stop()
    print("All services stopped.")

signal.signal(signal.SIGTERM, shutdown_handler)
signal.signal(signal.SIGINT, shutdown_handler)
```

Stop a single service by name:

```
runner.stop_service("cache_warmer")
```

---

8. Integrating with the App

Start the runner in your `src/app.py`:

```
# src/app.py
import atexit
from src.worker_services import runner

runner.start()

# ServiceRunner does not stop itself, so hand it to atexit. The interpreter
# runs this on a clean exit and on SIGTERM, which is what an orchestrator
# sends before it stops the container.
atexit.register(runner.stop)
```

There is no app object and no lifecycle decorator. Tina4 for Python registers routes with bare `@get` and `@post`, and `src/app.py` runs at import time, so starting the runner is a plain call. `runner.start()` launches every registered service in a background thread and returns, which leaves the web server free to carry on serving.

---

9. Exercise: Cache-Warming Background Service

Build a background service that keeps a product cache warm.

Requirements

- Create a `ProductCacheWarmer` service that:
- Runs every 60 seconds
- Fetches the top 20 products
- Stores them in cache with a 120-second TTL
- Logs a summary each time it runs
- Create a `GET /api/products/featured` route that:
- Returns products from cache (never hits the "database" directly)
- Returns a 503 if the cache is not yet warm
- Register the service with the runner and start it on app startup

Test with:

```
# Before the cache warms (first 60 seconds)
curl http://localhost:7146/api/products/featured
# 503 Service Unavailable -- cache not ready

# After the cache warmer runs
curl http://localhost:7146/api/products/featured
# 200 OK with product list
```

---

10. Solution

```
# src/workers/product_cache_warmer.py
import threading
from tina4_python.cache import cache_set
from tina4_python.debug import Log


FEATURED_PRODUCTS = [
    {"id": i, "name": f"Product {i}", "price": round(9.99 * i, 2)}
    for i in range(1, 21)
]


class ProductCacheWarmer:
    def __init__(self, interval=60):
        self.interval = interval
        self._stop_event = threading.Event()

    def run(self):
        Log.info("ProductCacheWarmer started", interval=self.interval)
        while not self._stop_event.is_set():
            self._warm()
            self._stop_event.wait(timeout=self.interval)

    def stop(self):
        self._stop_event.set()
        Log.info("ProductCacheWarmer stopped")

    def _warm(self):
        products = FEATURED_PRODUCTS[:20]
        cache_set("products:featured", products, ttl=120)
        Log.info("Cache warmed", product_count=len(products))
```

```
# src/routes/featured_products.py
from tina4_python.core.router import get
from tina4_python.cache import cache_get


@get("/api/products/featured")
async def featured_products(request, response):
    products = cache_get("products:featured")

    if products is None:
        return response(
            {"error": "Service warming up, please retry in a moment"},
            503
```

```
        )

    return response({"products": products, "count": len(products)})
```

```
# src/app.py (startup integration)
from tina4_python.service import ServiceRunner
from src.workers.product_cache_warmer import ProductCacheWarmer

runner = ServiceRunner()
runner.register("product_cache_warmer", ProductCacheWarmer(interval=60))
runner.start()
```

---

11. Gotchas

1. Blocking the main thread

Problem: `runner.start()` blocks because a service's `run()` calls `runner.start()` again recursively, or a service does not return from `run()`.

Fix: `run()` must contain the loop internally. `runner.start()` is non-blocking; it launches threads and returns. Each service owns its own loop.

2. No stop() method

Problem: `runner.stop()` raises `AttributeError` because the service class does not implement `stop()`.

Fix: Always implement `stop()`. Set a flag or clear a threading event that the `run()` loop checks.

3. Exceptions crashing the service thread

Problem: An unhandled exception in `run()` kills the thread. The service stops silently.

Fix: Wrap the loop body in `try/except`. Log the error and continue. Only exit the loop on a stop signal.

4. Service starts before .env is loaded

Problem: A service reads an environment variable at construction time, before Tina4 loads `.env`.

Fix: Read environment variables inside `run()` or `_warm()`, not in `__init__`. Or use a factory pattern: construct the service inside an `on_startup` hook, after `.env` is loaded.

MCP Dev Tools

1. What is MCP?

The Model Context Protocol connects AI coding tools to your running application. Claude Code, Cursor, and other assistants speak this protocol. When they connect, they see your database, routes, templates, and files -- not as static text, but as live, queryable tools.

Tina4 ships a built-in MCP server. It starts automatically in dev mode. No configuration needed.

---

2. How It Works

Set `TINA4_DEBUG=true` in your `.env` file and start the server:

```
tina4 serve
```

The console prints:

```
MCP server available at http://localhost:7146/__dev/mcp
```

Tina4 writes the connection details to `.claude/settings.json` automatically. Claude Code discovers the server on its next restart. Cursor reads the same file.

Localhost-Only Security

The built-in MCP server activates only when two conditions hold:

- `TINA4_DEBUG=true`
- The server runs on `localhost`, `127.0.0.1`, or `0.0.0.0`

Deploy to a remote server and the MCP endpoint vanishes -- even with debug enabled. This prevents accidental exposure of database queries and file editing in production.

To enable on a staging server (rare, intentional), set:

```
TINA4_MCP_REMOTE=true
```

---

3. Connecting AI Tools

Claude Code

Tina4 auto-generates `.claude/settings.json` with the current Streamable HTTP transport:

```
{
  "mcpServers": {
    "tina4-dev": {
      "type": "http",
      "url": "http://localhost:7146/__dev/mcp"
    }
  }
}
```

Restart Claude Code. It connects and lists the tools.

Prefer the command line? Register the same server in one step:

```
claude mcp add --transport http tina4-dev http://localhost:7146/__dev/mcp
```

Cursor

Copy the same MCP config into Cursor's settings. The `type` and `url` are identical.

Manual Connection

Any modern MCP client talks to a single endpoint:

- **Streamable HTTP**: `POST http://localhost:7146/__dev/mcp` -- send JSON-RPC 2.0 and read the response inline. `initialize` returns an `Mcp-Session-Id` header; send it back on later calls. `DELETE` on the same URL ends the session.

Older clients that speak the 2024-11-05 HTTP+SSE transport keep working:

- **SSE handshake** (legacy): `GET http://localhost:7146/__dev/mcp/sse` -- returns the message endpoint URL
- **Message endpoint** (legacy): `POST http://localhost:7146/__dev/mcp/message` -- accepts JSON-RPC 2.0 messages

---

4. Built-in Tools

The MCP server exposes 48 tools organized by category. The exact list lives in `tina4_python/mcp/tools.py` -- the registration block at the bottom of that file is authoritative.

Database

Tool	Description
`database_query`	Execute a read-only SQL query (SELECT)
`database_execute`	Execute arbitrary SQL (INSERT, UPDATE, DELETE, DDL)
`database_tables`	List all tables in the database
`database_columns`	Get column definitions for a specific table

Example -- an AI assistant queries your database directly:

```
Tool: database_query
Arguments: {"sql": "SELECT id, name, email FROM users WHERE active = 1"}
```

The `database_execute` tool handles write operations. It commits automatically after each statement.

Routes

Tool	Description
`route_list`	List all registered routes with methods and auth status
`route_test`	Call a route and return status, body, and headers
`swagger_spec`	Return the full OpenAPI 3.0.3 specification

The `route_test` tool lets an AI assistant verify endpoints without leaving the conversation:

```
Tool: route_test
Arguments: {"method": "GET", "path": "/api/users"}
```

Templates

Tool	Description
`template_render`	Render a Frond template string with provided data

Useful for testing template snippets:

```
Tool: template_render
Arguments: {"template": "Hello {{ name }}!", "data": "{\"name\": \"Alice\"}"}
```

Files

Tool	Description
`file_read`	Read a project file (relative to project root)
`file_write`	Write or update a project file (full overwrite)
`file_patch`	Targeted edit -- replace `old_string` with `new_string` in a file
`file_list`	List files in a directory
`asset_upload`	Upload a file to `src/public/`

File operations are sandboxed. Paths that escape the project directory are rejected. An AI assistant cannot read `/etc/passwd` or write outside your project.

Migrations

Tool	Description
`migration_status`	List pending and completed migrations
`migration_create`	Create a new migration file
`migration_run`	Run all pending migrations

Data and ORM

Tool	Description
`orm_describe`	List all ORM models with fields, types, and relationships
`seed_table`	Seed a table with fake data

Infrastructure

Tool	Description
`queue_status`	Queue size by status (pending, completed, failed)
`session_list`	Active sessions with data
`cache_stats`	Response cache hit/miss statistics

Debugging

Tool	Description
`log_tail`	Read recent log entries
`error_log`	Recent errors and exceptions with stack traces
`env_list`	Environment variables (sensitive values redacted)
`system_info`	Framework version, Python version, project info

The `env_list` tool redacts any variable containing "secret", "password", "token", "key", or "credential" in its name.

Project introspection

Tool	Description
`git_status`	Show current branch, modified/untracked files, and recent commits
`deps_list`	List the project's declared Python dependencies (from `pyproject.toml`)
`project_overview`	One-shot snapshot: dependencies, routes, models, tables, errors, git

Persistent code index

Tool	Description
`index_rebuild`	Refresh the persistent project index (lazy, mtime-based)
`index_search`	Find files by path, symbol, route, or summary -- use FIRST for "where is X"
`index_file`	Full index entry for one file: symbols, routes, imports
`index_overview`	Project shape: files by language, routes, models, recent edits

Framework documentation (prose)

Tool	Description
`docs_list`	List framework documentation files
`docs_search`	Search Tina4 framework docs for a query string -- use before guessing
`docs_section`	Return a full markdown section from a framework doc file

Live API RAG (reflection)

`api_*` tools query the live framework reflection -- actual class/method signatures, file/line locations, with a `framework` vs `user` source tag. Use these for "what's the signature of X?" questions; `docs_*` is for "explain queues conceptually".

Tool	Description
`api_search`	Live API search -- finds framework + user classes/methods by name/summary

`api_class`	Live class reflection -- returns full method list and signatures for an FQN
`api_method`	Live method reflection -- returns signature, params, return type, file, line

Plan management

The `plan_*` family lets an AI assistant cooperate with a markdown plan file in `plan/`. Tina4 uses these to keep a steady record of in-progress refactors and multi-session work.

Tool	Description
`plan_current`	The active plan: title, steps (done/not), next step, progress
`plan_list`	All plans in `plan/` with progress and which one is active
`plan_create`	Create a new markdown plan and make it active
`plan_switch_to`	Make a different plan the active one
`plan_complete_step`	Tick a step as done (call the moment the step finishes)
`plan_add_step`	Append a new unchecked step to the current plan
`plan_note`	Append a timestamped note/breadcrumb to the current plan
`plan_archive`	Move a finished plan to `plan/done/` and clear the current pointer
`plan_read`	Full structured view of any plan by filename
`plan_flesh`	Auto-generate concrete build steps via qwen and append them to a plan

---

5. Protocol Details

The MCP server speaks JSON-RPC 2.0 over the current Streamable HTTP transport (protocol versions `2025-06-18` and `2025-03-26`). It still answers the legacy `2024-11-05` HTTP+SSE transport so older clients keep working. The server negotiates: it echoes the version a client asks for when it can speak it, and otherwise falls back to the newest one.

Streamable HTTP (current)

One endpoint carries the whole conversation:

```
POST /__dev/mcp
Content-Type: application/json
```

`initialize` mints a session and returns it in a header:

```
Mcp-Session-Id: 0f9a...c31
```

Send that header on every later request. A request bearing an unknown session gets a `404`, the client's cue to initialize again. A notification (a message with no `id`) returns `202` with an empty body. Every other request answers inline with the JSON-RPC response as `application/json`. `GET /__dev/mcp` returns `405` with an `Allow: POST, DELETE` header; `DELETE /__dev/mcp` ends the session.

Legacy HTTP+SSE (still supported)

```
GET /__dev/mcp/sse
Content-Type: text/event-stream
```

Returns the message endpoint URL as a server-sent event:

```
event: endpoint
data: http://localhost:7146/__dev/mcp/message
```

Then POST JSON-RPC messages to that endpoint:

```
POST /__dev/mcp/message
Content-Type: application/json
```

Messages

Both transports carry the same JSON-RPC 2.0 requests:

```
{
  "jsonrpc": "2.0",
  "id": 1,
  "method": "tools/list",
  "params": {}
}
```

Returns JSON-RPC 2.0 responses:

```
{
  "jsonrpc": "2.0",
  "id": 1,
  "result": {
    "tools": [
      {"name": "database_query", "description": "Execute a read-only SQL query", "inputSchema":
    ]
  }
}
```

Lifecycle

- Client sends `initialize` -- server responds with capabilities
- Client sends `notifications/initialized` -- server acknowledges

- Client calls `tools/list` -- server returns all registered tools with schemas
- Client calls `tools/call` with tool name and arguments -- server executes and returns results
- Client calls `resources/list` and `resources/read` for data resources

---

6. Troubleshooting

MCP server not starting:

- Check `TINA4_DEBUG=true` in `.env`
- Verify the server is running on localhost (not a remote host)

Claude Code not detecting the server:

- Restart Claude Code after starting the Tina4 server
- Check `.claude/settings.json` exists with the correct URL
- Verify the port matches your server's port

Tools returning errors:

- Database tools require `bind_database(db)` in `app.py`
- File tools are sandboxed to the project directory
- `database_execute` only works on localhost

Remote server, MCP disabled:

- This is intentional. Set `TINA4_MCP_REMOTE=true` only on staging environments you trust
- Never enable MCP on production servers

Building Custom MCP Servers

1. Beyond Dev Tools

Chapter 28 covered the built-in MCP server that ships with Tina4. It exposes framework internals for AI-assisted development. This chapter goes further: you build your own MCP servers that expose your application's business logic.

A CRM system exposes customer lookup. An accounting system exposes invoice queries. A warehouse system exposes inventory checks. Any domain logic that an AI assistant should access becomes an MCP tool.

---

2. Creating an MCP Server

Import `McpServer` and create an instance on any path:

```
from tina4_python.mcp import McpServer

mcp = McpServer("/api/my-tools", name="My App Tools", version="1.0.0")
```

The server registers HTTP endpoints at:

- `POST /api/my-tools` -- Streamable HTTP (the current transport): send JSON-RPC and read the response inline. `initialize` returns an `Mcp-Session-Id` header, `DELETE` ends the session.
- `POST /api/my-tools/message` -- legacy JSON-RPC message handler
- `GET /api/my-tools/sse` -- legacy SSE discovery handshake

Register it with the router in `app.py` before `run()`:

```
from tina4_python.core.router import get, post
mcp.register_routes(__import__("tina4_python.core.router", fromlist=["router"]))
```

---

3. Registering Tools with @mcp_tool

The `@mcp_tool` decorator turns a function into an MCP tool. Type hints become the input schema automatically:

```
from tina4_python.mcp import McpServer, mcp_tool

mcp = McpServer("/crm/mcp", name="CRM Tools")

@mcp_tool("lookup_customer", description="Find a customer by email", server=mcp)
def lookup_customer(email: str):
    """Search the customer database by email address."""
    return db.fetch_one("SELECT * FROM customers WHERE email = ?", [email])

@mcp_tool("recent_orders", description="Get recent orders for a customer", server=mcp)
def recent_orders(customer_id: int, limit: int = 10):
```

```
    """Fetch the most recent orders for a customer."""
    result = db.fetch(
        "SELECT * FROM orders WHERE customer_id = ? ORDER BY created_at DESC",
        [customer_id], limit=limit
    )
    return result.to_array()
```

The decorator extracts:

- **Parameter names** from the function signature
- **Types** from type hints (`str` -> `"string"`, `int` -> `"integer"`, etc.)
- **Required vs optional** -- parameters with defaults are optional
- **Description** from the `description` argument or the docstring

An AI assistant sees these tools and their schemas:

```
{
  "name": "lookup_customer",
  "description": "Find a customer by email",
  "inputSchema": {
    "type": "object",
    "properties": {
      "email": {"type": "string"}
    },
    "required": ["email"]
  }
}
```

---

4. Registering Resources with @mcp_resource

Resources are read-only data endpoints. They expose reference data that AI assistants can browse:

```
from tina4_python.mcp import mcp_resource

@mcp_resource("crm://product-catalog", description="All active products", server=mcp)
def product_catalog():
    return db.fetch("SELECT id, name, price, category FROM products WHERE active = 1").to_array(

@mcp_resource("crm://tax-rates", description="Current tax rates by region", server=mcp)
def tax_rates():
    return db.fetch("SELECT region, rate FROM tax_rates").to_array()
```

Resources are accessed via `resources/list` and `resources/read` in the MCP protocol.

---

5. Class-Based MCP Services

Group related tools into a service class. Each method with `@mcp_tool` becomes a tool:

```
from tina4_python.mcp import McpServer, mcp_tool

mcp = McpServer("/accounting/mcp", name="Accounting Tools")
```

```
class AccountingService:
    def __init__(self, db):
        self.db = db

    @mcp_tool("invoice_lookup", description="Find an invoice by number", server=mcp)
    def lookup(self, invoice_no: str):
        return self.db.fetch_one(
            "SELECT * FROM invoices WHERE invoice_no = ?", [invoice_no]
        )

    @mcp_tool("outstanding_balances", description="List all unpaid invoices", server=mcp)
    def balances(self, min_amount: float = 0.0):
        return self.db.fetch(
            "SELECT * FROM invoices WHERE paid = 0 AND total >= ?", [min_amount]
        ).to_array()

    @mcp_tool("monthly_summary", description="Revenue summary for a month", server=mcp)
    def summary(self, year: int, month: int):
        return self.db.fetch_one(
            "SELECT SUM(total) as revenue, COUNT(*) as invoice_count "
            "FROM invoices WHERE strftime('%Y', created_at) = ? "
            "AND strftime('%m', created_at) = ?",
            [str(year), f"{month:02d}"]
        )

# Create the service instance - methods are already registered via decorators
accounting = AccountingService(db)
```

---

6. Securing MCP Endpoints

By default, developer MCP servers are public. Add authentication using the standard Tina4 middleware:

```
from tina4_python.core.router import middleware, secured

# Secure the entire MCP path
@secured()
@middleware(AuthMiddleware)
def register_mcp():
    mcp.register_routes(router)
```

Or check the bearer token inside individual tools:

```
@mcp_tool("sensitive_data", description="Access restricted data", server=mcp)
def sensitive_data(token: str):
    payload = Auth.valid_token_static(token)
    if not payload:
        return {"error": "Unauthorized"}
    return db.fetch("SELECT * FROM sensitive_table").to_array()
```

---

7. Testing MCP Tools

Use `TestClient` to test MCP endpoints without starting a server, or test tool functions directly:

```
# Test the tool function directly
def test_lookup_customer():
    result = lookup_customer("alice@example.com")
    assert result is not None
    assert result["email"] == "alice@example.com"

# Test via MCP protocol
def test_mcp_tool_call():
    import json
    resp = json.loads(mcp.handle_message({
        "jsonrpc": "2.0",
        "id": 1,
        "method": "tools/call",
        "params": {
            "name": "lookup_customer",
            "arguments": {"email": "alice@example.com"}
        }
    }))
    assert "result" in resp
    content = resp["result"]["content"][0]["text"]
    assert "alice" in content.lower()
```

---

8. Complete Example: CRM MCP Server

Here is a full working example -- a CRM system with customer, order, and product tools:

```
# app.py
from tina4_python.core import run
from tina4_python.orm import bind_database
from tina4_python.database import Database
from tina4_python.mcp import McpServer, mcp_tool, mcp_resource

db = Database("sqlite:///crm.db")
bind_database(db)

# Create MCP server
crm_mcp = McpServer("/crm/mcp", name="CRM Assistant", version="1.0.0")

# Tools
@mcp_tool("find_customer", description="Search customers by name or email", server=crm_mcp)
def find_customer(query: str):
    return db.fetch(
        "SELECT * FROM customers WHERE name LIKE ? OR email LIKE ?",
        [f"%{query}%", f"%{query}%"]
    ).to_array()

@mcp_tool("customer_orders", description="Get all orders for a customer", server=crm_mcp)
def customer_orders(customer_id: int):
    return db.fetch(
        "SELECT o.*, GROUP_CONCAT(oi.product_name) as items "
        "FROM orders o LEFT JOIN order_items oi ON o.id = oi.order_id "
        "WHERE o.customer_id = ? GROUP BY o.id ORDER BY o.created_at DESC",
        [customer_id]
    ).to_array()

@mcp_tool("create_note", description="Add a note to a customer record", server=crm_mcp)
def create_note(customer_id: int, note: str):
```

```
    db.insert("customer_notes", {"customer_id": customer_id, "note": note})
    return {"success": True}

# Resources
@mcp_resource("crm://products", description="Product catalog", server=crm_mcp)
def products():
    return db.fetch("SELECT * FROM products WHERE active = 1").to_array()

@mcp_resource("crm://stats", description="CRM statistics", server=crm_mcp)
def stats():
    customers = db.fetch_one("SELECT COUNT(*) as count FROM customers")
    orders = db.fetch_one("SELECT COUNT(*) as count, SUM(total) as revenue FROM orders")
    return {
        "customers": customers["count"],
        "orders": orders["count"],
        "revenue": orders["revenue"],
    }

# Register routes
import tina4_python.core.router as router
crm_mcp.register_routes(router)

if __name__ == "__main__":
    run()
```

Connect Claude Code to `http://localhost:7146/crm/mcp` (Streamable HTTP) and ask:

> *"Find all customers named Smith and show their recent orders"*

The AI calls `find_customer` with `query: "Smith"`, then `customer_orders` for each result. No custom API needed. The MCP protocol handles it.

---

9. Best Practices

- **One server per domain** -- CRM tools on `/crm/mcp`, accounting on `/accounting/mcp`
- **Keep tools focused** -- one query per tool, not a Swiss-army-knife tool
- **Use type hints** -- they become the schema. An AI assistant cannot call a tool correctly without knowing the parameter types
- **Return structured data** -- dicts and lists, not formatted strings. Let the AI format for the user
- **Secure production endpoints** -- use `@secured()` or middleware for any MCP server that runs outside localhost
- **Test tools directly** -- call the Python function in your test suite, not just through the MCP protocol

Dev Tools

1. Debugging at 2am

2am. Production monitoring pings you. A 500 error on checkout. You pull up the dev dashboard. The request inspector shows the failing request. The stack trace points to line 47 of `src/routes/checkout.py` -- an `AttributeError` on the shipping address because the user skipped the form field. You add a None check. Push the fix. Back to sleep. Total time: 30 seconds.

Tina4's dev tools are not an afterthought. They ship with the framework from day one. When `TINA4_DEBUG=true`, you get a development dashboard, an error overlay with source code, live reload, a request inspector with replay, a SQL query runner, a queue monitor, and system info -- all without installing extra packages.

---

2. Enabling the Dev Dashboard

Set `TINA4_DEBUG=true` in your `.env`:

```
TINA4_DEBUG=true
```

Restart your server and navigate to:

```
http://localhost:7146/__dev
```

No token or additional environment variables needed. The dashboard runs only when debug mode is on. Set `TINA4_DEBUG=false` in production and the entire dashboard disappears.

---

3. Dashboard Overview

The dev dashboard has several sections. Navigation tabs run along the top.

System Overview

The landing page shows at a glance:

- **Framework version** -- The installed Tina4 Python version
- **Python version** -- The running Python version and loaded packages
- **Uptime** -- How long the server has been running
- **Memory usage** -- Current and peak memory consumption
- **Database status** -- Connection status, database engine, file size (for SQLite)
- **Environment** -- Current `.env` variables (sensitive values masked)
- **Project structure** -- Directory listing of your project with file counts

Check here first when something feels off. Is the database connected? Is the right Python version running? Are the environment variables loaded?

---

4. The Dev Toolbar

Visit any HTML page in your application. A debug toolbar appears at the bottom of the page. Thin bar. Click it to expand.

The toolbar shows:

Field	What It Means
Request	HTTP method and URL of the current request
Status	HTTP response status code
Time	Total request processing time in milliseconds
DB	Number of database queries executed and total query time
Memory	Python memory used for this request
Template	The template rendered and how long rendering took
Session	Session ID and session data summary
Route	Which route handler matched this request

Click any section to expand it. Clicking "DB" shows every SQL query that ran during the request, with the query text, parameters, execution time, and row count.

Disabling the Toolbar

The toolbar hides when `TINA4_DEBUG=false`. You can also hide it for specific routes by returning a response with the `X-Debug-Toolbar: off` header:

```
from tina4_python.core.router import get

@get("/api/data")
async def api_data(request, response):
    return response({"data": "no toolbar"}, headers={
        "X-Debug-Toolbar": "off"
    })
```

This is useful for API endpoints that return JSON -- the toolbar only matters for HTML pages.

---

5. Error Overlay

An unhandled exception occurs. Tina4 does not show a generic "500 Internal Server Error" page. It shows a detailed error overlay:

- **Exception type and message** -- What went wrong, in plain language

- **Stack trace** -- Every function call that led to the error, from entry point to crash
- **Source code** -- The Python code around the line that threw the exception, with the failing line highlighted
- **Request data** -- HTTP method, URL, headers, body, and query parameters of the request that triggered the error
- **Environment** -- Relevant `.env` variables at the time of the error

Example Error

Write this handler:

```
@get("/api/users/{user_id}")
async def get_user(request, response):
    user_id = request.params["user_id"]
    user = get_user_from_db(user_id)
    return response({"name": user.name, "email": user.email})
```

If `get_user_from_db()` returns `None` for a missing user, the error overlay shows:

```
AttributeError: 'NoneType' object has no attribute 'name'

  File: src/routes/users.py, line 5

  3 | async def get_user(request, response):
  4 |     user_id = request.params["user_id"]
  5 |     user = get_user_from_db(user_id)
> 6 |     return response({"name": user.name, "email": user.email})
  7 |

  Request: GET /api/users/999
  Headers: {"Accept": "application/json"}
```

The highlighted line makes the cause obvious. `user` is `None` and the code accesses `.name` on it.

Error Overlay in Production

When `TINA4_DEBUG=false`, the error overlay is disabled. Users see a clean error page. Full error details go to `logs/error.log` for later investigation.

---

6. The Gallery

The gallery is a collection of ready-to-use code examples built into the dev dashboard. Each gallery item includes:

- A description of what it does
- The complete source code (routes, templates, models)
- A "Try It" button that installs the example into your project

Available gallery items:

- **JWT Authentication** -- Complete login/register flow with token management
- **CRUD API** -- Full REST API with ORM model

- **File Upload** -- Multipart form handling with file storage
- **WebSocket Chat** -- Real-time chat with rooms
- **Email Contact Form** -- Form with Messenger integration
- **Dashboard Template** -- Admin dashboard with tina4css

Click "Try It" on any gallery item. Tina4 creates the necessary files in your project. Modify them to fit your needs.

---

7. Live Reload

When `TINA4_DEBUG=true`, Tina4 watches your project files for changes and refreshes the browser automatically. Edit a route file. Save it. The browser refreshes with the new code. No manual restart required.

```
tina4 serve
```

```
  Tina4 Python v3.11.12
  HTTP server running at http://0.0.0.0:7146
  File watcher active - press Ctrl+C to stop
```

Live reload watches:

- `src/**` -- Routes, ORM, middleware, templates, SCSS
- `migrations/` -- SQL migrations
- `.env` -- Environment variables

How It Works

The `tina4` Rust CLI is the sole file watcher for the whole Tina4 stack: Python, PHP, Ruby, and Node.js all share it. There is no framework-side watcher (the old `dev_reload.py` was removed in 3.11.x).



- The CLI watches `src/`, `migrations/`, `.env` using the `notify` crate. Events are filtered to real source changes: metadata/access events, `__pycache__`, `.git`, `node_modules`, `logs`, `.log`/`.db*`/`.pyc`/`.swp` files are ignored. A real mtime check also defeats overlayfs / polling-mode spurious events (Podman, distrobox).
- On a real change, the CLI POSTs `/__dev/api/reload` to the framework. The server keeps running.
- The framework bumps an in-memory reload counter and broadcasts `{type: "reload"}` over WebSocket at `/__dev_reload`. `GET /__dev/api/mtime` returns the counter for browsers using the polling fallback.
- The dev-toolbar script in your page reloads the browser. SCSS/CSS changes are signalled as `type: "css"` and swap the stylesheet without a full reload.

The reload happens in under a second. Edit code. Switch to the browser. The changes are already there.

If you run without the Rust CLI (e.g. Docker), set `TINA4_OVERRIDE_CLIENT=true`; there is no automatic reload in that mode.

---

8. Hot-Patching with jurigged

For faster iteration, Tina4 supports hot-patching via jurigged. Hot-patching updates function definitions in the running server without restarting it:

- No connection drops (WebSocket clients stay connected)
- No cache loss (in-memory caches stay warm)
- No session interruption
- Sub-second updates

Install jurigged:

```
uv add --dev jurigged
```

Start the server with hot-patching:

```
tina4 serve --hot
```

```
  Tina4 Python v3.0.0
  HTTP server running at http://0.0.0.0:7146
  Hot-patching enabled (jurigged)
```

Edit any route handler and save. The function updates in place -- without restarting the server. The next request uses the new code.

When Hot-Patching Cannot Help

Hot-patching works for function body changes. It does not work for:

- Adding or removing route decorators (requires restart)
- Changing module-level variables (requires restart)
- Modifying class definitions (may require restart)
- Changing `.env` (requires restart)

For these changes, the server does a full reload.

---

9. Request Inspector

The request inspector records every HTTP request to your application. For each request:

- **Timestamp** -- When the request arrived
- **Method** -- GET, POST, PUT, DELETE

- **URL** -- The full URL including query parameters
- **Status** -- The HTTP status code returned (color-coded: green for 2xx, yellow for 4xx, red for 5xx)
- **Time** -- How long the request took to process
- **Request ID** -- A unique identifier for correlating logs

Click on any request to see its full details.

Request Details Panel

- **Headers** -- All request headers (Accept, Content-Type, Authorization)
- **Body** -- The request body (for POST/PUT/PATCH), formatted as JSON if applicable
- **Query parameters** -- Parsed URL query parameters
- **Route match** -- Which route definition matched this request
- **Middleware** -- Which middleware ran and how long each took
- **Database queries** -- Every SQL query executed during this request, with timing
- **Template renders** -- Which templates rendered and how long each took
- **Response headers** -- The response headers sent back
- **Response body** -- The first 1000 characters of the response body

Filtering Requests

The inspector supports filtering:

- **By status**: Click the status code badges at the top (e.g., show only 5xx errors)
- **By method**: Filter by GET, POST, PUT, DELETE
- **By path**: Search for a URL pattern (e.g., `/api/` for API requests only)
- **By time range**: Show requests from the last 5 minutes, 1 hour, or all time

Request Replay

Click "Replay" on any request to re-send it. The inspector fires the same method, URL, headers, and body. You reproduce an error without constructing the curl command by hand.

This is the fastest path from "what happened?" to "I can see it happen again." Find a failing request. Hit Replay. Watch the error overlay show the stack trace. Fix the code. Replay again. Green status code. Done.

No more `print()` statements scattered through your code. The inspector shows what happened. Every request. Every detail. With one-click replay.

---

10. SQL Query Runner

The dev dashboard includes a SQL query runner. Execute queries against your database:

- **Execute queries** -- Run any SQL statement
- **See results** -- Formatted table with column headers and row numbers
- **View query timing** -- How long each query took
- **Browse tables** -- A sidebar lists all tables with their column definitions
- **Export results** -- Download as CSV

```
SELECT * FROM products WHERE category = 'Electronics' ORDER BY price DESC;
```

The results display in a table with column headers, row numbers, and data type indicators. Copy results, export as CSV, or run another query.

Safety

The query runner is read-write in development. You can run INSERT, UPDATE, and DELETE statements. Be careful -- there is no undo. In shared environments, consider a read-only database connection for the dev dashboard.

The query runner only works when `TINA4_DEBUG=true`. It is disabled in production.

---

11. Queue Monitor

If your application uses background job queues (Chapter 12 -- Queues), the dev dashboard includes a queue monitor. The monitor shows five categories:

- **Pending jobs** -- Jobs waiting to be processed
- **Active jobs** -- Jobs currently being processed
- **Completed jobs** -- Recently completed jobs with timing
- **Failed jobs** -- Jobs that threw exceptions, with error details
- **Dead-letter queue** -- Jobs that failed too many times

For each job, you see:

- The job function name
- The payload (serialized arguments)
- When it was enqueued
- When it started processing (if active)
- The error message and stack trace (if failed)
- How many times it has been retried

You can also take action:

- **Retry a failed job** -- Click "Retry" to move it back to the pending queue
- **Delete a job** -- Remove it from any queue
- **Pause/resume the queue** -- Stop processing without losing jobs

The queue monitor gives you a window into your background work. A job fails. You see the error. You fix the code. You hit Retry. The job processes. No log file hunting. No guesswork about what payload caused the failure.

---

12. System Info

The System Info tab shows detailed information about your environment:

- **Python Configuration** -- Version, installed packages, virtual environment path, memory limit
- **Database Info** -- Engine, version, connection details, table sizes, index information
- **Server Info** -- OS, hostname, server software, document root
- **Tina4 Config** -- All loaded `.env` variables (sensitive values masked), auto-discovered routes, registered middleware, ORM models
- **Disk Usage** -- Size of your project directory, data directory, logs directory

This panel solves the "it works on my machine" problem. Compare the System Info output between two environments. Spot the difference. The wrong Python version. A missing package. A misconfigured environment variable. The answer is in the panel.

---

13. Exercise: Debug a Failing Route

Your colleague wrote a route that fails. Use the dev tools to find and fix the bug.

Setup

Create `src/routes/buggy.py`:

```
from tina4_python.core.router import get, post
from tina4_python.cache import cache_get, cache_set

@get("/api/buggy/users")
async def buggy_users(request, response):
    users = cache_get("all_users")
    if users:
        return response({"users": users, "source": "cache"})

    # Bug 1: This query has an error
    db = Database.get_connection()
    users = db.fetch_all("SELCT * FROM users ORDER BY name")

    cache_set("all_users", users, ttl=300)
    return response({"users": users, "source": "database"})


@post("/api/buggy/users")
async def buggy_create_user(request, response):
    body = request.body

    # Bug 2: Missing validation
    user = User()
```

```
    user.name = body["name"]
    user.email = body["email"]
    user.save()

    # Bug 3: Wrong status code
    return response({"message": "User created", "user": user.to_dict()})
```

Requirements

- Open the dev dashboard at `http://localhost:7146/__dev`
- Hit the `GET /api/buggy/users` endpoint and find Bug 1 using the error overlay
- Hit the `POST /api/buggy/users` endpoint without a body and find Bug 2 using the request inspector
- Fix all three bugs:
- Bug 1: Fix the SQL syntax error
- Bug 2: Add validation for required fields
- Bug 3: Return 201 instead of 200
- Use Request Replay to verify each fix

Solution

```
from tina4_python.core.router import get, post
from tina4_python.cache import cache_get, cache_set

@get("/api/buggy/users")
async def buggy_users(request, response):
    users = cache_get("all_users")
    if users:
        return response({"users": users, "source": "cache"})

    db = Database.get_connection()
    users = db.fetch_all("SELECT * FROM users ORDER BY name")  # Fixed: SELCT -> SELECT

    cache_set("all_users", users, ttl=300)
    return response({"users": users, "source": "database"})


@post("/api/buggy/users")
async def buggy_create_user(request, response):
    body = request.body

    # Fixed: Added validation
    if not body.get("name") or not body.get("email"):
        return response({"error": "Name and email are required"}, 400)

    user = User()
    user.name = body["name"]
    user.email = body["email"]
    user.save()

    return response({"message": "User created", "user": user.to_dict()}, 201)  # Fixed: 201
```

The dev tools made each bug visible. The error overlay showed the SQL syntax error with the exact line. The request inspector showed the missing body causing a `KeyError`. Request Replay

let you re-send the fixed requests without leaving the dashboard.

---

14. Gotchas

1. Dev Dashboard Accessible on Network

Problem: Anyone on your network can access the dev dashboard.

Cause: `TINA4_DEBUG=true` makes the dashboard available at `/__dev`.

Fix: In production, set `TINA4_DEBUG=false` to disable the dashboard. In shared development environments, restrict network access.

2. Live Reload Causes Connection Drops

Problem: WebSocket connections drop every time you save a file.

Cause: The server restarts on file change, which closes all active connections.

Fix: Use hot-patching (`--hot` flag) instead of live reload for WebSocket development. Hot-patching updates function bodies without restarting the server, so connections stay open.

3. Error Overlay Shows in Production

Problem: Users see Python stack traces when errors occur.

Cause: `TINA4_DEBUG=true` is set in the production environment.

Fix: Set `TINA4_DEBUG=false` in production. The error overlay is replaced by a clean error page. Full details go to `logs/error.log`.

4. Request Inspector Slows Down the Server

Problem: The server becomes slower as it runs longer.

Cause: The request inspector stores every request in memory. After thousands of requests, memory usage grows.

Fix: The inspector limits storage to the last 1000 requests. If you notice slowness, clear the inspector from the dashboard. In production, the inspector is disabled.

5. SQL Runner Executes Destructive Queries

Problem: Someone ran `DROP TABLE users` in the SQL query runner.

Cause: The SQL runner executes any valid SQL query. There is no confirmation step.

Fix: The SQL runner only works when `TINA4_DEBUG=true`. Never leave debug mode on in production. For sensitive development databases, use a read-only database connection for the SQL runner.

6. Hot-Patching Does Not Pick Up New Routes

Problem: You added a new route decorator but it does not respond to requests.

Cause: Hot-patching (jurigged) can update function bodies but cannot register new decorators. Adding `@get("/new-route")` requires the decorator to execute, which only happens at import time.

Fix: Restart the server when adding or removing routes. Hot-patching modifies existing route behavior. It does not add new routes.

7. Template Changes Not Reflected

Problem: You edited a template but the page still shows the old version.

Cause: Template caching is enabled (`TINA4_TEMPLATE_CACHE_TTL=true`). The compiled template serves from cache.

Fix: In development, set `TINA4_TEMPLATE_CACHE_TTL=false` (this is the default when `TINA4_DEBUG=true`). If you enabled template caching manually, disable it for development.

8. Queue Monitor Shows No Jobs

Problem: The Queue Monitor tab is empty even though your application uses queues.

Cause: The queue system is not running. Jobs are enqueued but no worker processes them.

Fix: Start a queue worker: `tina4 queue work`. The queue monitor reflects the state of the queue storage (database or Redis). If no worker is running, jobs sit in "Pending" and never move to "Active" or "Completed."

Tina4 CLI

1. Getting a New Developer Up to Speed

Monday morning. A new developer joins your team. You hand them the repo URL. By 10am they have a running project, a new database model, CRUD routes, a migration, and a deployment to staging. All from the command line. No documentation scavenger hunt. No boilerplate copy-paste.

The Tina4 CLI is a single Rust binary. It manages all four Tina4 frameworks (PHP, Python, Ruby, Node.js). The commands are identical across languages. Learn the CLI for Python. You know it for PHP.

---

2. tina4 init -- Project Scaffolding

You saw this in Chapter 1. Now the details.

```
tina4 init my-project
```

```
Creating Tina4 project in ./my-project ...
  Detected language: Python (pyproject.toml)
  Created .env
  Created .env.example
  Created .gitignore
  Created src/routes/
  Created src/orm/
  Created migrations/
  Created src/seeds/
  Created src/templates/
  Created src/templates/errors/
  Created src/public/
  Created src/public/js/
  Created src/public/css/
  Created src/public/scss/
  Created src/public/images/
  Created src/public/icons/
  Created src/locales/
  Created data/
  Created logs/
  Created secrets/
  Created tests/

Project created! Next steps:
  cd my-project
  uv sync
  tina4 serve
```

Language Detection

The CLI detects the language from existing files:

File Present	Language
`composer.json`	PHP

`pyproject.toml` or `requirements.txt`	Python
`Gemfile`	Ruby
`package.json`	Node.js

If no language-specific file exists, the CLI asks:

```
tina4 init my-project
```

```
No language detected. Which language?
  1. PHP
  2. Python
  3. Ruby
  4. Node.js
> 2

Creating Tina4 Python project in ./my-project ...
```

Explicit Language Selection

Skip the prompt by specifying the language:

```
tina4 init python my-project
```

This creates a Python project with `pyproject.toml`, `app.py`, and the full directory structure.

Init into an Existing Directory

Already have a project? Add Tina4 structure:

```
cd existing-project
tina4 init .
```

The CLI creates only files and directories that do not exist. It never overwrites.

---

3. tina4 serve -- Dev Server

```
tina4 serve
```

```
  Tina4 Python v3.0.0
  HTTP server running at http://0.0.0.0:7146
  WebSocket server running at ws://0.0.0.0:7146
  Live reload enabled
  Press Ctrl+C to stop
```

`tina4 serve` detects the language and starts the appropriate server. For Python, it spawns the Tina4 Python runtime with SCSS compilation, file watching, and live reload all wired in.

Options

```
tina4 serve --port 8080        # Custom port
tina4 serve -p 8080            # Same thing, short form
tina4 serve --host 127.0.0.1   # Bind to localhost only
tina4 serve --no-browser       # Don't open a browser tab on startup
tina4 serve --no-reload        # Don't reload the browser when files change
tina4 serve --dev              # Force the dev server even if a production server is installed
tina4 serve --production       # Production server, debug off, no live reload
```

Serving a Project by Name

You don't have to `cd` into a project first. Pass its folder name and the CLI will find it, change into it, and serve it:

```
tina4 serve my-app             # ./my-app, or my-app in your projects folder
tina4 serve my-app -p 7150     # Options still apply
```

The CLI looks for `./my-app` in the current folder first, then in the projects folder `tina4 setup` recorded. If neither has it, it stops with `Project 'my-app' not found` and names the folder it searched. Run a bare `tina4 serve` outside any project and it will look in the projects folder too: one project there gets served, and several get listed so you can pick one by name.

Bypassing the CLI

The framework refuses to start without the Rust CLI. For sandboxed environments (Docker images that already wrap the framework, CI runners, APM agents that wrap the Python process directly) set `TINA4_OVERRIDE_CLIENT=true` in `.env` and only then run:

```
TINA4_OVERRIDE_CLIENT=true uv run python app.py
```

This bypasses SCSS compilation and live reload, so use it only when the Rust CLI is genuinely unavailable.

---

4. tina4 generate model -- ORM Scaffolding

The `generate model` command creates an ORM model file and a matching migration. One command produces both.

```
tina4 generate model Product
```

```
Created src/orm/Product.py
Created migrations/20260322120000_create_product.sql
Created migrations/20260322120000_create_product.down.sql
Created tests/test_product_model.py
```

The generated model:

```
from tina4_python.orm import ORM

class Product(ORM):
    table_name = "products"
    id: int
    created_at: str
```

The generated migration:

```
-- UP
CREATE TABLE products (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    created_at TEXT DEFAULT CURRENT_TIMESTAMP
);

-- DOWN
DROP TABLE IF EXISTS products;
```

The model is ready to use. Add your fields and run migrations.

Adding Fields

Specify fields on the command line:

```
tina4 generate model Product --fields "name:string,price:float,category:string,in_stock:bool"
```

The generated model includes all the fields:

```
from tina4_python.orm import ORM

class Product(ORM):
    table_name = "products"
    id: int
    name: str
    price: float
    category: str
    in_stock: bool
    created_at: str
```

And the migration:

```
-- UP
CREATE TABLE products (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    name TEXT NOT NULL DEFAULT '',
    price REAL NOT NULL DEFAULT 0,
    category TEXT NOT NULL DEFAULT '',
    in_stock INTEGER NOT NULL DEFAULT 0,
    created_at TEXT DEFAULT CURRENT_TIMESTAMP
);

-- DOWN
DROP TABLE IF EXISTS products;
```

Field Types

CLI Type	Python Type	SQLite Column
`string`	`str`	`TEXT`
`int`	`int`	`INTEGER`
`float`	`float`	`REAL`
`bool`	`bool`	`INTEGER`
`text`	`str`	`TEXT`
`date`	`str`	`TEXT`

Options

Flag	Description	Example
`--fields`	Comma-separated field definitions	`--fields "name:string,price:float"`
`--table-name`	Use this table name instead of the one derived from the class name	`--table-name products`
`--no-migration`	Skip migration generation	`--no-migration`

Apart from these and the general `--dry-run`, `--json` and `--help`, `generate model` stops with `Unknown option` on any other flag. To add CRUD routes, run `tina4 generate route` after it (section 8).

---

5. tina4 generate route -- CRUD Route Scaffolding

The `generate route` command creates a complete CRUD route file with all five REST endpoints. It reads the model's properties and builds routes with proper imports and response handling.

```
tina4 generate route products
```

```
Created src/routes/products.py
Created tests/test_products.py
```

The generated route file:

```
from tina4_python.core.router import get, post, put, delete

@get("/api/products")
async def list_products(request, response):
    page = int(request.params.get("page", 1))
    per_page = int(request.params.get("per_page", 20))
    offset = (page - 1) * per_page

    products, total = Product.where("1=1", [], limit=per_page, offset=offset)
    results = [p.to_dict() for p in products]

    return response({
        "data": results,
        "page": page,
        "per_page": per_page,
        "count": len(results)
    })


@get("/api/products/{product_id}")
async def get_product(request, response):
    product = Product.find(request.params["product_id"])
```

```
    if product is None:
        return response({"error": "Product not found"}, 404)

    return response(product.to_dict())


@post("/api/products")
async def create_product(request, response):
    body = request.body

    product = Product()
    product.name = body.get("name", "")
    product.price = float(body.get("price", 0))
    product.category = body.get("category", "")
    product.in_stock = bool(body.get("in_stock", False))
    product.save()

    return response(product.to_dict(), 201)


@put("/api/products/{product_id}")
async def update_product(request, response):
    product = Product.find(request.params["product_id"])

    if product is None:
        return response({"error": "Product not found"}, 404)

    body = request.body
    if "name" in body:
        product.name = body["name"]
    if "price" in body:
        product.price = float(body["price"])
    if "category" in body:
        product.category = body["category"]
    if "in_stock" in body:
        product.in_stock = bool(body["in_stock"])
    product.save()

    return response(product.to_dict())


@delete("/api/products/{product_id}")
async def delete_product(request, response):
    product = Product.find(request.params["product_id"])

    if product is None:
        return response({"error": "Product not found"}, 404)

    product.delete()
    return response(None, 204)
```

The generator reads the model's properties and creates routes with type casting and None checks. Customize the generated code immediately. It is regular Python, not magic.

Options

Flag	Description	Example
`--prefix`	Custom route prefix (default: `/api`)	`--prefix /api/v2`
`--middleware`	Add middleware to all routes	`--middleware auth_middleware`

---

6. tina4 generate migration -- Migration Scaffolding

The `generate migration` command creates a timestamped migration file with `UP` and `DOWN` sections. The timestamp ensures migrations run in order.

```
tina4 generate migration add_category_to_products
```

```
Created migrations/20260322120500_add_category_to_products.sql
```

The generated file:

```
-- UP
-- Add your forward migration SQL here


-- DOWN
-- Add your rollback migration SQL here
```

Fill in the SQL:

```
-- UP
ALTER TABLE products ADD COLUMN category TEXT DEFAULT '';

-- DOWN
ALTER TABLE products DROP COLUMN category;
```

The timestamp prefix (`20260322120500`) ensures migrations run in order. Each migration runs once. The framework tracks which ones have been applied.

When to Generate a Migration

Generate a new migration when you need to change the database schema after the initial model migration. Adding a column. Creating an index. Renaming a table. Each change gets its own migration file with a unique timestamp.

---

7. tina4 generate middleware -- Middleware Scaffolding

The `generate middleware` command creates a middleware function with the correct signature and a placeholder for your logic.

```
tina4 generate middleware rate_limit
```

```
Created src/middleware/rate_limit.py
```

The generated file:

```
async def rate_limit(request, response, next_handler):
    # Add your middleware logic here

    # Continue to the next middleware or route handler
    return await next_handler(request, response)

    # Or return early to block the request:
    # return response({"error": "Rate limit exceeded"}, 429)
```

The middleware is a named function. Reference it in route definitions:

```
@get("/api/data", middleware=["rate_limit"])
async def protected_data(request, response):
    return response({"data": "protected"})
```

---

8. Generate a Model and Its Routes

`generate model` has no flag that adds routes. Run `generate route` after it:

```
tina4 generate model Product
tina4 generate route products
```

```
Created src/orm/Product.py
Created migrations/20260322120000_create_product.sql
Created migrations/20260322120000_create_product.down.sql
Created tests/test_product_model.py
Created src/routes/products.py
Created tests/test_products.py
```

Model, migration, CRUD routes, and a test for each. For admin pages over the model as well, use `tina4 generate crud` (see the Scaffolding chapter).

---

9. tina4 doctor -- Health Check

The `doctor` command checks your project for common issues:

```
tina4 doctor
```

```
Tina4 Doctor -- Checking your project...

  [OK] Python 3.12.0 detected
  [OK] uv package manager found
  [OK] tina4_python package installed (v3.0.0)
  [OK] .env file exists
  [OK] Database connection: sqlite:///data/app.db
  [OK] Database is accessible
  [OK] src/routes/ directory exists (3 route files)
  [OK] src/orm/ directory exists (2 model files)
  [OK] src/templates/ directory exists (5 templates)
  [OK] src/public/ directory exists (static files served)
  [OK] tests/ directory exists (4 test files)
```

```
  [WARN] No migrations found in migrations/
  [OK] AI context: Claude Code detected, CLAUDE.md present
  [OK] .gitignore includes .env, data/, logs/

  12 checks passed, 1 warning, 0 errors
```

Doctor checks:

- Python version and package manager
- Tina4 package installation and version
- `.env` file existence and critical variables
- Database connectivity
- Directory structure
- Missing files or configurations
- AI tool context files
- Git configuration

The warnings give actionable advice. If your database is not configured, it tells you exactly what to add to `.env`.

---

10. tina4 test -- Running Tests

```
tina4 test
```

```
Running tests...

  ProductTest
    [PASS] test_create_product
    [PASS] test_load_product

  2 tests, 2 passed, 0 failed (0.12s)
```

This runs all tests in the `tests/` directory. See Chapter 18 for full testing documentation.

Test Options

```
tina4 test --file tests/test_product.py           # Specific file
tina4 test --file tests/test_product.py --method test_create  # Specific method
tina4 test --verbose                                # Show assertion details
```

---

11. tina4 routes -- Route Listing

See all registered routes in your project:

```
tina4 routes
```

```
Registered Routes:

  Method  Path                        Handler                    Middleware
  ------  ------------------------------------------------------  ----------
```

```
  GET     /health                     health_check              -
  GET     /api/products               list_products             ResponseCache:300
  GET     /api/products/{product_id}  get_product               -
  POST    /api/products               create_product            auth_middleware
  PUT     /api/products/{product_id}  update_product            auth_middleware
  DELETE  /api/products/{product_id}  delete_product            auth_middleware
  GET     /api/auth/login             -                         -
  POST    /api/auth/login             login                     -
  GET     /admin                      admin_dashboard           -

  9 routes registered
```

This is useful for verifying that your routes are registered and for finding the handler function for a specific URL.

When debugging routing issues, check here first. If a route does not match, `tina4 routes` shows whether it was registered and what middleware is attached.

---

12. tina4 migrate -- Database Migrations

Run pending migrations:

```
tina4 migrate

Running migrations...
  [UP] 20260322000100_create_users_table.sql
  [UP] 20260322000200_create_products_table.sql
  [UP] 20260322000300_add_category_to_products.sql

  3 migrations applied
```

Rollback

```
tina4 migrate:rollback

Rolling back last migration...
  [DOWN] 20260322000300_add_category_to_products.sql

  1 migration rolled back
```

Migration Table Auto-Upgrade

If your project was created with an earlier version of Tina4, the `tina4_migration` tracking table may use the older v2 schema. Running `tina4 migrate` detects the old layout and adds the missing `migration_id`, `batch`, and `executed_at` columns, backfilling existing data. No manual intervention needed.

Status

```
tina4 migrate:status

Migration Status:

  Status    Migration
  --------  ----------------------------------------
  Applied   20260322000100_create_users_table.sql
  Applied   20260322000200_create_products_table.sql
```

```
  Applied   20260322000300_add_category_to_products.sql
  Pending   20260322000400_create_orders_table.sql

  3 applied, 1 pending
```

---

13. Exercise: Scaffold a Feature in 5 Commands

Scaffold a complete "Customer" feature from scratch using only CLI commands.

Requirements

Starting from an existing Tina4 Python project, run 5 commands to create:

- A Customer ORM model with name, email, phone, and company fields
- A CRUD route file with all five REST endpoints
- A migration that creates the customers table
- Run the migration to create the table
- Run the doctor to verify everything

Expected Commands

```
# 1. Generate the model, its migration, and the CRUD routes
tina4 generate model Customer
tina4 generate route customers

# 2. Edit the model to add fields (manual step)
# Add fields to src/orm/Customer.py

# 3. Edit the migration to add columns (manual step)
# Add columns to the migration file

# 4. Run the migration
tina4 migrate

# 5. Run the doctor to verify everything is set up
tina4 doctor
```

Solution

Command 1: Generate the model, its migration, and the CRUD routes:

```
tina4 generate model Customer
tina4 generate route customers
```

Command 2: Edit `src/orm/Customer.py`:

```
from tina4_python.orm import ORM

class Customer(ORM):
    table_name = "customers"

    id: int
    name: str
    email: str
    phone: str
```

```
    company: str
    created_at: str
```

Command 3: Edit the migration file:

```
-- UP
CREATE TABLE customers (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    name TEXT NOT NULL,
    email TEXT NOT NULL UNIQUE,
    phone TEXT,
    company TEXT,
    created_at TEXT DEFAULT CURRENT_TIMESTAMP
);

CREATE INDEX idx_customers_email ON customers(email);

-- DOWN
DROP TABLE IF EXISTS customers;
```

Command 4: Run the migration:

```
tina4 migrate
```

```
Running migrations...
  [UP] 20260322120000_create_customers_table.sql

  1 migration applied
```

Command 5: Verify with doctor:

```
tina4 doctor
```

```
  [OK] src/orm/ directory exists (3 model files)
  [OK] src/routes/ directory exists (4 route files)
  [OK] Database connection: sqlite:///data/app.db
  ...
```

Now test the API:

```
curl -X POST http://localhost:7146/api/customers \
  -H "Content-Type: application/json" \
  -d '{"name": "Alice Corp", "email": "alice@corp.com", "phone": "+1-555-0100", "company": "Alic
```

```
{
  "id": 1,
  "name": "Alice Corp",
  "email": "alice@corp.com",
  "phone": "+1-555-0100",
  "company": "Alice Corp",
  "created_at": "2026-03-22 12:00:00"
}
```

From zero to a working CRUD API. Five commands. Under two minutes.

---

14. Gotchas

1. tina4 Command Not Found

Problem: Running `tina4` gives "command not found".

Cause: The Tina4 CLI is not installed or not in your PATH.

Fix: Install the CLI: `curl -fsSL https://tina4.com/install.sh | sh`. Verify with `tina4 --version`. If installed but not found, add the installation directory to your PATH:

```
echo 'export PATH="$HOME/.tina4/bin:$PATH"' >> ~/.zshrc
source ~/.zshrc
```

2. Wrong Language Detected

Problem: `tina4 init` creates a PHP project instead of Python.

Cause: A `composer.json` file exists in the directory from a previous project.

Fix: Use explicit language selection: `tina4 init python my-project`. Or delete the conflicting language file before running `init`.

3. Generated Files Overwrite Existing Code

Problem: Running `tina4 generate route products` overwrites your custom route file.

Cause: The generate command creates files at fixed paths. If the file exists, it is overwritten.

Fix: The CLI warns you before overwriting. Check if the file exists first. If you need to regenerate, rename the existing file: `mv src/routes/products.py src/routes/products_backup.py`.

4. Migration Order Issues

Problem: A migration fails because it references a table that does not exist yet.

Cause: Migration files run in alphabetical (timestamp) order. If migration B depends on a table created by migration A, but A has a later timestamp, B runs first and fails.

Fix: Use consistent timestamps. The `tina4 generate migration` command uses the current timestamp. Generating migrations in order ensures correct execution order. If you need to fix ordering, rename the migration files to adjust their timestamps.

5. tina4 serve Uses Wrong Port

Problem: `tina4 serve` starts on port 7146 but you need port 8080.

Cause: The default port is 7146 unless overridden.

Fix: Set it in `.env`: `TINA4_PORT=8080`. Or pass it as a flag: `tina4 serve --port 8080`. The `.env` value takes precedence over the default. The command-line flag overrides everything.

6. Doctor Shows False Warnings

Problem: `tina4 doctor` warns about missing migrations, but your project does not use migrations.

Cause: Doctor checks for common conventions. It warns about missing migrations regardless of your approach.

Fix: These are warnings, not errors. Ignore warnings that do not apply to your project. Doctor is a guide, not a gatekeeper.

7. Generate Creates Python 3.12+ Syntax

Problem: Generated code uses `type: int` syntax that does not work in Python 3.10.

Cause: The generator targets the latest Python version supported by Tina4.

Fix: Use Python 3.12 or later with Tina4 Python. Check your Python version with `python --version`. If you must use an older version, modify the generated code to use compatible syntax.

8. Model Name Must Be PascalCase

Problem: `tina4 generate model order_item` creates a model class named `order_item` which is not valid Python convention.

Cause: The CLI uses the argument as-is for the class name.

Fix: Use PascalCase for model names: `tina4 generate model OrderItem`. The CLI converts it to snake_case for the table name (`order_items`).

---

15. Documentation

```
tina4 docs      # Download framework-specific book chapters to .tina4-docs/
tina4 books     # Download the complete Tina4 book (all languages) to tina4-book/
```

`tina4 docs` detects your project language and downloads only the relevant chapters. The documentation is available in Markdown format, optimised for AI tools and local reference.

---

16. Test Port (Dual-Port Development)

When `TINA4_DEBUG=true`, Tina4 automatically starts a second HTTP server on `port + 1000`:

- **Main port** (e.g. 7146): hot-reload enabled, for AI dev tools
- **Test port** (e.g. 8145): stable, no hot-reload, for user testing

This prevents the browser from refreshing mid-test when AI tools edit files.

Setting	Effect
`TINA4_NO_AI_PORT=true`	Disables the test port entirely
`TINA4_NO_RELOAD=true`	Disables hot-reload on the main port too
`--no-reload`	CLI flag equivalent of TINA4*NO*RELOAD

Vibe Coding with AI

Why Tina4 is Built for AI-Assisted Development

Tell your AI assistant: "Add a product catalog with search, pagination, and category filtering." In most frameworks, the AI needs to guess which packages to install, which config files to create, which naming conventions to follow. It hallucinates half of it.

With Tina4, the AI reads one file -- `CLAUDE.md` -- and knows everything. Every method signature. Every import path. Every convention. It generates correct, runnable code on the first try because there is only one way to do things in Tina4.

This is not accidental. Tina4 was designed from the ground up for AI-assisted development.

This chapter explains coding assistants and the `tina4 ai` skills installer. To call a model from your application, see Chapter 40: AI Client.

The Zero-Dependency Advantage

When an AI generates code for Django, it might suggest `from django.core.cache import cache` or `from django.views.decorators.cache import cache_page`. Both exist. Both work differently. The AI guesses.

Tina4 eliminates the guesswork. One cache. One queue. One ORM. One template engine. No ambiguity. No alternatives. No "it depends on which package you installed." The AI knows.

```
# There is only one way to cache in Tina4
from tina4_python.cache import cache_get
products = cache_get("products")

# There is only one way to queue in Tina4
from tina4_python.queue import Queue
queue = Queue(topic="emails")
queue.push({"to": "user@test.com"})

# There is only one way to send email in Tina4
from tina4_python.messenger import Messenger
mailer = Messenger()
mailer.send(to="user@test.com", subject="Welcome", body="Hello!")
```

Zero dependencies means zero confusion for AI.

CLAUDE.md -- The AI's Instruction Manual

Every Tina4 project includes a `CLAUDE.md` file that tells AI assistants exactly how the framework works. It contains:

- Every method signature with parameters and return types
- The project structure convention (src/routes/, src/orm/, src/templates/)

- The .env variable reference
- Code style rules (routes return `response()`, ORM uses `to_dict()`)
- Database connection string formats
- Common gotchas and how to avoid them

When you open a Tina4 project in Claude Code, Cursor, or GitHub Copilot -- the AI reads this file first and immediately understands how to write correct Tina4 code.

Skills -- Deep Framework Knowledge

Beyond CLAUDE.md, Tina4 ships with three AI skill files:

tina4-maintainer

For framework maintenance -- porting features between languages, reviewing PRs, running benchmarks, checking parity.

tina4-developer

For application development -- creating routes, defining models, writing templates, setting up auth, configuring queues.

tina4-js

For frontend development -- tina4-js signals, Tina4Element components, reactive templates, WebSocket client.

These skills are automatically installed when AI tools are detected:

```
# In your project root
from tina4_python.ai import install_selected, install_context

install_selected(".", "all")    # Detects + installs context for every supported tool
# install_context(".")            # Or: install context for ALL known tools, even those not dete
```

The Convention Advantage

AI thrives on convention. Every Tina4 project follows the same structure. The AI never asks "where should I put this?"

```
src/
  routes/hello.py       -> AI knows: this is a route file
  orm/product.py        -> AI knows: this is an ORM model
  templates/home.html   -> AI knows: this is a template
  middleware/auth.py     -> AI knows: this is middleware
```

Tell the AI "add a User model" and it creates `src/orm/user.py` with the correct field definitions, the correct class structure, and the correct method stubs. Every time.

Real-World Vibe Coding Session

Here is an actual conversation with Claude Code in a Tina4 Python project:

You: "Add a blog with posts and comments. Posts belong to users. Comments belong to posts. I want a REST API and a template page that lists posts."

Claude Code generates:

- `src/orm/post.py` -- Model with relationships to user and comments
- `src/orm/comment.py` -- Model with relationship to post
- `src/routes/api/posts.py` -- Full CRUD API (list, get, create, update, delete)
- `src/routes/api/comments.py` -- Nested comments API
- `src/routes/blog.py` -- Template route for the blog page
- `src/templates/blog.html` -- Page with tina4css cards for each post
- `migrations/20260322_create_posts_and_comments.sql` -- Database schema

All correct. All runnable. First try.

You: "Add pagination to the posts list"

Claude adds `request.params.get("page", 1)` handling and `to_dict()` with limit/offset -- because CLAUDE.md told it exactly how Tina4 pagination works.

You: "Cache the post list for 5 minutes"

Claude adds `cache_get(f"posts_page_{page}")` with `cache_set(f"posts_page_{page}", data, ttl=300)` -- because it knows the cache API.

You: "Send an email when a new comment is posted"

Claude adds `mailer = Messenger(); mailer.send(...)` -- because it knows Messenger reads from .env.

No research. No Stack Overflow. No package debates. Describe the intent. The AI builds it.

Python-Specific Patterns the AI Knows

The CLAUDE.md file teaches AI assistants the correct Python patterns for Tina4:

Route Decorators

```
from tina4_python.core.router import get, post, put, delete, template

@get("/api/products")
async def list_products(request, response):
    products = Product().select("*")
    return response({"products": [p.to_dict() for p in products]})

@post("/api/products")
async def create_product(request, response):
    body = request.body
    product = Product()
    product.name = body["name"]
    product.price = body["price"]
    product.save()
    return response(product.to_dict(), 201)
```

Template Rendering

```
@get("/products")
async def products_page(request, response):
    products = Product.all(order_by="name ASC")
    return response.render("products.html", {"products": [p.to_dict() for p in products]})
```

Middleware Attachment

```
from tina4_python.core.router import get, middleware
from src.app.middleware import AuthMiddleware

@middleware(AuthMiddleware)
@get("/api/profile")
async def get_profile(request, response):
    user = User.find(request.user_id)
    return response(user.to_dict()) if user else response({"error": "User not found"}, 404)
```

`@middleware(...)` is a separate decorator stacked above the route decorator -- it is not a keyword argument on `@get`/`@post`.

WebSocket Handlers

```
from tina4_python.websocket import websocket_route

@websocket_route("/ws/updates")
async def updates_handler(connection):
    async for message in connection:
        await connection.broadcast(message)
```

The AI generates all of these patterns correctly because CLAUDE.md specifies:

- Handlers are always `async def`
- Route handlers always receive `(request, response)`
- WebSocket handlers receive a single `connection` object -- iterate it with `async for` to receive frames
- Responses use `response()` with the data as the first argument
- Templates render via `response.render("name.html", {...})`

- Middleware stacks above the route decorator via `@middleware(MiddlewareClass)`

Supported AI Tools

Tina4 auto-detects and installs context for eight tools. The exact list is in `tina4_python/ai/__init__.py` (`AI_TOOLS`):

Tool	Detection (context file)	Context installed
Claude Code	`CLAUDE.md`	CLAUDE.md + `.claude/` skills
Cursor	`.cursorules`	`.cursorules` (+ `.cursor/`)
GitHub Copilot	`.github/copilot-instructions.md`	`.github/copilot-instructions.md`
Windsurf	`.windsurfrules`	`.windsurfrules`
Aider	`CONVENTIONS.md`	`CONVENTIONS.md`
Cline	`.clinerules`	`.clinerules`
OpenAI Codex	`AGENTS.md`	`AGENTS.md`
Google Antigravity	`.antigravity/context.md`	`.antigravity/context.md`

Run `tina4 ai` to see which tools are detected in your project and install context files via the menu.

The AI Chat in Dev Dashboard

The dev dashboard at `/__dev` includes an AI chat tab. By default it talks to a local **qwen2.5-coder** model served via Ollama, grounded by Tina4's built-in RAG index of the framework source. Nothing leaves your machine.

Configure it in `.env`:

```
TINA4_AI_URL=http://localhost:11434      # Ollama HTTP endpoint
TINA4_AI_MODEL=qwen2.5-coder             # Default coding model
TINA4_RAG_URL=http://localhost:11434     # RAG embedding endpoint (defaults to TINA4_AI_URL)
```

If you prefer a remote provider, point `TINA4_AI_URL` at any OpenAI-compatible endpoint (LM Studio, vLLM, an OpenAI-compatible proxy, etc.) and set `TINA4_AI_MODEL` to the model name that endpoint serves.

The AI has full context of your Tina4 project -- routes, models, templates, configuration. Ask it:

- "Why is my /api/users route returning 500?"

- "How do I add WebSocket support?"
- "Write a migration to add an 'avatar' column to users"

The "Ask about this error" button on the error overlay sends the full stack trace and source code to the AI for instant debugging.

Tips for Effective Vibe Coding with Tina4

- **Be specific about what you want** -- "Add a product API with search by name and price range" works better than "add products"
- **Let the AI use tina4 generate** -- "Run tina4 generate model Product then add price and category fields" gives the AI a starting point
- **Reference the .env** -- "Configure the queue to use RabbitMQ" -- the AI knows to update .env with `TINA4_QUEUE_BACKEND=rabbitmq`
- **Ask for tests** -- "Write tests for the Product API" -- the AI uses Tina4's inline testing framework
- **Ask for the gallery** -- "Deploy the auth gallery example" -- the AI clicks Try It and you have a working JWT demo
- **Review, don't rewrite** -- The AI's first attempt is usually 90% correct. Tweak the details instead of starting over.

Exercise

Open your Tina4 Python project in Claude Code (or your preferred AI tool) and try these prompts:

- "Add a Contact model with name, email, phone, and message fields"
- "Create a contact form page at /contact with an HTML template using tina4css"
- "Add an API endpoint that accepts the form submission and saves it to the database"
- "Send an email notification when a new contact is submitted"
- "Add rate limiting to the contact form -- max 3 submissions per minute"

Each prompt should generate correct, runnable code on the first try. If it does not, check that CLAUDE.md is in your project root.

Gotchas

1. No CLAUDE.md?

Problem: The AI generates incorrect import paths or uses wrong API patterns.

Fix: Run `tina4 ai` (or `from tina4_python.ai import install_context; install_context(".")`) to generate the CLAUDE.md file. This gives the AI complete framework knowledge.

2. AI Suggests Wrong Import Paths

Problem: The AI writes `from tina4_python import Router` instead of `from tina4_python.core.router import get`.

Fix: The CLAUDE.md might be outdated. Regenerate it with `install_context(".")`. Check that the file includes the correct import paths for Python.

3. AI Hallucinates a Package

Problem: The AI suggests `pip install tina4-cache-redis` or similar packages that do not exist.

Fix: Remind it: "Tina4 has zero dependencies. Use only built-in features." Everything -- caching, queues, email, WebSocket, GraphQL -- is part of the framework.

4. AI Creates Files in Wrong Location

Problem: The AI creates routes in `routes/` instead of `src/routes/`.

Fix: Tell it: "Follow the src/routes, src/orm, src/templates convention." The CLAUDE.md file specifies this, but some AI tools need a reminder.

5. AI Code Does Not Run

Problem: The generated code has syntax errors or does not follow Tina4 patterns.

Fix: Check that route handlers use `async def` and return `response()`. Check that ORM models extend `ORM`. Check that templates use `template()` for rendering. These are the most common mistakes.

6. AI Uses Synchronous Code

Problem: The AI generates `def handler(request, response)` instead of `async def handler(request, response)`.

Fix: All Tina4 Python route handlers must be `async def`. Tell the AI: "All handlers are async in Tina4 Python."

7. AI Suggests Flask/Django Patterns

Problem: The AI writes `@app.route("/products")` or `return JsonResponse(data)`.

Fix: The AI is falling back to its general Python web framework knowledge. Point it to CLAUDE.md: "Read the CLAUDE.md file for Tina4 Python patterns. Do not use Flask or Django patterns."

The Philosophy

Tina4 was not made compatible with AI coding tools. It was designed for them.

Convention over configuration: the AI knows where things go. Zero dependencies: the AI never chooses between packages. A single CLAUDE.md: the AI has complete framework knowledge. Identical APIs across 4 languages: the AI's knowledge transfers instantly.

You describe the intent. The AI writes the code. You review and refine. That is the workflow Tina4 was built for.

The Intelligent Native Application 4ramework. Built for AI. Built for you.

Environment Variables

> **■■ *BREAKING CHANGE - Tina4 v3.12.0***
>
> *Every framework env var now requires the `TINA4_` prefix. The legacy un-prefixed names (`DATABASE_URL`, `SECRET`, `SMTP_HOST`, `HOST_NAME`, etc.) no longer work. Setting them at startup makes the framework refuse to boot with a list of renames.*
>
> *Run `tina4 env --migrate` to rewrite your existing `.env` automatically, or rename manually using the table below. The runtime guard prints the same mapping if it detects legacy names.*
>
> ***Conventional names stay un-prefixed:*** `PORT`, `HOST`, `NODE_ENV`, `RACK_ENV`, `RUBY_ENV`, `ENVIRONMENT`. *These are runtime/PaaS conventions, not framework config.*

Tina4 Python is configured through environment variables, read from `.env` at the project root. Every variable has a sensible default, and most projects set three or four values and leave the rest alone.

This chapter lists every variable the Python framework reads, grouped by subsystem. Start with the minimum-config examples at the end, then come back here when you need to tune something specific.

Core Server

Variable	Default	Description
`HOST`	`0.0.0.0`	Bind address. `0.0.0.0` listens on every interface. `127.0.0.1` restricts to localhost.
`TINA4_HOST`	*(inherits `HOST`)*	Explicit Tina4-specific bind address override. Takes precedence over `HOST` when both are set.
`PORT`	`7146`	HTTP server port. The Rust CLI prefers `TINA4_PORT` but falls back to `PORT`.
`TINA4_PORT`	*(inherits `PORT`)*	Explicit Tina4-specific port override. Takes precedence over `PORT` when both are set.
`TINA4_HOST_NAME`	`localhost:7146`	Fully-qualified host used in generated absolute URLs (Swagger, OAuth redirects, emails).
`TINA4_DEBUG`	`false`	Master debug toggle. Enables Swagger UI, dev dashboard, live reload, template dump filter, error overlay. Never set to `true` in production.
`TINA4_ENV`	`development`	Runtime environment label. Values like `development`, `staging`, `production` control dev-only features.
`TINA4_ENV_FILE`	`.env`	Alternative `.env` path. Read at boot before any other framework config; point at `.env.staging` or `.env.production` to switch the whole config tree.

`TINA4_SUPPRESS`	`false`	Suppresses the framework startup banner. Useful in CI runs or systemd units where stdout is parsed by another process.
`TINA4_HEALTH_PATH`	`/__health`	URL for the built-in liveness/readiness endpoint. The legacy `/health` path is kept as an alias.
`TINA4_NO_BROWSER`	`false`	Stops `tina4 serve` from opening your browser on every restart. Recommended during active development.
`TINA4_OPEN_BROWSER`	`true`	Alternative flag, set to `false` to prevent the browser opening on start.
`TINA4_NO_RELOAD`	`false`	Disables the dev hot-reload signal from the Rust CLI. Use when you want a stable server for debugging.
`TINA4_DEV_POLL_INTERVAL`	`1.0`	Seconds between dev-mode mtime polls. Lower for faster reload, higher to reduce CPU.
`TINA4_PUBLIC_DIR`	`src/public`	Directory served as static files under `/`.

Secrets and Authentication

Variable	Default	Description
`TINA4_SECRET`	`tina4-default-secret`	JWT signing secret. Must be long, random, and unique per environment. **Never commit.**
`TINA4_TOKEN_LIMIT`	`60`	JWT token lifetime in minutes.
`TINA4_TOKEN_EXPIRES_IN`	*(inherits `TOKEN_LIMIT`)*	Alias for `TINA4_TOKEN_LIMIT`.
`TINA4_API_KEY`	*(empty)*	Static API key used by `Auth.validate_api_key()` as a fallback to JWT.
`TINA4_API_KEY`	*(empty)*	Legacy alias for `TINA4_API_KEY`.

Database

Variable	Default	Description
`TINA4_DATABASE_URL`	`sqlite:///data/app.db`	Connection URL. Scheme selects the driver: `sqlite`, `postgres`, `mysql`, `firebird`.
`TINA4_DATABASE_USERNAME`	*(empty)*	Overrides the username embedded in `TINA4_DATABASE_URL`.
`TINA4_DATABASE_PASSWORD`	*(empty)*	Overrides the password embedded in `TINA4_DATABASE_URL`.
`TINA4_DATABASE_FIREBIRD_PATH`	*(empty)*	Overrides the database path/alias parsed from `TINA4_DATABASE_URL` for Firebird. Useful for Windows backslash paths and split-config setups.
`TINA4_DB_CACHE`	`false`	Enables in-memory query-result caching for read queries.
`TINA4_DB_CACHE_TTL`	`60`	Query cache TTL in seconds when `TINA4_DB_CACHE=true`.
`TINA4_DB_POOL`	`0`	Default `pool` size for `Database(url)` when the caller doesn't pass `pool=` explicitly. `0` disables pooling and uses a single connection. Set to a positive integer (e.g. `4`) to enable round-robin connection pooling.
`TINA4_ORM_PLURAL_TABLE_NAMES`	`true`	When `true`, the ORM pluralises class names into table names (`User` → `users`). Set `false` to keep them singular.

CORS

Variable	Default	Description
`TINA4_CORS_ORIGINS`	*(empty)*	Comma-separated allowed origins. Empty denies every cross-origin request; set the origins your front end is served from. `*` allows any origin and has to be set explicitly.
`TINA4_CORS_MAX_AGE`	`600`	Preflight cache lifetime in seconds.

Routing

Variable	Default	Description
`TINA4_TRAILING_SLASH_REDIRECT`	`false`	When truthy, requests to `/foo/` are 301-redirected to `/foo` so search engines and clients only see one canonical URL per route. The root `/` is exempt.

Security Headers

Variable	Default	Description
`TINA4_CSP`	`default-src 'self'`	`Content-Security-Policy` header.
`TINA4_CSRF`	`false`	CSRF token validation on POST/PUT/PATCH/DELETE.
`TINA4_HSTS`	*(empty/off)*	`Strict-Transport-Security` max-age in seconds. Set `31536000` in production with HTTPS.
`TINA4_FRAME_OPTIONS`	`DENY`	`X-Frame-Options` header.
`TINA4_REFERRER_POLICY`	`strict-origin-when-cross-origin`	`Referrer-Policy` header.

Rate Limiting

Variable	Default	Description
`TINA4_RATE_LIMIT`	`100`	Maximum requests per window per IP. Set `0` to disable.
`TINA4_RATE_WINDOW`	`60`	Rate-limit window in seconds.

Sessions

Variable	Default	Description
`TINA4_SESSION_BACKEND`	`file`	Storage backend. Options: `file`, `redis`, `valkey`, `mongo`, `database`.
`TINA4_SESSION_TTL`	`1800`	Session expiry in seconds (30 minutes).
`TINA4_SESSION_SAMESITE`	`Lax`	SameSite cookie attribute. Options: `Strict`, `Lax`, `None`.
`TINA4_SESSION_PATH`	`data/sessions`	Filesystem path for the file backend.
`TINA4_SESSION_NAME`	`tina4_session`	Name of the session cookie sent to the browser.
`TINA4_SESSION_HTTPONLY`	`true`	Sets the `HttpOnly` cookie attribute so JavaScript on the page cannot read the session ID.
`TINA4_SESSION_SECURE`	`false`	Sets the `Secure` cookie attribute so the session cookie is only sent over HTTPS. Turn on in production.

Redis/Valkey session backend

Variable	Default	Description
`TINA4_SESSION_REDIS_HOST`	`localhost`	Redis host.
`TINA4_SESSION_REDIS_PORT`	`6379`	Redis port.
`TINA4_SESSION_REDIS_PASSWORD`	*(none)*	Redis auth password.
`TINA4_SESSION_REDIS_DB`	`0`	Redis database number.
`TINA4_SESSION_VALKEY_HOST`	`localhost`	Valkey host.
`TINA4_SESSION_VALKEY_PORT`	`6379`	Valkey port.
`TINA4_SESSION_VALKEY_PASSWORD`	*(none)*	Valkey auth password.
`TINA4_SESSION_VALKEY_DB`	`0`	Valkey database number.

MongoDB session backend

Variable	Default	Description
`TINA4_SESSION_MONGO_URL`	`mongodb://localhost:27017`	MongoDB connection string.
`TINA4_SESSION_MONGO_DB`	`tina4`	MongoDB database name.
`TINA4_SESSION_MONGO_COLLECTION`	`sessions`	MongoDB collection name.

Templates

Variable	Default	Description
`TINA4_TEMPLATE_CACHE_TTL`	`0`	Frond template compile-cache TTL in seconds. `0` keeps compiled templates in memory permanently; set to a positive value if you want the engine to recompile after N seconds (rarely needed; `tina4 serve` invalidates the cache automatically on file change).

Cache

Variable	Default	Description
`TINA4_CACHE_BACKEND`	`memory`	Response cache backend. Options: `memory`, `file`, `redis`, `valkey`, `memcached`, `mongodb`, `database`. Falls back to `file` if the configured backend is unreachable.
`TINA4_CACHE_DIR`	`data/cache`	Cache directory for the file backend.
`TINA4_CACHE_TTL`	`60`	Default cache TTL in seconds.
`TINA4_CACHE_MAX_ENTRIES`	`1000`	Maximum cache entries. Oldest entries evicted first.
`TINA4_CACHE_URL`	`redis://localhost:6379`	Connection URL for remote cache backends. For `database`, falls back to `TINA4_DATABASE_URL` when unset.
`TINA4_CACHE_USERNAME`	*(none)*	Username for the cache backend. May also be embedded in `TINA4_CACHE_URL`.
`TINA4_CACHE_PASSWORD`	*(none)*	Password for the cache backend. May also be embedded in `TINA4_CACHE_URL` (e.g. `redis://:pass@host`). Memcached is unauthenticated.

Queues

Variable	Default	Description
`TINA4_QUEUE_BACKEND`	`lite`	Queue backend. Options: `lite` (file-based), `kafka`, `rabbitmq`, `mongo`, `database`.
`TINA4_QUEUE_PATH`	`data/queue`	Filesystem path for the lite backend.
`TINA4_QUEUE_URL`	*(none)*	Connection URL for remote backends.

Kafka queue backend

Variable	Default	Description
`TINA4_KAFKA_BROKERS`	`localhost:9092`	Comma-separated broker list.
`TINA4_KAFKA_GROUP_ID`	`tina4_consumer_group`	Kafka consumer group ID.

RabbitMQ queue backend

Variable	Default	Description
`TINA4_RABBITMQ_HOST`	`localhost`	RabbitMQ host.
`TINA4_RABBITMQ_PORT`	`5672`	RabbitMQ port.
`TINA4_RABBITMQ_USERNAME`	`guest`	RabbitMQ username.
`TINA4_RABBITMQ_PASSWORD`	`guest`	RabbitMQ password.
`TINA4_RABBITMQ_VHOST`	`/`	RabbitMQ virtual host.

MongoDB queue backend

Variable	Default	Description
`TINA4_MONGO_URI`	*(none)*	Full MongoDB connection string. Overrides host/port when set.
`TINA4_MONGO_HOST`	`localhost`	MongoDB host.
`TINA4_MONGO_PORT`	`27017`	MongoDB port.
`TINA4_MONGO_USERNAME`	*(none)*	MongoDB username.
`TINA4_MONGO_PASSWORD`	*(none)*	MongoDB password.
`TINA4_MONGO_DB`	`tina4`	MongoDB database name.
`TINA4_MONGO_COLLECTION`	`tina4_queue`	MongoDB collection name for jobs.

WebSocket

Variable	Default	Description
`TINA4_WS_BACKPLANE`	*(none)*	Backplane type. Set `redis` for multi-instance broadcasts.
`TINA4_WS_MAX_CONNECTIONS`	`1000`	Maximum concurrent WebSocket connections.
`TINA4_WS_MAX_FRAME_SIZE`	`65536`	Maximum WebSocket frame size in bytes.

Email

Variable	Default	Description
`TINA4_MAIL_HOST`	*(none)*	SMTP server hostname.
`TINA4_MAIL_PORT`	`587`	SMTP server port.
`TINA4_MAIL_USERNAME`	*(none)*	SMTP authentication username.
`TINA4_MAIL_PASSWORD`	*(none)*	SMTP authentication password.
`TINA4_MAIL_FROM`	*(none)*	Default sender email address.
`TINA4_MAIL_FROM_NAME`	*(none)*	Default sender display name.
`TINA4_MAIL_ENCRYPTION`	`tls`	Connection encryption. Options: `tls`, `ssl`, `none`.
`TINA4_MAIL_IMAP_HOST`	*(none)*	IMAP server for inbound mail.
`TINA4_MAIL_IMAP_PORT`	`993`	IMAP server port.
`TINA4_MAIL_IMAP_ENCRYPTION`	`tls`	IMAP transport encryption. Options: `tls`, `starttls`, `none`. Invalid values fall back to `tls` so a typo can't accidentally disable encryption.
`TINA4_MAIL_CAPTURE`	`false`	Force local DevMailbox capture and skip SMTP, even when an SMTP host exists. A missing host also captures.
`TINA4_MAIL_REDIRECT_TO`	*(none)*	Comma-separated safety recipients for real SMTP delivery. Replaces To, Cc, and Bcc.
`TINA4_MAILBOX_DIR`	`data/mailbox`	Storage directory for captured mail. This setting does not enable or disable capture.

> `TINA4_DEBUG` *does not suppress email. On staging, set* `TINA4_MAIL_CAPTURE=true` *to keep every message local, or set* `TINA4_MAIL_REDIRECT_TO=qa@example.com,product@example.com` *to exercise SMTP without reaching original recipients. Capture takes precedence when both are set.*

> `TINA4_MAIL_HOST`, `TINA4_MAIL_PORT`, `TINA4_MAIL_USERNAME`, `TINA4_MAIL_PASSWORD`, `TINA4_MAIL_FROM`, `TINA4_MAIL_FROM_NAME`, `TINA4_MAIL_IMAP_HOST`, `TINA4_MAIL_IMAP_PORT` *are accepted as legacy aliases. New projects should use the* `TINA4_MAIL_*` *names.*

Logging

Logs default to stdout in `text` format. Set `TINA4_LOG_OUTPUT=file` plus `TINA4_LOG_FILE=app.log` to write to disk; the framework rotates at `TINA4_LOG_ROTATE_SIZE` bytes and keeps `TINA4_LOG_ROTATE_KEEP` backups (`app.log.1` ... `app.log.N`). Switch `TINA4_LOG_FORMAT=json` for one structured record per line, perfect for shipping to Loki, Datadog, or any JSON-aware log aggregator.

Variable	Default	Description
`TINA4_LOG_LEVEL`	`INFO`	Console severity threshold. Options: `ALL`, `DEBUG`, `INFO`, `WARNING`, `ERROR`, `CRITICAL`, `NONE`.
`TINA4_LOG_FILE_LEVEL`	`ALL`	Independent severity threshold for the file sink.
`TINA4_LOG_STRICT`	`false`	Raise when a configured log sink fails instead of continuing with the remaining sinks. Accepts only `true` or `false`.
`TINA4_LOG_FILE`	*(empty - stdout only)*	Path to a log file. Empty leaves logs on stdout. Relative paths are resolved against `TINA4_LOG_DIR`; absolute paths are used verbatim.
`TINA4_LOG_DIR`	`logs`	Directory for log files. Joined with `TINA4_LOG_FILE` when the latter is a relative path.
`TINA4_LOG_FORMAT`	`text`	Output format. `text` writes the human-readable `[INFO] message` form; `json` writes one structured JSON record per line.
`TINA4_LOG_OUTPUT`	`stdout`	Where logs go. Options: `stdout`, `file`, `both`.

`TINA4_LOG_CRITICAL`	`false`	Enables the `Log.critical(...)` level above `error`. When off, calls to `Log.critical()` are silent no-ops.
`TINA4_LOG_ROTATE_SIZE`	`10485760`	Bytes per file before rotation (default 10 MB). `0` disables rotation entirely.
`TINA4_LOG_ROTATE_KEEP`	`5`	Number of rotated files to keep (`app.log.1` ... `app.log.N`). Older files are deleted on the next rotation.
`TINA4_LOG_MAX_SIZE`	`10485760`	Legacy alias for `TINA4_LOG_ROTATE_SIZE`. Per-file log size limit in bytes (10 MB). Rotated when exceeded.
`TINA4_LOG_KEEP`	`5`	Legacy alias for `TINA4_LOG_ROTATE_KEEP`. Number of rotated log files to retain.

Uploads

Variable	Default	Description
`TINA4_MAX_UPLOAD_SIZE`	`10485760`	Maximum multipart upload size in bytes (10 MB).

Localisation

Variable	Default	Description
`TINA4_LOCALE`	`en`	Default locale for the I18n module.
`TINA4_LOCALE_DIR`	`src/locale`	Directory containing locale JSON files.

Services (background tasks)

Variable	Default	Description
`TINA4_SERVICE_DIR`	`src/services`	Directory scanned for service classes.

Application AI Client and Developer AI Tools

The app-facing `Ai` client and the developer dashboard share the `TINA4_AI_*` namespace. The client supports local OpenAI-compatible servers, OpenAI, and Anthropic. See Chapter 40: AI Client for code examples.

Variable	Default	Description
`TINA4_AI_PROVIDER`	`local`	Provider used by the app-facing client: `local`, `openai`, or `anthropic`.
`TINA4_AI_URL`	Provider-specific	Base URL or full endpoint. Local defaults to `http://localhost:11437`; OpenAI and Anthropic use their public API bases.
`TINA4_AI_MODEL`	Provider-specific	Default model. Local uses `llama3.2`, OpenAI uses `gpt-4o-mini`, and Anthropic uses `claude-3-5-haiku-latest`.
`TINA4_AI_KEY`	*(none)*	Required before an OpenAI or Anthropic request. Local requests need no key.
`TINA4_AI_TIMEOUT`	`60`	Total request deadline in seconds, including retries and response reads.
`TINA4_AI_CONNECT_TIMEOUT`	`10`	Connection-establishment deadline in seconds.
`TINA4_AI_MAX_RETRIES`	`2`	Retries before output for connection failures, HTTP 429, and HTTP 5xx.
`TINA4_EMBED_URL`	*(inherits `TINA4_AI_URL`)*	Optional embedding base URL or full endpoint.

`TINA4_RAG_URL`	`http://localhost:11438`	RAG service used by the developer dashboard.
`TINA4_RAG_TOPK`	`4`	Number of RAG matches returned per query.
`TINA4_VISION_URL`	`http://localhost:11437/api/chat`	Vision endpoint used by developer tools, not the app-facing AI client.
`TINA4_IMAGE_URL`	`http://localhost:11437/api/generate`	Image endpoint used by developer tools, not the app-facing AI client.
`TINA4_SUPERVISOR_URL`	`http://localhost:9999`	Rust supervisor URL used by developer tools.
`TINA4_MCP_REMOTE`	`false`	Allows MCP to bind beyond localhost. Do not enable it on a public production interface.
`TINA4_NO_AI_PORT`	`false`	Disables the developer AI port listener.
`TINA4_OVERRIDE_CLIENT`	`false`	Lets the framework start without the Rust client in containers and CI.

Swagger / OpenAPI

Variable	Default	Description
`TINA4_SWAGGER_TITLE`	`Tina4 API`	OpenAPI spec title.
`TINA4_SWAGGER_DESCRIPTION`	*(empty)*	OpenAPI spec description.
`TINA4_SWAGGER_VERSION`	`1.0.0`	OpenAPI spec version.
`TINA4_SWAGGER_ENABLED`	*(inherits `TINA4_DEBUG`)*	Toggle the Swagger UI on or off independently of debug mode. Set `true` in production to keep API docs available without enabling the rest of the dev surface.
`TINA4_SWAGGER_CONTACT_EMAIL`	*(empty)*	Contact email rendered in the OpenAPI spec's `info.contact.email` field.
`TINA4_SWAGGER_LICENSE`	*(empty)*	License name in the OpenAPI spec's `info.license.name` field (e.g. `MIT`, `Apache-2.0`).

GraphQL

Variable	Default	Description
`TINA4_GRAPHQL_ENDPOINT`	`/graphql`	URL path the GraphQL handler is mounted on. POST serves queries, GET serves the GraphiQL IDE.
`TINA4_GRAPHQL_AUTO_SCHEMA`	`true`	When `true`, the framework auto-builds the GraphQL schema from every registered ORM model on boot. Set `false` if you want to define the schema manually with `gql.schema.add_type(...)`.

MCP (Model Context Protocol)

Variable	Default	Description
`TINA4_MCP`	*(inherits `TINA4_DEBUG`)*	Toggle the built-in MCP dev-tools server. Set explicitly to keep the MCP endpoint exposed in a debug-disabled deployment (e.g. for a remote AI assistant).
`TINA4_MCP_PORT`	*(framework port + 2000)*	TCP port for the MCP server. The default offset keeps it clear of both the main server (default `7145`) and the AI test port (`+1000`).

Configuration Recipes

The tables above list every knob. These are the setups most apps actually reach for, ready to paste into `.env`. Each block sets only what the feature needs. Everything else keeps its default.

PostgreSQL in production

One URL points the ORM, the migrations, and the query builder at Postgres. Credentials can ride in the URL or sit in their own variables, which keeps the password out of your shell history.

```
TINA4_DATABASE_URL=postgresql://localhost:5432/myapp
TINA4_DATABASE_USERNAME=myapp
TINA4_DATABASE_PASSWORD=changeme
TINA4_DB_POOL=4
```

`TINA4_DB_POOL=4` opens four connections and rotates across them. Leave it at `0` for a single connection on a small app.

A shared cache on Redis

The response cache and the cross-request query cache both speak to the same Redis. Point them at it and every instance shares one cache, invalidated globally on every write.

```
TINA4_CACHE_BACKEND=redis
TINA4_CACHE_URL=redis://localhost:6379
TINA4_DB_CACHE=true
TINA4_DB_CACHE_BACKEND=redis
TINA4_DB_CACHE_URL=redis://localhost:6379
```

If Redis is down or the driver is missing, the cache logs a warning and falls back to the file backend. It never silently stops caching.

Sessions that survive more than one box

The file backend is fine for a single server. Move sessions to Redis the moment you run more than one instance, so a user stays logged in whichever instance answers the next request.

```
TINA4_SESSION_BACKEND=redis
TINA4_SESSION_REDIS_HOST=localhost
TINA4_SESSION_REDIS_PORT=6379
TINA4_SESSION_SECURE=true
TINA4_SESSION_SAMESITE=Strict
```

`TINA4_SESSION_SECURE=true` keeps the cookie off plain HTTP. Turn it on once you have TLS.

A queue on RabbitMQ or Kafka

One URL is enough for RabbitMQ; the per-field variables only exist for split configs. The queue API stays identical whichever backend you pick.

```
TINA4_QUEUE_BACKEND=rabbitmq
TINA4_QUEUE_URL=amqp://guest:guest@localhost:5672/
```

Kafka reads a broker list instead:

```
TINA4_QUEUE_BACKEND=kafka
TINA4_KAFKA_BROKERS=localhost:9092
TINA4_KAFKA_GROUP_ID=myapp_workers
```

WebSocket broadcasts across instances

A single server broadcasts in memory. Add a backplane and a message sent on one instance reaches clients connected to every other instance.

```
TINA4_WS_BACKPLANE=redis
TINA4_WS_BACKPLANE_URL=redis://localhost:6379
TINA4_WS_ALLOWED_ORIGINS=https://myapp.com
```

Set the origin allow-list in production. Empty allows every origin, which is fine in dev and risky on the public internet.

Locked-down production headers

The defaults are already safe. These four tighten the screws for a public site on HTTPS.

```
TINA4_CORS_ORIGINS=https://myapp.com,https://www.myapp.com
TINA4_HSTS=31536000
TINA4_FRAME_OPTIONS=DENY
TINA4_SESSION_SECURE=true
```

Never pair `TINA4_CORS_CREDENTIALS=true` with `TINA4_CORS_ORIGINS=*`. Name your real origins instead.

The dev dashboard AI, kept local

The dashboard AI talks to a local model through Ollama by default, so nothing leaves your machine. Point the URLs elsewhere only when you run the hosted Tina4 AI services.

```
TINA4_AI_URL=http://localhost:11437/api/chat
TINA4_AI_MODEL=qwen2.5-coder:14b
```

```
TINA4_RAG_URL=http://localhost:11438
```

Minimal `.env` for Development

```
TINA4_DEBUG=true
TINA4_LOG_LEVEL=DEBUG
TINA4_NO_BROWSER=true
```

Debug mode lights up the Swagger UI, the dev dashboard, detailed error pages, and live reload. Keeping the browser flag on stops a new tab opening every time you save a file.

Minimal `.env` for Production

```
TINA4_SECRET=your-long-random-secret-here
TINA4_DATABASE_URL=postgresql://user:password@db-host:5432/myapp
TINA4_CORS_ORIGINS=https://myapp.com,https://www.myapp.com
TINA4_HSTS=31536000
TINA4_MAIL_HOST=smtp.example.com
TINA4_MAIL_PORT=587
TINA4_MAIL_USERNAME=noreply@myapp.com
TINA4_MAIL_PASSWORD=your-smtp-password
TINA4_MAIL_FROM=noreply@myapp.com
```

No `TINA4_DEBUG`. It defaults to `false`, which is what you want in production. Set a real secret, a real database, locked-down CORS origins, HSTS, and SMTP credentials if you send email. Everything else has a production-appropriate default.

Feature Module Settings

These settings are read by focused framework modules rather than the server bootstrap. They remain part of the public configuration surface.

Variable	Default	Description
`TINA4_DEFAULT_WEBSERVER`	`false`	Use Tina4's built-in asyncio server even when a production ASGI server is installed.
`TINA4_FEEDBACK_DEV_USER`	*(empty)*	Local-development identity for testing the feedback widget without authentication. Never use this as production authentication.
`TINA4_KAFKA_ASSIGN_TIMEOUT`	`15`	Seconds the first Kafka dequeue waits for partition assignment. Later polls use the normal short interval.

`TINA4_MCP_URL`	`https://mcp.tina4.com`	Grounding service URL shown and used by the developer admin.
`TINA4_SESSION_MEMCACHED_HOST`	`localhost`	Memcached host for the session backend.
`TINA4_SESSION_MEMCACHED_PORT`	`11211`	Memcached port for the session backend.
`TINA4_SESSION_MEMCACHED_PREFIX`	`tina4:session:`	Key prefix for Memcached sessions. Give each application a distinct prefix when sharing a server.
`TINA4_SESSION_VALKEY_PREFIX`	`tina4:session:`	Key prefix for Valkey sessions.
`TINA4_SWAGGER_CONTACT_TEAM`	*(empty)*	Contact name placed in the generated OpenAPI `info.contact` object.
`TINA4_SWAGGER_CONTACT_URL`	*(empty)*	Contact URL placed in the generated OpenAPI `info.contact` object.
`TINA4_TRUSTED_PROXIES`	*(empty)*	Comma-separated IP addresses and CIDR ranges allowed to supply forwarding headers. Empty trusts no proxy.

Deployment

1. From Development to Production

The app works on `localhost:7146`. Now it needs to run around the clock on a real server. Handle thousands of concurrent users. Survive restarts. Hold steady on memory. The gap between "works on my machine" and "works in production" is where projects stumble.

This chapter covers everything for a production deployment: environment configuration, ASGI server setup, Docker packaging, health checks, graceful shutdown, SSL/TLS, scaling, and monitoring.

When you run `tina4 init`, the framework generates a production-ready `Dockerfile` and `.dockerignore` in your project root. The Dockerfile uses a multi-stage build: the first stage installs dependencies and the second stage copies only runtime artifacts into a slim image. You do not need to write a Dockerfile from scratch -- the generated one is a solid starting point.

---

2. Production .env Configuration

Development defaults optimize for debugging. Production defaults optimize for performance and security. The first deployment step: configure `.env` for production.

Create a production `.env`:

```
# Core
TINA4_DEBUG=false
TINA4_LOG_LEVEL=WARNING
TINA4_PORT=7146

# Database
TINA4_DATABASE_URL=sqlite:///data/app.db

# Security
TINA4_CORS_ORIGINS=https://yourdomain.com
JWT_SECRET=your-long-random-secret-at-least-32-characters
TINA4_RATE_LIMIT=120

# Performance
TINA4_TEMPLATE_CACHE_TTL=true
```

Key Differences from Development

Setting	Development	Production	Why
`TINA4_DEBUG`	`true`	`false`	Hides stack traces, disables toolbar
`TINA4_LOG_LEVEL`	`ALL`	`WARNING`	Reduces log noise
`TINA4_CORS_ORIGINS`	Your dev front end (for example `http://localhost:3000`), or empty	Your domain	Only listed origins can call the API from a browser

Sensitive Values

Production secrets never go into version control. The `.env` file is gitignored by default. For deployment, use environment variables from your hosting platform, CI/CD secrets, or a secrets manager.

```
# Docker: pass env vars at runtime
docker run -e JWT_SECRET=your-secret -e TINA4_DATABASE_URL=sqlite:///data/app.db my-app

# Fly.io: set secrets
fly secrets set JWT_SECRET=your-secret

# Railway: use the dashboard or CLI
railway variables set JWT_SECRET=your-secret
```

---

3. ASGI Server Auto-Detection

Tina4 Python auto-detects ASGI servers at startup. ASGI servers deliver production-grade performance:

- Multiple worker processes
- Async request handling
- Graceful shutdown
- Better error recovery

How Auto-Detection Works

When you run `tina4 serve` or `tina4 serve --production`, Tina4 checks for installed ASGI servers:

- If `uvicorn` is installed, the framework uses uvicorn with optimal settings
- If `hypercorn` is installed, the framework uses hypercorn
- If neither is installed, the framework falls back to the built-in development server

```
# Install uvicorn for production
uv add uvicorn
```

```
# With uvicorn installed
tina4 serve

  Tina4 Python v3.0.0
  Server: uvicorn (4 workers)
  Running at http://0.0.0.0:7146

# Without uvicorn
tina4 serve

  Tina4 Python v3.0.0
  Server: built-in (development)
  Running at http://0.0.0.0:7146
  WARNING: Do not use the built-in server in production
```

Uvicorn Configuration

Fine-tune uvicorn through environment variables:

Or pass options directly:

```
uvicorn app:app --workers 4 --host 0.0.0.0 --port 7146
```

How Many Workers?

Start with `(2 * CPU cores) + 1`:

```
import multiprocessing
workers = (2 * multiprocessing.cpu_count()) + 1
```

CPU Cores	Workers	Use Case
1	3	Small VPS, hobbyist
2	5	Small production app
4	9	Medium production app
8	17	High-traffic app

---

4. Docker Deployment

Docker is the most portable deployment path. Your app runs the same way on your laptop, in CI, and on the production server.

Official Base Image

Tina4 Python provides an official Docker Hub base image: `tina4stack/tina4-python:v3`. It is a lean, Alpine-based image (~56MB) with Python 3.13, SQLite, and the Tina4 framework pre-installed. Your app Dockerfile extends it and adds only your application code.

The base image includes these environment variables pre-configured:

- `TINA4_OVERRIDE_CLIENT=true` - bypasses the CLI guard for Docker
- `TINA4_DEBUG=false` - production mode by default

- `PYTHONUNBUFFERED=1` - ensures logs appear in `docker logs`
- `TINA4_NO_BROWSER=true` - prevents browser auto-open attempts
- `HOST=0.0.0.0` and `PORT=7146` - server binds to all interfaces

Dockerfile

Create `Dockerfile`:

```
FROM tina4stack/tina4-python:v3
WORKDIR /app

# Copy application code
COPY app.py .
COPY .env .
COPY migrations/ migrations/
COPY src/ src/

# Create data directories for SQLite, sessions, queue, and mailbox
RUN mkdir -p data data/sessions data/queue data/mailbox

EXPOSE 7146
CMD ["python", "app.py"]
```

That is the entire Dockerfile. The base image handles Python, dependencies, and framework setup.

.dockerignore

Create `.dockerignore`:

```
.venv
__pycache__
*.pyc
data/
tests/
.tina4/
.DS_Store
*.db
*.db-wal
*.db-shm
logs/
.git
.env.development
.pytest_cache
htmlcov
.claude
```

Building and Running

```
# Build the image
docker build -t my-tina4-app .

# Run the container
docker run -d \
  --name my-app \
  -p 7146:7146 \
  -e JWT_SECRET=your-production-secret \
  -e TINA4_DATABASE_URL=sqlite:///data/app.db \
  -v $(pwd)/data:/app/data \
  my-tina4-app
```

Adding Database Drivers

The base image ships with SQLite only. To use PostgreSQL, MySQL, MSSQL, or Firebird, install the driver in your Dockerfile.

PostgreSQL:

```
FROM tina4stack/tina4-python:v3
WORKDIR /app
RUN python -m pip install --no-cache-dir psycopg2-binary
COPY app.py .
COPY .env .
COPY migrations/ migrations/
COPY src/ src/
RUN mkdir -p data data/sessions data/queue data/mailbox
EXPOSE 7146
CMD ["python", "app.py"]
```

MySQL:

```
FROM tina4stack/tina4-python:v3
WORKDIR /app
RUN apk add --no-cache mariadb-connector-c-dev && \
    python -m pip install --no-cache-dir mysqlclient
COPY app.py .
COPY .env .
COPY migrations/ migrations/
COPY src/ src/
RUN mkdir -p data data/sessions data/queue data/mailbox
EXPOSE 7146
CMD ["python", "app.py"]
```

MSSQL:

```
FROM tina4stack/tina4-python:v3
WORKDIR /app
RUN apk add --no-cache unixodbc-dev freetds-dev && \
    python -m pip install --no-cache-dir pymssql
COPY app.py .
COPY .env .
COPY migrations/ migrations/
COPY src/ src/
RUN mkdir -p data data/sessions data/queue data/mailbox
EXPOSE 7146
CMD ["python", "app.py"]
```

Firebird:

```
FROM tina4stack/tina4-python:v3
WORKDIR /app
# Pure Python driver - no system dependencies needed
RUN python -m pip install --no-cache-dir firebird-driver
COPY app.py .
COPY .env .
COPY migrations/ migrations/
COPY src/ src/
RUN mkdir -p data data/sessions data/queue data/mailbox
EXPOSE 7146
CMD ["python", "app.py"]
```

Docker Compose

For a complete setup with supporting services, use Docker Compose.

Create `docker-compose.yml`:

```
services:
  app:
    build: .
    ports:
      - "7146:7146"
    environment:
      - TINA4_DEBUG=false
      - TINA4_LOG_LEVEL=WARNING
      - JWT_SECRET=${JWT_SECRET}
      - TINA4_DATABASE_URL=sqlite:///data/app.db
    volumes:
      - app-data:/app/data
    restart: unless-stopped
    healthcheck:
      test: ["CMD", "python", "-c", "import urllib.request; urllib.request.urlopen('http://local
      interval: 30s
      timeout: 5s
      retries: 3

volumes:
  app-data:
```

```
# Start everything
docker compose up -d

# View logs
docker compose logs -f app

# Stop everything
docker compose down
```

---

5. Health Checks

Production deployments need a health check endpoint. Load balancers, container orchestrators, and monitoring tools all rely on it to verify the app is running.

Create a health check route:

```
from datetime import datetime, timezone
from tina4_python.core.router import get

@get("/health")
async def health_check(request, response):
    db = Database.get_connection()
    db_ok = False

    try:
        db.fetch_one("SELECT 1")
        db_ok = True
    except Exception:
        pass

    status = "ok" if db_ok else "degraded"
    status_code = 200 if db_ok else 503

    return response({
        "status": status,
        "version": "1.0.0",
        "database": "connected" if db_ok else "disconnected",
        "timestamp": datetime.now(timezone.utc).isoformat()
    }, status_code)
```

This endpoint:

- Returns `200` when everything is healthy
- Returns `503` when the database is down (so the load balancer stops routing traffic)
- Includes version information for deployment tracking
- Runs fast (no heavy queries, no authentication)

---

6. Graceful Shutdown

Deploy a new version. The old process must stop. Graceful shutdown finishes active requests before terminating.

Tina4 handles this when it receives a `SIGTERM` signal (the standard shutdown signal from Docker, Kubernetes, and systemd):

- Stop accepting new connections
- Wait for active requests to complete (up to 30 seconds)
- Close database connections
- Flush logs
- Exit

Configuring Shutdown Timeout

```
TINA4_SHUTDOWN_TIMEOUT=30
```

If active requests do not finish within the timeout, the server terminates them. Set this based on your longest expected request time. For most applications, 30 seconds is generous.

Docker Stop Grace Period

Docker sends `SIGTERM`, waits for the grace period, then sends `SIGKILL`. Match the Docker grace period to your shutdown timeout:

```
services:
  app:
    stop_grace_period: 30s
```

---

7. Log Rotation

In production, logs grow without limit unless rotated. Tina4 writes logs to `logs/app.log` and `logs/error.log`.

Using logrotate (Linux)

Create `/etc/logrotate.d/tina4`:

```
/app/logs/*.log {
    daily
    rotate 14
    compress
    delaycompress
    missingok
    notifempty
    create 0640 www-data www-data
    postrotate
        kill -USR1 $(cat /app/tina4.pid) 2>/dev/null || true
    endscript
}
```

This rotates logs daily, keeps 14 days of history, and compresses old logs. The `USR1` signal tells Tina4 to reopen its log files after rotation.

Docker Logging

Docker captures stdout/stderr automatically. Configure log rotation in the Docker daemon or compose file:

```
services:
  app:
    logging:
      driver: json-file
      options:
        max-size: "10m"
        max-file: "5"
```

This keeps at most 50MB of logs (5 files of 10MB each).

---

8. Reverse Proxy with Nginx

In production, Tina4 runs behind Nginx. Nginx handles:

- SSL/TLS termination (HTTPS)

- Static file serving (faster than Python)
- Request buffering
- Rate limiting
- WebSocket proxying

Create `/etc/nginx/sites-available/my-app`:

```
server {
    listen 80;
    server_name yourdomain.com;
    return 301 https://$host$request_uri;
}

server {
    listen 443 ssl;
    server_name yourdomain.com;

    ssl_certificate /etc/letsencrypt/live/yourdomain.com/fullchain.pem;
    ssl_certificate_key /etc/letsencrypt/live/yourdomain.com/privkey.pem;

    # Static files (served by Nginx, faster than Python)
    location /css/ {
        alias /app/src/public/css/;
        expires 30d;
        add_header Cache-Control "public, immutable";
    }

    location /js/ {
        alias /app/src/public/js/;
        expires 30d;
        add_header Cache-Control "public, immutable";
    }

    location /images/ {
        alias /app/src/public/images/;
        expires 30d;
        add_header Cache-Control "public, immutable";
    }

    # WebSocket upgrade
    location /ws/ {
        proxy_pass http://127.0.0.1:7146;
        proxy_http_version 1.1;
        proxy_set_header Upgrade $http_upgrade;
        proxy_set_header Connection "upgrade";
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_read_timeout 86400;
    }

    # Application
    location / {
        proxy_pass http://127.0.0.1:7146;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
```

```
    }
}
```

---

9. SSL/TLS with Let's Encrypt

HTTPS is non-negotiable in production. Let's Encrypt provides free SSL certificates with automatic renewal.

With Nginx and Certbot

Install Certbot:

```
sudo apt install certbot python3-certbot-nginx
```

Obtain a certificate:

```
sudo certbot --nginx -d yourdomain.com
```

Certbot modifies your Nginx configuration to include SSL settings and sets up automatic renewal. Verify auto-renewal works:

```
sudo certbot renew --dry-run
```

Certbot renews certificates 30 days before expiry. The renewal runs via a systemd timer or cron job that Certbot creates during installation.

With Docker (Using Traefik as Reverse Proxy)

Traefik handles SSL termination and automatic certificate provisioning. Add it to your Docker Compose setup:

```
services:
  reverse-proxy:
    image: traefik:v3.0
    command:
      - "--providers.docker=true"
      - "--entrypoints.web.address=:80"
      - "--entrypoints.websecure.address=:443"
      - "--certificatesresolvers.letsencrypt.acme.tlschallenge=true"
      - "--certificatesresolvers.letsencrypt.acme.email=you@example.com"
      - "--certificatesresolvers.letsencrypt.acme.storage=/letsencrypt/acme.json"
    ports:
      - "80:80"
      - "443:443"
    volumes:
      - /var/run/docker.sock:/var/run/docker.sock:ro
      - letsencrypt:/letsencrypt

  app:
    build: .
    labels:
      - "traefik.http.routers.app.rule=Host(`yourdomain.com`)"
      - "traefik.http.routers.app.tls=true"
      - "traefik.http.routers.app.tls.certresolver=letsencrypt"
    environment:
      - TINA4_DEBUG=false
      - JWT_SECRET=${JWT_SECRET}
```

```
volumes:
  letsencrypt:
```

Traefik detects your app container through Docker labels, provisions a certificate from Let's Encrypt, and handles all HTTPS traffic. No Nginx configuration needed.

Certificate Monitoring

Certificates expire. Even with auto-renewal, things go wrong -- DNS changes, firewall rules blocking port 80 for ACME challenges, or a crashed renewal service. Set up monitoring:

```
# Check certificate expiry manually
echo | openssl s_client -connect yourdomain.com:443 2>/dev/null | openssl x509 -noout -dates

# Verify Certbot timer is active
sudo systemctl list-timers | grep certbot
```

Use an external monitoring service (Uptime Robot, Better Uptime) that checks certificate expiry and alerts you 14 days before it expires.

---

10. Scaling

A single server handles many applications. When traffic outgrows one server, you scale.

Multiple Workers

Uvicorn runs multiple worker processes by default. Configure the count in `.env`:

Start with the number of CPU cores on your server. For I/O-heavy applications (database queries, external API calls), double or quadruple the core count. CPU-bound work benefits less from extra workers.

Load Balancing with Nginx

When you run multiple Tina4 instances, Nginx distributes traffic across them:

```
upstream tina4_backend {
    server 127.0.0.1:7146;
    server 127.0.0.1:7246;
    server 127.0.0.1:7346;
    server 127.0.0.1:7446;
}

server {
    listen 80;
    server_name yourdomain.com;

    location / {
        proxy_pass http://tina4_backend;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
    }
}
```

Start four instances on different ports. The framework refuses to start without the Rust CLI unless `TINA4_OVERRIDE_CLIENT=true`, so for a multi-process setup either set that override or run each instance through `tina4 serve --port`:

```
TINA4_PORT=7146 tina4 serve --port 7146 &
TINA4_PORT=7246 tina4 serve --port 7246 &
TINA4_PORT=7346 tina4 serve --port 7346 &
TINA4_PORT=7446 tina4 serve --port 7446 &
```

Nginx distributes requests in round-robin order by default. If a backend goes down, Nginx routes traffic to the remaining instances.

Docker Scaling

With Docker Compose, scale horizontally with a single command:

```
docker compose up -d --scale app=4
```

This starts four containers. Place a load balancer (Traefik, Nginx, or a cloud load balancer) in front of them:

```
services:
  reverse-proxy:
    image: traefik:v3.0
    command:
      - "--providers.docker=true"
      - "--entrypoints.web.address=:80"
    ports:
      - "80:80"
    volumes:
      - /var/run/docker.sock:/var/run/docker.sock:ro

  app:
    build: .
    labels:
      - "traefik.http.routers.app.rule=Host(`yourdomain.com`)"
    environment:
      - TINA4_DEBUG=false
      - TINA4_DATABASE_URL=sqlite:///data/app.db
    volumes:
      - app-data:/app/data

volumes:
  app-data:
```

Scaling Considerations

Scaling introduces shared-state problems. When four instances serve requests, each must agree on the state of the world.

Sessions: Store sessions in Redis, not in-memory. Otherwise, a user who logs in on instance 1 appears logged out on instance 2.

Database: SQLite handles one writer at a time. Under high load with multiple instances, switch to PostgreSQL or MySQL. If you must use SQLite, enable WAL mode.

File uploads: Store uploaded files in shared storage (S3, a mounted volume) -- not the local filesystem of a single container.

Cache: Use Redis as the cache backend so all instances share the same cache.

---

11. Monitoring

Your app runs in production. You need to know when it breaks, slows down, or runs out of resources.

Log Aggregation

Switch to JSON-formatted logs for production. Structured logs feed into aggregation services:

```
TINA4_LOG_FORMAT=json
```

Example JSON log entry:

```
{
  "timestamp": "2026-03-22T14:30:00.123Z",
  "level": "WARNING",
  "message": "Rate limit exceeded",
  "request_id": "req-abc123",
  "ip": "203.0.113.42",
  "path": "/api/products",
  "method": "GET"
}
```

Services that ingest JSON logs:

Service	Strengths
Grafana Loki	Open source, pairs with Grafana dashboards
Elastic Stack (ELK)	Full-text search across logs
Datadog	Managed service, correlates logs with metrics
AWS CloudWatch	Native for AWS deployments

Docker makes log aggregation straightforward. Configure the logging driver to send container output to your chosen service:

```
services:
  app:
    logging:
      driver: "fluentd"
      options:
        fluentd-address: "localhost:24224"
        tag: "tina4.app"
```

Uptime Monitoring

Point an external monitoring service at your health endpoint:

```
https://yourdomain.com/health
```

Services like Uptime Robot, Pingdom, or Better Uptime ping this endpoint every 30-60 seconds. When it stops responding or returns a non-200 status, you receive an alert via email, SMS, or

Slack.

The health endpoint from section 5 serves double duty: container orchestrators use it for restart decisions, and uptime monitors use it for alerting.

Application Performance Monitoring (APM)

Uptime monitoring tells you the app is running. APM tells you how well it performs. APM agents track:

- Request latency (average, p95, p99)
- Database query performance (slow queries, connection pool usage)
- Error rates (which endpoints fail, how often)
- Memory and CPU usage over time

Since Tina4 Python runs on standard Python, any Python APM agent works:

- **Datadog APM**: `uv add ddtrace` and run with `TINA4_OVERRIDE_CLIENT=true ddtrace-run uv run tina4 serve` (the override is required because APM wrappers spawn Python directly rather than going through the Rust CLI)
- **New Relic**: `uv add newrelic` and run with `TINA4_OVERRIDE_CLIENT=true newrelic-admin run-program uv run tina4 serve`
- **Elastic APM**: `uv add elastic-apm` and configure in your app startup, then `TINA4_OVERRIDE_CLIENT=true tina4 serve --production`

A basic monitoring stack for a small team: Uptime Robot for availability alerts (free tier covers it), JSON logs shipped to Grafana Loki for debugging, and `docker stats` for resource usage. Add APM when your application serves enough traffic to warrant the cost.

---

12. Exercise: Docker Deploy

Deploy a Tina4 Python application using Docker.

Requirements

- Create a `Dockerfile` that:
- Uses `tina4stack/tina4-python:v3` as the base image
- Copies the application code
- Creates data directories
- Exposes port 7146
- Create a `docker-compose.yml` that:
- Builds and runs the app
- Mounts volumes for data persistence
- Sets environment variables for production

- Create a `/health` endpoint that checks database connectivity
- Build, run, and verify:

```
# Build
docker compose build

# Start
docker compose up -d

# Test health
curl http://localhost:7146/health

# Test the app
curl http://localhost:7146/api/products

# View logs
docker compose logs -f app

# Stop
docker compose down
```

Solution

The Dockerfile and docker-compose.yml are shown in section 4. The health check route is shown in section 5. Combine them in your project, then:

```
docker compose up -d --build
```

```
[+] Building 4.2s
[+] Running 1/1
  Container my-app   Started
```

```
curl http://localhost:7146/health
```

```
{
  "status": "ok",
  "version": "1.0.0",
  "database": "connected",
  "timestamp": "2026-03-22T14:30:00+00:00"
}
```

The app runs in production mode with persistent data volumes, automatic restarts, and health monitoring.

HTTP Caching, Compression, and ETags

Tina4 applies gzip to eligible dynamic responses and then computes a strong ETag from the final post-compression bytes. The identity and gzip representations therefore have different ETags, and `Vary: Accept-Encoding` keeps intermediary caches from mixing them. A matching `If-None-Match` returns `304 Not Modified` with an empty body while preserving the validators.

Static files use a weak validator derived from file size and modification time. Streaming responses bypass dynamic compression and ETag generation because their complete representation is not buffered. Test both identity and gzip requests when validating cache behavior.

13. Gotchas

1. .env Not Loaded in Docker

Problem: Environment variables from `.env` are not available in the container.

Cause: Docker does not read `.env` files automatically. The `.env` file belongs in `.dockerignore` (never ship secrets in the image).

Fix: Pass environment variables via `docker run -e`, the `environment` section in `docker-compose.yml`, or an `env_file` directive. For secrets, use Docker secrets or your platform's secret management.

2. SQLite Database Lost on Container Restart

Problem: All data disappears when the container restarts.

Cause: The SQLite database file sits inside the container. When the container is recreated, the file is gone.

Fix: Mount a volume for the data directory: `-v $(pwd)/data:/app/data`. In Docker Compose, use a named volume: `volumes: [app-data:/app/data]`.

3. WebSocket Connections Drop Behind Nginx

Problem: WebSocket connections fail or drop immediately behind Nginx.

Cause: Nginx does not proxy WebSocket by default. It treats the upgrade request as a regular HTTP request.

Fix: Add WebSocket proxy headers in your Nginx config:

```
proxy_http_version 1.1;
proxy_set_header Upgrade $http_upgrade;
proxy_set_header Connection "upgrade";
proxy_read_timeout 86400;
```

4. Built-in Server Used in Production

Problem: The server logs show "WARNING: Do not use the built-in server in production".

Cause: No ASGI server (uvicorn or hypercorn) is installed. The built-in server is single-threaded and not designed for production load.

Fix: Install uvicorn: `uv add uvicorn`. Tina4 auto-detects it and uses it with multiple workers.

5. Static Files Slow

Problem: CSS, JS, and images load slowly in production.

Cause: Python serves static files. Every static file request goes through the Python process.

Fix: Serve static files from Nginx (see the Nginx config in section 8). Nginx serves static files from memory-mapped files, which is orders of magnitude faster than Python.

6. Memory Usage Grows Over Time

Problem: The container's memory usage climbs until it crashes with OOM (out of memory).

Cause: Memory leaks in the application -- unclosed database connections, growing caches without TTL, accumulating request data in global variables.

Fix: Set TTLs on all cache entries. Close database connections properly. Avoid storing request data in module-level variables. Use `docker stats` to monitor memory usage. Set memory limits in Docker Compose: `deploy: {resources: {limits: {memory: 512M}}}`.

7. Container Starts Before Database Is Ready

Problem: The app crashes on startup because the database is not ready.

Cause: Docker Compose starts services in parallel. The app container starts before the database container finishes initializing.

Fix: For SQLite, this is not an issue (the file is created automatically). For PostgreSQL or MySQL, use a startup script that waits for the database, or use Docker Compose healthcheck on the database service with `depends_on: {db: {condition: service_healthy}}`.

8. SSL Certificate Not Renewing

Problem: Your HTTPS certificate expires and the site goes down.

Cause: The auto-renewal process (Certbot or Traefik) failed. Common reasons: DNS changes, firewall blocking port 80 for ACME challenges, or the renewal service crashed.

Fix: Monitor certificate expiry with an external service. Check renewal logs and verify the renewal timer is active:

```
sudo certbot renew --dry-run
sudo systemctl list-timers | grep certbot
```

9. Scaled Instances Have Different State

Problem: Users see inconsistent data across requests when running multiple app instances.

Cause: In-memory sessions and cache are not shared between instances. A user who logs in on instance 1 appears logged out when the load balancer routes the next request to instance 2.

Fix: Store sessions in Redis. Use Redis as the cache backend. Store uploaded files in shared storage (S3 or a mounted volume). All instances must read from and write to the same data stores.

Building a Complete App

1. Putting It All Together

Twenty chapters of building blocks. Routing. Templates. Databases. ORM. Authentication. Middleware. Queues. WebSocket. Caching. Frontend. GraphQL. Testing. Dev tools. CLI scaffolding. Deployment. Now all of it works together in one application.

TaskFlow -- a task management system with:

- User registration and JWT authentication
- Task creation, assignment, and tracking
- A dashboard with real-time updates
- Email notifications when tasks are assigned
- Caching for dashboard performance
- A full test suite
- Docker deployment

Not a toy project. A production-ready application. Every major Tina4 feature in one codebase.

---

2. Planning the App

Models

Model	Table	Fields
User	users	id, name, email, password*hash, role, created*at
Task	tasks	id, title, description, status, priority, created*by, assigned*to, due*date, completed*at, created*at, updated*at

Relationships

- A User has many Tasks (created by them)
- A User has many Tasks (assigned to them)
- A Task belongs to a User (creator)
- A Task belongs to a User (assignee)

Routes

Method	Path	Description	Auth
POST	/api/auth/register	Register a new user	public
POST	/api/auth/login	Login, get JWT	public
GET	/api/profile	Get current user profile	secured
GET	/api/tasks	List tasks (with filters)	secured
GET	/api/tasks/{id}	Get a single task	secured
POST	/api/tasks	Create a task	secured
PUT	/api/tasks/{id}	Update a task	secured
DELETE	/api/tasks/{id}	Delete a task	secured
GET	/api/dashboard/stats	Dashboard statistics	secured
GET	/admin	Dashboard HTML page	public

Templates

- `base.html` -- Base layout with sidebar and topbar
- `dashboard.html` -- Dashboard with stats, task list, quick actions
- `login.html` -- Login page
- `register.html` -- Registration page

---

3. Step 1: Init Project and Set Up Database

```
tina4 init python taskflow
cd taskflow
uv sync
```

Update `.env`:

```
TINA4_DEBUG=true
TINA4_SECRET=taskflow-dev-secret-change-in-production
TINA4_TOKEN_EXPIRES_IN=1440
```

Create Migrations

Create `migrations/20260322000100_create_users_table.sql`:

```
-- UP
CREATE TABLE users (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    name TEXT NOT NULL,
    email TEXT NOT NULL UNIQUE,
```

```
    password_hash TEXT NOT NULL,
    role TEXT NOT NULL DEFAULT 'user',
    created_at TEXT DEFAULT CURRENT_TIMESTAMP
);

CREATE INDEX idx_users_email ON users(email);

-- DOWN
DROP TABLE IF EXISTS users;
```

Create `migrations/20260322000200_create_tasks_table.sql`:

```
-- UP
CREATE TABLE tasks (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    title TEXT NOT NULL,
    description TEXT,
    status TEXT NOT NULL DEFAULT 'pending',
    priority TEXT NOT NULL DEFAULT 'medium',
    created_by INTEGER NOT NULL,
    assigned_to INTEGER,
    due_date TEXT,
    completed_at TEXT,
    created_at TEXT DEFAULT CURRENT_TIMESTAMP,
    updated_at TEXT DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (created_by) REFERENCES users(id),
    FOREIGN KEY (assigned_to) REFERENCES users(id)
);

CREATE INDEX idx_tasks_status ON tasks(status);
CREATE INDEX idx_tasks_assigned_to ON tasks(assigned_to);
CREATE INDEX idx_tasks_created_by ON tasks(created_by);

-- DOWN
DROP TABLE IF EXISTS tasks;
```

Run migrations:

```
tina4 migrate
```

```
Running migrations...
  [UP] 20260322000100_create_users_table.sql
  [UP] 20260322000200_create_tasks_table.sql

  2 migrations applied
```

Verify the database:

```
curl http://localhost:7146/health
```

```
{"status": "ok", "database": "connected"}
```

---

4. Step 2: ORM Models

Create `src/orm/user.py`:

```
from tina4_python.orm import ORM
import hashlib
```

```
import hmac
import os

class User(ORM):
    table_name = "users"
    auto_crud = True

    id: int
    name: str
    email: str
    password_hash: str
    role: str
    created_at: str

    def set_password(self, password):
        salt = os.urandom(32)
        key = hashlib.pbkdf2_hmac("sha256", password.encode(), salt, 100000)
        self.password_hash = salt.hex() + ":" + key.hex()

    def verify_password(self, password):
        if not self.password_hash:
            return False
        salt_hex, key_hex = self.password_hash.split(":")
        salt = bytes.fromhex(salt_hex)
        key = bytes.fromhex(key_hex)
        new_key = hashlib.pbkdf2_hmac("sha256", password.encode(), salt, 100000)
        return hmac.compare_digest(key, new_key)

    def safe_dict(self):
        data = self.to_dict()
        data.pop("password_hash", None)
        return data
```

Create `src/orm/task.py`:

```
from tina4_python.orm import ORM

class Task(ORM):
    table_name = "tasks"
    auto_crud = True

    id: int
    title: str
    description: str
    status: str
    priority: str
    created_by: int
    assigned_to: int
    due_date: str
    completed_at: str
    created_at: str
    updated_at: str

    STATUSES = ["pending", "in_progress", "completed", "cancelled"]
    PRIORITIES = ["low", "medium", "high", "urgent"]
```

---

5. Step 3: Authentication Routes

Create `src/routes/auth.py`:

```
from tina4_python.core.router import post, get
from tina4_python.auth import Auth

@post("/api/auth/register")
async def register(request, response):
    body = request.body

    if not body.get("name") or not body.get("email") or not body.get("password"):
        return response({"error": "Name, email, and password are required"}, 400)

    # Check for existing user
    results, count = User.where("email = ?", [body["email"]])
    if results:
        return response({"error": "Email already registered"}, 409)

    user = User()
    user.name = body["name"]
    user.email = body["email"]
    user.set_password(body["password"])
    user.role = "user"
    user.save()

    return response({
        "message": "Registration successful",
        "id": user.id,
        "name": user.name,
        "email": user.email
    }, 201)


@post("/api/auth/login")
async def login(request, response):
    body = request.body

    if not body.get("email") or not body.get("password"):
        return response({"error": "Email and password are required"}, 400)

    results, count = User.where("email = ?", [body["email"]])

    if not results:
        return response({"error": "Invalid credentials"}, 401)

    user = results[0]
    if not user.verify_password(body["password"]):
        return response({"error": "Invalid credentials"}, 401)

    token = Auth.get_token({
        "user_id": user.id,
        "email": user.email,
        "role": user.role
    })

    return response({
        "token": token,
        "user": user.safe_dict()
```

```
    })
```

Test Registration and Login

```
# Start the server
tina4 serve
```

```
# Register a user
curl -X POST http://localhost:7146/api/auth/register \
  -H "Content-Type: application/json" \
  -d '{"name": "Alice Johnson", "email": "alice@example.com", "password": "securepass123"}'
```

```
{
  "message": "Registration successful",
  "id": 1,
  "name": "Alice Johnson",
  "email": "alice@example.com"
}
```

```
# Login
curl -X POST http://localhost:7146/api/auth/login \
  -H "Content-Type: application/json" \
  -d '{"email": "alice@example.com", "password": "securepass123"}'
```

```
{
  "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...",
  "user": {
    "id": 1,
    "name": "Alice Johnson",
    "email": "alice@example.com",
    "role": "user"
  }
}
```

Authentication works. Save the token for the next steps.

---

6. Step 4: Auth Middleware

Create `src/middleware/auth.py`:

```
from tina4_python.auth import Auth

async def auth_middleware(request, response, next_handler):
    auth_header = request.headers.get("Authorization", "")

    if not auth_header.startswith("Bearer "):
        return response({"error": "Authentication required"}, 401)

    token = auth_header.replace("Bearer ", "")

    payload = Auth.valid_token(token)
    if payload is None:
        return response({"error": "Invalid or expired token"}, 401)

    request.user = payload
    request.user_id = payload["user_id"]

    return await next_handler(request, response)
```

Verify the middleware blocks unauthenticated requests:

```
curl http://localhost:7146/api/tasks

{"error": "Authentication required"}
```

The middleware stands guard. No token, no access.

---

7. Step 5: Task CRUD Routes

Create `src/routes/tasks.py`:

```
from datetime import datetime, timezone
from tina4_python.core.router import get, post, put, delete
from tina4_python.cache import cache_get, cache_set, cache_delete

@get("/api/tasks", middleware=["auth_middleware"])
async def list_tasks(request, response):
    status = request.params.get("status")
    assigned_to = request.params.get("assigned_to")
    page = int(request.params.get("page", 1))
    limit = int(request.params.get("limit", 20))
    offset = (page - 1) * limit

    where = "1=1"
    params = []

    if status:
        where += " AND status = ?"
        params.append(status)
    if assigned_to:
        where += " AND assigned_to = ?"
        params.append(int(assigned_to))

    tasks, total = Task.where(where, params, limit=limit, offset=offset)

    return response({
        "tasks": [t.to_dict() for t in tasks],
        "page": page,
        "limit": limit
    })


@get("/api/tasks/{task_id}", middleware=["auth_middleware"])
async def get_task(request, response):
    task_id = request.params["task_id"]

    task = Task.find(task_id)

    if task is None:
        return response({"error": "Task not found"}, 404)

    # Load creator and assignee names
    creator = User.find(task.created_by)

    result = task.to_dict()
    result["creator_name"] = creator.name if creator else "Unknown"
```

```
    if task.assigned_to:
        assignee = User.find(task.assigned_to)
        result["assignee_name"] = assignee.name if assignee else "Unassigned"

    return response(result)


@post("/api/tasks", middleware=["auth_middleware"])
async def create_task(request, response):
    body = request.body

    if not body.get("title"):
        return response({"error": "Title is required"}, 400)

    task = Task()
    task.title = body["title"]
    task.description = body.get("description", "")
    task.status = body.get("status", "pending")
    task.priority = body.get("priority", "medium")
    task.created_by = request.user_id
    task.assigned_to = body.get("assigned_to")
    task.due_date = body.get("due_date")
    task.save()

    # Invalidate dashboard cache
    cache_delete("dashboard:stats")

    # Send email notification if assigned
    if task.assigned_to:
        await notify_assignee(task)

    # Push WebSocket update
    await push_task_update("created", task)

    return response(task.to_dict(), 201)


@put("/api/tasks/{task_id}", middleware=["auth_middleware"])
async def update_task(request, response):
    task_id = request.params["task_id"]
    body = request.body

    task = Task.find(task_id)

    if task is None:
        return response({"error": "Task not found"}, 404)

    if "title" in body:
        task.title = body["title"]
    if "description" in body:
        task.description = body["description"]
    if "status" in body:
        if body["status"] not in Task.STATUSES:
            return response({"error": f"Invalid status. Must be one of: {Task.STATUSES}"}, 400)
        task.status = body["status"]
        if body["status"] == "completed":
            task.completed_at = datetime.now(timezone.utc).isoformat()
    if "priority" in body:
        if body["priority"] not in Task.PRIORITIES:
```

```
            return response({"error": f"Invalid priority. Must be one of: {Task.PRIORITIES}"}, 4
        task.priority = body["priority"]
    if "assigned_to" in body:
        old_assignee = task.assigned_to
        task.assigned_to = body["assigned_to"]
        if body["assigned_to"] != old_assignee and body["assigned_to"]:
            await notify_assignee(task)
    if "due_date" in body:
        task.due_date = body["due_date"]

    task.updated_at = datetime.now(timezone.utc).isoformat()
    task.save()

    # Invalidate dashboard cache
    cache_delete("dashboard:stats")

    # Push WebSocket update
    await push_task_update("updated", task)

    return response(task.to_dict())


@delete("/api/tasks/{task_id}", middleware=["auth_middleware"])
async def delete_task(request, response):
    task_id = request.params["task_id"]

    task = Task.find(task_id)

    if task is None:
        return response({"error": "Task not found"}, 404)

    task.delete()

    cache_delete("dashboard:stats")

    return response(None, 204)
```

Test the Task API

```
# Set your token from the login step
TOKEN="eyJhbGciOiJIUzI1NiIs..."

# Create a task
curl -X POST http://localhost:7146/api/tasks \
  -H "Content-Type: application/json" \
  -H "Authorization: Bearer $TOKEN" \
  -d '{"title": "Design database schema", "priority": "high"}'
```

```
{
  "id": 1,
  "title": "Design database schema",
  "priority": "high",
  "status": "pending",
  "created_by": 1,
  "created_at": "2026-03-22 10:00:00"
}
```

```
# Create more tasks
curl -X POST http://localhost:7146/api/tasks \
  -H "Content-Type: application/json" \
```

```
  -H "Authorization: Bearer $TOKEN" \
  -d '{"title": "Build API endpoints", "priority": "high"}'

curl -X POST http://localhost:7146/api/tasks \
  -H "Content-Type: application/json" \
  -H "Authorization: Bearer $TOKEN" \
  -d '{"title": "Write documentation", "priority": "medium", "due_date": "2026-04-01"}'

# List all tasks
curl http://localhost:7146/api/tasks \
  -H "Authorization: Bearer $TOKEN"

# Update a task status
curl -X PUT http://localhost:7146/api/tasks/1 \
  -H "Content-Type: application/json" \
  -H "Authorization: Bearer $TOKEN" \
  -d '{"status": "completed"}'

# Delete a task
curl -X DELETE http://localhost:7146/api/tasks/3 \
  -H "Authorization: Bearer $TOKEN"
```

Every endpoint responds. Create. Read. Update. Delete. The CRUD cycle works.

---

8. Step 6: Dashboard Stats with Caching

Create `src/routes/dashboard.py`:

```
from tina4_python.core.router import get, template
from tina4_python.cache import cache_get, cache_set

@get("/api/dashboard/stats", middleware=["auth_middleware"])
async def dashboard_stats(request, response):
    cached = cache_get("dashboard:stats")
    if cached:
        return response({**cached, "source": "cache"})

    db = Database.get_connection()

    total = db.fetch_one("SELECT COUNT(*) as count FROM tasks")
    pending = db.fetch_one("SELECT COUNT(*) as count FROM tasks WHERE status = 'pending'")
    in_progress = db.fetch_one("SELECT COUNT(*) as count FROM tasks WHERE status = 'in_progress'
    completed = db.fetch_one("SELECT COUNT(*) as count FROM tasks WHERE status = 'completed'")
    overdue = db.fetch_one(
        "SELECT COUNT(*) as count FROM tasks WHERE due_date < date('now') AND status != 'complet
    )
    users = db.fetch_one("SELECT COUNT(*) as count FROM users")

    recent = db.fetch_all(
        "SELECT t.*, u.name as assignee_name FROM tasks t "
        "LEFT JOIN users u ON t.assigned_to = u.id "
        "ORDER BY t.created_at DESC LIMIT 10"
    )

    stats = {
        "total_tasks": total["count"],
        "pending": pending["count"],
```

```
        "in_progress": in_progress["count"],
        "completed": completed["count"],
        "overdue": overdue["count"],
        "total_users": users["count"],
        "recent_tasks": recent
    }

    cache_set("dashboard:stats", stats, ttl=30)

    return response({**stats, "source": "database"})


@get("/admin")
async def admin_dashboard(request, response):
    return response(template("dashboard.html"))
```

Verify the stats endpoint:

```
curl http://localhost:7146/api/dashboard/stats \
  -H "Authorization: Bearer $TOKEN"

{
  "total_tasks": 2,
  "pending": 1,
  "in_progress": 0,
  "completed": 1,
  "overdue": 0,
  "total_users": 1,
  "recent_tasks": [...],
  "source": "database"
}
```

Hit it again. The second response comes from cache:

```
curl http://localhost:7146/api/dashboard/stats \
  -H "Authorization: Bearer $TOKEN"

{
  "total_tasks": 2,
  "pending": 1,
  "completed": 1,
  "source": "cache"
}
```

The cache holds for 30 seconds. Creating or updating a task invalidates it.

---

9. Step 7: WebSocket Real-Time Updates

Create `src/routes/task_ws.py`:

```
import json
from tina4_python.core.router import websocket, push_to_websocket

@websocket("/ws/tasks")
async def task_updates(connection, event, data):
    if event == "open":
        await connection.send(json.dumps({
            "type": "connected",
```

```
            "message": "Listening for task updates"
        }))

    if event == "message":
        message = json.loads(data)
        if message.get("type") == "ping":
            await connection.send(json.dumps({"type": "pong"}))


async def push_task_update(action, task):
    """Push a task update to all connected WebSocket clients."""
    await push_to_websocket("/ws/tasks", json.dumps({
        "type": "task_update",
        "action": action,
        "task": task.to_dict()
    }))
```

Any user creates, updates, or deletes a task. All connected dashboard users see the change.

---

10. Step 8: Email Notifications

Create `src/routes/notifications.py`:

```
from tina4_python.messenger import Messenger
from tina4_python.core.router import template

async def notify_assignee(task):
    """Send email notification when a task is assigned."""
    assignee = User.find(task.assigned_to)

    if assignee is None:
        return

    creator = User.find(task.created_by)

    html_body = template("emails/task-assigned.html",
        assignee_name=assignee.name,
        task_title=task.title,
        task_description=task.description or "No description",
        task_priority=task.priority,
        task_due_date=task.due_date or "No due date",
        creator_name=creator.name if creator else "Unknown",
        task_url=f"http://localhost:7146/admin#task-{task.id}"
    )

    mailer = Messenger()
    mailer.send(
        to=assignee.email,
        subject=f"Task assigned: {task.title}",
        body=html_body,
        text_body=f"Hi {assignee.name},\n\n"
                  f"{creator.name} assigned you a task: {task.title}\n"
                  f"Priority: {task.priority}\n"
                  f"Due: {task.due_date or 'No due date'}\n\n"
                  f"Description:\n{task.description or 'No description'}"
    )
```

Create `src/templates/emails/task-assigned.html`:

```
<!DOCTYPE html>
<html>
<head><meta charset="UTF-8"></head>
<body>
    <div class="container">
        <h2>New Task Assigned</h2>
        <p>Hi {{ assignee_name }},</p>
        <p><strong>{{ creator_name }}</strong> assigned you a new task:</p>

        <div class="card">
            <div class="card-body">
                <h3>{{ task_title }}</h3>
                <p>{{ task_description }}</p>
                <table class="table">
                    <tr><td><strong>Priority:</strong></td><td>{{ task_priority }}</td></tr>
                    <tr><td><strong>Due:</strong></td><td>{{ task_due_date }}</td></tr>
                </table>
            </div>
        </div>

        <p><a href="{{ task_url }}">View Task</a></p>
    </div>
</body>
</html>
```

---

11. Step 9: Dashboard Template

Create `src/templates/dashboard.html`:

```
{% extends "base.html" %}

{% block title %}TaskFlow Dashboard{% endblock %}

{% block content %}
<h1>Dashboard</h1>

<div id="stats" class="row mb-4">
    <div class="col-md-2"><div class="card"><div class="card-body">
        <h6 class="text-muted">Total</h6><h2 id="stat-total">--</h2>
    </div></div></div>
    <div class="col-md-2"><div class="card"><div class="card-body">
        <h6 class="text-muted">Pending</h6><h2 id="stat-pending">--</h2>
    </div></div></div>
    <div class="col-md-2"><div class="card"><div class="card-body">
        <h6 class="text-muted">In Progress</h6><h2 id="stat-progress">--</h2>
    </div></div></div>
    <div class="col-md-2"><div class="card"><div class="card-body">
        <h6 class="text-muted">Completed</h6><h2 id="stat-completed">--</h2>
    </div></div></div>
    <div class="col-md-2"><div class="card"><div class="card-body">
        <h6 class="text-muted">Overdue</h6><h2 id="stat-overdue" class="text-danger">--</h2>
    </div></div></div>
    <div class="col-md-2"><div class="card"><div class="card-body">
        <h6 class="text-muted">Users</h6><h2 id="stat-users">--</h2>
    </div></div></div>
```

```
</div>

<div class="row">
    <div class="col-md-8">
        <div class="card">
            <div class="card-header d-flex justify-content-between">
                <span>Recent Tasks</span>
                <button class="btn btn-sm btn-primary" data-toggle="modal" data-target="#newTask
                    New Task
                </button>
            </div>
            <div class="card-body">
                <table class="table table-striped" id="task-table">
                    <thead>
                        <tr><th>Title</th><th>Assignee</th><th>Priority</th><th>Status</th></tr>
                    </thead>
                    <tbody id="task-list"></tbody>
                </table>
            </div>
        </div>
    </div>
    <div class="col-md-4">
        <div class="card">
            <div class="card-header">Live Updates</div>
            <div class="card-body" id="live-updates">
                <p class="text-muted">Connecting...</p>
            </div>
        </div>
    </div>
</div>

<script src="/js/frond.min.js"></script>
<script>
    var token = localStorage.getItem("token");
    if (token) frond.setToken(token);

    // Load dashboard stats
    frond.get("/api/dashboard/stats", function (data) {
        document.getElementById("stat-total").textContent = data.total_tasks;
        document.getElementById("stat-pending").textContent = data.pending;
        document.getElementById("stat-progress").textContent = data.in_progress;
        document.getElementById("stat-completed").textContent = data.completed;
        document.getElementById("stat-overdue").textContent = data.overdue;
        document.getElementById("stat-users").textContent = data.total_users;

        var tbody = document.getElementById("task-list");
        tbody.innerHTML = "";
        (data.recent_tasks || []).forEach(function (task) {
            var tr = document.createElement("tr");
            var badgeColor = {pending: "secondary", in_progress: "warning",
                              completed: "success", cancelled: "danger"};
            tr.innerHTML = "<td>" + task.title + "</td>" +
                "<td>" + (task.assignee_name || "Unassigned") + "</td>" +
                "<td>" + task.priority + "</td>" +
                "<td><span class='badge bg-" + (badgeColor[task.status] || "secondary") +
                "'>" + task.status + "</span></td>";
            tbody.appendChild(tr);
        });
    });
```

```
    // WebSocket for live updates
    var ws = frond.ws("/ws/tasks");
    var updates = document.getElementById("live-updates");

    ws.on("open", function () {
        updates.innerHTML = "<p class='text-success'>Connected - listening for updates</p>";
    });

    ws.on("message", function (raw) {
        var msg = JSON.parse(raw);
        if (msg.type === "task_update") {
            var div = document.createElement("div");
            div.innerHTML = "<strong>" + msg.action + ":</strong> " + msg.task.title;
            updates.prepend(div);

            // Refresh stats
            frond.get("/api/dashboard/stats", function () {});
        }
    });
</script>
{% endblock %}
```

Verify the dashboard:

```
curl http://localhost:7146/admin
```

The server returns the rendered HTML. Open `http://localhost:7146/admin` in your browser to see the full dashboard with stats, task list, and live update panel.

---

12. Step 10: Tests

Create `tests/test_taskflow.py`:

```
import uuid
import json
from tina4_python.test import Test, assert_equal, assert_true, assert_not_none

class TaskFlowTest(Test):

    def _register_and_login(self):
        email = f"test-{uuid.uuid4().hex[:8]}@example.com"
        self.post("/api/auth/register", {
            "name": "Test User",
            "email": email,
            "password": "TestPassword123!"
        })
        login_resp = self.post("/api/auth/login", {
            "email": email,
            "password": "TestPassword123!"
        })
        return json.loads(login_resp.body)["token"]

    def test_register(self):
        resp = self.post("/api/auth/register", {
            "name": "Alice",
            "email": f"alice-{uuid.uuid4().hex[:8]}@example.com",
            "password": "SecurePass123!"
```

```
        })
        assert_equal(resp.status_code, 201, "Registration should succeed")

    def test_login(self):
        email = f"login-{uuid.uuid4().hex[:8]}@example.com"
        self.post("/api/auth/register", {
            "name": "Bob",
            "email": email,
            "password": "SecurePass123!"
        })
        resp = self.post("/api/auth/login", {
            "email": email,
            "password": "SecurePass123!"
        })
        assert_equal(resp.status_code, 200, "Login should succeed")
        body = json.loads(resp.body)
        assert_not_none(body.get("token"), "Should return token")

    def test_create_task(self):
        token = self._register_and_login()
        resp = self.post("/api/tasks", {
            "title": "Test Task",
            "description": "A test task",
            "priority": "high"
        }, headers={"Authorization": f"Bearer {token}"})

        assert_equal(resp.status_code, 201, "Task creation should succeed")
        body = json.loads(resp.body)
        assert_equal(body["title"], "Test Task", "Title should match")
        assert_equal(body["priority"], "high", "Priority should match")

    def test_list_tasks(self):
        token = self._register_and_login()
        self.post("/api/tasks", {"title": "List Task 1"}, headers={"Authorization": f"Bearer {to
        self.post("/api/tasks", {"title": "List Task 2"}, headers={"Authorization": f"Bearer {to

        resp = self.get("/api/tasks", headers={"Authorization": f"Bearer {token}"})
        assert_equal(resp.status_code, 200, "Should return 200")
        body = json.loads(resp.body)
        assert_true(len(body["tasks"]) >= 2, "Should have at least 2 tasks")

    def test_update_task_status(self):
        token = self._register_and_login()
        create_resp = self.post("/api/tasks", {"title": "Status Task"},
                                headers={"Authorization": f"Bearer {token}"})
        task_id = json.loads(create_resp.body)["id"]

        resp = self.put(f"/api/tasks/{task_id}", {"status": "completed"},
                        headers={"Authorization": f"Bearer {token}"})
        assert_equal(resp.status_code, 200, "Update should succeed")
        body = json.loads(resp.body)
        assert_equal(body["status"], "completed", "Status should be updated")
        assert_not_none(body.get("completed_at"), "Should have completion timestamp")

    def test_delete_task(self):
        token = self._register_and_login()
        create_resp = self.post("/api/tasks", {"title": "Delete Me"},
                                headers={"Authorization": f"Bearer {token}"})
        task_id = json.loads(create_resp.body)["id"]
```

```
        resp = self.delete(f"/api/tasks/{task_id}",
                           headers={"Authorization": f"Bearer {token}"})
        assert_equal(resp.status_code, 204, "Delete should succeed")

    def test_dashboard_stats(self):
        token = self._register_and_login()
        self.post("/api/tasks", {"title": "Stats Task"},
                  headers={"Authorization": f"Bearer {token}"})

        resp = self.get("/api/dashboard/stats",
                        headers={"Authorization": f"Bearer {token}"})
        assert_equal(resp.status_code, 200, "Should return stats")
        body = json.loads(resp.body)
        assert_true(body["total_tasks"] >= 1, "Should have at least 1 task")

    def test_unauthorized_access(self):
        resp = self.get("/api/tasks")
        assert_equal(resp.status_code, 401, "Should reject unauthenticated request")
```

Run the tests:

```
tina4 test

Running tests...

  TaskFlowTest
    [PASS] test_register
    [PASS] test_login
    [PASS] test_create_task
    [PASS] test_list_tasks
    [PASS] test_update_task_status
    [PASS] test_delete_task
    [PASS] test_dashboard_stats
    [PASS] test_unauthorized_access

  8 tests, 8 passed, 0 failed (0.84s)
```

All green. The application works.

---

13. Step 11: Docker Deployment

Create `Dockerfile`:

```
FROM python:3.12-slim

COPY --from=ghcr.io/astral-sh/uv:latest /uv /usr/local/bin/uv

WORKDIR /app

COPY pyproject.toml uv.lock ./
RUN uv sync --frozen --no-dev

COPY . .
RUN mkdir -p data logs

EXPOSE 7146

HEALTHCHECK --interval=30s --timeout=5s --retries=3 \
```

```
  CMD curl -f http://localhost:7146/health || exit 1

CMD ["uv", "run", "python", "app.py"]
```

Create `docker-compose.yml`:

```
services:
  app:
    build: .
    ports:
      - "7146:7146"
    environment:
      - TINA4_DEBUG=false
      - JWT_SECRET=${JWT_SECRET:-change-me-in-production}
      - TINA4_DATABASE_URL=sqlite:///data/app.db
      - TINA4_CACHE_BACKEND=redis
      - TINA4_CACHE_URL=redis://redis:6379
    volumes:
      - taskflow-data:/app/data
      - taskflow-logs:/app/logs
    depends_on:
      - redis
    restart: unless-stopped

  redis:
    image: redis:7-alpine
    restart: unless-stopped

volumes:
  taskflow-data:
  taskflow-logs:
```

Deploy:

```
docker compose up -d --build
```

Verify:

```
curl http://localhost:7146/health
```

```
{"status": "ok", "version": "1.0.0", "database": "connected"}
```

TaskFlow runs in production. Authenticated APIs. Real-time WebSocket updates. Email notifications. Cached dashboard stats. Tested. Dockerized.

---

14. The Complete Project Structure

```
taskflow/
■■■ .env
■■■ .env.example
■■■ .gitignore
■■■ pyproject.toml
■■■ uv.lock
■■■ app.py
■■■ Dockerfile
■■■ docker-compose.yml
■■■ src/
■   ■■■ routes/
■   ■   ■■■ auth.py               # Registration, login
■   ■   ■■■ tasks.py              # Task CRUD
■   ■   ■■■ dashboard.py          # Dashboard stats + page
■   ■   ■■■ notifications.py      # Email notification helpers
■   ■   ■■■ task_ws.py            # WebSocket event handlers
■   ■■■ orm/
■   ■   ■■■ user.py               # User model with auth methods
■   ■   ■■■ task.py               # Task model with relationships
■   ■■■ middleware/
■   ■   ■■■ auth.py               # JWT auth middleware
■   ■■■ migrations/
■   ■   ■■■ 20260322000100_create_users_table.sql
■   ■   ■■■ 20260322000200_create_tasks_table.sql
■   ■■■ templates/
■   ■   ■■■ base.html             # Base layout
■   ■   ■■■ dashboard.html        # Dashboard page
■   ■   ■■■ emails/
■   ■   ■   ■■■ task-assigned.html # Assignment notification
■   ■   ■■■ errors/
■   ■       ■■■ 404.html
■   ■       ■■■ 500.html
■   ■■■ public/
■   ■   ■■■ css/
■   ■   ■   ■■■ tina4.css
■   ■   ■■■ js/
■   ■       ■■■ tina4.min.js
■   ■       ■■■ frond.min.js
■   ■■■ locales/
■       ■■■ en.json
■■■ data/
■   ■■■ app.db
■■■ logs/
■■■ secrets/
■■■ tests/
    ■■■ test_taskflow.py
```

Every file has a purpose. Every directory follows the convention. A new developer looks at this structure and knows where to find things.

---

15. What You Built

This chapter used every major concept from the book:

Feature	Chapter
Route decorators (`@get`, `@post`, `@put`, `@delete`)	Chapter 2
Request/response handling	Chapter 3
Jinja templates	Chapter 4
Database queries	Chapter 5
ORM models (User, Task)	Chapter 6
JWT authentication	Chapter 8
Auth middleware	Chapter 10
Email notifications (Messenger)	Chapter 16
Cache (dashboard stats)	Chapter 11
Frontend (tina4css dashboard)	Chapter 17
WebSocket (live updates)	Chapter 23
Testing (full test suite)	Chapter 18
Docker deployment	Chapter 34

---

16. What to Build Next

TaskFlow is a solid foundation. Here are ideas for extending it.

Features:

- **Task comments** -- Add a Comment model with a `task_id` foreign key. Display comments on the task detail page.
- **File attachments** -- Let users upload files to tasks. Store them in `data/uploads/` and serve them via a route.
- **Team management** -- Add a Team model. Users belong to teams. Tasks are scoped to teams.
- **Task labels/tags** -- Many-to-many relationship between tasks and labels for categorization.
- **Due date reminders** -- Use the queue system to schedule reminder emails 24 hours before a task's due date.
- **Activity log** -- Record every change to a task (who changed what, when) for audit trails.
- **Search** -- Full-text search across task titles and descriptions.
- **Calendar view** -- Render tasks on a calendar based on their due dates.

- **Mobile API** -- The API already works for mobile apps. Add push notification support via Firebase Cloud Messaging.

Technical improvements:

- **Rate limiting per user** -- Replace the global rate limiter with per-user limits.
- **Database upgrade** -- Switch from SQLite to PostgreSQL for better concurrency.
- **CI/CD pipeline** -- Add GitHub Actions to run tests on every push.
- **API documentation** -- Generate OpenAPI/Swagger docs from your route definitions.
- **Internationalization** -- Add `src/locales/` files for multiple languages.

---

17. Closing Thoughts -- The Tina4 Philosophy

You built a complete application. User auth. CRUD. Real-time updates. Email. Caching. Tests. Deployment. Your project has one dependency: `tina4_python`.

No separate ORM package. No template engine package. No authentication library. No WebSocket server. No caching library. No testing framework. No CLI tool. No CSS framework. No JavaScript helpers. All built in.

One framework. Zero extra dependencies. Everything you need.

The same patterns work in PHP, Ruby, and Node.js. Same project structure. Same CLI commands. Same `.env` variables. Same template syntax. Learn Tina4 once. Use it everywhere.

Build things. Ship them. Keep it simple.

Release Notes

v3.13.138 (2026-09-24) - Safer requests, templates, and database operations

Licensing from this release. Tina4 is available under MPL-2.0, with separate commercial terms available from Code Infinity. Copyright Code Infinity. Previously published releases retain their original licences.

This release brings together the framework hardening work, database correctness fixes, measured-time guidance for the AI skills, and verifiable release artifacts.

Request and template boundaries. Frond escapes output by default and preserves only explicitly trusted output. Review templates that intentionally render HTML when upgrading; never mark untrusted input as safe. Header validation, request-body limits, security headers, CORS handling, model-field validation, and diagnostic redaction follow the shared contracts. Explicit Content-Type headers survive response conversion and binary bodies preserve bytes.

Credentials and development access. API clients retain credentials only for their configured origin, including uploads, downloads and streams. Use a separate client when authenticating to another service. Forwarded host and protocol require a trusted proxy. Development reads and writes enforce the raw-peer, Host and browser-origin boundary; remote non-MCP development access requires the dedicated token. The documented MCP transport API-key fallback is unchanged. File access checks resolved paths so symlinks cannot expose secrets or escape the project.

Database and service correctness. Statements that return rows retain those rows, filtered counts and write results agree across adapters, SQL placeholders preserve literal text, and ORM fields mapped to database columns read back correctly. Database operations and explicit transactions reserve their connection; a fully occupied pool fails clearly instead of sharing an in-use connection. Session transports reuse MongoDB connections and handle replica-set replies. Mail transports verify certificates and require STARTTLS when configured. Optional driver errors identify the needed install, and unconfirmed broker writes fail clearly.

Skills and release evidence. The maintainer skill estimates work from measured agent durations. Release packages carry checksums, an SPDX software inventory, and build provenance; release Actions are pinned to commits. The updated PowerShell skills installer is digitally signed. The ISO programme remains in progress; this release makes no certification or conformance claim.

The accompanying CLI 3.8.91 verifies update checksums before replacing an installed binary, refuses missing checksum manifests, and corrects generated Node.js package imports. It also includes the browser-opening and CI behavior fixes. Tina4 JS 1.7.2 ships the merged client hardening and verified browser bundle. Refresh the AI skills with `curl -fsSL https://tina4.com/install-skills.sh | sh` or the documented PowerShell installer.

v3.13.105 (2026-08-19) - Cross-API invariants, honest logs, portable tests

A bug release with teeth. Five audit bugs closed in the queue and ORM cache seams — the places where two spellings of the same intent quietly disagreed. Route inspection stops booting the app. The startup log stops lying about `@noauth`. Hot reload reaches the modules that captured the changed one. Firebird's migration ledger tolerates any case the driver hands back. And every framework developer skill gained a spine: it announces every step before taking it, and it warns with ■ when it is out of date.

Queue and ORM audit bugs — five fixed

- `Model.clear_cache()` **invalidates the DB layer too.** When both

`TINA4_AUTO_CACHING` and `TINA4_DB_CACHE` are enabled, a manual `clear_cache()` used to leave the DB-layer cache holding stale rows. It now cascades to `db.cache_clear()` on this model's bound connection.

- `Queue.retry()` **revives every dead letter.** The no-arg form used a

generator inside `any(...)`, which short-circuits after the first success. Now it materialises the loop so all N dead letters revive, not just one.

- `Job.retry()` **removes the dead-letter file.** Iterating `dead_letters()`

and calling `.retry()` on each used to leave the failed directory carrying every revived job. The next `dead_letters()` call re-reported them and consumers processed each job twice.

- **MongoDB** `retry_job(id)` **finds the dead letter.** The old filter looked

for `{_id: job_id, topic: self._topic, status: "failed"}` — three reasons that combination could never match a real dead-letter doc (fresh `_id`, `.dead_letter` topic namespace, `dead` status). The new filter reads the dead-letter namespace by `data.id`, deletes the record, and upserts the original back to pending.

- **MongoDB** `purge(status)` **returns a count.** Used to return `None` and

honour only the `pending` status. Now returns `deleted_count`, honours every status, and scopes the delete so `purge("pending")` no longer nukes completed or reserved docs under the same topic.

Route inspection scans, never boots

`tina4 routes` walks canonical route files without executing `app.py` or starting the server. The prior implementation booted the app on the same port and terminated whichever process was holding it — a nasty surprise for anyone running `tina4 routes` in a live shell. Closes tina4-python #104.

Startup log tells the truth about @noauth / @secured

Python decorators apply bottom-up, so `@post()` logged the route as `auth=required` a microsecond before the outer `@noauth()` flipped the flag. `@noauth()` and `@secured()` now emit a corrective `Route auth updated:` line whenever they change the auth default on a route another decorator already logged. Closes tina4-python #103, where a public login endpoint appeared to log its own security bug. Python-only — PHP, Ruby and Node do not log an auth field at registration time.

Hot reload reaches every module that imports the changed one

A route file that did `from src.orm.Todo import Todo` kept its stale `Todo` reference after the model changed — the API then lied about the schema on the next request. `_auto_discover` now cascades a re-import to every module that captured a binding from the changed one, recursive with a visited set, bounded to the discovery scope, fail-loud. Closes tina4-python #102.

Firebird migration ledger is case-agnostic

`tina4_migration` reads and writes work regardless of the case the Firebird driver returns for the `migration_name` column. Uses the atomic sequence pattern the other engines already have.

Test-harness fixes

- **SIGTERM port probe checks both interfaces.** The graceful-shutdown test

probed only `0.0.0.0` with `SO_REUSEADDR`, which succeeds on macOS while the framework's default `127.0.0.1` listener holds the port — a silent false negative on every Mac dev machine. Now probes both interfaces and refuses to conclude the port is free until both agree.

- **MySQL connect-timeout assertion tolerates driver cleanup time.** The

assertion compared the outer clock to the wrapper's rendered message. mysql-connector adds around 0.7 seconds of cleanup after aborting, so the two stopwatches legitimately differed. The assertion now parses the reported elapsed via regex and checks it against the configured bound, not against the outer clock.

Skills grow a spine

- **Announce before you act.** Every framework developer skill now instructs

itself to say what it is about to do — one line before each file write, migration, or dependency install — so the developer can stop the work before it spends their afternoon. Same rhythm across Python, PHP, Ruby and Node.

- ■ **stale-skill detection.** Every framework developer skill also checks

whether a newer skill is available (compared against the latest published skill version at `https://tina4.com/skills/<name>/version`, never against the project's framework version) and prepends ■ to every reply if it is behind. Your framework version stays your call; the skill just refuses to pretend its advice is current.

Upgrade notes

- No API changes. All fixes preserve existing signatures.
- Consumers iterating `dead_letters()` and calling `job.retry()` on each — now safe. Previously it produced duplicates on the next `dead_letters()` call.
- `Model.clear_cache()` under `TINA4_DB_CACHE=true` now also flushes the DB-layer cache. This is the expected behaviour.
- Lab full-suite verified: 25,263 tests across all four frameworks, zero failures, zero skips.

v3.13.104 (2026-08-17) - OIDC SSO and PostGIS land

Two big features, uniform across all four frameworks: a provider-neutral OIDC handoff for SSO logins, and a shared `Point` contract for PostGIS geospatial queries. Both wire through the same small API in Python, PHP, Ruby and Node.

OIDC / OpenID Connect — provider-neutral, session handoff

`Sso.from_issuer(...)` discovers a real OIDC provider (Keycloak, Auth0, Okta, anything speaking the standard) via its `.well-known/openid-configuration` endpoint and hands a validated identity into an existing Tina4 `Session`. The `login` / `callback` / `refresh` / `logout` cycle runs against the real provider — no mock. PKCE with S256 by default. Reserved SSO session keys (`Sso.PENDING_KEY`, `Sso.SESSION_KEY`) never leak through `session.all()`.

Opt in with `TINA4_SSO_ISSUER` plus the client id, secret and redirect URI.

PostGIS spatial support

`PointField` on an ORM model creates the right column for whichever engine you are on — PostGIS `geometry(Point, SRID)` on PostgreSQL, JSON on the others. `select_distance`, `intersects` and `bbox` predicates read the same way across engines but use each engine's native functions where available. GeoJSON in, GeoJSON out. Refuses hostile input (garbage geometry, injected column names, NaN or Inf ordinates) before any SQL runs.

Opt in by declaring a `PointField` on a model. Existing models unchanged.

Swagger is SSO-aware

The Swagger generator recognises OIDC-secured routes and emits the right `security` scheme in the OpenAPI 3.0.3 spec.

v3.13.102 (2026-08-15) - Unified client and CI hardening

Small release. Framework developer skills teach the unified `tina4 init` / `tina4 serve` workflow consistently across the four languages, and CI now verifies the downloaded native CLI asset before running the metrics handoff.

v3.13.103 (2026-08-16) - Metrics you can trust, releases that cannot lie

This release restores one public version across Python, PHP, Ruby, and Node.js. Version 3.13.102 reached PyPI and Packagist, but its RubyGems and npm packages still contained 3.13.101. Skip 3.13.102 and upgrade to 3.13.103.

What changed

• Dev-admin hands metrics work to the signed Tina4 client 3.8.76. Frameworks do

not keep a second metrics analyser or silently fall back to one.

• `has_referencing_test` means that test source refers to the file. It does not

claim that the test ran, passed, or covered the file.

• Frond expression parsing and evaluation are split into smaller internal steps,

while public APIs and the shared 84-case expression corpus remain unchanged.

• Tina4 skills lead with `tina4 init` and `tina4 serve`, and keep scaffolding

prominent in every language.

• Release workflows now reject a tag that differs from the source package

version before publishing.

No runtime packages were added. Language extensions remain extensions, not framework dependencies.

+## v3.13.101 (2026-08-14) - One AI client, four languages

One class. Three methods. No provider SDK. Tina4 applications can now call local models, OpenAI, and Anthropic through the same small API: `Ai.chat`, `Ai.complete`, and `Ai.embed`.

What ships

- The native `tina4 metrics` CLI now owns every code-health calculation. Framework-level metrics commands and quick census implementations are gone.
- Dev-admin keeps its metrics tab through two thin native-JSON adapters: full analysis and file detail.
- `Ai.chat` returns one `ChatResponse` with text, model, token usage, finish reason, and the raw provider response.
- `Ai.complete` sends one user message and returns its text.
- `Ai.embed` accepts one string or a list and preserves that input shape in its output.
- `stream=True` yields ordered text deltas from OpenAI-compatible and Anthropic streams.

Failure is data

Hosted providers refuse to send a request without `TINA4_AI_KEY`. Connection and total-request deadlines stay separate. Retries cover connection failures, HTTP 429, and HTTP 5xx responses, but stop after the first streamed delta. Errors never include the key, prompt, or provider response body.

Upgrade notes

This release adds a new API. It does not add the unimplemented `Llm.ask` or `Llm.ask_json` proposal. Vision, image generation, tools, agents, and conversation state remain outside this release. The client uses Python's standard library; your dependency tree does not grow.

v3.13.100 (2026-08-14) - Frond keeps the whole page

This fast-follow release fixes template inheritance and puts a ceiling on Frond's caches. The template engine now rejects a second `{% extends %}` tag, preserves nested root blocks, and evicts stale compiled state instead of letting a long-running process grow without limit.

Template inheritance

- A template may declare one parent. A second `{% extends %}` now raises a clear error instead of changing the parent without warning (`9bb7279`)
- The final block-substitution pass now tracks nesting depth. A block nested inside another root block no longer causes the outer content to disappear (`91828ce`)

Bounded caches

- Template, fragment, and expression caches now have size bounds and TTL sweeps. Long-running workers release stale entries while rendered output stays unchanged (`82e02de`)

Frond extension scope

- Class calls to `add_filter`, `add_global`, and `add_test` remain process-global. Instance calls now affect that renderer only, so one request or test cannot seed every Frond instance created later (`ecf66e6`)

AI skill installer

- Skill downloads retry transient network and HTTP failures. A short GitHub raw-content outage no longer drops the whole install (`09dd6b8`)

Release consistency

- `pyproject.toml`, `uv.lock`, the packaged fallback version, and the AI-facing guide now agree on `3.13.100`. New version guards stop a partial bump before release (`1aec8c7`, `44188a6`)

Upgrade notes

No valid template syntax changes. Code that relied on an instance extension leaking globally must register on `Frond` itself. A template with two `{% extends %}` tags was already contradictory; it now fails at the line that needs fixing. Cache eviction may trigger a reparse, but it does not change the rendered bytes.

The Frond AOT compiler is not part of this release. Parser and compiler parity for Ruby and Node.js remains planned for `3.13.101`.

v3.13.99 (2026-08-13) - Stability, parity, and secure by default

The biggest step yet on the road to **3.14 stable**. This release closes out Phases 1 through 5 of the v3 parity audit: roughly thirty breaking changes, and every one of them is a security fix, a parity fix, or a correctness fix, never a change made for its own sake. Two conformance grids, logging and database adapters, are now fully proven against real services in all four frameworks, and a small batch of genuinely new bugs gets fixed alongside them. Read "Possible breaking" before you upgrade: several of these changes correct a footgun that was silently doing the wrong thing.

Security

Security now defaults on instead of requiring opt-in.

- Security headers emit by default, including `Content-Security-Policy: default-src 'self'`, which blocks inline scripts and third-party CDNs. Relax the policy with `TINA4_CSP`; HSTS emits only on HTTPS, when `TINA4_HSTS` is set (`bfc1597`)
- `TINA4_CSRF=true` now actually attaches the CSRF middleware, where it used to be inert. A blank `TINA4_SECRET` fails closed now instead of minting a forgeable public-default token (`3249495`)
- The dev server binds `127.0.0.1` by default; set `TINA4_HOST=0.0.0.0` to expose it. A cross-origin `/__dev` mutation is refused, and `.env` is never served through the file endpoints (`41b3aeb`)
- A symlink whose real path escapes the public directory is refused, and dotfiles (`.env`, `.git`) 404 instead of serving (`c51c686`)
- `{% include %}`, `{% extends %}`, and `{% import %}` in a Frond template are confined to the templates directory. A path that escapes it, with `..`, an absolute path, or a symlink, now raises (`f858c11`)
- `tina4 serve` on a busy port no longer kills whatever holds it. It reclaims only a port held by an identifiable Tina4 dev server, refuses a foreign holder, and honours `TINA4_NO_TAKEOVER` / `--no-kill` (`fdd7d86`)
- A hostile inbound `X-Request-ID` (CRLF, illegal characters, an over-long value) is sanitized to a fresh id instead of echoed back raw, closing a response-header and log-injection path (`1d324d5`)
- The dead `render_production_error` function is removed; nothing called it. The dev error overlay now redacts `Authorization`, `Cookie`, and `Set-Cookie` headers plus secret-looking body fields, and caps the rendered stack at 50 frames (`7b460ea`)

• A repeated multipart file field now yields a list instead of silently dropping every upload but the last. The new safe-save helper rejects `..` and absolute filenames (`9fd483c`)

Data integrity

Four footguns that used to fail silently now fail loud, or stop failing at all.

• An unparseable or unsupported MongoDB WHERE clause now raises instead of silently matching every document. A DELETE or UPDATE with no WHERE is rejected outright. Write an explicit WHERE, or call `truncate()` for the whole collection (`3315d1c`)

• `truncate()` on a Mongo collection now actually empties it. It used to report success while leaving every document in place (`079873e`)

• Firebird write results are correct now: `db.insert().last_id` and `db.update()` / `db.delete() .affected_rows` return real values instead of `None` or `0` (`de441c5`)

• MSSQL pagination converges on `OFFSET`/`FETCH` in all four frameworks (`4491cf8`)

ORM and validation

Eight fixes bring the ORM's write path, validation, and generated schema into agreement across all four frameworks.

• `create_table()` now injects the soft-delete column (`is_deleted INTEGER DEFAULT 0`) for a `soft_delete = True` model that does not declare it itself (`6c190c2`)

• A soft-deleted child is no longer returned through relationship traversal, lazy or eager. The phantom `on_delete=` foreign-key parameter is removed and now raises a clear "read-side-only" error instead of silently doing nothing (`2e9a69a`)

• The imperative `has_many` default row cap changes from a silent 100 to the whole result set. A caller relying on the old truncation now gets every row (`558a000`)

• A REST list `?page` below 1 clamps to page 1 instead of handing a negative offset to the driver; an oversized `?limit` or `?per_page` is capped at 100 (`761873a`)

• Loading a row from the database no longer re-enforces write-path business constraints (required, length, range, choices). A stored row that violates a constraint tightened after it was written now loads instead of aborting the whole page; the write path is unchanged and still rejects a bad save (`6095243`)

• AutoCrud returns `422` with field errors for an invalid create or update, where it used to return `400`. The write body is now allow-listed: `is_deleted` is never client-writable, and a client-supplied primary key is stripped on both create and update, closing a mass-assignment hole and an IDOR-shaped redirect (`9941536`)

• `seed_table`'s `seed` parameter is removed and now raises if you pass a non-default value. Seed your own `FakeData` instance for determinism (`80e897b`)

• The validation message vocabulary is unified across all four frameworks; `in_list` now renders a compact JSON list (`must be one of ["a","b","c"]`) (`b9fbd7e`)

Request model - the big one

`request.params` is route-params-only now, in all four frameworks. Python used to merge the query string and the route params into `params`, so `request.params["id"]` could silently return a client-supplied `?id=` value that shadowed the real route parameter, a param-pollution surface. Client input lives only in `request.query` and `request.body` now; `request.params` holds route params and nothing else. A handler or middleware reading the old merged `params` must read `query` or `body` explicitly.

Three related fixes ship with it: a malformed JSON body returns the raw string it failed to parse, the empty-body sentinel is Python's native `None`, `Request` gains a mutable `user` field, and `header()` lookup is case-fold only now (it no longer also converts `-` to `_`) (`7df99c8`).

Migrations

- PHP, Ruby, and Node's `rollback` is fail-safe now too, matching Python's already-correct behaviour: a missing `.down.sql` file or a failed down script raises and leaves the `tina4_migration` ledger row in place, instead of deleting the tracking row and leaving the schema applied but untracked (`9289017`)

HTTP

- Responses gzip-compress when eligible in PHP, Ruby, and Node (body over 1024 bytes, `Accept-Encoding: gzip`, a compressible content type); a cacheable 200 gets a strong ETag and a matching `If-None-Match` gets a 304. The static-file ETag format is unified to `W/"<size>-<mtime>"` on all four, so a cache revalidates once instead of on every deploy. Python's 304 now carries `ETag` and `Last-Modified`, and `If-None-Match` follows RFC 7232 (weak, list, or `*`) instead of an exact-string match (`4a389f9`)

- A `403` now negotiates HTML versus JSON the same way `404` and `500` already do (`7af21c6`)

- The OpenAPI spec converges on one shape across languages: a secured operation documents a `401`, and `summary`/`tags` always populate (`47fbcf2`)

- A route group's prefix join is normalized, so a route no longer mis-registers on a bare concatenation or an uncollapsed double slash (`14df0ed`)

Dev tooling

- The inline `@tests` descriptor builders are renamed: `assert_equal`/`assert_raises`/`assert_true`/`assert_false` become `expect_equal`, `expect_raises`, `expect_true`, `expect_false`. `tina4 python test` now discovers and runs the inline surface with a real exit code, so a previously-green run that executed none of your inline tests may now actually run, and fail, them (`d854b9d`)

- `TestResponse` preserves duplicate headers; a new multi-value accessor sits alongside the unchanged single-value `headers[name]` (`21bc198`)

- MCP `serverInfo.version` and outbound HTTP requests now report the real framework version instead of a placeholder; the framework client sends `User-Agent: Tina4/<version>` unless the caller already supplies one (`4275713`)

Proven in all four

Two conformance grids close out fully proven, tested against real services in every framework, no mocks:

- **Logging.** A real per-language runner now exercises the shared logger contract end to end. Closing it surfaced and fixed real bugs: bootstrap `configure()` no longer invents defaults, and a request id is now cleared in a `finally` block so it can never bleed into the next request (`f809b38`)
- **Database adapters** (ADR-0044). A real runner against SQLite, PostgreSQL, MySQL, MSSQL, and Firebird proved every adapter implements the same interface. It found a real bug on the way: `MySQLAdapter` ran with `autocommit=False` under InnoDB's REPEATABLE-READ, so a plain `fetch()` / `fetch_one()` opened an implicit transaction that a later `start_transaction()` then collided with (`f29579a`, `543d8c1`)

Bug fixes

- A route path containing a literal parenthesis, like `/products/(sale)`, now matches correctly everywhere. Python and Ruby were already correct; PHP and Node compiled the literal characters as regex syntax (`8534b27`)

Possible breaking

Read these before you upgrade. Every entry here is a security, parity, or correctness fix, none is a change made for its own sake.

- `request.params` **is route-params-only.** Read query-string values from `request.query` and body values from `request.body`. This is the single largest behaviour change in the release.
- **Security headers, including CSP, emit by default.** An app depending on inline scripts or a third-party CDN needs `TINA4_CSP` to relax the policy.
- `TINA4_CSRF=true` **now actually attaches the CSRF middleware.** If you set it and never noticed CSRF enforcement, it enforces now.
- **A blank** `TINA4_SECRET` **fails closed.** Set a real secret; the framework no longer falls back to a guessable public default.
- **The dev server binds** `127.0.0.1` **by default.** Set `TINA4_HOST=0.0.0.0` to expose it on your network.
- **An unparseable Mongo WHERE now raises instead of matching everything.** Add an explicit WHERE, or call `truncate()`.
- `create_table()` **adds** `is_deleted` **for a soft-delete model automatically.** A model that already declares the column is unaffected.
- **The** `on_delete=` **foreign-key parameter is removed and raises.** Relationships are read-side only; enforce integrity in your migrations.
- **The imperative** `has_many` **no longer caps at 100 rows.** A caller relying on the old truncation now gets every row.

• `?page=0` **clamps to page 1, and** `?limit` **caps at 100.** A client can no longer request the whole table in one page.

• **AutoCrud returns** `422`**, not** `400`**, on invalid input**, and never accepts `is_deleted` or a client-supplied primary key in the write body.

• `seed_table`**'s** `seed` **parameter is removed and raises.** Seed your own `FakeData` instance instead.

• **Validation messages changed wording**, including `in_list`'s compact JSON rendering.

• **A malformed JSON body returns the raw string**, not an empty dict.

• **The static-file** `ETag` **format changed**, so every cache revalidates once on upgrade.

• **A route group's prefix join is normalized.** A route depending on the old mis-registered, doubled-slash path must use the corrected one.

• **The inline testing descriptors are renamed** `assert_*` to `expect_*`, and `tina4 python test` now actually runs them.

Coming in 3.13.100

Frond's compiler, extensibility, auto-escaping, sandboxing, and caching work (features 48 through 60) is deferred to the 3.13.100 fast-follow, alongside a refreshed Carbonah benchmark harness and a configurable database column-name casing option.

v3.13.98 (2026-08-11) - Skills discipline

A maintenance release. No API changed, and no behaviour changed in your app.

The bundled AI coding skills - the ones that install with `curl -fsSL https://tina4.com/install-skills.sh | sh` - moved onto one shared plan discipline. A plan file has one format (Scope, Tests, Bugs, Commits), and a checkbox is ticked only when the work is verified green at HEAD, never on a claim. That keeps an agent's plan file honest, so it always reflects what actually shipped.

The framework package also shed internal dead weight: duplicate design-system source files that were never compiled or served at runtime. Nothing your code imports or renders changed.

Nothing to upgrade. If you use the Tina4 AI skills, re-run the installer.

v3.13.97 (2026-08-07) - Behaviour corrections

A small bug-fix release on the road to **3.14 stable**. No new surface, no renames. Two behaviours come back into line with the contract. A broker queue refuses an operation it cannot honour instead of losing your data, and a parity test locks DocStore's date handling so it stays correct. Read "Possible breaking" if your app calls `clear()` or `purge()` on a Kafka or RabbitMQ queue.

Queues

- `clear()` and `purge()` on a broker backend raise a named refusal. A broker delivers messages to consumers and cannot address them by status, so neither call can be honoured. Kafka's `clear()` returned `0` and `purge()` did nothing; RabbitMQ drained the live queue and destroyed its messages. Both raise now, rather than return a meaningless count or throw away data you meant to keep (`2a0cb8c`)

DocStore

- `distinct()` dedups values by equality, so two equal dates collapse to one row. Python was already correct; a parity test now locks it across all four frameworks (`8de6025`)

Possible breaking

- `clear()` **and** `purge()` **raise on a broker queue (Kafka, RabbitMQ).** The result never matched the call: Kafka returned `0`, RabbitMQ destroyed the live queue. Catch the refusal, or guard the call. A database-backed queue still clears and purges as before.

v3.13.96 (2026-08-07) - One paginate envelope, one message shape

Another step toward **3.14 stable**, and the theme is still parity: the same call means the same thing in Python, PHP, Ruby and Node. This release settles two subsystems that had drifted into four shapes. Pagination returns one envelope of seven keys with a true total. The Messenger IMAP path returns one message shape, addresses mail by a real IMAP UID, and hands back attachments you can write straight to disk. Some of these are behaviour changes, so read "Possible breaking" at the end before you upgrade.

Pagination

- `to_paginate()` takes no arguments and reads every field from the query that ran; pass one and it raises (`790f6bc`)
- The envelope is exactly seven snake_case keys: `records`, `total`, `page`, `per_page`, `total_pages`, `limit`, `offset` (`790f6bc`)
- `total` is a true `COUNT(*)` for the filter, never the row count the page returned (`790f6bc`)
- The AutoCrud REST list endpoint returns the same seven keys (`891e9a2`)

```
# Fetch the page you want, then describe it. No arguments.
result = db.fetch("SELECT * FROM orders", [], 20, 40)   # 20 per page, page 3
return response(result.to_paginate())
# {"records": [...], "total": 250, "page": 3, "per_page": 20,
#  "total_pages": 13, "limit": 20, "offset": 40}
```

Messenger

- `send()` returns `{success, message, id}` on both the real-send and dev-capture paths. `error` became `message`, `message_id` became `id`, and the capture-only `dev` key is gone (`5e912ca`)
- `id` carries a real `Message-ID` header the message actually sets, so it is populated instead of always empty (`5e912ca`)

- `inbox()` and `search()` return `snippet` as decoded, tag-stripped plain text truncated to 200 characters, not raw base64 (`5e912ca`)
- `read()` of a UID that does not exist returns `None`, not `{}` (`5e912ca`)
- IMAP credentials are their own now: `TINA4_MAIL_IMAP_USERNAME` and `TINA4_MAIL_IMAP_PASSWORD`, falling back to the SMTP pair, plus `imap_username` / `imap_password` and an `imap_encryption` constructor argument (`5e912ca`)
- The `uid` from an IMAP read is a real IMAP UID, so it still addresses the right message after another client expunges the mailbox (`3237383`)
- `read()` folds each attachment's decoded bytes into `attachments[i]["content"]`, so you write an attachment straight to disk (`dabd87d`)

Api client

- `Api.send()` is renamed `Api.send_request()`, the one name PHP, Ruby and Node already use (Ruby's `send` is `Object#send` and cannot be an HTTP verb). There is no alias, the primary is renamed. The verb methods `get` / `post` / `put` / `patch` / `delete`, `upload` and `download` are unchanged (`e54ae00`)

Swagger

- `servers[0].url` defaults to `/`, which resolves under any port, host or proxy, instead of a hard-coded `http://localhost:7145` (`d19fcac`)
- `info.version` defaults to `1.0.0` and `info.description` to an empty string (`d19fcac`)
- `components.schemas` is keyed by the model class name, with a `required` array derived from column nullability (`d19fcac`)
- `operationId` keeps a path's leading underscores, so `/__health` and `/health` stay distinct (`d19fcac`)
- An operation documents only the response codes the framework really returns (`d19fcac`)

Migrations

- `code` is the canonical kind for a code migration, the same word in all four frameworks. An unknown kind raises and names the valid ones instead of being quietly accepted (`7ce89dd`)

Server

- An oversized request body answers `413`, not `500` (`d448027`)
- `TINA4_PORT` is the port variable and wins; bare `PORT` still works but is deprecated and goes in 3.14 (`a0b0dab`)
- A public `asgi()` bootstrap exposes the app to any ASGI server (`a653908`)
- A synchronous route handler runs in a worker thread, so a slow handler no longer blocks the event loop (`867b8d9`)

Build and skills

- The framework SCSS compilers are gone; the `tina4` Rust CLI owns SCSS now (`9773bcc`)
- tina4-css is pinned to one artefact and one URL (`546448f`)

- The skills point at the ADRs and make `tina4 metrics`, Carbonah and mcp.tina4.com the lean, green, grounded workflow (`5cca039`)

Proven in all four

The Messenger, Swagger and pagination behaviours are locked by machine-checked contract suites that run against real services in every framework: a live GreenMail server for the mailbox, a real 250-row SQLite table for pagination. No mocks.

Possible breaking

Read these before you upgrade. Each affects an app that relied on the old shape:

- `Api.send()` **is now** `Api.send_request()` with no alias. Replace `api.send(method, ...)` with `api.send_request(method, ...)`.
- `to_paginate()` **takes no arguments.** The old `to_paginate(page, per_page)` form raises. Fetch the page you want, then call `to_paginate()`.
- `total` **is the true count.** It was the page's row count in some frameworks; it is now a `COUNT(*)` for the filter. Read `len(records)` for the page size.
- **The paginate envelope is seven keys.** A reader of the dropped alias keys must move to `records`, `total`, `page`, `per_page`, `total_pages`, `limit` and `offset`.
- **A Messenger** `uid` **is a real IMAP UID.** Discard any `uid` stored by an older version and re-read it; the old values were never stable across an expunge.
- `send()` **and** `read()` **changed keys.** `send()` returns `{success, message, id}`; `read()` of a missing UID returns `None`; dates are ISO-8601.
- **Swagger defaults moved.** `servers[0].url` is `/`, `info.version` is `1.0.0`, `info.description` is empty. Set `TINA4_SWAGGER_SERVERS`, `TINA4_SWAGGER_VERSION` or `TINA4_SWAGGER_DESCRIPTION` to restore the old values.
- **An unknown migration kind raises** instead of being ignored.
- **The framework no longer compiles SCSS.** Use the `tina4` Rust CLI.

v3.13.95 (2026-08-06) - Preparing for the 3.14 stable release

One of several releases still to come before **3.14 stable**. The theme is parity: the same call now means the same thing in Python, PHP, Ruby and Node. Method shapes and inputs are unchanged - this is bug fixes, optimizations, and a much harder test suite behind them. See "Possible breaking" at the end for the handful of behaviour corrections that could affect an app relying on the old, wrong result.

Queues

- The dev dashboard queue panel lists the store it counts (`f539be2`)
- The AMQP vhost is read as the path segment (`6f4ab2a`)

Logging

- An explicit argument beats the environment, which beats the default (`c56d3a1`)

Databases

- Every connect that can block is bounded, so a wedged server fails instead of hanging (`7c51d3d`)
- `count()` returns the true total, not the last page's row count (`f9b8280`)
- `to_paginate` reports the page it is on (`6f85c3f`)
- `update()` matches the primary key in data case-insensitively (`c122e6d`)
- Firebird: a column name is folded back only when Firebird folded it (`ef6139d`)
- Firebird: `table_exists` matches either spelling Firebird may have stored (`1c4dc34`)
- Firebird: `commit`/`rollback` with no open transaction are no-ops, not crashes (`4b2e38d`)

Sessions

- A backend failure is loud, then degrades, on the request path too (`59bee59`)
- No network I/O in a handler constructor (`f423ec5`)
- Every backend has a zero-dependency transport (`7e8416f`)
- `TINA4_SESSION_REDIS_PREFIX` and `_VALKEY_PREFIX` are honoured (`f7a5ba2`)
- The database backend's DDL is portable to MySQL (`c0886f5`)

Document store and cache

- `find()` is deferred, and there is one sort contract (`cae5484`, `0b2dfc1`)
- A missing MongoDB driver raises instead of falling through (`219fa36`)
- memcached TTL beyond 30 days no longer vanishes instantly (`f1abf81`)
- redis `stats()` size was a constant on the raw transport (`8c0cd5b`)
- `boto3` is declared, so a sync cannot silently delete live-S3 coverage (`fbd7dd2`)

Local by default, production by environment variable

The document store runs on a local SQLite file with no configuration and no services, and becomes a real MongoDB collection when you set one environment variable. The call sites are identical either way. The same shape holds across the framework: SQLite is the default database, the queue is file-backed until you point it at a broker, and the cache is in-memory until you give it a URL.

Bulk insert

Pass a list of rows to `insert()` and the framework builds one prepared statement and runs it inside a single transaction on a single connection, instead of a round trip per row. Same call in all four languages.

```
$db->insert("orders", [
    ["customer_id" => 1, "total" => 9.99],
    ["customer_id" => 2, "total" => 14.50],
    ["customer_id" => 3, "total" => 22.00],
]);
```

Tooling

The `tina4` CLI reached 3.8.67 over this cycle:

- `tina4 init` now wires the project up rather than printing instructions for it
- `tina4 deploy docker` gained `--runtime cli|fpm|swoole` for PHP
- The skills installer worked on macOS but installed nothing on Debian/Ubuntu. Fixed
- Skills install for Codex as well as Claude
- `tina4 metrics` gained cross-file duplication detection, Rust support, and now refuses a file it cannot parse instead of scoring it
- `tina4 serve` no longer opens a duplicate browser tab

The documentation site you are reading runs on **tina4press**, our own zero-Vue static site generator built on tina4-js. 271 pages, no framework runtime on the page.

Possible breaking

Method shapes and inputs did not change. These are behaviour corrections, so they only affect an application that depended on the previous, incorrect result:

- `count()` **in Ruby and Node** returned the number of rows the last page

produced. It now returns the true total. Code that treated it as a page size will see a different number.

- **The AMQP vhost** was read as `"/"` plus the path segment, so

`amqp://host/production` looked for `//production`. If you worked around this by naming your vhost with a leading slash, remove it.

- **Configuration precedence.** An explicit argument now beats the environment.

If you both set `TINA4_LOG_DIR` and passed a directory to `configure()`, the code now wins. Setting only one is unaffected. Ruby deployments that read `tina4.log` from the project root should read `logs/tina4.log`.

- **The Mongo session database default in Node** is now `tina4`, matching the

other three. Set it explicitly if you relied on the old default.

- **An unknown connection scheme, or an unknown** `TINA4_SESSION_BACKEND`**,**

raises instead of silently falling back to SQLite or to file sessions.

- **A missing MongoDB driver** raises rather than falling through.

v3.13.94 (2026-07-29) - A write with no filter is an error

`update()` and `delete()` with no WHERE clause used to be accepted. Two of the four frameworks then wrote every row in the table. The other two changed nothing and said they had succeeded. Neither answer is a write, and neither told you which one you got.

An unfiltered write now refuses to run and names what is missing. `truncate()` is the explicit spelling for the case where you really did mean every row.

Breaking. Read the migration note below before upgrading.

The write path (BREAKING)

A write needs a target. `update(table, data)` with no filter now takes the primary key out of `data` and uses it as the WHERE clause. With neither a filter nor the complete primary key, it raises and names the columns it wanted. `delete(table)` with no filter raises too. `truncate(table)` is the whole-table spelling, and it always was.

A failed write raises rather than returning a falsy result the caller never inspected.

Primary keys are read as a LIST, because a key can span several columns and every one of them belongs in the WHERE. Match a composite key on its first column alone and you hit every row that shares that value: the same data-loss shape the fix exists to prevent. Two of the four SQLite drivers had a second bug underneath that one. `PRAGMA table_info` returns `pk` as the 1-BASED POSITION within the key, not a boolean, so a composite key reports `pk=1, pk=2`. Code that tested `pk == 1` saw a key one column wide and truncated the rest.

Migration. Two changes to make, both mechanical:

- An `update()` that relied on the primary key travelling inside `data` still

works, and now works on composite keys. An `update()` with neither a filter nor a full primary key was already broken. Add the filter.

- Replace a deliberate delete-everything with `truncate(table)`.

Nothing else in the write path moved. A filtered write behaves as it did before.

The Frond sandbox revokes capability, it does not skip a step

`sandbox()` filtered the tag and filter lists but let a denied name fall through to the default path, so a template could still reach an escape filter it was never granted. A denied filter cannot confer safety now, and the tag gate collapsed to one check at dispatch instead of several that could disagree.

The shared render corpus grew from 72 cases to 84, covering `|safe` and `|escape`. It is one committed fixture with one answer key, byte-identical across all four frameworks, so a divergence is a failing test rather than a discovery six months later.

Messenger: one name, one signature

The dev branch and the send branch had drifted into two shapes of the same method, and capture keyed off debug mode rather than whether a transport was available. One signature behind one name now, and capture follows availability.

Base images that boot

No CI had ever built or run a base image. Two of the four had therefore never served a request at all, and the two that did boot were each wrong in their own way. `docker run` is a supported entry point, so all four are now gated in CI on Docker Hub, on `/health`, and on the version they actually serve.

The `FROM` to `COPY` recipe for adding your own database driver is documented in all four image headers, and in one case that documented line did not work until this release. An install instruction nobody runs is a claim, not a feature.

Frond, measured against the engines it replaces

The benchmark page carried seven categories and none of them timed a template. That absence flattered us, because template rendering is the one axis where Frond competes head-on with the engine it replaced. It is a headline category now, and the numbers are not kind.

Every engine renders the same page: a 20-row product list with a loop, an index, an even/odd class, an uppercase filter, two-decimal money, and a conditional footer. Output is compared byte for byte and proven identical before any timing counts, so nothing here is a strawman.

Frond loses in all four languages, on its own compiled path rather than the interpreter fallback. The competition compiles a template to code the host runtime optimises. Frond walks a tree and calls back into engine primitives per hole. What Frond buys is the zero in the dependency column, and one template language that renders the same across four runtimes.

The per-language multiples are in each framework's own note below. Publishing them costs us the comparison and buys the reader a real number.

Also in this release

MQTT test infrastructure locks its password file to `0600` and the broker's own user, so a run cannot leave a world-readable credential behind.

Python specifics

`primary_key()` took `next(c for c in columns if c.primary_key)`, the FIRST key column. On a composite key that reintroduced the exact bug the change existed to prevent. Verified against real SQLite on `order_items (order_id, product_id)`: the update popped `order_id`, built `WHERE order_id = 1`, and tried to set `product_id = 5` on both rows in that group. A UNIQUE constraint caught it. Without one it would have corrupted both rows in silence.

The Kafka raw path only runs when `confluent-kafka` is absent, and CI installs it, so that code had never executed under test. It does now: a test forces the raw path against a real broker. CRC32C is built in-process, because the standard library has none and the framework carries no third-party runtime dependency.

The published 3.13.92 image served `"version": "3.13.56"` on `/health`, 36 releases stale. `_resolve_version()` falls back from `pyproject.toml` to `importlib.metadata`, then to a floor literal. The Dockerfile pruned `dist-info`, which removed the only metadata path, so the literal won. The literal had never been bumped, because in a source checkout the first path always wins and no test ever reached it. The prune bought 20 KB of a 41 MB image. It is gone,

and a test now reads the literal out of the source and checks it against `pyproject.toml`.

Frond: 14.5x slower than Jinja2 and 37x slower than Mako on identical output. The harness confirms the AOT compiler engaged, so this is Frond's fastest path.

v3.13.92 (2026-07-27) - A token header that tells the truth about its signature

Configure `Auth` for HS512 and Tina4 minted a token whose header said HS512 over a signature computed with HMAC-SHA256. The signer reached for `hashlib.sha256` and never asked the header what it had promised.

Tina4 accepted its own tokens, because it made the identical mistake on the way back in. Anything else that reads the header and computes the digest it names got different bytes and threw the token out. Three reported issues, one contract, shipped in all four frameworks.

The header names the algorithm that signed (python#105)

`_sign()` hardcoded the digest while the header carried `self.algorithm`. HS512 produced a 32-byte signature under a header promising 64.

The digest now comes from `_HMAC_ALGORITHMS[self.algorithm]`, so the `alg` in the header is always the one that produced the bytes. HS256, HS384 and HS512 all sign and verify, and the whole family lives in `hashlib`, so nothing new was installed to get it.

The assertion that catches this measures the signature's byte length: 32, 48, 64. Comparing one algorithm's token against another's proves nothing, because the header is part of the signing input, so two tokens differ either way.

TINA4_JWT_ALGORITHM is read for real (python#106)

The variable sat in the CLI's list of known variables and was never read. Set HS512, see it acknowledged as a setting Tina4 knows about, and get HS256 tokens anyway.

`_resolve_algorithm()` reads it now. An explicit argument wins, then the environment, then HS256. An algorithm Tina4 cannot sign raises and names the supported set, because the silent downgrade was the whole defect: a weaker token than the one you asked for, and nothing said a word.

RS256 is not in the Python set, and it is not coming. PHP has OpenSSL and Node has `node:crypto`, so RSA signing costs those two nothing and stays available there. Python and Ruby would need a third-party package, and the core takes no dependencies. RS256 is a documented PHP and Node extra, permanently.

nbf is enforced, with 60 seconds of leeway (python#107)

A token stamped not-valid-until-noon was accepted at nine. `valid_token()` checked `exp` and walked straight past `nbf`.

It now refuses a post-dated token until its `nbf` arrives, and accepts one up to 60 seconds early. RFC 7519 allows a small leeway, and without one a token minted on a host a second ahead is rejected for nothing at all.

A token carrying no `nbf` is unconstrained, which is what keeps this non-breaking for every token already in circulation. `get_token()` does not stamp one. Ruby used to, and no longer does, so all four now issue the same claims.

The header cannot choose the verifier

`valid_token()` now rejects a token whose header names any algorithm other than the configured one, `alg: "none"` included, before it spends a single HMAC.

This closed no live hole. Every framework already verified with its own configured algorithm and never read the header to pick one, so a forged header changed the signing input and broke the signature anyway. What the check buys is parity with Ruby, which always had it, an exit before the wasted work, and a test that fails the day someone refactors `valid_token()` into trusting `header.alg`. That mistake has its own CVE list. Now it has a test here.

Overrides on authenticate_request are honoured

`authenticate_request(headers, secret=..., algorithm=...)` accepted both overrides and dropped them. The body called `self.valid_token()`, so a caller asking for a particular secret or algorithm quietly got the instance's own.

Both are honoured now. Their defaults moved to `None`, because the override branch tests for `is not None` and a real default recursed until the stack ran out.

A misconfigured algorithm raises (Breaking)

While the variable was ignored, `TINA4_JWT_ALGORITHM=HS-256` did nothing at all and you signed HS256 without being told. Now that the setting is honoured, a value Tina4 cannot sign has to go somewhere, and it raises a `ValueError` naming the variable and the values it accepts. A typo surfaces at the first token operation, which is where a deployment mistake belongs.

A malformed or forged token still returns `None`, exactly as before. Only the configuration error escapes.

Migration: set `TINA4_JWT_ALGORITHM` to `HS256`, `HS384` or `HS512`, or leave it unset for the HS256 default. A value that used to be swallowed now stops the first token operation and tells you why.

The tests that hold it

Twenty-seven named regression tests, a positive and a negative case per issue, no doubles anywhere. Each ran against the old code first and was watched to fail: restoring the hardcoded digest fails four of them. Full suite on macOS with Python 3.13: 3,967 passing, 114 skipped.

v3.13.91 (2026-07-27) - The offenders list finally points at code worth fixing

`tina4 metrics` ranked `public/js/frond.js` as the worst code in the framework, at cyclomatic complexity 191. Second place went to `register_dev_tools` at 139. Neither number was real. The first belongs to a file wrapped in one anonymous function. The second belongs to a registrar that declares twenty small handlers inside itself.

Both scores were the same arithmetic mistake, and it had been in every framework since the metrics module shipped.

A function is no longer charged for the functions inside it

Complexity was measured across a function's whole span. A branch inside a nested function landed on that function and on every function wrapping it. Count the same branch twice, three times, four, once per level of nesting.

The deeper the nesting, the worse the lie:

```
def outer(a):
    def inner1(x):
        if x: return 1
        if x > 2: return 2
        return 3
    def inner2(y):
        if y: return 1
        if y > 2: return 2
        return 3
    return inner1(a) + inner2(a)
```

`outer` branches on nothing. It scored 5. It now scores 1, and each inner keeps its own 3. The branches moved to where they belong. None went missing.

Python and the Rust engine walk a real syntax tree, so they stop descending at a nested function or class. PHP, Ruby and Node scan text, where that surgery would be three risky rewrites. They apply an identity instead:

```
own(F) = raw(F) - sum over direct children C of (raw(C) - 1)
```

A raw score is 1 plus every decision in the span, so `raw - 1` is the total decision count of a whole subtree. Subtract that for each direct child and what remains is the function's own work. It needs only the line and LOC each extractor already reports.

Closures, blocks, lambdas and arrow functions stay exactly where they are. None of them appear in the function list, so nothing subtracts them, and their decisions remain with the function that contains them.

Breaking: every complexity number drops, and so does every file total. A `tina4 metrics --fail-on` gate that failed on an inflated score may now pass. Run the gate once before you trust the old threshold, and lower it if the inflation was holding your ceiling up.

Ruby complexity was double what it should have been

In the tree-sitter Ruby grammar, `if`, `unless`, `while`, `when` and `rescue` name two things: the construct, and the keyword token inside it. The engine matched on the name alone, so it counted both.

`return 1 if y` scored 3. It has one branch.

Every Ruby number the engine produced came out at roughly double. The fix ignores anonymous nodes, which is what a bare keyword token is. The engine and `metrics.rb` now agree on the same file, method for method.

LOC means one thing again

Every framework counted file LOC as code lines, skipping blanks and comments. Every framework counted function LOC as a raw line span. One word, two units, sitting side by side in the same payload.

The dashboard sized its bubbles in one unit and printed its function table in the other. A function documented with care looked bigger than a function with no comments at all.

Each language now has one definition of a code line, shared by both levels, so they cannot drift apart again. `Swagger.generate` reports 167 lines in Python and 167 in the Rust engine, which is the first time two implementations have been asked the same question and given the same answer.

Fixing it in PHP surfaced a third bug. A bare `}` is a token with no line number, so the scan for a function's last line landed on the whitespace before it. Every PHP function was one line short. That line is back.

Nothing but the dashboard reads function LOC. Violations and the maintainability index both use file LOC, which was right all along, so no threshold moves.

The dev dashboard reads the coupling it is given

The bubble chart now uses the numbers the engine resolves. Files that everything depends on drift to the centre. A ring around each bubble runs blue for stable and amber for unstable. Hovering prints the real figures behind them.

The chart draws none of this unless the coupling actually resolved. The older metrics modules never mapped an import to a file, so they reported zero dependents for everything and an instability of exactly 1.0. Drawing that would paint every ring at maximum instability, which is a confident lie, so the chart leaves those channels dark instead.

Thirty-three regression tests hold all of it in place across the four frameworks, and seven more in the engine. Each was run against the old code first and watched to fail. A test that has never failed has never proved anything.

v3.13.90 (2026-07-27) - A scaffolded model and its migration finally agree

`tina4 generate crud Todo` wrote a model that declared a `name` column and a migration that never created it. The first write died:

```
table todo has no column named name
```

This sits on the path https://tina4.com/llms.txt hands to AI assistants, so the documented way to stand up a REST API returned a 500 on its first POST.

One default, defined once (php#186, ruby#33, nodejs#38)

The default field set lived inside the model template. Other generators each carried their own copy, and the migration builder had none, so it built the table from an empty field list: `id` and `created_at`, nothing else. The default now lives in a single constant that flows into the model, the migration, the form, the view and the co-emitted test alike.

Ruby hid a second trap. An empty array is truthy in Ruby, so `fields_override || parse_fields(...)` short-circuited to the empty array and the fallback never fired. That check now tests for content, not truthiness.

Python shipped this fix in 3.13.89 and missed one generator: the form kept its own inline copy of the default. The output matches while the default happens to be `name`, and diverges the moment it changes, at which point the form renders an input for a column the table does not have. Fixed, with a source invariant that fails if any generator restates the literal again.

A failed write no longer reports success

The generated create and update routes serialised their result without checking it. `create()` and `save()` report failure by return value; they do not raise. In Node a failed insert therefore returned HTTP 201 carrying unsaved data. In Ruby it surfaced as a NoMethodError on `false` and buried the real cause. Both now check the result and return 400 with a clear message. PHP already checked, and a test pins that behaviour in place.

Every generator test in all four frameworks passed `--fields` explicitly, which is why one bug survived in four languages. Each framework now exercises the no-fields path and asserts that every field the model declares reaches the migration, ending with a real write against a real database.

v3.13.89 (2026-07-27) - A typo'd tag no longer renders what it was hiding

Two Frond bugs, both present identically in all four frameworks since v3, both compatibility gaps against Twig and Jinja2. One of them was quietly showing people content that was supposed to be gated.

An unknown tag raises instead of leaking its body (Breaking, Security)

Look at this template and decide what a visitor sees:

```
{% iff user.is_admin %}
    <a href="/admin">Admin console</a>
{% endiff %}
```

Everyone. `iff` is a typo, and Frond did not recognise it, so the tag rendered nothing and its body rendered as ordinary content. The admin link went to every visitor. Worse than a broken page: the template reads as guarded, so a reviewer scrolling past sees a check that is not there.

The same shape hid behind any misspelling. `{% ifff %}`, `{% unles %}`, `{% i %}`, a tag copied from another engine. Each one silently removed its own condition.

An unrecognised tag now raises, naming it and listing what Frond does know:

```
Frond: unknown tag "iff" -- known tags are: autoescape, block, cache, extends,
for, from, if, import, include, live, macro, raw, set, spaceless
```

Frond has no plugin system for tags, so an unrecognised name is always a typo, never an extension. Twig and Jinja2 both raise on one. Frond now does too.

Two things deliberately still render nothing. A stray terminator, an `{% endif %}` with no `{% if %}`, and an empty `{% %}`. Neither has a body, so neither can expose anything, and both have always been silent.

What to do: run your templates once. A typo'd tag now fails on the page that contains it, which is where you want to find it.

set works as a block (Fixed)

```
{% set badge %}
    <span class="badge">{{ order.status|upper }}</span>
{% endset %}

<td>{{ badge }}</td>
<td class="mobile-only">{{ badge }}</td>
```

That is core syntax in both Twig and Jinja2, and until now it did the wrong thing in all four frameworks: the body printed inline where the block stood, and the variable was never bound. `{% set g %}Hello{% endset %}[{{ g }}]` rendered `Hello[]` instead of `[Hello]`.

It captures now. The captured value is marked safe, so markup you wrote in the body renders as markup rather than as escaped angle brackets, matching both reference engines. A value interpolated into the body is still escaped on the way in, which is the right place for it: the escaping happens once, where the untrusted value enters.

Blocks nest, and a loop inside the body renders into the capture rather than to the page:

```
{% set rows %}{% for i in items %}<li>{{ i }}</li>{% endfor %}{% endset %}
<ul>{{ rows }}</ul>
```

One rule decides which form you get: an `=` anywhere in the tag means assignment. So `{% set label = "a = b" %}` stays an assignment and is never read as a block. The inline form is untouched.

The expression corpus grew to 82

Three entries cover the block form: a plain capture, a capture that must not be double-escaped, and an inline `set` whose value contains an `=`. The unknown-tag rule is locked by named tests in each framework rather than the corpus, because the corpus compares rendered output and this one raises.

v3.13.88 (2026-07-27) - json_encode gives you JSON again

3.13.87 HTML-escaped the `json_encode` filter in all four frameworks. That was wrong, and this release reverts it.

Entity-encoded JSON is not JSON. `{"a":1}` inside a `<script>` block is a SyntaxError, and a script block is most of what the filter is for. One change broke all four

languages at once.

The same pass fixed a bug that had been there far longer: a value JSON cannot represent used to escape as itself, or as nothing at all.

json_encode emits JSON, not entities (Breaking, reverts 3.13.87)

`{{ product | json_encode }}` renders `{"id":1,"name":"Widget"}` again. Put it straight into a script block:

```
<script>
    const product = {{ product | json_encode }};
</script>
```

The output is still safe on the page. Frond escapes the four characters that can break out of HTML, as JSON unicode escapes rather than entities. A `</script>` inside a string arrives as `\u003c/script\u003e` and cannot close the block early. A single quote becomes `\u0027`, so the value also drops into a single-quoted attribute untouched:

```
<div data-product='{{ product | json_encode }}'>
```

Use single quotes there. JSON carries its own double quotes, so a double-quoted attribute needs `| e` on top. This is the model Jinja2 uses for `tojson`, and it is why the result is marked safe.

`| raw` after `json_encode` is now a no-op. If you added one for 3.13.87 you can delete it, and nothing breaks if you leave it.

A value JSON cannot represent becomes null (Fixed)

Reported as tina4-php#184 by justin-k-bruce, who hit it in production: two infinite sort keys in a 657-row grid blanked the whole screen.

JSON has no Infinity and no NaN. All four frameworks handled that differently and all four were wrong. Python wrote a bare `Infinity`, which no parser reads. PHP's `json_encode` returned false, which arrived as an empty string, so the payload vanished with no error anywhere. Ruby fell back to Ruby inspect output. Only Node.js already did the right thing.

Every framework now writes `null`, which is what `JSON.stringify` has always produced and the only answer the grammar allows:

```
{{ readings | json_encode }}
```

```
{"low":1.5,"high":null}
```

The rule behind it is wider than one value type. This filter never returns an empty string and never returns a token that would fail to parse. A payload always arrives, in the worst case as `null`. An empty script assignment is at least a loud error; a silently wrong one is not.

Two smaller differences went with it. Forward slashes stay unescaped, so `a/b` no longer comes back as `a\/b` from PHP. Non-ASCII text stays raw, so `cafe` with an accent is itself rather than a `\u00e9` escape from Python.

to_json and tojson are the same filter

All three names now share one serializer. They cannot drift apart again.

The `to_json` indent argument is gone. PHP's pretty printer has a fixed four-space indent and cannot honour an arbitrary width, so supporting the argument in three frameworks and not the fourth broke byte parity on the one filter whose entire job is a wire format.

The expression corpus grew to 79

The cross-framework expression fixture added seven entries: the json_encode escape shape, a non-finite value, a NaN nested in an object, the `to_json` alias, a filter pipe feeding a `~` concatenation, that same expression inside a ternary, and `number_format` with locale separators.

The last three close tina4-php#170 and tina4-php#171. Both were reported against 3.13.68 and both were already fixed by the 3.13.87 expression work; they are locked in now so they cannot come back.

What to change in your templates

Grep for `json_encode`. Three cases:

- Inside a `<script>` block: nothing to do. It works again.
- Inside a single-quoted attribute: nothing to do.
- Inside a double-quoted attribute: add `| e`.

v3.13.87 (2026-07-27) - Frond expressions agree in all four frameworks

Frond has always been described as one template language with four implementations. That was an assumption. Nobody had ever measured it.

So we measured it. Seventy-two expressions covering filters, operators, ternaries, null coalescing, concatenation, comparisons, math precedence, missing values, dotted paths, arrays, hashes, escaping and filter chains, rendered through Python, PHP, Ruby and Node.js against one identical dataset. Eleven of the seventy-two disagreed.

All eleven are fixed. Every framework now renders all seventy-two identically.

The corpus is no longer a one-off script. It ships as a test fixture in all four repositories, byte for byte identical, with a single shared answer key. If one framework drifts, its own suite turns red while the other three stay green, and the failure names the expression. Changing the contract on purpose now means changing the answer key in four repositories in the same change, which is the point.

Booleans print as true or false (Breaking)

`{{ flag }}` used to print Python's `True` and `False`. It now prints `true` and `false`, lowercase.

Lowercase is the only form a template can use directly. `data-active="true"` reads back from JavaScript as `el.dataset.active === "true"`. `True` needs translating at every site that touches it, and it is a Python artifact leaking into a template language.

This was a deliberate break with the reference implementation. The four frameworks had drifted to four different answers, so one of them had to change, and the master was the one that was wrong for a template.

Search your templates and tests for comparisons against the rendered `True` or `False`.

Both macro import forms now agree (Fixed)

`{% import "macros.twig" as forms %}` shifted every macro argument by one position. `{{ forms.greet("Andre") }}` arrived at the macro with the alias as the first argument and `"Andre"` as the second.

The macros were installed as class attributes, so Python bound them as methods and passed the namespace object in as `self`. They are now installed on a plain namespace object, which binds nothing.

`{% from "macros.twig" import greet %}` was never affected. Both forms now behave identically, which is what the two forms were always supposed to mean.

The not operator works in output (Fixed)

`{{ not user.active }}` rendered as nothing.

Every logical operator was matched with spaces on both sides, so a leading `not` with nothing to its left matched none of them, fell through to variable lookup, and was resolved as a variable literally named "not user.active". Finding no such variable, it rendered empty.

`{% if not x %}` and `{{ x and not y }}` always worked, so the operator itself was never broken. Only the standalone output expression was lost, and before booleans printed lowercase a dropped expression and a false value looked identical.

A leading `not` now routes to the same evaluator `{% if %}` uses, so a condition means the same thing in a condition and in an output expression.

json_encode escaping

Python has always escaped `{{ data | json_encode }}`, and still does. Use `{{ data | json_encode | raw }}` inside a `<script>` block.

The note records that PHP returned raw JSON from this filter and was brought in line.

The gate that keeps this true

Every one of these bugs survived because each implementation looked correct on its own. Reading the code four times would not have found them. Rendering the same expression through all four and diffing the bytes found eleven in an afternoon.

That diff is now a test. `frond_expression_corpus.txt` and `frond_expression_expected.txt` live in every framework's test directory as identical bytes, and every suite renders the corpus and compares against the shared answer key. Alongside it sit named regression tests for each bug above, each carrying the negative case that was failing before the fix.

One more thing worth naming, because it is a pattern rather than an incident. Every boolean bug was a falsy guard: `|| ""`, `a[k] || a[k.to_sym]`, `true ? '1' : ''`. In a template engine, "absent" and "false" are different things, and code that conflates them prints nothing where it should print something.

v3.13.86 (2026-07-25) - The write-result contract is now consistent across the family

Nothing changed in Python. `db.insert`, `db.update`, and `db.delete` already returned a `DatabaseResult` carrying `.last_id` and `.affected_rows`, and they still do. This note records that the other Tina4 frameworks were brought in line: PHP (which used to return a bool) and Ruby (which used to return a Hash) now return a result object from these writes too, so a write returns a result in every language.

v3.13.85 (2026-07-24) - The dev-admin bundle ships once

Every install carried the dev-admin dashboard twice. `tina4-dev-admin.js` and `tina4-dev-admin.min.js` sat side by side in the framework's public assets, byte-for-byte identical, and nothing referenced the first one. The route that serves the dashboard, the SPA shell that loads it, and the asset resolver that finds it all name the `.min.js`.

The unminified copy is gone. That is 0.92MB removed from every install, in Python, Ruby and Node.js. PHP had already dropped its copy and is unchanged.

Nothing about the dashboard changes. The file that ships is the same file that was always being served, and the two were identical anyway, so the only difference is what you download.

Why the surviving file is still called `.min.js`

It is not minified, and it never was. The two files had the same SHA-256, so the `.min` suffix described an intention rather than a fact.

Renaming it would break every project that references the asset by path, for no benefit, so the name stays. If the bundle is ever genuinely minified the name will finally be honest; until then it is simply the name of the bundle.

A gate, so the duplicate cannot come back

Nothing compared the two files, which is how 0.92MB per package went unnoticed. All four frameworks now test the shipped assets directly: the `.min.js` is present, the unminified duplicate is absent, exactly one `tina4-dev-admin*.js` exists (so a differently-named copy cannot slip through), and the surviving file is the real bundle rather than a stub.

The Ruby half of this was quietly broken in a way worth naming. Two specs read the unminified file behind a `skip ... unless File.exist?` guard. Deleting the file would have turned both into silent skips: a green suite with two dead assertions, and no signal that the asset had vanished. They now read the shipped bundle with no skip guard, so a missing asset fails loudly. A skip that hides a missing file is not a passing test.

v3.13.84 (2026-07-24) - Every generated Dockerfile actually starts

`tina4 deploy docker` wrote Dockerfiles that could not run. Building each one for real found that of the eight Dockerfile generators in the stack (four templates in the `tina4` CLI, plus one inside each framework's own CLI), exactly one was correct.

The Python image is the clearest case. Its `CMD` was:

```
CMD ["python", "-m", "tina4_python.cli", "serve", "--production"]
```

`tina4_python/cli.py` used to be a module, and `python -m` is valid for a module. The v3 restructure replaced it with the package `tina4_python/cli/`, which has no `__main__.py`, so `-m` cannot execute it. Containers died on startup with "'tina4_python.cli' is a package and cannot be directly executed". The code moved; the hard-coded string in a different repo did not, and nothing tested the generated file.

Every generator now names an entry point that exists, and asks for production mode:

language	CMD
Python	`tina4python serve --production`
PHP	`php vendor/bin/tina4php serve --host 0.0.0.0 --port 7145 --production`
Ruby	`bundle exec tina4ruby serve --production`
Node.js	`npx tina4nodejs serve --production`

All four were verified by scaffolding a project, running `tina4 deploy docker`, building the image, and starting a container.

`serve` no longer kills PID 1 (all four frameworks)

Before starting, the CLI reclaims the port from a stale dev server by reading `lsof -ti` and signalling what it finds. It never validated that output. Where `lsof` exists but prints a different shape, a non-numeric field coerced to 0 or 1, and signalling PID 0 sends the signal to every process in the caller's own process group.

In a container the server is PID 1, so it killed itself. The Node image logged `Killed existing process on port 7148 (PID: 1 ...)` and exited 143. The PHP image logged the same attempt and survived by luck.

Port reclaiming is now skipped entirely inside a container, where there is no stale sibling to reclaim from. Outside one, only all-digit PIDs are accepted, and PID 0, PID 1 and the current process are never signalled. The message reports only what was really killed.

A gate, so this cannot rot again

Nothing tested a generated artifact. The CLI's deploy tests covered argument parsing and nothing else, and the docs truth-check inspects prose, not template payloads.

The CLI now carries five contract tests over the Dockerfile templates that fail on exactly the mistakes above: no `python -m` on a package, no dev-only runner such as tsx in a production CMD, every CMD names a published entry point, every CMD requests production, and every template sets `TINA4_OVERRIDE_CLIENT`.

The port-reclaim rule is pinned down too, in all four frameworks. Deciding which PIDs to signal now lives in one pure function per language (`selectable_pids`, `tina4SelectablePids`, `selectablePids`), so the safety rule is testable without touching a real process: 43 tests assert that a junk token, PID 0, PID 1, the current process and the current process group are each refused, and that a genuine stale sibling is still reclaimed. One of them feeds in the exact line from the container log that started this.

Also in the CLI (tina4 v3.8.58)

The four bundled Dockerfile templates carry the corrected commands, and the Ruby template gains its build toolchain. Each framework's own `docker` generator was corrected to match, so the two paths agree. The PHP generator now detects whether a project has `bin/tina4php` or `vendor/bin/tina4php` and writes whichever exists, because composer does not link a root package's own bin into `vendor/bin`.

v3.13.83 (2026-07-24) - Swagger UI stops serving itself in production

Security. With swagger disabled, `/swagger/openapi.json` returned 404 and `/swagger` returned 200. The gate was real. The static files walked around it.

The framework ships the Swagger UI as files inside its own public directory. Static serving runs before route matching, and it resolves a directory to its index, so `/swagger` became `swagger/index.html` and never reached the gated route. A production server kept serving the whole UI on four paths:

```
/swagger                       200
/swagger/                      200
/swagger/index.html            200
/swagger/oauth2-redirect.html  200
```

The tell was the mismatch. The spec route obeyed `TINA4_SWAGGER_ENABLED`; the UI ignored it. Static serving now checks the swagger gate before it resolves an index, so all four paths return 404 when swagger is off and serve as before when it is on. Fixed in all four frameworks, each with a lock-in test that fails against the old code. (python#97)

The banner advertises only what answers (python#99)

Every boot printed both dev links, whatever the environment:

```
Swagger:   http://localhost:7146/swagger
Dashboard: http://localhost:7146/__dev
```

In production both URLs 404. That misled two readers at once. An operator scanning a production log believed a dev surface was exposed. A developer clicking the link landed on a dead page.

The banner now prints a row only when that surface answers. Each framework builds those rows through one pure function of the port and two booleans, so the contract is unit tested instead of inferred from stdout: `banner_surface_lines` in Python, `App::bannerSurfaceLines` in PHP, `Tina4.banner_surface_lines` in Ruby, `bannerSurfaceLines` in Node.

The CLI runs no watcher in production (tina4 v3.8.57)

`tina4 serve --production` started the file watcher and the SCSS watcher anyway. Both burn CPU, both contend with the single server process, and neither has anything to do: production compiles SCSS once at boot and never re-imports code. `--production` now starts neither. The CLI stays to supervise the server process and shut its tree down cleanly on Ctrl-C.

A stale CA no longer turns a missing certificate into six failures (python#98)

The MQTT TLS tests trusted whatever CA file happened to sit in the shared temp directory. Once that file went stale, the tests did not skip. They failed, six of them, in all four frameworks, pointing at TLS code that was correct. Each suite now verifies that the CA actually validates the broker certificate before it treats the TLS environment as present, and skips when it does not.

v3.13.82 (2026-07-23) - MQTT 3.1.1: talk to any broker, no dependency

Tina4 now speaks MQTT. The new `Mqtt` client publishes and subscribes to any MQTT 3.1.1 broker - Mosquitto, EMQX, HiveMQ, AWS IoT - and adds nothing to your dependency tree. It is built on the standard library alone (`socket`, `struct`, `ssl`), and it is the same client in all four frameworks.

```
from tina4_python.mqtt import Mqtt

mqtt = Mqtt()   # reads TINA4_MQTT_URL, default mqtt://127.0.0.1:1883
mqtt.publish("fleet/meter-42/telemetry", '{"kwh":12.5}', qos=1)

for message in mqtt.consume("fleet/+/telemetry", qos=1):
    if message.duplicate:
        continue
    store(message.topic, message.payload)
```

- **QoS 0 and QoS 1, retained messages, and a Last Will.** `publish`, `subscribe`, and `consume` mirror the Queue you already know. `consume` acknowledges a message only after your handler has processed it, so a handler that fails leaves the message for redelivery - at-least-once, which is what QoS 1 is for.
- **QoS 2 is refused, loudly.** Asking for exactly-once raises an error that names the limit and the fix - a QoS 1 consumer keyed on device id and timestamp - rather than silently downgrading to at-least-once and double-processing forever.
- **TLS with a per-client trust store.** An `mqtts://` connection verifies the broker against the CA you supply and no other. A self-signed certificate is rejected unless you provide its CA, and a CA loaded for one client never leaks into the next.

- **Username and password auth**, over plain or TLS connections, from the URL or from explicit arguments.
- **No mocks.** Every test runs against a real Mosquitto broker - the anonymous, authenticated, and TLS listeners - so the wire protocol is verified for real, not simulated.

This release also:

- **Counts 98 built-in features.** MQTT is the newest. 97 features are identical across all four languages; Ruby adds ERB as a native second template engine for 98.
- **Stops and deregisters a single background task.** `background()` returns a handle whose `stop()` ends just that task and removes it from the registry.
- **Dispatches the in-process test client through the real front controller**, so a test sees the same routing, middleware, and 404 path a real request does.
- **Ships the compiled Tina4 CSS assets** and honours SCSS `!default` instead of leaking it into the output.
- **Reaches `doctor`, `setup`, and `deploy`** from the framework CLI by delegating to the shared Rust CLI.

v3.13.81 (2026-07-21) - `tina4 test` fails the build when tests fail

A green build could hide a red suite. `tina4 test` ran pytest, then threw pytest's exit code away and always returned 0. A `tina4 test || exit 1` gate in CI or a pre-push hook passed while tests failed underneath it. This release makes the command exit with pytest's own return code, so a failing suite fails the gate.

- **The exit code now propagates.** `tina4 test` returns whatever pytest returns. A red suite exits non-zero and stops the pipeline; a green suite exits 0. Nothing else about the command changes.
- **A real lock-in test pins it.** The test spawns the actual CLI against a temp project with a failing test and asserts a non-zero exit, then repeats with a passing test and asserts 0. No mocks - the CLI runs for real.

This release re-aligns all four frameworks on one version. Python, Ruby, and Node skip 3.13.80, a PHP-only patch that shipped the `\Tina4\Test` base class; PHP moves from 3.13.80 to 3.13.81. Everyone is back on 3.13.81.

Reported as python#96.

v3.13.79 (2026-07-19) - Session cookies get Secure behind a proxy, and a renamed cookie is read back

If you run Python behind nginx, an ALB, or any TLS-terminating proxy, your session cookie shipped without `Secure` on the very deployments that were encrypted. This release fixes that and reads a renamed cookie back.

- **Security:** `Secure` **now follows the scheme the client actually used.** The response emit path in `server.py` hand-wrote the session `Set-Cookie` header and never called the cookie builder, so the documented `TINA4_SESSION_SECURE` did nothing and there was no proxy detection at all. The emit path now routes through the one cookie builder (`cookie_header`). `Secure` is set when `TINA4_SESSION_SECURE` is truthy, when `SameSite` is `None` (browsers reject `None` without `Secure`), or when the request is HTTPS - detected proxy-aware through `x-forwarded-proto`, whose first hop is the client-facing one, resolved by `Request.is_secure_scheme()`, else the native scheme.
- **Plain HTTP is unchanged.** Without a proxy header and without TLS, the cookie stays non-Secure, so the browser keeps returning it and local sessions never break silently.
- `TINA4_SESSION_NAME` **is now read back.** The write side honoured `TINA4_SESSION_NAME`, but the incoming-cookie read used a hardcoded `tina4_session` literal, so an operator who renamed the cookie wrote one name and read another, and the session silently never resumed. Both sides now resolve the name through one function (`session_cookie_name()`); the default is byte-identical to before.

Reported by justin-k-bruce (python#95). Real-server tests read the actual `Set-Cookie` off a live server driven with a real `X-Forwarded-Proto` header; the name test resumes a renamed cookie across two requests and proves the default is unchanged.

v3.13.77 (2026-07-16) - A slow background task no longer runs on top of itself

- `background()` **never overlaps a task with itself.** A sync callback that took longer than the loop's timeout used to have a second copy started alongside it, and every later tick piled on another. A callable already running in a thread pool cannot be cancelled, so the timeout abandoned the wrapper while the thread kept working. Each run is now awaited to completion before the next tick, and an overrun only logs a warning. The old warning claimed the task "was interrupted" when it was still running, which pointed away from the cause; it now says the task is still running and the next run is deferred.
- **Why it matters.** This is silent double-execution of any slow periodic sweep, in a single process, with no clustering involved. A queue drainer that reads rows and marks them one by one would process every row twice.

Reported by justin-k-bruce, who traced it to the exact lines and supplied a repro. Real-thread regression tests now pin the no-overlap invariant.

v3.13.76 (2026-07-16) - Migrations apply again on a database created before 3.13.55

If your database was created by Tina4 v3 3.13.54 or earlier, every new migration failed and none could ever be applied. This release fixes that. It is the framework that created the legacy column, so Python is where it bit.

- **The bookkeeping insert now writes the columns your table actually has.** The 3.13.55 rename added `migration_name` and left the old `migration_id` column in place, calling it harmless. It was harmless on reads and anything but harmless on writes: that column is `NOT`

`NULL`, the insert never filled it, so recording a migration raised a not-null violation, the migration rolled back, and the database was stuck. The runner now builds its insert from the table's real columns and fills any legacy one it finds. No schema change, no `ALTER`, every engine.

- **Fresh databases are untouched.** A table with no legacy column still gets exactly the six canonical columns. That is the case CI and every new project exercised, which is why this only ever bit long-lived staging and production databases.

Thanks to justin-k-bruce, who reported it against 3.13.75 with a full compatibility matrix and a working patch. Real-database regression tests now cover a pending migration on a legacy table in all four frameworks.

v3.13.75 (2026-07-14) - Static assets revalidate, so a deploy reaches users without a hard refresh

The built-in static file handler (everything under public/) now lets a browser cache an asset but forces it to revalidate on every use. A redeployed CSS or JS file reaches the browser on the next page load, with no manual hard refresh - and an unchanged file costs a cheap 304 Not Modified, not a full re-download.

- **Cache-Control and validators on every static response.** Each asset carries `Cache-Control: no-cache, must-revalidate`, an `ETag`, and a `Last-Modified`. Before, the static path set no Cache-Control and no Last-Modified.

- **Conditional requests get a 304.** The handler answers `If-None-Match` and `If-Modified-Since` with a `304 Not Modified` and no body, so a revalidation is a small round trip rather than a re-download. Real-file, real-request tests lock the behaviour in.

This lands identically across Python, PHP, Ruby, and Node.js. It closes the class of "I already reported this" where a browser kept serving a fixed-but-cached front-end asset.

v3.13.74 (2026-07-13) - The dev dashboard connection tester works again

The dev dashboard has a "Test connection" panel that opens a database URL and reports the server version and the table count. On Python it failed on every call: the handler imported `tina4_python.Database` (capital D), a module that does not exist, so it always returned `{"success": false}` and never reached `db.get_tables()`. This release imports the real `tina4_python.database` module, so the panel connects, lists the tables, and shows the version.

- **The connection tester reports the real table count.** A real-SQLite test (no mocks) now drives the endpoint end to end against a live database with two tables and asserts the count and version come back. The endpoint had no end-to-end coverage before, which is exactly how the broken import shipped unnoticed.

The same panel was broken in different ways in PHP and Node.js and is fixed there too; Ruby was already correct and gained the same lock-in test.

v3.13.73 (2026-07-13) - A failed migration re-applies cleanly

This release makes a previously-failed migration run again on the next migrate, at full parity across the four frameworks.

- **A leftover `passed = 0` row no longer wedges the next run.** When a migration succeeds, the runner deletes any existing bookkeeping row for that migration name before it writes the fresh `passed = 1` row. A migration that failed earlier - whether you recorded it with `record_migration(name, batch, passed=0)` or carried it over from a v2 table - re-applies cleanly instead of colliding on the unique `migration_name`. The `tina4_migration` table holds at most one row per migration, and the latest run wins. The delete-then-insert path is identical on every engine, so there is no dialect-specific behaviour to reason about.

- **The v2 upgrade tells you what happens next.** When the v2 to v3 upgrade finds `passed = 0` rows, it logs that those migrations re-apply on the next migrate, instead of asking you to clear them by hand.

v3.13.72 (2026-07-12) - Frond number_format, filter precedence, and a sandbox fix

This release sharpens the Frond template engine, locks in a database error contract, and brings the dev dashboard to parity across the four frameworks.

- `number_format` **reads all three arguments.** The filter now honours the full Twig signature, `number_format(decimals, decimalPoint, thousandsSep)`:

``twig {{ 1234.5 | number_format(2, ',', '.') }} {# renders 1.234,50 #} ``

The one-argument form is unchanged, so every existing template behaves as before. (php#170)

- **The filter pipe binds tighter than concat.** `|` now groups before `~`:

``twig {{ amount|number_format(2) ~ ' EUR' }} {# (amount|number_format(2)) ~ ' EUR' -> 1,234.50 EUR #} ``

The rule holds at any nesting depth, including both branches of a ternary. On Python it also clears a latent case where a pipe inside parentheses rendered empty. (php#171)

- **The sandbox allow-list covers every filter path (Security).** A filter applied inside a `~` concatenation or a ternary condition now respects the `{% sandbox %}` filter allow-list. A filter you did not allow-list no longer runs its code in sandbox mode.

- **A malformed request path was already safe here.** The Node worker gained a guard this release for a path like `//` (or `///`, `/\`); Python never crashed on it and returns a normal 404.

- **Database errors still fail loud (#57).** `execute()` and `fetch()` raise on failure and record the message on `get_error()` rather than returning `False` or an empty result. This shipped in 3.13.38; this release adds a real-PostgreSQL regression test across all four frameworks so it can never slip back to a silent failure.

- **Dev dashboard parity (`TINA4_DEBUG`).** The dev-admin dependency installer (`deps/install`), the grounding-token proxy, and the Migrate, Test, and Seed run-chips now

match across all four frameworks. This is development-only; nothing changes in production.

v3.13.71 (2026-07-11) - AI skills: sharper tina4_code guidance

A skills-and-docs release; no change to the Python package. The bundled Tina4 AI skills now state WHY `tina4_code` is deprecated: in a boot-and-verify gate (scaffold the output, boot it, run it) `tina4_code` failed where a strong model grounded with `tina4_context` passed, so the tools point to grounding plus a strong model over the self-hosted coder. The recommendation is unchanged - ground with `tina4_context` and write the code yourself; only the rationale is sharper. Running `curl -fsSL https://tina4.com/install-skills.sh | sh` now installs these updated skills by default.

v3.13.70 (2026-07-11) - Column defaults on INSERT, a Firebird charset override, and stacked Swagger metadata

ORM honours column defaults on INSERT (#165)

An INSERT now omits any column you never set on the model, so the database applies that column's own DEFAULT. Previously `save()` serialised every declared column, sending an explicit NULL for the ones you left alone; a `NOT NULL DEFAULT <x>` column then failed the insert, because a DB default applies only when the column is omitted, not when NULL is passed. The rule is now precise: a column you never touch is omitted and the database fills it in; a column you explicitly set to `None` is written as NULL; a column with a non-None ORM default is still written. When every insertable column is unset, the row inserts with the engine's all-defaults form. UPDATE is unchanged. Shipped across all four frameworks, verified with real-SQLite positive and negative tests.

Firebird connection charset override (#160)

The Firebird adapter no longer hardcodes the connection charset to UTF8. You can override it, in precedence order, with a `?charset=` query on the connection URL (`firebird://host:port/path?charset=NONE`), an explicit `charset=` argument on `connect()`, or the `TINA4_DATABASE_CHARSET` environment variable. The default stays UTF8, so nothing changes unless you ask for it. This fixes double-encoded UTF-8 bytes read from a legacy NONE database. Shipped across all four frameworks.

Stacked Swagger metadata all survives (#59)

Stacking `@summary`, `@description`, and `@tags` on one route now keeps every decorator's value in the generated OpenAPI spec, whatever the order. Python and Ruby were already correct; the fix landed in PHP and Node, and all four now carry a lock-in test so the behaviour cannot drift again.

v3.13.69 (2026-07-10) - The Api client grows up: uploads, downloads, and safe redirects

The built-in HTTP `Api` client gained four zero-dependency capabilities, shipped across all four frameworks. All are opt-in and non-breaking in Python.

- **Multipart upload.** `Api.upload()` POSTs a `multipart/form-data` body from a file on disk (`file_path`) OR from in-memory bytes (`file_bytes` plus `filename`), with optional extra text fields, so you never need a temp file. The part Content-Type is guessed from the filename. A missing file or no source returns a clean error dict and never raises.
- **Streaming download.** `Api.download()` streams a GET body straight to disk in 64KB chunks, so a multi-megabyte file never lands in memory whole. It returns the status, headers, and the on-disk `path` (there is no `body` key), and writes nothing on an error status.
- **Transport seam.** An injectable `transport` lets application developers unit-test code that calls an `Api` without a live server. Tina4's own suite never injects a fake (the no-mock rule stands); every framework test hits a real local server.
- **Opt-in cookie jar.** Pass `cookies=True` for a per-client in-memory jar: the client parses `Set-Cookie` (leading `name=value`, last write wins) and replays the accumulated `Cookie` header on later requests.

The cross-origin redirect protection now also covers the cookie jar: on a redirect to a different scheme, host, or port the client strips BOTH the `Authorization` and the `Cookie` header, so neither a bearer token nor a session cookie can leak to a host you did not authenticate to. Same-origin redirects keep them.

Also shipping

- **REPL database env fix.** The interactive console now connects using the project database URL, credentials, and bind, matching the running application, instead of a hand-rolled connection that ignored them.
- **AI coder rule-path skill.** The AI coding-assistant scaffolder writes each tool's rule and context files to the correct path, across all four frameworks.

v3.13.68 (2026-07-10) - Parity release

A version bump to keep all four frameworks on the same number. No functional changes to the Python package since 3.13.67; the accompanying change is a fix to the Tina4 maintainer AI skill so it links plan and source files by a path that resolves when clicked.

v3.13.67 (2026-07-10) - The MCP table browser, locked down

The `database_tables` dev-tool now has a real behavioural test. A sibling bug in the PHP framework (#164) fataled the same tool: it called a method that does not exist instead of listing tables, and the test never caught it because it only checked the tool was registered, never that it ran. This framework already listed tables correctly, but carried the same blind spot in its own tests. The new test invokes the real handler against a real SQLite database and asserts a table list comes back, so this class of drift is caught here too. Shipped across all four frameworks.

v3.13.66 (2026-07-10) - A self-describing CLI, and generators that ship their tests

The command line grew up. The `tina4` client no longer keeps its own copy of what each framework can do. It asks the framework, then forwards the request. A command you add to the framework shows up in the client automatically; one you remove simply stops being offered. The whole class of client-versus-framework drift is gone.

The self-describing CLI

- `commands` / `commands --json`**.** Every framework CLI now prints its own command

table from a single source. The `tina4` client reads that manifest to render an accurate `tina4 --help`, caches it by a cheap fingerprint of the resolved CLI, and refreshes with `--refresh`. Dispatch never depends on the manifest, so a discovery miss shortens the help listing and never breaks a command.

- **Pass-through dispatch.** The client keeps its own conductor commands (serve, scss,

setup, init, deploy, agent, doctor, install, update, build) and forwards everything else to the framework verbatim. It carries no per-command flag knowledge, so it can never fall out of parity.

- `queue` **is now a top-level command** in all four frameworks: `queue work`,

`queue stats`, `queue retry`, `queue clear`, wired to the real queue. Run a worker straight from the CLI instead of only scaffolding one.

- `build` **builds the deployable Docker image** (`docker build`), replacing the old

library-packaging behaviour. `build` produces the image; `deploy` ships it.

- **Ruby gains** `migrate:create`, matching the other three.

Generators now ship a test with the code

Every code-producing `generate` subcommand writes a real, passing test next to the code it scaffolds. `generate model Product` also gives you a `Product` test that talks to a real database. The tests use real collaborators (real SQLite, a real test client, a real queue), never mocks, and pass the moment they are generated.

Writing those tests exposed real bugs in the scaffolds that no string-matching test would have caught: the generated `auth` current-user endpoint rejected valid tokens in Python and PHP, and the generated migration created then immediately dropped its table in Ruby and PHP. All four are fixed and locked in by the new tests.

Databases

- **PHP silent PDO fallback.** SQLite and PostgreSQL adapters prefer the native

extension and fall back to the matching PDO driver when it is missing. The developer gets a working database either way, with identical behaviour (native types, raw-byte BLOBs, last insert id, transactions, fail-loud errors).

- **ORM** `where()` **takes an order.** `Model.where(...)` now accepts `order_by` /

`orderBy` (and Ruby also gains `limit`/`offset`), matching `find()`, `all()`, and the query builder.

Fixes

- The Frond browser helper now applies a 30 second request timeout by default.
- SQLite datetime adapters no longer emit a deprecation warning on Python 3.12+.
- Node route handlers accept a `void` return in TypeScript without a type error.

Breaking

- **Frond `request()` now times out after 30 seconds by default.** A request that

used to hang forever now fails after 30 seconds and calls `onError`. Pass `timeout: 0` to restore the old unbounded behaviour, or a millisecond value to set your own.

- `build` **changed target.** It now builds a Docker image rather than packaging the

framework as a library. Projects that relied on the old behaviour should call their packaging tool directly.

- `generate` **writes an extra test file per scaffold.** If you script generation and

assert on the exact set of created files, expect one more file (the co-emitted test).

v3.13.56 (2026-07-08) - Skills that own up when they drift

Every AI skill now tells the assistant how to report itself when it is wrong. A skill is documentation, and documentation drifts. When a skill still describes a method, default, or column the framework no longer has, an assistant writes confident code against an API that is gone. This release closes that loop.

Every skill, and every project context file the AI installer writes (CLAUDE.md, .cursorules, .github/copilot-instructions.md, .windsurfrules, CONVENTIONS.md, .clinerules, AGENTS.md), now carries one instruction: if Tina4 behaves differently from what the skill describes, that is a bug in the skill. Tell the developer, then report it at https://tina4.com/report-a-skill. The report lands as an issue on the documentation repository, gets fixed at the source, and ships to everyone.

The skills themselves are corrected too. The ORM soft-delete guidance now names the real `is_deleted` column (it wrongly said `deleted_at`), the tina4-js signal-persistence reference ships alongside the skill, and the per-framework skill copies are back in sync with the canonical set.

The web framework runtime does not change in this release. Update your installed skills with `curl -fsSL https://tina4.com/install-skills.sh | sh` (re-run to refresh in place).

v3.13.55 (2026-07-07) - One migration tracking schema on every engine

The `tina4_migration` bookkeeping table now has the same shape on every framework and every engine. Before this release the four frameworks each named and typed the tracking table a little differently. A project that moved between them, or a tool that read the table directly, met a different schema each time.

The canonical table is six columns: an auto-increment `id`, a `migration_name` (unique, the migration file stem), a `description`, a `batch`, an `executed_at` timestamp, and a `passed` flag. The auto-increment and the column types follow the engine: `AUTOINCREMENT` on SQLite, `SERIAL` on PostgreSQL, `AUTO_INCREMENT` on MySQL, `IDENTITY(1,1)` on SQL Server, and a generator on Firebird.

Existing installs upgrade in place, and no applied migration re-runs. The runner detects the old name column (`migration_id` in Python, `migration` in PHP, `name` in Node; Ruby already used `migration_name`), adds `migration_name`, copies the values across, and backfills the new columns. The old column stays where it is, ignored. A migration already marked applied stays applied.

No new third-party dependencies. Shipped across all four frameworks.

v3.13.54 (2026-07-07) - Migrations honour the SET TERM directive

A Firebird trigger or stored procedure now survives the migration splitter. Those bodies end their inner statements with a semicolon, the same character the runner uses to separate one statement from the next. Run under the default terminator, a trigger body split apart on its own punctuation and the migration failed.

A `SET TERM` line fixes it. Wrap the block in the universal isql idiom and the runner switches its active terminator, so the whole body travels as one statement:

```
SET TERM ^ ;
CREATE OR ALTER TRIGGER t_bi FOR t ACTIVE BEFORE INSERT AS
BEGIN
  IF (NEW.id IS NULL) THEN NEW.id = GEN_ID(GEN_T, 1);
END^
SET TERM ; ^
```

The runner consumes each `SET TERM` line instead of sending it to the engine, restores the previous terminator when the block ends, and handles a multi-character terminator such as `!!`. A migration with no `SET TERM` splits on the semicolon exactly as before. Shipped across all four frameworks.

PHP and Ruby also repair the Firebird v2 to v3 upgrade. Firebird returns column names in upper case, so the migration tracker read a null migration name and treated every applied migration as pending, re-running the lot. The tracking-table reads now normalise to one key shape. PHP additionally records a row on an upgraded table with its original 14-character id and detects the Firebird dialect through the Database facade.

Thanks to justin-k-bruce for the contribution.

v3.13.53 (2026-07-06) - JSONField: JSON document columns

Store a JSON document in a column. A model field can now hold a whole object or array. The framework encodes it to JSON when it writes and decodes it back to a native dict when it reads, so the attribute is always live data, never a raw string.

```
class Event(ORM):
    payload = JSONField()          # dict or list
```

The column type follows the engine. PostgreSQL gets native `JSONB`, MySQL native `JSON`, SQL Server `NVARCHAR(MAX)`, Firebird a text `BLOB`, and SQLite `TEXT` (queryable through JSON1). One field declaration, the right column on every database.

A value that cannot be encoded to JSON does not slip through. `save()` fails loud: it rolls back, returns `False`, and records the cause, so a half-written row never reaches the table. A mutable default is copied per instance, so two records never share and mutate the same object.

No new third-party dependencies.

v3.13.52 (2026-07-04) - Frond live blocks, pgsql:// URL scheme, SCSS colour functions

Frond live blocks. A page can now carry a region that keeps itself current. Wrap the region in `{% live %}` and Frond paints it on the server with the first request, then refreshes it over the transport you name.

```
{% live "prices" poll 5 %}
  <ul>{% for row in rows %}<li>{{ row.name }}: {{ row.price }}</li>{% endfor %}</ul>
{% endlive %}
```

The block renders server-side on first paint, so a crawler and a client with no JavaScript both see real content. After that it refreshes on its own. `poll N` re-fetches every N seconds. `sse` streams updates over Server-Sent Events. `ws "/path"` rides a WebSocket route you already own. A data provider feeds every refresh: `@live_source` in Python, `Frond::liveSource` in PHP, `Frond.live_source` in Ruby, `Frond.liveSource` in Node. The provider re-runs with the live request, so a block that reads the signed-in user reads it again on every refresh, and an authenticated block cannot serve one user another user's data. For poll and SSE, Tina4 mounts one always-on endpoint, `GET /__frond/live/{name}`. For a WebSocket block, `push_live(name, data)` re-renders the block and broadcasts the fresh HTML to every client on that path. Nested live blocks are rejected, and a block's optional `src` attribute is same-origin only.

One client script drives it, `frond.js`, byte-identical across Python, PHP, Ruby, and Node. No build step. No framework on the page.

pgsql:// is a Postgres URL scheme again (#58). A connection string like `pgsql://user:pass@host/db` was rejected. v3 registered only `postgresql://` and `postgres://` and dropped the older spelling, but `pgsql` is the scheme PDO, Laravel, and Doctrine all use, so real config files carried it and Tina4 refused to start. `pgsql://` now resolves

to the PostgreSQL driver in all four frameworks, next to the two existing spellings. Same driver, three accepted names.

SCSS colour functions evaluate at compile time. `rgba(#3498db, 0.5)` used to pass through to the stylesheet as literal text, and the browser dropped the whole rule because `rgba()` cannot take a hex string. The built-in SCSS compiler now evaluates the colour functions: `rgba(#hex, a)` and `rgb(#hex)` expand to real channel values, `mix(c1, c2, weight)` blends two colours, and `lighten()` and `darken()` shift a colour through HSL. The output is byte-identical across all four compilers, down to the same integer rounding, so a shared stylesheet renders the same colour whichever framework served it.

No new third-party dependencies.

v3.13.51 (2026-07-03) - MCP Streamable HTTP transport, Firebird fixes

The built-in dev MCP server now speaks the current MCP Streamable HTTP transport, the one Claude Code and today's MCP clients expect. It still answers the older 2024-11-05 HTTP+SSE transport, so nothing that already worked stops working.

One endpoint carries the whole session. A client POSTs JSON-RPC to `/__dev/mcp` and reads the response inline. `initialize` mints a session and returns it in an `Mcp-Session-Id` header; the client sends that header back on every later call. A request with an unknown session gets a `404`, its cue to initialize again. A notification returns `202`. `GET` on the endpoint returns `405` with `Allow: POST, DELETE`, and `DELETE` ends the session. The server negotiates the protocol version: it echoes the version a client asks for when it can speak it, otherwise it picks the newest one it knows.

Tina4 writes the connection details to `.claude/settings.json` for you, now with `"type": "http"` and the bare `/__dev/mcp` URL. Prefer the command line? Run `claude mcp add --transport http tina4-dev http://localhost:7146/__dev/mcp`. The change lands uniformly across Python, PHP, Ruby, and Node. Python and Node keep a full persistent legacy SSE stream; PHP and Ruby serve the current transport plus a one-shot legacy handshake, with no long-lived connection required.

Why the transport slipped past us. Our MCP tests spoke our own JSON-RPC shape over the endpoints we built, never the wire a real client speaks. They stayed green while a real Claude Code client could not connect. Every framework now ships a no-mock transport test that drives the real session lifecycle, and each was verified end to end against a live server booted through the tina4 CLI.

Firebird (PHP). Three reported issues are fixed. The migration runner and the ORM now call `execute()` rather than the old `exec()` (#120). Parameterized DML no longer throws a type error when it fetches the last insert id (#121). NULL parameters bind correctly again, rewritten to a literal `NULL` because the ibase driver cannot bind a PHP null (#123). The `exec()` method stays as a deprecated alias for `execute()`.

Migration recording on upgraded schemas (PHP). The fail-loud `execute()` surfaced a quiet bug. On a database whose `tina4_migration` table had been upgraded in place from the old v2

layout, a new migration never recorded itself, so it re-ran on every boot. The old `exec()` had swallowed the constraint error. The runner now supplies the legacy `migration_id` column when it is present, so a migration records once and stays recorded.

No new third-party dependencies.

v3.13.50 (2026-07-02) - Path route params match INTEGER primary keys on SQLite (Ruby fix)

A route path parameter like `{id}`, matched against a real HTTP request, must find an INTEGER primary-key row. On Ruby it did not. Rack delivers the request path as ASCII-8BIT, so an untyped `{id}` capture reached the SQL bind as a binary string, and the sqlite3 gem bound it as a BLOB. SQLite gives a BLOB no numeric affinity, so `WHERE id = ?` never matched an INTEGER column - `GET /api/users/{id}` returned 404 for a row that plainly existed (and `GET /api/users` listed it). The router now relabels path captures as UTF-8 so they bind as TEXT, which SQLite coerces to the column's integer affinity, and the row matches. Typed `{id:int}` params were never affected - they cast to an Integer. The SQLite driver is left alone on purpose: coercing every binary string there would corrupt genuine BLOB writes, so the encoding is fixed at the source (the router).

Python, PHP, and Node were confirmed unaffected - their string path params already bind as TEXT - and each gains a real regression test: real router extraction feeding a real SQLite integer-primary-key lookup, no mocks, so the contract cannot silently drift. No new third-party dependencies.

v3.13.47 (2026-06-25) - Open-issue batch: migration comment splitting, global middleware, SCSS interpolation

Four reported bugs, fixed and locked in with tests against the real thing.

Migration statement splitter (#54). A migration whose SQL carried a `;` inside a `-- ...` line comment fragmented into broken pieces, because the runner split on the `;` delimiter before it stripped comments. A `CREATE TABLE` with a trailing `-- drop then re-add; old way` comment raised "incomplete input" on SQLite. The splitter is now a single-pass, quote- and comment-aware scanner: it strips `--` line and `/* */` block comments, copies single- and double-quoted string literals verbatim (honouring the `''`/`""` escape), keeps `$$`/`//` stored-procedure blocks intact, and splits on the delimiter only outside all of that. A `;` or `--` inside a comment or a string literal can no longer split or corrupt a statement. Named regression tests plus an end-to-end migrate against a real temp SQLite database lock it in, with no mocks.

Global middleware now runs (#55). Middleware registered globally with `Middleware.use(...)` or `Router.use(...)` never executed - the request dispatcher only iterated each route's own middleware list and never read the global registry, and `Router.use` wrote to a private list nothing consulted. Global middleware now folds into every route's before and after run, in registration order, deduped against route-level middleware. PHP, Ruby, and Node already dispatched globals; each gains a lock-in test so the contract never regresses.

Hot-reload no longer duplicates modules (#53). The dev-reload auto-discover re-imported a `src` module that another file had already pulled in transitively, minting a fresh module object while the earlier importer kept the old one - module-level singletons silently diverged. Discovery now records a baseline for a transitively-loaded module instead of re-importing it, and only ever reloads a module it loaded itself. A genuine edit still hot-reloads on the next pass; a regression test covers both paths.

SCSS `#{}` interpolation (#116). The SCSS compiler did not support interpolation, so `calc(100% - #{$gap})` left the `#{...}` in the output and corrupted the CSS around it. The compiler now resolves `#{ ... }` before variable substitution and nesting: a `$variable` inside the braces resolves to its value and anything else inlines verbatim, so `calc(100% - #{$gap})` becomes `calc(100% - 20px)` and `.icon-#{$name}` becomes `.icon-home`. Shipped across all four frameworks for parity.

No new third-party dependencies. Full suite: 3,183 passing.

v3.13.46 (2026-06-24) - Atomic batch insert on SQLite + full batch-contract tests

A batch insert that hits a bad row must leave the table untouched, not half-written. SQLite's execute_many ran the batch with no transaction wrapper, so a row that violated a constraint mid-batch raised but left the rows before it applied in an open, uncommitted transaction - a partial write. It now wraps a standalone batch in one transaction and rolls the whole batch back on any error, all-or-nothing, matching PostgreSQL, MySQL and MSSQL (which already committed the batch as a unit). The batch-insert tests now assert the full contract against every live engine - all rows read back, the affected-row count, a single-row insert, an empty-array no-op, and the atomic rollback - with no mocks. No new third-party dependencies.

v3.13.45 (2026-06-24) - MSSQL insert row-count fix + real-service test hardening

Running the batch-insert tests against a live SQL Server for the first time exposed a real adapter bug. `execute()` read the affected-row count at the end of the call, but for an INSERT it had already run `SELECT SCOPE_IDENTITY()` on the same cursor, so the count it returned reflected the identity probe rather than the insert. Every MSSQL INSERT reported an `affected_rows` of zero. The rows landed and `last_id` was correct, but a batch insert summed those zeros and looked like it had done nothing. The adapter now captures the count straight after the main statement, before the identity probe overwrites it. The fix is INSERT-only - UPDATE and DELETE never run that probe and were already correct. The live MySQL and MSSQL batch tests now build their connection from the standard `TINA4_TEST_*` host and port variables, so they run for real against the provisioned engines instead of skipping on an absent URL. No new third-party dependencies.

v3.13.44 (2026-06-24) - Real-service bug-fix sweep (no mocks)

Standing up live infrastructure - PostgreSQL, MongoDB, Redis, Valkey, Memcached, RabbitMQ, Kafka - and running the suites against the real services surfaced a batch of bugs that mock-based and skipped tests had hidden. This release fixes them across the family and makes the no-mock rule absolute: a test that touches a dependency exercises the real service, never a stand-in. **Migrations on PostgreSQL/MySQL/MSSQL:** the migration runner built its bookkeeping table (`tina4_migration`) with SQLite-only DDL (`id INTEGER PRIMARY KEY AUTOINCREMENT`) for every non-Firebird engine, so `migrate()` failed on PostgreSQL with a syntax error and never applied a single file. The tracking-table id is now engine-aware - `SERIAL` on PostgreSQL, `AUTO_INCREMENT` on MySQL, `IDENTITY` on MSSQL, `AUTOINCREMENT` on SQLite. The existing migration tests only covered SQLite, which is why this shipped; a gated live-PostgreSQL migration test now guards the engine-aware path. **Kafka:** dequeue waits for the consumer-group partition assignment on first subscribe, so a single short poll right after produce no longer returns nothing. **RabbitMQ:** the raw-socket fallback negotiates `Connection.TuneOk` from the broker's proposal - the hardcoded `channel-max=0` made a standard broker abort the handshake. The pika path was already correct. No new third-party runtime dependencies. All four suites pass against live services.

A parity sweep shipped in the same release. The live Redis, MongoDB, and Valkey session round-trips run by default. They gate on the service being reachable rather than on an opt-in environment variable, so the real write, read, and destroy path runs on every test pass with the infrastructure up. Under `TINA4_REQUIRE_SERVICES` a provisioned backend that is unreachable becomes a hard failure rather than a silent skip.

MySQL and MSSQL join the provisioned test services (#262). Both engines now run live round-trip tests by default, gated on reachability the same way the other services are. The non-skippable real-service gate fails on a MySQL or MSSQL skip under `TINA4_REQUIRE_SERVICES`, so a missing engine in CI breaks the build instead of passing quiet. CI gained a MySQL 8 container and a SQL Server 2022 container. Running the suites against these two engines for the first time surfaced adapter bugs that no prior test could reach.

One of those bugs lived in the master itself. The MSSQL row-count probe wrapped the user query in `SELECT COUNT(*) FROM (...)`, and a query ending in `ORDER BY` put that clause inside a derived-table subquery. SQL Server rejects an `ORDER BY` in a subquery without `TOP`, so the probe reported a count of zero. The probe now strips a trailing top-level `ORDER BY` before it wraps the query. This was a genuine Python bug, not a mirror artefact - the master carried it too. Two adapter behaviours were already correct here and earned regression tests to lock them in: a raw boolean binds as `1`/`0` at the parameter boundary, and `last_id` after an insert returns the new auto-increment or `IDENTITY` value and survives a later `SELECT`.

The no-mock rule reached further this release (#250). The messenger SMTP and IMAP tests now talk to a real GreenMail mail server, the WebSocket backplane tests run against a real Redis backplane, and the HTTP-client tests hit a real loopback HTTP server. Every in-test double in those paths is gone. The dev mailbox gained a non-ASCII round-trip regression test for parity with the family: write a message with an accented name, read it back, confirm the bytes survive. Python escapes non-ASCII to ASCII in its message JSON, so it never carried the decode bug that bit Ruby, but the test now guards ~~the contract on every engine.~~

v3.13.43 (2026-06-22) - Queue lifecycle correctness + cross-framework unification

A live MongoDB producer/consumer test exposed four queue bugs, and fixing them across the family surfaced a deeper split: the four frameworks ran three different queue-lifecycle architectures. This release unifies them on one model - the active backend (whatever `TINA4_QUEUE_BACKEND` selects) owns the whole lifecycle - and fixes the bugs. `consume(topic)`/`process(topic)` **now honour the topic argument** - they previously read the construction-time topic, so consuming a topic the queue was not built with silently drained the wrong queue (every backend). **MongoDB retry timing:** a failed job is retryable on the next poll instead of being stranded for the full visibility window - `reject`/requeue now resets `available_at` (or `now + retry_backoff`) and clears `reserved_at`. **MongoDB attempts:** `dequeue` surfaces the live document attempt count, not the push-time snapshot, so `fail()` dead-letters at `max_retries` instead of retrying forever. `dead_letters()`/`retry_failed()` **signatures:** the MongoDB, RabbitMQ, and Kafka adapters now accept the `max_retries` keyword the queue passes - calling `queue.dead_letters()`/`retry_failed()` on those backends previously raised `TypeError`. A cross-backend contract test locks the signatures in. Regression tests cover topic targeting (engine-agnostic) and the MongoDB retry/attempts/dead-letter paths; verified end-to-end against a live MongoDB. Full suite: 3,280 passing.

v3.13.42 (2026-06-22) - Swagger configurability for external and public APIs

Closes four gaps that pushed teams to hand-roll their own OpenAPI spec instead of using the built-in generator. **Configurable security schemes:** the built-in `bearerAuth` scheme honours `TINA4_SWAGGER_BEARER_FORMAT` (default `JWT`; set `opaque` for `sk_live_`-style keys), and setting `TINA4_SWAGGER_API_KEY_NAME` emits an `apiKeyAuth` scheme (`TINA4_SWAGGER_API_KEY_IN` is `header`/`query`/`cookie`). Register any scheme - including an `oauth2` flow with scopes - programmatically with `Swagger.add_security_scheme(name, definition)` (`Swagger.reset_registry()` clears it). **Per-route security:** a route declares its own requirement with `@security("oauth2", scopes=["read:users"])`, a `{name: [scopes]}` map, a list of maps for an OR requirement, or `@security("public")` to mark a write route open (emits `security: []`). A secured route with no explicit declaration falls back to `TINA4_SWAGGER_DEFAULT_SCHEME` (default `bearerAuth`). Scopes stay spec-valid: only `oauth2`/`openIdConnect` schemes carry them, every other type gets `[]`, so the output validates against 3.0 and 3.1. **Path filtering:** `TINA4_SWAGGER_INCLUDE` documents only routes whose path starts with one of its comma-separated prefixes; `TINA4_SWAGGER_EXCLUDE` drops matching prefixes; framework internals (`/swagger`, `/__dev`) are always excluded. **OpenAPI 3.1 opt-in:** `TINA4_SWAGGER_OPENAPI` (default `3.0.3`) emits `3.1.0` when set to `3.1`/`3.1.0`. **Reusable component schemas:** register a shared shape with `Swagger.add_schema(name, schema)` and reference it from a route with `@request_schema(name)` / `@response_schema(name, status, is_list)`, extending the ORM-model `$ref` mechanism to arbitrary schemas. Identical behaviour and tests across all four frameworks. Zero new third-party dependencies. Full suite: 3,266 passing.

---

v3.13.41 (2026-06-22) - Queue reservation/visibility timeout (at-least-once delivery)

A targeted fix for silent job loss in multi-replica / rolling-deploy setups. When a consumer reserved a queue message and then died before acknowledging - a crash, an OOM kill, a Kubernetes pod eviction - the message was stranded forever: never re-delivered, never retried, never dead-lettered. The file backend deleted the job on pop (lost outright); the MongoDB backend flipped the document to `reserved` without advancing `available_at` and never re-evaluated it. Now a popped job is held as a reservation with `available_at = now + visibility_timeout` (plus a `reserved_at` stamp). If the consumer does not acknowledge in time, the next pop reclaims the abandoned reservation: it increments `attempts` and re-enqueues the job, or dead-letters it once it has hit `max_retries`. A dead consumer can no longer strand a job - standard at-least-once delivery, the contract SQS and RabbitMQ already provide. The window is configurable via `TINA4_QUEUE_VISIBILITY_TIMEOUT` (default 300 seconds; `<= 0` disables the reclaim) or the per-queue `visibility_timeout` option; `complete()`/`fail()`/`retry()` clear the reservation. RabbitMQ and Kafka are unchanged - the broker already owns redelivery there. Regression tests lock the behaviour in across all four frameworks (file backend: reclaim after the timeout, no reclaim before it, dead-letter past max_retries, complete/fail clear the reservation, env override, disable-at-zero; MongoDB: dequeue advances `available_at` and the reclaim requeues or dead-letters). Zero new third-party dependencies. Full suite: 3,250 passing.

v3.13.40 (2026-06-22) - MCP security hardening + Swagger/OpenAPI overhaul

A coordinated cross-framework release with two themes: MCP transport security and a full Swagger/OpenAPI sweep. **MCP security:** the built-in dev MCP server now authorises every request on the raw socket peer rather than a configured host name, closing a remote-reach surface where a debug box bound to `0.0.0.0` exposed the file and database tools to unauthenticated callers. The gate is two layers - a host-independent capability check (`TINA4_MCP`, else `TINA4_DEBUG`) and a per-request authorisation: loopback always passes, while a remote caller needs `TINA4_MCP_REMOTE=true` AND a token matching `TINA4_MCP_TOKEN` (fallback `TINA4_API_KEY`), sent as `Authorization: Bearer`, `X-MCP-Token`, or `X-Api-Key`. Every MCP surface returns 404 to a disallowed caller, the SQL tool is read-only (SELECT/WITH, no stacked statements), and the file tools are sandboxed to the project root. **Swagger:** the production on/off switch is wired for real - set `TINA4_SWAGGER_ENABLED=false` to disable `/swagger` and `/swagger/openapi.json` in any environment, or `true` to expose them in production (it falls back to `TINA4_DEBUG` when unset). Secured routes now carry a `bearerAuth` security requirement, so the documentation no longer presents protected endpoints as public. ORM models become reusable `components.schemas` referenced by `$ref` across all four frameworks. The spec is valid where it was not: wildcard and splat routes emit proper `{name}` path parameters, `operationId` values are de-duplicated, and WebSocket routes no longer leak an invalid method. **Swagger configuration:** `TINA4_SWAGGER_SERVERS` (comma-separated) drives a multi-server block, `TINA4_SWAGGER_UI_CDN` points the UI assets at a self-hosted mirror for air-gapped use, and the generator adds typed-parameter formats, enums, top-level tags, and multipart request bodies. **SqliteDocStore (new):** a pymongo-style document store with a zero-config SQLite fallback. `get_collection(name)` returns a real Mongo collection when a Mongo URI is set (`TINA4_MONGO_URI`, then `TINA4_SESSION_MONGO_URI`, then the legacy `TINA4_SESSION_MONGO_URL`), otherwise a SQLite-backed collection over a local file (`TINA4_DOC_STORE_PATH`, default `data/tina4_docstore.db`) - the call sites are identical, only the backend differs. It pushes filters down to JSON1 `json_extract` (equality, `$in`, `$nin`, `$gt`/`$gte`/`$lt`/`$lte`, `$ne`, `$exists`, `$regex`, `$or`, `$and`, dotted nested keys), supports `$set`/`$unset`/`$inc`/replace/upsert with lazy `sort`/`limit`/`skip`/projection cursors, ships a zero-dependency 12-byte ObjectId, and round-trips datetimes and ObjectIds so values stay queryable. Develop against the local store and switch to MongoDB in production by setting one env var. **Developer experience:** the blank-`TINA4_SECRET` warning now explains why it fired - the run was not detected as development - and gives both fixes (set `TINA4_SECRET`, or set `TINA4_DEBUG=true` to auto-generate one into `.env.local`); the legacy-env strict check now hints the names may come from a `.env` baked into a Docker image and points at `tina4 env --migrate`. **Session Mongo env parity:** the session and DocStore Mongo URI is canonical as `TINA4_SESSION_MONGO_URI` across all four frameworks, with `TINA4_SESSION_MONGO_URL` kept as a back-compat legacy alias (Python and PHP historically read `_URL`). **Tests:** new DB contract tests (execute raises on a bad statement, read-after-write, generator monotonicity, transaction bracketing) and a queue isolation contract (a job in one topic never leaks into another, and a queue on a fresh storage path starts empty). **Breaking:** the Swagger and MCP production gates are now enforced - if you relied on `/swagger` being reachable in production, set `TINA4_SWAGGER_ENABLED=true`, and a remote MCP caller now needs

`TINA4_MCP_REMOTE=true` and a token. Zero new third-party dependencies. Full suite: 3,238 passing.

v3.13.39 (2026-06-21) - Auto-migration, unified critical log level, fail-loud ORM, per-route WebSocket auth

A cross-framework parity sweep that hardens the data layer and tightens a few safe-by-default behaviours. **Migrations:** pending migrations can now run on startup, gated by `TINA4_AUTO_MIGRATE` and off by default so existing apps are untouched. A footgun pass adds numeric-aware ordering (so `10_` sorts after `2_`, not after `1_`), `CREATE TABLE` idempotency, a URL-safe `//` delimiter, and smart/curly-quote normalization in migration SQL before it runs. **ORM:** `save()`, `create()`, `QueryBuilder`, and the Mongo path now fail loud instead of swallowing errors - `save()` validates first and returns `False` with the reason on `get_error()`, `create()` returns `False` when the write fails, and the silent fallbacks are gone. **Logging:** `critical` is now a first-class top-level severity, a new `Log.is_enabled(level)` lets callers skip building an expensive log payload, and logs default to stdout-only in production and container environments to avoid file bloat. **WebSockets:** a route can require authentication on the upgrade itself, before the socket is established. **Security:** the `/__dev/mcp` endpoint enforces its localhost guard and honors `TINA4_MCP_REMOTE`, and the built-in `Api` client strips the `Authorization` header on a cross-origin redirect and gains opt-in retry with backoff. **Env:** defaults align to the canonical `TINA4_` manifest with a uniform `.env.example` - CORS credentials are opt-in and AI hosts default to localhost. **Tooling:** the `tina4 metrics` coverage detection now counts full-package-path imports and short (3-character) class names, ending a class of false "untested" flags. Breaking: `critical` is now a top-level severity and the previous toggle is removed - update any logging configuration that relied on it. Full suite: 3,149 passing.

v3.13.38 (2026-06-19) - Coordinated security & robustness release

A large bundled release closing a cross-framework hardening sweep. **WebSockets:** the Redis/NATS backplane is now wired for real - local-first delivery, then a published envelope on the shared `tina4:ws` channel, relayed by sibling instances with an origin guard (no own-echo, no cluster loop) - plus an origin allow-list (`TINA4_WS_ALLOWED_ORIGINS`), an idle reaper (`TINA4_WS_IDLE_TIMEOUT`), resilient broadcast (a dead/slow client is pruned, never aborts the loop), and SSE hardening (client-disconnect + generator-error). **Sessions:** a log-loud-and-degrade backend-failure policy (set `TINA4_SESSION_STRICT=true` to re-raise). **GraphQL/WSDL:** a SOAP `<!DOCTYPE>` is rejected before parsing (SOAP 1.1 forbids DTDs - this closes the XML entity-expansion / billion-laughs and external-entity surface), a recursion-depth guard (`TINA4_GRAPHQL_MAX_DEPTH`, default 50) catches deep queries **and** circular fragments, and resolver/SOAP faults are masked in production with the real cause logged (full detail only under `TINA4_DEBUG`). **Tooling:** a new `tina4 metrics` command reports the top-N code-health offenders (complexity, large files, low maintainability, untested) with `--top/--json/--fail-on/--path`, and the coverage test-detection is now precise - a real import / defined symbol, not a name-substring scan. **Queue:** Kafka TLS/SASL on the confluent producer **and** consumer (`TINA4_KAFKA_SECURITY_PROTOCOL` / `_SSL_CA_LOCATION` / `_SASL_MECHANISM|USERNAME|PASSWORD`, with bare `KAFKA_*` as a fallback - community PR #52). Zero new third-party dependencies. Full suite: 3,080 passing.

v3.13.37 (2026-06-18) - Dev-admin editor: Ruby + more syntax highlighting

The dev-admin dashboard's code editor (CodeMirror) gained the grammars it was missing: `.rb` files - plus `.rs`/`.go`/`.java`/`.scss` - now highlight in the file viewer instead of falling through to plain text (the editor bundle had no Ruby/Rust/Go/Java case). Python's file-read endpoint already reported the correct `language` per extension, so this ships the rebuilt editor bundle that can render them. Dev-mode tooling only; no framework runtime change. Full suite: 2,952 passing.

v3.13.36 (2026-06-18) - Instant WebSocket dev-reload

Dev-reload is now a WebSocket push instead of a poll. On a file change `tina4 serve` POSTs `/__dev/api/reload`; the server re-imports just the changed route module in-process - mtime-tracked, same PID, **no respawn** - then broadcasts `{type, file, mtime}` to every browser on the `/__dev_reload` WebSocket, and the injected toolbar client reloads instantly (CSS changes hot-swap the `<link>` href without a full reload). The `/__dev/api/mtime` poll is now a fallback only, used when the socket is down. `Router.add` replaces a re-registered `(method, path)` in place so the fresh handler wins instead of being shadowed by a stale duplicate. Debug-mode only - production is untouched. Full suite: 2,952 passing.

v3.13.35 (2026-06-17) - Live MCP endpoint for AI agents

The built-in MCP server is now actually reachable. It was fully built - 50+ dev tools (live DB queries, file I/O sandboxed to the project, route list, project overview, framework docs search) - but never mounted, so no MCP client could connect. `tina4 serve` now exposes it at `/__dev/mcp` (JSON-RPC) + `/__dev/mcp/sse`, gated on debug mode, giving an AI agent (Claude Desktop/Code) live access scoped to the running project. Also fixed the `route_list` dev tool, which referenced a non-existent `Router._routes` (now `Router.get_routes()`) and errored for every caller - caught by new tool-coverage tests. Full suite: 2,943 passing.

v3.13.34 (2026-06-17) - Demo + onboarding fixes

The example store crashed on boot: `app.py` imported `orm_bind`, which was renamed to `bind_database` in 3.13 (no alias). Switched it, so the demo boots, migrates, seeds, and serves real data again. Corrected stale env-var names in the README and `example/.env` to the names the framework actually reads (`TINA4_SECRET`, `TINA4_LOG_LEVEL`, `TINA4_LOCALE`, `TINA4_SESSION_BACKEND`, `TINA4_SWAGGER_*`) - the demo had been signing JWTs with a blank secret - and unified project creation on the `tina4` CLI. Examples/docs only; framework unchanged.

v3.13.33 (2026-06-17) - Queues: priority pop + automatic dead-lettering (Warning: behavioural change)

Behavioural change. `job.fail()` now **re-enqueues** the job (incrementing `attempts`) until `attempts >= max_retries`, then moves it to the dead-letter store - so a `for job in queue.consume(topic): ... job.fail(e)` loop retries `max_retries` times and dead-letters automatically (no manual `retry_failed()`). Previously `fail()` only marked the job failed. Also: `pop`/`consume` now return the **highest-priority** available job first (ties oldest-first) instead of FIFO; new additive `Queue(..., retry_backoff=0)` delays the auto re-enqueue. Only the file/lite backend changed (brokers delegate retry/dead-lettering). The queue chapter was rewritten to match (the documented retry->dead-letter flow is now real). Full suite: 2,933 passing.

v3.13.32 (2026-06-17) - Caching: per-query bypass + X-Cache headers (chapter rewritten to match code)

Added a per-query cache bypass - `db.fetch(... , no_cache=True)` (also `fetch_one`/`fetch_all`) skips both the lookup and the store for that one call. The HTTP `ResponseCache` now stamps `X-Cache: HIT|MISS` and `X-Cache-TTL: <seconds>` on cached responses (no `Cache-Control`). The caching chapter was substantially rewritten to match the code: the real `cache_stats()` shapes, all seven backends + file-backend fallback, the three cache layers (request-scoped auto, persistent DB, response), and accurate env/defaults - removing earlier aspirational claims (a fictional stats shape, stale-while-revalidate, a `/__dev` per-key panel, auto `Cache-Control`). Full suite: 2,924 passing.

v3.13.31 (2026-06-17) - Documentation fixes (no functional change)

Corrected the developer guide: `Response.add_header` is an instance method - the class-level `Response.add_header(...)` shown previously raises `TypeError`, so it's now `response.add_header(...)` (including six middleware examples in Chapter 10). Removed a stale `fieldName` key from the `request.files` upload example (the dict has `filename`, `type`, `content`, `size`). Code is unchanged. Full suite: 2,914 passing.

v3.13.30 (2026-06-16) - Typed route params now arrive coerced (Warning: behavioural change)

Behavioural change. A typed path param now arrives **coerced to its type** instead of as a raw string: `{id:int}` / `{id:integer}` -> `int`, `{price:float}` / `{x:number}` -> `float`. Every other type (`string`, `alpha`, `alnum`, `slug`, `uuid`, `path`) and an untyped `{id}` stay strings; URL matching is unchanged (`{id:int}` still 404s on non-digits). Previously `{id:int}` matched only digits but still handed the handler the string `"42"` - code that did string operations on a typed param must adjust. This brings Python in line with Ruby (which already coerced) and the documented "auto-converted" behaviour, now matched by PHP and Node too. Also fixed a reversed `check_password` argument-order line in the dev guide. Full suite: 2,914 passing.

v3.13.29 (2026-06-16) - Live API search (`api_search`/`api_class`/`api_method`) now finds what you ask for

The live reflection index behind the `api_*` MCP tools - what AI assistants query for real method signatures instead of guessing - had three gaps, now fixed:

- **Metaprogrammed methods were invisible.** `Frond.add_filter` / `add_global` / `add_test` are defined through a custom class/instance descriptor, and the reflector's `__qualname__` owner check skipped them - they never entered the index. Reflection now walks `obj.__dict__`, unwraps staticmethod/classmethod/property/descriptor wrappers, and strips receiver params, so `api_method("Frond", "add_test")` returns `add_test(name, fn)`.
- **Class-qualified queries weren't steered.** `api_search("Frond.add_test")` returned unrelated `add_*`/`*test*` methods because only the bare name was scored. Ranking now weights the owning class, fqn segments, and an exact `Class.method` match, so the right method ranks first.
- **Lookups only matched the deep fqn.** `api_class`/`api_method` now resolve the documented public import path and a bare class name (`Database`), not just `tina4_python.database.connection.Database`.

The bundled AI skills (developer/maintainer/js) now instruct assistants to query `api_*` before guessing a signature. Full suite: 2,905 passing.

v3.13.28 (2026-06-16) - Frond: custom `add_test` now honoured by `is`

Python only. A test registered with `Frond.add_test("positive", fn)` was ignored by `{% if x is positive %}` - the `is` evaluator checked a hardcoded built-in table (`even`, `odd`, `defined`, ...) and never consulted the instance's custom-test registry, so every custom test silently returned false. It now merges the registered tests (reachable via the bound evaluator) over the built-ins, so custom registrations work and can override built-ins - matching PHP, Ruby, and Node. Built-in tests are unchanged. Surfaced by a cross-engine host-API check (`add_filter`/`add_global`/`add_test`/`form_token`) while building the verified cheatsheet. Full suite: 2,901 passing.

v3.13.27 (2026-06-16) - Frond template-engine parity fixes

A 50-case cross-engine audit (every Frond tag, filter, and test rendered through all four frameworks with identical templates) surfaced six places where Python's output diverged from the Twig/Jinja standard. All are now fixed to match:

- `{{ x | e }}` / `escape` no longer double-escapes - the filter returns a `SafeString`, so the auto-escaper leaves it alone (`<b>` -> `<b>`, not `<b>`).
- `{{ "%.2f" | format(value) }}` now resolves a *variable* argument to its value (it previously errored). Unquoted filter arguments are treated as variable references; quoted literals stay literal.
- `{%- ... -%}` **whitespace control** now actually trims - a single up-front pass applies every trim marker, including on closing tags (`endif`/`endfor`) and block-body boundaries.
- `json_encode` emits compact `[1,2,3]` (no spaces); `round` at precision 0 renders the integer `4` (not `4.0`); `nl2br` escapes its input, inserts `<br />`, and is marked safe - all matching PHP.

Behavioural note: these change rendered output for the affected filters - they are correctness fixes toward the documented Twig/Jinja behaviour. Full suite: 2,900 passing.

v3.13.26 (2026-06-16) - pooling fix: standalone writes auto-commit; explicit transactions stay atomic

Behavioural default change. A standalone write - `execute`/`insert`/`update`/`delete` made **outside** an explicit transaction - now **auto-commits on its own connection before returning**. Previously the default was autocommit *off*, which broke connection pooling: a standalone `INSERT` landed uncommitted on one pooled connection, then the next read round-robined to a different connection and saw nothing. Standalone writes are now durable and visible across the pool.

Explicit transactions are unchanged and stay atomic - inside `start_transaction()` ... `commit()`/`rollback()` the per-statement commit is suppressed (the commit branches are gated on `not self._in_transaction`), so a `rollback()` still discards everything. The psycopg2 connection still runs with `connection.autocommit = False`, so the framework owns commit boundaries and the v3.13.15 idle-in-transaction read-rollback still applies.

Set `TINA4_AUTOCOMMIT=false` in `.env` for strict manual-commit mode (every write needs an explicit `commit()`).

Verified live on PostgreSQL: standalone write visible from a separate connection, explicit rollback discards, explicit commit persists, and pooled standalone writes visible across every round-robin connection. Full suite: 2,894 passing.

v3.13.24 (2026-06-15) - unified cache backends across response, KV, and persistent DB cache

The response/KV cache now supports **seven backends**, selected by `TINA4_CACHE_BACKEND`: `memory` (default), `file`, `redis`, `valkey`, `memcached`, `mongodb`, and `database`. `TINA4_CACHE_URL` carries the connection string for `redis`/`valkey`/`memcached`/`mongodb`, or a SQL URL for the `database` backend (which falls back to `TINA4_DATABASE_URL`). Credentials can be embedded in the URL (`redis://user:pass@host`, `redis://:pass@host`, `mongodb://user:pass@host`) or supplied via `TINA4_CACHE_USERNAME` / `TINA4_CACHE_PASSWORD` (mirroring `TINA4_DATABASE_USERNAME`/`_PASSWORD`); memcached is unauthenticated. The usual `TINA4_CACHE_TTL` (60), `TINA4_CACHE_MAX_ENTRIES` (1000), and `TINA4_CACHE_DIR` (`data/cache`) still apply.

Graceful fallback: if a configured backend's driver is missing or the service/credentials are unreachable or wrong, the cache logs a warning and falls back to the **file** backend - a real persistent cache, never a silent no-op.

The **persistent DB query cache** (`TINA4_DB_CACHE=true`) now routes through the same backend set via `TINA4_DB_CACHE_BACKEND` + `TINA4_DB_CACHE_URL`, so multiple instances share one cache with global write-invalidation. `cache_stats()` now reports a `backend` field alongside `mode`.

Full suite: 2,899 passing.

v3.13.23 (2026-06-15) - request-scoped DB query cache, on by default

A new **request-scoped query cache** protects your database from rapid repeat reads. Within a single request, identical `SELECT`s and ORM reads are deduped automatically - the DB is hit once and subsequent identical reads are served from memory. The cache is **cleared at the start of every request** (so it never serves stale rows across requests) and **flushed on any write** (insert/update/delete/execute). For non-request contexts (scripts, workers) a short safety TTL applies.

It is **on by default** via `TINA4_AUTO_CACHING=true` (off-switch `TINA4_AUTO_CACHING=false`); the in-request TTL is `TINA4_AUTO_CACHING_TTL` (default 5 seconds). The existing `TINA4_DB_CACHE` (default `false`) remains the separate *persistent* cross-request cache (TTL `TINA4_DB_CACHE_TTL`, default 30s) and is not cleared per request. `cache_stats()` now reports a `mode` field: `"request"` (default), `"persistent"`, or `"off"`.

Full suite: 2,866 passing.

---

v3.13.22 (2026-06-15) - session default TTL standardised to 1 hour

The default session lifetime now matches across all four frameworks: **3600 seconds (1 hour)**. Python previously defaulted to 1800s (30 min). The session cookie `Max-Age` and the file-handler garbage-collection window both follow `TINA4_SESSION_TTL` (default now 3600) - override it in your `.env`. PHP and Node already used 3600 and are unchanged.

v3.13.21 (2026-06-15) - security: never sign JWTs with a guessable default secret

Security fix. When `TINA4_SECRET` was unset, Tina4 silently signed JWTs **and** CSRF form tokens with a hardcoded built-in default secret - so anyone who knew that default could forge valid tokens, and the developer got no warning. It affected `Auth`, the Frond `form_token()` filter, and the CSRF middleware (four copies of the same fallback).

Token signing now reads `TINA4_SECRET` and, when it is unset, **warns loudly and uses a blank secret** - matching what the PHP and Node frameworks already did. There is no longer a guessable built-in secret. **Always set `TINA4_SECRET` in production.**

Also corrected stale docs: the JWT secret env var is `TINA4_SECRET` (some docs still said `SECRET`), and `$response->template()` references are fixed to `response.render()`.

Full suite: 2,857 passing.

v3.13.19 (2026-06-15) - return domain objects, construct from JSON, and one database binder

Three ergonomic improvements surfaced by the live side-by-side review of the book's own examples across all four frameworks.

`response(...)` serializes domain objects

Return an ORM model, a list of models, or a query result straight from a route - Tina4 serializes it to JSON. No more hand-rolled `to_dict()` / `to_json()`:

```
@get("/api/users")
async def users(request, response):
    return response(User.all())        # list of models -> JSON array
```

A single model becomes a JSON object; a list of models or a `DatabaseResult` becomes a JSON array. Plain dicts, lists and strings behave exactly as before - this is purely additive.

Construct a model from a JSON object string

The model constructor now accepts a JSON object string, alongside a dict or keyword args:

```
User('{"name": "Alice", "email": "alice@example.com"}')   # parsed into one record
User({"name": "Alice"})                                    # still works
User(name="Alice")                                         # still works
```

Passing a **list/array** to a single-record constructor now raises a clear `TypeError` instead of a cryptic `'list' object has no attribute 'items'`. To build many records, map over the list.

Warning: Breaking - one database binder: `bind_database`

The ORM-to-database binder is now `bind_database` across all four frameworks (was `orm_bind` in Python). The default is unchanged - models still auto-bind to `TINA4_DATABASE_URL`, so apps that rely on the `.env` default need **no change at all**.

```
# Most apps: nothing to do - the .env default is auto-bound.

# Override the default explicitly:
bind_database(Database("sqlite:///app.db"))

# Register a NAMED connection and point a model at it:
bind_database(Database("postgres://.../analytics", "u", "p"), name="analytics")

class Visit(ORM):
    _db = "analytics"        # this model uses the analytics connection
```

A model can live on a different database from the default - `bind_database(db, name="...")` registers it and `_db = "..."` selects it. A missing named connection raises a clear error.

Migration: rename `orm_bind(...)` -> `bind_database(...)`. That is the only change; the `name=` argument, per-model `_db`, and `.env` resolution are new or unchanged.

Full suite: 2,852 passing. Shipped with parity across all four frameworks.

v3.13.16 (2026-06-15) - `create_table()` works on PostgreSQL + `DatabaseResult` index access

Found by the live documentation-verification pass - running the book's own samples against a real PostgreSQL database. The documented code-first schema path, `ORM.create_table()`, was silently broken on PostgreSQL: it emitted SQLite-only DDL, PG rejected it, the error was swallowed, and the method returned `True` while creating **no table**.

`create_table()` is now engine-aware

- `DateTimeField` **->** `TIMESTAMP` on PostgreSQL (and Firebird) - they have no `DATETIME` type (`type "datetime" does not exist`); `DATETIME` stays on SQLite/MySQL/MSSQL.
- `BooleanField` **-> native** `BOOLEAN` on PostgreSQL/MySQL, `BIT` on MSSQL, `INTEGER` on SQLite/Firebird. The engine check previously compared against `"postgres"` but `get_database_type()` returns `"postgresql"`, so bool columns silently got `INTEGER` on PG - fixed. Boolean column `DEFAULT`s are engine-aware too (`TRUE`/`FALSE` vs `1`/`0`).
- **A failed** `CREATE` **now returns** `False` **(and logs)** instead of masquerading as success.
- The PostgreSQL adapter no longer rewrites `TRUE`/`FALSE` -> `1`/`0` (`boolean_to_int`) - PG has a native boolean, and that rewrite had broken `DEFAULT FALSE` and `WHERE active = TRUE` on `BOOLEAN` columns.

`DatabaseResult` is subscriptable

`result[0]` (documented in chapter 5, "Index Access") raised `TypeError: 'DatabaseResult' object is not subscriptable`. Added `__getitem__` - index and slice access now delegate to `.records`. The bundled guide's wrong "no `len()` support" note is corrected too.

Verified against PostgreSQL 16: a model with `id` (auto-increment) + `StringField` + `BooleanField` + `DateTimeField` creates, inserts, and round-trips natively (real `bool`, `TIMESTAMP`). New PG-backed test suite (skip-if-no-PG) + always-run subscript suite. Full suite: 2,840 passing. Shipped with parity across all four frameworks.

v3.13.15 (2026-06-15) - Python only: PostgreSQL idle-in-transaction leak (#51)

Python only. psycopg2 follows the DB-API contract - a connection starts with `autocommit = False`, so even a bare `SELECT` opens a transaction. Tina4's `fetch()` / `fetch_one()` never closed it, so every read left the connection `idle in transaction` for its lifetime, pinning a pool slot and any locks it touched. PHP (`pg_query`), Ruby (`pg`), and Node (`node-postgres`) run on libpq autocommit - each statement is its own transaction - so the leak can't happen there. They stay at v3.13.14.

The slow-motion outage

The migration runner's batch lookup runs at boot:

```
row = db.fetch_one("SELECT MAX(batch) as max_batch FROM tina4_migration")
```

That read opened a transaction and left it open. Each short-lived pod/process leaked one `idle in transaction` connection holding locks on `tina4_migration`. Over enough restarts the pool filled - `FATAL: remaining connection slots are reserved for roles with the SUPERUSER attribute` - and then module autodiscovery failed mid-boot, so `/health-check` still passed (pod marked Ready) while every real route 404'd. A silent "ready but broken" state; #51 reported sessions sitting idle for 2+ days.

The fix

After a successful read **outside** an explicit transaction, the PostgreSQL adapter now rolls back the implicit transaction - a `SELECT` has nothing to persist, so a rollback is the clean close that returns the connection to plain `idle`. Inside `start_transaction()` the caller owns the transaction, so it's left alone. `execute()` (writes) is deliberately untouched: with autocommit off a write must still be committed explicitly, and auto-closing it would silently drop data.

This is distinct from the v3.13.8 fix, which healed *aborted* transactions - a clean idle read-transaction never hit that path.

Tests

- Python: 2,836 passed (+7 - `_end_read_txn` unit, `fetch`/`fetch_one` wiring, in-transaction deferral). No live PostgreSQL required: a fake connection records the rollback, and psycopg2 is stubbed when the optional driver is absent (so the guard runs in CI).

v3.13.14 (2026-06-13) - Logs reach stdout in containers + per-request logging + schema-qualified tables (#48)

Cross-framework release (all four). Deployed Docker containers were getting no application logs. The cause was the same architectural decision in every framework: in production/container mode Tina4 either **suppressed stdout** or wrote logs **only to a file inside the container** - but `docker logs` (and Kubernetes) read PID 1's stdout, so operators saw nothing. A follow-on report - "logs stop after `Development server: asyncio`" - surfaced a second gap: the dev server logged its startup banner but **never logged requests**, so it looked dead under traffic.

This is a 12-factor correction: a container's stdout *is* the log sink.

Per-request logging - on by default in dev

Every request now logs one line through the Tina4 `Log` (-> stdout), on by default in development and opt-in for production via `TINA4_LOG_REQUESTS`:

```
2026-06-12T10:15:03.221Z [INFO   ] GET /api/users -> 200 (12.3ms)
```

- Format is identical across all four frameworks: `METHOD /path -> STATUS (Nms)`.
- **Default**: on when `TINA4_DEBUG` is truthy (dev), off in production - so prod doesn't pay the per-request cost unless you opt in.
- `TINA4_LOG_REQUESTS=true` forces it on (production debugging); `=false` forces it off.
- Routed through `Log`, so it's coloured human-readable in dev and structured JSON in production, like every other line.

Two bugs fixed here: Python's `RequestLoggerMiddleware` emitted via an **unconfigured stdlib `logging` logger** (silently dropped - never reached stdout), and the dev server only fed request data to the `/__dev` dashboard inspector, never to a log. Both now go through `Log`.

What changed (stdout)

- **stdout is no longer suppressed in production.** Logs are written to stdout regardless of `TINA4_ENV`/`TINA4_DEBUG`. Production emits clean JSON (no ANSI colour) so aggregators can parse it; dev keeps the human-readable coloured format. `TINA4_LOG_OUTPUT=file` still opts out of stdout entirely.
- **stdout is flushed per line.** `print(..., flush=True)` - logs appear immediately on a non-TTY pipe (every container) instead of sitting in Python's block buffer until the process exits (or vanishing on an abrupt stop).
- **Default log level is now `INFO`** (was `ERROR`). An app that logged at info/debug previously looked silent in production. INFO surfaces request/startup/warning/error without debug noise. Override with `TINA4_LOG_LEVEL`.

```
# In a container (TINA4_ENV=production), with the default config:
Log.info("worker started")
# pre-v3.13.14: written only to logs/tina4.log inside the container -> docker logs empty
# v3.13.14:    {"timestamp":"...","level":"INFO","message":"worker started"}  -> on stdout
```

Why it spanned all four

The bug was the *same* decision in~~ each framework, so the fix is~~ too:

Framework	Pre-v3.13.14 cause	Fix
Python	`not _is_production` gate suppressed stdout; default ERROR	stdout always on (flushed); default INFO
PHP	`$stdout = $development` (file-only in prod); no `TINA4_LOG_LEVEL` read	stdout default on + `fflush`; reads `TINA4_LOG_LEVEL`; default INFO
Ruby	stdout written but **never flushed** (block-buffered on non-TTY); default ALL	`$stdout.sync = true`; default INFO; accepts plain + bracket level names
Node	`!isProduction()` gate suppressed console; default DEBUG	console always on; production emits JSON; default INFO

The Rust `tina4` CLI was already correct - it inherits child stdio, so child logs flow to the container.

Request logging parity

Framework	Pre-v3.13.14	Fix
Python	no request log; `RequestLoggerMiddleware` used a dead stdlib logger	log in dispatch via `Log`; middleware routed through `Log`
PHP	`RequestLogger` not default-on, line lacked status	log in `Router::dispatch` via `Log`; status added to the line
Ruby	dev inspector only; `RequestLoggerMiddleware` not wired; `[RequestLogger]` prefix	log in `rack_app` via `Log`; prefix dropped for parity
Node	`requestLogger` always-on via bare `console.log`, status-first format	gated (dev-default + env), routed through `Log`, standard format

Schema-qualified tables (#48) + a PostgreSQL `fetch()` regression

Issue #48 - *"Database Table Does Not Exist"* on PostgreSQL. A model whose table lives in a non-default schema (`gift_cards.gift_card`, MSSQL `dbo.widget`, MySQL `otherdb.table`, SQLite ATTACH `extra.widget`) was invisible to the framework's introspection. `table_exists`, `get_tables`, and `get_columns` hardcoded the default namespace (`public`) and matched the whole dotted string as one flat name - so plain reads worked, but `create_table`, migrations, and auto-CRUD were blind to the table and reported it missing.

All introspection is now schema-aware on every affected engine:

- **PostgreSQL** - `table_exists` uses `to_regclass()` (honours schema + `search_path`); `get_columns` filters by `table_schema`; `get_tables` lists every non-system schema and returns non-`public` tables schema-qualified.
- **MySQL** - schema = database; a qualified name checks that catalog, a bare name defaults to `DATABASE()`.
- **MSSQL** - honours `dbo.table`; a bare name matches in any schema.
- **SQLite** - honours an ATTACH alias (`extra.widget`) for both `table_exists` and `get_columns`.
- **Firebird** - N/A (no schemas).

Verified against a live PostgreSQL 16 container: `table_exists('gift_cards.gift_card') -> True`, `get_tables -> ['gift_cards.gift_card', 'gift_cards.transaction']`, `get_columns -> 12 columns` - identical results across all four frameworks.

> ***PHP also fixed a v3.13.12 regression found while cross-checking #48.*** `PostgresAdapter` *referenced* `stripTrailingSemicolons()` *(added in v3.13.12) and the new* `splitSchema()` *but never mixed in* `SqlNormalizerTrait` *- so* ***every PostgreSQL*** `fetch()` **/** `fetchOne()` **/** `getColumns()` ***fatalled*** *with "Call to undefined method". It shipped silently because the PHP PostgreSQL test suite skips without a live server. Fixed with a one-line trait mix-in and pinned by server-free reflection guards that assert all five SQL adapters expose the normalizer helpers.*

Tests

- Python: 2,829 passed (+18 new - stdout-in-prod, JSON shape, level filter, file opt-out; request-log gate + middleware; #48 schema split + SQLite ATTACH introspection)
- PHP: 2,394 passed (+63 new - stdout/level/file gating; request-log format + gate; #119 cli-server crash fix + LegacyEnvGuard suite now CI-gated; #48 schema-qualified + PG trait regression guards)
- Ruby: 2,999 passed (+23 new - level resolution + `$stdout.sync`; request-log gate + dispatch; #48 schema split + SQLite ATTACH introspection)
- Node: 3,628 passed (+16 net - production JSON stdout; request-log gating + format; #48 schema split + SQLite ATTACH introspection)

11,850 tests across the family, zero regressions.

> *PHP also fixed #119 in this release -* `App::checkLegacyEnvVars()` *crashed with* `Undefined constant Tina4\STDERR` *under the built-in* `cli-server` *(bare* `STDERR` *is only auto-defined for the* `cli` *SAPI). Now uses the* `php://stderr` *stream. PHP-specific; the other three don't reference that constant.*

---

v3.13.12 (2026-06-11) - SQL safety + implicit ORM binding + `fetch_all` correctness

Three high-impact fixes that close out long-standing footguns. All three ship with full parity across all four frameworks.

`fetch_all` actually fetches ALL rows now (no silent 100-row truncation)

Pre-v3.13.12 the convenience method defaulted to `limit=100` and silently truncated. The name says `fetch_all` - it should fetch them all:

```
# 150 rows in the table
db.fetch_all("SELECT * FROM rows")
# pre-v3.13.12: returns 100 rows, silently drops the other 50
# v3.13.12:    returns all 150 rows
```

The new default is `limit=0`, which the adapter interprets as "no pagination injection" - your SQL runs verbatim. To opt back into a cap, pass `limit=N` explicitly:

```
db.fetch_all("SELECT * FROM events", limit=500)   # capped
db.fetch_all("SELECT * FROM users")               # all rows
```

`db.fetch()` (the paginated sibling that returns a `DatabaseResult` with count metadata) keeps its 100-row default - pagination is its job. Only the `fetch_all` convenience changed.

Breaking change: callers who relied on the silent 100-row cap now get every row. For very large tables, switch to `fetch()` (which paginates with metadata) or pass an explicit `limit`. Per the v3 parity rule, breaking changes are OK when correctness is the win.

Trailing `;` is now stripped from user SQL in `fetch()` / `fetch_one()`

The framework appends `LIMIT n OFFSET m` to the user-supplied query (and wraps it in `SELECT COUNT(*) FROM (...) AS subq` for the count probe). When the user's query already ended with a `;`, both rewrites broke:

```
db.fetch("SELECT * FROM users;")
# pre-v3.13.12: syntax error near "LIMIT" - the appended LIMIT followed a ;
# v3.13.12:    works - trailing ; is stripped before LIMIT is appended
```

The strip is conservative: only trailing whitespace + semicolons are removed (any number of them, including `;;`), nothing inside the statement is touched. Parameters and quoting are unchanged - the existing parameter-binding defense against injection still does all the heavy lifting.

Applied at the top of `fetch()` and `fetch_one()` on all five adapters: PostgreSQL, MySQL, SQLite, MSSQL, Firebird.

Ruby ORM now auto-discovers `TINA4_DATABASE_URL` like the other three

When `TINA4_DATABASE_URL` was set in `.env` but `Tina4.bind!` had never been called, Ruby ORM operations returned `nil` from the model's `db` accessor - every save / find / where silently no-op'd. Python, PHP, and Node all already discovered the env var on first use; Ruby had the helper (`auto_discover_db`) defined but never called.

```
# .env has TINA4_DATABASE_URL=sqlite://./app.db, no explicit Tina4.bind! anywhere
```

```
User.find(1)
# pre-v3.13.12: nil  (db accessor returned nil, query never ran)
# v3.13.12:     <User id: 1, ...>  (auto-discovered on first model access)
```

Explicit `Tina4.bind!(db)` still takes precedence - use it to bind a second database or override the env-driven default. The behaviour now matches Python's `database_url_auto_discover()`, PHP's adapter auto-init, and Node's `initDatabase()` env fallback.

Cross-framework parity

Fix	Python	PHP	Ruby	Node
`fetch_all` returns ALL rows by default	[x] `limit=0` default	[x] `$limit = 0` default	[x] `limit: nil` default	[x] already correct (`limit?` undefined)
Strip trailing `;` from fetch SQL	[x] shared helper on `DatabaseAdapter`	[x] `SqlNormalizerTrait` on 5 adapters	[x] `Tina4::Database.strip_trailing_semicolons`	[x] exported `stripTrailingSemicolons`
Implicit ORM binding from env	[x] already worked	[x] already worked	[x] **fixed** (wired `auto_discover_db`)	[x] already worked

Tests

- Python: 2,811 passed (+24 new)
- PHP: 2,316 passed (+13 new)
- Ruby: 2,980 passed (+18 new)
- Node: 3,612 passed across 95 files (+16 new)

11,719 tests across the family, +71 new for v3.13.12, zero regressions.

---

v3.13.11 (2026-06-11) - ORM correctness pass

Five fixes bundled into one release. Two are Andre's own ORM bugs (#50), two close out follow-on gaps from Michael's error-visibility report (#49), and one fixes a PostgreSQL-level mismatch that BooleanField columns were creating.

#50.1 - Callable field defaults are now resolved per-instance

```
class GiftCard(ORM):
    created_at = DateTimeField(default=lambda: datetime.now())
```

Pre-v3.13.11 this stored the lambda object verbatim and crashed on save:

```
psycopg2.ProgrammingError: can't adapt type 'function'
```

Now the framework invokes the callable **per instance** at construction time, so every row gets a fresh value. Types are excluded (`default=int` is preserved verbatim - that's almost never intended as "call `int()`").

#50.2 - `save()` correctly INSERTs natural (non-auto-increment) PKs

For models with a user-supplied PK (e.g. `gift_card_number = "GC-100"` set before the first save), pre-v3.13.11 `save()` always chose UPDATE - matched zero rows - and silently returned success without inserting anything. The framework now checks `cls.exists(pk_value)` for non-auto-increment PKs:

```
gc = GiftCard()
gc.gift_card_number = "GC-100"
gc.save()                          # -> INSERT (pre-v3.13.11 was a silent UPDATE no-op)
GiftCard.find_by_id("GC-100")      # -> returns the row
```

Auto-increment behaviour is unchanged: `PK is None -> INSERT`, `PK set -> UPDATE`. Saving an existing natural-key row still UPDATEs (and doesn't duplicate).

#49.1 - Original cause logged when failure is inside an explicit transaction

When a query fails inside `db.start_transaction()`, the auto-rollback correctly defers to the user. But the visibility half no longer goes with it - the framework now emits a Log.warning marker so operators can spot the upstream cause that's about to be buried by the cascade. The `fetch()` COUNT probe also now logs original-cause failures via `Log.warning` before swallowing.

#49.2 - `Database.fetch()` populates `last_error` (mirror of `execute()`)

```
try:
    db.fetch("SELECT * FROM does_not_exist")
except Exception:
    pass

db.get_error()   # pre-v3.13.11: None  (adapter had the cause, wrapper never read it)
                 # v3.13.11:     "relation \"does_not_exist\" does not exist"
```

BooleanField - engine-aware DDL on PG / MySQL / MSSQL

Pre-v3.13.11 `BooleanField` mapped to `INTEGER` on every engine. That caused PostgreSQL to throw `operator does not exist: boolean = integer` when Python `bool` values bound via psycopg2 met the `INTEGER` column - because psycopg2 adapts `True`/`False` to PG `boolean`, not to integer.

v3.13.11 makes `BooleanField -> create_table()` engine-aware:

Engine	DDL type
PostgreSQL	`BOOLEAN`
MySQL	`BOOLEAN` (alias for `TINYINT(1)`)
MSSQL	`BIT`
SQLite	`INTEGER` (no native bool)

Firebird	`INTEGER` (driver round-trip uneven for native BOOLEAN)

Breaking change: callers writing literal `= 0` / `= 1` against tables created by `create_table()` on PG / MySQL / MSSQL will need to update to `= false` / `= true` (or the engine's native bool literal). Tables created via migration with explicit DDL aren't affected - the framework only sets the type when it creates the table itself.

Cross-framework parity

Fix	Python	PHP	Ruby	Node
#50.1 callable defaults	[x] fixed	N/A (PHP property defaults are constants)	[x] fixed (Procs)	N/A (no auto-default at construction)
#50.2 natural-key INSERT	[x] fixed	[x] already correct (`recordExists`)	[x] already correct (`@persisted` flag)	[x] fixed
#49.1 + #49.2 PG visibility	[x] fixed (Python-only - libpq autocommit means cascade never happens on PHP/Ruby/Node)			
BooleanField DDL	[x] fixed	N/A (PHP createTable is migration-driven)	[x] fixed	[x] already engine-aware

Tests

- Python: 2,807 passed, 31 skipped (+34 new)
- PHP: 2,888 passed (no changes)
- Ruby: 2,962 passed, 7 pending (+10 new)
- Node: 3,596 passed across 94 files (+10 new)

12,253 tests across the family, +54 new for v3.13.11, zero regressions.

---

v3.13.10 (2026-06-11) - Python only

Three Python-only housekeeping items.

1. Google Antigravity removed from `AI_TOOLS` - it reads `AGENTS.md`, not `.antigravity/context.md`

Per Google's official Antigravity docs (rules-workflows), as of Antigravity **v1.20.3 (March 2026)** the IDE reads `AGENTS.md` from the repo root - the same cross-tool standard Codex pioneered, Cursor adopted, and Claude Code reads as a fallback. Our pre-v3.13.10 installer wrote to `.antigravity/context.md`, **a path nothing actually reads.** Antigravity has been silently producing dead files in users' projects since the entry was added in commit `04fba18`.

The fix in v3.13.10 is simply to **remove the antigravity entry from** `AI_TOOLS`. The existing `codex` entry (`AGENTS.md`) already writes the Tina4 skill block to the file Antigravity actually consumes - one file, four tools (Codex + Cursor + Claude Code + Antigravity all read it).

If you want Antigravity-specific tuning beyond the shared `AGENTS.md`, write it to `.agents/rules/tina4.md` by hand - that's the documented per-workspace rules folder.

2. `uv.lock` drift caught

The lockfile had quietly fallen out of sync - it tracked `tina4-python` at `3.13.8` while `pyproject.toml` was at `3.13.9`. The mistake was on my side: I didn't re-stage `uv.lock` on the v3.13.7 / v3.13.8 / v3.13.9 commits, and `uv` only regenerates it lazily. **Cosmetic only** (the PyPI artefact was always built from `pyproject.toml`, so the published packages were correct), but the lockfile would have drifted further every release. Now refreshed to `3.13.10` and committed.

3. `.gitignore` for runtime broken-route artefacts

`Tina4::BrokenTracker` writes import-time and route-time failure dumps to `data/.broken/` and `data/broken/` at the project root. The pre-v3.13.10 `.gitignore` only covered `/broken` (older convention) and the `example/store/` paths. The newer `data/`-rooted paths weren't ignored, so test runs that intentionally threw (the `tina4.request.error` tests in v3.13.7) were leaving untracked `.broken` files lying around in the framework's own repo.

Now ignored:

```
/data/broken/
/data/.broken/
```

Why Python-only

All three items are Python-specific. PHP/Ruby/Node never had the Antigravity entry, don't use uv, and have their own gitignore conventions covered separately.

Tests

- `tests/test_ai.py::TestAITools::test_antigravity_is_handled_via_codex_entry` - new test that asserts (a) Antigravity is NOT a separate `AI_TOOLS` entry, (b) the Codex entry's `context_file` remains `AGENTS.md`. Keeps the design intent visible so nobody reintroduces a dedicated Antigravity entry without checking the docs.
- Existing `test_tools_count_matches_known_set` updated from 8 -> 7.

2,773 passed, 47 skipped - no regressions.

---

v3.13.9 (2026-06-10)

Non-destructive AI installer across all four frameworks.

The bug

Pre-v3.13.9 the installer wrote a full developer guide to `CLAUDE.md` (and the equivalent context files for Cursor / Copilot / Windsurf / Aider / Cline / Codex / Antigravity) on every run, clobbering whatever the user had put there. If a user kept project-specific notes in `CLAUDE.md` - branch naming, deploy URLs, "don't touch this", reminders about a flaky service - re-running `install_context()` wiped all of that.

The fix

The installer now uses a **marker-bracketed skill block**:

```
<!-- tina4-skills:start -->
## Tina4 Skills

When working on this Tina4 project, these skills give the assistant project-aware behaviour:

- **tina4-developer** - Read `.claude/skills/tina4-developer/SKILL.md` before building features.
- **tina4-js** - Read `.claude/skills/tina4-js/SKILL.md` for frontend work.
- **tina4-maintainer** - Read `.claude/skills/tina4-maintainer/SKILL.md` for framework-level cha

See https://tina4.com for full docs.
<!-- tina4-skills:end -->
```

Four behaviours:

- **Fresh install** - file doesn't exist -> write the framework guide plus the skill block.
- **Marker refresh** - file exists with our markers -> replace just the bracketed block. **Idempotent**: re-running the installer keeps the skill references current as new skills are added.
- **One-time migration** - file starts with the pre-v3.13.9 framework header (`# Tina4 Python - Developer Guidelines`, `# Tina4 PHP`, `# Tina4 Ruby`, `# CLAUDE.md - AI Developer Guide for tina4-nodejs`) -> replace the old dump with the new framework guide + skill block.
- **Preserve user content** - file exists with the user's own content (no markers, no old header) -> append the skill block to the end. Everything else is preserved verbatim.

Markdown files (`CLAUDE.md`, `.github/copilot-instructions.md`, `CONVENTIONS.md`, `AGENTS.md`, `.antigravity/context.md`) get HTML-comment markers (`<!-- ... -->`). Rule files (`.cursorules`, `.windsurfrules`, `.clinerules`) get `#`-prefixed markers so rule loaders treat them as comments.

The actual skill content lives in `.claude/skills/tina4-*/SKILL.md` - those are framework-owned and still get cleanly overwritten so re-runs upgrade the skill content. `CLAUDE.md` itself becomes a thin pointer, not a re-rendered guide.

Cross-framework parity

Same algorithm, same marker syntax, same four branches in Python, PHP, Ruby, and Node. Same canonical action verbs in the log output (`Installed` / `Refreshed skill block in` / `Migrated (replaced old framework dump in)` / `Appended skill block to`).

Tests

99 new tests across the family covering all four branches plus marker detection, block replacement, idempotency, old-header detection, encoding edge cases, and rule-file vs markdown-file behaviour.

- Python: 2,772 passed, 47 skipped (24 new)
- PHP: 2,888 passed (11 new - verified via reflection so private helpers stay private)
- Ruby: 2,952 passed, 7 pending (18 new)
- Node: 3,586 passed across 93 files (46 assertions new)

What you'll see when you re-install

Existing users running the installer for the first time after upgrading will hit branch 3 - they'll see this in the output:

```
[OK] Migrated (replaced old framework dump in) CLAUDE.md
```

On any subsequent run, branch 2 kicks in:

```
[OK] Refreshed skill block in CLAUDE.md
```

Users who curated their own `CLAUDE.md` and never ran the old installer will see branch 4:

```
[OK] Appended skill block to CLAUDE.md
```

---

v3.13.8 (2026-06-10) - Python only

Follow-on for issue #46. Schalk on the 24rent team upgraded to v3.13.7 and still hit the cascade message on the FIRST query of a function - meaning the PostgreSQL connection had been poisoned **before** the wrapper saw any failure.

The gap in v3.13.6 / v3.13.7

v3.13.6 added auto-rollback **inside** `_on_query_error` - that clears the abort *on* a failure the framework's wrapper catches. But the connection can still arrive poisoned from sources the wrapper never observed:

- A boot-time query that failed before request handling started (migration probe, ORM `information_schema` lookup, etc.)
- A failure inside an explicit transaction where the user owned the rollback but never issued one
- A direct `cursor.execute` call in another adapter method (the `SELECT lastval()` SAVEPOINT probe in `execute`) that managed to leave the txn dirty

In all three cases, the *next* `db.fetch` / `db.execute` - even one routed through the wrapper - hits `InFailedSqlTransaction` immediately, and the cascade message buries whatever the original cause was. Exactly what Schalk reported:

```
[ERROR] PostgreSQL query failed: InFailedSqlTransaction: current transaction is aborted, command
{"sql": "SELECT * FROM gift_cards.gift_card WHERE created_by_email = %s AND is_deleted = 0 LIMIT
```

The fix: pre-flight heal

`_exec_with_handling` and `fetch` now call `_heal_aborted_txn()` before executing. That checks the psycopg2 connection's `transaction_status` against `TRANSACTION_STATUS_INERROR` and rolls back if poisoned:

```
def _heal_aborted_txn(self):
    if self._in_transaction or self._conn is None:
        return
    import psycopg2.extensions as _ext
    if self._conn.info.transaction_status != _ext.TRANSACTION_STATUS_INERROR:
        return
    self._conn.rollback()
    Log.warning(
        "PostgreSQL connection arrived in aborted-transaction state - "
        "issued pre-flight ROLLBACK so the next query starts clean. "
        "Look back in the log for the original PostgreSQL query failed entry."
    )
```

- **Only fires outside an explicit transaction** - same rule as the failure-time auto-rollback. Callers running SAVEPOINT/retry stay in charge.
- **Logs a warning when it triggers** so the operator can correlate against the upstream failure.
- **Defensive**: even if a code path bypasses the wrapper entirely, the next path that does *use* the wrapper heals the connection on the way in.

Cross-framework

Python only. PHP `pg_query`, Ruby `pg`, and Node `node-postgres` use libpq autocommit by default - each statement is its own transaction, so the cascade never happens there.

Tests

3 new tests in `tests/test_postgres_error_visibility.py::TestPoisonedConnectionIsHealed`:

- `test_fetch_heals_poisoned_connection` - manually poison the connection with a raw `cursor.execute`, then verify `db.fetch` succeeds
- `test_execute_heals_poisoned_connection` - same for `db.execute`
- `test_explicit_transaction_skips_heal` - confirms the heal defers when the user owns the txn

2,764 passing, 31 skipped (skipped tests require PG container; verified against `postgres:16-alpine` on `localhost:55432`).

---

v3.13.7 (2026-06-10)

Two changes from an external app-platform team (24rent, PLATFORM-2159) - one observability hook, one production-safety fix. Both ship across **all four frameworks** with identical event payload shape.

NEW: `tina4.request.error` event

When the router catches a thrown exception, it now emits `tina4.request.error` **before** rendering the 500 page. Listeners receive `{exception, request}` and can ship the failure to CloudWatch / Sentry / Slack - even though the framework caught the throwable.

```
from tina4_python.core.events import on
from tina4_python.debug import Log

@on("tina4.request.error")
def report_to_observability(payload):
    exc = payload["exception"]
    req = payload["request"]
    Log.error(f"Route error: {type(exc).__name__}: {exc}", path=req.path)
    # ...or POST to your centralised logging pipeline
```

- **Fires for caught route exceptions** (and warnings escalated to exceptions). Does NOT fire for 404s - those aren't server errors.
- **Listener errors are swallowed + logged via** `Log.warning` so a broken listener can never break the 500 render.
- **Listeners fire in priority order** (higher priority first, matching the existing `@on(event, priority=N)` contract).
- **Same payload shape in every framework** - only the calling syntax differs:

```
// PHP
Events::on('tina4.request.error', function ($payload) {
    Log::error('Route error: ' . $payload['exception']->getMessage());
});
```

```
# Ruby
Tina4::Events.on("tina4.request.error") do |payload|
  Tina4::Log.error("Route error: #{payload[:exception].message}")
end
```

```
// Node.js
import { Events, Log } from "@tina4/core";
Events.on("tina4.request.error", (payload) => {
  Log.error(`Route error: ${(payload as any).exception.message}`);
});
```

FIX: Stack trace removed from production 500 body (CWE-209)

Before v3.13.7, an unhandled route exception would render the **full stack trace** into the HTTP 500 response body - file paths, function chain, the exception message - **regardless of** `TINA4_DEBUG`. That's CWE-209 / OWASP A05: information disclosure.

The framework's own `500.twig` now guards the trace block with `{% if error_message %}`. When `TINA4_DEBUG=false`, callers pass an empty `error_message` and the trace block doesn't render. The trace stays in ~~`Log.error` (server-side) and~~ reaches observability via the new

event.

When `TINA4_DEBUG=true`, the rich `ErrorOverlay` page is unchanged.

Tests

Each framework added 6-14 regression tests covering: event payload shape, dev/prod symmetry, listener priority ordering, listener-error safety, and CWE-209 (no trace markers in the prod body).

- Python: 2,748 passed, 44 skipped
- PHP: 2,877 passed
- Ruby: 2,934 passed, 7 pending (PG container)
- Node: 3,540 passed across 92 files

Background

Reported by DevProx on the 24rent platform - they centralise observability by scraping structured JSON lines from stderr -> CloudWatch -> a Slack notifier. Route-level exceptions weren't surfacing because the framework caught them silently. The event hook fixes that without forcing any team's logging convention; the trace-leak fix is independently a security concern.

---

v3.13.6 (2026-06-09)

Two small reliability fixes - both tracked across all four frameworks.

PostgreSQL transaction errors no longer cascade (#46)

A failed PostgreSQL query outside an explicit transaction used to leave the connection in an aborted state. Every subsequent query then failed with `current transaction is aborted, commands ignored until end of transaction block`, masking the original cause and making the bug effectively invisible to operators.

The fix wraps every PostgreSQL `cursor.execute` in error-aware machinery:

```
# Bad query
try:
    db.execute("SELECT * FROM table_that_does_not_exist")
except Exception:
    pass

# Before v3.13.6: this raised InFailedSqlTransaction
# v3.13.6 onward: succeeds - the framework auto-rolled back
result = db.fetch("SELECT 1 AS one")
assert result.records[0]["one"] == 1

# The original error is still visible:
db.last_error  # -> 'relation "table_that_does_not_exist" does not exist'
```

The framework now:

- Logs the original failure via `Log.error` with the SQL and params.
- Stores the message on `db.last_error` so observability tools can read it.

- Auto-rollbacks **only** when the caller is not inside an explicit transaction - explicit transactions are left to the user (so SAVEPOINT / retry patterns still work).

Cross-framework note: this cascade behaviour is psycopg2-specific (DB-API 2.0 mandates an implicit transaction on first statement). PHP `pg_query`, Ruby `pg` gem, and Node `node-postgres` all run in libpq autocommit by default - no cascade, no fix needed.

Better driver install hints (#47)

Missing-driver `ImportError` messages now suggest a `uv add` command alongside `pip install`:

```
psycopg2 is required for PostgreSQL connections. Install one of:
    uv add tina4-python[postgres]   # extra for projects using uv
    pip install psycopg2-binary    # bare driver
    uv add tina4-python[all-db]    # all five database drivers
```

Applies to the PostgreSQL, MySQL, MSSQL, Firebird, ODBC, and MongoDB drivers, plus the MongoDB queue backend.

Tests

2,741 passing, 44 skipped (Postgres / MySQL / MongoDB containers).

---

v3.13.5 (2026-06-05)

Frond static-facade parity across PHP, Ruby, Node.js. Closes the last documented v3 parity gap (tina4-python task #32). Python's `Frond.add_filter` / `add_global` / `add_test` have worked as classmethods since v3.13.0 - now PHP / Ruby / Node match.

What changes

Filters, globals, and tests registered at app-startup persist across `new Frond()` instances. Every framework now supports the same pattern:

```
// PHP
\Tina4\Frond::addFilter("money", fn($v) => number_format((float)$v, 2));
\Tina4\Frond::addGlobal("APP_NAME", "My App");
\Tina4\Frond::addTest("positive", fn($v) => $v > 0);
```

```
# Ruby
Tina4::Frond.add_filter("money") { |v| "%.2f" % v.to_f }
Tina4::Frond.add_global("APP_NAME", "My App")
Tina4::Frond.add_test("positive") { |v| v > 0 }
```

```
// Node.js
Frond.addFilter("money", (v) => Number(v).toFixed(2));
Frond.addGlobal("APP_NAME", "My App");
Frond.addTest("positive", (v) => Number(v) > 0);
```

```
# Python - already shipped in v3.13.0
Frond.add_filter("money", lambda v: f"{float(v):.2f}")
Frond.add_global("APP_NAME", "My App")
Frond.add_test("positive", lambda v: v > 0)
```

In every framework, registering at the class level updates a static registry. The next `new Frond()` drains that registry into its own filter/global/test maps automatically. No need to thread a single `Frond` instance through the application - register at startup, render everywhere.

Instance form still works

Existing per-instance registration continues to work, and now propagates to the class registry too - so the lifecycle is symmetric:

```
$frond = new \Tina4\Frond();
$frond->addFilter("currency", $fn);
// Future `new Frond()` instances also see "currency"
```

`clearRegistry()` for test fixtures

Every framework exposes a class-level method to wipe user-registered filters/globals/tests without touching the built-ins (upper, lower, length, defined, even, ...). Useful in test setup/teardown to prevent state leaks between specs.

```
\Tina4\Frond::clearRegistry();
```

```
Tina4::Frond.clear_registry
```

```
Frond.clearRegistry();
```

```
Frond.clear_registry()
```

Implementation notes per framework

Framework	Mechanism
Python	`_ClassOrInstanceMethod` descriptor - one method, dual-callable via `__get__`
PHP	`__call` + `__callStatic` magic-method pair - PHP can't have same-name static and instance methods
Ruby	Same-name class method and instance method - Ruby naturally allows this
Node.js	TypeScript class supports same-name `static foo()` and `foo()` instance methods - distinct lookup spaces

Test count

Framework	Before	After	New
Python	2,741	2,741	0 (already covered)
PHP	2,858	2,871	+13
Ruby	2,907	2,928	+21
Node.js	3,508	3,526	+18
Total	**12,014**	**12,066**	**+52**

Upgrade

Drop-in patch. No breaking changes. Existing instance-form code (`$frond->addFilter(...)` / `frond.add_filter` / `frond.addFilter`) keeps working unchanged. The new static form is purely additive.

v3.13.4 (2026-06-04)

Three middleware/header bug fixes across all four frameworks, plus Python chapter 10 + 18 docs rewrites. Reported in tina4-book#140 and tina4-book#141 by MichaelC8E.

PY-10-02 - `@middleware()` no longer silently disables auth (SECURITY)

Before: Applying `@middleware(...)` to a POST/PUT/PATCH/DELETE route silently flipped `auth_required = false`, removing the framework's built-in Bearer-token gate. A developer adding custom logging or rate-limiting middleware to an admin endpoint would, with no warning, open it to unauthenticated callers.

After: Middleware is purely additive. Write routes stay Bearer-token-gated by default. Use `@noauth()` to open a write route, `@secured()` to lock a read route. Same rule across all four frameworks.

This is a **behaviour change** - if your code relied on the old auto-disable to handle auth in custom middleware, add `@noauth()` (and have your middleware enforce auth on its own).

PY-10-03 - `request.headers` is now case-insensitive

Before: `request.headers["Content-Type"]` returned `None`/`undefined`/`nil`. The dict was lowercase-only; mixed-case lookups silently failed. Six chapter 10 examples (`Content-Type`, `X-API-Key`, `Authorization`, `User-Agent`) were broken.

After: HTTP headers are case-insensitive per RFC 7230 Section 3.2. Same is true in every framework:

Framework	Implementation
Python	`CaseInsensitiveDict` (dict subclass, normalises string keys to lowercase on read/write)

PHP	`Tina4\CaseInsensitiveArray` (ArrayAccess + IteratorAggregate + Countable)
Ruby	`Tina4::CaseInsensitiveHash < Hash` (overrides `[]`, `[]=`, `key?`, `delete`, etc.)
Node	Proxy wrapper around `http.IncomingHttpHeaders`

`request.headers.get("Content-Type")`, `request.headers.get("content-type")`, and `request.headers.get("CONTENT-TYPE")` all return the same value. Existing lowercase code keeps working unchanged.

PY-10-01 - Function-based middleware now runs

Before: Chapter 10 taught Express-style `async def mw(req, resp, next_handler)` in 8+ examples, but the Python framework's dispatcher only looked for class-based `before_*`/`after_*` methods. Function-style middleware was silently inert - body never executed. PHP and Ruby had similar gaps (closures ran but no `next` continuation).

After: Express-style continuation chain is implemented across the family. Python adds `_is_function_middleware()` + `_invoke_handler_with_middleware()`. PHP wraps closures with `array_reverse` continuation. Ruby uses lambdas + `reverse_each`. Node already had `next()` continuation - added a regression test to keep it green.

```
@middleware(my_mw)
@post("/api/orders")
async def create_order(req, resp):
    ...

async def my_mw(req, resp, next_handler):
    if not authorised(req):
        return resp.json({"error": "forbidden"}, 403)
    result = await next_handler(req, resp)   # continue the chain
    return result
```

First-declared middleware is the outermost layer; calling `next_handler` descends to the next layer (or the route handler if last). Omitting the `next_handler` call short-circuits the chain.

Python chapter rewrites - book + docs

- **Chapter 18 (Testing)** - Fixed PY-18-04 (test runner output now shows real pytest output, not the fictional `[PASS] test_addition` format), PY-18-07a (added missing `from src.orm.Product import Product` import), PY-18-08 (`resp.status_code` -> `resp.status` across 14+ call sites, positional body `self.post(path, dict)` -> keyword `self.post(path, json=dict)`).
- **Chapter 10 (Middleware)** - Added two callouts: headers are case-insensitive in v3.13.4+; `@middleware()` is purely additive (does not change auth_required). Existing mixed-case header examples now work against v3.13.4.

Test count

Framework	Before	After	New
Python	2,725	2,741	+16
PHP	2,844	2,858	+14
Ruby	2,887	2,906	+19
Node.js	3,477	3,508	+31
Total	**11,933**	**12,013**	**+80**

Upgrade

PY-10-02 is a behaviour change with a security implication. Audit routes that use `@middleware()` on POST/PUT/PATCH/DELETE: if you rely on custom middleware to handle auth, add `@noauth()` above `@middleware()` (and make sure your middleware enforces auth). Otherwise, no action - your write routes were always supposed to require Bearer tokens.

PY-10-01 and PY-10-03 are purely additive - no breaking changes.

v3.13.3 (2026-06-03)

Two reporter-driven ergonomic additions, shipped across all four frameworks with full parity per `feedback_parity`.

`Env` typed env-var helpers (tina4-python#43)

Reading env vars by hand gets old fast: every boolean flag becomes a `os.getenv("TINA4_DEBUG", "false").lower() in ("1", "true", "on", "yes")` incantation. Every numeric tuning knob needs a try/except around `int()`. `Env` centralises it:

```
from tina4_python import Env

debug   = Env.bool("TINA4_DEBUG", default=False)
workers = Env.int("WORKERS", default=4)
rate    = Env.float("RATE_LIMIT", default=10.0)
region  = Env.str("AWS_REGION", default="us-east-1")
```

Same API across all four frameworks:

- **Python** - `from tina4_python import Env`
- **PHP** - `Tina4\Env::bool / int / float / str`
- **Ruby** - `Tina4::Env.bool / int / float / str`
- **Node.js** - `import { Env } from "@tina4/core"`

Truthy tokens (case-insensitive after `strip`/`trim`): `1`, `true`, `on`, `yes`, `y`, `t`. Falsy: `0`, `false`, `off`, `no`, `n`, `f`, empty string. Anything else returns the `default` - never raises. `int`/`float` parse failures log a warning via `Log` and fall back to default.

Function-name in log lines (tina4-python#41)

Opt-in via `TINA4_LOG_FUNC=true`. When enabled, the calling function name is injected into every log line so a `tail -f` gives you free context:

```
2026-06-03T14:22:18.341Z [INFO   ] [super_trooper] Hello from inside the function
```

Or in JSON mode:

```
{"timestamp":"...","level":"INFO","function":"super_trooper","message":"Hello..."}
```

Default off - zero overhead unless opted in. When on, ~5% per-call cost from the stack walk.

Per-framework implementation:

- **Python** - `inspect.currentframe()` walk past Log's own frames
- **PHP** - `debug_backtrace(DEBUG_BACKTRACE_IGNORE_ARGS)` + `{closure}` filter
- **Ruby** - `caller_locations(2, 16)` + block-noise regex
- **Node.js** - `new Error().stack` regex parse + anonymous filter

Anonymous frames (`<lambda>`, `<module>`, `{closure}`, anonymous IIFEs) are filtered as noise - showing `[<lambda>]` would be uglier than nothing.

Test count

Framework	Before	After	New
Python	2,675	2,725	+50
PHP	2,780	2,844	+64
Ruby	2,839	2,887	+49 (+1 pre-existing rack_app fail unchanged)
Node.js	3,420	3,477	+57
Total	**11,714**	**11,933**	**+220**

Upgrade

Drop-in patch. No breaking changes. Two new exports (`Env`, plus one new env var `TINA4_LOG_FUNC`). Existing logs and existing code keep working unchanged.

v3.13.2 (2026-06-03)

Bug-fix patch - three field reports, fixed with full cross-framework parity audit per `feedback_crosscheck_bugs`.

SCSS calc() with mixed units (tina4-python#42, tina4-php#116, tina4-nodejs#1)

The SCSS math evaluator silently folded mixed-unit arithmetic by keeping operand 1's unit and dropping operand 2's, producing wrong CSS:

- `max-height: calc(100vh - 170px)` -> `calc(-70vh)` (negative, layout-breaking)
- `width: 100% - 20px` -> `80%` (pixel term silently lost)
- `padding: 1rem + 4px` -> `5rem`

Fixed in Python, PHP, and Node - the evaluator now captures both operand units, only folds when units match (or one side is unitless for `*`/`/`), and masks `calc(...)` ranges so the browser computes them as intended. Ruby unaffected (delegates to libsass).

Router.group docs taught a crashing pattern (tina4-python#44)

The Python book and docs site showed `Router.group("/api/v1", lambda: [...])` with a zero-arg lambda. Source intentionally passes a `RouteGroup` instance to the callback, so users hit `TypeError: <lambda>() takes 0 positional arguments but 1 was given`. Docs rewritten to `lambda group: [group.get(...), group.post(...)]` matching the real contract (Node has always taught this correctly; PHP and Ruby use ambient state, no group arg needed).

DATABASE*URL -> TINA4*DATABASE_URL drift (tina4-python#45)

Three real bugs:

- **Python ORM error message** told users to "set DATABASE_URL in .env" - but the v3.12 boot guard rejects that bare name. Users following the error message hit a hard stop.
- **Python dev-admin `.env` writer** stripped the `TINA4_` prefix when updating existing rows, actively corrupting the config every time the user saved a new connection through the dashboard.
- **Node** `Database.fromEnv()` defaulted to `"DATABASE_URL"` as the env-var key, missing the project's actual connection. The ORM error message had the same drift.

All fixed. PHP and Ruby audited - already correct in both.

Test count

Framework	Before	After	New
Python	2,665	2,675	+10
PHP	2,774	2,780	+6
Ruby	2,839	2,839	0 (parity bump only)
Node.js	3,406	3,420	+14
Total	**11,684**	**11,714**	**+30**

Upgrade

Drop-in patch. No breaking changes. No new public API.

v3.13.1 (2026-06-02)

Cross-framework parity patch. Closes the remaining audit-flagged docs-vs-code gaps that didn't make 3.13.0 - the documentation claimed APIs across PHP / Ruby / Node that only Python had. This release ships those APIs everywhere and rewrites the PHP chapters that referenced fictional symbols.

Convenience parity additions (Groups A / B)

Three highest-impact cross-framework methods every documentation set already claimed existed. PHP / Ruby / Node now match Python:

- `db.fetchAll(sql, params)` / `db.fetch_all` / `$db->fetchAll` - returns the records list directly. Symmetric with `fetch_one`. For the 80% case where you don't need the `DatabaseResult` metadata.
- `Database.getConnection(url)` / `.get_connection` / `::getConnection` - classmethod factory matching SQLAlchemy's `engine.connect()`. Falls back to in-memory SQLite when no URL or env resolves.
- `Api(bearerToken=, username=, password=, headers=, verifySsl=)` **ergonomic kwargs** - three setter calls collapse to one constructor. Bearer wins over basic-auth when both are passed. `verifySsl=False` is the positive form of `ignoreSsl=true`.

Decorator-style GraphQL resolvers across the family

Python `@GraphQL.resolve` shipped in 3.13.0. This release adds:

- **PHP** - `GraphQL::resolve("Type", "field", $callable)` static method + class-level resolver registry that `new GraphQL()` drains into its schema.
- **Ruby** - `Tina4::GraphQL.resolve("Type", "field") { |root, args, ctx| ... }` with block-based registration.
- **Node.js** - `GraphQL.resolve(typeName, fieldName, resolver)` matching the cross-framework shape.

All four frameworks now support the FastAPI / Strawberry / Ariadne pattern where resolvers register at module-import time before any `GraphQL` instance is constructed, and where post-startup registrations land in the active default singleton via `setDefault(gql)` / `Tina4::GraphQL.default_instance = gql`.

Class-based service pattern across the family

`class FooWorker extends Service { run() { ... } }` - chapter 27 / equivalent docs have long taught this pattern. Until 3.13.1, only the runner was real:

- **PHP** - new `Tina4\Service` abstract base class + `ServiceRunner::registerService($name, $service)` static helper.
- **Ruby** - new `Tina4::Service` class + `Tina4::ServiceRunner.register_service(name, service)`.

- **Node.js** - new `Tina4Service` abstract class + `ServiceRunner.registerService(name, service)`.
- **Python** - new `tina4_python.service.Service` base + `ServiceRunner.register_service(name, service)` (this release closes the gap; Python had only the function-style runner before).

All four ship `run()` (abstract), `stop()`, and `should_stop()` / `shouldStop()` helpers backed by an internal flag. Function-style services using bare callables continue to work alongside the new class-based pattern.

PHP chapter rewrites (`docs/php/` and `book-2-php/`)

The 3.13.0 audit found that the PHP testing-chapter disaster was the tip of a larger pattern - multiple PHP chapters taught APIs that didn't exist. 3.13.1 rewrites all seven of them:

- **Chapter 15 - Logging** - primary surface now `Tina4\Log::info()/warning()/error()` instead of the legacy `Tina4\Debug::message()` shim (still works).
- **Chapter 18 - Testing** - `$response->statusCode` -> `$response->status` across 23 occurrences; CLI section updated (`tina4 test` runs the suite; `vendor/bin/phpunit` for targeted runs).
- **Chapter 19 - Scaffolding** - v2 `Tina4\Get::add()` / `Post::add()` / `Put::add()` / `Delete::add()` syntax replaced with `Tina4\Router::get/post/put/delete`; fictional `->description()` chain replaced with real `->swagger([...])`.
- **Chapter 22 - GraphQL** - chapter's decorator pattern (`GraphQL::resolve("Type", "field", $fn)`) now matches real source (built this release).
- **Chapter 25 - WSDL** - `@wsdl_operation` docblock replaced with `#[WSDLOperation([...])]` PHP attribute; methods now return associative arrays matching the response-shape spec; `Router::soap()` -> `Router::any()` + manual `(new Service($request))->handle()`.
- **Chapter 27 - ServiceRunner** - `new ServiceRunner()` + `->add()` instance API replaced with `ServiceRunner::registerService()` + `ServiceRunner::start()` static API. The `Tina4\Service` base class the chapter teaches now exists.
- **Chapter 34 - Deployment** - un-prefixed env vars (`SECRET`, `CORS_ORIGINS`, `SMTP_USER`, `JWT_SECRET`, `API_KEY`, `SWAGGER_TITLE`) replaced with `TINA4_`-prefixed forms. The v3.12 boot guard rejects the legacy names with `exit(2)`.

Test count

Net new across the family this release:

Framework	Before	After	New
Python	2,654	2,665	+11
PHP	2,749	2,774	+25
Ruby	2,827	2,839	+12
Node.js	3,384	3,406	+22

Total	**11,614**	**11,684**	**+70**

Upgrade

Drop-in patch - no breaking changes. Existing source-code patterns from 3.13.0 continue to work; the new methods are additive. Documentation rewrites in chapters 15 / 18 / 19 / 22 / 25 / 27 / 34 redirect copy-paste examples to the real APIs the framework actually ships.

v3.13.0 (2026-06-01)

The docs-vs-code parity release. A cross-framework audit of 381 markdown files surfaced 146 hallucinations, signature drifts, and stale references across Python, PHP, Ruby, Node, and tina4-js. 3.13.0 closes the chapter-18 disaster pattern - where documentation taught a class-based API that didn't exist - by shipping the missing pieces, renaming the misnamed pieces, and rewriting the aspirational chapters.

The headline fire: `Test` class with HTTP helpers

Every framework's chapter 18 has long shown integration tests like:

```
class UserApiTest(Test):           # tina4_python.test.Test
    def test_health(self):
        resp = self.get("/health")
        assert_equal(resp.status, 200)
```

Until 3.13.0, only `Test` (the bare assertion base) existed - calling `self.get(...)` crashed with `AttributeError`. The HTTP test client lived in a separate `TestClient` class that the docs never mentioned.

This release mixes `TestClient.get / post / put / patch / delete` into the `Test` base across every framework:

- **Python** - `tina4_python.test.Test` (extends `unittest.TestCase`, pytest auto-discovers)
- **PHP** - `Tina4\Test` (extends `PHPUnit\Framework\TestCase`)
- **Ruby** - `Tina4::Test` (zero-dep; built-in `run_all` runner)
- **Node.js** - `Tina4Test` (zero-dep; built-in `Tina4Test.runAll()` runner)

Plus positional assertions on every framework - `assertEqual(actual, expected, message)`, `assertNotEqual`, `assertTrue`, `assertFalse`, `assertNull`/`assertNullValue`, `assertNotNull`/`assertNotNullValue`, `assertRaises` - matching the documented `(actual, expected, message)` shape.

`Auth.valid_token` now returns the payload, not a bare bool - BREAKING

The most common silent-fail pattern caught by the audit. Every framework's docs claimed `valid_token` returned the decoded JWT payload; every framework's source returned `bool` and forced a second `get_payload` call.

Framework	Before	After	

Python	`Auth.valid_token(token) -> bool`	`Auth.valid_token(token) -> dict \	`None`
PHP	`Auth::validToken(token) -> bool`	`Auth::validToken(token) -> array \	`null`
Ruby	`Auth.valid_token(token) -> Boolean`	`Auth.valid_token(token) -> Hash \	`nil`
Node	`validToken(token) -> boolean`	`validToken(token) -> Record<string, unknown> \	`null`

Matches PyJWT / firebase-jwt-ruby / firebase/php-jwt / jsonwebtoken conventions. Truthy/falsy contract preserved - existing `if (validToken(t))` callers keep working because a non-null object is truthy and null is falsy.

Python-specific groups (mirrored to PHP/Ruby/Node in follow-up patches)

The Python framework is the reference per `feedback_python_master`. Six groups landed in tina4-python:

- **Group A - ergonomic additions**: `Database.get_connection()`, `db.fetch_all()`, `db.pool`, `DatabaseResult.columns`, `Job.error`, `Queue.produce(delay_until=datetime)`, module-level `migrate(db)`/`rollback(db)`/`status(db)`, module-level `i18n.t()`, dict-style `session[key]`, `WebSocketConnection.connection_count`. All zero-risk additions - no signatures changed.
- **Group B - signature expansions**: `Api(bearer_token=, username=, password=, headers=, verify_ssl=)` kwargs, `Model.find(pk)` int overload (Active Record convention), `@description(summary, detail=, params=, query=)`, `@tags(str | list)`, `@example_response(status_code, data)`, `response.render(template, data, status_code)`, `response.cookie(name, value, options_dict)`, `response(data, headers={})`, `@get(path, description=, middleware=["ResponseCache:300"])` with string-form middleware parser.
- **Group C - mixins + decorators**: the Test HTTP mixin (covered above), `Frond.add_filter / add_global / add_test` callable as classmethod OR instance method via a `_ClassOrInstanceMethod` descriptor, `@GraphQL.resolve("Type", "field")` decorator with class-level registry - chapter 22's pattern now works as documented.
- **Group D - return-type changes (BREAKING)**: `Container.reset()` now clears singleton cache only (factories survive); new `Container.reset_all()` for the old wipe-everything behaviour. `queue.dead_letters()` returns `list[Job]` with `.error` populated, not `list[dict]`. `Model.where(..., with_count=True)` returns `(list, int)` tuple for pagination UIs.
- **Group E - renames (BREAKING)**: `ai.install_all()` -> `ai.install_context()`; new `ai.detect_ai()`, `ai.detect_ai_names()`, `ai.status_report()`. `queue.consume(id=)` -> `queue.consume(job_id=)`. `Api.send_request()` -> `Api.send()`. `I18n(locale=, path=)` preferred over `I18n(locale_dir=,`

`default_locale=)` (legacy kept). `TINA4_TOKEN_EXPIRES_IN` preferred over `TINA4_TOKEN_LIMIT` for JWT expiry (both honoured; new wins; constructor arg overrides both).

• **Group F - top-level re-exports + scaffolder**: `from tina4_python import Api, WSDL, wsdl_operation, GraphQL, AutoCrud, Messenger, on, emit, once, off, tests` now resolve. `Model.select()` with no args defaults to `SELECT * FROM <table>` so the CRUD-list scaffolder template's emitted code actually runs.

PHP-specific: `Tina4\Debug` shim

Chapter 15 of the PHP logging docs taught `Tina4\Debug::message($msg, TINA4_LOG_INFO, [...])`. Neither the class nor the constants existed. Real logger is `Tina4\Log`.

This release ships a `Tina4\Debug` compatibility shim that forwards to `Tina4\Log`, plus defines the `TINA4_LOG_*` level constants - so the chapter's code samples run as-written. For new code, prefer `\Tina4\Log::info()` etc.

Documentation sweep

Aside from the source-side changes, the audit caught hundreds of stale references in docs site + book + AI skills + CLAUDE.md files. All fixed in this release:

• ~80 occurrences of `from tina4 import` -> `from tina4_python import` (the Python package is `tina4_python`, not `tina4`)

• `from tina4_python.router` -> `from tina4_python.core.router`

• `TINA4_SESSION_HANDLER` -> `TINA4_SESSION_BACKEND` (matches the env var the framework actually reads)

• `DATABASE_NAME=` -> `TINA4_DATABASE_URL=` (legacy un-prefixed names get rejected by the v3.12 boot guard)

• `@cached(True, max_age=N)` -> `@cached(max_age=N)` (bogus first arg)

• `Template.render()` -> `response.render()` (Template class doesn't exist; renamed to Frond)

• `Debug.error()` -> `Log.error()` in Python (Debug class doesn't exist)

• `Producer` / `Consumer` (removed in v3.2.0) -> `Queue.push / consume`

• `Email` -> `Messenger`, `event.fire / @listener` -> `emit / @on`, `gql` singleton -> `GraphQL()` + `@GraphQL.resolve`

• **Security fix**: `Auth.check_password(hash, password)` -> `(password, hash)` in skill ref - the bcrypt comparison was returning False every time due to reversed args (silent-failure auth)

• `request.files['content']` is **raw bytes** - drop `base64.b64decode()` from upload examples

• Deployment chapter env vars all `TINA4_`-prefixed (un-prefixed names brick boot under v3.12 guard)

Aspirational chapters rewritten

Two Python chapters were built on APIs that didn't exist:

- **Chapter 22 (GraphQL)**: rewritten around the new `@GraphQL.resolve("Type", "field")` decorator (the FastAPI/Strawberry pattern). The previous `gql.schema.add_query("name", {dict})` form still works but is no longer the primary documented path.
- **Chapter 25 (WSDL)**: rewritten around the real subclass pattern (`class Calculator(WSDL): @wsdl_operation({"Result": int}) def Add(self, ...): ...; Calculator(request).handle()`). The previous `WSDL(service_name=, namespace=, endpoint=)` constructor + `handle_wsdl` / `handle_request` API was entirely fictional.

tina4-js doc drift caught

The cross-framework audit's synthesis pass dropped tina4-js findings; the raw agent transcripts had 23 real findings:

- Every import in CLAUDE.md and `docs/js/09-graphql.md` used `"tina4-js"` (with hyphen). The npm package is named `tina4js`. Fixed.
- `pwa({...})` was treated as callable; real API is `pwa.register({...})`. `PWAConfig.icon` is a single string, not an `icons: [...]` array.
- `static props = { label: { type: String, default: "..." } }` - the `{ type, default }` wrapper is fictional. Real shape is `static props = { label: String }`.
- `router.navigate('/users/42')` - `navigate` is a top-level export, not a method on `router`.
- Chapter 14's `<slot>` inside a `static shadow = false` component - slots are a Shadow DOM feature. Chapter 14 contradicted chapter 4. Switched to `shadow = true`.

Test counts

Net new across the family:

Framework	Before	After	New
Python	2,537	2,654	+117
PHP	2,742	2,749	+20
Ruby	2,800	2,816	+16 (1 pre-existing unrelated rack_app failure)
Node.js	3,366	3,384	+18
Total	**11,445**	**11,603**	**+171**

Upgrade

`Auth.validToken` is the breakage to know about - your `if Auth::validToken($t)` style code keeps working unchanged because non-null arrays are truthy and null is falsy. If you do `=== true` / `=== false` strict comparisons, switch to `!== null` / `=== null`.

Python: `ai.install_all()` -> `ai.install_context()`, `queue.consume(id=)` -> `consume(job_id=)`, `Api.send_request()` -> `Api.send()`, `Container.reset()` semantic change (use `reset_all()` for old behaviour).

Everything else is additive - new properties, new kwargs, new convenience methods that match what the docs have promised for years.

v3.12.14 (2026-06-01)

Two independent fixes ship together as 3.12.14. **Python** - the `tina4_python.test` class-based xUnit testing surface that the chapter 18 documentation has always promised but never actually existed. Reports came in of developers copy-pasting `from tina4_python.test import Test, assert_equal, assert_true` straight out of the book and getting `ModuleNotFoundError`. The fix was to build the module to match the documentation, not the other way around. **PHP** - `:named` placeholder translation for the four non-PDO adapters where `ORM::save()` was silently failing.

Python - `tina4_python.test` xUnit testing surface

The testing chapter taught a `Test` base class with positional assertions:

```
from tina4_python.test import Test, assert_equal, assert_true

class BasicTest(Test):
    def test_addition(self):
        assert_equal(2 + 2, 4, "Basic addition should work")

    def test_string_contains(self):
        greeting = "Hello, World!"
        assert_true("World" in greeting, "Greeting should contain 'World'")
```

The module did not exist. Every developer who followed the chapter hit an immediate import error. The other surface, `tina4_python.Testing` with the inline `@tests` decorator, has always existed - but the two are for different purposes and the docs only documented one of them.

The fix ships the missing module - `tina4_python/test/__init__.py` - with the `Test` base class (inherits `unittest.TestCase`, so pytest discovers any subclass regardless of class-name convention) and 13 positional assertions. The signatures are uniform: `(actual, expected, message)`. The 2-arg legacy `(value, message)` form keeps working - a type-based dispatch detects which shape the caller used. `assert_raises` accepts three forms: docs form (`callable, exception, message`), context-manager form (`with assert_raises(X):`), and unittest order (`exception, callable`). Lifecycle hooks come in both flavours - snake_case `set_up`/`tear_down` (the Tina4 idiom) and camelCase `setUp`/`tearDown` (for users coming from unittest) - without double-calling when a subclass uses either one.

```
# 13 assertions, all uniform (actual, expected, message)
assert_equal(actual, expected, message="")
```

```
assert_not_equal(actual, expected, message="")
assert_true(actual, expected=True, message="")
assert_false(actual, expected=False, message="")
assert_none(actual, expected=None, message="")
assert_not_none(actual, expected="not None", message="")
assert_in(item, container, message="")
assert_not_in(item, container, message="")
assert_is_instance(value, expected_type, message="")
assert_greater(actual, expected, message="")
assert_less(actual, expected, message="")
assert_almost_equal(actual, expected, places=7, message="")
assert_raises(callable, exception_class, message="")
```

51 new tests in `tests/test_test_module.py` pin the contract: BasicTest from the chapter runs verbatim, every assertion fails when it should and passes when it should, both 2-arg and 3-arg shapes work, snake_case and camelCase lifecycle hooks fire once each (never both). Full Python suite: 2,537 passing.

`tina4 test` continues to run pytest (`subprocess.run([sys.executable, "-m", "pytest", "tests/"] + args)`).

Cross-framework parity check (testing)

PHP / Ruby / Node testing chapters already teach native conventions correctly - PHPUnit, RSpec, and `node:test` respectively. No fake API to fix. The Python-specific gap was that Tina4 had two testing surfaces (`tina4_python.Testing` for inline `@tests` decorator, `tina4_python.test` for class-based suites) and only one of the two existed. The other three frameworks defer to a single native runner each, so the same trap doesn't apply.

PHP - :named placeholder translation across non-PDO adapters

The ORM's `save()` emits `:named` placeholders because PDO would accept them. Four of the five PHP database adapters do not use PDO. `MySQLAdapter` (mysqli), `MSSQLAdapter` (sqlsrv), `FirebirdAdapter` (ibase/fbird), and `PostgresAdapter` (pgsql) all bind positionally. Every INSERT/UPDATE through `save()` against those four engines failed silently. Reads worked because read paths typically use `?` or no params.

A single helper, `SqlTranslation::namedToPositional($sql, $params)`, translates `:name` to `?` and reorders `$params` to match the SQL order. Wired into the four affected adapters at the top of their prepare/execute paths. The helper skips string literals and SQL comments, so a literal `:colon` inside a value stays as a value. Duplicate names bind once per occurrence, so `WHERE id = :id AND parent_id = :id` works as expected.

`SQLite3Adapter` is untouched. ext-sqlite3 natively accepts `:name` via `SQLite3Stmt::bindValue`. The other four did not, and now do.

15 unit tests pin the helper in `tests/SqlTranslationNamedToPositionalTest.php`: order preservation, duplicate names, quoted strings, line and block comments, unknown placeholders, null values, and the `0`-as-value case. Full PHP suite: 2,290 passing.

Cross-framework parity check (`:named` placeholders)

Python (`mysql-connector-python` uses `%s`), Ruby (`mysql2` uses `?`), and Node (`mysql2` uses `?`) build their INSERT/UPDATE SQL with positional placeholders from the ORM down. No `:named` ever emitted. Audited the MySQL adapter and `save()` path in each before shipping; confirmed clean. PHP-only fix.

Upgrade

Drop in for both Python and PHP. No `.env` changes, no API changes.

Python users who followed chapter 18 and hit `ModuleNotFoundError` - bump to `3.12.14`, the `from tina4_python.test import Test, assert_equal, ...` import now resolves. Existing tests written against `tina4_python.Testing` (the inline `@tests` decorator) continue to work - that surface was not touched.

PHP users - `:named` and `?` both work, and the framework picks the right form for whichever driver is underneath. Existing ORM `save()` calls start succeeding on MariaDB/MySQL, PostgreSQL, MSSQL, and Firebird.

v3.12.13 (2026-05-29)

Consolidated parity release. PHP ran ahead through two independent patch releases (3.12.11-3.12.12) while Python / Ruby / Node stayed at 3.12.10. This release realigns all four frameworks on **3.12.13** and ships the cross-framework dev-admin parity sweep - five tiers of work that bring PHP, Ruby and Node up to Python's AI-assisted development surface.

Cross-framework dev-admin parity sweep (Tier 1-5)

The Python framework had pulled ahead on a series of dev-admin features driven by real frustration with the AI coder loop ("Applying a small patch went and messed up my whole file", "Says it is creating files but then doesn't", repeated import-error spirals). This release ports the full set to PHP, Ruby, and Node - same intent, language-idiomatic implementations.

Tier 1 - MCP defensive write layer. `file_write` and `file_patch` now refuse prose-as-filenames (the LLM occasionally emits `## FILE: I'll implement Step 1 by creating the database migration` and the parser used to write a zero-byte file with that sentence as its filename), normalise bare top-level `routes/` / `orm/` / `templates/` / `seeds/` / `controllers/` / `middleware/` paths to their canonical `src/<dir>/` form (auto-discovery only scans `src/`, so a file at `templates/foo.twig` was dead weight), back up existing files to `.tina4/backups/<flat-path>.<ISO-ts>.bak` before overwrite, and refuse suspicious truncations (>200B file -> <30% size = almost always a truncated LLM response). Every attempt logs to `.tina4/agent.log` with a structured category (`write.ok` / `write.refused` / `write.path_normalized` / `write.import_failed`) - the supervisor reads that file on every turn so it sees what broke last time and can self-correct without asking the developer "what's the error?".

Tier 2 - Post-write syntax verification. PHP shells out to `php -l`, Ruby to `ruby -c`, Node to `node --check` (and single-file `tsc --noEmit --allowJs --skipLibCheck` for `.ts`). On parse error the tool result gets an `import_error` field AND a `write.import_failed` log entry

surfaces in the next supervisor turn's failure context. Catches hallucinated framework APIs (`CharField` doesn't exist in `tina4_python.orm.fields` - should be `StrField`; `auto_now_add` keyword on `Field.__init__()`) at write time instead of letting them propagate to a runtime 500 the user only discovers by hitting the URL.

Tier 3 - `/__dev/api/threads` + `/__dev/api/chat` proxy. The SPA now talks to the Rust supervisor agent the same way regardless of framework. `_supervisor_base_url()` matches Python's 4-step ladder (`TINA4_SUPERVISOR_URL` -> `TINA4_AGENT_PORT` -> `PORT+2000` -> `9145`). `active_file` rides through `/chat` POST verbatim so deictic phrases ("fix this", "explain this") bind to the editor's open file without the supervisor asking. The Node port forwards SSE chunks as they arrive; PHP and Ruby buffer (functional - EventSource parses fine - but feels less snappy until a future round of Rack/PHP-FPM streaming work).

Tier 4 - Customer feedback widget. A floating bubble for end-users of a shipped Tina4 app, gated by `TINA4_ENABLE_FEEDBACK=true` AND a non-empty `TINA4_FEEDBACK_WHITELIST`. The framework's response middleware injects `<script src="/__feedback/widget.js" data-tina4-feedback></script>` immediately before the LAST `</body>` tag on text/html responses, ONLY for whitelisted users, NEVER on `/__dev` or `/__feedback` paths (no double-bubble UX on the developer dashboard). One conversational turn at a time POSTs to `/__feedback/api/turn` -> server-side identity stamp from the verified JWT (clients cannot fake `sender`) -> forward to the Rust agent's intake-only agent (zero tools, JSON-only output). Finalised tickets land in the dev admin sidebar with `kind:"feedback"`. Rate-limited at 5 turns/hour per user.

Tier 5 - Stale-source overlay badge + `list_plans()` merge. The error overlay now stamps `captured_at` on render and tags each stack frame whose source file has been modified since: "FILE MODIFIED @ HH:MM:SS UTC - source may not match what failed". Stops the user from chasing ghosts when the AI coder rewrote the file between the error and the page reload. `list_plans()` reads from BOTH `plan/` (user-curated canonical) AND `.tina4/plans/` (AI-planner output), dedupes by filename with `plan/` winning on collision, sorts newest-first, and returns a `path` field so the SPA can open the right file regardless of source dir.

Test counts. Per-framework deltas across the sweep:

Framework	Before -> After (full suite)
Python	2453 -> 2453 (canonical - no new tests, just released)
PHP	2235 -> 2714 (+479)
Ruby	2747 -> 2800 (+53)
Node	3263 -> 3368 (+105)

PHP's larger delta reflects new tests + the 3.12.11 + 3.12.12 lineage rolling forward.

Why all four frameworks at once. Per the cross-framework parity rule: a feature that exists in only one framework is technical debt. The Python-only Tier 1-5 surface had been accumulating for two weeks while the UX was settling. With it settled, this release closes the gap in one

coordinated sweep.

Folded-in from PHP 3.12.11 - file upload regression (`tina4-book#139`)

`WebSocket::parseHttpHeaders()` previously split the entire raw HTTP request on `\r\n` and iterated every line for a `:` to fill the headers map. Multipart body parts have their own `Content-Type`, `Content-Disposition`, and `Content-Transfer-Encoding` headers - those lines matched the parser and overwrote the real request `Content-Type: multipart/form-data; boundary=...` with whatever the last body part's content type was (typically `application/pdf`, `image/png`). Downstream `str_contains($contentType, 'multipart/form-data')` then failed, the multipart branch was skipped, `$parsedFiles` was never set, and `$request->files` came out empty. Every file upload through the stream-socket server was silently lost - the body landed in `$request->body` as a raw multipart string with no way to parse it.

Fix. Stop the parser at the first `\r\n\r\n` (RFC 9112 Section 2.2 boundary between headers and body) before splitting into lines. One logical change in `Tina4/WebSocket.php`. 9 regression tests in `tests/BookIssue139Test.php` cover single-part, multi-part, and mixed-header cases.

Cross-framework parity check. Python (`http.server`), Ruby (`webrick`/`puma`), and Node (built-in `http` module) all delegate header parsing to upstream stdlib HTTP parsers that already split headers from body correctly. PHP was the only framework with a hand-rolled HTTP parser in this code path. No port needed.

Folded-in from PHP 3.12.12 + Python 3.12.13 - v2 `tina4_migration` auto-upgrade (#115)

Projects upgrading from tina4 ^2.x to ^3.x carried a v2-shaped `tina4_migration` table that v3's `ensureMigrationsTable()` left untouched (the `CREATE TABLE IF NOT EXISTS` short-circuited). The v3 reader then selected columns that didn't exist, fell into the "never seen this migration, run it" branch, and re-applied already-applied migrations - typically failing on duplicate-column / table-already-exists errors when the SQL was non-idempotent. The AirOffices ~190-migration codebase tripped on this in March 2026 and needed a manual SQL backfill at the time.

Framework	v2 schema	v3 schema
PHP	`migration_id VARCHAR(14)`, `description`, `content BLOB`, `passed`	`id INT PK`, `migration`, `batch`, `applied_at`
Python	`description` as identifier, `content`, `passed`	`migration_id`, `migration_name`, `executed_at`

Fix. `ensureMigrationsTable()` (PHP) and `_ensure_tracking_table()` (Python) now detect a v2-shaped table (v2 columns present, v3 columns absent) and call an in-place upgrade that ALTERs in the v3 columns alongside the v2 ones, then backfills v3 fields from the v2 data. v2

columns are kept in place so a manual rollback path stays open - they're simply ignored by v3 readers. The match is by file stem: a v2 row's identifier is matched against `migrations/` files by basename (Python uses `000001_create_users.sql` -> stem `000001_create_users` -> v2 description `create_users`).

Cross-framework parity check. Ruby and Node never shipped a v2 migration table with the trapping shape - their v2 lineages used a different column layout that v3's tracker tolerated. Nothing to port.

Folded-in from PHP 3.12.11 - request URL parity

`$request->url` now returns the full absolute URL (`https://host:port/path?query`) instead of just the path. `$request->queryString` (raw query bytes) added for parity with `request.query_string` on the other frameworks. Drop-in - old code that read `$request->path` (untouched) keeps working.

Upgrade

Drop in. No `.env` changes, no API changes.

For projects upgrading from v2.x: the v2 `tina4_migration` auto-upgrade runs once on first boot against v3 - back up your migrations table beforehand if you're paranoid. The upgrade is non-destructive (v2 columns are kept alongside the new v3 ones).

For projects using the dev admin AI coder loop: the new MCP defensive layer will silently rewrite `## FILE: routes/foo.py` to `src/routes/foo.py` and log a `write.path_normalized` entry. If you were relying on the old behaviour (writes landing wherever the LLM emitted them), this will move some files. Run `tail -n 50 .tina4/agent.log | grep path_normalized` after upgrading to see what got rewritten.

For shipping apps that want the customer feedback widget: set `TINA4_ENABLE_FEEDBACK=true` AND `TINA4_FEEDBACK_WHITELIST=alice@example.com,bob@example.com` in `.env`. The widget appears only for those users on non-`/__dev` pages.

v3.12.10 (2026-05-14)

Version-alignment release. PHP ran ahead through three independent patch releases (3.12.7-3.12.9) while Python / Ruby / Node stayed at 3.12.6. This release realigns all four frameworks on **3.12.10** and ships the ORM `save()` fix.

PHP - `ORM->save()` no longer swallows write failures (#114)

`ORM->save()` called `update()`/`insert()` but ignored their `bool` return - it only caught exceptions. The PHP adapter's `exec()` returns `false` on a bad statement instead of throwing, so a failed `UPDATE` (commonly: one referencing a public model property with no matching DB column, since `getDbData()` includes every public property) slipped through. The empty transaction got committed and `save()` returned `$this` - the documented success signal. Callers relying on the `save(): static|false` contract believed the row persisted when nothing changed. **Silent data loss** - no exception, no log.

Fix. `save()` now captures the `bool` return of `update()`/`insert()`, rolls back, and returns `false` on a falsy result.

```
$ok = $this->_exists || ... ? $this->update() : $this->insert();
if ($ok === false) { $this->_db->rollback(); return false; }
$this->_db->commit();
```

Cross-framework parity check. Python, Ruby and Node don't have this exact failure mode - they build the write payload from declared fields only (not all public properties), and their DB adapters raise on bad SQL, which the existing `try/except` already catches. PHP was the outlier on both counts. 3 regression tests in `tests/Issue114Test.php`; PHP suite 2235 -> 2238 passing.

Also in the PHP 3.12.7-3.12.9 patch line

These shipped to PHP between 3.12.6 and this release; folded into the consolidated 3.12.10 line:

- **3.12.7** - `Request` now normalises caller-provided header keys to lowercase. Some upstream entry points (Apache+PHP-FPM custom mappings, certain proxies, hand-written test fixtures) hand headers in with original case. The constructor only looks them up by lowercase key, so without normalisation `multipart/form-data` content-type detection silently missed and the body fell through as raw bytes - a follow-up to the #135 fix.
- **3.12.8 / 3.12.9** - Router gained RFC 9110 HTTP method conformance: proper `HEAD` and `OPTIONS` handling, `405 Method Not Allowed` with an `Allow` header listing the methods a route does support.

Python / Ruby / Node

Version-only bump 3.12.6 -> 3.12.10 to realign with PHP. No behavioural changes in these three since 3.12.6.

Upgrade

Drop in. No `.env` changes, no API changes. PHP users on 3.12.9 get the `save()` fix; everyone else gets a version-number realignment.

v3.12.6 (2026-05-06)

Python-only fix release. PHP / Ruby / Node ship the same version stamp for parity but carry no behavioural changes.

Python - psycopg2 `%` substitution no longer trips PL/pgSQL function bodies (#40)

A migration containing a PL/pgSQL function with literal `%` characters in a `RAISE EXCEPTION` (or `format()`) call used to fail with the misleading:

> *RuntimeError: Migration failed: list index out of range*

The error message gave no hint that the `%` chars were the problem. The user-facing failure looked like a tina4 internal bug - actually psycopg2's argument-substitution system tripping on the literal percent signs.

Root cause. `PostgreSQLAdapter.execute(sql, params)` always called `cursor.execute(sql, params or [])`. psycopg2 interprets `%` as parameter placeholders WHENEVER the `params` arg is supplied - even an empty list `[]`. So a function body containing `RAISE EXCEPTION 'thing % conflicts with %', a, b` (perfectly valid PL/pgSQL) blew up because psycopg2 thought `%` was a placeholder and there were no values to substitute.

Fix. New `PostgreSQLAdapter._safe_execute(cursor, sql, params)` helper routes empty/None params through `cursor.execute(sql)` (no second arg), which makes psycopg2 skip the substitution pass entirely. Literal `%` chars flow through untouched. Applied at every `cursor.execute(...)` call site in the adapter (5 spots across `execute`, `fetch`, `fetch_one`).

Tests. 5 new unit tests in `tests/test_postgres_percent_substitution.py` pin the helper's branching. 3 live-Postgres regression tests in `tests/test_postgres_plpgsql_percent.py` exercise a real CREATE FUNCTION + trigger flow with literal `%` in the body - skipped automatically when no Postgres is reachable. Full suite: 2453 passing (was 2448).

Cross-framework parity check. PHP (`pg_query` vs `pg_query_params`) and Ruby (`exec` vs `exec_params`) already branch on params presence so they don't have this bug. Node uses `$1` placeholders not `%`, so the same class of bug doesn't apply.

Long-standing tina4-js #37 confirmed fixed

`frond.form.submit` not following 3xx redirects - fixed in frond v2.1.2 back on April 11, 2026 (`xhr.responseURL` comparison + `window.location.href` navigation). All four framework `public/js/frond.min.js` copies carry the fix. The original issue stayed open because the reporter never confirmed against the patched build.

Upgrade

Drop in. No `.env` changes, no API changes.

v3.12.5 (2026-05-06)

PHP-only bug fix release. Python / Ruby / Node ship the same version stamp for parity but carry no behavioural changes.

PHP - multipart bodies with file uploads now parse correctly (#135)

Two stacked bugs in `Tina4\Request::__construct` made `$request->body` come through as the raw multipart bytes (~11 KB blobs starting with `------WebKitFormBoundary...`) whenever the request included a file upload:

- The constructor called `$this->parseBody()` BEFORE initialising `$this->files`. Inside parseBody's multipart branch, the line `$this->files = array_merge($this->files, $parsed['files'])` read an uninitialised typed property - fatal `Error`.
- After fixing the init order, that same line tried to mutate the `readonly` `$files` property - another fatal `Error`.

Both errors got swallowed by the upstream error handler and the route handler received the raw multipart payload instead of the parsed associative array. Routes that worked fine for ordinary

form posts broke the moment a file field came along.

Fix. Move `$this->files` initialisation AFTER `parseBody()` runs. parseBody stashes extracted multipart files on a new private mutable `$multipartFiles`; the constructor merges them into the readonly `$files` in a single assignment that respects the readonly contract.

4 new regression tests in `tests/Issue135Test.php` pin the constructor's contract. Full PHP suite: 2235 passing (was 2231).

Upgrade

Drop in. No `.env` changes, no API changes, no other framework changes.

v3.12.4 (2026-05-06)

Documentation-truth release. The `audit-truth.py` CI gate (introduced post-3.12.3) flagged 39 env vars referenced in docs that no framework actually read. This release closes that gap: 25 of them now exist in code, the other 14 are deleted from docs (11 hallucinations + 6 clustering vars deferred to tina4#2). Both audit gates (CLI drift + env-var drift) are now strict in CI.

25 new env vars across all 4 frameworks

Server: `TINA4_HOST`, `TINA4_SUPPRESS`, `TINA4_ENV_FILE`. Health: `TINA4_HEALTH_PATH` (default `/__health`, with `/health` kept as a legacy alias), `TINA4_TRAILING_SLASH_REDIRECT`. Sessions: `TINA4_SESSION_HTTPONLY`, `TINA4_SESSION_NAME`, `TINA4_SESSION_SECURE`. Templates: `TINA4_TEMPLATE_CACHE_TTL` (`0` = permanent). GraphQL: `TINA4_GRAPHQL_AUTO_SCHEMA`, `TINA4_GRAPHQL_ENDPOINT`. Mail: `TINA4_MAIL_IMAP_ENCRYPTION` (`tls`/`starttls`/`none`). MCP: `TINA4_MCP`, `TINA4_MCP_PORT`. Swagger: `TINA4_SWAGGER_ENABLED`, `TINA4_SWAGGER_CONTACT_EMAIL`, `TINA4_SWAGGER_LICENSE`. Database: `TINA4_DB_POOL` (env override on the existing `Database(url, pool=N)` constructor argument).

Logging - env-driven file output + rotation

Six new vars give you full control over logging without touching code:

Var	Default	What it does
`TINA4_LOG_FILE`	*(empty - stdout only)*	Path to a log file. Empty leaves you on stdout.
`TINA4_LOG_DIR`	`logs`	Directory for log files (joined with `_LOG_FILE` if relative).
`TINA4_LOG_FORMAT`	`text`	`text` or `json`. JSON mode emits one structured record per line.
`TINA4_LOG_OUTPUT`	`stdout`	`stdout`, `file`, or `both`. Strict - `stdout` means stdout only.

`TINA4_LOG_CRITICAL`	`false`	Enables a `Log.critical()` level above `error`. Off = no-op.
`TINA4_LOG_ROTATE_SIZE`	`10485760` (10 MB)	Rotate when the file exceeds this many bytes. `0` disables rotation.
`TINA4_LOG_ROTATE_KEEP`	`5`	Number of rotated files to retain (`app.log.1` ... `app.log.N`). Older ones are deleted.

Implementation uses each language's stdlib - Python's `logging.handlers.RotatingFileHandler`, Ruby's `Logger.new(path, shift_age, shift_size)`, and a roll-your-own atomic-rename pattern in PHP and Node. Zero new dependencies in any framework.

Documentation-truth CI gate now strict on both axes

The `audit-truth.py` script now blocks merges to `main` of `tina4-documentation` whenever a doc references a `tina4 <command>` or `TINA4_*` env var that doesn't exist in source. Previously CLI drift was strict; env drift was warn-only. Today both are strict.

Tests added

- Python: +53 tests in `tests/test_env_vars.py` (2395 -> 2448)
- PHP: +59 tests in `tests/EnvVarTest.php` (2172 -> 2231)
- Ruby: +51 examples in `spec/env_vars_spec.rb` (2696 -> 2747)
- Node: +59 tests in `test/envVars.test.ts` (3204 -> 3263)

Cross-framework total: 10,689 tests passing, +222 from 3.12.3.

Upgrade path

Drop in. No breaking changes - every new env var is opt-in with a sensible default. If you were setting any of the 17 deleted vars in your `.env`, the boot guard will warn (then ignore) - clean them out at your leisure.

v3.12.3 (2026-05-05)

Cross-framework parity sweep. Two minor breaking changes in the Ruby and PHP public API that bring all four frameworks onto the same shape.

Breaking changes (Ruby + PHP only)

Ruby Container - predicate now uses `?` suffix.

```
# before (3.12.2 and earlier)
Tina4::Container.has(:mailer)        # outdated
```

```
# after (3.12.3)
Tina4::Container.has?(:mailer)       # idiomatic Ruby predicate
```

This brings Ruby in line with Python (`has()`), PHP (`has()`), and Node (`has()`) while still respecting Ruby's `?`-suffix idiom for predicates returning bool. The pre-existing `resolve` -> `get` rename happened earlier; only the predicate was lagging.

ResponseCache public surface - middleware-only across all four frameworks.

The cache has always been middleware. Two of the four frameworks (PHP, Ruby) historically exposed lookup/store as public methods, which let users couple to internals. The public API is now consistent across all four: use the middleware on a route, and read stats with module-level helpers.

```
# Ruby - module-level helpers (parity with Python)
Tina4.cache_stats   # -> { hits:, misses:, size:, backend:, keys: }
Tina4.clear_cache   # flush all entries

# PHP - static methods on the class
\Tina4\Middleware\ResponseCache::cacheStats();
\Tina4\Middleware\ResponseCache::clearCache();
```

Internal methods that used to be public (`get`, `lookup`, `store`, `cache_response`) are now private. Tests that needed them retain access via `_internal*` test seams marked `@internal`.

Doc parity - CLAUDE.md and book chapter 33

- **CLAUDE.md**: every framework's "Key Method Stubs" section now covers the same surface area Python documents - Queue, QueryBuilder, Frond, Api, Background Tasks, ResponseCache, etc. PHP added 4 sections; Ruby added 5; Node added 13.
- **Book chapter 33**: env var tables are now grounded in source. Each framework's chapter 33 lists every `TINA4_*` var its source actually reads. Found and fixed several gaps - Ruby was missing `TINA4_CACHE_*`, `TINA4_QUEUE_*`, `TINA4_KAFKA_*`, `TINA4_RABBITMQ_*`, `TINA4_MONGO_*`, `TINA4_WS_BACKPLANE`, and the entire `TINA4_SESSION_VALKEY_*` block.

Other fixes

- **Ruby** `lib/tina4/ai.rb` - subprocess output is now force-encoded to UTF-8 before `String#strip`, fixing `Encoding::CompatibilityError` that crashed 4 ai specs on systems with non-ASCII pip output.
- **Node** `test/serverParity.test.ts` - sets `TINA4_OVERRIDE_CLIENT=true` so `start()` actually runs, plus emits the `N passed, M failed` summary line the runner expects. The test was effectively a no-op before; now it's recorded properly.

Genuine gaps surfaced by the parity audit (follow-up, not blocking 3.12.3)

The chapter 33 audit flagged env vars Python documents that no other framework actually reads - Ruby/PHP/Node lack `TINA4_OPEN_BROWSER`, `TINA4_DEV_POLL_INTERVAL`, `TINA4_PUBLIC_DIR`, `TINA4_TOKEN_EXPIRES_IN` alias, plus a few framework-specific gaps (Ruby has no Mongo session backend; Node `TINA4_CSRF` defaults to `false` vs Python's `true`). Tracked for a future patch.

Upgrade path

Symptom	Fix
Ruby: `NoMethodError: undefined method 'has' for Tina4::Container`	Replace `has(:key)` with `has?(:key)`
PHP: `BadMethodCallException` calling `$cache->lookup(...)`	Use the middleware: `[ResponseCache::class, 'beforeCache']` / `[..., 'afterCache']`. Or call `_internalLookup` if you really need direct access (test code only - `@internal`).
Ruby: `NoMethodError: undefined method 'get' for ResponseCache instance`	Use `Tina4.cache_stats` / `Tina4.clear_cache` for stats. Lookup goes through the middleware.

No `.env` changes from 3.12.2.

v3.12.2 (2026-05-05)

Quality-of-life patch. Two related portability fixes - no breaking changes from 3.12.1.

Firebird URL auto-detect

Firebird is the awkward one in the stack. Every other engine has a server-side database name (`postgres://host:port/dbname`), but Firebird wants either an absolute file path on the server, a Windows drive-letter path, or an alias. The classic URI form needs a double slash to keep the leading `/` of an absolute path through the URL parser - unintuitive to anyone used to the way postgres / mysql / mssql encode the database name.

The framework now accepts five equivalent forms and normalises all of them transparently:

URL path you write	Resolved Firebird identifier
`//abs/path/db.fdb` (classic double-slash)	`/abs/path/db.fdb`
`/abs/path/db.fdb` (single-slash, intuitive)	`/abs/path/db.fdb`
`/C:/Data/db.fdb` (Windows drive letter)	`C:/Data/db.fdb`
`/C%3A/Data/db.fdb` (URL-encoded colon)	`C:/Data/db.fdb`
`/employee` (Firebird alias)	`employee`

For ops setups that keep server URL and DB location in separate config layers - or for Windows backslash paths that fight URL encoding - set `TINA4_DATABASE_FIREBIRD_PATH`. The env override wins over whatever path is in the URL.

```
TINA4_DATABASE_FIREBIRD_PATH=C:\firebird\data\app.fdb
TINA4_DATABASE_URL=firebird://SYSDBA:masterkey@localhost:3050/ignored
```

Shipped to all 4 frameworks. 11 regression tests per framework (8 unit + 3 live).

Bug fix specific to PHP - `mysqli` localhost+port quirk

PHP's `mysqli` has a long-standing quirk where `host == "localhost"` triggers a Unix socket lookup and IGNORES the port argument entirely. Connecting to `mysql://...:53306` against a Docker container fails with "No such file or directory" - `mysqli` is hunting for `/tmp/mysql.sock` instead of opening a TCP connection. `MySQLAdapter::rewriteHostForTcp()` now rewrites `localhost` to `127.0.0.1` when a non-zero port is specified, forcing the TCP code path. Bare `mysql:///db` (no port) is preserved so existing socket-based setups keep working.

Other fixes

- **chore(python):** `pyproject.toml` had drifted to `3.10.41` while `__init__.py` read `3.12.1`. Synced both to 3.12.2 so `uv build` and runtime introspection now agree.
- **chore(claude.md, all 4):** stale framework version banners in `CLAUDE.md` headers updated.

No `.env` changes from 3.12.1, no migration needed. Existing 3.12.1 installs upgrade by changing one version number.

v3.12.1 (2026-05-04)

CI-only patch - no framework code changes from 3.12.0.

- **fix(ci, all 4):** every `publish.yml` workflow now declares `permissions: contents: write` on the publish job. Without this, `softprops/action-gh-release` 403'd against the default `GITHUB_TOKEN` on repos whose default Workflow permissions setting was read-only (Ruby and Node hit this every release; PHP and Python worked by luck of repo settings). The explicit declaration makes the workflow self-sufficient.
- **chore(ci):** bumped `softprops/action-gh-release` from `@v1` (unmaintained) to `@v2`.

No `.env` changes, no API changes, no migration needed. Existing 3.12.0 installs can upgrade without touching anything else.

The version-bump itself is the test: a successful 3.12.1 release proves the workflow fix works on Ruby and Node where 3.12.0 needed manual `gh release create`.

v3.12.0 (2026-05-04)

> ***Warning: Breaking change - read before upgrading.*** *Every framework env var now uses the `TINA4_` prefix. Existing `.env` files set with `DATABASE_URL`, `SECRET`, `SMTP_HOST`, `HOST_NAME`, etc. will cause the framework to refuse to boot. Run `tina4 env --migrate` to rewrite, or follow the rename table below.*

Why this release

Tina4's env vars had grown inconsistent. Some had the `TINA4_` prefix (`TINA4_DEBUG`, `TINA4_LOCALE`, `TINA4_CACHE_BACKEND`), others didn't (`DATABASE_URL`, `SECRET`, `SMTP_HOST`). Newcomers had to guess which convention applied to which feature. Existing tools and PaaS dashboards collided with un-prefixed names like `SECRET` and `API_KEY` that other libraries also read. Documentation drifted - 91 env-var names appeared in the docs that didn't exist in any framework, and 22 framework-specific env vars in the code didn't match the names users were told to set.

This release closes all three gaps with a single hard rename. No deprecation period, no fallback chain. The framework refuses to boot if it detects a legacy name in the environment, prints a list of every var to rename, and tells you which command to run.

What changed

• **22 env vars renamed** to `TINA4_*` form. See the migration table below.

• `tina4 env --migrate` **CLI** added to all four frameworks. Reads your `.env`, rewrites it in place, leaves a `.env.bak` backup, prints a diff. Idempotent.

• **Boot-time guard** scans `os.environ` (or the language equivalent) for the 22 legacy names. If any are present, prints the rename map and exits with code 2. Bypass with `TINA4_ALLOW_LEGACY_ENV=true` for migration scripts that need both names set during transition.

• **All 4 framework books rewritten.** Chapter 33 (Environment Variables) is now a clean canonical list - every var prefixed, descriptions current, legacy names removed.

• **Doc-vs-code drift closed.** Of the 91 stale env vars previously documented, 61 were renames (corrected), 32 were never implemented (removed). The `audit-links.py` CI gate stays at 0 broken links / 0 broken anchors.

• **Frond bundle** rebuilt at v2.1.3 - `frond.min.js` footer now shows the version explicitly so users can verify what they have.

Bug fixes shipped alongside the rename

• **#38 PostgreSQL UUID-PK transaction abort** - the post-INSERT `lastval()` probe is now wrapped in a SAVEPOINT, so UUID-PK INSERTs no longer poison the outer transaction with `InFailedSqlTransaction`. Live regression test against PostgreSQL 16. (Affects all 4 frameworks where the PG adapter does this probe.)

• **#39 Landing page + template auto-routing** -

• Auto-routing now scans `src/templates/pages/` only. Partials, layouts, base.twig, errors/, components/, and `_*` files never auto-serve from a URL.

• `TINA4_TEMPLATE_ROUTING=off` kills the feature entirely.

• `src/public/index.html` auto-serves at `/` (and `/foo/` serves `src/public/foo/index.html`) - SPA hosting Just Works.

• The framework landing page only renders when `TINA4_DEBUG=true`. Production never shows it; framework version, dev-admin link, and gallery don't leak to real users.

• The malformed `HTTP/1.1 404 OK` status line is fixed - every status code now uses its canonical RFC 7231/9110 reason phrase.

• **#37 frond.form.submit redirect handling** - verified shipped at v2.1.x; `xhr.responseURL` change triggers `window.location` navigation correctly.

• **#36 Session file handler** - re-verified safeguards (lazy save, WebSocket skip, probabilistic GC, new-and-empty skip) all still in place.

Migration - every renamed var

Legacy name	New name
`DATABASE_URL`	`TINA4_DATABASE_URL`
`DATABASE_USERNAME`	`TINA4_DATABASE_USERNAME`
`DATABASE_PASSWORD`	`TINA4_DATABASE_PASSWORD`
`DB_URL`	`TINA4_DATABASE_URL` (alias dropped)
`SECRET`	`TINA4_SECRET`
`API_KEY`	`TINA4_API_KEY`
`JWT_ALGORITHM`	`TINA4_JWT_ALGORITHM`
`SMTP_HOST`	`TINA4_MAIL_HOST`
`SMTP_PORT`	`TINA4_MAIL_PORT`
`SMTP_USERNAME`	`TINA4_MAIL_USERNAME`
`SMTP_PASSWORD`	`TINA4_MAIL_PASSWORD`
`SMTP_FROM`	`TINA4_MAIL_FROM`
`SMTP_FROM_NAME`	`TINA4_MAIL_FROM_NAME`
`IMAP_HOST`	`TINA4_MAIL_IMAP_HOST`
`IMAP_PORT`	`TINA4_MAIL_IMAP_PORT`
`IMAP_USER`	`TINA4_MAIL_IMAP_USERNAME`
`IMAP_PASS`	`TINA4_MAIL_IMAP_PASSWORD`
`HOST_NAME`	`TINA4_HOST_NAME`
`SWAGGER_TITLE`	`TINA4_SWAGGER_TITLE`
`SWAGGER_DESCRIPTION`	`TINA4_SWAGGER_DESCRIPTION`
`SWAGGER_VERSION`	`TINA4_SWAGGER_VERSION`
`ORM_PLURAL_TABLE_NAMES`	`TINA4_ORM_PLURAL_TABLE_NAMES`

Names that stay un-prefixed (not framework config)

`PORT`, `HOST`, `NODE_ENV`, `RACK_ENV`, `RUBY_ENV`, `ENVIRONMENT` - these are runtime / PaaS conventions, not framework config. Heroku, Railway, Vercel, and friends set them; we keep reading them.

How to upgrade

- **Backup your `.env`:** `cp .env .env.bak.pre-v3.12`
- **Run the migration:** `tina4 env --migrate` - rewrites your `.env` in place.
- **Update PaaS dashboards:** Heroku, Railway, Vercel, Render, Fly.io etc - rename the same vars in your provider's env-var UI.
- **Restart your app.** The boot guard verifies nothing legacy remains.

If your app uses `SECRET`, `DATABASE_URL`, or any other listed name in places besides `.env` (e.g. your CI pipeline's `env:` blocks), update those too - the boot guard checks `os.environ`, not just `.env`.

Parity

All 4 frameworks aligned at **3.12.0**:

- tina4-python 3.11.32 -> 3.12.0
- tina4-php 3.11.32 -> 3.12.0
- tina4-ruby 3.11.32 -> 3.12.0
- tina4-nodejs 3.11.32 -> 3.12.0

Coordinated release across PyPI, Packagist, RubyGems, npm.

v3.11.32 (2026-04-25)

Critical fix - pool + transactions are now actually atomic. Plus a coordinated parity release that aligns all four frameworks at the same version after months of drift.

Before this release, creating a `Database` with `pool > 0` silently broke transactions. The pool's round-robin checkout rotated to a different adapter on every call - so `start_transaction()` pinned its flag on adapter A, the executes autocommitted on adapters B and C, and the final `commit()` / `rollback()` landed on adapter D, which had nothing to commit. Result: `rollback()` was a no-op, writes leaked through, and no error or log surfaced the problem.

The fix pins one adapter to the calling context for the lifetime of a transaction. Each language uses its own primitive:

- **Python** - `threading.local()` on the `Database` instance
- **Ruby** - `Thread.current[:tina4_pinned_adapter_<obj_id>]`
- **Node.js** - `AsyncLocalStorage` from `node:async_hooks` (async-safe across overlapping awaits)
- **PHP** - per-instance property (PHP-FPM is one process per request; threading.local is unnecessary)

While pinned, every database call routes to the same adapter. `commit()` and `rollback()` release the pin so subsequent calls round-robin again.

- **fix (database / all 4):** adapter pinning across transaction scope in `Database._get_adapter()` (and language equivalents). Every backend is affected - SQLite, PostgreSQL, MySQL, MSSQL, Firebird. Firebird exposed it loudest because of its honest "commit-empty-txn is a real no-op" semantics; the others mostly hid the bug behind eager autocommits but still lost rollback atomicity.
- **tests (all 4):** new regression suite - three INSERTs followed by `rollback()` under `pool=4` now leaves zero rows (was leaking three). Three INSERTs followed by `commit()` persists exactly three. Pin-release after commit/rollback verified. `pool=0` regression test added so single-connection mode stays unaffected.
- **parity / version alignment:** all 4 frameworks bumped to 3.11.32 - closes the cross-framework version drift that had built up (PHP at 3.11.31, Python at 3.11.24, Ruby and Node at 3.11.19). A single coordinated release across all four registries: PyPI, Packagist, RubyGems, npm.

No migration needed. Code using `pool=0` (the default for every adapter except where explicitly raised) is unaffected. Code using `pool>0` will now actually honour transactions instead of silently dropping them.

If you've been seeing intermittent "writes vanished" or "rollback didn't help" reports on a pooled `Database`, this release is the cause and the cure.

v3.11.13 (2026-04-16)

Issue-driven release. Everything reported in the open tina4-book issues either was fixed in this version or is already fixed in 3.11.12; this release consolidates the remaining bits and corrects documentation drift.

- **feat (router / all 4):** Explicit typed-parameter system shared across Python, PHP, Ruby, Node. Adds `alpha`, `alnum`, `slug`, `uuid`, and explicit `string` types in addition to the existing `int`/`integer`, `float`/`number`, `path`/`.*`. **Unknown type names now throw at registration** - `{name:str}`, `{id:inetger}`, etc. raise with a clear message listing the valid types instead of silently falling through to the default matcher. Fixes tina4-book#125. +45 new tests across the four suites.
- **fix (gallery / python+php+ruby):** Gallery Try-It / View buttons now open the deployed example in a new tab (`window.open(url, '_blank')`) instead of navigating away from the gallery home. Fixes tina4-book#115.
- **fix (ruby gemspec):** `sqlite3` promoted from `add_development_dependency` to `add_dependency`. Matches the "zero-config SQLite on first run" promise. Fixes tina4-book#100.
- **docs (tina4-book):** PHP Chapter 2 updated - correct port (7145), `->noAuth()` on write-method examples, and an explicit callout explaining the secure-by-default policy for POST/PUT/PATCH/DELETE. Addresses tina4-book#87, #94, #123.
- **docs (tina4-book):** Python `@template` decorator ordering corrected (must sit BELOW the route decorator) in book chapters 04 and 10; Python `request->query` vs

`request->params` distinction in PHP chapter 1.

• **tests (python):** Session-handler tests updated to reflect the real default TTL of 3600s (were stale at 1800s).

• **verified already fixed in earlier 3.11.x releases** - closed comments posted on all of these:

• #79 dotted numeric index (`{{ items.0.name }}`)

• #80 `truncate` filter

• #82 `{{ parent() }}` / `{{ super() }}` across all 4 frameworks

• #83 Ruby dashboard - WEBrick is runtime dep

• #89 `load_dotenv` rename, `DatabaseResult` methods, SQLite WAL locking

• #91 Ruby `request.params` symbol + string keys via `IndifferentHash`

• #93 Ruby `/docs/*` and bare `/*` wildcard routes

• #97 Frond ternary operator

• **parity:** All 4 frameworks bumped to 3.11.13.

v3.11.12 (2026-04-16)

Breaking: `sqlite:///X` URLs are now relative to the project root (cwd), matching the documented convention. For absolute paths use four slashes (`sqlite:////abs/path.db`) or a Windows drive letter (`sqlite:///C:/Users/app.db`).

Before this release, `TINA4_DATABASE_URL=sqlite:///data/app.db` was interpreted differently by every framework. Python/Node/Ruby tried to open `/data/app.db` (absolute) which crashed on macOS with `OSError: [Errno 30] Read-only file system: '/data'`. PHP did the same under the hood. All four frameworks now agree: three slashes = relative, four slashes = absolute.

• **fix (all 4):** `sqlite:///X` resolves under cwd; parent directory auto-created only when inside cwd. Absolute paths are trusted and never mkdir'd at root.

• **fix (python):** `_ensure_folders` no longer creates a bogus `src/migrations/` directory. The migration runner always looks at `migrations/` at the project root - there is only one correct location.

• **parity (php, ruby, node):** Same `sqlite:///X` parsing as Python. Dedicated `resolve_path` / `resolveSqlitePath` helpers in each framework so adapters consistently handle `:memory:`, `./` forms, Windows drive letters.

• **tests:** 9 new Python tests in `TestSQLiteConnectionPath` + `TestProjectFolders`. 4 new PHP tests in `DatabaseUrlTest` covering relative/absolute/Windows/bruce-regression. 6 new Ruby specs in `database_drivers_spec.rb :: SqliteDriver.resolve_path`. Node URL tests expanded in `database.test.ts` with the full relative/absolute/Windows/:memory: matrix.

• **parity:** All 4 frameworks bumped to 3.11.12.

Migration note: If your `.env` has `TINA4_DATABASE_URL=sqlite:///data/app.db`, it will now create `./data/app.db` in the project root (which is what most users actually want). If you

genuinely want an absolute path, change to `sqlite:////data/app.db` (four slashes).

v3.11.11 (2026-04-16)

• **fix (python ORM):** `Field.validate` no longer re-coerces values that are already the correct type. Previously, any PostgreSQL/MSSQL read of a row containing a `DateTimeField` crashed because `datetime(datetime_instance)` raises `TypeError`. The fix accepts native driver types (`datetime`, `bytes`, `int`, `bool`, `float`, `str`) without re-wrapping, and parses ISO-8601 strings into `datetime` for SQLite. See `tina4-python/plan/orm-field-validate-native-types.md`.

• **fix (python ORM):** `BooleanField` vs `IntegerField` ordering handled explicitly. `BooleanField(1)` still coerces to `True`, `IntegerField(True)` still coerces to `1`; no regression for either direction (bool is a subclass of int in Python).

• **tests (python):** 10 new `TestFieldsNativeTypes` cases covering datetime/int/bool/float/bytes/string/ForeignKey round-trips.

• **tests (parity):** Regression-guard "datetime round-trip on read path" tests added to PHP (`ORMV3Test`), Ruby (`orm_spec`) and Node.js (`orm.test.ts`) so an equivalent bug can't creep in there later.

• **parity:** All 4 frameworks bumped to 3.11.11.

v3.11.10 (2026-04-15)

• **fix (php):** Hot-reload loop - DevAdmin's polling fallback used `mt=0` as the baseline, so the first poll after every page load triggered `location.reload()`, which reset `mt=0` again. Loop now initialises the baseline on the first poll.

• **fix (php):** Reload sentinel removed - PHP was the only framework recursively walking `src/` and touching `src/.reload_sentinel` on every reload POST. The sentinel lived inside the Rust CLI's watched tree and fed back into the watcher, triggering a second loop. Replaced with the same in-memory counter used by Python/Ruby/Node.

• **fix (php):** Polling no longer starts more than once when the WebSocket reconnect retry budget is exhausted (added a `pollStarted` guard).

• **feat (parity):** `GET /__dev/api/queue/topics` and `GET /__dev/api/queue/dead-letters` added to PHP, Ruby and Node (previously only in Python). PHP queue endpoints now read from the real `Tina4\Queue` backend instead of returning stubs.

• **feat (devadmin):** Refreshed `tina4-dev-admin.js` bundle (87.8 KB) across all 4 frameworks - adds the topic selector dropdown, inline payload expand/copy, and corrected version display.

• **tests:** 4-way parity tests for hot-reload: mtime starts at 0, POST /__dev/api/reload bumps the counter, no sentinel file is written to disk, mtime is monotonic across successive reloads. Mirrored in `tina4-php/tests/DevAdminTest.php`, `tina4-python/tests/test_dev_admin.py`, `tina4-ruby/spec/dev_admin_spec.rb`, `tina4-nodejs/test/devAdmin.test.ts`.

• **parity:** All 4 frameworks bumped to 3.11.10.

v3.11.9 (2026-04-15)

Catch-up release covering v3.11.0 -> v3.11.9 across all 4 frameworks.

• **feat (websocket):** Full WebSocket parity across Python/PHP/Node/Ruby - `get_client_rooms()` / `getClientRooms()`, `route()` usable as decorator or direct handler registration, matching room/broadcast semantics, plus new parity tests on all 4.

• **feat (graphql):** Input validation and field-level `@auth` directives with context threading.

• **feat (graphql):** Auto-discovery of schemas; removed legacy DevAdmin HTML/JS in favour of the new UI.

• **feat (devadmin - Python):** Queue tab with topic selector, dead-letter listing and replay endpoints, inline payload expand/copy, version display.

• **feat (cli):** Rust CLI now owns file watching - frameworks receive `POST /__dev/api/reload` and internal watchers are disabled when launched by the Rust CLI (`--managed`).

• **fix (cli):** `parseFlags` / `parse_flags` / `parseCliArgs` no longer swallow `host:port` or positional args after boolean flags.

• **fix (scss):** SCSS recompilation loop fixed; output path corrected to `src/public/css/` to match CLI and static serving.

• **fix (frond - Python):** Numeric dotted index for lists (`items.0.name`) now resolves correctly.

• **fix (router - Ruby):** Bare `/*` wildcard capture exposed under `"*"` key for parity.

• **fix (orm - PHP):** Three data-sync bugs fixed: `load()` double-fill, `getPrimaryKeyValue`, `save()` ID sync.

• **fix (graphql):** `from_orm` / `fromOrm` list resolver used `select(skip=)` instead of `all(offset=)`.

• **fix (metrics):** Windows backslash paths normalised to forward slashes.

• **fix (app - PHP):** No longer crashes on notices/deprecations in loaded files; `run()` now prints the banner when starting the server directly.

• **chore:** Example demo store ships with the repo; Windows-friendly setup; `.env.example` and setup scripts added.

• **parity:** All 4 frameworks bumped to 3.11.9. PHP aligned to the 3.x tag scheme on `v3`.

v3.10.99 (2026-04-12)

• **breaking:** `auto_map` now defaults to `True` - ORM models automatically map between camelCase properties and snake_case DB columns. Set `auto_map = False` on your model to restore the old behaviour.

• **feat:** `to_dict(case=)` parameter - pass `case='camel'` to get camelCase keys (for JSON APIs) or `case='snake'` (default) for snake_case keys matching DB columns.

• **feat:** Frond `replace` filter now accepts dict args - `{{ v|replace({"T": " ", "-": "/"}) }}` for multiple substitutions in one call.

• **feat:** `background(callback, interval)` - register periodic tasks that run cooperatively in the asyncio event loop. Replaces `threading.Thread` for background work.

• **feat:** Background task protection - sync callbacks run in a `ThreadPoolExecutor` via `run_in_executor()` with `asyncio.wait_for()` timeout, preventing blocking functions from freezing the server.

• **feat:** Docker image now bundles the example store demo - `docker run tina4stack/tina4-python:v3` starts a working app out of the box.

• **fix:** Cart nav badge now updates reactively on quantity change and item removal (tina4-js `signal`/`computed`/`effect`).

• **fix:** Non-blocking queue consumer - `process_orders()` uses `queue.pop()` (single job per tick) instead of blocking `queue.consume()`.

• **tests:** 6 new parity tests covering `to_dict(case=)`, `auto_map` default, `replace` filter (dict + positional), and `background()` registration.

• **parity:** All features shipped identically across Python, PHP, Ruby, Node.js. 2,304 tests passing.

v3.10.97 (2026-04-11)

• **fix:** frond.form.submit redirect handling - XHR transparently follows 3xx redirects, so the callback received redirected HTML instead of navigating. Fixed by comparing `xhr.responseURL` with the original URL and calling `window.location.href` when a redirect is detected.

• **fix:** Currency placeholder - locale files now default to `$` for the currency symbol.

• **fix:** Admin sidebar alignment - widened sidebar to 220px with `min-width` to prevent label truncation.

• **fix:** Admin table overflow - added `min-width: 0` and `overflow-x: auto` on `.admin-main` to prevent content clipping.

• **fix:** Order detail template - corrected variable names (`items` instead of `order.items`, `item.name` instead of `item.product_name`) and used `.records` from `DatabaseResult`.

• **fix:** Status badges - dashboard recent orders and order list now show colored badge pills with translated status labels (pending, processing, shipped, delivered, cancelled).

• **fix:** Date formatting - admin order/dashboard dates trimmed to `YYYY-MM-DD HH:MM:SS` instead of raw ISO with microseconds.

• **feat:** Cart quantity spinner - reactive qty controls using tina4-js signals, computed values, and effects.

• **feat:** Multi-currency pricing - forex conversion via Api client (frankfurter.app), `|currency` template filter, currency selector in navbar.

• **feat:** MCP server tools - `check_stock`, `low_stock_report`, `search_products` tools and `store://categories`, `store://inventory-summary` resources for AI assistant integration.

• **feat:** Contact form - built with `HtmlElement` and `add_html_helpers()` to demonstrate programmatic HTML generation.

• **feat:** ORM named scopes - `Product.scope("active")`, `Product.scope("low_stock")`, `Product.scope("expensive")`.

• **feat:** Database connection pooling - `Database("sqlite:data/store.db", pool=4)`.

• **feat:** Inline tests - `@tests` decorators on `cart_service.py` and `forex_service.py`.

• **feat:** Language toggle - flag button (FR/GB) in navbar to switch locale.

• **feat:** Helpdesk chat persistence - chat messages stored in DB, history API (`GET /api/chat/history`).

• **dep:** Updated frond.min.js to v2.1.2 across all 4 frameworks (Python, PHP, Ruby, Node.js).

• **parity:** All 4 frameworks bumped to 3.10.97.

v3.10.93 (2026-04-11)

• **fix:** Frond array/dict literal support - `{% set items = ["a", "b"] %}` and `{% set obj = {"k": "v"} %}` now parse correctly.

• **fix:** Frond bracket depth tracking in `_find_outside_quotes()` and `_split_outside_quotes()` - expressions like `arr[i % 2]` no longer treated as top-level arithmetic.

• **fix:** Frond subscript expression evaluation - bracket content uses `_eval_expr()` instead of `_resolve()`, enabling `arr[loop.index0 % 2]`.

• **fix:** Frond slice with variable bounds - `items[start:end]` evaluates bounds through `_eval_expr()`.

• **fix:** Frond multiline `{% set %}` - `_SET_RE` regex now uses `re.DOTALL` flag.

• **docs:** Developer skills updated - Metrics Dashboard guidance, Frond Template Parity rules, `@noauth` security warnings.

• **demo:** Complete e-commerce store example (`example/store/`) with GraphQL search, SSE, WebSocket, Queue, Events, 13 test files.

• **parity:** All Frond fixes applied identically across Python, PHP, Ruby, Node.js. 2,304 tests passing.

v3.10.92 (2026-04-10)

• **refactor:** Extract `RateLimiter` from `core/middleware.py` into its own file `core/rate_limiter.py`. The old import path still works via re-export.

• **feat:** Add `RateLimiterMiddleware` wrapper class with `before_rate_limit()` and `check()` static methods.

• **breaking:** Rename `ErrorOverlay` methods - `render()` -> `render_error_overlay()`, `render_production()` -> `render_production_error()`, `debug_mode()` -> `is_debug_mode()`.

• **feat:** Add `Server.start()` and `Server.stop()` for cross-framework parity.

• **feat:** Add `DatabaseResult.size()`, `to_array()`, `to_json()`, `to_csv()` methods.

• **feat:** Add `ScssCompiler` class with `compile()`, `compile_file()`, `add_import_path()`, `set_variable()`.

• **feat:** Add `DevAdmin.unresolved_count()`, `clear_all()`, `reset()`, `capture()` (5-param), `register()`.

• **fix:** GraphQL test API - update `add_query()` calls to use positional args (args, return_type, resolver).

• **parity:** 44/44 cross-framework features green. 2,263 tests passing.

v3.10.91 (2026-04-10)

• **feat:** Add parity methods - `GraphQLType.parse()`, `Response.send()` params, `QueryBuilder.from_()`, `Debug.configure()`.

• **breaking:** Remove alias methods `from_`, `configure`, `template` - use canonical names only (`from_table`, etc.).

v3.10.90 (2026-04-09)

• **docs:** Chapter 4 (Templates) - new "Dumping Values for Debugging" section covering both `{{ x|dump }}` and `{{ dump(x) }}` forms, their shared `<pre>`-wrapped output, and the `TINA4_DEBUG=true` production gate. Filter table entry updated to reference the new section.

• **docs:** `plan/parity/parity-template.md` updated with a cross-framework dump helper comparison table and marks dump parity as confirmed across all 4 frameworks at v3.10.89.

• **chore:** Version sync release - brings all 4 frameworks to the same patch version (3.10.90) so downstream users can upgrade PHP/Python/Ruby/Node.js in lockstep without hunting version mismatches.

v3.10.89 (2026-04-09)

• **feat:** `{{ dump(value) }}` global function form added to Frond alongside the existing `{{ value|dump }}` filter. Both call a single `_render_dump()` helper and produce identical output.

• **security:** Dump is now **gated on** `TINA4_DEBUG=true`. In production (env var unset or `false`) both the filter and function silently return an empty string. This prevents accidental leaks of internal state, object shapes, and sensitive values into rendered HTML when a developer leaves a `{{ dump(x) }}` call in a template.

• **refactor:** Dump output is wrapped in `<pre>` and HTML-escaped via a single shared code path.

• **test:** 6 new tests in `test_frond.py` (`TestDump`) covering debug-mode output, production silencing, unset-env default-to-production, function/filter parity, and circular references.

v3.10.86 (2026-04-09)

• **feat:** `ForeignKeyField` is now a proper `Field` subclass that auto-wires both sides of the relationship. Declaring `author_id = ForeignKeyField(to=Author)` injects `belongs_to` on the declaring model and `has_many` on the referenced model via `ORMMeta` - no manual descriptor calls required. Override the has-many name with `related_name=`.

• **feat:** Cross-framework parity - same FK auto-wiring semantics now available in PHP (`$foreignKeys`), Ruby (`foreign_key_field`), and Node.js (`type: "foreignKey"`)

• **fix:** `@orm_bind(db)` no longer nulls the decorated class - returns a pass-through decorator

• **fix:** `Auth.get_token`/`valid_token`/`get_payload`/`refresh_token`/`authenticate_request` can now be called on the class (e.g. `Auth.get_token(payload)`) or on an instance via the `_DualMethod` descriptor

• **fix:** `SQLiteAdapter` uses a class-level `threading.Lock` + `PRAGMA busy_timeout = 30000` + `timeout=30` on connect to eliminate `SQLITE_BUSY` deadlocks in the dev server under concurrent writes

• **docs:** Chapter 6 (ORM) updated with a new "ForeignKeyField - Auto-Wired Relationships" section

v3.10.85 (2026-04-09)

• **refactor:** Split queue adapters into separate files - `queue/rabbitmq_backend.py`, `queue/kafka_backend.py`, `queue/mongo_backend.py` (one class per file, aligning with PHP/Node/Ruby architecture)

• **fix:** Updated remaining tests to use bool `valid_token()` + `get_payload()` pattern

v3.10.84 (2026-04-09)

• **fix:** Router/middleware was setting `request.user` / `request.auth` / auth payload to `true` (boolean) instead of the actual JWT payload dict after `validToken()` was changed to return bool - any code reading `request.user["sub"]` etc. would have failed silently or crashed

• **fix:** CSRF middleware was not correctly rejecting invalid tokens (nil check on bool result always passed)

• **fix:** `AuthMiddleware.before_request` called `get_payload` incorrectly - would TypeError at runtime on valid token

• **add:** Headless routing auth payload integration tests to prevent regression

v3.10.83 (2026-04-08)

• **fix:** prevent orphaned session files on WebSocket and anonymous requests (#36)

• **feat:** WebSocket rooms - `join_room`, `leave_room`, `broadcast_to_room`, `room_count`, `get_room_connections`

• **feat:** queue signature parity - instance-scoped `push`/`pop`/`retry`, no topic params on public methods

v3.10.70 (2026-04-06)

• **New:** SSE (Server-Sent Events) support via `response.stream()` - pass a generator, framework handles chunked transfer encoding, keep-alive, and `text/event-stream` content type

• **New:** Chapter 24 added to documentation: Server-Sent Events

• Feature count: 45 (was 44)

• Full parity across Python, PHP, Ruby, Node.js

Version History

Tina4 Python follows semantic versioning. The major version (3) marks the ground-up rewrite from v2. Minor versions (3.1, 3.2, ...) introduce features. Patch versions fix bugs and polish edges.

Every release ships through PyPI. Upgrade with:

```
uv add tina4-python@latest

# or with pip

## v3.10.70 (2026-04-06)

- **New:** SSE (Server-Sent Events) support via `response.stream()` - pass a generator, framewor
- **New:** Chapter 24 added to documentation: Server-Sent Events
- Feature count: 45 (was 44)
- Full parity across Python, PHP, Ruby, Node.js

pip install --upgrade tina4-python
```

Check your current version:

```
import tina4_python
print(tina4_python.__version__)
```

---

v3.10.68 (2026-04-03) - Full Parity Release

• **100% API parity** across Python, PHP, Ruby, Node.js - 30+ issues fixed

• **ORM:** save() returns self/false, all/select/where return arrays, toDict/toAssoc standardized, scope() registers reusable method, where()/all() on Node, count() on PHP

• **Auth:** expires*in in minutes, PBKDF2 260k iterations, env TINA4*SECRET fallback, API key fallback in authenticateRequest

• **Session:** dual-mode flash(), get_flash/getFlash, cookieHeader on all, getSessionId on Node, save() public on Node

• **Database:** execute() returns bool/DatabaseResult for RETURNING, get/*ast*id/get_error on all, getColumns on PHP, cacheStats on Node

• **Request/Response:** Node files as dict, Python query property, cookies on PHP/Node, contentType on Node, xml() on PHP/Node, Ruby callable response

• **Queue:** consume() poll_interval (long-running generator with built-in sleep)

• **WebSocket:** event naming standardized (open/message/close/error), connection ip/headers/params, Python on() string API

• **GraphQL:** schema_sdl()/schemaSdl() and introspect() on all 4

• **WSDL:** Node.js zero-dep DOM parser (replaced regex)

• **Events:** emitAsync()/emit_async() on all 4

• **i18n:** zero-dep YAML locale file support on Python/PHP/Node (Ruby already had it)

v3.10.67 (2026-04-03)

• **BREAKING: request.files content is now raw bytes** - previously base64-encoded; remove any `base64.b64decode()` calls when saving uploaded files. Write `file["content"]` directly to disk

• **load() is now an instance method** - `model.load(sql, params)` calls selectOne internally, populates the instance, returns `True`/`False`. Use `Model.find(id)` for PK lookups

• **api.upload()** added to tina4-js - sends FormData with Bearer token auth for multipart file uploads

• **ORM CLAUDE.md rewrite** - all method stubs now match actual API signatures

• **tina4-js skill** - critical input binding warning, routing docs (`{param}` not `:param`), file upload pattern

v3.10.66 (2026-04-03)

• **Metrics file detail fix** - clicking bubbles in framework scanning mode now resolves paths correctly via scan root tracking

v3.10.65 (2026-04-03)

• **Metrics 3-stage test detection** - filename, path, and content matching

• **Metrics framework mode** - scans framework source with correct relative paths

• **tina4 console** - interactive REPL with framework loaded

• **tina4 env** - interactive environment configuration

• **Brand** - "TINA4 - The Intelligent Native Application 4ramework"

• **Quick references** - 36 sections, DotEnv API documented

• **37 chapters** - 7 new (Events, Localization, Logging, API Client, WSDL/SOAP, DI Container, Service Runner)

• **MongoDB + ODBC adapters** across all 4 frameworks

• **Pagination standardized** - limit/offset primary, merged dual-key response

• **Port kill-and-take-over** on startup

---

v3.10.60 (2026-04-03)

- **tina4 console** - interactive Python REPL with framework loaded (db, Router, ORM, Auth, Api, Log)
- **tina4 env** - interactive environment configuration (database, cache, session, queue, mail)
- **Brand update** - "TINA4 - The Intelligent Native Application 4ramework"
- **Quick reference** - 36 sections covering every framework feature
- **Chapter reshuffle** - 37 chapters, 7 new (Events, Localization, Logging, API Client, WSDL/SOAP, DI Container, Service Runner)
- **RouteGroup fix** - double prefix bug resolved
- **Port kill-and-take-over** - default port always reclaimed on startup
- **Metrics test detection** - expanded to check spec/, tests/, test/ directories
- **MongoDB adapter** (pymongo), **ODBC adapter** (pyodbc)
- **Pagination standardized** - limit/offset primary, merged dual-key response
- **9,138 tests** across all 4 frameworks

---

v3.10.57 (2026-04-02)

- **MongoDB adapter** - `Database("mongodb://host:port/db")`, requires `pip install pymongo`
- **ODBC adapter** - parity with PHP/Ruby/Node
- **RouteGroup class** - `group.get()`/`group.post()` syntax matching PHP/Ruby/Node
- **Pagination standardized** - limit/offset primary, merged dual-key toPaginate() response
- **Test port at +1000** - user testing port (e.g. 8145) stable, no hot-reload
- **Dynamic version** - `__version__` read at runtime, no hardcoded constants
- **ORM TINA4*DATABASE*URL discovery** - auto-connect from env
- **Firebird path parsing** - preserves absolute paths
- **108 features at 100% parity**, 2,112 tests

---

v3.10.54 (2026-04-02)

- **Auto AI dev port** - second listener on port+1 with no-reload when TINA4_DEBUG=true
- **TINA4*NO*RELOAD** env var + --no-reload CLI flag
- **Firebird path parsing** - preserve absolute paths
- **ORM TINA4*DATABASE*URL discovery** - auto-connect from env
- **SQLite transaction safety** - commit() no-op without transaction

- **QueryBuilder docs** - added to ORM chapter

---

v3.10.48 - April 2, 2026

Bug Fixes

Production server explicit opt-in - All frameworks now require an explicit `--production` flag to use production servers (Puma for Ruby, FrankenPHP for PHP, cluster mode for Node.js). Previously, production servers activated automatically when `TINA4_DEBUG=false`, which was surprising behaviour. Now `tina4 serve` always uses the dev server unless `--production` is passed.

Python `--no-browser` - The `run()` function now accepts `no_browser=True` and respects the `TINA4_NO_BROWSER` env var to prevent browser auto-opening on server start.

Test Coverage

Python: 2,132. PHP: 1,992. Ruby: 2,387. Node.js: 2,546. Total: 9,057 tests, 0 failures.

---

v3.10.46 - April 1, 2026

Test Coverage

Massive test parity push across all 4 frameworks. CSRF middleware tests expanded to 29+ per framework. Dedicated test suites added for FakeData, Cache, DevMailbox, Static files, Metrics, CLI scaffolding, and all remaining gap areas. Python: 2,132 tests. PHP: 1,937. Ruby: 2,274. Node.js: 2,546. Total: 8,889 tests, 0 failures, 49 core areas with full parity.

---

v3.10.45 - April 1, 2026

Bug Fixes

PHP CLI serve hijack - When `index.php` calls `App::run()`, the CLI `serve` command now sets a `TINA4_CLI_SERVE` constant so `run()` returns early, letting the CLI manage the server lifecycle (port, debug mode, browser open). Previously, `index.php`'s `run()` would start its own server and block the CLI's serve logic.

---

v3.10.44 - April 1, 2026

New Features

Database tab redesign - The dev admin Database panel now uses a split-screen layout. Tables are listed on the left as a navigation sidebar with click-to-select highlighting. The query editor, toolbar, and results occupy the right panel. Results render immediately below the query box with no gap.

Copy CSV / Copy JSON - Two new buttons in the database toolbar copy query results to the clipboard. CSV uses proper comma-separated format with quoting; JSON copies a formatted array of objects.

Paste data - A new Paste button opens a modal where you can paste JSON arrays or CSV/tab-separated data. The tool auto-detects the format and generates INSERT statements. If a table is selected on the left, it targets that table. If no table is selected, it prompts for a name and generates a CREATE TABLE statement for new tables. If you paste SQL directly, it passes through to the query box unchanged.

Multi-statement execution - The query runner now handles multiple SQL statements separated by semicolons. CREATE TABLE + INSERT batches run in a single transaction with automatic rollback on error.

Database badge on load - The Database tab count badge now shows the table count immediately when the dev admin opens, without needing to click the tab first.

Star wiggle animation - The GitHub star button on the landing page uses an empty star (*) with a playful wiggle animation: 3-second delay on page load, then wiggles at random 3-18 second intervals.

Bug Fixes

Default port - Python default port changed from 7145 to 7146 to avoid clashes when running multiple Tina4 frameworks (PHP=7145, Python=7146, Ruby=7147, Node=7148).

SQLite LIMIT fix - The SQLite adapter now checks if a query already contains a LIMIT clause before appending one, preventing double-LIMIT errors in the database browser.

browseTable quote escaping - Fixed broken onclick handlers for table names in the database panel. Now uses `addEventListener` instead of inline onclick with escaped quotes.

Migration UP/DOWN separation - Migration generator no longer puts DOWN SQL in the .sql file. UP SQL stays in the .sql file; DOWN SQL goes in the separate .down.sql file.

Test Coverage

Major test expansion across all 4 frameworks - 8,107 total tests (up from ~5,200), with full parity across 49 core feature areas. New dedicated test suites added for FakeData, Cache, DevMailbox, Static files, Metrics, and CLI scaffolding.

Python: 2,132 tests passing (12 skipped).

---

v3.10.40 - April 1, 2026

Bug Fixes

Dev overlay version check - Fixed misleading "You are up to date" message when running a version ahead of what's published on PyPI (e.g. running v3.10.39 locally while PyPI still has v3.10.24). The overlay now shows a purple "ahead of PyPI" message. Also added a breaking changes warning (red banner with changelog link) when a major or minor version update is available, so developers know to check for breaking changes before upgrading.

---

v3.10.39 - April 1, 2026

Breaking Changes

This release aligns the Python framework with the other three Tina4 implementations. Two breaking changes affect existing code:

`Auth.check_password()` **parameter order reversed**

```
# BEFORE (v3.10.38 and earlier)

## v3.10.70 (2026-04-06)

- **New:** SSE (Server-Sent Events) support via `response.stream()` - pass a generator, framewor
- **New:** Chapter 24 added to documentation: Server-Sent Events
- Feature count: 45 (was 44)
- Full parity across Python, PHP, Ruby, Node.js

Auth.check_password(hashed, password)

# AFTER (v3.10.39+)

## v3.10.70 (2026-04-06)

- **New:** SSE (Server-Sent Events) support via `response.stream()` - pass a generator, framewor
- **New:** Chapter 24 added to documentation: Server-Sent Events
- Feature count: 45 (was 44)
- Full parity across Python, PHP, Ruby, Node.js

Auth.check_password(password, hashed)  # password first - matches PHP, Ruby, Node.js
```

`Router.all()` **removed - use** `get_routes()` **or** `list_routes()`

```
# BEFORE

## v3.10.70 (2026-04-06)

- **New:** SSE (Server-Sent Events) support via `response.stream()` - pass a generator, framewor
- **New:** Chapter 24 added to documentation: Server-Sent Events
- Feature count: 45 (was 44)
- Full parity across Python, PHP, Ruby, Node.js

routes = Router.all()

# AFTER
```

```
## v3.10.70 (2026-04-06)

- **New:** SSE (Server-Sent Events) support via `response.stream()` - pass a generator, framewor
- **New:** Chapter 24 added to documentation: Server-Sent Events
- Feature count: 45 (was 44)
- Full parity across Python, PHP, Ruby, Node.js

routes = Router.get_routes()   # or Router.list_routes()
```

New Features

`Auth.validate_api_key(provided, expected=None)`

Compare API keys with constant-time comparison. Optionally pass `expected`; if omitted, reads `TINA4_API_KEY` (or `TINA4_API_KEY`) from environment.

```
Auth.validate_api_key("sk-abc123")              # check against env
Auth.validate_api_key("sk-abc123", "sk-abc123") # check against explicit value
```

`Auth.authenticate_request(headers)`

One-call header authentication: checks Bearer JWT, Bearer API key, and Basic auth in order.

```
payload = Auth.authenticate_request(request.headers)
# Returns: dict on success, None on failure
```


```
## v3.10.70 (2026-04-06)

- **New:** SSE (Server-Sent Events) support via `response.stream()` - pass a generator, framewor
- **New:** Chapter 24 added to documentation: Server-Sent Events
- Feature count: 45 (was 44)
- Full parity across Python, PHP, Ruby, Node.js
```


`ORM.find_by_id(pk)` **with** `find()` **and** `load()` **as aliases**

`find_by_id()` is now the explicit primary method. Both `find()` and `load()` continue to work as aliases, ensuring backward compatibility.

Test Coverage

2,054 tests passing (up from 2,051 in v3.10.38).

---

v3.10.38 - April 1, 2026

Code Metrics & Bubble Chart

The dev dashboard (`/__dev`) now includes a **Code Metrics** tab with a PHPMetrics-style bubble chart visualization. Files are represented as animated bubbles - sized by lines of code, colored by maintainability index (green = healthy, red = needs attention). Click any bubble to drill down into per-function cyclomatic complexity.

The metrics engine uses Python's `ast` module for zero-dependency static analysis:

- Cyclomatic complexity per function
- Halstead volume and maintainability index

- Afferent/efferent coupling and instability
- Violation detection (CC > 20, LOC > 500, MI < 20)

File analysis is sorted worst-first so problem areas surface immediately. Results are cached for 60 seconds with mtime-based invalidation.

AI Context Installer

The `tina4python ai` command now presents a simple numbered menu instead of unreliable auto-detection:

```
1. Claude Code        CLAUDE.md              [installed]
2. Cursor             .cursorules
3. GitHub Copilot     copilot-instructions.md
...
8. Install tina4-ai tools (requires Python)
```

Select by number (comma-separated or `all`). Already-installed tools show green. The generated context now includes the full skills table across all frameworks.

Dashboard Improvements

- Full-width layout (removed 1400px max-width constraint)
- Sticky header and tab bar when scrolling
- Dashboard overlay fills the screen (was constrained to 1200px)

Cleanup

- Removed `demo/` directories from all framework repos (demos live in documentation)
- Removed old `plan/` spec documents, replaced with `PARITY.md` and `TESTS.md`
- Removed junk sample files (broken migrations, test templates)
- Central parity matrix added to tina4-book

---

v3.10.x - Previous Releases (March 28-31, 2026)

The v3.10 line carries the most patches of any minor release. It refined the Frond template engine, hardened the ORM, and completed cross-framework parity.

Highlights

- **Singleton Frond engine** (v3.10.0). The template engine creates one instance and reuses it. Previous versions spawned a new engine per render call. This cut template rendering overhead across the board.
- **ORM `auto_map` flag** (v3.10.1). Models translate between `snake_case` and `camelCase` column names without manual mapping.

```
from tina4_python import ORM

class UserProfile(ORM):
    auto_map = True  # created_at <-> createdAt handled for you
```

```
    table_name = "user_profiles"
```

- **Frond method calls and slice syntax** (v3.10.2). Templates gained the ability to call methods on objects and use Python slice syntax.

```
<!-- Method calls on template variables -->
{{ user.get_display_name() }}

<!-- Slice syntax -->
{{ long_text[:100] }}...
```

- **Frond quote-aware operator matching** (v3.10.5). Operators inside quoted strings no longer break the parser. Before this fix, a string containing `>=` or `==` could confuse the template engine.

- **ORM auto-commit on write operations** (v3.10.13). Save, delete, and update operations commit their transactions without an explicit call.

- `to_json` **and** `js_escape` **filters** (v3.10.16). Templates gained filters for safe JSON embedding and JavaScript string escaping.

```
<script>
  const config = {{ settings|to_json }};
  const message = "{{ user_input|js_escape }}";
</script>
```

- `formTokenValue()` **helper** (v3.10.23). CSRF tokens gained a dedicated template function for cleaner form markup.

```
<form method="post">
  <input type="hidden" name="form_token" value="{{ formTokenValue() }}">
  <!-- form fields -->
</form>
```

- **MCP server and TestClient parity** (v3.10.32). The built-in MCP server and integration test client reached feature parity with the PHP and Ruby implementations.

- **Arithmetic in** `{% set %}` **and expressions** (v3.10.31). The template engine handles math operations inside assignment blocks.

```
{% set total = price * quantity + shipping %}
```

Bug Fixes

Middleware not applied to routes (fixed in v3.10.1). Middleware functions registered with `@middleware` were silently skipped during route dispatch. Routes ran without their middleware.

```
# Before (broken) - middleware was silently skipped

## v3.10.70 (2026-04-06)

- **New:** SSE (Server-Sent Events) support via `response.stream()` - pass a generator, framewor
- **New:** Chapter 24 added to documentation: Server-Sent Events
- Feature count: 45 (was 44)
- Full parity across Python, PHP, Ruby, Node.js

@app.middleware
def log_request(request, response):
    print(f"Request: {request.method} {request.url}")
    return request, response
```

```
@app.get("/users")
def get_users(request, response):
    # middleware never ran
    return response("OK")

# After (fixed in v3.10.1) - middleware runs on every matching route

## v3.10.70 (2026-04-06)

- **New:** SSE (Server-Sent Events) support via `response.stream()` - pass a generator, framewor
- **New:** Chapter 24 added to documentation: Server-Sent Events
- Feature count: 45 (was 44)
- Full parity across Python, PHP, Ruby, Node.js

@app.middleware
def log_request(request, response):
    print(f"Request: {request.method} {request.url}")
    return request, response

@app.get("/users")
def get_users(request, response):
    # log_request fires before this handler
    return response("OK")
```

Wildcard route matching (fixed in v3.10.1). Routes with wildcard segments failed to match incoming requests. The router now handles wildcards as expected.

`Auth.valid_token` **reference error (fixed in v3.10.9).** The internal server module referenced the wrong attribute name when calling token validation, which caused a `TypeError` on every secured request. The fix resolved the reference so `Auth.valid_token` works as expected.

Frond `dict[variable_key]` **access (fixed in v3.10.11).** Accessing a dictionary with a variable key inside templates raised a `KeyError`. The engine now resolves the variable before the lookup.

```
<!-- Before (broken) - raised KeyError -->
{% set key = "name" %}
{{ user[key] }}

<!-- After (fixed in v3.10.11) - resolves variable, then looks up the key -->
{% set key = "name" %}
{{ user[key] }}  <!-- now outputs user["name"] -->
```

ORM transaction errors on SQLite (fixed in v3.10.25). Calling `save()` or `delete()` on an ORM model raised `"cannot commit -- no transaction is active"` on SQLite. The ORM now wraps every write operation in a proper `start_transaction` / `commit` / `rollback` cycle.

```
# Before (broken on SQLite)

## v3.10.70 (2026-04-06)

- **New:** SSE (Server-Sent Events) support via `response.stream()` - pass a generator, framewor
- **New:** Chapter 24 added to documentation: Server-Sent Events
- Feature count: 45 (was 44)
- Full parity across Python, PHP, Ruby, Node.js

user = User()
user.name = "Alice"
```

```
user.save()  # raised "cannot commit -- no transaction is active"

# After (fixed in v3.10.25) - save() wraps in a transaction automatically

## v3.10.70 (2026-04-06)

- **New:** SSE (Server-Sent Events) support via `response.stream()` - pass a generator, framewor
- **New:** Chapter 24 added to documentation: Server-Sent Events
- Feature count: 45 (was 44)
- Full parity across Python, PHP, Ruby, Node.js

user = User()
user.name = "Alice"
user.save()  # works on all database engines including SQLite
```

Stale templates in dev mode (fixed in v3.10.24). The dev server cached rendered templates and did not pick up file changes until restart. Templates now reload on every request in debug mode.

Macro output HTML escaping (fixed in v3.10.27). Frond macros returned raw strings that the auto-escaper then double-escaped. Macro output now wraps in `SafeString` to preserve the intended HTML.

DevReload performance (fixed in v3.10.28). The live-reload watcher polled too fast and triggered duplicate reloads. It now uses a 3-second default interval with debouncing.

Breaking Changes

`@noauth` **and** `@secured` **decorator behavior (v3.10.1).** Before this fix, these decorators did not update the route's auth flags. If you relied on the broken behavior (routes ignoring auth decorators), your routes will now enforce authentication as intended.

```
# Before (broken) - decorator had no effect

## v3.10.70 (2026-04-06)

- **New:** SSE (Server-Sent Events) support via `response.stream()` - pass a generator, framewor
- **New:** Chapter 24 added to documentation: Server-Sent Events
- Feature count: 45 (was 44)
- Full parity across Python, PHP, Ruby, Node.js

@app.get("/public")
@noauth
def public_page(response):
    return response("Open to all")  # still required auth

# After (fixed in v3.10.1) - decorator works as expected

## v3.10.70 (2026-04-06)

- **New:** SSE (Server-Sent Events) support via `response.stream()` - pass a generator, framewor
- **New:** Chapter 24 added to documentation: Server-Sent Events
- Feature count: 45 (was 44)
- Full parity across Python, PHP, Ruby, Node.js

@app.get("/public")
@noauth
def public_page(response):
```

```
    return response("Open to all")  # accessible without token
```

---

v3.9.x - QueryBuilder and Sessions (March 26-27, 2026)

Features

QueryBuilder with fluent API (v3.9.0). SQL construction through method chaining, integrated with the ORM.

```
from tina4_python import ORM

class User(ORM):
    table_name = "users"

# Fluent query through the ORM

## v3.10.70 (2026-04-06)

- **New:** SSE (Server-Sent Events) support via `response.stream()` - pass a generator, framewor
- **New:** Chapter 24 added to documentation: Server-Sent Events
- Feature count: 45 (was 44)
- Full parity across Python, PHP, Ruby, Node.js

admins = User.query() \
    .where("role = ?", ["admin"]) \
    .order_by("name") \
    .limit(10) \
    .get()

# Standalone query

## v3.10.70 (2026-04-06)

- **New:** SSE (Server-Sent Events) support via `response.stream()` - pass a generator, framewor
- **New:** Chapter 24 added to documentation: Server-Sent Events
- Feature count: 45 (was 44)
- Full parity across Python, PHP, Ruby, Node.js

from tina4_python import QueryBuilder

results = QueryBuilder.table("orders") \
    .select("customer_id", "SUM(total) as revenue") \
    .where("status = ?", ["completed"]) \
    .group_by("customer_id") \
    .having("revenue > ?", [1000]) \
    .get()
```

Path parameter injection (v3.9.0). Route handlers receive path parameters as named function arguments. No more digging through `request.params`.

```
# Before v3.9.0

## v3.10.70 (2026-04-06)

- **New:** SSE (Server-Sent Events) support via `response.stream()` - pass a generator, framewor
- **New:** Chapter 24 added to documentation: Server-Sent Events
```

```
- Feature count: 45 (was 44)
- Full parity across Python, PHP, Ruby, Node.js

@app.get("/users/{id}")
def get_user(request, response):
    user_id = request.params["id"]
    return response(f"User {user_id}")

# v3.9.0 and later

## v3.10.70 (2026-04-06)

- **New:** SSE (Server-Sent Events) support via `response.stream()` - pass a generator, framewor
- **New:** Chapter 24 added to documentation: Server-Sent Events
- Feature count: 45 (was 44)
- Full parity across Python, PHP, Ruby, Node.js

@app.get("/users/{id:int}")
def get_user(id, request, response):
    return response(f"User {id}")  # id is already an int
```

Auto-start sessions (v3.9.0). Every route handler receives `request.session` with zero configuration. The session API covers `get`, `set`, `delete`, `has`, `clear`, `destroy`, `regenerate`, `flash`, and `get_flash`.

```
@app.get("/dashboard")
def dashboard(request, response):
    visits = request.session.get("visits", 0)
    request.session.set("visits", visits + 1)
    return response(f"Visit #{visits + 1}")
```

CSRF middleware and form tokens (v3.9.1). Session-bound CSRF tokens protect forms by default. Toggle with the `TINA4_CSRF` environment variable.

```
# CSRF is on by default in v3.9.1+

## v3.10.70 (2026-04-06)

- **New:** SSE (Server-Sent Events) support via `response.stream()` - pass a generator, framewor
- **New:** Chapter 24 added to documentation: Server-Sent Events
- Feature count: 45 (was 44)
- Full parity across Python, PHP, Ruby, Node.js

# Disable for API-only apps:

## v3.10.70 (2026-04-06)

- **New:** SSE (Server-Sent Events) support via `response.stream()` - pass a generator, framewor
- **New:** Chapter 24 added to documentation: Server-Sent Events
- Feature count: 45 (was 44)
- Full parity across Python, PHP, Ruby, Node.js

# TINA4_CSRF=false

## v3.10.70 (2026-04-06)

- **New:** SSE (Server-Sent Events) support via `response.stream()` - pass a generator, framewor
- **New:** Chapter 24 added to documentation: Server-Sent Events
- Feature count: 45 (was 44)
```

```
- Full parity across Python, PHP, Ruby, Node.js
```

File-based queue backend (v3.9.1). Replaced the SQLite queue with a JSON file-based backend. Zero dependencies. Full cross-platform parity with the PHP and Ruby implementations.

NoSQL QueryBuilder (v3.9.2). The fluent API gained `to_mongo()` to generate MongoDB queries from the same builder syntax.

```
query = QueryBuilder.table("users") \
    .where("age > ?", [21]) \
    .order_by("name") \
    .limit(10) \
    .to_mongo()
# Returns a MongoDB-compatible query dict

## v3.10.70 (2026-04-06)

- **New:** SSE (Server-Sent Events) support via `response.stream()` - pass a generator, framewor
- **New:** Chapter 24 added to documentation: Server-Sent Events
- Feature count: 45 (was 44)
- Full parity across Python, PHP, Ruby, Node.js
```

WebSocket backplane (v3.9.2). Redis pub/sub for scaling WebSocket broadcast across multiple server instances.

SameSite cookie default (v3.9.2). Session cookies default to `SameSite=Lax`. Override with the `TINA4_SESSION_SAMESITE` environment variable.

Breaking Changes

Session API rename (v3.9.0). The `unset()` method became `delete()` for cross-framework parity. `unset()` still works as an alias but will show a deprecation warning in future releases.

```
# Before

## v3.10.70 (2026-04-06)

- **New:** SSE (Server-Sent Events) support via `response.stream()` - pass a generator, framewor
- **New:** Chapter 24 added to documentation: Server-Sent Events
- Feature count: 45 (was 44)
- Full parity across Python, PHP, Ruby, Node.js

request.session.unset("cart")

# After

## v3.10.70 (2026-04-06)

- **New:** SSE (Server-Sent Events) support via `response.stream()` - pass a generator, framewor
- **New:** Chapter 24 added to documentation: Server-Sent Events
- Feature count: 45 (was 44)
- Full parity across Python, PHP, Ruby, Node.js

request.session.delete("cart")
```

Environment variable standardization (v3.9.1). All framework environment variables now follow the `TINA4_*` naming convention. `TOKEN_LIMIT` became `TINA4_TOKEN_LIMIT`. Check your `.env` file and rename any bare variables.

Queue backend change (v3.9.1). The SQLite queue backend no longer exists. If you used it, the file-based backend is a drop-in replacement. Queue data from the old SQLite store must be migrated manually.

---

v3.8.x - Pooling, Validation, and Security (March 25-26, 2026)

Features

Connection pooling (v3.8.1). Pass `pool=N` to the `Database` constructor for round-robin, thread-safe connection pooling.

```
from tina4_python import Database

db = Database("sqlite:///app.db", pool=4)
# Four connections rotate across requests

## v3.10.70 (2026-04-06)

- **New:** SSE (Server-Sent Events) support via `response.stream()` - pass a generator, framewor
- **New:** Chapter 24 added to documentation: Server-Sent Events
- Feature count: 45 (was 44)
- Full parity across Python, PHP, Ruby, Node.js
```

Validator class (v3.8.1). Input validation with an error response envelope.

```
from tina4_python import Validator

errors = Validator.validate(request.body, {
    "email": "required|email",
    "age": "required|integer|min:18"
})

if errors:
    return response({"errors": errors}, 422)
```

Upload size limit (v3.8.1). The `TINA4_MAX_UPLOAD_SIZE` environment variable caps file uploads. Set it in bytes.

TestClient for integration tests (v3.8.1). Test your routes without starting the server.

```
from tina4_python import TestClient

client = TestClient(app)
result = client.get("/api/users")
assert result.status_code == 200
```

SecurityHeadersMiddleware (v3.8.1). One import adds `X-Frame-Options`, `Strict-Transport-Security`, `Content-Security-Policy`, and `X-Content-Type-Options` to every response.

```
from tina4_python import SecurityHeadersMiddleware

app.use(SecurityHeadersMiddleware())
```

Zero core dependencies (v3.8.x). Database drivers, queue backends, and session handlers became optional installs. The core framework runs on Python's standard library alone.

---

v3.7.x - Template Auto-Serve and Firebird Fixes (March 25, 2026)

Features

Template auto-serve at `/` (v3.7.0). Place an `index.html` or `index.twig` in `src/templates/` and the framework serves it at the root path. User-registered `GET /` routes take priority.

```
src/
  templates/
    index.html   <- served at / with no route needed
```

Firebird idempotent migrations (v3.7.0). `ALTER TABLE ADD` statements on Firebird check `RDB$RELATION_FIELDS` before executing. Columns that already exist are skipped. Other databases and statement types are not affected.

---

v3.6.x - API Parity (March 24, 2026)

Breaking Changes

Auth method renames (v3.6.0). The authentication API aligned with the PHP, Ruby, and Node.js implementations.

```
# Before v3.6.0

## v3.10.70 (2026-04-06)

- **New:** SSE (Server-Sent Events) support via `response.stream()` - pass a generator, framewor
- **New:** Chapter 24 added to documentation: Server-Sent Events
- Feature count: 45 (was 44)
- Full parity across Python, PHP, Ruby, Node.js

from tina4_python import Auth

token = Auth.create_token(payload)       # old name
valid = Auth.validate_token(token)       # old name
```

```
# v3.6.0 and later

## v3.10.70 (2026-04-06)

- **New:** SSE (Server-Sent Events) support via `response.stream()` - pass a generator, framewor
- **New:** Chapter 24 added to documentation: Server-Sent Events
- Feature count: 45 (was 44)
- Full parity across Python, PHP, Ruby, Node.js

from tina4_python import Auth

token = Auth.get_token(payload)          # new primary name
valid = Auth.valid_token(token)          # new primary name
# create_token and validate_token still work as aliases
```

```
## v3.10.70 (2026-04-06)

- **New:** SSE (Server-Sent Events) support via `response.stream()` - pass a generator, framewor
- **New:** Chapter 24 added to documentation: Server-Sent Events
- Feature count: 45 (was 44)
- Full parity across Python, PHP, Ruby, Node.js
```

Pagination parameter rename (v3.6.0). `skip` became `offset` across all query methods.

```
# Before

## v3.10.70 (2026-04-06)

- **New:** SSE (Server-Sent Events) support via `response.stream()` - pass a generator, framewor
- **New:** Chapter 24 added to documentation: Server-Sent Events
- Feature count: 45 (was 44)
- Full parity across Python, PHP, Ruby, Node.js

users = User.find(skip=10, limit=5)

# After

## v3.10.70 (2026-04-06)

- **New:** SSE (Server-Sent Events) support via `response.stream()` - pass a generator, framewor
- **New:** Chapter 24 added to documentation: Server-Sent Events
- Feature count: 45 (was 44)
- Full parity across Python, PHP, Ruby, Node.js

users = User.find(offset=10, limit=5)
```

Token expiry parameter rename (v3.6.0). `token_expiry` became `expires_in` and now accepts minutes instead of seconds.

```
# Before

## v3.10.70 (2026-04-06)

- **New:** SSE (Server-Sent Events) support via `response.stream()` - pass a generator, framewor
- **New:** Chapter 24 added to documentation: Server-Sent Events
- Feature count: 45 (was 44)
- Full parity across Python, PHP, Ruby, Node.js

token = Auth.create_token(payload, token_expiry=3600)  # seconds

# After

## v3.10.70 (2026-04-06)

- **New:** SSE (Server-Sent Events) support via `response.stream()` - pass a generator, framewor
- **New:** Chapter 24 added to documentation: Server-Sent Events
- Feature count: 45 (was 44)
- Full parity across Python, PHP, Ruby, Node.js

token = Auth.get_token(payload, expires_in=60)          # minutes
```

Locale environment variable (v3.6.0). `LOCALE` became `TINA4_LOCALE`. Update your `.env` file.

---

v3.5.x - Bundled Frontend (March 24, 2026)

Features

tina4js bundled (v3.5.0). The reactive frontend library (13.6 KB minified) ships with the framework. No CDN link needed. Import it from your templates and build reactive UIs with signals, components, and declarative routing.

AutoCrud Swagger metadata (v3.5.0). Routes generated by `AutoCrud` now include Swagger annotations. They appear in the auto-generated API docs without extra configuration.

---

v3.3.x - WebSockets, File Uploads, and Frond Improvements (March 24, 2026)

Features

Route-based WebSocket handlers (v3.3.0). Define WebSocket endpoints with the same decorator pattern as HTTP routes.

```
@app.websocket("/ws/chat")
def chat_handler(message, client):
    # message is the incoming data
    # client.send() to reply
    client.send(f"Echo: {message}")
```

File upload improvements (v3.3.0). Uploaded files include raw bytes, a `data_uri` template filter, and consistent property names across all frameworks.

```
@app.post("/upload")
def handle_upload(request, response):
    file = request.files[0]
    print(file.filename)    # standardized from file_name
    print(file.type)        # new in v3.3.0
    raw = file.content      # raw bytes
    return response("Uploaded")
```

Lazy `column_info()` on DatabaseResult (v3.3.0). Query results expose schema metadata on demand. Call `result.column_info()` to inspect column names, types, and sizes without a separate query.

`@any` **decorator alias (v3.3.0).** `@any` works as shorthand for `@any_method`, matching the PHP, Ruby, and Node.js API.

Ternary-with-filter support in Frond (v3.3.0). Templates handle inline conditionals that pipe their result through a filter.

```
{{ user.name if user else "Anonymous"|upper }}
```

Breaking Changes

`job.data` **renamed to** `job.payload` **(v3.3.0).** Queue jobs use `payload` as the primary attribute. `job.data` remains as a read-only property alias but will be removed in a future release.

```
# Before
```

```
## v3.10.70 (2026-04-06)

- **New:** SSE (Server-Sent Events) support via `response.stream()` - pass a generator, framewor
- **New:** Chapter 24 added to documentation: Server-Sent Events
- Feature count: 45 (was 44)
- Full parity across Python, PHP, Ruby, Node.js

@app.consume("emails")
def send_email(job):
    to = job.data["to"]

# After

## v3.10.70 (2026-04-06)

- **New:** SSE (Server-Sent Events) support via `response.stream()` - pass a generator, framewor
- **New:** Chapter 24 added to documentation: Server-Sent Events
- Feature count: 45 (was 44)
- Full parity across Python, PHP, Ruby, Node.js

@app.consume("emails")
def send_email(job):
    to = job.payload["to"]
```

File upload property rename (v3.3.0). `file_name` became `filename` (no underscore). The old name is no longer available.

```
# Before

## v3.10.70 (2026-04-06)

- **New:** SSE (Server-Sent Events) support via `response.stream()` - pass a generator, framewor
- **New:** Chapter 24 added to documentation: Server-Sent Events
- Feature count: 45 (was 44)
- Full parity across Python, PHP, Ruby, Node.js

name = request.files[0].file_name

# After

## v3.10.70 (2026-04-06)

- **New:** SSE (Server-Sent Events) support via `response.stream()` - pass a generator, framewor
- **New:** Chapter 24 added to documentation: Server-Sent Events
- Feature count: 45 (was 44)
- Full parity across Python, PHP, Ruby, Node.js

name = request.files[0].filename
```

Route params merged into `request.params` (v3.3.0). Path parameters now merge into `request.params` alongside query parameters. If a query parameter shares a name with a path parameter, the path parameter wins.

---

v3.2.x - Flexible Route Handlers and DevReload (March 24, 2026)

Features

Flexible handler signatures (v3.2.0). Route handlers accept any combination of parameters. The framework inspects the signature and injects what you ask for.

```
# No parameters - fire and forget

## v3.10.70 (2026-04-06)

- **New:** SSE (Server-Sent Events) support via `response.stream()` - pass a generator, framewor
- **New:** Chapter 24 added to documentation: Server-Sent Events
- Feature count: 45 (was 44)
- Full parity across Python, PHP, Ruby, Node.js

@app.get("/ping")
def ping():
    return "pong"

# Response only

## v3.10.70 (2026-04-06)

- **New:** SSE (Server-Sent Events) support via `response.stream()` - pass a generator, framewor
- **New:** Chapter 24 added to documentation: Server-Sent Events
- Feature count: 45 (was 44)
- Full parity across Python, PHP, Ruby, Node.js

@app.get("/hello")
def hello(response):
    return response("Hello")

# Request only (type-hinted)

## v3.10.70 (2026-04-06)

- **New:** SSE (Server-Sent Events) support via `response.stream()` - pass a generator, framewor
- **New:** Chapter 24 added to documentation: Server-Sent Events
- Feature count: 45 (was 44)
- Full parity across Python, PHP, Ruby, Node.js

@app.post("/echo")
def echo(request: Request):
    return request.body

# Both

## v3.10.70 (2026-04-06)

- **New:** SSE (Server-Sent Events) support via `response.stream()` - pass a generator, framewor
- **New:** Chapter 24 added to documentation: Server-Sent Events
- Feature count: 45 (was 44)
- Full parity across Python, PHP, Ruby, Node.js

@app.get("/users")
def users(request, response):
```

```
    return response({"users": []})
```

DevReload with SCSS compilation (v3.2.0). The development server watches for file changes and reloads the browser. SCSS files compile on change.

Route groups (v3.2.0). Group routes under a shared prefix with shared middleware.

MongoDB queue backend (v3.2.0). Use MongoDB as a queue backend alongside the existing file-based, RabbitMQ, and Kafka options.

Migration naming convention (v3.2.0). Migration files follow the `YYYYMMDDHHMMSS` timestamp format. The `tina4 migrate --status` command shows which migrations have run.

Auto-increment port (v3.2.0). If the default port is in use, the framework picks the next available port and opens your browser.

Breaking Changes

Queue constructor simplified (v3.2.0). The `db` parameter was removed from the Queue constructor. The queue reads its backend from the environment.

```
# Before

## v3.10.70 (2026-04-06)

- **New:** SSE (Server-Sent Events) support via `response.stream()` - pass a generator, framewor
- **New:** Chapter 24 added to documentation: Server-Sent Events
- Feature count: 45 (was 44)
- Full parity across Python, PHP, Ruby, Node.js

from tina4_python import Queue, Database
db = Database("sqlite:///queue.db")
queue = Queue("emails", db=db)

# After

## v3.10.70 (2026-04-06)

- **New:** SSE (Server-Sent Events) support via `response.stream()` - pass a generator, framewor
- **New:** Chapter 24 added to documentation: Server-Sent Events
- Feature count: 45 (was 44)
- Full parity across Python, PHP, Ruby, Node.js

from tina4_python import Queue
queue = Queue("emails")
# Backend set via TINA4_QUEUE_BACKEND env var

## v3.10.70 (2026-04-06)

- **New:** SSE (Server-Sent Events) support via `response.stream()` - pass a generator, framewor
- **New:** Chapter 24 added to documentation: Server-Sent Events
- Feature count: 45 (was 44)
- Full parity across Python, PHP, Ruby, Node.js
```

Producer/Consumer classes removed (v3.2.0). Use `queue.produce()` and `queue.consume()` directly.

```
# Before
```

```
## v3.10.70 (2026-04-06)

- **New:** SSE (Server-Sent Events) support via `response.stream()` - pass a generator, framewor
- **New:** Chapter 24 added to documentation: Server-Sent Events
- Feature count: 45 (was 44)
- Full parity across Python, PHP, Ruby, Node.js

from tina4_python import Producer, Consumer
producer = Producer(queue)
producer.send({"to": "user@example.com"})

# After

## v3.10.70 (2026-04-06)

- **New:** SSE (Server-Sent Events) support via `response.stream()` - pass a generator, framewor
- **New:** Chapter 24 added to documentation: Server-Sent Events
- Feature count: 45 (was 44)
- Full parity across Python, PHP, Ruby, Node.js

queue.produce({"to": "user@example.com"})
```

---

v3.1.x - Benchmarks and Internal Improvements (March 22, 2026)

Features

Automated benchmark suite (v3.1.0). A reproducible benchmark compares Tina4 Python against 17 frameworks across four languages. Run it yourself with `python benchmarks/run.py`.

No user-facing API changes in this release. Internal improvements to test infrastructure and benchmark tooling.

---

v3.0.0 - The Rewrite (March 22, 2026)

The v3.0.0 release replaced the entire v2 codebase. Zero external dependencies. Pure Python standard library. 38 features. Over 6,000 tests.

What Changed

- **New module structure.** `tina4_python.core` replaces the old flat namespace.
- **Frond template engine.** Built-in Twig-compatible templates replace the Jinja2 dependency. Pre-compilation caches tokens for a 2.8x speedup on file renders.
- **Decorator-based routing.** `@app.get`, `@app.post`, `@app.put`, `@app.delete`, `@app.patch` replace the old `Route` class.
- **Built-in Dev Admin.** A browser-based dashboard shows routes, database tables, and queue status.

- **Error overlay in debug mode.** Stack traces render in the browser with source context, request details, and suggested fixes.
- **Swagger auto-registration.** Decorated routes appear in the Swagger UI without manual annotation.
- **Unified queue system.** Switch between file-based, RabbitMQ, and Kafka backends through environment variables. No code changes.
- **Database query caching.** Set `TINA4_DB_CACHE=true` for transparent caching with a 4x speedup on repeated queries.
- `tina4 generate` **scaffolding.** Generate models, routes, migrations, and middleware from the command line.
- **Custom error pages.** Self-contained 404, 403, and 500 pages with clean, framework-neutral design.

For the full migration guide, see Chapter 37: Upgrading from v2 to v3.

Release Candidate History

Five release candidates preceded the v3.0.0 stable release between March 21-22, 2026. They resolved test failures, polished the Dev Admin UI, added the benchmark suite, and stabilized the Frond template engine. If you tested a release candidate, upgrade to v3.0.0 or later. The RC builds are not supported.

Upgrading from v2 to v3

1. Overview

Tina4 v3 is a ground-up rewrite. Zero external dependencies. Pure Python stdlib. The framework does more with less.

The concepts are the same -- routing, ORM, templates, migrations, authentication. The APIs are cleaner. The internals are simpler. If you built something with v2, you will recognise everything in v3. But the import paths, decorators, and project layout have all changed.

This chapter covers every breaking change and the migration path for each one. Read it top to bottom before you start, then use the checklist at the end.

Step 0: Run the Automated Upgrade Command

Before doing anything manually, run the automated upgrade tool:

```
cd your-v2-project
tina4 i-want-to-stop-using-v2-and-switch-to-v3
```

This command automatically:

- Moves `routes/`, `orm/`, `templates/`, `scss/`, `public/`, `app/`, `locales/`, `seeds/` into `src/`
- Updates your `pyproject.toml` dependency from v2 to v3

After running the command, continue with the steps below for the changes that require manual attention.

---

2. Package and Installation

v2 installed with pip and pulled in external dependencies:

```
# v2
pip install tina4_python
```

v3 uses uv (recommended) or pip. Zero external dependencies:

```
# v3
uv add tina4-python

# or with pip
pip install tina4-python
```

v3 requires Python 3.12 or later. Check your version:

```
python --version
```

If you are on 3.11 or earlier, upgrade Python first. v3 uses features from 3.12 that cannot be backported.

---

3. Project Structure Changes

v2 used a flat structure. Routes, models, and templates could live anywhere. v3 expects a standard layout:

```
project/
  .env
  app.py
  src/
    routes/
      products.py
      users.py
    orm/
      product.py
      user.py
    templates/
      index.twig
      layout.twig
    app/
      helpers.py
      services.py
```

v3 auto-discovers every `.py` file in `src/` and its subdirectories. No manual imports. No registration. Drop a file in `src/routes/` and it loads at startup.

The framework adds your project root (CWD) to `sys.path`, so imports like this work everywhere:

```
from src.app.helpers import format_currency
from src.orm.product import Product
```

If you have v2 code scattered across the project root, move it into the appropriate `src/` subdirectory. The `tina4 init python .` command creates this structure for you.

---

4. Routing Changes

Decorators

v2 used `@route.get()`. v3 uses standalone decorators:

```
# v2
from tina4_python import route

@route.get("/products")
def list_products(request, response):
    return response.json({"products": []})

# v3
from tina4_python.core.router import get

@get("/products")
async def list_products(request, response):
    return response.json({"products": []})
```

Import all HTTP methods from one place:

```
from tina4_python.core.router import get, post, put, patch, delete
```

Auth Defaults

v3 changes the default auth behaviour. GET routes are public. POST, PUT, PATCH, and DELETE routes require authentication.

To make a write route public, use `@noauth()`:

```
from tina4_python.core.router import post, noauth

@noauth()
@post("/api/feedback")
async def submit_feedback(request, response):
    return response.json({"status": "received"}, 201)
```

To protect a GET route, use `@secured()`:

```
from tina4_python.core.router import get, secured

@secured()
@get("/admin/dashboard")
async def admin_dashboard(request, response):
    return response.render("admin/dashboard.twig")
```

Decorator Order

Order matters. Stack them outermost to innermost:

- `@noauth()` or `@secured()` (auth control)
- `@description("...")` (Swagger docs)
- `@get("/path")` or `@post("/path")` (route binding)

```
@noauth()
@description("Submit anonymous feedback")
@post("/api/feedback")
async def submit_feedback(request, response):
    return response.json({"status": "received"}, 201)
```

Middleware

Middleware uses a decorator. It works in any order relative to the route decorator:

```
from tina4_python.core.router import get, middleware
from src.app.rate_limiter import RateLimiter

@middleware(RateLimiter)
@get("/api/data")
async def get_data(request, response):
    return response.json({"data": []})
```

Wildcard Routes

Wildcard routes now work correctly in v3:

```
@get("/api/*")
async def catch_all(request, response):
    return response.json({"path": request.path})
```

---

5. Database Changes

Connection URL

v2 imported `Database` from the top-level package. v3 imports from `tina4_python.database`:

```
# v2
from tina4_python import Database
db = Database("sqlite3", "data/app.db")

# v3
from tina4_python.database import Database
db = Database("sqlite:///data/app.db")
```

URL format examples:

```
# SQLite
db = Database("sqlite:///data/app.db")

# PostgreSQL
db = Database("postgresql://localhost:5432/myapp", "user", "password")

# Firebird
db = Database("firebird://localhost:3050//var/data/app.fdb", "SYSDBA", "masterkey")
```

Keyword Arguments

v3 passes `**kwargs` through to the underlying driver. Useful for Firebird charset, connection timeouts, and other driver-specific options:

```
db = Database("firebird://localhost:3050//var/data/app.fdb", "SYSDBA", "masterkey", charset="ISO
```

Firebird Column Names

v3 lowercases all Firebird column names. This makes Firebird consistent with every other adapter. Update your code:

```
# v2
email = row["EMAIL"]
first_name = row["FIRST_NAME"]

# v3
email = row["email"]
first_name = row["first_name"]
```

Search your codebase for uppercase column access. Every instance needs updating.

Firebird Drivers

v3 supports both `firebird-driver` (modern, recommended) and `fdb` (legacy). It tries `firebird-driver` first, falls back to `fdb`. Install the one you prefer:

```
uv add firebird-driver    # recommended
# or
uv add fdb                # legacy fallback
```

Transactions

Use the database object's transaction methods. Never use raw SQL for transaction control:

```
# Correct
db.start_transaction()
db.execute("INSERT INTO products (name) VALUES (?)", ["Widget"])
db.commit()

# Wrong -- do not do this
db.execute("BEGIN")
db.execute("INSERT INTO products (name) VALUES (?)", ["Widget"])
db.execute("COMMIT")
```

Connection Pooling

v3 adds connection pooling. Pass the `pool` parameter:

```
db = Database("postgresql://localhost:5432/myapp", "user", "password", pool=4)
```

This creates a pool of 4 connections. The framework manages checkout and return.

---

6. ORM Changes

Field Definitions

v2 field definitions varied. v3 uses typed field classes:

```
# v2
class Product:
    table_name = "products"
    id = None
    name = None
    price = None

# v3
from tina4_python.orm import ORM, bind_database, IntegerField, StringField, FloatField

class Product(ORM):
    table_name = "products"
    id = IntegerField(primary_key=True)
    name = StringField()
    price = FloatField()
```

Binding the Database

Call `bind_database(db)` before using any ORM class. Do this once in `app.py`:

```
from tina4_python.database import Database
from tina4_python.orm import bind_database

db = Database("sqlite:///data/app.db")
bind_database(db)
```

Auto Mapping

Set `auto_map = True` on your ORM class for automatic snake_case to camelCase field mapping. This exists for cross-language parity (Python, PHP, Ruby, Node.js all share the same ORM concepts):

```
class UserProfile(ORM):
    table_name = "user_profiles"
    auto_map = True
    id = IntegerField(primary_key=True)
    first_name = StringField()
    last_name = StringField()
```

Field Mapping

For columns that do not follow conventions, use `field_mapping`:

```
class LegacyUser(ORM):
    table_name = "tbl_users"
    field_mapping = {
        "user_id": "usr_id",
        "email": "usr_email",
        "name": "usr_full_name"
    }
    user_id = IntegerField(primary_key=True)
    email = StringField()
    name = StringField()
```

Relationships

v3 adds `has_many`, `has_one`, and `belongs_to` with eager loading:

```
from tina4_python.orm import ORM, IntegerField, StringField, has_many, belongs_to

class Customer(ORM):
    table_name = "customers"
    id = IntegerField(primary_key=True)
    name = StringField()
    orders = has_many("Order", "customer_id")

class Order(ORM):
    table_name = "orders"
    id = IntegerField(primary_key=True)
    customer_id = IntegerField()
    total = FloatField()
    customer = belongs_to("Customer", "customer_id")
```

---

7. Template Engine Changes

Frond Replaces Template

v2 used a `Template` class. v3 uses the Frond engine, which is Jinja2/Twig-compatible:

```
# v2
return response.template("page.html", {"title": "Home"})

# v3
```

```
return response.render("page.twig", {"title": "Home"})
```

Frond is a singleton. It is created once at startup and reused for every request.

Custom Filters

Register filters in `app.py` before calling `run()`:

```
from tina4_python.frond import Frond

def money(value):
    return f"${value:,.2f}"

Frond.add_filter("money", money)
```

Use in templates:

```
{{ product.price | money }}
```

Custom Globals

Add global variables available in every template:

```
Frond.add_global("APP_NAME", "My Store")
Frond.add_global("YEAR", 2026)
```

```
<footer>&copy; {{ YEAR }} {{ APP_NAME }}</footer>
```

New Template Features

v3 Frond supports method calls on dict values:

```
{{ user.t("greeting") }}
```

Python slice syntax works:

```
{{ text[:10] }}
{{ items[1:3] }}
```

---

8. Migration Tracking Table

v2 used a `tina4_migration` table with a `description` column.

v3 uses an expanded schema:

Column	Type	Description
`migration_id`	text	Unique identifier
`description`	text	Migration description
`batch`	integer	Batch number
`executed_at`	timestamp	When it ran
`passed`	boolean	Whether it succeeded

You do not need to alter the table yourself. Run `tina4 migrate` and v3 auto-detects the v2 schema. It adds the missing columns and backfills `migration_id` from `description`. Your existing migration history is preserved.

```
tina4 migrate
```

That is it. No manual SQL. No data loss.

---

9. Authentication Changes

v2 had various auth approaches. v3 consolidates into an `Auth` class:

```
from tina4_python.auth import Auth

# Generate a token
token = Auth.get_token({"user_id": 42, "role": "admin"})

# Validate a token
is_valid = Auth.valid_token(token)

# Extract the payload
payload = Auth.get_payload(token)
```

Password hashing:

```
hashed = Auth.hash_password("my-secret-password")
matches = Auth.check_password("my-secret-password", hashed)  # True
# Signature: check_password(password, hashed) -- plain text first, hash second
```

JWT uses HMAC-SHA256. Set the signing key in `.env`:

```
TINA4_SECRET=your-long-random-secret-key
```

Token lifetime defaults to 60 minutes. Override with:

```
TINA4_TOKEN_LIMIT=120
```

---

10. Session Changes

v2 had basic file-based sessions. v3 supports pluggable backends.

Set the backend in `.env`:

```
TINA4_SESSION_BACKEND=file
```

Available backends:

Value	Backend	Package Required
`file`	Local filesystem (default)	None
`redis`	Redis	`redis`
`valkey`	Valkey	`valkey`

`mongodb`	MongoDB	`pymongo`
`database`	Database table	None

Session cookies default to `SameSite=Lax`. Override with:

```
TINA4_SESSION_SAMESITE=Strict
```

---

11. New Features in v3

v3 adds capabilities that did not exist in v2. Each has its own chapter:

- **Events system** -- publish and subscribe to application events (Chapter 13)
- **GraphQL engine** -- schema-first GraphQL with resolvers (Chapter 22)
- **WSDL/SOAP services** -- consume and expose SOAP endpoints
- **WebSocket with Redis backplane** -- real-time with horizontal scaling (Chapter 23)
- **Response caching middleware** -- cache responses with TTL and invalidation (Chapter 11)
- **DI container** -- dependency injection for services and repositories
- **Queue system** -- RabbitMQ, Kafka, and MongoDB backends (Chapter 12)
- **Swagger/OpenAPI auto-generation** -- live API docs from route decorators (Chapter 20)
- **Auto-CRUD endpoint generator** -- CRUD routes from ORM models with one line
- **Seeder/fake data** -- populate databases with realistic test data
- **i18n translations** -- multi-language support with translation files
- **AI coding assistant context** -- `tina4 ai` generates context for LLM coding tools
- **Error overlay in dev mode** -- stack traces rendered in the browser
- **SCSS auto-compilation** -- `.scss` files compiled to CSS on change

You do not need to adopt all of these at once. They are opt-in. Migrate your existing code first, then add new features as you need them.

Common Pitfalls

1. POST/PUT/DELETE routes now require authentication

This is the most common upgrade issue. In v2, all routes were public by default. In v3, only GET routes are public -- POST, PUT, PATCH, and DELETE require a Bearer JWT token.

Symptom: Working v2 endpoints return `401 Unauthorized` after upgrading.

Fix: Add `@noauth` to any write route that should remain public.

Find affected routes:

```
grep -rn "@post\|@put\|@patch\|@delete" src/routes/
```

Review each match -- if the endpoint should be public (webhooks, public forms, etc.), add the `@noauth()` decorator.

2. Database connection strings changed

v2 used driver-specific classes. v3 uses URL format:

```
# v2
db = DatabaseSqlite3("data/app.db")
# v3
db = Database("sqlite:///data/app.db")
```

3. Template engine renamed

v2: `Template.render()` -- v3: `Frond.render()` (or `response.render()`)

The Twig syntax is the same -- your `.twig` files work unchanged. Only the Python API call changes.

---

12. Step-by-Step Migration Checklist

Follow this order. Each step builds on the previous one.

- **Install v3**

``bash uv add tina4-python ``

- **Create the v3 project structure**

``bash tina4 init python . ` This creates` src/routes/`,` src/orm/`,` src/templates/`, and` src/app/` if they do not exist. It will not overwrite existing files.

- **Move route files to** `src/routes/`

Update imports from `from tina4_python import route` to `from tina4_python.core.router import get, post, put, patch, delete`. Replace `@route.get()` with `@get()`. Make handler functions `async`.

- **Move ORM models to** `src/orm/`

Replace raw field definitions with typed fields: `IntegerField`, `StringField`, `FloatField`, `TextField`. Add `bind_database(db)` in `app.py`.

- **Move templates to** `src/templates/`

Replace `response.template()` calls with `response.render()`. Templates are Twig-compatible -- most existing templates work without changes.

- **Update** `app.py`

``python from tina4_python.core import run run() ``

- **Update database connections**

Switch to URL format in `.env`: ``~~bash TINA4_DATABASE_URL=~~sqlite:///data/app.db ``

- **Run migrations**

``bash tina4 migrate `` The tracking table upgrades automatically.

- **Fix Firebird column names**

Search for uppercase column access (`row["UPPER"]`) and change to lowercase (`row["upper"]`). Only applies if you use Firebird.

- **Start the server and test**

``bash tina4 serve `` Hit every route. Check the logs for errors.

- **Run the doctor**

``bash tina4 doctor `` This verifies your project structure, database connection, and configuration.

That covers the migration. Most projects take under an hour. The biggest time sink is updating import paths and decorators -- a find-and-replace handles the bulk of it. Once you are running on v3, you get zero dependencies, faster startup, and access to every new feature listed in section 11.

Feature Catalog

Tina4 3.14 has **140 numbered catalog entries**. The number describes the framework family's implementation and audit inventory. It does not mean that 140 features have reached four-language parity, and it does not mean that every entry lives inside each backend package.

This chapter is the map. The earlier chapters explain the public APIs available to Python applications. The numbered audit packets define the parity work and the clean-room formula for another language.

Read the status correctly

A feature moves through four distinct states:

- **Catalogued** means the feature has a stable number, name, owner, and audit packet.
- **Shipped** means code exists in one or more released components.
- **Audited** means the team has measured all relevant implementations and recorded contradictions and decisions.
- **Contract-proven** means a shared fixture passes in every applicable implementation with real dependencies.

Do not collapse those states into one green tick. At version 3.13.101, the contract ledger reports **55 fixtures and 282 proven invariants, with 0 owed and 0 broken inside those fixture-covered contracts**. That result proves the named contracts. It does not certify the remaining catalog entries.

Where the features live

Range	Owner	Meaning
1-109, 126-132, 135-140	Backend frameworks	Runtime, developer, testing, and application-facing capabilities. Availability and maturity still require per-feature evidence.
110-125 and 134	Shared Rust CLI	One language-neutral client used with all four backends. These are not four separate backend implementations.
133	Verification contract	Carbonah benchmark and report shape, not an application runtime API.
Provider entries	Selected integration	Each selectable provider has its own number because another language must implement and test it separately. Providers can require an external service, driver, language extension, or extra package.

Feature 63 includes browser helpers and the separate tina4-js frontend package. The backend books explain their integration, but tina4-js is not embedded four times.

Current parity boundary

Evidence	Python	PHP	Ruby	Node.js
Catalog membership	Inventory source	Audit in progress	Audit in progress	Audit in progress
Fixture-covered contracts	Proven	Proven	Proven	Proven
Entire 140-entry catalog	Not yet proven	Not yet proven	Not yet proven	Not yet proven

The initial catalog followed the Python module inventory, but Python is not a permanent master. Tina4 promotes the best implementation after audit and captures that decision in an ADR and shared fixture.

Known missing feature surfaces remain tracked in PHP issue 185 and Node.js issue 37. An empty issue list in another repository is not parity evidence. The fixture ledger is the evidence.

Dependency boundary

The Python package declares no required third-party dependencies. Database, cache, queue, MongoDB, and S3 integrations use optional extras when selected.

"Core dependency" and "provider dependency" are different facts. A local SQLite application can stay small while PostgreSQL, RabbitMQ, Kafka, Redis, MongoDB, S3, or hosted AI still requires the driver or service that speaks that protocol.

Numbered catalog

Foundation

#	Catalog entry
1	DotEnv and typed environment
2	Structured logger

Database and providers

#	Catalog entry
3	Database adapter interface
4	Database URL parser
5	Database facade and safe writes
6	Query builder
7	SQL translator
8	SQLite provider
9	PostgreSQL provider
10	MySQL provider
11	MSSQL provider
12	Firebird provider
13	ODBC provider
14	MongoDB SQL-translation provider
15	Migrations
16	Race-safe database sequences

ORM and data layer

#	Catalog entry
17	ORM base class
18	ORM fields and column mapping
19	Input and request validation
20	Soft delete
21	Declarative ORM relationships
22	Imperative ORM relationships
23	ORM scopes
24	Paginated database and ORM results
25	ORM result caching
26	ORM instance loading
27	Automatic CRUD from models
28	Seeder and fake data

HTTP core

#	Catalog entry
29	HTTP request model
30	HTTP response model and representation types
31	Router and dispatch
32	Route groups
33	Middleware pipeline

HTTP policies

#	Catalog entry
34	CORS middleware
35	Rate limiting
36	Security headers middleware
37	CSRF protection

HTTP runtime

#	Catalog entry
38	Health and readiness endpoints
39	Graceful shutdown
40	HTTP compression and ETag
41	Static assets and cache revalidation
42	Configurable error pages
43	Request ID tracking
44	File upload contract
45	Swagger and OpenAPI
46	Default landing page
47	In-process background tasks

Frond template engine

#	Catalog entry
48	Frond lexer
49	Frond parser
50	Frond compiler
51	Frond runtime
52	Frond filters
53	Frond tags
54	Frond expression tests
55	Frond functions
56	Frond extensibility API
57	Frond auto-escaping
58	Frond sandboxing
59	Frond template caching
60	Frond fragment caching

Frontend assets

#	Catalog entry
61	SCSS compiler
62	Tina4 CSS
63	Frond and Tina4 browser helpers

Authentication

#	Catalog entry
64	JWT and request authentication

Sessions and providers

#	Catalog entry
65	Session lifecycle
66	File session provider
67	Redis session provider
68	Valkey session provider
69	MongoDB session provider
70	Database session provider
71	Memcached session provider

Cache and providers

#	Catalog entry
72	Cache interface and provider selection
73	Memory cache provider
74	File cache provider
75	Redis cache provider
76	Valkey cache provider
77	Memcached cache provider
78	MongoDB cache provider
79	Database cache provider
80	HTTP response cache

Integrations and storage

#	Catalog entry
81	Standard-library HTTP API client
82	GraphQL
83	WebSocket protocol and server
84	Redis WebSocket backplane
85	NATS WebSocket backplane
86	WSDL and SOAP
87	Localization and i18n
88	Email and messenger
89	Queue lifecycle
90	Lite queue provider
91	RabbitMQ queue provider
92	Kafka queue provider
93	MongoDB queue provider
94	MQTT client
95	Document store interface
96	SQLite document store provider
97	MongoDB document store provider
98	Realtime collaboration
99	Local realtime attachment storage
100	S3 realtime attachment storage
101	Model Context Protocol server
102	Local source and documentation context index
103	Live framework and application API index
135	App-facing LLM client
137	GIS spatial points and queries
138	RBAC (role/permission guards)

139	Graph databases (Ultipa, Neo4j, Memgraph, ArangoDB)
140	Provider-neutral Web Push notifications

Enterprise authentication

#	Catalog entry
136	Configuration-first OpenID Connect SSO

Application runtime

#	Catalog entry
104	Event and listener system
105	Dependency injection container
106	Service runner
107	HTML element builder
108	AI coding-tool integration
109	Lazy feature loading and preload manifest

CLI

#	Catalog entry
110	CLI project initialization
111	CLI development server
112	CLI migrations
113	CLI seeding
114	CLI test runner
115	CLI route inspection
116	CLI interactive console
117	CLI environment management
118	CLI queue management
119	CLI container build
120	CLI code generation
121	CLI code metrics
122	CLI command discovery and help
123	CLI doctor
124	CLI guided setup
125	CLI deployment
134	CLI AI-skills installation and refresh

Developer runtime

#	Catalog entry
126	Development error overlay
127	Development admin dashboard
128	Dual development and test ports
129	Development port takeover
130	Dynamic framework version

Testing and verification tools

#	Catalog entry
131	In-process HTTP test client
132	Inline testing API
133	Carbonah benchmark contract

What parity means

Parity applies to observable contracts: response shapes, status codes, environment keys, error messages, file formats, and security rules. Host-language code remains idiomatic. A Python method can use snake_case while PHP and Node.js use camelCase, but all three must return the same public data.

The catalog gives another language a checklist. The audit packet supplies the decisions. The fixture supplies the proof. A name alone proves nothing; a green contract earns the claim.

Real-time Collaboration (WebRTC)

1. The Media Server You Do Not Have to Run

You want a call button. Two people click it and they are talking: audio, video, a shared screen. Next to the call sits a chat panel and a place to drop a file. This is the Slack shape, the Teams shape, and the usual advice is heavy. Stand up a media server. Wire in TURN. Operate all of it.

You do not need any of that to start. Modern browsers already speak WebRTC to each other directly, peer to peer. What they cannot do is *find* each other. One browser has to hand its connection offer to the other and get an answer back. That hand-off is called **signalling**, and it is a tiny amount of text relayed through a server. The media itself, the actual audio and video, never touches your server. It flows browser to browser.

Tina4's realtime module is that relay, plus the two things every collaboration tool needs around a call: persistent **chat** and permissioned **file** transfer. It shipped in 3.13.57, carries zero extra dependencies, and mounts with one line. Media is peer to peer, a **mesh**, by default. Tina4 carries no media and never parses your SDP. It forwards the handshake and nothing more. Outgrow mesh later and an SFU backend drops in with no route changes.

---

2. What You Get, and How It Differs from Raw WebSocket

The WebSocket chapter gave you the raw `@websocket` primitive: a decorator, a handler, a path. That is the tool you reach for when you build your *own* protocol. The realtime module is the opposite end. It is a *pre-built* protocol for calls, chat, and files, assembled from that same primitive underneath.

The module gives you three surfaces, and you enable only the ones you want:

- **calls** - a WebRTC signalling relay (mesh, peer to peer) plus a self-describing ICE-config endpoint. The relay forwards offer, answer, and ICE frames between peers in a room. It is public: no login to join a room.
- **chat** - persistent channels and messages backed by framework-owned ORM models, a secured chat WebSocket with live presence, typing, and read receipts, and a history endpoint for catch-up on reconnect.
- **files** - permissioned upload and download through a pluggable storage backend (local filesystem by default, S3-compatible optional).

The distinction in one line: with `@websocket` you write the handler and invent the message protocol; with `realtime()` the protocol is written for you and the browser discovers every path from one config endpoint. Under the hood the realtime WebSockets *are* Tina4 WebSockets. Same `(connection, event, data)` handler convention, same room manager. You are handed the handlers pre-written.

This backend pairs with the frontend tina4-js `rtc` module, whose `rtcConfig()` helper fetches the config endpoint so the client and server never drift on paths. Do the backend here. Do the

browser side in tina4-js, or with the plain browser APIs shown in section 10.

---

3. Mounting the Module: `realtime()`

Call `realtime()` once in `app.py`, **before** `run()`. It registers the routes and returns the resolved path map.

```
from tina4_python.realtime import realtime

realtime()                                                          # calls only (default)
realtime(features=["calls", "chat"])                                # add persistent chat
realtime(prefix="/api/collab", features=["calls", "chat", "files"])  # relocate the whole surfac
```

The full signature:

```
realtime(prefix="", *, media=None, authorize=None, storage=None, features=None) -> dict
```

Parameter	Meaning
`prefix`	Mounts the whole surface under `/<prefix>` (default: root). Internally `prefix.strip("/")`, so `"/api/collab"` and `"api/collab"` behave the same.
`media`	An `RtcMediaBackend`. Defaults to the env-selected backend (`mesh` in Phase 1). Pass one to override the env entirely.
`authorize`	Membership guard `authorize(identity, channel_id) -> bool` (sync or async) used by `chat` and `files`. Defaults to a `ChannelMember` check. `identity` is the string user id from the JWT.
`storage`	A `StorageBackend` for the `files` feature. Defaults to the env-selected store (`local`).
`features`	List; any of `"calls"`, `"chat"`, `"files"`. Defaults to `["calls"]`.

The call returns the resolved path map, the same map the config endpoint serves, so you can log it or assert against it:

```
realtime()
# -> {'backend': 'mesh', 'config': '/api/rtc/config', 'signalling': '/ws/rtc'}

realtime(features=['calls', 'chat'])
# -> {'backend': 'mesh', 'config': '/api/rtc/config',
#     'signalling': '/ws/rtc', 'chat': '/ws/chat', 'messages': '/api/channels'}
```

`config` is added by any enabled feature. `calls` sets it outright; `chat` and `files` add it with `setdefault`. So even a chat-only mount exposes `/api/rtc/config`.

What each feature wires

Feature	Route registered	Auth
any	`GET {p}/api/rtc/config` -> `rtc_config`	public (no auth)
`calls`	`WS {p}/ws/rtc/{room}` -> `rtc_signalling`	public (unauthenticated)
`chat`	`WS {p}/ws/chat/{channel}` -> `chat_ws`	secured (valid JWT on upgrade)
`chat`	`GET {p}/api/channels/{id}/messages` -> `chat_history`	`auth_required=True`
`files`	`POST {p}/api/files` -> `files_upload`	`auth_required=True`
`files`	`GET {p}/api/files/{key}` -> `files_download`	`auth_required=True`

If `chat` or `files` is enabled, the framework creates its chat tables at mount time. A missing database logs an error but does not crash boot (section 11).

---

4. The Config Bootstrap: `GET /api/rtc/config`

The client never hardcodes a URL. It fetches this one public endpoint, and everything else comes back in the response: the signalling path, the ICE servers, the chat, messages, and files paths. Move `prefix` on the server and the client follows automatically.

The body is feature-gated. Only keys for enabled features appear.

```
{
  "backend": "mesh",
  "iceServers": [ /* ...ice_servers()... */ ],   // calls
  "signalling": "/ws/rtc/{room}",                 // calls
  "chat": "/ws/chat/{channel}",                   // chat
  "messages": "/api/channels/{id}/messages",      // chat
  "files": "/api/files"                           // files
}
```

The `{room}`, `{channel}`, and `{id}` are literal template tokens. The client substitutes the real room name, channel id, or message id before connecting.

This endpoint is public, and it returns your ICE and TURN configuration including freshly-minted ephemeral TURN credentials (section 5). That is intentional. The browser needs those before it can authenticate anywhere, which is exactly why the credentials are short-lived by design.

---

5. ICE and TURN Servers

Before two browsers connect, each gathers candidate addresses using **ICE**. A **STUN** server tells a browser its public address. A **TURN** server relays media when a direct path is impossible: strict NAT, symmetric firewalls. The realtime module builds this list from environment variables via `ice_servers()`.

`ice_servers()` always includes a STUN entry. It adds a TURN entry only when both `TINA4_RTC_TURN_URL` and `TINA4_RTC_TURN_SECRET` are set, using coturn's `use-auth-secret` scheme with time-limited credentials:

- `username = str(int(time.time()) + ttl)`, an expiry epoch.
- `credential = base64(HMAC_SHA1(secret, username))`.

```
# No TURN env - STUN only:
[{'urls': ['stun:stun.l.google.com:19302']}]

# TINA4_RTC_TURN_URL + TINA4_RTC_TURN_SECRET set:
[{'urls': ['stun:stun.l.google.com:19302']},
 {'urls': ['turn:turn.example.com:3478'],
  'username': '1783546725',
  'credential': 'ie7Mm...=='}]
```

Environment variables

```
# .env
TINA4_RTC_BACKEND=mesh                          # only 'mesh' ships in Phase 1
TINA4_RTC_STUN_URLS=stun:stun.l.google.com:19302
TINA4_RTC_TURN_URL=turn:turn.example.com:3478   # enables TURN when paired with the secret
TINA4_RTC_TURN_SECRET=your-coturn-shared-secret
TINA4_RTC_TURN_TTL=3600                          # ephemeral credential lifetime (seconds)
```

Var	Default	Effect
`TINA4_RTC_BACKEND`	`mesh`	Media backend name. Only `mesh` ships in Phase 1. An unknown name falls back to `mesh`, never failing boot.
`TINA4_RTC_STUN_URLS`	`stun:stun.l.google.com:19302`	Comma-separated STUN URLs.
`TINA4_RTC_TURN_URL`	-	Comma-separated TURN URLs; enables TURN when set together with the secret.
`TINA4_RTC_TURN_SECRET`	-	coturn `use-auth-secret` shared secret (drives the ephemeral credentials).
`TINA4_RTC_TURN_TTL`	`3600`	Ephemeral TURN credential lifetime, in seconds.

The media backend is a strategy object. `MeshBackend` is the default, zero-dependency backend: browsers connect peer to peer in a mesh. An explicit `media=` argument wins; otherwise the module reads `TINA4_RTC_BACKEND`, and any unknown name falls back to `MeshBackend`. It never fails boot. An SFU or LiveKit backend that returns a real join token is the documented Phase-2 drop-in, and because signalling paths are unchanged, the client keeps working.

---

6. Signalling: The Mesh Relay

The signalling handler is registered at `WS {p}/ws/rtc/{room}` and is public. Anyone can join any room. It follows the framework's WebSocket handler convention:

```
async def rtc_signalling(connection, event, data):
    # connection : the WebSocketConnection
    # event      : "open" | "message" | "close"
    # data       : payload (str for "message", None for "open"/"close")
    ...
```

Its behaviour is deliberately minimal, a raw relay:

- Reads `room = connection.params.get("room", "")`. An empty room is a no-op.
- On `"open"` it calls `connection.join_room("rtc:<room>")`.
- On `"message"` it calls `await connection.broadcast_to_room("rtc:<room>", data, exclude_self=True)`, forwarding the raw payload to the other peers in the room, untouched.

Tina4 never parses the SDP. Peers put a `to` field in their own frames and filter for themselves. Rooms are namespaced `rtc:<room>` so signalling rooms never collide with chat channels, which use `chat:<channel>`, on the shared WebSocket manager.

Because signalling is public, the room name is your only barrier. If a call must be private, gate access at your app layer: issue unguessable room ids, or check membership before you hand the client a room name.

For a private call, do not mount the built-in `calls` feature and assume its `authorize` hook protects signalling: that hook applies only to chat and files. Register an application-owned signalling route with `@secured()` above `@websocket(...)`, then verify the authenticated identity in `connection.auth` belongs to the requested room on open and on every relayed frame. Browser clients carry the JWT with `new WebSocket(url, ["bearer", token])`.

The current `tina4-js` `rtc.call()` helper opens the built-in public signalling socket and has no signalling-token option. Use a small application WebSocket client for the protected route (or add token support to that client API in a separately versioned change). Unguessable room names reduce discovery; they are not authorization.

---

7. Chat: Secured Channels, Presence, History

Chat is the opposite of signalling: secured. The handler `chat_ws` is marked `_secured = True`, so a valid JWT is required on the WebSocket upgrade. The router rejects an unauthenticated upgrade before your handler runs.

- Channels are addressed by integer id: `int(connection.params["channel"])`. A non-integer `{channel}` makes the handler return silently. The socket opens and does nothing.
- Identity comes from the verified token: `identity = _identity(connection.auth)` (section 9).
- The room key is `chat:<channel_id>`.

Every frame is JSON; every broadcast is a `json.dumps(...)` string. The event flow:

Event / message	Server behaviour
`open`	Authorize. Fail: send `{"type":"error","error":"not a member of this channel"}` then `close()`. OK: `join_room`, send the caller the roster, then broadcast a `join` presence event (exclude self).
`close`	Broadcast a `leave` presence event (exclude self).
message `typing`	Broadcast `{"type":"typing","user_id":<id>}` (exclude self).
message `read`	Advance the member's read cursor (`last_read_at = now`), broadcast a `read` event (exclude self).
message `message`	Trim `body`; empty is ignored. Persist a `Message` row; on success broadcast the saved message to everyone including the sender, so an optimistic client reconciles with the server `id` and `created_at`.

`type` defaults to `"message"` when absent. Unknown types are ignored. Authorization is re-checked on every inbound frame, not just on join. Membership can be revoked mid-session, and the server never trusts an identity carried in the payload.

The saved-message JSON shape, also what history returns:

```
{ "id": 42, "channel_id": 7, "user_id": "12", "body": "hi team",
  "thread_id": null, "created_at": "2026-07-08T10:15:00+00:00" }
```

`thread_id` is `null` for a top-level message, or the parent message id for a threaded reply.

Chat history: `GET {p}/api/channels/{id}/messages`

A catch-up-on-reconnect endpoint (`auth_required`). Identity comes from `authenticate_request(request.headers)`. An invalid channel id returns 400; not authorized returns 403. Query params: `before` (return messages with `id < before`) and `limit` (default 50, capped at 200). Messages come back newest-first (`ORDER BY id DESC`), the standard infinite-scroll-backwards shape.

```
curl -H "Authorization: Bearer $JWT" \
  "http://localhost:7146/api/channels/7/messages?limit=50"

# older page: pass the smallest id you already have as `before`
curl -H "Authorization: Bearer $JWT" \
  "http://localhost:7146/api/channels/7/messages?before=42&limit=50"
```

---

8. Files: Upload and Download

Enable with `features=["files"]`. Both routes are `auth_required` and go through a pluggable `StorageBackend`: the `storage=` argument, or the env-selected store (default `LocalStorage`).

`POST {p}/api/files` **- upload.** Multipart: a file field named `file`, plus a form field `channel_id` (required, integer). Missing or invalid `channel_id` returns 400; not a channel member returns 403; no file returns 400. It stores the blob under an opaque, collision-free `storage_key` (a uuid plus a sanitized extension, never a user-controlled path), inserts an `Attachment` row (metadata only), and responds 201:

```
{ "id": 9, "key": "3f8c...e1.png", "filename": "diagram.png",
  "mime": "image/png", "size": 20481,
  "url": "/api/files/3f8c...e1.png" }
```

`url` is `store.url(key)` when the backend exposes a direct URL (for example an S3 presigned URL), otherwise the app download route.

`GET {p}/api/files/{key}` **- download.** Looks up the `Attachment` by `storage_key`; missing returns 404. Authorizes against the attachment's `channel_id`; a non-member returns 403. If the backend has a direct URL it returns a 302 redirect; otherwise it streams the bytes (200) with `Content-Disposition: inline` and the stored `Content-Type`.

Storage backends

`select_storage(storage=None)` resolves from the `storage=` argument or `TINA4_STORAGE_BACKEND`. An `s3` backend that cannot be built (for example, no `TINA4_STORAGE_BUCKET`) falls back to `LocalStorage` with a warning: a real store, never a silent no-op.

```
# .env - local (default)
TINA4_STORAGE_BACKEND=local
TINA4_STORAGE_DIR=data/rt_storage

# .env - S3-compatible (MinIO, AWS, ...)
TINA4_STORAGE_BACKEND=s3
```

```
TINA4_STORAGE_URL=https://s3.example.com
TINA4_STORAGE_KEY=...
TINA4_STORAGE_SECRET=...
TINA4_STORAGE_BUCKET=my-bucket
TINA4_STORAGE_REGION=us-east-1
```

`LocalStorage` resolves every key inside its root and rejects path traversal; its `url()` returns `None`, so files serve through the permissioned download route. `S3Storage` returns a presigned GET URL from `url()`, so clients fetch large blobs straight from object storage and downloads become a 302 redirect.

---

9. Auth, Identity, and the Data Model

Chat and files are membership-gated. Two pieces make that work: how an identity is extracted, and how membership is checked.

Extracting identity, `_identity(auth)`. It pulls a stable string user id from a verified JWT payload, trying the claims `user_id`, then `sub`, then `id`, and returns `None` if none are present. Identities round-trip as strings, so an integer id, a UUID, or an email all work. WebSocket identity comes from `connection.auth`, the verified payload the router attached on the secured upgrade. HTTP identity comes from `authenticate_request(request.headers)` inside each handler.

Checking membership, `_default_authorize(identity, channel_id)`. The secure default: the user must be a member of the channel (`ChannelMember.count(...) > 0`). Any exception logs and denies. Override it with `authorize=`, a `authorize(identity, channel_id) -> bool` guard, sync or async. The internal wrapper denies first whenever `identity is None`, so an unauthenticated caller is always rejected regardless of your guard. Because it runs on every inbound chat frame, keep a custom guard cheap.

```
# Open every channel to any logged-in user (skip per-channel membership).
async def allow_any_authenticated(identity, channel_id):
    return True

realtime(features=["calls", "chat"], authorize=allow_any_authenticated)
```

Data model

The chat surface persists to framework-owned ORM models, all carrying the `tina4_rt_` table prefix so they never collide with your app's tables. Tables are created on demand at mount time.

Model	Table	Key fields
`Workspace`	`tina4_rt_workspaces`	`id`, `name`, `created_at`
`Channel`	`tina4_rt_channels`	`id`, `workspace_id`, `name`, `kind` (`public`/`private`/`dm`), `created_at`

`ChannelMember`	`tina4_rt_channel_members`	`id`, `channel_id`, `user_id` (string), `role`, `last_read_at`
`Message`	`tina4_rt_messages`	`id`, `channel_id`, `user_id`, `body`, `thread_id`, `created_at`, `edited_at`
`Attachment`	`tina4_rt_attachments`	`id`, `channel_id`, `message_id`, `storage_key`, `filename`, `mime`, `size`

`user_id` is a string everywhere so any JWT identity shape fits. Because these are ordinary Tina4 ORM models, you seed a workspace, a channel, and its members exactly as in the ORM chapter:

```
from tina4_python.realtime.models import Workspace, Channel, ChannelMember

ws = Workspace(name="Acme"); ws.save()
channel = Channel(workspace_id=ws.id, name="general", kind="public"); channel.save()
ChannelMember(channel_id=channel.id, user_id="12", role="owner").save()
```

Now user `12` can open `WS /ws/chat/{channel.id}` and pass the history and file checks.

---

10. A Complete Example

A mesh video call with a chat side-panel, using only the browser's built-in `RTCPeerConnection` and `WebSocket`. No client library required.

Backend - `app.py`

Mount the surface before `run()`, and seed one channel so chat has somewhere to land.

```
from tina4_python import run, bind_database, Database
from tina4_python.realtime import realtime
from tina4_python.realtime.models import Workspace, Channel, ChannelMember

# Chat/files need a bound DB BEFORE realtime(features=[...]) - see section 11.
bind_database(Database("sqlite:data/app.db"))

paths = realtime(features=["calls", "chat", "files"])
print("realtime mounted:", paths)

# One-time seed so a channel + member exist.
if not Channel.where("name = ?", ["general"], limit=1):
    ws = Workspace(name="Acme"); ws.save()
    channel = Channel(workspace_id=ws.id, name="general", kind="public"); channel.save()
    ChannelMember(channel_id=channel.id, user_id="12", role="owner").save()

if __name__ == "__main__":
    run()
```

Browser - bootstrap from the config endpoint

The client fetches `/api/rtc/config` first and never hardcodes a path.

```
// 1. Discover paths + ICE servers (public endpoint).
const cfg = await (await fetch("/api/rtc/config")).json();

// 2. Open the signalling socket for a room (public, no token).
const room = "standup";
const wsUrl = location.origin.replace(/^http/, "ws")
            + cfg.signalling.replace("{room}", room);
const signal = new WebSocket(wsUrl);

// 3. One RTCPeerConnection per call, using the server's ICE list.
const pc = new RTCPeerConnection({ iceServers: cfg.iceServers });

pc.onicecandidate = (e) => {
  if (e.candidate) signal.send(JSON.stringify({ kind: "ice", candidate: e.candidate }));
};
pc.ontrack = (e) => { document.getElementById("remote").srcObject = e.streams[0]; };

// 4. Handle relayed signalling frames. Tina4 forwards raw payloads verbatim.
signal.onmessage = async (evt) => {
  const msg = JSON.parse(evt.data);
  if (msg.kind === "offer") {
    await pc.setRemoteDescription(msg.sdp);
    const answer = await pc.createAnswer();
    await pc.setLocalDescription(answer);
    signal.send(JSON.stringify({ kind: "answer", sdp: answer }));
  } else if (msg.kind === "answer") {
    await pc.setRemoteDescription(msg.sdp);
  } else if (msg.kind === "ice") {
    await pc.addIceCandidate(msg.candidate);
  }
};

// 5. The caller adds their camera/mic and sends an offer.
async function startCall() {
  const media = await navigator.mediaDevices.getUserMedia({ video: true, audio: true });
  document.getElementById("local").srcObject = media;
  media.getTracks().forEach((t) => pc.addTrack(t, media));
  const offer = await pc.createOffer();
  await pc.setLocalDescription(offer);
  signal.send(JSON.stringify({ kind: "offer", sdp: offer }));
}
```

Browser - the chat side-panel (secured, needs a JWT)

```
const channelId = 7;                       // integer id, not a name
const chatUrl = location.origin.replace(/^http/, "ws")
              + cfg.chat.replace("{channel}", channelId)
              + "?token=" + encodeURIComponent(jwt);   // WS carries the JWT
const chat = new WebSocket(chatUrl);

chat.onmessage = (evt) => {
  const m = JSON.parse(evt.data);
  if (m.type === "message")  addLine(m.message.user_id, m.message.body);
  if (m.type === "presence") renderRoster(m);   // roster / join / leave
  if (m.type === "typing")   showTyping(m.user_id);
  if (m.type === "error")    console.warn(m.error);   // e.g. not a member
};

function send(text) { chat.send(JSON.stringify({ type: "message", body: text })); }
```

The signalling socket is public: anyone with the room name joins. The chat socket and the history and file endpoints require a valid JWT and channel membership. Two browser tabs, both members of channel `7`, joined to room `standup`, will see each other's video peer to peer and each other's messages through the relay.

---

11. Footguns and Hard Rules

Chat needs a bound database, but a missing one does not crash boot. With `features=["chat"]` or `["files"]`, table creation runs at mount. If no database is bound, it logs an ERROR and continues. `realtime()` still returns the full path map and registers every route, and the failure only resurfaces at query time. Bind a database before calling `realtime(features=[...])`.

The signalling WebSocket is public. `/ws/rtc/{room}` is not secured. Anyone can join any room and receive the relayed frames. Only the chat WebSocket is JWT-secured. If a call must be private, gate the room name at your app layer.

The config endpoint is public. `/api/rtc/config` returns your ICE and TURN configuration, including freshly-minted ephemeral TURN credentials. This is by design, which is exactly why the TURN credentials are time-limited (`TINA4_RTC_TURN_TTL`).

The WS handler signature is `(connection, event, data)`. `event` is `"open"`, `"message"`, or `"close"`; `data` is a `str` on `"message"` and `None` otherwise. This is the framework convention, not `(connection, data, event)`.

Channels are addressed by integer id. A non-integer `{channel}` makes the chat handler return silently, no error frame. Pass the numeric channel id, never its name.

Authorization is re-checked on every chat frame. Membership can be revoked mid-session, and identity is always taken from the verified token, never from the payload. A custom `authorize=` must be cheap.

Empty messages are dropped. A `message` with an empty or whitespace-only `body` is not persisted and not broadcast.

An unknown `TINA4_RTC_BACKEND` falls back to mesh. A typo will not error; you just get `mesh`. Only `mesh` exists in Phase 1, and its `mint_join` returns `None`, since there is no SFU join token yet.

Where to Go Next

- **The WebSocket chapter** - the raw `@websocket` primitive these handlers are built on, and the connection API (`join_room`, `broadcast_to_room`, `send_json`).
- **The ORM and Authentication chapters** - you seed workspaces, channels, and members with the ORM, and the JWT that secures chat is the same one from the auth chapter.
- **The tina4-js `rtc` module** - the browser side, whose `rtcConfig()` helper wraps `GET /api/rtc/config` so the client and server never drift.

You started this chapter wanting a call button. You end it with calls, chat, presence, read receipts, history, and file transfer, mounted in one line, carrying no media, running no server you did not already have. The hard part of real-time was never the video. It was finding the other browser, and Tina4 relays that in a few lines of text.

AI Client

One class. Three methods. No provider SDK. The Tina4 AI client sends chat, completion, embedding, and streaming requests through one provider-neutral API.

This chapter covers the application-facing `Ai` class. The `tina4 ai` command serves a different purpose: it installs Tina4 skills and context for coding assistants.

Configure a provider

The client supports `local`, `openai`, and `anthropic`. Local mode targets an OpenAI-compatible endpoint and needs no API key.

```
TINA4_AI_PROVIDER=local
TINA4_AI_URL=http://localhost:11437
TINA4_AI_MODEL=llama3.2
TINA4_AI_TIMEOUT=60
TINA4_AI_CONNECT_TIMEOUT=10
TINA4_AI_MAX_RETRIES=2
```

Use the hosted OpenAI service by changing three values:

```
TINA4_AI_PROVIDER=openai
TINA4_AI_URL=https://api.openai.com/v1
TINA4_AI_MODEL=gpt-4o-mini
TINA4_AI_KEY=your-api-key
```

Anthropic uses the same keys:

```
TINA4_AI_PROVIDER=anthropic
TINA4_AI_URL=https://api.anthropic.com/v1
TINA4_AI_MODEL=claude-3-5-haiku-latest
TINA4_AI_KEY=your-api-key
```

An explicit method argument wins over the environment. The environment wins over the built-in default. This lets one request use another model without changing the rest of the process.

Complete one prompt

`Ai.complete` sends one user message and returns the response text.

```
from tina4_python import Ai

answer = Ai.complete("Explain why this query needs an index")
print(answer)
```

Pass request options when one call needs a different model or provider:

```
answer = Ai.complete(
    "Summarize this incident report",
    provider="openai",
    model="gpt-4o-mini",
    temperature=0.2,
    max_tokens=300,
    timeout=20,
)
```

Hold a chat

`Ai.chat` accepts a non-empty list of messages. Each message needs a `system`, `user`, or `assistant` role and string content.

```
from tina4_python import Ai

response = Ai.chat([
    {"role": "system", "content": "Answer as a concise database engineer."},
    {"role": "user", "content": "When should I use a composite index?"},
])

print(response.text)
print(response.model)
print(response.usage["total_tokens"])
print(response.finish_reason)
```

The `ChatResponse` object carries five fields:

Field	Meaning
`text`	Normalized response text
`model`	Model reported by the provider
`usage`	`prompt_tokens`, `completion_tokens`, and `total_tokens`
`finish_reason`	Provider finish reason, or `None`
`raw`	Original provider response

Use `raw` only when you need provider-specific metadata. Keep application logic on the normalized fields so a provider change does not spread through your code.

Stream text

Set `stream=True` to receive ordered text deltas. The iterator yields text only and ignores provider metadata events.

```
from tina4_python import Ai

chunks = Ai.chat(
    [{"role": "user", "content": "Write a short release announcement."}],
    stream=True,
)

for chunk in chunks:
    print(chunk, end="", flush=True)
```

Tina4 may retry before the first delta arrives. It never retries after yielding text because that could duplicate content the caller has already displayed.

Create embeddings

`Ai.embed` preserves the input shape. One string returns one vector. A list returns one vector per input, in the same order.

```
from tina4_python import Ai

vector = Ai.embed("Tina4 keeps application code small")

vectors = Ai.embed([
    "The router finds a matching handler",
    "The ORM maps a row to a model",
])
```

Set `TINA4_EMBED_URL` when the embedding service uses a different base URL. Anthropic does not expose embeddings through this contract, so `provider="anthropic"` raises `AiConfigError`.

Handle failures

All client failures inherit from `AiError`:

Error	Meaning
`AiConfigError`	Missing key, invalid provider, bad option, or unsupported capability
`AiHTTPError`	Provider returned a failing HTTP status or the transport failed
`AiTimeoutError`	Connection or total request deadline expired
`AiParseError`	A successful response did not match the provider contract

```
from tina4_python import Ai, AiError

try:
    print(Ai.complete("Create a migration plan"))
except AiError as error:
    print(f"AI request failed: {error}")
```

Hosted providers fail before sending when `TINA4_AI_KEY` is missing. Error text never includes the key, prompt, or provider response body. A failure stays a failure; Tina4 never turns it into an empty answer.

Timeouts and retries

`TINA4_AI_CONNECT_TIMEOUT` bounds connection setup. `TINA4_AI_TIMEOUT` bounds the whole request, including retries and response reads. `TINA4_AI_MAX_RETRIES` applies only to connection failures, HTTP 429, and HTTP 5xx responses.

Other HTTP 4xx responses and malformed successful responses run once and fail. The client carries transient trouble for a bounded distance, then hands the error back to your code.

OpenID Connect SSO

Tina4 completes one provider-neutral OpenID Connect flow. You configure an issuer. Tina4 discovers the provider, runs Authorization Code with PKCE, verifies the result, regenerates the Session, and hands the normalized identity to the existing secured-route gate.

No provider SDK. No second authentication system. The browser keeps the normal Tina4 Session cookie.

Configure SSO

```
TINA4_SSO_ISSUER=https://identity.example.com/realms/my-app
TINA4_SSO_CLIENT_ID=my-app
TINA4_SSO_CLIENT_SECRET=replace-me
TINA4_SSO_REDIRECT_URI=https://app.example.com/auth/callback
TINA4_SSO_SCOPES=["openid", "profile", "email"]
TINA4_SSO_VERIFY=introspection
TINA4_SSO_POST_LOGOUT_REDIRECT_URI=https://app.example.com/
```

SSO stays off until the issuer, client id, and redirect URI exist. A confidential client also needs its secret. Production issuer and callback URLs require HTTPS. Loopback HTTP remains available for local development.

When configured, Tina4 mounts these routes:

Method	Route	Purpose
`GET`	`/auth/login`	Start login and create one-use state, nonce, and PKCE values.
`GET`	`/auth/callback`	Validate state, exchange the code, and create the Session identity.
`POST`	`/auth/logout`	Destroy the local Session, then request provider logout when supported.

An application route that collides with one of these paths stops startup with an error. Tina4 never replaces part of the login flow in silence.

Protect a route

SSO uses the same `@secured()` gate as local Tina4 authentication.

```
from tina4_python import get, secured

@get("/account")
@secured()
async def account(request, response):
    return response.json(request.user)
```

`request.user` contains:

```
{
  "issuer": "https://identity.example.com/realms/my-app",
  "subject": "provider-stable-subject",
  "username": "andre",
  "email": "andre@example.com",
  "name": "Andre",
  "roles": ["admin", "developer"],
  "groups": ["/engineering"]
}
```

Issuer and subject are required. Roles and groups are sorted, de-duplicated lists. Provider tokens never enter `request.user`, logs, error pages, Swagger, or `session.all()`.

Use the SSO client

Automatic routes cover the normal browser flow. The public client remains available when an application needs to start or inspect the flow itself.

```
from tina4_python import Sso

if Sso.configured():
    sso = Sso.from_issuer()
    metadata = sso.discover()
    identity = sso.identity(request)
```

Method	Result
`Sso.configured()`	Reports whether the required environment values exist.
`Sso.from_issuer()`	Creates the client and discovers the configured issuer.
`discover(force=False)`	Returns validated provider metadata. `force=True` refreshes it.
`login(request, return_to="/")`	Stores one-use login state and returns the authorization URL.
`callback(request, query=None)`	Validates the callback, regenerates Session, and returns identity plus the local return path.
`identity(request)`	Returns the normalized identity or `None`.
`refresh(request)`	Replaces provider credentials and identity atomically.
`logout(request, return_to="/")`	Destroys local Session first and returns the safe logout target.

`return_to` accepts a local absolute path. Tina4 rejects schemes, hosts, protocol-relative URLs, backslashes, and control characters.

Map provider claims

Use `TINA4_SSO_CLAIM_MAP` when a provider stores a normalized field under another claim path:

```
TINA4_SSO_CLAIM_MAP={"username":"preferred_username","roles":"realm_access.roles","groups":"grou
```

The value is a native `.env` object, not a comma-separated string.

Provider recipes

The runtime does not name providers. Recipes only supply standard OIDC values.

Keycloak

```
TINA4_SSO_ISSUER=https://sso.example.com/realms/my-realm
TINA4_SSO_REDIRECT_URI=https://app.example.com/auth/callback
```

Enable Authorization Code flow, register the exact callback URI, and map realm roles, client roles, or groups into claims when the application needs them.

Microsoft Entra ID

```
TINA4_SSO_ISSUER=https://login.microsoftonline.com/your-tenant-id/v2.0
TINA4_SSO_REDIRECT_URI=https://app.example.com/auth/callback
```

Use the tenant-specific v2 issuer. Add a claim map only when the tenant exposes roles or groups under different names.

Verification and failure rules

Version 3.13.104 supports introspection without an extra Python package. It requires a confidential client secret. Selecting `jwks` fails during configuration until the application installs a supported cryptography capability.

State, nonce, authorization code, and PKCE verifier are one-use. A mismatch or replay creates no identity. A failed refresh removes SSO identity rather than continuing with expired credentials. Logout always destroys the local Session before it contacts the provider.

Swagger

When SSO is configured, Swagger publishes the issuer discovery URL as an `openIdConnect` scheme. Secured routes accept the normal Tina4 Session cookie or the configured bearer scheme. Swagger never presents a provider access token as a Tina4 Session.

GIS and PostGIS

Tina4 stores geographic points in PostGIS, measures distance in metres, and returns map-ready GeoJSON. Public coordinates always use longitude first: `(longitude, latitude)`.

GIS describes the spatial model and queries. GPS may supply coordinates, but it does not define this feature.

Prepare PostGIS

The first GIS provider is PostgreSQL with PostGIS:

```
CREATE EXTENSION IF NOT EXISTS postgis;
```

Point fields fail with a clear error on SQLite, MySQL, SQL Server, Firebird, or PostgreSQL without PostGIS. Tina4 does not replace a spatial query with an approximation over two numeric columns.

Define a point field

```
from tina4_python.orm import ORM, IntegerField, StringField, PointField, Point

class ChargePoint(ORM):
    id = IntegerField(primary_key=True, auto_increment=True)
    name = StringField()
    location = PointField()

site = ChargePoint({
    "name": "V&A Waterfront",
    "location": Point(18.4241, -33.9249),
})
site.save()
```

`PointField()` creates `geography(Point,4326)` and an idempotent GiST index. Set `spatial_index=False` only when another migration owns the index.

Point values

`Point.parse()` accepts a `Point`, a two-number pair, WKT or EWKT, GeoJSON Point or Feature, and WKB or EWKB. It rejects booleans, NaN, infinity, invalid ranges, non-point geometry, and conflicting SRIDs before SQL runs.

```
Point.parse([18.4241, -33.9249])
Point.parse("POINT(18.4241 -33.9249)")
Point.parse("SRID=4326;POINT(18.4241 -33.9249)")
Point.parse({"type": "Point", "coordinates": [18.4241, -33.9249]})
```

Property	Value
`.lon`, `.lat`, `.srid`	Validated point coordinates and reference id.
`.wkt`	`POINT(lon lat)`

`.ewkt`	`SRID=4326;POINT(lon lat)`
`.geojson`	RFC 7946 Point geometry.
`.to_tuple()`	`(longitude, latitude)`

SQL `NULL` remains `None`. `(0, 0)` remains a real point.

Query by distance

```
nearby = (
    ChargePoint.query()
    .within_distance("location", (18.42, -33.92), 5_000)
    .select_distance("location", (18.42, -33.92), alias="metres")
    .order_by_distance("location", (18.42, -33.92))
    .get()
)
```

Query method	Meaning
`within_distance(column, point, metres)`	Keep rows within the non-negative spheroid radius.
`select_distance(column, point, alias="metres")`	Add the distance in metres to each row.
`order_by_distance(column, point, descending=False)`	Sort by distance with a stable primary-key tie-break.
`intersects(column, geometry)`	Match a bound WKT, EWKT, or GeoJSON query geometry.
`bbox(column, min_lon, min_lat, max_lon, max_lat)`	Match a validated bounding box.

Spatial values use bound parameters. Tina4 validates column and alias names as identifiers. A bounding box rejects inverted or out-of-range coordinates.

Return GeoJSON

```
from tina4_python.orm import feature_collection

feature = site.to_feature()
collection = feature_collection(nearby)
return response.json(collection)
```

`to_feature()` moves the chosen Point field into `geometry` and keeps the other selected fields in `properties`. A null point produces `"geometry": null`, not coordinates at Null Island. `feature_collection()` preserves query order.

Pass `geometry_field` when a model has more than one Point field. Pass `include` to select the properties that leave the application.

Limits in 3.13.104

Tina4 persists Point fields only. It does not persist lines, polygons, rasters, tiles, geocoding results, or routes. `intersects()` may still accept a polygon as bound query geometry. PostGIS is the only supported storage provider in this release.

IoT and MQTT

Tina4 includes a zero-dependency MQTT 3.1.1 client for telemetry, device state, asset tracking, and EV charging applications. It connects to an external broker such as Mosquitto, EMQX, HiveMQ, or AWS IoT. Tina4 is not an MQTT broker.

Configure the broker

```
TINA4_MQTT_URL=mqtt://127.0.0.1:1883
TINA4_MQTT_CLIENT_ID=warehouse-api
TINA4_MQTT_KEEPALIVE=60
TINA4_MQTT_TLS_VERIFY=true
TINA4_MQTT_CA_FILE=/run/secrets/mqtt-ca.pem
```

Use `mqtt://` or `tcp://` for plain TCP and `mqtts://` for TLS. Credentials belong in URL user information or explicit constructor arguments:

```
TINA4_MQTT_URL=mqtts://device-user:device-password@broker.example.com:8883
```

Explicit constructor values beat URL values and environment values. Broker credentials do not have dedicated environment variables.

Publish telemetry

Python connects when `Mqtt()` is constructed unless `connect=False`.

```
from tina4_python.mqtt import Mqtt

mqtt = Mqtt()
packet_id = mqtt.publish(
    "fleet/meter-42/telemetry",
    '{"kwh":12.5,"timestamp":"2026-08-18T08:00:00Z"}',
    qos=1,
)
mqtt.disconnect()
```

QoS 0 returns `None` and waits for no acknowledgement. QoS 1 returns the packet id after the matching PUBACK. Tina4 refuses QoS 2 instead of downgrading it. Use QoS 1 with an idempotency key such as `(device_id, device_timestamp)`.

Subscribe and consume

```
mqtt = Mqtt(clean_session=False)

for message in mqtt.consume("fleet/+/telemetry", qos=1):
    process(message.topic, message.text())
```

`consume()` acknowledges after the loop resumes from a successfully handled message. If processing raises, the broker can redeliver the message. Use `receive(ack=True)` when the application wants immediate acknowledgement, or `receive(ack=False)` followed by `message.acknowledge()` for manual control.

`MqttMessage` exposes `topic`, byte `payload`, `qos`, `packet_id`, `retained`, `duplicate`, `acknowledge()`, `text()`, and `to_dict()`.

Retained state and Last Will

```
mqtt.publish("fleet/meter-42/status", "online", qos=1, retain=True)
```

A retained publish gives new subscribers the latest device state. Publishing an empty retained payload clears it.

Use a Last Will to report an unclean disconnect:

```
mqtt = Mqtt(
    will_topic="fleet/meter-42/status",
    will_payload="offline",
    will_qos=1,
    will_retain=True,
)
```

`disconnect()` sends MQTT DISCONNECT and suppresses the will. `kill()` drops the socket so the broker publishes it.

TLS and trust

Every client builds its own TLS trust store. A CA loaded for one connection cannot leak into another client. Verification defaults to true. Disabling it logs a warning because encrypted traffic without peer verification remains open to a man-in-the-middle attack.

After a TLS connection, `mqtt.tls()`, `mqtt.cipher()`, and `mqtt.tls_version()` report the negotiated channel.

Public operations

Operation	Purpose
`connect()` / `connected()`	Open the broker connection and inspect its state.
`publish()`	Send QoS 0 or QoS 1 data, optionally retained.
`subscribe()`	Register a topic filter and return its packet id.
`receive()`	Read one message with immediate or manual acknowledgement.
`consume()`	Iterate messages and acknowledge after successful handling.
`acknowledge()`	Send PUBACK for a packet id.
`ping()` / `start_keepalive()`	Check or maintain an idle connection.
`disconnect()` / `kill()`	Close cleanly or simulate an unclean device loss.

One client has one socket reader. Do not run two receive loops on the same instance. Create another client when independent consumers need the same topic.